

Texinfo

O Formato GNU de Documentação
para Texinfo versão 7.3, 1º de março de 2026

Robert James Chassell
Richard Matthew Stallman
Jamenson Espindula (tradução para português brasileiro)

Publicado pela Fundação do Software Livre
Rua 31 Milk nº 960789
Boston, Massachussets 02196
Estados Unidos da América do Norte
ISBN 1-882114-67-1

Este manual é para o GNU Texinfo (versão 7.3, 1º de março de 2026), um sistema de documentação que consegue produzir informações online e um manual impresso a partir de uma fonte usando marcação semântica.

Copyright © 1988-2026 Fundação do Software Livre, Inc.

É concedida permissão para copiar, distribuir e (ou) modificar este documento sob os termos da Licença GNU de Documentação Livre, Versão 1.3 ou qualquer versão posterior publicada pela Free Software Foundation; sem Seções Invariantes, sem Textos de Capa Frontal e sem Textos de Contracapa. Uma cópia da licença está incluída na seção intitulada “Licença GNU de Documentação Livre”.

Sumaríssimo

Condições de Cópia do Texinfo	1
1 Visão Geral do Texinfo	2
2 Escrevendo um Arquivo do Texinfo	8
3 Nós	26
4 Estruturamento de Capítulo	37
5 Referências cruzadas	43
6 Marcação de Texto, Palavras e Frases	54
7 Citações e Exemplos	65
8 Listas e Tabelas	74
9 Exibições Especiais	81
10 Índices	88
11 Inserções Especiais	95
12 Formatando Expressões Matemáticas	109
13 Forçando e Impedindo Quebras	110
14 Comandos de Definição	114
15 Internacionalização	126
16 Texto Condicionalmente Visível	129
17 Definindo Novos Comandos do Texinfo	138
18 Arquivos de Inclusão	148
19 Formatando e Imprimindo com $\text{\TeX}$	150
20 <code>texi2any</code> : O Tradutor para Texinfo	157
21 Criando e Instalando Arquivos Info	183
22 Gerando HTML	192
A Detalhes do Comando <code>@</code>	221
B Dicas e Sugestões	245
C Arquivos de Amostra do Texinfo	249
D Usando Modo Texinfo	252
E Comandos Globais de Documento	274
F Especificação do Formato Info	282
G Licença GNU de Documentação Livre	289
Índice de Comandos	297
Índice de Variáveis	302
Índice Geral	305

Sumário

Condições de Cópia do Texinfo	1
1 Visão Geral do Texinfo	2
1.1 Informando Defeitos	2
1.2 Formatos de Saída	3
1.3 Arquivos do Info	4
1.4 Livros Impressos	5
1.5 Histórico	5
1.6 Adicionando Formatos de Saída	7
2 Escrevendo um Arquivo do Texinfo	8
2.1 Convenções Sintáticas Gerais	8
2.2 Comentários	8
2.3 O Que um Arquivo do Texinfo Precisa Ter	9
2.4 Amostra Curta	10
2.5 Cabeçalho do Arquivo do Texinfo	11
2.5.1 A Primeira Linha de um Arquivo do Texinfo	11
2.5.2 @setfilename: Configure o Nome do Arquivo da Saída Gerada	12
2.5.3 @settitle: Configura o Título do Documento	12
2.5.4 Preâmbulo	13
2.5.5 Início e Fim do Cabeçalho para Emacs	14
2.6 Categoria de Diretório	14
2.7 Permissões do Documento	15
2.7.1 @copying: Declarar Permissões de Cópia	16
2.7.2 @insertcopying: Incluir Texto de Permissões	17
2.8 Páginas de Título e de Direitos Autorais	17
2.8.1 Metadados do Documento e o Comando @maketitle	18
2.8.2 @titlepage	18
2.8.2.1 @title, @subtitle e @author	19
2.8.2.2 @titlefont, @center e @sp	20
2.8.3 Exibindo a Versão ou a Edição na Página de Título	21
2.8.4 Geração de Títulos	21
2.9 Gerando um Sumário	21
2.10 O Nó ‘Top’	22
2.10.1 Partes de um Menu Mestre	23
2.11 O Corpo do Documento	23
2.12 Finalizando um Arquivo do Texinfo	25
3 Nós	26
3.1 Escrevendo uma Linha de @node	26
3.2 Escolhendo Nomes de Nós	27
3.3 Exigências da Linha de @node	28
3.4 O Primeiro Nó	29
3.5 O Comando de Seccionamento @top	29
3.6 Estrutura do Documento Texinfo	30
3.7 Ilustração de Nó e Menu	31
3.8 Descrições de Nó	32

3.9	Menus	33
3.9.1	Escrevendo um Menu	33
3.9.2	Um Exemplo de Menu	34
3.9.3	Local de Menu	34
3.9.4	As Partes de um Menu	35
3.9.5	Entrada de Menu Menos Desordenada	35
3.9.6	Referenciando Outros Arquivos do Info	35
4	Estruturamento de Capítulo	37
4.1	Estrutura de Árvore das Seções	37
4.2	Tipos de Comandos Estruturantes	37
4.3	@chapter: Estruturamento de Capítulo	38
4.4	@unnumbered, @appendix: Capítulos com Outra Rotulagem	38
4.5	@majorheading, @chapheading: Cabeçalhos de Nível de Capítulo	39
4.6	@section: Seções Abaixo de Capítulos	39
4.7	@unnumberedsec, @appendixsec, @heading	39
4.8	@subsection: Subseções Abaixo de Seções	40
4.9	Os Comandos do Tipo @subsection	40
4.10	@subsubsection e Outros Comandos Subsub	40
4.11	@part: Grupos de Capítulos	40
4.12	Elevar/rebaixar Seções: @raisesections e @lowersections	41
4.13	Nós Desestruturados	42
5	Referências cruzadas	43
5.1	Diferentes Comandos de Referência Cruzada	43
5.2	Partes de Uma Referência Cruzada	43
5.3	@xref com Um Argumento	44
5.4	@xref com Dois Argumentos	45
5.5	@xref com Três Argumentos	45
5.6	@xref com Quatro e Cinco Argumentos	46
5.7	Referenciamento para um Manual como um Todo	47
5.8	@xref	48
5.9	@ref	48
5.10	@pxref	48
5.11	@anchor: Definindo Alvos Arbitrários de Referências Cruzadas	49
5.12	@link: Simples, hyperlink desadornado	50
5.13	@inforef: Referências cruzadas para material unicamente Info	50
5.14	@url, @uref{url[, texto][, substituição]}	50
5.14.1	Exemplos @url	51
5.14.2	Quebra de Linha de URL	51
5.14.3	@url Formato de Saída PDF	52
5.15	@cite{referência}	52
5.16	Cores do PDF	52
6	Marcação de Texto, Palavras e Frases	54
6.1	Indicando Definições, Comandos, etc.	54
6.1.1	Comandos de Realçamento são Úteis	54
6.1.2	@code{código-amostra}	55
6.1.3	@kbd{caracteres-de-teclado}	56
6.1.4	@key{nome-tecla}	57

6.1.5	<code>@samp{texto}</code>	57
6.1.6	<code>@verb{chartextchar}</code>	58
6.1.7	<code>@var{variável-metassintática}</code>	58
6.1.8	<code>@env{variável-ambiente}</code>	59
6.1.9	<code>@file{nome-arquivo}</code>	59
6.1.10	<code>@command{nome-comando}</code>	59
6.1.11	<code>@option{nome-opção}</code>	60
6.1.12	<code>@dfn{termo}</code>	60
6.1.13	<code>@abbr{abreviação[, significado]}</code>	60
6.1.14	<code>@acronym{sigla[, significado]}</code>	60
6.1.15	<code>@indicateurl{localizador-uniforme-de-recurso}</code>	61
6.1.16	<code>@email{endereço-correio-eletrônico[, texto-exibido]}</code>	62
6.2	Enfatizando Texto	62
6.2.1	<code>@emph{texto}</code> e <code>@strong{texto}</code>	62
6.2.2	<code>@sc{texto}</code> : A Fonte de Versaletes	62
6.2.3	Fontes para Impressão	63
7	Citações e Exemplos	65
7.1	Comandos de Cercamento de Blocos	65
7.2	<code>@quotation</code> : Citações de Bloco	66
7.3	<code>@indentedblock</code> : Blocos recuados de texto	67
7.4	<code>@example</code> : Texto de Exemplo	67
7.5	<code>@verbatim</code> : Texto Literal	68
7.6	<code>@lisp</code> : Marcando um Exemplo da Lisp	68
7.7	<code>@display</code> : Exemplos Usando a Fonte de Texto	69
7.8	<code>@format</code> : Exemplos Usando a Largura Total da Linha	69
7.9	<code>@exdent</code> : Desfazendo o Recuo de Uma Linha	69
7.10	<code>@flushleft</code> e <code>@flushright</code>	70
7.11	<code>@raggedright</code> : Texto Irregular à Direita	70
7.12	<code>@noindent</code> : Omitindo Recuo	71
7.13	<code>@indent</code> : Forçando Recuo	71
7.14	<code>@cartouche</code> : Retângulos Arredondados	72
7.15	<code>@small...</code> Comandos de Bloco	73
8	Listas e Tabelas	74
8.1	Listas Introducentes	74
8.2	<code>@itemize</code> : Construindo Uma Lista de Itens	74
8.3	<code>@enumerate</code> : Fazendo Uma Lista Numerada ou Com Letras	76
8.4	Fazendo Uma Tabela de Duas Colunas	77
8.4.1	Usando o Comando <code>@table</code>	77
8.4.2	<code>@ftable</code> e <code>@vtable</code>	78
8.4.3	<code>@itemx</code> : Segundo e Itens Subsequentes	78
8.5	<code>@multitable</code> : Tabelas Multi Colunas	79
8.5.1	Larguras de Colunas de Multi Tabelas	79
8.5.2	Linhas de Multi Tabelas	79
9	Exibições Especiais	81
9.1	Flutuadores	81
9.1.1	<code>@float [tipo][,rótulo]</code> : Material Flutuante	81
9.1.2	<code>@caption</code> e <code>@shortcaption</code>	82

9.1.3	<code>@listoffloats</code> : Sumários para Flutuadores.....	82
9.2	Inserindo Imagens.....	83
9.2.1	Sintaxe da Imagem.....	83
9.2.2	Escalonamento da Imagem.....	84
9.3	Notas de Rodapé.....	85
9.3.1	Comandos de Notas de Rodapé.....	85
9.3.2	Estilos de Notas de Rodapé.....	86
10	Índices.....	88
10.1	Índices Predefinidos.....	88
10.2	Definindo as Entradas de um Índice.....	89
10.3	Comandos Avançados de Indexação.....	90
10.4	Criando Entradas de Índice.....	90
10.5	Imprimindo Índices e Menus.....	91
10.6	Combinando Índices.....	92
10.6.1	<code>@syncindex</code> : Combinando Índices Usando <code>@code</code>	92
10.6.2	<code>@synindex</code> : Combinando Índices.....	93
10.7	Definindo Novos Índices.....	93
11	Inserções Especiais.....	95
11.1	Caracteres Especiais: Inserindo <code>@{}</code> , <code>\</code> , <code>#</code> &.....	95
11.1.1	Inserindo <code>'@'</code> com <code>@@</code> e <code>@atchar{}</code>	95
11.1.2	Inserindo <code>'{'</code> com <code>@{ @}</code> e <code>@l rbracechar{}</code>	95
11.1.3	Inserindo <code>' '</code> com <code>@comma{}</code>	95
11.1.4	Inserindo <code>'\'</code> com <code>@backslashchar{}</code>	96
11.1.5	Inserindo <code>'#'</code> com <code>@hashchar{}</code>	96
11.1.6	Inserindo <code>'&'</code> com <code>@&</code> e <code>@ampchar{}</code>	96
11.2	Inserindo Caracteres de Aspas.....	97
11.3	Inserindo Espaço.....	97
11.3.1	Espaços Múltiplos.....	98
11.3.2	Não Finalizando uma Frase.....	98
11.3.3	Finalizando uma Frase.....	99
11.3.4	<code>@frenchspacing val</code> : Espaçamento de Controle de Frase.....	99
11.3.5	<code>@dmn{dimensão}</code> : Formatar uma Dimensão.....	100
11.4	Inserindo Acentos.....	100
11.5	Inserindo Aspas.....	101
11.6	<code>@sub</code> e <code>@sup</code> : Inserindo Subscritos e Sobrescritos.....	102
11.7	Glifos para Texto.....	102
11.7.1	<code>@TeX{}</code> ($\TeX$) e <code>@LaTeX{}</code> ($\LaTeX$).....	103
11.7.2	<code>@copyright{}</code> (©).....	103
11.7.3	<code>@registeredsymbol{}</code> (®).....	103
11.7.4	<code>@dots (...)</code> e <code>@enddots (...)</code>	103
11.7.5	<code>@bullet (•)</code>	103
11.7.6	<code>@euro (€)</code> : Símbolo da Moeda Euro.....	103
11.7.7	<code>@pounds (£)</code> : Libras Esterlinas.....	103
11.7.8	<code>@textdegree (°)</code> : Símbolo de Graus.....	104
11.7.9	<code>@minus (-)</code> : Inserindo um Sinal de Menos.....	104
11.7.10	<code>@geq ($\geq$)</code> e <code>@leq ($\leq$)</code> : Inserindo Relações.....	104
11.8	Glifos para Programação.....	104
11.8.1	Sumário de Glifos.....	104
11.8.2	<code>@result{}</code> ($\Rightarrow$): Resultado de uma Expressão.....	104

11.8.3	<code>@expansion{}</code> ($\mapsto$): Indicando uma Expansão.....	105
11.8.4	<code>@print{}</code> ($\dashv$): Indicando Saída Gerada.....	105
11.8.5	<code>@error{}</code> (<code>\error</code>): Indicando uma Mensagem de Erro	106
11.8.6	<code>@equiv{}</code> ($\equiv$): Indicando Equivalência	106
11.8.7	<code>@point{}</code> ($\star$): Indicando Ponto em um Buffer	106
11.8.8	Sequências de Cliques.....	107
11.9	Inserindo Unicode: <code>@U</code>	107
12	Formatando Expressões Matemáticas	109
13	Forçando e Impedindo Quebras.....	110
13.1	Comandos de Quebra.....	110
13.2	<code>@*</code> e <code>@/</code> : Gerar e Permitir Quebras de Linha	110
13.3	<code>@-</code> e <code>@hyphenation</code> : Hifenização em Saída Impressa.....	111
13.4	<code>@allowcodebreaks</code> : Controle Quebras de Linha no <code>@code</code>	111
13.5	<code>@w{texto}</code> : Impedir Quebras de Linha	111
13.6	<code>@tie{}</code> : Inserindo um Espaço Inquebrável	112
13.7	<code>@sp n</code> : Inserir Linhas em Branco.....	112
13.8	<code>@page</code> : Comece uma Nova Página	112
13.9	<code>@group</code> : Impedir Quebras de Página.....	112
13.10	<code>@need mils</code> : Impedir Quebras de Página.....	113
14	Comandos de Definição.....	114
14.1	O Modelo Para Uma Definição.....	114
14.2	Linhas de Continuação de Comando de Definição	115
14.3	Argumentos Opcionais e Repetidos	115
14.4	Omitindo o Espaço Depois de um Nome de Definição	116
14.5	<code>@deffnx</code> , et al.: Duas ou Mais ‘Primeiras’ Linhas	116
14.6	Os Comandos de Definição	116
14.6.1	Funções e Entidades Similares	116
14.6.2	Funções em Linguagens Tipadas	117
14.6.3	Variáveis e Entidades Similares	119
14.6.4	Variáveis em Linguagens Tipadas	119
14.6.5	Tipos de Dados.....	120
14.6.6	Programação Orientada a Objetos	120
14.6.6.1	Variáveis Orientadas a Objetos.....	120
14.6.6.2	Métodos Orientados a Objetos	121
14.7	Comandos Genéricos de Definição.....	123
14.8	Convenções para Escrita de Definições	123
14.9	Uma Definição de Amostra de Função	124
15	Internacionalização	126
15.1	<code>@documentlanguage ll[_cc]</code> : Configurar o Idioma do Documento	126
15.2	<code>@documentencoding</code> <i>codificação</i> : Configurar Codificação de Entrada.....	127
16	Texto Condicionalmente Visível	129
16.1	Comandos Condicionais	129
16.2	Comandos Condicionais Não.....	130
16.3	Comandos Brutos do Formatador.....	131

16.4	Conditonais Inline: <code>@inline</code> , <code>@inlineifelse</code> , <code>@inlineraw</code>	131
16.5	Sinalizadores: <code>@set</code> , <code>@clear</code> , condiconais e <code>@value</code>	132
16.5.1	<code>@set</code> e <code>@value</code>	133
16.5.2	<code>@ifset</code> e <code>@ifclear</code>	134
16.5.3	<code>@inlineifset</code> e <code>@inlineifclear</code>	134
16.5.4	Exemplo de <code>@value</code>	135
16.6	Testagem para Comandos do Texinfo: <code>@ifcommanddefined</code> , <code>@ifcommandnotdefined</code>	136
16.7	Aninhamento de Condicional	136
17	Definindo Novos Comandos do Texinfo	138
17.1	Definindo Macros	138
17.2	Invocando Macros	139
17.3	Detalhes e Ressalvas de Macro	141
17.4	<code>'@alias novo=existente'</code>	144
17.5	Macros de Linha	144
17.6	<code>@definfoenclose</code> : Realçamento Personalizado	145
17.7	Processadores Externos de Macro: Diretivas de Linha	146
17.7.1	Diretiva <code>'#line'</code>	146
17.7.2	<code>'#line'</code> e $\text{\TeX}$	147
17.7.3	Detalhes da Sintaxe da <code>'#line'</code>	147
18	Arquivos de Inclusão	148
18.1	Como Usar Arquivos de Inclusão	148
18.2	Arquivo de Amostra com <code>@include</code>	148
18.3	<code>arquivo @verbatiminclude</code> : Incluir um Literal de Arquivo	149
19	Formatando e Imprimindo com $\text{\TeX}$	150
19.1	Use $\text{\TeX}$	150
19.2	Formatar com <code>texi2dvi</code> ou <code>texi2pdf</code>	150
19.3	Formatar com <code>tex/texindex</code>	152
19.3.1	Formatando Documentos Parciais	153
19.3.2	Detalhes do <code>texindex</code>	153
19.4	Preparando para $\text{\TeX}$	154
19.5	“hboxes” lotados	155
19.6	Imprimindo Japonês e Chinês	155
20	<code>texi2any</code>: O Tradutor para Texinfo	157
20.1	Invocando <code>texi2any</code> a partir de um Shell	157
20.2	Variáveis de Ambiente Reconhecidas pelo <code>texi2any</code>	164
20.3	Saída Impressa do <code>texi2any</code>	165
20.4	Variáveis de Personalização	166
20.4.1	Variáveis de Personalização para Comandos <code>@</code>	166
20.4.2	Variáveis e Opções de Personalização	167
20.4.3	Seleção do Formato de Saída	167
20.4.4	Variáveis Globais de Personalização	168
20.4.5	Variáveis de Personalização do Info e de Texto Simples	173
20.4.6	Variáveis de Personalização do $\text{\LaTeX}$	175
20.4.7	Outras Variáveis de Personalização de Formatos	176
20.4.8	Variáveis de Personalização para Múltiplos Formatos	176

20.5	Internacionalização de Sequências de Caracteres de Documentos.....	178
20.6	Invocando <code>pod2texi</code> : Converter Pod para Texinfo	179
20.6.1	<code>pod2texi</code>	179
20.7	<code>texi2html</code> : Ancestral do <code>texi2any</code>	181
21	Criando e Instalando Arquivos Info	183
21.1	Instalando um Arquivo do Info	183
21.1.1	O Arquivo de Diretório <code>dir</code>	183
21.1.2	Listando um Novo Arquivo Info.....	183
21.1.3	Arquivos Info em Outros Diretórios	184
21.1.4	Instalando Arquivos de Diretório Info.....	185
21.1.5	Invocando <code>install-info</code>	186
21.2	Arquivos de Etiqueta e Arquivos Divididos.....	188
21.3	Perguntas Frequentes do Formato Info.....	189
22	Gerando HTML.....	192
22.1	Tradução do HTML	192
22.2	Dividindo HTML.....	193
22.3	CSS do HTML	193
22.4	<code>@documentdescription</code> : Texto do Sumário.....	194
22.5	Gerando EPUB	195
22.5.1	Diretório do Contêiner e Arquivos da Saída Gerada	195
22.5.2	Referências Cruzadas do EPUB.....	195
22.5.3	Obtendo estrita conformação do EPUB	196
22.5.4	HTML Gerado para EPUB	196
22.6	Realçamento de Sintaxe de Exemplos de Códigos no HTML.....	196
22.7	Referências Cruzadas do HTML	197
22.7.1	Básicos do Linque de Referência Cruzada do HTML.....	197
22.7.2	Expansão de Nome de Nó de Referência Cruzada do HTML.....	198
22.7.3	Expansão do Comando de Referência Cruzada do HTML.....	199
22.7.4	Expansão do Caractere de 8 Bits da Referência Cruzada do HTML.....	201
22.7.5	Configuração de Referência Cruzada do HTML.....	201
22.8	Personalização da Saída Gerada HTML	204
22.8.1	Personalização da Estrutura da Saída Gerada HTML.....	204
22.8.2	Personalização de Nomes de Arquivo e de Linques para HTML.....	205
22.8.3	Personalização da Navegação e de Cabeçalhos.....	208
22.8.4	Personalização de Recursos do HTML.....	209
22.8.5	Personalização de Código HTML Inserido.....	209
22.8.6	Personalização da Saída Gerada Específica do HTML	210
22.8.7	Personalização do HTML para Matemática	211
22.8.7.1	Variáveis de Personalização da MathJax.....	212
22.8.7.2	Variáveis de Personalização do <code>latex2html</code>	213
22.8.7.3	Variáveis de Personalização do <code>tex4ht</code>	213
22.8.8	Interface e Licenças JavaScript	214
22.8.9	Lista de Variáveis de Personalização do HTML.....	214
Anexo A	Detalhes do Comando @.....	221
A.1	Sintaxe de Comando @.....	221
A.2	Lista de Comandos @	222
A.3	Contextos do Comando @.....	243
A.4	Comandos @ Obsoletos.....	244

Anexo B Dicas e Sugestões..... 245

Anexo C Arquivos de Amostra do Texinfo..... 249

C.1	Textos GNU de Amostra	249
C.2	Licença de Cópia Literal	251
C.3	Licença de Cópia Toda Permissiva.....	251

Anexo D Usando Modo Texinfo 252

D.1	Visão Geral do Modo Texinfo	252
D.2	Os Comandos Usuais de Edição do GNU Emacs.....	252
D.3	Inserindo Comandos Frequentemente Usados	253
D.4	Mostrando a Estrutura de Seccionamento de um Arquivo.....	254
D.4.1	Usando <code>texinfo-show-structure</code>	255
D.4.2	Usando <code>occur</code>	256
D.5	Atualizando Nós e Menus	256
D.5.1	Os Comandos de Atualização.....	257
D.5.2	Exigências de Atualização.....	259
D.5.3	Atualizar Arquivo Externo e Incluir Arquivos.....	259
D.5.4	Exigências de Arquivos de Inclusão.....	260
D.5.5	Outros Comandos de Atualização.....	261
D.6	Formatação para Info.....	261
D.6.1	Executando <code>texi2any/makeinfo</code> no Emacs.....	261
D.6.2	Os comandos <code>texinfo-format</code>	262
D.7	Formatando e Imprimindo com o Emacs.....	263
D.7.1	Formatação e Impressão no Modo Texinfo	263
D.7.2	Usando a Lista de Variáveis Locais	265
D.8	Sumário do Modo Texinfo.....	265
D.9	Formatação Direta de arquivos do Info	267
D.9.1	Etiquetando um Arquivo	268
D.9.2	Dividindo um Arquivo Manualmente	268
D.10	Capturando Erros	269
D.10.1	<code>texi2any</code> Preferido	269
D.10.2	Capturando Erros com formatação do Info	269
D.10.3	Depuração com <code>TeX</code>	270
D.10.4	Encontrando Nós Mal Referenciados.....	272
D.10.4.1	Usando <code>Info-validate</code>	272
D.10.4.2	Criando um Arquivo Desdividido e Adicionando uma Tabela de Etiquetas	272
D.11	Formatação em Lote	273

Anexo E Comandos Globais de Documento 274

E.1	<code>@setchapternewpage</code> : Páginas em Branco Antes dos Capítulos	274
E.2	Cabeçalhos de Página.....	275
E.2.1	O Comando <code>@headings</code>	275
E.2.2	Formatos Padrão de Cabeçalho.....	276
E.2.3	Como Fazer Teus Próprios Cabeçalhos.....	276
E.3	<code>@paragraphindent</code> : Controlando Recuo do Parágrafo.....	278
E.4	<code>@firstparagraphindent</code> : Recuo Depois de Cabeçalhos.....	279
E.5	<code>@exampleindent</code> : Recuo do Ambiente.....	279
E.6	<code>@smallbook</code> : Imprimindo Livros “Pequenos”	280
E.7	Imprimir em Papel A4	280

E.8	<code>@pagesizes [largura][, altura]</code> : Tamanhos Personalizados de Página	280
E.9	Microtipografia	280
E.10	Ampliação	281
Anexo F Especificação do Formato Info		282
F.1	Formato Info: Um Manual Inteiro	282
F.2	Formato Info: Preâmbulo	283
F.3	Formato Info: Tabela Indireta	283
F.4	Formato Info: Tabela de Etiquetas	283
F.5	Formato Info: Variáveis Locais	284
F.6	Formato Info: Nós Regulares	284
F.7	Formato Info: Menu	285
F.8	Formato Info: Imagem	285
F.9	Formato Info: Impressão de Índice	286
F.10	Formato Info: Referência Cruzada	287
Anexo G Licença GNU de Documentação Livre		289
Índice de Comandos		297
Índice de Variáveis		302
Índice Geral		305

Condições de Cópia do Texinfo

GNU Texinfo é *software livre*; isso significa que qualquer pessoa é livre para usá-lo e livre para redistribuí-lo sob certas condições. Texinfo não está em domínio público; ele está sob direitos autorais e existem restrições acerca da distribuição dele; porém essas restrições estão projetadas para permitir qualquer coisa que um(a) bom(a) cidadão(ã) cooperador(a) desejasse fazer. O que não for permitido é para tentar impedir que outros(as) adicionalmente compartilhem qualquer versão do Texinfo que eles(as) possam obter de você.

Especificamente, nós queremos ter certeza que você tem o direito de doar cópias dos programas que se relacionem ao Texinfo; que você receba código fonte ou, do contrário, consiga obtê-lo se assim o desejar; que você pode mudar esses programas ou usar pedaços deles em novos programas livres; e que você sabe que pode fazer essas coisas.

Para ter certeza de que qualquer pessoa tem tais direitos, nós temos que proibir que você prive alguém mais desses direitos. Por exemplo, se você distribuir cópias dos programas relacionados ao Texinfo, você precisa dar para os(as) receptores(as) todos os direitos que você tem. Você precisa assegurar-se de que eles(as), também, recebam ou possam conseguir o código fonte. E você precisa informá-los(as) dos direitos deles(as).

Além disso, para nossa própria proteção, nós devemos estar certos(as) de que qualquer pessoa sabe que não existe garantia para os programas que se relacionem ao Texinfo. Se esses programas forem modificados por qualquer outra pessoa e passados adiante, nós queremos que os(as) receptores(as) deles saibam que o que eles(as) tem não é o que nós distribuímos, de forma que quaisquer problemas introduzidos por outros(as) não refletirão na nossa reputação.

As condições precisas das licenças para os programas atualmente sendo distribuídos que se relacionem com o Texinfo são encontradas nas Licenças Gerais Públicas que os acompanham. Este manual está coberto pela Licença GNU de Documentação Livre (veja-se Anexo G [Licença GNU de Documentação Livre], Página 289).

1 Visão Geral do Texinfo

Texinfo é um sistema de documentação que usa um arquivo fonte único para produzir informações online e saída impressa. Isso significa que, em vez de escrever vários documentos, um para cada formato de saída, você precisa escrever somente um documento.

Usando Texinfo, você consegue criar um documento impresso (via sistema tipográfico $\text{T}_{\text{E}}\text{X}$) em formato PDF, incluindo capítulos, seções, referências cruzadas e índices. A partir do mesmo arquivo fonte do Texinfo, você consegue criar saída HTML adequada para uso com um navegador web; você consegue criar um arquivo do Info para uso no GNU Emacs ou outros programas de leitura do Info; e também criar arquivos DocBook, EPUB 3 ou $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$.

Um arquivo fonte do Texinfo é um arquivo de texto simples contendo texto intercalado com *comandos @* (palavras precedidas por um '@') que informam aos processadores Texinfo o que fazer. Os comandos de marcação do Texinfo são quase inteiramente *semânticos*; isto é, eles especificam o significado pretendido do texto no documento, em vez de instruções de formatação física.

A saída para a maioria dos formatos de saída é manuseada pelo programa `texi2any`. Veja-se Capítulo 20 [Tradutor Genérico `texi2any`], Página 157. A exceção é quando o código do Texinfo é processado pelo $\text{T}_{\text{E}}\text{X}$ para DVI ou PDF (veja-se Seção 1.4 [Livros Impressos], Página 5, abaixo). O Emacs também oferece comandos para converter código Texinfo para Info (veja-se Seção D.6.2 [Comandos `texinfo-format`], Página 262).

O GNU Emacs tem um modo especial, chamado modo Texinfo, que fornece vários recursos relacionados ao Texinfo. (Veja-se Anexo D [Modo Texinfo], Página 252).

Texinfo foi concebido especificamente para o propósito de escrever documentação e manuais de software. Se você quer escrever um bom manual para teu programa, Texinfo tem muitos recursos que nós esperamos que tornarão teu trabalho mais fácil. No entanto, Texinfo não é destinado para ser um programa de formatação de propósito geral. Ele quase não fornece comandos para controlar a formatação final, de forma que pode ser inapropriado para tuas necessidades se você quiser esquematizar um jornal, conceber um anúncio brilhante de revista ou seguir as exigências exatas de formatação de uma editora.

Escreva-se “Texinfo” com um “T” maiúsculo e as outras letras em minúsculas. A primeira sílaba de “Texinfo” é pronunciada como “speck”, e não “hex”. Essa pronúncia estranha é derivada da pronúncia de $\text{T}_{\text{E}}\text{X}$. Pronuncie-se $\text{T}_{\text{E}}\text{X}$ como se o ‘X’ fosse o último som no nome ‘Bach’. Na palavra $\text{T}_{\text{E}}\text{X}$, o ‘X’ é, em vez da letra do Inglês “ex”, na verdade a letra Grega “chi”.

Texinfo é o formato oficial de documentação do projeto GNU. Mais informações, incluindo manuais para pacotes GNU, estão disponíveis na página web da documentação GNU (<https://www.gnu.org/doc/>).

1.1 Informando Defeitos

Nós apreciamos informes de defeitos e sugestões para qualquer aspecto do sistema Texinfo: programas, documentação, instalação, etc. Por favor, envie-os para bug-texinfo@gnu.org. Você consegue obter a versão mais recente do Texinfo por meio da página inicial dele, <https://www.gnu.org/software/texinfo>.

Para informes de defeitos, por favor, inclua informações suficientes para os(as) mantenedores(as) reproduzirem o problema. Falando genericamente, isso significa:

- O número da versão do Texinfo e o(s) programa(s) ou manual(is) envolvido(s).
- O conteúdo de quaisquer arquivos de entrada necessários para reproduzir o defeito.
- Precisamente como você executou quaisquer programa(s) envolvido(s).
- Uma descrição do problema e amostras de quaisquer saídas errôneas.

- Nomes e versões do hardware e do sistema operacional.
- Qualquer outra coisa que você pense que pudesse ajudar.

Quando em dúvida se alguma coisa é necessária ou não, inclua-a. É melhor incluir muito mais que deixar de fora algo importante.

É crítico enviar um arquivo de entrada atual que reproduza o problema.

Quaisquer problemas com o leitor do Info no Emacs deveriam ser informados para os(as) desenvolvedores(as) do Emacs: veja-se Seção “Defeitos” em *O Manual do GNU Emacs*.

Remendos são bem-vindos; se possível, por favor, faça-os com ‘diff -c’, ‘diff -u’ (veja-se *Comparando e Mesclando Arquivos*), ou ‘git diff’ e inclua entradas do *ChangeLog* (veja-se Seção “Registros das Mudanças” em *Padrões GNU de Codificação*), e siga o estilo existente de codificação.

1.2 Formatos de Saída

Aqui está uma visão geral dos formatos de saída atualmente suportados por Texinfo.

- Info** (Gerado via `texi2any`). O formato Info é em sua maior parte transliteração de texto plano do fonte Texinfo. Ele adiciona uns poucos caracteres de controle para fornecer informação de navegação para referências cruzadas, índices, e assim por diante. O subsistema Emacs Info (veja-se *Info*), e o programa autônomo `info` (veja-se *GNU Info*), entre outros, conseguem ler esses arquivos. Veja-se Seção 1.3 [Arquivos do Info], Página 4, e Capítulo 21 [Criando e Instalando Arquivos Info], Página 183.
- Texto simples** (Gerado via `texi2any --plaintext`). Essa é quase a mesma que a saída Info com os caracteres de controle de navegação omitidos.
- HTML** (Gerado via `texi2any --html`). HTML, que significa Hyper Text Markup Language, é a linguagem para escrever documentos na World Wide Web. Os navegadores da Web conseguem renderizar essa linguagem online. Existem muitas versões de HTML, tanto diferentes usos mais comuns quanto variações específicas de navegador. `texi2any` usa um subconjunto da linguagem que consegue ser interpretado por qualquer navegador comum, intencionalmente não usando muitas etiquetas mais novas ou menos amplamente suportadas. Embora a saída nativa seja, portanto, bastante simples, ela pode ser personalizada em vários níveis, se desejado. Veja-se Capítulo 22 [Gerando HTML], Página 192.
- EPUB 3** (Gerada via `texi2any --epub3`). EPUB é um formato projetado para leitura de livros eletrônicos em dispositivos portáteis. É um derivado do HTML. O formato foi desenvolvido pelo International Digital Publishing Forum (IDPF), que agora faz parte do World Wide Web Consortium (W3C). A mais recente grande revisão, EPUB 3, data de 2011.
- DVI** (Gerada via `texi2dvi`). O formato binário DeVIce Independent é liberado pelo programa tipográfico T_EX (<http://tug.org>). Essa é então lida por um ‘controlador’ DVI, o qual conhece os atuais comandos específicos de dispositivo que podem ser visualizados ou impressos, notadamente Dvips para tradução para PostScript (veja-se *Dvips*) e Xdvi para visualização em uma tela do X (<http://sourceforge.net/projects/xdvi/>). Veja-se Capítulo 19 [Impresso com T_EX], Página 150. (Esteja ciente de que a linguagem do Texinfo é muito diferente das linguagens usuais do T_EX: T_EX simples, L^AT_EX, ConT_EXt, entre outras).
- PostScript** (Gerada via `texi2dvi --ps`). PostScript é uma linguagem de descrição de página que se tornou amplamente usada por volta de 1985 e ainda é usada hoje em dia.

<https://en.wikipedia.org/wiki/PostScript> dá uma descrição básica e mais preferências. Por padrão, Texinfo usa o programa `dvips` para converter a saída DVI do $\text{\TeX}$ para PostScript. Veja-se *Dvips*.

PDF (Gerada via `texi2dvi --pdf` ou `texi2pdf`). Esse formato foi desenvolvido pela Adobe Systems para intercâmbio de documento portátil, baseado na linguagem PostScript prévia deles. Ele consegue representar a exata aparência de um documento, incluindo fontes e gráficos, e suportar escalonamento arbitrário. Ele é entendido como sendo independente de plataforma e facilmente visualizável, entre outros objetivos de projeto; https://en.wikipedia.org/wiki/Portable_Document_Format e <http://tug.org/TUGboat/tb22-3/tb72beebe-pdf.pdf> tem algum conhecimento. Por padrão, Texinfo usa o programa `pdftex`, uma extensão de $\text{\TeX}$, para gerar PDF; veja-se <http://tug.org/applications/pdftex>. Veja-se [Saída PDF], Página 150.

$\text{\LaTeX}$ (Gerada via `texi2any --latex`). Esse é um sistema de tipografia construído sobre o $\text{\TeX}$. Foi lançado originalmente por Leslie Lamport em 1984. O $\text{\LaTeX}$ adiciona mais definições para aquelas do $\text{\TeX}$ e tem uma ampla gama de pacotes construídos sobre ele. O $\text{\LaTeX}$ é onipresente na literatura acadêmica. A versão atual do $\text{\LaTeX}$ está sob desenvolvimento ativo; mais informações estão disponíveis online em <https://www.latex-project.org/>.

A saída do $\text{\LaTeX}$ pode ser processada posteriormente em DVI, PostScript ou PDF. Em teoria, a saída do $\text{\LaTeX}$ deveria permitir muito mais personalização da saída do que seria possível com a implementação $\text{\TeX}$ simples do Texinfo.

DocBook (Gerada via `texi2any --docbook`). Esse é um formato baseado em XML, primariamente para documentação técnica. Ele, por conseguinte, assume alguma semelhança, em linhas gerais, com Texinfo. Veja-se <http://www.docbook.org>. Vários conversores de DocBook para Texinfo também foram desenvolvidos; vejam-se as páginas web do Texinfo.

XML (Gerada via `texi2any --xml`). A saída XML do `texi2any`, diferente de todos os outros formatos de saída, é uma transliteração do fonte do Texinfo, em vez de uma saída finalizada. Os arquivos XML do Texinfo não podem ser visualizados em navegadores da web ou outros programas.

XML é uma especificação genérica de sintaxe usável para qualquer tipo de conteúdo. (Uma referência está em <http://www.w3.org/XML>). O propósito da saída XML do Texinfo é o de permitir processamento posterior por ferramentas XML. A sintaxe de saída é definida em um DTD de XML, que está contido em um arquivo `texinfo.dtd` incluído na distribuição do fonte do Texinfo.

A distribuição do fonte do Texinfo inclui um script utilitário `txixml2texi` para fazer uma transformação reversa para recriar o conteúdo Texinfo original (exceto para macros e condicionais do Texinfo).

1.3 Arquivos do Info

Conforme mencionado acima, o formato Info é em sua maioria uma transliteração de texto plano do fonte Texinfo, com a adição de uns poucos caracteres de controle para separar nós e fornecer informação de navegação, de forma que os programas de leitura Info consigam operar sobre ele.

Os arquivos Info são quase sempre criados pelo processamento de um documento fonte do Texinfo. `texi2any`, também conhecido como `makeinfo`, é o comando principal que converte um arquivo Texinfo em um arquivo Info; veja-se Capítulo 20 [Tradutor Genérico `texi2any`], Página 157.

Geralmente, você adentra um arquivo Info via um nó que, por convenção, é chamado ‘Top’. Esse nó normalmente contém somente um sumário curto do propósito do arquivo, e um menu

amplo por meio do qual o restante do arquivo é alcançado. A partir desse nó, você ou pode atravessar o arquivo sistematicamente, indo de nó a nó; ou você pode ir até um nó específico listado no menu principal; ou você pode pesquisar no menu de índice e então ir diretamente ao nó que tenha a informação que você deseja. Alternativamente, com programa autônomo Info, você consegue especificar itens específicos de menu na linha de comando (veja-se *Info*).

Se você desejar ler ao longo de um arquivo Info em sequência, como se ele fosse um manual impresso, você pode teclar **ESPAÇO** repetidamente, ou você percorre o arquivo inteiro com o comando Info avançado **g ***. (Veja-se Seção “Comandos avançados do Info” em *Info*).

O arquivo **dir** no diretório **info** serve como o ponto de chegada para o sistema Info inteiro. A partir dele, você consegue alcançar os nós ‘Top’ de cada um dos documentos em um sistema Info completo.

Se você desejar se referir a um arquivo Info via URI, você pode usar a sintaxe (não oficial) exemplificada pelo seguinte. Isto funciona com Emacs/W3, por exemplo:

```
info:emacs#Dissociated%20Press
info:///usr/info/emacs#Dissociated%20Press
info://localhost/usr/info/emacs#Dissociated%20Press
```

O próprio programa **info** não segue URIs de qualquer tipo.

1.4 Livros Impressos

Um arquivo Texinfo pode ser formatado e tipografado como um livro ou um manual impresso. Para fazer isso, você precisa do **T_EX**, um sofisticado programa de tipografia escrito por Donald Knuth da Universidade Stanford. Ele não é parte da distribuição do Texinfo.

O Texinfo fornece um arquivo **texinfo.tex** que contém as definições que o **T_EX** usa quando tipografa um arquivo do Texinfo. Você consegue obter a versão mais recente do **texinfo.tex** a partir da página inicial do Texinfo, <https://www.gnu.org/software/texinfo/>.

Um livro baseado em Texinfo é similar a qualquer outro trabalho tipografado, impresso: ele pode ter uma página de título, página de direitos autorais, sumário, e prefácio, bem como capítulos, seções e subseções numeradas ou não numeradas, cabeçalhos de páginas, referências cruzadas, notas de rodapé, e índices.

T_EX é muito poderoso e tem muitos recursos ótimos. No entanto, como um arquivo do Texinfo precisa ser capaz de apresentar informações tanto em um terminal somente de caracteres no formato do Info quanto em um livro tipografado, os comandos de formatação que o Texinfo suporta são necessariamente limitados.

Veja-se Capítulo 19 [Impresso com **T_EX**], Página 150, para mais informações acerca do processamento de um manual com **T_EX**.

1.5 Histórico

Richard M. Stallman inventou o formato Texinfo, escreveu os processadores iniciais e criou a Edição 1.0 deste manual. Robert J. Chassell revisou e estendeu muito o manual, começando com a Edição 1.1. Brian Fox foi responsável pela distribuição autônoma do Texinfo até a versão 3.8. Karl Berry continuou a manutenção a partir do Texinfo 3.8 (edição do manual 2.22), e Gavin Smith continuou a manutenção desde o Texinfo 6.0.

Começo

Richard Stallman incluiu um sistema de ajuda de hipertexto online chamado Info na implementação original do Emacs (em 1975/6). Stallman foi inspirado depois de ver uma apresentação alguns anos antes acerca do sistema de hipertexto “NLS” de Douglas Engelbart.

Em outro desenvolvimento, na década de 1970 na CMU, Brian Reid desenvolveu um programa e formato chamado Scribe para marcar documentos para impressão. Ele usava o caractere @ para

introduzir comandos, como o Texinfo faz. Muito mais consequentemente, ele se esforçou para descrever o conteúdo do documento em vez da formatação, uma ideia adotada de todo o coração pelo Texinfo.

Enquanto isso, pessoas no MIT desenvolveram outro formato chamado Bolio. Richard Stallman (RMS) trabalhou na conversão do Bolio para usar $\text{\TeX}$ como a linguagem dele de tipografia, resultando no $\text{\BoTeX}$. A versão mais antiga do $\text{\BoTeX}$ parece ter sido 0.02 em 31 de outubro de 1984.

O $\text{\BoTeX}$ só podia ser usado como uma linguagem de marcação para documentos a serem impressos, não para documentos online. RMS combinou o $\text{\BoTeX}$ e o Info para criar o Texinfo, uma linguagem de marcação para texto que se destina a ser lido online e como cópia impressa.

O tradutor original para criar o Info foi escrito (principalmente por RMS e Bob Chassell) em Emacs Lisp, ou seja, o `texinfo-format-buffer` e outras funções. No início da década de 1990, Brian Fox reimplementou o programa de conversão em C, agora chamado `makeinfo`, bem como o programa autônomo `info`.

Reimplementando em Perl

Em 2012, o `makeinfo` do C foi ele próprio substituído por uma implementação Perl genericamente chamada `texi2any`. Essa versão foi inicialmente baseada no `texi2html`, um programa independente escrito originalmente por Lionel Cons, mais tarde com trabalho substancial de muitos(as) outros(as). Os recursos necessários para tornar `texi2html` um substituto para `makeinfo` foram implementados por Patrice Dumas. Esse programa, no entanto, nunca foi lançado.

A atual implementação (também escrita por Patrice Dumas) herdou o projeto de personalização e outros recursos do `texi2html` (para mais acerca da compatibilidade do `texi2html`, veja-se Seção 20.7 [`texi2html`], Página 181). No entanto, o `texi2any` foi uma reimplementação completa: diferente do `texi2html`, ele constrói uma representação baseada em árvore do documento de entrada para todas as estruturas de retaguarda trabalharem.

O novo programa Perl pretendia ter as seguintes vantagens sobre o antigo programa C:

- Personalização do HTML. Muitos pacotes GNU e outros softwares livres têm sido felizes no uso dos recursos de personalização de HTML no `texi2html` por anos. Assim, na verdade, duas implementações independentes da linguagem Texinfo foram desenvolvidas, e mantê-las em sincronia não era simples. Adicionar a possibilidade de personalização de HTML no `texi2html` a um programa C teria sido um esforço enorme na época.
- Unicode e suporte multilíngue em geral, especialmente de idiomas do leste asiático. Naquela época, fazer isso em C teria sido equivalente a reescrever o programa inteiro.
- Estruturas adicionais de retaguarda. O código do `makeinfo` se tornou complicado até o ponto onde adicionar-se uma nova estrutura de retaguarda era bastante complexo, exigindo interações complexas com estruturas de retaguarda existentes. Em contraste, a nova implementação fornece uma representação limpa baseada em árvore para todas as estruturas de retaguarda trabalharem. Pessoas têm solicitado numerosas estruturas de retaguarda ($\text{\LaTeX}$, o mais recente (X)HTML, ...); essa mudança as tornou muito mais viáveis de implementar.
- `texi2any` foi concebido para ser uma implementação de referência que define partes da linguagem não totalmente especificadas pelo manual e para fornecer uma referência para opções de linha de comando para qualquer outro programa que possa processar o Texinfo no futuro.

Na época, o novo programa Perl era muito mais lento que o antigo programa C. Desde então, o analisador, parte do conversor Info e o conversor HTML como um todo foram traduzidos para C, levando a importantes ganhos de velocidade comparado com a implementação Perl pura do `texi2any`. O programa principal também foi traduzido para C. A implementação C do `texi2any` embute um interpretador Perl para as partes que exigem Perl.

1.6 Adicionando Formatos de Saída

Os formatos de saída nas seções anteriores lidam com uma variedade ampla de usos, porém, certamente, sempre existe espaço para mais.

Se você for um(a) programador(a) e gostaria de contribuir para com o projeto GNU implementando formatos adicionais de saída para o Texinfo, isso seria excelente. A maneira de fazer isso que seria mais útil é a de escrever uma estrutura de retaguarda nova para `texi2any`, nossa implementação de referência de um analisador Texinfo; ela cria uma representação de árvore da entrada de Texinfo que você consegue usar para a conversão. A documentação no arquivo fonte `tta/perl/Texinfo/Convert/Converter.pm` é um bom lugar para começar se você estiver disposto(a) a desenvolver em Perl (veja-se Seção “Texinfo::Convert::Converter” em *documentação dos módulos do Texinfo*). Capítulo 20 [Tradutor Genérico `texi2any`], Página 157.

Outra abordagem viável é a de usar a saída XML do Texinfo oriunda do `texi2any` como tua entrada. Esse XML é uma representação essencialmente completa da entrada, porém sem as peculiaridades da sintaxe e de opção do Texinfo, conforme descrito acima.

Se você ainda não conseguir resistir à tentação de escrever um programa novo que leia diretamente o fonte do Texinfo, permita-nos te dar mais algumas advertências: por favor, não subestime a quantidade de trabalho exigida. Texinfo não é de forma alguma uma linguagem simples de se analisar corretamente, e permanece sob desenvolvimento, de maneira que você estaria se comprometendo com uma tarefa em andamento.

Testes extensivos da linguagem e do processador são desenvolvidos com `texi2any`; nós encorajamos qualquer pessoa pensando em escrever um programa para processar o Texinfo a usar esses casos de teste e verificar se eles fornecem resultados corretos com teu novo programa.

De tempos em tempos, propostas são feitas para gerar páginas de manual Unix tradicionais a partir do fonte do Texinfo. Entretanto, pelo motivo de que as páginas de manual tem um formato convencional estrito, criar uma página de manual boa exige um fonte completamente diferente daquele necessário para as aplicações Texinfo típicas de escrita de um tutorial de usuário(a) bom e (ou) um bom manual de referência. Isso torna a geração de páginas de manual incompatível com o objetivo do projeto do Texinfo de não ter de documentar a mesma informação de maneiras diferentes para diferentes formatos de saída. Você poderia também escrever a página de manual diretamente.

Como um meio alternativo para suportar páginas de manual, você possivelmente ache que o programa `help2man` seja útil. Ele gera uma página de manual tradicional a partir da saída ‘`--help`’ de um programa. De fato, as páginas de manual para os programas na distribuição do Texinfo são geradas com isso. Ele é software GNU escrito por Brendan O’Dea, disponível a partir de <https://www.gnu.org/software/help2man>.

2 Escrevendo um Arquivo do Texinfo

Este capítulo descreve a sintaxe do Texinfo e o que é exigido em um arquivo do Texinfo, e dá um arquivo curto de amostra.

2.1 Convenções Sintáticas Gerais

Esta seção descreve as convenções gerais usadas em todos os documentos do Texinfo.

- Todos os caracteres ASCII imprimíveis, exceto ‘@’, ‘{’ e ‘}’, podem aparecer em um arquivo do Texinfo e representam eles mesmos. ‘@’ é o caractere de encapsulamento que introduz comandos, enquanto ‘{’ e ‘}’ são usados para envolver argumentos a certos comandos. Para colocar um desses caracteres especiais no documento, coloque um caractere ‘@’ em frente a ele, como isto: ‘@@’, ‘@{’, e ‘@}’.
- Em um arquivo do Texinfo, os comandos que você escreve para descrever o conteúdo do manual são precedidos por um caractere ‘@’; eles são chamados de *comandos @*. (O ‘@’ no Texinfo tem o mesmo significado que o ‘\’ tem no $\text{\TeX}$ simples).

Dependendo do que eles fazem ou de quais argumentos recebam, você precisa escrever comandos @ em linhas próprias, ou como parte de frases. Como regra geral, um comando exige chaves se ele se misturar com outro texto; mas não precisa de chaves se estiver em uma linha própria. Para mais detalhes da sintaxe de comandos do Texinfo, veja-se Seção A.1 [Sintaxe de Comando], Página 221.

- Espaço em branco seguindo um nome de comando @ é opcional e (normalmente) ignorado se presente. As exceções são contextos quando o espaço em branco é significativo, por exemplo, um ambiente de `@example`.
- Texinfo suporta as aspas normais usadas em Inglês e em outros idiomas; veja-se Seção 11.5 [Inserindo Aspas], Página 101.
- Use três hifens em uma linha, ‘---’, para produzir um traço longo—como esse (chamado um *travessão*), usado para pontuação em frases. Use dois hifens, ‘--’, para produzir um traço médio (chamado um *traço de ligação*), usado primariamente para intervalos numéricos, como em “Junho 25–26”. Use um hífen simples, ‘-’, para produzir um hífen padrão usado em palavras compostas. Para exibir na tela, Info reduz três hifens para dois e dois hifens para um (não transitivamente!). Certamente, qualquer número de hifens no fonte permanece como estiverem em contextos literais, tais como `@code` e `@example`.
- *Espaço em branco*. Arquivos do Texinfo são arquivos de texto simples compostos de linhas terminadas pelo usual caractere de nova linha (avanço de linha). Processadores do Texinfo leem a entrada uma linha por vez. Parágrafos são terminados por uma linha vazia ou uma linha contendo somente espaços. Uma sequência de vários espaços no texto geralmente é tratada da mesma forma que um espaço (exceto em modos literais).

Caracteres de avanço de formulário (*CTRL-1*) encerram qualquer parágrafo aberto. Outros avanços de formulários, bem como outros espaços em branco ASCII (tabulação, retorno de carro) são tratados da mesma forma que caracteres de espaço (embora os resultados possam diferir dependendo do formato de saída). Portanto, não existe muito sentido em usá-los em documentos. Espaços não ASCII, como Unicode “em space”, não são reconhecidos como espaços em branco e serão tratados como caracteres regulares, não espaços em branco.

No entanto, em modos literais, por exemplo, em amostras de código, caracteres de tabulação podem produzir a formatação correta na saída gerada.

2.2 Comentários

Você pode escrever comentários em um arquivo do Texinfo usando o comando `@comment`, o qual pode ser abreviado para `@c`. Tais comentários são para uma pessoa examinando o arquivo fonte

do Texinfo. Todo o texto em uma linha que se seguir ou a `@comment` ou a `@c` é um comentário; o restante da linha não aparece na saída gerada visível. (Para ser preciso(a), o caractere depois do `@c` ou do `@comment` deve necessariamente ser algo que não um traço ou alfanumérico, ou será tomado como sendo parte do comando).

Frequentemente, você pode escrever o `@comment` ou o `@c` no meio de uma linha, e somente o texto que se seguir depois do comando `@comment` ou `@c` não aparece; porém alguns comandos, tais como `@settitle`, funcionam sobre uma linha inteira. Você não pode usar `@comment` ou `@c` dentro de uma linha que se inicie com um tal comando.

Em casos de invocações aninhadas de comando, definições complicadas de macro, etc., `@c` e `@comment` podem provocar um erro quando do processamento com $\text{\TeX}$. Por conseguinte, você também pode usar o caractere `DEL` (decimal 127 ASCII, hexadecimal 0x7f, octal 0177) como um verdadeiro caractere de comentário do $\text{\TeX}$ (código de categoria 14, nos internos do $\text{\TeX}$). Tudo na linha depois de `DEL` será ignorado e a próxima linha será mesclada.

Você também pode ter trechos longos de texto ignorados pelos processadores Texinfo com os comandos `@ignore` e `@end ignore`. Escreva cada um desses comandos em uma linha própria, iniciando cada comando no começo da linha. O texto entre esses dois comandos não aparece na saída gerada processada. Você pode usar `@ignore` e `@end ignore` para escrever comentários. (Para algumas advertências relativamente ao aninhamento de tais comandos, veja-se Seção 16.7 [Aninhamento de Condicional], Página 136).

2.3 O Que um Arquivo do Texinfo Precisa Ter

Por convenção, o nome de um arquivo do Texinfo termina com uma das extensões `.texi`, `.texinfo`, `.txi` ou `.tex`. Usar `.tex` é desencorajado, pois essa extensão já é usada pelos arquivos de entrada do $\text{\TeX}$ e do $\text{\LaTeX}$. A extensão mais comum e recomendada é `.texi`. O nome de um arquivo do Texinfo deveria conter somente caracteres ASCII.

O nome da saída gerada é baseado no nome do arquivo de entrada, no caso padrão. Primeiro, quaisquer das extensões `.texi`, `.tex`, `.txi` ou `.texinfo` são removidas do nome do arquivo de entrada; a seguir, a extensão específica do formato da saída gerada é adicionada—`.html` ao gerar HTML, `.info` ao gerar Info, etc. O nome da saída gerada deveria conter somente caracteres ASCII¹.

Para a finalidade de ser transformado em um manual impresso, um arquivo do Texinfo precisa iniciar com uma linha como esta:

```
\input texinfo
```

O conteúdo do arquivo segue esse início, e então você precisa finalizar o fonte do Texinfo com uma linha como esta:

```
@bye
```

A linha `@bye` no final do arquivo em uma linha própria informa ao $\text{\TeX}$ que o arquivo está terminado e para parar a formatação. Se deixar isso de fora, você será despejado(a) no prompt do $\text{\TeX}$ no final da execução.

Além disso, você normalmente fornecerá um arquivo do Texinfo com um título, uma página de título, índices e similares, todos os quais estão explicados neste manual. Mas o mínimo, que pode ser útil para documentos curtos, é apenas a linha no início e a linha no final.

Sem informações adicionais, as codificações de entrada e saída são assumidas como UTF-8, um conjunto universal de códigos compatível com ASCII de 7 bits.

¹ `texi2any` consegue manusear caracteres não ASCII em nomes de arquivo de entrada, mas caracteres não ASCII no nome da saída gerada criarão problemas para alguns formatos de saída, especialmente para referências cruzadas.

2.4 Amostra Curta

Aqui está um arquivo curto de amostra do Texinfo.

```
\input texinfo
@settitle Manual do Sample 1.0

@copying
Este é um exemplo curto de um arquivo completo do Texinfo.

Copyright @copyright{} 2026 Free Software Foundation, Inc.
@end copying
@documentinfo
@title Sample Title
@end documentinfo
@maketitle

@contents

@node Top
@top GNU Sample

Este manual é para GNU Sample
(versão @value{VERSION}, @value{UPDATED}).

@node Primeiro Capítulo
@nodedescription O primeiro capítulo é o único capítulo nesta amostra.
@chapter Primeiro Capítulo

@cindex capítulo, primeiro
Este é o primeiro capítulo.
@cindex entrada de índice, outra

Aqui está uma lista numerada.

@enumerate
@item
Este é o primeiro item.

@item
Este é o segundo item.
@end enumerate

@node Primeira Seção
@section Primeira Seção

Primeira seção do primeiro capítulo.

@node Segunda Seção
@section Segunda Seção
```

Segunda seção do primeiro capítulo.

```
@node Index
@nodedescription Complete Index
@unnumbered Index

@printindex cp

@bye
```

2.5 Cabeçalho do Arquivo do Texinfo

Arquivos do Texinfo iniciam com a linha:

```
\input texinfo
```

A linha ‘`\input texinfo`’ diz ao $\text{\TeX}$ para usar o arquivo `texinfo.tex`, que diz ao $\text{\TeX}$ como traduzir os comandos `@` do Texinfo para comandos de tipografia do $\text{\TeX}$. (Observe o uso da barra invertida, ‘`\`’; isso é correto para o $\text{\TeX}$).

Faz sentido incluir qualquer comando que afete a formatação do documento como um todo no cabeçalho. A linha `@settitle` geralmente está presente no começo do cabeçalho:

```
@settitle Manual do Sample 1.0
```

A linha `@settitle` especifica um título para os cabeçalhos de página (ou rodapés) do manual impresso, e o título padrão e a descrição do documento para ‘`<head>`’ em HTML. `@synindex` (veja-se Seção 10.6.2 [`@synindex`], Página 93), por exemplo, é outro comando frequentemente incluído no cabeçalho.

O início do arquivo do Texinfo até o primeiro conteúdo que é gerado como parte do corpo principal do documento é o *preâmbulo*. Ele inclui o cabeçalho, Seção 2.7 [Permissões do Documento], Página 15, e a especificação Seção 2.8 [Página de Título e Página de Copirraite], Página 17. É importante para o formato de saída do $\text{\LaTeX}$, pois o fim do preâmbulo é onde a linha `\begin{document}` é gerada. Em outros formatos de saída, ele pode ser usado para determinar como alguma saída especial é formatada, por exemplo, a saída Seção 2.7.1 [`@copying`], Página 16, como um comentário no início dos arquivos de saída ou o idioma usado nos cabeçalhos de arquivo.

2.5.1 A Primeira Linha de um Arquivo do Texinfo

Cada arquivo do Texinfo que deva ser a entrada de nível superior para $\text{\TeX}$ precisa começar com uma linha semelhante a esta:

```
\input texinfo
```

Quando o arquivo for processado pelo $\text{\TeX}$, o comando ‘`\input texinfo`’ diz ao $\text{\TeX}$ para carregar as macros necessárias para processar um arquivo do Texinfo. Elas estão em um arquivo chamado `texinfo.tex`, que deveria ter sido instalado no teu sistema ou junto com o software $\text{\TeX}$ ou com o Texinfo. O $\text{\TeX}$ usa a barra invertida, ‘`\`’, para marcar o início de um comando, exatamente como o Texinfo usa ‘`@`’. O arquivo `texinfo.tex` causa a comutação de ‘`\`’ para ‘`@`’; antes que a comutação ocorra, o $\text{\TeX}$ exige ‘`\`’, e é por isso que ela aparece no início do arquivo.

Opcionalmente, você pode seguir essa linha com um comentário para informar ao GNU Emacs para usar o modo Texinfo quando o arquivo for editado:

```
\input texinfo @c -*-texinfo*-
```

Isso possivelmente seja útil quando o Emacs não detectar automaticamente o tipo do arquivo a partir da extensão do arquivo.

2.5.2 @setfilename: Configure o Nome do Arquivo da Saída Gerada

A linha `@setfilename` especifica o nome do arquivo de saída a ser gerado por `texi2any`. Esse comando é ignorado para formatação `TEX`. Quando presente, deveria ser o primeiro comando do Texinfo (isto é, depois de `\input texinfo`). Escreva o comando `@setfilename` no início de uma linha e siga-o na mesma linha pelo nome do arquivo do Info.

```
@setfilename nome-arquivo-info
```

O nome precisa ser diferente do nome do arquivo do Texinfo. Existem duas convenções para escolher o nome: você pode ou remover a extensão (como `.texi`) inteiramente do nome do arquivo de entrada ou (recomendado) substituí-la pela extensão `.info`. Não é aconselhável basear o nome de `@setfilename` em um nome totalmente diferente do nome do arquivo de entrada.

Quando uma linha `@setfilename` está presente, os processadores do Texinfo ignoram tudo escrito antes da linha `@setfilename`. É por isso que a primeiríssima linha do arquivo (a linha `\input`) não aparece na saída gerada.

Se não existir nenhuma linha `@setfilename`, `texi2any` usará o nome do arquivo de entrada para determinar o nome da saída (veja-se Seção 2.3 [Mínimo], Página 9). A linha `\input` ainda será ignorada nesse processamento, assim como as linhas em branco iniciais.

Ao produzir outro formato de saída, `texi2any` substituirá qualquer extensão final pela extensão específica do formato de saída (`.html` ao gerar HTML, por exemplo) ou adicionará um ponto seguido pela extensão (`.html` para HTML) se o nome fornecido não tiver extensão.

Comandos `@` não são permitidos em `@setfilename`, exceto para `@@`, `@{`, `@}` e comandos `@` associados, como `@atchar{}`.

`@setfilename` costumava ser exigido pelos processadores do Texinfo e por alguns outros programas. Isso não mais deveria ser o caso; `@setfilename` pode ser omitido. Se a entrada do Texinfo for processada a partir da entrada padrão, sem um nome de arquivo de entrada para deduzir o nome base do arquivo, `@setfilename` ainda poderia ser relevante. No entanto, essa não é a única maneira: a opção `--output` especifica o nome do arquivo de saída na linha de comando do `texi2any` (veja-se Seção 20.1 [Invocando `texi2any`], Página 157).

Embora uma extensão `.info` explícita seja preferível, alguns sistemas operacionais não conseguem lidar com nomes longos de arquivo. Você pode ter um problema mesmo quando o nome do arquivo que especificar for curto o suficiente. Isso ocorre porque os formatadores do Info dividem um arquivo longo do Info em subarquivos curtos indiretos e os nomeiam pospondo `-1`, `-2`, ..., `-10`, `-11` e assim por diante ao nome original do arquivo. (Veja-se Seção 21.2 [Arquivos de Etiqueta e Divididos], Página 188). O nome do subarquivo `texinfo.info-10`, por exemplo, é muito longo para sistemas antigos com um limite de 14 caracteres sobre nomes de arquivo; portanto, o nome do arquivo do Info para esse documento poderia ser `texinfo` em vez de `texinfo.info` em tal sistema. `@setfilename` é uma maneira de especificar um nome alternativo.

2.5.3 @settitle: Configura o Título do Documento

Um arquivo Texinfo deveria conter uma linha que se pareça com isto:

```
@settitle título
```

Escreva o comando `@settitle` no início de uma linha e siga-o na mesma pelo título. Não escreva nada mais na linha. O comando `@settitle` deveria preceder tudo o que gera saída atual. O melhor lugar para ele é logo depois do comando `@setfilename`, se presente (descrito na seção anterior).

Esse comando fornece o título para se usar em um cabeçalho ou rodapé para saída impressa frente e verso, caso tais títulos sejam produzidos. Para mais acerca de títulos para saída impressa, veja-se Seção 2.8.4 [Geração de Títulos], Página 21.

Em HTML, *title* serve como o ‘<title>’ do documento e se torna a descrição padrão do documento na parte ‘<head>’.

Quando a página de título for usada na saída gerada, como geralmente é o caso para saída impressa, o título no comando `@settitle` não afeta o título conforme ele aparece na página de título. Assim, os dois não precisam não corresponder exatamente. Uma prática que nós recomendamos é a de incluir o número da versão ou o da edição do manual no título `@settitle`; na página de título, o número da versão geralmente aparece como um `@subtitle`, de forma que ele seria omitido do `@title`. Veja-se Seção 2.8 [Página de Título e Página de Copirraite], Página 17.

2.5.4 Preâmbulo

O *preâmbulo* começa no início do arquivo do Texinfo e continua até o primeiro material diretamente produzido. Ele tipicamente inclui o cabeçalho do arquivo (veja-se Seção 2.5 [Cabeçalho do Arquivo do Texinfo], Página 11), o bloco `@copying` especificando as permissões do documento (veja-se Seção 2.7.1 [`@copying`], Página 16) e as especificações da página de título (veja-se Seção 2.8 [Página de Título e Página de Copirraite], Página 17).

O preâmbulo pode conter comandos que afetam a formatação do documento como um todo, mas que não produzem saída, ou não produzem saída imediatamente, como `@settitle` (veja-se Seção 2.5.3 [`@settitle`], Página 12), `@documentlanguage`, (veja-se Seção 15.1 [`@documentlanguage`], Página 126), comandos que definem os títulos, controlam o recuo ou a hifenização, ou o sumário (veja-se Seção 2.9 [Conteúdo], Página 21).

Qualquer texto que inicia um parágrafo, comandos `@` que são formatados como citações, tabelas, listas e assim por diante, e `@node` (veja-se Capítulo 3 [Nós], Página 26) e comandos de estruturação de capítulos (veja-se Capítulo 4 [Estruturamento de Capítulo], Página 37) encerram o preâmbulo.

O conceito de preâmbulo é significativo para a saída do L^AT_EX, pois a linha `\begin{document}` é gerada no final do preâmbulo.

Em texto simples, o preâmbulo é simplesmente gerado como de costume no início do documento; por exemplo, um `@contents` no preâmbulo é gerado como o sumário (veja-se Seção 2.9 [Conteúdo], Página 21).

Também não existe muito tratamento especial do preâmbulo para saída HTML e Info. No entanto, algumas configurações atuais no finalzinho do preâmbulo podem ser usadas para o documento como um todo, independentemente do que se segue. Isso pode se aplicar a comandos que especificam o recuo ou o idioma (veja-se Seção 15.1 [`@documentlanguage`], Página 126).

Por exemplo, para o documento a seguir, os comentários de cópia de HTML e de Info são formatados com `@documentlanguage` configurado como ‘pt’, pois é o último `@documentlanguage` antes do final do preâmbulo.

```

\input texinfo
@documentlanguage fr

@copying
A informação de cópia @error{} algum texto
@end copying

@documentlanguage pt

Texto finalizando o preâmbulo

@documentlanguage de

@node Top

```

2.5.5 Início e Fim do Cabeçalho para Emacs

No Emacs, as linhas de início e de fim de cabeçalho podem ser usadas para cercar comandos que afetam globalmente o documento no preâmbulo do Texinfo. Isso permite que você formate somente parte de um arquivo do Texinfo para Info ou impressão. Veja-se Seção D.6.2 [Comandos `texinfo-format`], Página 262.

Uma linha de início de cabeçalho é um comentário do Texinfo que se parece com isto:

```
@c %**start of header
```

Escreva a linha de início de cabeçalho na segunda linha de um arquivo do Texinfo. Siga os comandos de início de cabeçalho que afetam globalmente a formatação do documento, como `@settitle`, `@synindex` ou `@footnotestyle`; e então por uma linha `end-of-header`.

Uma linha de fim de cabeçalho é um comentário do Texinfo que se parece com isto:

```
@c %**end of header
```

A estranha sequência de caracteres, ‘%**’, é para garantir que nenhum outro comentário seja acidentalmente tomado como uma linha de início de cabeçalho. Você pode mudá-la se desejar, configurando as variáveis do Emacs `tex-start-of-header` e (ou) `tex-end-of-header`. Veja-se Seção D.7.1 [Impressão no Modo Texinfo], Página 263.

As linhas de início e fim do cabeçalho não são parte da especificação do formato do Texinfo, e é por isso que são implementadas com comentários.

2.6 Categoria de Diretório

Use o comando `@dircategory` para especificar uma categoria para o manual.

Comandos `@dircategory` geralmente são seguidos por blocos `@direntry`, que são usados pelo `install-info`. Veja-se Seção 21.1.4 [Instalando Entradas de Diretório], Página 185, para um exemplo de como esses comandos são usados.

O primeiro comando `@dircategory` em um manual é a categoria para o manual inteiro. Usos subsequentes de `@dircategory` configuram a categoria somente para blocos `@direntry` seguintes.

Não existe consistência no uso de categorias de diretório entre manuais, e é improvável que alguma vez exista, já que os(as) autores(as) trabalham na documentação do Texinfo independentemente, sem nenhuma coordenação central que resultaria em tal consistência.

É possível para usuários(as) ou para distribuições de sistemas operacionais editarem categorias em arquivos do Info ou do Texinfo, se uma organização lógica dentro de arquivos `dir` for algo que eles(as) valorizem.

Autores(as) de manuais podem considerar usar algumas das seguintes categorias para manuais. Para um pacote que instale muitos manuais (como o Emacs), pode ser uma boa ideia usar categoria específica de pacote, ou categorias, para agrupar os manuais dele.

- Arquivamento
- Áudio
- Negócios e produtividade
- Compressão
- Base de dados
- Dicionários
- Tradução de documentação
- Editores
- Educação
- Correio eletrônico
- Fontes
- Jogos
- Gráficos
- Saúde
- Interface
- Aplicações de Internet
- Comunicações ao vivo
- Localização
- Matemática
- Música
- Aplicações de rede de intercomunicação
- Impressão
- Segurança
- Ciência
- Desenvolvimento de software
- Bibliotecas de software
- Planilhas
- Administração do sistema
- Telefonia
- Criação e manipulação de texto
- Controle de versão
- Vídeo
- Autoria web

2.7 Permissões do Documento

Este segmento descreve o documento e contém o aviso de direitos autorais e permissões de cópia. Isso é feito com o comando `@copying`. Um manual real inclui mais texto aqui, de acordo com a licença sob a qual ele é distribuído.

`@copying`

Este é um exemplo curto de um arquivo completo do Texinfo, versão 1.0.

Copyright @copyright{} 2026 Free Software Foundation, Inc.

`@end copying`

O aviso de direitos autorais e as permissões de cópia para um documento precisam aparecer em vários lugares nos vários formatos da saída gerada do Texinfo. Portanto, o Texinfo fornece um comando (`@copying`) para declarar esse texto uma vez, e outro comando (`@insertcopying`) para inserir o texto nos pontos apropriados.

Para saída de texto simples, você precisa inserir explicitamente as informações de copirraite se quiser que elas apareçam, pois de outra forma não seriam exibidas. Por exemplo, você poderia ter o seguinte próximo ao início do arquivo Texinfo:

```
@ifplaintext
@insertcopying
@end ifplaintext
```

Você poderia incluir outras informações semelhantes as de título para a saída de texto simples no mesmo lugar.

Se o documento for um manual de software, o software tipicamente está sob uma licença diferente—para GNU e muitos outros pacotes de software livre, o software geralmente é lançado sob a GNU GPL, e os manuais são lançados sob a GNU FDL. É útil declarar a licença do software do manual, mas fornecer o texto completo da licença do software não é necessariamente exigido.

2.7.1 @copying: Declarar Permissões de Cópia

O comando `@copying` deveria ser fornecido bem no início do documento; o local recomendado é logo depois do material do cabeçalho (veja-se Seção 2.5 [Cabeçalho do Arquivo do Texinfo], Página 11). Ele consiste convencionalmente em uma ou duas frases acerca do que é o programa, identificação da documentação em si, a linha jurídica de direitos autorais e as permissões de cópia. Aqui está um exemplo esquelético:

```
@copying
Este manual é para programa (versão versão, atualizado data), que ...
Copyright @copyright{} anos titular-copyright.

@quotation
Permissão é concedida para ...
@end quotation
@end copying
```

O `@quotation` não tem significado jurídico; ele está lá para melhorar a legibilidade em alguns contextos.

O texto de `@copying` aparece como um comentário no início dos arquivos da saída gerada do Info e de HTML. Essas informações também são geradas no início dos arquivos da saída gerada do DocBook usando-se marcação apropriada. Elas *não* são geradas implicitamente em texto simples ou em saída impressa; cabe inteiramente a você usar `@insertcopying` para emitir as informações de cópia. Veja-se a próxima seção para detalhes.

O comando `@copyright{}` gera um ‘c’ dentro de um círculo quando o formato da saída gerada suportar esse glifo (impressa e HTML sempre suportam, por exemplo). Quando o glifo não for suportado na saída gerada, ele gera a sequência de três caracteres ‘(C)’.

O aviso de direitos autorais em si tem o seguinte formato juridicamente prescrito:

```
Copyright © anos titular-copyright.
```

A palavra ‘Copyright’ precisa sempre estar escrita em inglês, mesmo que o documento esteja escrito em outro idioma. Isso é devido ao direito internacional.

A lista de anos deveria incluir todos os anos nos quais uma versão foi concluída (mesmo se ela foi lançada em um ano subsequente). É mais simples que cada ano esteja escrito individualmente e por completo, separado por vírgulas.

O(A) proprietário(a) (ou proprietários(as)) dos direitos autorais é quem titularizar os direitos autorais sobre a obra. No caso de obras atribuídas à FSF, o(a) proprietário(a) é ‘Free Software Foundation, Inc.’.

A ‘linha’ de direitos autorais pode, na verdade, ser dividida em várias linhas, tanto no documento fonte quanto na saída gerada. Isso geralmente acontece para documentos com um longo histórico, tendo muitos anos de publicação. Se você usar várias linhas, não recue nenhuma delas (ou qualquer outra coisa no bloco `@copying`) no arquivo fonte.

Veja-se Seção “Copyright Notices” em *Informações do(a) Mantenedor(a) GNU*, para informações adicionais. Veja-se Seção C.1 [Textos GNU de Amostra], Página 249, para o texto completo a ser usado nos manuais GNU. Veja-se Anexo G [Licença GNU de Documentação Livre], Página 289, para a licença em si sob a qual o GNU e outros manuais livres são distribuídos.

2.7.2 @insertcopying: Incluir Texto de Permissões

O comando `@insertcopying` simplesmente é escrito em uma linha própria, como esta:

```
@insertcopying
```

Isso insere o texto previamente definido por `@copying`. Para atender às exigências jurídicas, ele precisa ser usado na página de direitos autorais no manual impresso (veja-se [Direitos Autorais], Página 19).

O próprio comando `@copying` faz com que o texto das permissões apareça em um arquivo Info *antes* do primeiro nó. O texto também é copiado no início de cada arquivo dividido de saída Info, como é juridicamente necessário. Esse local implica que um humano lendo o manual usando Info *não* vê esse texto (exceto quando usar o comando avançado `g *` do Info). Isso não importa para propósitos jurídicos, pois o texto está presente. Mas, para obter um texto visível na saída gerada, `@insertcopying` deveria ser usado.

Similarmente, o texto `@copying` é automaticamente incluído no início de cada arquivo de saída HTML, como um comentário HTML. De novo, esse texto não é visível sem `@insertcopying` (a menos que o(a) leitor(a) visualize o fonte HTML).

O texto de permissões definido por `@copying` também aparece automaticamente no início dos arquivos de saída DocBook usando-se marcação apropriada. `@insertcopying` pode ser usado para gerar o texto de permissão dentro do texto normal.

2.8 Páginas de Título e de Direitos Autorais

A primeira página de um livro impresso geralmente é a *página de título*, a qual contém o nome do livro e o(a) autor(a). As informações de copirraite e de publicação geralmente são impressas na página seguinte; nós nos referimos a essa página como a *página de copirraite*. Como essa está atrás (no verso) da página de título, essa página também é conhecida como o *verso do título*.

O Texinfo fornece comandos para você fornecer o título do documento, qualquer subtítulo, o nome do(a) autor(a) (ou nomes de vários(as) autores(as)) e informações de cópia e de publicação para o documento. Esses *metadados* são usados para o conteúdo da página de título e da página de copirraite. Da mesma forma, essas informações aparecem na saída gerada, em alguma forma, para outros formatos de saída, mesmo se não exibidas como a página de título.

O Texinfo fornece pelo menos três maneiras para fornecer esses metadados e para gerar as páginas de título e de copirraite, as quais estão descritas nas seguintes subseções.

Em HTML, a melhor maneira de obter uma página de título semelhante a manuais impressos é a de configurar a variável de personalização `NO_TOP_NODE_OUTPUT` (veja-se [NO_TOP_NODE_OUTPUT], Página 176).

Para documentos suficientemente simples, e para a página órfã de título na tradicional página preliminar de livro, o Texinfo também fornece um comando, `@shorttitlepage`, que leva o restante da linha como o título. O argumento é tipografado em uma página própria e seguido por uma página em branco. No HTML, `@shorttitlepage` consegue desempenhar o mesmo papel que `@settitle`, se `@settitle` não estiver configurado. Veja-se Seção 2.5.3 [`@settitle`], Página 12.

2.8.1 Metadados do Documento e o Comando @maketitle

A maneira mais simples para definir metadados de documento usados para a página de título é a de usar-se o comando @documentinfo. Aqui está um modelo:

```
@documentinfo
@title nome-do-manual-quando-impresso
@subtitle subtítulo-se-existir
@subtitle segundo-subtítulo
@author autor(a)
@end documentinfo
```

Você só deveria usar o comando @title uma vez dentro do bloco @documentinfo. Você pode usar o comando @subtitle mais que uma vez, e a ordem em que quaisquer comandos @subtitle aparecem no bloco é mantida quando o arquivo do Texinfo for processado. Da mesma forma, você pode usar @author mais que uma vez. (No entanto, a ordem dos comandos @title, @subtitle e @author em relação um ao outro não é significativa).

As informações para a página de copirraite aparecem dentro dos comandos @copying e @publication. Veja-se Seção 2.7 [Permissões do Documento], Página 15, para mais discussão do @copying.

O comando @publication funciona muito similarmente a @copying. Os processadores de Texinfo salvam o conteúdo desse bloco quando o encontram no fonte de Texinfo. Você usa esse comando para fornecer informações acerca da publicação do documento, como o endereço do(a) editor(a) ou o ISBN. Você usa esse bloco para informações que seriam “impressas” ou “colofões” em um livro impresso.

Finalmente, para realmente gerar as páginas de título e de copirraite, você usa o comando @maketitle. Escreva esse comando sozinho em uma linha. Isso reúne as informações disponíveis para produzir essas duas páginas.

Esses comandos seriam usados assim:

```
@copying
informações-cópia
@end copying
@publication
Publicado por nome-editor(a)
endereço-editor(a)
@end publication
@maketitle
```

2.8.2 @titlepage

Uma alternativa ao uso de @maketitle para criar páginas de título e de copirraite é a de usar o comando @titlepage. Qualquer material entre as linhas @titlepage e @end titlepage é enviado diretamente para um manual impresso. Aqui está um exemplo básico:

```
@titlepage
@title Título de Amostra
@c Os dois comandos a seguir iniciam a página de copirraite.
@page
@vskip 0pt plus 1filll
@insertcopying

Publicado por ...

Arte de capa por ...
@end titlepage
```

Inicie o material para a página de título e seguinte à página de direitos autorais com `@titlepage` em uma linha própria e finalize-a com `@end titlepage` em uma linha própria.

O material antes da linha `@page` é usado para a página de título. As subseções a seguir explicam as duas maneiras pelas quais o comando `@titlepage` pode ser usado em um documento do Texinfo para produzir uma página de título, a primeira mais próxima do método anterior com `@documentinfo`, a segunda com formatação mais explícita.

Porém, a página de copirraite é sempre produzida da mesma maneira com `@titlepage`.

Página de Direitos Autorais

Usando o comando `@page`, você consegue forçar uma quebra de página dentro da região delimitada pelos comandos `@titlepage` e `@end titlepage` e assim criar mais que uma página não numerada. É assim que a página de direitos autorais é produzida. (O comando `@titlepage` talvez pudesse ter sido melhor nomeado de comando `@titleandadditionalpages`, porém isso poderia ter sido até certo ponto longo!)

Para empurrar o aviso de copirraite e o outro texto na página de copirraite em direção ao final da página, use o seguinte encantamento depois de `@page`:

```
@vskip 0pt plus 1filll
```

O comando `@vskip` insere espaço em branco na saída $\text{\TeX}$; ele é ignorado em todos os outros formatos de saída. O `'0pt plus 1filll'` significa colocar em zero pontos de espaço em branco obrigatório, e tantos espaços em branco opcionais quantos sejam necessários para empurrar o texto seguinte para o pé da página. Observe o uso de três `'l's` na palavra `'filll'`; isso está correto.

Para inserir o próprio texto de direitos autorais, escreva `@insertcopying` depois (veja-se Seção 2.7 [Permissões do Documento], Página 15):

```
@insertcopying
```

(Isso é exigido para gerar o texto mediante cópia ao se usar `@titlepage`, ao contrário de `@maketitle` onde esse texto é gerado automaticamente).

Siga o texto de direitos autorais pelo(a) editor(a), números ISBN, créditos da arte de capa, e outras tais informações.

2.8.2.1 @title, @subtitle e @author

Uma maneira de se usar `@titlepage` é a de se usar os comandos `@title`, `@subtitle` e `@author` dentro do bloco do `@titlepage`. Com esse método, você não especifica nenhuma formatação real da página de título. Você especifica o texto que quiser e o Texinfo faz a formatação. A formatação usual consiste de riscos pretos sob as linhas do título e do autor e o texto do subtítulo definido alinhado para o lado direito da página.

Você consegue usar os comandos `@title`, `@subtitle`, e `@author` para criar uma página de título na qual o espaçamento vertical e horizontal seja feito para você automaticamente.

Escreva os comandos `@title`, `@subtitle`, ou `@author` no início de uma linha seguido pelo título, subtítulo, ou autor. O comando `@author` pode ser usado para uma citação em um bloco `@quotation` (veja-se Seção 7.2 [`@quotation`], Página 66); exceto para isso, é um erro usar quaisquer desses comandos fora de `@titlepage`.

O comando `@title` normalmente produz uma linha na qual o título é configurado rente ao lado esquerdo da página em uma fonte mais larga que a normal. O título é sublinhado com uma linha preta. O título deve necessariamente ser dado em uma linha no arquivo fonte; ele será quebrado em múltiplas linhas de saída se necessário.

Para títulos longos, o comando `@*` pode ser usado para especificar as quebras de linha em títulos longos se as quebras automáticas não servirem. Tais quebras explícitas de linha geralmente

são refletidas em todos os formatos de saída; se você quiser somente especificá-las para a saída impressa, use um condicional (veja-se Capítulo 16 [Condicionais], Página 129). Por exemplo:

```
@title Este Título Longo@inlinefmt{tex,*} É Quebrado no @TeX{}
```

O comando `@subtitle` normalmente configura subtítulos em uma fonte de tamanho normal rente ao lado direito da página.

O comando `@author` normalmente configura os nomes do(a) autor(a) ou autores(as) em uma fonte de tamanho médio rente ao lado esquerdo da página em uma linha próxima ao pé da página de título. Os nomes são seguidos por uma linha preta que é mais fina que a linha que normalmente sublinha o título.

Existem duas maneiras de se usar o comando `@author`: você consegue escrever o nome ou nomes na parte restante da linha que inicia com um comando `@author`:

```
@author por Jane Smith e John Doe
```

ou você pode escrever os nomes um acima do outro, usando múltiplos comandos `@author`:

```
@author Jane Smith
@author John Doe
```

Um modelo para esse método se parece com isto:

```
@titlepage
@title nome-do-manual-quando-impresso
@subtitle subtítulo-se-algum
@subtitle segundo-subtítulo
@author autor
@page
...
@end titlepage
```

2.8.2.2 @titlefont, @center e @sp

O Texinfo também fornece um segundo método para criar uma página de título, usando comandos de tipografia que não são usados no texto principal. Esse método usa os comandos `@titlefont`, `@sp` e `@center` para gerar uma página de título na qual as palavras na página ficam centralizadas.

Use o comando `@titlefont` para selecionar uma fonte grande adequada para o título em si. Você pode usar `@titlefont` mais que uma vez se tiver um título especialmente longo.

Para saída HTML, cada comando `@titlefont` produz um cabeçalho `<h1>`, porém o `<title>` do documento HTML não é afetado. Para isso, você poderia colocar um comando `@settitle` antes do comando `@titlefont` (veja-se Seção 2.5.3 [`@settitle`], Página 12).

Por exemplo:

```
@titlefont{Texinfo}
```

Use o comando `@center` no início de uma linha para centralizar o restante do texto naquela linha. Assim,

```
@center @titlefont{Texinfo}
```

centraliza o título, o qual neste exemplo é “Texinfo” impresso na fonte de título.

Use o comando `@sp` para inserir espaço vertical. Por exemplo:

```
@sp 2
```

Isso insere duas linhas em branco na página impressa. (Veja-se Seção 13.7 [`@sp`], Página 112, para mais informações acerca do comando `@sp`).

Um modelo para esse método se parece com isto:


```

@titlepage
@sp 10
@center @titlefont{nome-do-manual-quando-impresso}
@sp 2
@center subtítulo-se-algum
@sp 2
@center autor
...
@end titlepage

```

O espaçamento do exemplo se encaixa em um manual de 8,5 por 11 polegadas.

2.8.3 Exibindo a Versão ou a Edição na Página de Título

Quando você escrever um manual acerca de um programa de computador, você deveria escrever, na página de título, a versão do programa para a qual o manual se aplica. Você tipicamente incluiria isso em um subtítulo. Se o manual mudar mais frequentemente que o programa ou for independente dele, você também deveria incluir um número de edição² para o manual. Isso ajuda os(as) leitores(as) a se manterem informados(as) sobre qual manual é para qual versão do programa. (O nó ‘Top’ também deveria conter essa informação; veja-se Seção 2.10 [O Nó Top], Página 22).

de direitos autorais.

2.8.4 Geração de Títulos

Texinfo tem dois formatos padrão de cabeçalho de página, um para documentos impressos em um lado de cada folha de papel (impressão de lado único), e o outro para documentos impressos em ambos os lados de cada folha (impressão de lado duplo).

Em generalidade plena, você consegue controlar os cabeçalhos de diferentes maneiras:

- A maneira convencional é a de escrever um comando `@setchapternewpage` antes dos comandos de página de título.

A maioria dos documentos é formatada com os cabeçalhos padrão lado único ou lado duplo, (ocasionalmente) usando `@setchapternewpage odd` para impressão em lado duplo e (quase sempre) sem comando `@setchapternewpage` para impressão de lado único (veja-se Seção E.1 [`@setchapternewpage`], Página 274).

- Alternativamente, você pode usar o comando `@headings` para evitar que os cabeçalhos de página sejam gerados ou para iniciá-los para, ou impressão de lado único, ou duplo. Para desligar os cabeçalhos, escreva `@headings off`. Veja-se Seção E.2.1 [`@headings`], Página 275.
- Ou, você pode especificar o teu próprio formato de cabeçalho e de rodapé de página. Veja-se Seção E.2 [Cabeçalhos], Página 275.

2.9 Gerando um Sumário

`@chapter`, `@section`, e outros comandos de estruturamento (veja-se Capítulo 4 [Estruturamento de Capítulo], Página 37) fornecem a informação para produzir um sumário, porém eles não fazem com que uma tabela real apareça no manual. Para fazer isso, você deve necessariamente usar o(s) comando(s) `@contents` e (ou) `@summarycontents`.

`@contents`

Gera um sumário em um manual impresso, incluindo todos os capítulos, seções, subseções, etc., bem como apêndices e capítulos não numerados. Os cabeçalhos

² Entendeu-se que é útil se referir a versões de manuais independentes como ‘edições’ e versões de programas como ‘versões’; do contrário, nós achamos que estamos sujeitos a confundir cada um em conversas, referenciando a ambos, a documentação e o software, com as mesmas palavras.

gerados por `@majorheading`, `@chapheading`, e os outros comandos `@...heading` não aparecem no sumário (veja-se Seção 4.2 [Tipos de Comandos Estruturantes], Página 37).

`@shortcontents`

`@summarycontents`

(`@summarycontents` é um sinônimo para `@shortcontents`).

Gera um índice curto ou sumário que lista somente os capítulos, apêndices e capítulos não numerados. Seções, subseções e subsubseções são omitidas. Somente um manual longo precisa de um sumário curto em adição ao sumário completo.

Ambos os comandos de conteúdo deveriam ser escritos em uma linha para cada um deles, e colocados próximo do início do arquivo, depois do `@end titlepage` (veja-se Seção 2.8.2 [`@titlepage`], Página 18), antes de qualquer comando de seccionamento. Os comandos de conteúdo automaticamente geram um cabeçalho estilo capítulo no topo da primeira página do sumário, de forma que não inclua qualquer comando de seccionamento, tal qual `@unnumbered` antes deles.

Dado que um arquivo Info usa menus em vez de sumários, os comandos de formatação do Info ignoram os comandos de conteúdo. Porém, o conteúdo é incluído na saída de texto simples e em outros formatos de saída, tais como HTML.

Na saída HTML, os links no sumário curto apontam para as entradas correspondentes no sumário completo, em vez do texto do documento. Os links no sumário completo apontam para o texto principal do documento.

`@shortcontents` não está implementado para saída do L^AT_EX.

2.10 O Nó ‘Top’

O nó ‘Top’ é o nó no qual um(a) leitor(a) acessa um manual Info. Como tal, ele deveria conter uma descrição muito breve do manual (incluindo o número de versão). O conteúdo do nó ‘Top’ não aparece em saída impressa nem na saída DocBook.

É convencional escrever uma linha com um comando `@top` contendo o título do documento imediatamente depois da linha `@node Top` (veja-se Seção 3.5 [Comando `@top`], Página 29).

Nós repetimos a descrição curta desde o início do texto ‘`@copying`’, mas não existe necessidade de repetir as informações de direitos autorais, de forma que nós não usamos ‘`@insertcopying`’ aqui.

Na saída Info (e às vezes na saída HTML), o nó ‘Topo’ geralmente contém um *menu* de nível superior listando os capítulos, e possivelmente um *menu detalhado* listando todos os nós em todo o documento. Geralmente não é necessário fornecer tal menu no fonte do Texinfo, pois `texi2any` gera esses menus automaticamente. No entanto, o exemplo a seguir mostra como você usaria o comando `@menu` para fornecer um menu no nó ‘Topo’:

```
@node Top
```

```
@top Amostra Curta
```

```
Esta é uma amostra curta do arquivo do Texinfo.
```

```
@menu
```

```
* Primeiro Capítulo:: O primeiro capítulo é o
                        único capítulo nesta amostra.
```

```
* Index::             Índice completo.
```

```
@end menu
```

2.10.1 Partes de um Menu Mestre

Um *menu mestre* é o menu principal. É costumeiro incluir um menu detalhado listando todos os nós no documento nesse menu. Como qualquer outro menu, um menu mestre é envolvido em `@menu` e `@end menu` e não aparece na saída impressa nem na saída DocBook.

O menu mestre contém entradas para os principais nós no arquivo do Texinfo: os nós para os capítulos, seções do tipo capítulo e os apêndices, seguidos por nós para os índices.

Você pode escolher seguir essas entradas com um *menu detalhado*. Isso lista outros nós de nível inferior, frequentemente ordenados pelo capítulo. Esses itens podem ser uma conveniência para um inquirido que pode ir diretamente para um nó específico ao se procurar por informações específicas, em vez de se passar por um menu intermediário. Se você usar um menu detalhado no teu menu mestre, marque-o com o ambiente `@detailmenu ... @end detailmenu`.

Cada seção no menu pode ser introduzida por uma linha descritiva. Contanto que a linha não se inicie com um asterisco, ela não será tratada como uma entrada de menu. (Veja-se Seção 3.9.1 [Escrevendo um Menu], Página 33, para mais informações).

Por exemplo, o menu mestre para este manual se parece com o seguinte (porém tem muito mais entradas):

```
@menu
* Condições de Cópia:: Teus direitos.
* Visão Geral::          Texinfo em resumo.
...
* Índice de Variáveis::
* Índice Geral::

@detailmenu
--- A Listagem Detalhada do Nó ---

Visão geral do Texinfo

* Informando Defeitos:: ...
...

Iniciando um Arquivo do Texinfo

* Início de Amostra:: ...
...
@end detailmenu
@end menu
```

2.11 O Corpo do Documento

O segmento do corpo contém todo o texto do documento. Um manual é dividido em um ou mais nós (veja-se Capítulo 3 [Nós], Página 26). O exemplo ilustra um capítulo feito de três nós, um para material introdutório no capítulo e duas seções. O material introdutório contém uma lista enumerada.

```
@node Primeiro Capítulo
@chapter Primeiro Capítulo

@cindex capítulo, primeiro
Este é o primeiro capítulo.
@cindex entrada de índice, outra
```

Aqui está uma lista numerada.

```
@enumerate
@item
Este é o primeiro item.
```

```
@item
Este é o segundo item.
@end enumerate
```

```
@node Primeira Seção
@section Primeira Seção
```

Primeira seção do primeiro capítulo.

```
@node Segunda Seção
@section Segunda Seção
```

Segunda seção do primeiro capítulo.

Na saída Info, o nó ‘Primeiro Capítulo’ conterá um menu listando as duas seções no capítulo. Da mesma forma, quando esse nó for gerado no próprio arquivo HTML, ele conterá um sumário para o capítulo.

Aqui está com o que o conteúdo desse capítulo se parecerá:

1. Primeiro Capítulo

Este é o primeiro capítulo

Aqui está uma lista numerada.

1. Este é o primeiro item
2. Este é o segundo item

1.1 Primeira Seção

Primeira seção do primeiro capítulo.

1.2 Segunda Seção

Segunda seção do primeiro capítulo.

(Na saída Info e HTML, o capítulo também seria dividido em nós).

2.12 Finalizando um Arquivo do Texinfo

O final de um arquivo Texinfo deveria incluir comandos para criar índices (veja-se Seção 10.5 [Imprimindo Índices e Menus], Página 91), e o comando `@bye` para marcar a última linha a ser processada. Por exemplo:

```
@node Índice
@unnumbered Índice

@printindex cp

@bye
```

Um comando `@bye` termina o processamento Texinfo. Ele deveria estar em uma linha própria. Qualquer coisa seguinte a `@bye` é completamente ignorada.

3 Nós

Um *nó* é uma região de texto que inicia no comando `@node`, e continua até o próximo comando `@node`. Para especificar um nó, escreva um comando `@node` no início de uma linha, e siga-o com o nome do nó. Os leitores Info exibem um nó por vez, e fornecem comandos para o(a) usuário(a) se movimentar para nós relacionados. A saída HTML pode ser navegada similarmente.

Nós são usados como os alvos de referências cruzadas. Referências cruzadas, tais quais aquelas ao final desta frase, são feitas com `@xref` e comandos relacionados; veja-se Capítulo 5 [Referências Cruzadas], Página 43. Referências cruzadas podem ser colocadas em qualquer parte do texto. Outros comandos `@` também podem ser o alvo de referências cruzadas (veja-se Seção 5.11 [`@anchor`], Página 49, veja-se Seção 9.1 [Flutuadores], Página 81).

Normalmente, você coloca um comando de nó imediatamente antes de cada comando de estruturação de capítulo—por exemplo, uma linha `@section` ou `@subsection`. (Veja-se Capítulo 4 [Estruturamento de Capítulo], Página 37). Você deve necessariamente fazer isso, ainda que você não pretenda formatar o arquivo para Info. Isso é por que saída impressa usa ambos, nomes `@node` e nomes de estruturação de capítulo, na saída para referências cruzadas. A única vez que você similarmente usa os comandos de estruturação de capítulo sem também usar nós é se você estiver escrevendo um documento que não contenha referências cruzadas e somente será impresso, não transformado em Info, HTML, ou outros formatos.

3.1 Escrevendo uma Linha de `@node`

Escreva `@node` no início de uma linha seguida pelo nome do nó, como isto:

```
@node nome-do-nó
```

Depois de se ter inserido uma linha `@node`, você deveria imediatamente escrever um comando `@` para o capítulo ou seção associado (se existir) e inserir o nome desse(a).

Você pode opcionalmente seguir o argumento de nome de nó para `@node` com até três argumentos opcionais no restante da mesma linha, separando os argumentos com vírgulas. Esses são os nomes dos ponteiros ‘Próximo’, ‘Anterior’, e ‘Acima’, nessa ordem. Assim, o modelo para uma linha de nó completamente preenchida com os ponteiros ‘Próximo’, ‘Anterior’, e ‘Acima’ se parece com isto:

```
@node nome_do_nó, next, previous, up
```

O argumento *nome_do_nó* precisa estar presente, porém os outros são opcionais. Se você desejar especificar algum, porém não outros, apenas insira vírgulas conforme necessário, como em: `@node meu_nó,,,nó_superior`. Entretanto, se teu documento Texinfo for hierarquicamente organizado, como virtualmente todos são, nós recomendamos deixar desligados todos os ponteiros e permitir que `texi2any` os determine.

O programa `texi2any` automaticamente determina ponteiros de nó para um documento hierarquicamente organizado. Para ele fazer isso, cada comando `@node` deveria ser seguido imediatamente por um comando de seccionamento, como `@chapter` ou `@section` (exceto que linhas de comentários podem entrevir). Finalmente, você precisa seguir a linha `@node` do ‘Top’ com uma linha iniciando com `@top` para marcar o nó de nível superior no arquivo. Veja-se Seção 3.5 [Comando `@top`], Página 29.

Mesmo quando especificar explicitamente todos os ponteiros, você não pode escrever os nós no arquivo fonte do Texinfo em uma ordem arbitrária. Você precisa escrever os nós na ordem que deseja que eles apareçam na saída gerada. Para o formato Info, alguém pode imaginar que a ordem possivelmente não importe, mas importa para os outros formatos.

Na maioria dos casos, você desejará tirar vantagem do recurso de criação de ponteiros e não redundantemente especificar ponteiros de nó que os programas possam determinar. Entretanto, não são exigidos que documentos Texinfo estejam organizados hierarquicamente ou, de fato,

tenham comandos de seccionamento (por exemplo, se você nunca pretender que o documento seja impresso), de forma que os ponteiros de nó ainda podem ser especificados explicitamente, em completa generalidade.

Se você estiver usando o GNU Emacs e quiser ponteiros explícitos, poderá usar os comandos de atualização de nós fornecidos pelo modo Texinfo para inserir os nomes dos ponteiros. (Veja-se Seção D.5 [Atualizando Nós e Menus], Página 256).

Alternativamente, você pode inserir você mesmo(a) os ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’. Se você fizer isso no Emacs, poderá achar útil usar o comando de teclado do modo Texinfo `C-c C-c n`. Esse comando insere `@node` e uma linha de comentários listando os nomes dos ponteiros na ordem apropriada deles. A linha de comentários ajuda você a acompanhar quais argumentos são para quais ponteiros.

3.2 Escolhendo Nomes de Nós

O nome de um nó identifica o nó. Para todos os detalhes dos nomes de nós, veja-se Seção 3.3 [Exigências da Linha do Nó], Página 28).

Aqui estão algumas sugestões para nomes de nós:

- Tente escolher nomes de nó que sejam informativos, porém curtos.
No arquivo do Info, o nome do arquivo, o nome do nó e os nomes de ponteiros são todos inseridos em uma linha, que podem ser encontrados na borda direita da janela. (Isso não causa um problema com o Info, mas é feio).
- Tente escolher nomes de nó que diferem entre si próximos ao início de seus nomes. Dessa forma, é fácil usar a completção automática de nomes no Info.
- Convencionalmente, os nomes dos nós são capitalizados da mesma maneira que os títulos de seção e de capítulo. Neste manual, palavras iniciais e significativas são capitalizadas; outros não são. Em outros manuais, apenas palavras iniciais e substantivos adequados são capitalizados. De qualquer maneira está bem; nós recomendamos apenas ser consistente.
- Os nomes dos nós não deveriam conter macros definidas por usuário(a). Veja-se Seção 17.3 [Detalhes de Macro], Página 141, para outras restrições no uso de macros.
- Na saída gerada HTML, quaisquer caracteres no nome do nó que não sejam letras, números ou espaços ASCII simples serão mudados no nome do arquivo. (Veja-se Seção 22.7.2 [Expansão de Nome de Nó do Xref do HTML], Página 198). Isso pode tornar os URLs para as páginas no teu manual menos amigáveis; por exemplo, neste manual, o nó ‘@dots’ é gerado como `__0040dots.html`.

Como os nomes dos nós são usados em referências cruzadas, não é desejável mudá-los casualmente uma vez publicados. Quando você deletar ou renomear um nó, geralmente é uma boa ideia definir um `@anchor` com o nome antigo. Dessa forma, as referências a partir de outros manuais, a partir de arquivamentos de correio e assim por diante não serão invalidados. Veja-se Seção 5.11 [`@anchor`], Página 49.

Os ponteiros a partir de um dado nó te habilitam a atingir outros nós e consistem simplesmente dos nomes desses nós.

Normalmente, um ponteiro do nó ‘Acima’ contém o nome do nó cujo menu menciona esse nó. O ponteiro do nó ‘Próximo’ contém o nome do nó que segue o nó atual nesse menu e o ponteiro ‘Anterior’ dele contém o nome do nó que o precede nesse menu. Quando um nó ‘Anterior’ do nó é o mesmo que o nó dele ‘Acima’, ambos os ponteiros nomeiam o mesmo nó.

Geralmente, o primeiro nó de um arquivo do Texinfo é o nó ‘Top’, e ponteiro dele ‘Acima’ aponta para o arquivo `dir`, que contém o menu principal para todos do Info.

3.3 Exigências da Linha de @node

Os nomes usados com @node tem várias exigências:

- Todos os nomes de nó em um arquivo Texinfo precisam ser únicos.
Isso significa, por exemplo, que, se você finalizar cada capítulo com um sumário, você precisa nomear cada nó de sumário diferentemente. Você não pode simplesmente chamá-los todos de “Sumário”. Você pode, entretanto, duplicar os títulos dos capítulos, seções, e afins. Assim, você pode finalizar cada capítulo com uma seção chamada “Sumário”, contanto que os nomes de nó para tais seções sejam todos diferentes.

Nomes de nós, nomes de âncoras (veja-se Seção 5.11 [`@anchor`], Página 49), e rótulos de flutuadores (veja-se Seção 9.1.1 [`@float`], Página 81) precisam todos serem únicos.

- Nomes de nós podem conter comandos @¹. Por exemplo, usar-se `@TeX{}` em um nome de nó resulta no logotipo do TeX sendo exibido, como seria em texto normal. Referências cruzadas deveriam usar `@TeX{}`, assim como o nome de nó usa.

Alguns comandos não fazem sentido em nomes de nó; por exemplo, ambientes (por exemplo, `@quotation`), comandos que leem uma linha inteira como argumento deles (por exemplo, `@sp`), e muitos outros. Para a lista completa dos comandos que são permitidos, e as expansões deles para identificadores HTML e nomes de arquivo, veja-se Seção 22.7.3 [Expansão de Comando do Xref do HTML], Página 199.

- Um nome de nó não pode iniciar com um abre parênteses precedendo um fecha parênteses, como em (não)permitido, dado que tal sintaxe é usada para especificar um manual externo.
- Infelizmente, você não pode confiavelmente usar pontos, vírgulas ou dois pontos dentro de um nome de nó; esses podem confundir alguns leitores Info. `texi2any` aspeia nomes e rótulos problemáticos de nós por padrão, mas alguns leitores Info não reconhecem essa sintaxe. O aspeamento de nomes e etiquetas de nó faz com que caracteres DEL (`‘Ctrl-?’`, número do caractere 127, geralmente renderizado como `‘^?’`) apareçam em torno do nome. Para remover o aspeamento de nomes e rótulos de nó, você pode configurar a variável de personalização `INFO_SPECIAL_CHARS_QUOTE` como `‘0’` (veja-se Seção 20.4.5 [Variáveis de Personalização do Info e de Texto Simples], Página 173).

`texi2any` alerta a respeito de tais usos problemáticos em nomes de nó, itens de menu e referências cruzadas. Se não quiser ver os alertas, você pode configurar a variável de personalização `INFO_SPECIAL_CHARS_WARNING` para `‘0’` (veja-se Seção 20.4.5 [Variáveis de Personalização do Info e de Texto Simples], Página 173).

Se insistir em usar tais caracteres em nomes de nó, para a finalidade de não confundir os processadores Texinfo, você precisa ainda encapsular tais caracteres, usando ou inserções especiais (veja-se Seção 11.1.3 [Inserindo uma Vírgula], Página 95) ou `@asis` (veja-se [`@asis`], Página 77). Por exemplo:

```
@node foo@asis{::}bar@comma{ } baz
```

Como um exemplo de se evitar os caracteres especiais, o seguinte é um título de seção neste manual:

```
@section @code{@@unnumbered}, @code{@@appendix}: Capítulos com...
```

Porém, correspondente nome de nó carece de vírgulas e do subtítulo:

```
@node @code{@@unnumbered @@appendix}
```

- Maiúsculas e minúsculas são significativas em nomes de nó.
- Os espaços antes e depois dos nomes na linha `@node` são ignorados. Caracteres espaços em branco múltiplos “dentro” de um nome são substituídos por um espaço único. Por exemplo:

```
@node foo bar
```

¹ Antes do lançamento do Texinfo 5 em 2013, esse recurso era suportado de uma maneira peculiar (a opção `--commands-in-node-names` para `makeinfo`). Agora ele é parte da linguagem.


```
@node foo bar,
@node foo bar ,
@node foo bar,
@node foo bar ,
```

todas definem o mesmo nó, nominadamente ‘foo bar’. Em entradas de menu, um espaço interno deveria ser usado em nomes de nó ou algumas versões de alguns leitores Info não encontrarão o nó.

3.4 O Primeiro Nó

O primeiro nó de um arquivo Texinfo é o nó *Top*, exceto em um arquivo incluído (veja-se Capítulo 18 [Arquivos de Inclusão], Página 148). O nó *Top* deveria conter um sumário curto e um menu mestre. Veja-se Seção 2.10 [O Nó Top], Página 22, para mais informações acerca do conteúdo e exemplos de nó *Top*. Texto reto antes do nó *Top* fora de qualquer nó deveria ser evitado. Tal texto, se presente, não é gerado para DocBook.

Aqui está uma descrição dos ponteiros de nó a serem usados no nó *Top*:

- O nó *Top* (o qual precisa ser nomeado ‘top’ ou ‘Top’) deveria ter como nó dele ‘Acima’ o nome de um nó em outro arquivo, onde existe um menu que leva a esse arquivo. Especifique o nome de arquivo entre parênteses.

Usualmente, todos os arquivos Info estão disponíveis por meio de uma árvore virtual única do Info, construída a partir de múltiplos diretórios. Nesse caso, use ‘(dir)’ como o ancestral do nó *Top*; isso especifica o nó de nível superior no arquivo *dir*, o qual contém o menu principal para o sistema Info como um todo. (Cada diretório com arquivos Info é destinado a conter um arquivo chamado *dir*).

Isso está bom para Info, porém para saída HTML, alguém bem pode querer que o linque Acima oriundo do nó *Top* vá para algum lugar específico. Por exemplo, para GNU, o lugar natural seria <https://www.gnu.org/manual/> (uma página da web que coleciona linques para a maioria dos manuais GNU), melhor especificada que apenas */manual/*, se o manual será instalado em www.gnu.org. Isso pode ser especificado com a variável de personalização *TOP_NODE_UP_URL* (veja-se Seção 22.8.2 [Personalização de Nomes de Arquivo e de Linques para HTML], Página 205), como em

```
$ texi2any --html -c TOP_NODE_UP_URL=/manual/ ...
```

- O nó ‘Anterior’ do nó *Top* é usualmente omitido.
- O nó ‘Próximo’ do nó *Top* deveria ser o primeiro capítulo em teu documento.

Veja-se Seção 21.1 [Instalando um Arquivo do Info], Página 183, para mais informações acerca de instalar um arquivo Info no diretório *info*.

Geralmente, é melhor deixar os ponteiros completamente desligados e deixar que as ferramentas os definam implicitamente, com este resultado simples:

```
@node Top
```

3.5 O Comando de Seccionamento @top

O comando *@top* é um comando especial de seccionamento que você deveria usar somente depois de uma linha ‘@node Top’ no início de um arquivo Texinfo.

Ele produz o mesmo tipo de saída que *@unnumbered* (veja-se Seção 4.4 [*@unnumbered @appendix*], Página 38). No L^AT_EX *\part** é usado.

@top é ignorado ao se elevar ou rebaixar seções. Ou seja, ele nunca é rebaixado e nada pode ser elevado a ele (veja-se Seção 4.12 [Elevar/rebaixar seções], Página 41).

Costumava ser convencional envolver o nó `@top` em um condicional `@ifnottex`, de forma que ele não aparecesse em saída impressa (veja-se Capítulo 16 [Condicionais], Página 129). Assim, um nó Top frequentemente se parecia com isto:

```
@ifnottex
@node Top
@top título-teu-manual

sumário-altíssimo-nível
@end ifnottex
```

Isso não mais é necessário, pois o nó ‘Top’ agora nunca é gerado para saída impressa. O nó ‘Top’ também não é gerado para DocBook.

3.6 Estrutura do Documento Texinfo

Os nós podem conter *menus*, os quais contêm os nomes de *nós descendentes* dentro do nó ancestral; por exemplo, um nó correspondente a um capítulo teria um menu de seções nesse capítulo. Os menus permitem que o(a) usuário(a) se movimente para os nós descendentes na saída Info.

Adicionalmente, os nós contêm *ponteiros de nó* que nomeiam outros nós. Os ponteiros ‘Próximo’ e ‘Anterior’ vinculam nós no mesmo nível de seccionamento em uma cadeia. Conforme você pode imaginar, o ponteiro ‘Próximo’ vincula ao próximo nó, e o ponteiro ‘Anterior’ vincula ao nó anterior. Em geral, ‘Próximo’ e ‘Anterior’ referenciam nós no *mesmo nível hierárquico* no manual, não necessariamente ao próximo nó dentro do arquivo Texinfo. No arquivo Texinfo, o nó subsequente possivelmente esteja em um nível inferior—um nó ao nível de seção frequentemente precisa seguir um nó ao nível de capítulo, por exemplo. Assim, por exemplo, todos os nós que estejam ao nível de seções dentro de um capítulo são vinculados juntos, e a ordem nessa cadeia é a mesma que a ordem dos descendentes no menu do capítulo ancestral. Cada nó descendente grava o nome de nó do ancestral como ponteiro dele ‘Acima’.

Como o nó ‘Top’ é o único nó nesse nível, ‘Próximo’ se refere ao primeiro nó a seguir, que quase sempre é um nó de capítulo ou ao nível de capítulo. Essa é uma exceção à regra de ‘Próximo’ estar no mesmo nível hierárquico.

A saída Info e HTML para cada nó inclui vínculos para os nós ‘Próximo’, ‘Anterior’, e ‘Acima’. O HTML também usa o atributo `accesskey` com os valores ‘n’, ‘p’, e ‘u’, respectivamente. Isso permite que pessoas que usam navegadores web sigam a navegação usando (tipicamente) *M-letra*, por exemplo, *M-n* para o nó ‘Próximo’, a partir de qualquer lugar dentro do nó. Os ponteiros e menus de nó fornecem estrutura para arquivos Info, exatamente como capítulos, seções, subseções e semelhantes fornecem estrutura para livros impressos. As duas estruturas teoricamente são distintas; na prática, entretanto, a estrutura de árvore dos livros impressos é também essencialmente sempre usada para o nó e estrutura de menu, pois isso conduz a um documento que é fácil de seguir.

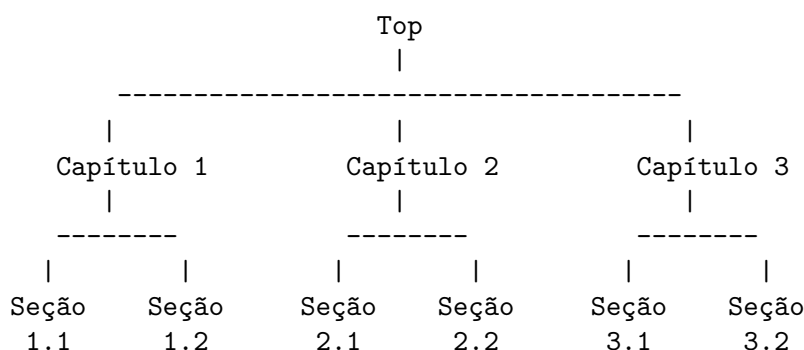
Normalmente, a estrutura de seccionamento e a estrutura de nó são completamente paralelas, com um nó para cada capítulo, seção etc., e com os nós seguindo o mesmo arranjo hierárquico que o seccionamento. Assim, se um nó estiver no nível lógico de um capítulo, os nós descendentes dele estarão no nível das seções; similarmente, os nós descendentes das seções estarão no nível das subseções.

É tecnicamente possível criar documentos do Texinfo com somente uma estrutura ou a outra, ou para as duas estruturas não serem paralelas ou para que a estrutura de seccionamento ou a de nó seja diferente da estrutura convencional. Até onde sabemos, no entanto, todos os manuais do Texinfo atualmente em uso geral seguem a estrutura paralela convencional.

3.7 Ilustração de Nó e Menu

Aqui está um diagrama que ilustra um arquivo texinfo com três capítulos, cada um dos quais contém duas seções.

A “raiz” está no topo do diagrama e as “folhas” estão na parte inferior. É assim que esse diagrama é desenhado convencionalmente; ele ilustra uma árvore de cabeça para baixo. Por esse motivo, o nó raiz é chamado de nó ‘Top’ e os ponteiros do nó ‘Acima’ levam você mais perto da raiz.



Usando-se ponteiros explícitos (não recomendados, mas mostrados para fins do exemplo), o comando totalmente escrito para iniciar o Capítulo 2 seria o este:

```
@node      Capítulo 2, Capítulo 3, Capítulo 1, Top
@comment   nome-nó, próximo, anterior, acima
```

Essa linha `@node` diz que o nome desse nó é “Capítulo 2”, o nome do nó ‘Próximo’ é “Capítulo 3”, o nome do nó ‘Anterior’ é “Capítulo 1” e o nome do nó ‘Acima’ é “Top”. Você pode (e deveria) omitir escrever esses nomes de nós se teu documento estiver hierarquicamente organizado, mas os relacionamentos do ponteiro ainda obtém.

Para ir para as seções 2.1 e 2.2 usando Info, você precisa de um menu dentro do Capítulo 2. (Veja-se Seção 3.9 [Menus], Página 33). Você escreveria o menu pouco antes do início da Seção 2.1, como este:

```
@menu
* Sect. 2.1:: Descrição desta seção.
* Sect. 2.2:: Descrição.
@end menu
```

Os ponteiros automáticos para o nó para Sect. 2.1 correspondem a:

```
@node      Sect. 2.1, Sect. 2.2, , Capítulo 2
@comment   nome-nó, próximo, anterior, acima
```

Observe que nenhum ponteiro ‘Anterior’ é gerado, pois não existe outro nó no mesmo nível hierárquico antes de Sect. 2.1.

Usando ponteiros explícitos, o nó para Sect. 2.1 poderia ser escrito assim:

```
@node      Sect. 2.1, Sect. 2.2, Capítulo 2, Capítulo 2
@comment   nome-nó, próximo, anterior, acima
```

Com ponteiros automáticos, os ponteiros ‘Próximo’ e ‘Anterior’ de um nó levam a outros nós no mesmo nível—de capítulo para capítulo ou de seção para seção. Como mostrado, ao usar-se ponteiros explícitos, os ponteiros também podem liderar em outro lugar, aqui, por exemplo, o ponteiro ‘Anterior’ aponta para cima. Um ponteiro ‘Acima’ geralmente leva a um nó no nível acima (mais próximo do nó ‘Top’; e um ‘Menu’ leva a nós em um nível abaixo (mais próximo das ‘folhas’). (Uma referência cruzada pode apontar para um nó em qualquer nível; veja-se Capítulo 5 [Referências Cruzadas], Página 43).

Tecnicamente, os ponteiros explícitos de nós podem te levar a qualquer nó, independentemente da estrutura do documento; mesmo para nós em um arquivo Info diferente. No entanto, seria

muito confuso para leitores ter ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ levando a nós que não correspondem, mesmo vagamente, ao nó próximo, anterior e acima.

Um comando `@node` e um comando de estruturamento de capítulo são usados convencionalmente juntos, nessa ordem, geralmente seguidos por comandos de indexação. (Como mostrado no exemplo acima, você pode seguir a linha `@node` com uma linha de comentários, por exemplo, para mostrar qual ponteiro é qual se ponteiros explícitos forem usados). Os processadores Texinfo usam esse construto para determinar as relações entre nós e comandos de seccionamento.

Aqui está o começo do capítulo neste manual chamado “Finalizando um Arquivo Texinfo”. Isso mostra uma linha `@node` seguida por uma linha `@chapter` e, em seguida, por linhas de indexação.

```
@node Finalizando um Arquivo
@chapter Finalizando um Arquivo Texinfo
@cindex Finalizando um arquivo Texinfo
@cindex Finalização de arquivo Texinfo
@cindex Finalização de arquivo
```

3.8 Descrições de Nó

Você pode fornecer uma breve descrição do propósito de um nó usando o comando `@nodedescription` seguindo a linha `@node`. Essa descrição pode elaborar ou estender as informações no próprio nome do nó.

Você também pode usar um ambiente `@nodedescriptionblock` para fornecer uma descrição do nó. Isso possivelmente seja útil para descrições mais longas.

`texi2any` usa o conteúdo que você fornecer com esses comandos ao gerar menus para o formato de saída Info (e, opcionalmente, para HTML). `texi2any` usa a descrição depois de uma entrada de menu para o nó se estiver gerando o menu automaticamente ou se nenhuma descrição para a entrada de menu foi fornecida em um bloco explícito `@menu`. (Veja-se Seção 3.9 [Menus], Página 33).

Aqui está um exemplo do uso desses comandos:

```
@node Ferramentas
@chapter Ferramentas
```

Este capítulo é a respeito de diferentes ferramentas que você pode usar.

```
@node Chaves de fenda
@nodedescription Ponta Chata e Phillips.
@section Chaves de fenda
```

Esta seção é a respeito de chaves de fenda.

```
@node Brocas
@nodedescriptionblock
Fazendo buracos em coisas com chaves elétricas de fenda, controladores de
broca, brocas combi, controladores de impacto, brocas martelo, disjuntores
e brocas de demolição.
@end nodedescriptionblock
@section Brocas
```

Esta seção é a respeito de brocas.

Na saída gerada Info, `texi2any` geraria o nó ‘Ferramentas’ com um menu conforme segue:

```
* Menu:
```

```
* Chaves de fenda::      Ponta Chata e Phillips.
* Brocas::                Fazendo buracos em coisas com chaves
                           elétricas de fenda, controladores de broca,
                           brocas combi, controladores de impacto, brocas
                           martelo, disjuntores e brocas de demolição.
```

3.9 Menus

Menus contém ponteiros para subordinar nós. Na saída gerada Info, você usa menus para ir para tais nós. `texi2any` consegue gerar menus na saída gerada HTML, porém não faz isso por padrão (veja-se Seção 22.8.1 [Personalização da Estrutura da Saída Gerada HTML], Página 204, `FORMAT_MENU`). Os menus não têm efeito em manuais impressos ou em outros formatos de saída.

Os menus são gerados automaticamente pelo `texi2any` ao gerar Info ou HTML com menus para nós seguidos por um comando de seccionamento, sem um bloco explícito `@menu`, e com ponteiros automáticos.

Frequentemente é mais conveniente permitir que `texi2any` gere menus para você, pois você então não tem o ônus de atualizar blocos de menus no teu fonte do Texinfo quando adicionar, remover ou realocar nós. No caso usual de um manual organizado hierarquicamente com comandos de seccionamento associados a nós e com ponteiros de nós deixados de fora, você deveria escrever menus somente se quiser controle exato sobre o conteúdo e a formatação de menus no Info.

3.9.1 Escrevendo um Menu

Um menu consiste de um comando `@menu` em uma linha própria, seguido por linhas de entrada de menu ou de linhas de comentários de menu, e então seguido por um comando `@end menu` em uma linha própria.

Um menu se parece com isto:

```
@menu
Unidades Maiores de Texto

* Arquivos::                Tudo acerca de manuseio de arquivos.
* Múltiplos: Buffers.      Múltiplos buffers; editando
                           vários arquivos de uma vez.

@end menu
```

Em um menu, cada linha que inicie com um ‘*’ é uma *entrada de menu*. (Observe o espaço depois do asterisco). No exemplo acima, as duas linhas iniciando com ‘*’ são entradas de menu.

Uma linha que não se inicie com um ‘*’ também pode aparecer em um menu. Tal linha não é uma entrada de menu, mas sim uma linha do *comentário de menu*. No exemplo acima, a linha ‘Unidades Maiores de Texto’ é uma linha de comentário de menu. Linhas de comentário de menu e linhas vazias são mantidas na saída gerada.

Tecnicamente, menus podem te transportar para qualquer nó, independentemente da estrutura do documento; mesmo para nós em um arquivo Info diferente. Entretanto, o recurso de criação implícita de ponteiros do `texi2any` (veja-se Seção 3.1 [Escrevendo um Nó], Página 26) e os comandos de atualização do modo Texinfo do GNU Emacs funcionam somente para criar menus de nós subordinados em um documento hierarquicamente estruturado. Em um documento hierarquicamente estruturado, é muito melhor usar referências cruzadas para referenciar nós arbitrários.

No Info, um(a) usuário(a) seleciona um nó com o comando `m` (**Info-menu**). O nome de entrada de menu é o que o(a) usuário(a) digita depois do comando `m`. Na saída HTML, o atributo `accesskey` é usado com os valores ‘1’...‘9’ para as primeiras nove entradas. Isso permite que

as pessoas usando navegadores web sigam as primeiras entradas de menu usando (tipicamente) *M-dígito*, por exemplo, *M-1* para a primeira entrada.

3.9.2 Um Exemplo de Menu

Um menu se parece com isto no Texinfo:

```
@menu
* nome de entrada de menu: Nome de nó. Uma descrição curta.
* Nome de nó::                      Essa forma é preferida.
@end menu
```

Isso produz:

```
* Menu:

* nome de entrada de menu: Nome de nó. Uma descrição curta.
* Nome de nó::                      Essa forma é preferida.
```

Aqui está um exemplo como você pode vê-lo em um arquivo Texinfo:

```
@menu
Unidades Maiores de Texto

* Arquivos::                      Tudo acerca de manuseio de arquivos.
* Múltiplos: Buffers.             Múltiplos buffers; editando
                                  vários arquivos de uma vez.

@end menu
```

Isso produz:

```
* Menu:
Unidades Maiores de Texto

* Arquivos::                      Tudo acerca do manuseio de arquivos.
* Múltiplos: Buffers.             Múltiplos buffers; editando
                                  vários arquivos de uma vez.
```

Nesse exemplo, o menu tem duas entradas. ‘Arquivos’ tanto é um nome de entrada de menu quanto o nome do nó referenciado por esse nome. ‘Múltiplos’ é o nome de entrada de menu; ele se refere ao nó chamado ‘Buffers’. A linha ‘Unidades Maiores de Texto’ é um comentário; ela aparece no menu, porém não é uma entrada.

Como nenhum nome de arquivo está especificado nem com ‘Arquivos’ nem com ‘Buffers’, eles precisam ser os nomes de nós no mesmo arquivo Info (veja-se Seção 3.9.6 [Referenciando Outros Arquivos do Info], Página 35).

3.9.3 Local de Menu

Pode existir ao menos um menu em um nó. Um menu é convencionalmente localizado ao final de um nó, sem qualquer texto regular ou comandos adicionais entre o `@end menu` e o início do próximo nó.

Essa convenção é útil, dado que um(a) leitor(a) que use o menu poderia facilmente se esquecer de quaisquer desses textos. Também, quaisquer desses textos pós menu serão considerados parte do menu na saída Info (a qual não tem marcador para o final de um menu). Assim, uma linha iniciando com ‘*’ será igualmente incorretamente manuseada.

Geralmente é melhor se um nó com um menu não contiver muito texto. Se você se encontrar com um monte de texto antes de um menu, geralmente nós recomendamos mover tudo, exceto alguns parágrafos, para um novo sub nó. Do contrário, é fácil para os(as) leitores(as) perderem o menu.

3.9.4 As Partes de um Menu

Uma entrada de menu tem três partes, somente a segunda das quais é exigida:

1. O nome de entrada de menu (opcional).
2. O nome do nó (exigido).
3. Uma descrição do item (opcional).

O modelo para uma entrada genérica de menu se parece com isto (porém, veja-se a próxima seção para mais uma possibilidade):

```
* nome_de_entrada_de_menu: nome_de_nó.    descrição
```

Siga o nome de entrada de menu com dois pontos, e siga o nome de nó com tabulação, vírgula, nova linha ou os dois caracteres ponto e espaço (‘. ’).

A terceira parte de uma entrada de menu é uma frase descritiva ou sentença. Os nomes de entrada de menu e os nomes de nó frequentemente são curtos; a descrição explica para o(a) leitor(a) a respeito de que se trata o nó. Uma descrição útil complementa o nome de nó em vez de repeti-lo. A descrição, que é opcional, pode ser espalhada por várias linhas; se isso acontecer, alguns(mas) autores(as) preferem recuar a segunda linha enquanto outros(as) preferem alinhá-la com a primeira (e todas as outras). É completamente com você. Uma linha vazia, ou a próxima entrada de menu, finaliza a descrição.

Os caracteres de espaço em um menu são preservados como estão na saída Info; isso te permite formatar o menu como você desejar. Infelizmente, você precisa digitar os nomes de nó sem quaisquer espaços extras ou algumas versões de alguns leitores Info não encontrarão o nó (veja-se Seção 3.3 [Exigências da Linha do Nó], Página 28).

`texi2any` alerta e protege nomes quando o texto de um item de menu (e nomes e referências cruzadas de nó) contiver uma construção problemática que puder interferir com a análise dele no Info. (Veja-se [Restrições de Nomes de Nó do Info], Página 28).

3.9.5 Entrada de Menu Menos Desordenada

Quando o nome de entrada de menu e o nome de nó são os mesmos, você pode escrever o nome imediatamente depois do asterisco e espaço no início da linha e seguir com o nome com dois “dois pontos”.

Por exemplo, escreva

```
* Nome::                                descrição
```

em vez de

```
* Nome: Nome.                          descrição
```

Nós recomendamos usar o nome de nó para o nome de entrada de menu sempre que possível, pois isso reduz a confusão visual no menu.

3.9.6 Referenciando Outros Arquivos do Info

Você pode criar uma entrada de menu que habilite um leitor no Info a ir para um nó em outro arquivo Info, escrevendo o nome de arquivo entre parênteses um pouco antes do nome do nó. Alguns exemplos:

```
@menu
* nome-primeira-entrada:(nome-arquivo)nome-nó.    descrição
* (nome-arquivo)segundo-nó::                        descrição
@end menu
```

Por exemplo, para referenciar diretamente os nós ‘Outlining’ e ‘Rebinding’ no *Manual do Emacs*, você poderia escrever um menu como este:

```

@menu
* Outlining: Modo Contorno(emacs). O principal modo para
                                se editar contornos.
* (emacs)Rebinding::           Como redefinir o
                                significado de uma tecla.

@end menu

```

Se você não listar o nome de nó, mas somente nomear o arquivo, então Info presume que você está se referindo ao nó ‘Top’. Exemplos:

```

* Info: (info).                Sistema de navegação de documentação.
* (emacs)::                    O editor de texto, extensível,
                                auto-documentador.

```

Os comandos de atualização de menu do modo Texinfo do GNU Emacs somente funcionam com nós dentro do buffer atual, de forma que você não consegue usá-los para criar menus que referenciem outros arquivos. Você precisa escrever tais menus manualmente. Veja-se Seção D.5 [Atualizando Nós e Menus], Página 256.

4 Estruturamento de Capítulo

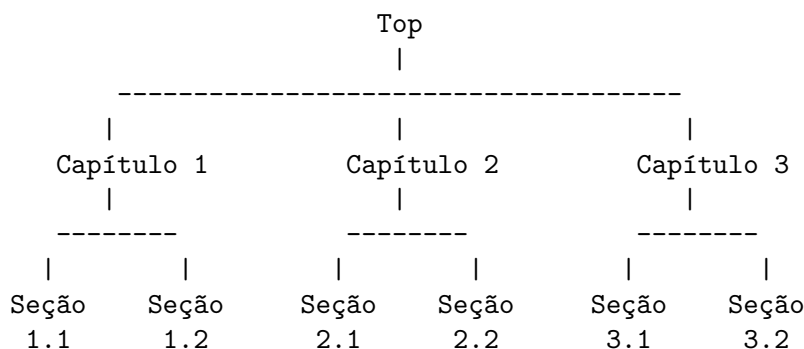
Os comandos de *estruturamento de capítulo* do Texinfo dividem um documento em uma hierarquia de capítulos, seções, subseções e subsubseções. Esses comandos geram cabeçalhos grandes no texto, como o acima. Eles também fornecem informações para gerar o sumário (veja-se Seção 2.9 [Gerando Um Sumário], Página 21).

Normalmente você coloca um comando `@node` imediatamente antes de cada comando de estruturamento de capítulo. Veja-se Capítulo 3 [Nós], Página 26.

4.1 Estrutura de Árvore das Seções

Um arquivo Texinfo geralmente é estruturado como um livro com capítulos, seções, subseções, e afins. Essa estrutura pode ser visualizada como uma árvore (ou melhor, como uma árvore de cabeça para baixo) com a raiz no topo e os níveis correspondendo aos capítulos, seções, subseções e subsubseções.

Aqui está um diagrama que mostra um arquivo Texinfo com três capítulos, cada qual com duas seções.



Em um arquivo Texinfo que tenha essa estrutura, o início do Capítulo 2 poderia ser escrito assim:

```
@node    Capítulo 2
@chapter Capítulo 2
```

Para propósitos de exemplo, aqui está como poderia ser escrito com ponteiros explícitos de nó:

```
@node    Capítulo 2, Capítulo 3, Capítulo 1, Top
@chapter Capítulo 2
```

Os comandos de estruturamento de capítulo estão descritos nas seções que seguem; o comando `@node` está descrito no capítulo anterior (veja-se Capítulo 3 [Nós], Página 26).

4.2 Tipos de Comandos Estruturantes

Os comandos de estruturamento de capítulo se dividem em quatro grupos, cada um dos quais contendo comandos de estruturamento nos níveis de capítulos, seções, subseções e subsubseções:

- Os comandos tipo `@chapter` e comandos tipo `@appendix` produzem entradas numeradas ou letradas, ambas no corpo de um documento e no sumário dele.
- Os comandos tipo `@unnumbered` produzem entradas não numeradas, ambas no corpo de um documento e no sumário dele. O comando `@top`, o qual tem um uso especial, é um membro desse grupo (veja-se Seção 3.5 [Comando `@top`], Página 29). Uma seção `@unnumbered` é uma parte normal da estrutura do documento.
- Os comandos tipo `@heading` produzem cabeçalhos não numerados simples que não aparecem em um sumário, não são associados com nós e não podem ser referenciados cruzadamente. Esses comandos de cabeçalhos nunca iniciam uma nova página.

Na saída gerada impressa, os comandos de estruturamento de capítulo produzem cabeçalhos no documento. Quando um comando `@setchapternewpage` diz para fazer isso, os comandos `@chapter`, `@unnumbered` e `@appendix` iniciam novas páginas no manual impresso; os comandos `@heading` não. Veja-se Seção E.1 [`@setchapternewpage`], Página 274.

Na saída Info e de texto simples, o comando faz com que o título apareça em uma linha por si só, com uma linha de um caractere ASCII (*, =, ...) inserido por baixo. Por exemplo, o cabeçalhos de “Estruturamento de Capítulo” poderia ser:

```
5 Estruturamento de Capítulo
*****
```

O caractere sublinhado é o mesmo para todos os comandos no mesmo nível. Por exemplo, é o mesmo para os comandos de nível de capítulo `@chapter`, `@appendix`, `@unnumbered` e `@chapheading`.

No HTML, os comandos de nível de capítulo produzem um cabeçalho de nível `<h2>` por padrão (controlado pela variável de personalização `CHAPTER_HEADER_LEVEL`, veja-se Seção 22.8.6 [Personalização da Saída Gerada Específica do HTML], Página 210). O nível do elemento de cabeçalho é ajustado para os outros comandos.

Na saída gerada DocBook, o nível apropriado do elemento é usado. O elemento produzido inclui todas as seções seguintes até o próximo comando no mesmo nível ou superior. Por exemplo, um elemento `<chapter>` é produzido para `@chapter`, e contém quaisquer seções ou subseções no capítulo.

Aqui está um sumário:

<i>Numerada</i>	<i>Não numerada</i>	<i>Letrada/numerada</i>	<i>Sem nova página</i>
No conteúdo	No conteúdo	No conteúdo	<i>Não numerada</i>
	<code>@top</code>		Não no conteúdo
<code>@chapter</code>	<code>@unnumbered</code>	<code>@appendix</code>	<code>@majorheading</code>
<code>@section</code>	<code>@unnumberedsec</code>	<code>@appendixsec</code>	<code>@chapheading</code>
<code>@subsection</code>	<code>@unnumberedsubsec</code>	<code>@appendixsubsec</code>	<code>@heading</code>
<code>@subsubsection</code>	<code>@unnumberedsubsubsec</code>	<code>@appendixsubsubsec</code>	<code>@subheading</code>
			<code>@subsubheading</code>

4.3 @chapter: Estruturamento de Capítulo

`@chapter` identifica um capítulo no documento—o nível mais alto da hierarquia de estruturamento normal do documento. Escreva o comando no início de uma linha e o siga na mesma linha pelo título do capítulo. O capítulo é numerado automaticamente, iniciando a partir de 1.

Por exemplo, o capítulo presente neste manual é intitulado “Estruturamento de Capítulo”; a linha `@chapter` se parece com isto:

```
@chapter Estruturamento de Capítulo
```

4.4 @unnumbered, @appendix: Capítulos com Outra Rotulagem

Use o comando `@unnumbered` para iniciar um elemento de nível de capítulo que aparece sem números de capítulos de qualquer tipo. Use o comando `@appendix` para iniciar um apêndice que é rotulado por letras (‘A’, ‘B’, ...) em vez de números; apêndices também estão ao nível de estruturamento do capítulo.

Escreva um comando `@appendix` ou `@unnumbered` no início de uma linha e siga-o, na mesma linha, pelo título, exatamente como com `@chapter`.

Texinfo também fornece um comando `@centerchap`, o qual é análogo a `@unnumbered`, porém centraliza o argumento dele nas saídas impressa e HTML. Esse tipo de escolha estilística geralmente não é oferecida pelo Texinfo. Você está recomendado(a) a não usar esse comando, pois ele possivelmente seja removido em futuros lançamentos do Texinfo.

Com `@unnumbered`, se o nome do nó associado for uma destas palavras do Inglês (não diferencia maiúsculas de minúsculas):

`Acknowledgements` `Colophon` `Dedication` `Preface`

então a saída DocBook usará etiquetas especiais correspondentes (`<preface>`, entre outras) em vez da padrão `<chapter>`. O argumento para o próprio `@unnumbered` pode ser qualquer coisa, e é gerado como o elemento título como de costume.

4.5 `@majorheading`, `@chapheading`: Cabeçalhos de Nível de Capítulo

Os comandos `@majorheading` e `@chapheading` produzem cabeçalhos tipo capítulo no corpo de um documento.

Entretanto, nenhum comando produz uma entrada no sumário, e nenhum comando faz com que $\text{\TeX}$ inicie uma nova página em um manual impresso.

No $\text{\TeX}$, um comando `@majorheading` gera um espaço em branco vertical mais largo, antes do cabeçalho, que um comando `@chapheading`, porém, do contrário, é o mesmo.

Em outros formatos de saída, os comandos `@majorheading` e `@chapheading` produzem uma saída semelhante a `@chapter`. A diferença é a falta de numeração e a falta de qualquer associação com nós. Veja-se Seção 4.3 [`@chapter`], Página 38.

4.6 `@section`: Seções Abaixo de Capítulos

Um comando `@section` identifica uma seção dentro de uma unidade de capítulo, seja ela criada com `@chapter`, `@unnumbered`, ou `@appendix`, seguindo o esquema de numeração do comando de nível de capítulo. Assim, dentro de um capítulo `@chapter` numerado ‘1’, as seções são numeradas ‘1.1’, ‘1.2’, etc.; dentro de um “capítulo” `@appendix` rotulado ‘A’, as seções são numeradas ‘A.1’, ‘A.2’, etc.; dentro de um capítulo `@unnumbered`, a seção não é numerada.

Para criar uma seção, escreva o comando `@section` no início de uma linha e siga-o, na mesma linha, pelo título da seção. Por exemplo:

```
@section Isto é uma seção
```

Títulos de seção são listados no sumário.

4.7 `@unnumberedsec`, `@appendixsec`, `@heading`

Os comandos `@unnumberedsec`, `@appendixsec` e `@heading` são, respectivamente, os equivalentes não numerado, tipo apêndice e tipo cabeçalho do comando `@section` (veja-se a seção anterior).

`@unnumberedsec` e `@appendixsec` não precisam ser usados em circunstâncias normais, pois `@section` também pode ser usado dentro de capítulos `@unnumbered` e `@appendix`; novamente, veja-se a seção anterior.

`@unnumberedsec`

O comando `@unnumberedsec` pode ser usado dentro de um capítulo não numerado ou dentro de um capítulo ou apêndice normal para produzir uma seção não numerada.

`@appendixsec`

`@appendixsection`

`@appendixsection` é uma escrita mais longa do comando `@appendixsec`; os dois são sinônimos.

Convencionalmente, o comando `@appendixsec` ou `@appendixsection` é usado somente dentro de apêndices.

@heading Você pode usar o comando **@heading** (quase) em qualquer lugar para um cabeçalho estilo seção que não aparecerá no sumário. Os comandos da série **@heading** podem aparecer dentro da maioria dos ambientes, por exemplo, embora locais patológicos e inúteis como um argumento para outro comando, etc., não sejam permitidos.

4.8 @subsection: Subseções Abaixo de Seções

Subseções estão para seções como seções estão para capítulos; veja-se Seção 4.6 [**@section**], Página 39. Por exemplo:

@subsection Esta é uma subseção

Títulos de subseções são listados no sumário.

4.9 Os Comandos do Tipo @subsection

Os comandos **@unnumberedsubsec**, **@appendixsubsec** e **@subheading** são, respectivamente, os equivalentes não numerados, tipo apêndice e tipo cabeçalho do comando **@subsection**. (Veja-se Seção 4.8 [**@subsection**], Página 40).

@unnumberedsubsec e **@appendixsubsec** não precisam ser usados em circunstâncias normais, pois **@subsection** também pode ser usado dentro de seções de capítulos **@unnumbered** e **@appendix** (veja-se Seção 4.6 [**@section**], Página 39).

Um comando **@subheading** produz um cabeçalho semelhante àquele de uma subseção, exceto que ele não é numerado e não aparece no sumário. Similarmente, um comando **@unnumberedsubsec** produz um cabeçalho não numerado semelhante àquele de uma subseção e um comando **@appendixsubsec** produz um cabeçalho tipo subseção rotulado com uma letra e números; ambos esses comandos produzem cabeçalhos que aparecem no sumário.

4.10 @subsubsection e Outros Comandos Subsub

O quarto e mais baixo nível de comandos de seccionamento no Texinfo são os comandos ‘subsub’. Eles são:

@subsubsection

Subsubseções estão para subseções como subseções estão para seções. (Veja-se Seção 4.8 [**@subsection**], Página 40). Títulos de subsubseções aparecem no sumário.

@unnumberedsubsubsec

Títulos de subsubseções não numeradas aparecem no sumário, porém carecem de números. Do contrário, subsubseções não numeradas são o mesmo que subsubseções.

@appendixsubsubsec

Convencionalmente, os comandos de apêndice são usados somente para apêndices e são letrados e numerados apropriadamente. Eles também aparecem no sumário.

@subsubheading

O comando **@subsubheading** pode ser usado em qualquer lugar que você quiser um pequeno cabeçalho que não aparecerá no sumário.

Como com subseções, **@unnumberedsubsubsec** e **@appendixsubsubsec** não precisam ser usados em circunstâncias normais, pois **@subsubsection** também pode ser usado dentro de subseções de capítulos **@unnumbered** e **@appendix** (veja-se Seção 4.6 [**@section**], Página 39).

4.11 @part: Grupos de Capítulos

O comando final de seccionamento é o **@part**, para marcar uma *parte* de um manual, isto é, um grupo de capítulos ou (raramente) apêndices. Isso se comporta bem diferentemente de

outros comandos de seccionamento, para se encaixar com a maneira com que tais “partes” convencionalmente são usadas em livros.

Nenhum comando `@node` é associado com `@part`. Apenas escreva o comando em uma linha própria, incluindo o título da parte, no lugar no documento que você deseja marcar como o início daquela parte. Por exemplo:

```
@part Parte I:@* 0 Início
```

Como pode ser inferido a partir desse exemplo, nenhuma numeração ou rotulamento automático do texto do `@part` é feito. O texto é tomado como está.

Por causa das partes não serem associadas com nós, nenhum texto geral pode seguir a linha `@part`. Para produzir a saída pretendida, ela precisa ser seguida por um comando de nível de capítulo (incluindo o nó dela). Assim, para continuar o exemplo:

```
@part Parte I:@* 0 Início
```

```
@node Introdução
@chapter Introdução
...
```

Na saída gerada do `TeX`, o texto do `@part` é incluído em ambos os sumários, o normal e o curto, (veja-se Seção 2.9 [Conteúdo], Página 21), sem um número de página (dado que é a convenção normal). Adicionalmente, uma “página de parte” é gerada no corpo do documento, com apenas o texto do `@part`. No exemplo acima, o `@*` causa uma quebra de linha na página de parte (porém, é substituída por um espaço no sumário). Essa página de parte sempre é forçada a estar em uma página ímpar (a direita), independentemente da paginação do capítulo (veja-se Seção E.1 [`@setchapternewpage`], Página 274). Na saída gerada do `LATEX`, o `@part` é gerado como `\part`.

Na saída gerada de `HTML`, o texto do `@part` similarmente é incluído no sumário, e um cabeçalho é incluído no texto principal do documento, como parte do nó seguinte do capítulo ou apêndice.

Na saída gerada `DocBook`, o elemento `<part>` inclui todos os capítulos seguintes, até o próximo `<part>`. Um `<part>` contendo capítulos também é fechado em um apêndice.

Na saída gerada de `Info` e de texto simples, `@part` não tem efeito.

`@part` é ignorado quando do levantamento ou do rebaixamento de seções (veja-se a próxima seção). Isto é, ele nunca é rebaixado e nada pode ser levantado para ele.

4.12 Elevar/rebaixar Seções: `@raisesections` e `@lowersections`

Os comandos `@raisesections` e `@lowersections` implicitamente elevam e rebaixam o nível hierárquico dos seguintes capítulos, seções e dos outros comandos de seccionamento (excluindo as partes).

Isto é, o comando `@raisesections` muda seções para capítulos, subseções para seções, e assim por diante. Por outro lado, o comando `@lowersections` muda capítulos para seções, seções para subseções, e assim por diante. Assim, um comando `@lowersections` cancela um comando `@raisesections`, e vice versa.

Por uma questão prática, geralmente você quer somente elevar ou rebaixar grandes pedaços, normalmente em arquivos externos. Você consegue usar `@lowersections` para incluir texto escrito, como um arquivo do `Texinfo` externo ou independente, em outro arquivo do `Texinfo`, como um arquivo interno e incluído (veja-se Capítulo 18 [Arquivos de Inclusão], Página 148). O uso típico se parece com o isto:

```
@lowersections
@include algum_arquivo.texi
```

@raisections

(Sem o **@raisections**, todas as seções subsequentes no arquivo principal também poderiam ser rebaixadas).

Se o arquivo de inclusão sendo rebaixado tiver um nó **@top**, você precisará condicionar a inclusão dele com um sinalizador (veja-se Seção 16.5.1 [**@set @value**], Página 133).

Quaisquer menus no resultado final tem que levar em conta a elevação e o rebaixamento, de forma que intercalar arbitrariamente comandos **@raisections** e **@lowersections** por todo o documento muito provavelmente levará a erros (a menos que os menus no teu documento sejam todos automaticamente gerados).

O uso repetido dos comandos continua a elevar ou a rebaixar o nível hierárquico um passo de cada vez. Uma tentativa de elevar ‘capítulo’ acima reproduz comandos de capítulo; uma tentativa de rebaixar ‘subsubseção’ abaixo reproduz comandos subsubseção. Também, subsubseções rebaixadas e capítulos elevados não funcionarão com o recurso do **texi2any** de implicitamente determinar ponteiros de nó, dado que a estrutura de menu não pode ser representada corretamente.

Escreva cada comando **@raisections** e **@lowersections** em uma linha própria.

4.13 Nós Desestruturados

Todos os exemplos de nós neste capítulo até agora usaram uma linha **@node** seguida por um comando de estruturação (como **@chapter**).

No entanto, você também pode criar nós sem usar um comando de estruturação. Você pode achar isso útil se não quiser que teu manual (ou partes de um manual) tenha uma estrutura hierárquica, com capítulos superiores às seções, seções superiores às subseções e assim por diante.

Para fazer isso, siga a linha **@node** com um dos comandos de cabeçalho. Por exemplo:

```
@node Tópico
@heading Tudo acerca do Tópico
...
```

O argumento para o comando de cabeçalho será impresso normalmente. Quando o sinalizador **xrefautomaticsectiontitle** estiver ativado (veja-se [**xrefautomaticsectiontitle**], Página 46), esse argumento também será usado como o texto para quaisquer referências cruzadas para o nó.

Esses nós não são parte de uma estrutura hierárquica de documento. Isso significa que eles não estão listados em nenhum sumário e os ponteiros automáticos de direção do nó (**‘Próximo’**, **‘Anterior’** ou **‘Acima’**; veja-se Seção 3.1 [Escrevendo um Nó], Página 26) não lincam para ou a partir desses nós.

Se desejar criar um nó não estruturado *sem* imprimir um cabeçalho no início, você pode usar o comando **@xrefname** no lugar do comando de cabeçalho. Por exemplo:

```
@node Tópico
@xrefname Tudo acerca do Tópico
...
```

O comando **@xrefname** fornece o texto para referências cruzadas para o nó quando o sinalizador **xrefautomaticsectiontitle** está ativado, de modo que **‘@xref{Tópico}’** produziria uma saída como “Veja-se ‘Tudo acerca do Tópico’”.

5 Referências cruzadas

Referências cruzadas são usadas para guiar o(a) leitor(a) para outras partes do mesmo ou de diferentes arquivo(s) Texinfo.

Use referências cruzadas para fornecer acesso a informações que sejam detalhadas demais para o contexto atual, ou incidental. Um sistema de ajuda online ou um manual de referência não é como um romance; poucos(as) leem tais documentos em sequência a partir do início até o final. Em vez disso, as pessoas buscam o que precisam. Por essa razão, tais criações deveriam conter muitas referências cruzadas para ajudar os(as) leitores(as) a achar outras informações que eles(as) podem não ter lido.

Em um manual impresso, uma referência cruzada resulta em uma referência de página, a menos que ela seja completamente para outro manual, caso no qual a referência cruzada nomeia esse manual. No Info, uma referência cruzada resulta em uma entrada que você pode seguir usando o comando ‘f’ do Info. (Veja-se Seção “Seguindo referências cruzadas” em *Info*). Em HTML, uma referência cruzada resulta em um hiperlink. No DocBook, o elemento `<link>` é usado para referências cruzadas, a menos que ele seja para outro manual, caso no qual a referência cruzada nomeia esse manual.

Os vários comandos de referências cruzadas usam nós, âncoras (veja-se Seção 5.11 [`@anchor`], Página 49) ou rótulos de flutuação (veja-se Seção 9.1.1 [`@float`], Página 81) para definir locais de referências cruzadas. Quando o TeX gera um arquivo DVI, ele grava cada número de página do local da referência cruzada e usa os números de página na produção das referências. Assim, mesmo se você estiver escrevendo um manual que será somente impresso, e não usado online, você precisa, ainda assim, escrever linhas `@node` (ou âncoras `@anchor`) para a finalidade de nomear os lugares para os quais você faz referências cruzadas.

5.1 Diferentes Comandos de Referência Cruzada

Existem três comandos de referência cruzada:

- `@xref` Usado para iniciar uma frase com uma referência cruzada do Info dizendo ‘***Observe nome: nó.**’ ou com ‘Veja-se ...’ em outros formatos de saída.
- `@ref` Usado dentro de, ou mais frequentemente, ao final de uma frase; produz uma referência cruzada do Info dizendo ‘***observação nome: nó.**’, e apenas a referência em outros formatos de saída, sem o precedente ‘Veja-se’.
- `@pxref` Usado dentro de parênteses, ao final de uma frase, ou, do contrário, antes de pontuação, para fazer uma referência. A saída gerada dele inicia com um ‘***observação**’ em letras minúsculas no Info, e com um ‘veja-se’ em outros formatos de saída. (‘p’ é para ‘parênteses’).

Adicionalmente, existem comandos para produzir referências para documentos fora do sistema Texinfo. O comando `@cite` é usado para fazer referências para livros e manuais. `@url` produz um URL, por exemplo uma referência para uma página na Rede Mundial de Computadores.

5.2 Partes de Uma Referência Cruzada

Um comando de referência cruzada exige somente um argumento, que é o nome do nó para o qual ele se refere. Um comando de referência cruzada pode conter até quatro argumentos adicionais. O modelo para uma referência cruzada completa de cinco argumentos se parece com isto:

```
@xref{nome-nó, rótulo-online, rótulo-impresso,
nome-manual, título-manual-impresso}
```

Os cinco possíveis argumentos para uma referência cruzada são:

1. O nome do nó ou da âncora. Isso é o local para onde a referência cruzada leva você. Em um documento impresso, o local do nó fornece a referência de página somente para referências dentro do mesmo documento. Use `@node` para definir o nó (veja-se Seção 3.1 [Escrevendo um Nó], Página 26), `@anchor` (veja-se Seção 5.11 [`@anchor`], Página 49), ou `@float` (veja-se Seção 9.1.1 [`@float`], Página 81) com um rótulo. Esse argumento é exigido (exceto para referência para manuais inteiros).
- Escreva um nome de nó em uma referência cruzada exatamente da mesma maneira que na linha `@node`, incluindo a mesma capitalização; caso contrário, os processadores possivelmente não encontrem a referência.
2. Um rótulo para saída online. Geralmente ele é omitido; então a descrição do tópico (terceiro argumento) será usada se for especificada; Se isso também foi omitido, o nome do nó será usado.
3. Um rótulo para saída impressa. Frequentemente, esse é o título ou tópico da seção. Isso é usado como o nome da referência no manual impresso. Se omitido, o nome do nó será usado.
4. O nome do manual para o qual a referência se refere, se estiver fora do manual atual, em um arquivo Texinfo diferente.
5. O título do manual impresso para o qual a referência se refere, a partir de um arquivo Texinfo diferente.

O espaço em branco antes e depois das vírgulas separando esses argumentos é ignorado. Para incluir uma vírgula em um dos argumentos, use `@comma{}` (veja-se Seção 11.1.3 [Inserindo uma Vírgula], Página 95).

Referências cruzadas com um, dois, três, quatro e cinco argumentos estão descritas separadamente em seções seguintes.

Quando do processamento com TeX, uma vírgula automaticamente é inserida depois do número da página para referências cruzadas para dentro do mesmo manual, a menos que a chave de fechamento do argumento não seja seguida por espaços em branco (como uma vírgula ou um ponto). Isso te dá a escolha de quando ter uma vírgula lá em outros formatos de saída. Por exemplo,

```
@xref{Outra Seção} para mais informações
```

com TeX produz ‘Veja-se Outra Seção, página *ppp*, para mais informações’, e na saída Info produz ‘*Observe Outra Seção:: para mais informações’.

Se uma vírgula indesejada for adicionada, siga o argumento com um comando como ‘@:’. Por exemplo, ‘`@xref{Furacões}@: --- para os detalhes`’ produz

Veja-se Furacões, página *ppp* — para os detalhes
em vez de ‘Veja-se Furacões, página *ppp*, — para os detalhes’.

`texi2any` alerta e protege nomes quando o texto de uma referência cruzada (e nomes de nó e itens de menu) contiver uma construção problemática que poderia interferir com a análise dele no Info. Veja-se [Restrições de Nomes de Nó do Info], Página 28.

5.3 `@xref` com Um Argumento

A forma mais simples do `@xref` pega um argumento, o nome de outro nó no mesmo arquivo Texinfo.

Por exemplo,

```
@xref{Tempestades Tropicais}.
```

produz

```
*Observe Tempestades Tropicais::.
```

no Info e

Veja-se Seção 3.1 [Tempestades Tropicais], página 24.

em um manual impresso.

5.4 @xref com Dois Argumentos

Com dois argumentos, o segundo é usado como um rótulo para a saída gerada online.

O modelo é semelhante a isto:

```
@xref{nome-nó, rótulo-online}.
```

Por exemplo,

```
@xref{Efeitos Elétricos, Relâmpago}.
```

produz:

```
*Observe Relâmpago: Efeitos Elétricos.
```

no Info e

Veja-se Seção 5.2 [Efeitos Elétricos], página 57.

em um manual impresso, onde o nome do nó é impresso.

O segundo argumento para referências cruzadas compartilha restrições com nomes de nó. O carácter potencialmente problemático nesse contexto é o dois pontos. Veja-se [Restrições de Nomes de Nó do Info], Página 28.

5.5 @xref com Três Argumentos

Um terceiro argumento substitui o nome de nó na saída gerada impressa. O terceiro argumento deveria ser o nome da seção na saída impressa, ou, pelo contrário, indicar o tópico debatido pela aquela seção.

O modelo é semelhante a isto:

```
@xref{nome-nó, rótulo-online, rótulo-impresso}.
```

Por exemplo,

```
@xref{Efeitos Elétricos, Relâmpago, Trovão e Relâmpago},  
para detalhes.
```

produz

```
*Observe Relâmpago: Efeitos Elétricos, para detalhes.
```

no Info e

Veja-se Seção 5.2 [Trovão e Relâmpago], página 57, para detalhes.

em um manual impresso.

Se um terceiro argumento for dado e o segundo estiver vazio, então o terceiro argumento serve para ambos. (Observe como duas vírgulas, lado a lado, marcam o segundo argumento vazio).

```
@xref{Efeitos Elétricos, , Trovão e Relâmpago},  
para detalhes.
```

produz

```
*Observe Trovão e Relâmpago: Efeitos Elétricos, para detalhes.
```

no Info e

Veja-se Seção 5.2 [Trovão e Relâmpago], página 57, para detalhes.
em um manual impresso.

O terceiro argumento para referências cruzadas compartilha restrições com nomes de nó. O carácter potencialmente problemático nesse contexto é o dois pontos. Veja-se [Restrições de Nomes de Nó do Info], Página 28.

De um ponto de vista prático, geralmente é melhor escrever referências cruzadas apenas com o primeiro argumento se o nome do nó e o título da seção forem os mesmos (ou quase), e com o primeiro e terceiro argumentos somente se o nome de nó e título forem diferentes.

Texinfo oferece uma configuração para usar o título da seção em vez dos nomes de nó por padrão em referências cruzadas (um terceiro argumento explicitamente especificado ainda tem precedência):

```
@xrefautomaticsectiontitle on
```

Essa configuração também é usada para referências cruzadas para âncoras definidas com `@namedanchor` (veja-se Seção 5.11 [`@anchor`], Página 49).

Tipicamente essa linha seria dada próxima do início do documento e usada para o manual inteiro. Porém, você pode desativá-la se quiser (`@xrefautomaticsectiontitle off`), por exemplo, se estiver incluindo algum outro subdocumento que não tenha nomes adequados de seção. Essa configuração também se aplica a cabeçalhos de nó no HTML: se `@xrefautomaticsectiontitle` estiver ligado, os nomes de seções serão usados em cabeçalhos de nó em vez dos nomes de nó quando possível. Essa configuração não tem efeito na saída Info ou na de texto simples.

5.6 @xref com Quatro e Cinco Argumentos

Em uma referência cruzada, um quarto argumento especifica o nome de outro manual, diferente do arquivo no qual a referência aparece, e um quinto argumento especifica o título dele como um manual impresso.

O modelo completo é:

```
@xref{nome-nó, rótulo-online, rótulo-impresso,  
nome-manual, título-manual-impresso}.
```

Por exemplo,

```
@xref{Efeitos Elétricos, Relâmpago, Trovão e Relâmpago,  
clima, Uma Introdução à Meteorologia}.
```

produz esta saída no Info:

```
*Observe Relâmpago: (clima)Efeitos Elétricos.
```

Como você pode ver, o nome do manual está entre parênteses e precede o nome do nó. No HTML, o nome do manual e o nome do nó são usados para construir o URL de hiperlink (veja-se Seção 22.7 [Xref do HTML], Página 197), enquanto o texto do link é baseado no rótulo.

Em um manual impresso, a referência se parece com isto:

Veja-se seção “Trovão e Relâmpago” em *Uma Introdução à Meteorologia*.

O título do manual impresso é tipografado como `@cite`; e a referência carece de um número de página, pois a página a qual uma referência se refere quando essa referência é para outro manual não pode ser conhecida.

Próximo caso: frequentemente você deixará de fora o segundo argumento quando usar a versão longa de `@xref`. Nesse caso, o terceiro argumento, a descrição do tópico, será usado como o nome da referência cruzada em formatos online. Por exemplo,

```
@xref{Efeitos Elétricos, , Trovão e Relâmpago,  
clima, Uma Introdução à Meteorologia}.
```

produz

```
*Observe Trovão e Relâmpago: (clima)Efeitos Elétricos.
```

no Info e

Veja-se seção “Trovão e Relâmpago” em *Uma Introdução à Meteorologia*.
em um manual impresso.

Próximo caso: se o nome do nó e o título da seção forem os mesmos no outro manual, você também pode deixar de fora o título da seção. Nesse caso, o nome de nó é usado em ambas as instâncias. Por exemplo,

```
@xref{Efeitos Elétricos,,,
      clima, Uma Introdução à Meteorologia}.
```

produz

```
*Observe (clima)Efeitos Elétricos::.
```

no Info e

Veja-se seção “Efeitos Elétricos” em *Uma Introdução à Meteorologia*.
em um manual impresso.

Em geral, não existe razão para ter um argumento de nome de manual sem um argumento de manual impresso, a menos que nenhum manual impresso seja gerado. Você também pode querer se referir a outro arquivo de manual que esteja dentro de um manual impresso—quando vários arquivos do Texinfo forem incorporados ao mesmo manual impresso, mas puderem criar arquivos de saída separados em outros formatos de saída. Nesse caso, você precisa especificar somente o quarto argumento, e não o quinto. Se o argumento de título do manual impresso estiver ausente, o nome do manual será usado na saída impressa.

Um argumento de título de manual impresso sem um argumento de manual online é de pouco uso, a menos que somente um manual impresso seja gerado a partir do fonte do Texinfo. O resultado em formatos online depende do formato e pode ser, por exemplo, um nome vazio de manual ou uma referência para o manual impresso formatado de maneira semelhante para a saída impressa.

Finalmente, também é permitido deixar de fora todos os argumentos, *exceto* o quarto e quinto, para referenciar outro manual como um todo. Veja-se a próxima seção.

5.7 Referenciamento para um Manual como um Todo

Normalmente, você sempre precisa nomear um nó em uma referência cruzada. Entretanto, não é incomum querer se referir a outro manual como um todo, em vez de uma seção em particular dentro dele. Nesse caso, dar-se algum nome de seção é uma distração desnecessária.

Assim, com referências cruzadas para outros manuais (veja-se Seção 5.6 [Quatro e Cinco Argumentos], Página 46), se o primeiro argumento for ou ‘Top’ (primeira letra maiúscula exatamente dessa maneira) ou inteiramente omitido, e o terceiro argumento for omitido, a saída impressa não incluirá nome de nó ou de seção. (A saída Info incluirá ‘Top’ se foi fornecido). Por exemplo,

```
@xref{Top,,, make, O Manual do GNU Make}.
```

produz

```
*Observe (make)Top::.
```

e

Veja-se *O Manual do GNU Make*.

Leitores Info irão para o nó Top do manual se o nó ‘Top’ estiver explicitamente especificado ou não.

Também é possível (e é prática histórica) referenciar um manual inteiro especificando-se o nó ‘Top’ e uma entrada apropriada para o terceiro argumento para o comando `@xref`. Usando esse dialeto, para fazer uma referência cruzada para *O Manual do GNU Make*, você poderia escrever:

```
@xref{Top,, Visão Geral, make, O Manual do GNU Make}.
```

que produz:

```
*Observe Visão Geral: (make)Top.
```

no Info e

Veja-se seção “Visão Geral” in *O Manual do GNU Make*.

em um manual impresso.

Nesse exemplo, ‘Top’ é o nome do primeiro nó e ‘Visão Geral’ é o nome da primeira seção do manual. Não existe convenção amplamente usada para nomear a primeira seção em um manual impresso; isso é apenas o que o manual do Make usa. Essa arbitrariedade do primeiro nome é a razão principal pela qual é preferível omitir-se o terceiro argumento em referências cruzadas de manual inteiro.

5.8 @xref

O comando `@xref` gera uma referência cruzada para o início de uma frase. Exemplo de uso do `@xref` estão em seções anteriores.

5.9 @ref

`@ref` é aproximadamente o mesmo que `@xref`, exceto que não gera um ‘Veja-se’ na saída gerada, apenas a referência em si. Isso a torna útil como a última parte de uma frase.

Por exemplo,

```
Para mais informações, @pxref{Isso}, e @ref{Aquilo}.
```

produz em Info:

```
Para mais informações, *observe Isso::, e *observe Aquilo::.
```

e em saída impressa:

```
Para mais informações, veja-se Seção 1.1 [Isso], página 1, e Seção 1.2 [Aquilo], página 2.
```

O comando `@ref` pode instigar os(as) escritores(as) a se expressarem de uma maneira que seja adequada para um manual impresso, porém parecerem estranhos(as) no formato Info. Tenha em mente que teu público poderia estar usando ambos, o impresso e outros formatos de saída, como o Info. Por exemplo:

```
Ondas do mar estão descritas em @ref{Furacões}.
```

aparenta ok na saída impressa:

```
Ondas do mar estão descritas na Seção 6.7 [Furacões], página 72.
```

porém, é estranho ler no Info, “observe” sendo um verbo:

```
Ondas do mar estão descritas em *observe Furacões::.
```

5.10 @pxref

O comando de referência entre parênteses, `@pxref`, é aproximadamente o mesmo que `@xref`, porém ele é melhor usado dentro de parênteses. O comando difere do `@xref` em que a referência é tipografada com um ‘veja’ em letras minúsculas em vez de um ‘Veja’ em letra maiúscula. No Info, ‘*observe’ é gerado.

Com um argumento, uma referência cruzada entre parênteses se parece com isto:

```
... tempestades causam inundações (@pxref{Furacões}) ...
```

que produz:

```
... tempestades causam inundações (*observe Furacões::) ...
```

no Info e

```
... tempestades causam inundações (veja-se Seção 6.7 [Furacões], página 72) ...
```

em um manual impresso.

O efeito de ‘@pxref{nome-nó}’ é idêntico ao de ‘@ref{nome-nó}’ na saída gerada do Info. Portanto, pode ser preferível, quando não estiver entre parênteses, escrever ‘veja-se @ref{nome-nó}’ em vez de ‘@pxref{nome-nó}’ quando você quiser que a palavra “veja-se” ocorra na saída gerada, pois os leitores Info não conseguem confiavelmente inserir essa palavra com base no marcador de referência cruzada ‘*observação’ (por exemplo, usando o variável `Info-hide-note-references`; veja-se Seção “Texto invisível no Info do Emacs” em *Info: Uma Introdução*).

Nas versões anteriores do Texinfo, não era permitido escrever pontuação depois de um @pxref, de modo que poderia ser usado *somente* antes de um parêntese direito. Esse não mais é o caso.

5.11 @anchor: Definindo Alvos Arbitrários de Referências Cruzadas

Talvez você deseje criar referências cruzadas para posições arbitrárias em um documento, não apenas para o início dos nós. O comando @anchor marca essa posição no teu documento, chamada *âncora*. Coloque um comando @anchor logo antes da posição que você deseja se referir. Por exemplo:

```
Isso marca o ponto @anchor{x-spot}.
```

```
...
```

```
@xref{x-spot,,o ponto}.
```

Isso produz:

```
Isso marca o ponto.
```

```
...
```

```
Veja-se [o ponto], página 1.
```

O argumento delimitado por chaves para @anchor, aqui ‘x-spot’, é o *nome da âncora*, que desempenha o mesmo papel que um nome de nó. O nome da âncora é usado para criar referências cruzadas para a âncora, como com um @xref ou com outro comando de referência cruzada, conforme mostrado (veja-se Capítulo 5 [Referências Cruzadas], Página 43). Como você pode ver, o comando @anchor em si não produz saída.

Nomes de âncora compartilham as mesmas restrições que nós acerca de caracteres que podem ser incluídos (veja-se [Restrições de Nomes de Nó do Info], Página 28). Nomes de âncora, nomes de nós e rótulos de flutuadores não podem conflitar.

Você pode colocar o comando @anchor em uma linha própria se isso ajudar a legibilidade do fonte. Espaços em branco (incluindo novas linhas) são ignorados depois do @anchor.

Quando você deletar ou renomear um nó, geralmente é uma boa ideia definir um @anchor com o nome antigo. Dessa maneira, quaisquer links para o nó antigo, sejam originários de outros manuais Texinfo ou de páginas web em geral, continuarão funcionando.

Por padrão, os links para âncoras não são afetados pela configuração de `xrefautomaticsectiontitle` (veja-se Seção 5.5 [xrefautomaticsectiontitle], Página 45). Isso significa que o nome da âncora será usado como texto de rótulo na referência cruzada.

No entanto, você pode definir uma *âncora nominada* usando @namedanchor. O modelo é:

```
@namedanchor{nome-âncora, nome-xref}
```

Use isso exatamente como faria com @anchor, exceto que o parâmetro adicional *nome-xref* fornece o texto para usar se `xrefautomaticsectiontitle` estiver ativado. Por exemplo:

```
Isso marca o @namedanchor{x-spot, o ponto}ponto.
```

```
...
@set xrefautomaticsectiontitle on
@xref{x-spot}.
```

Isso produz:

```
Isso marca o ponto.
...
Veja-se [o ponto], página 1.
```

5.12 @link: Simples, hiperlink desadornado

@link produz um hiperlink simples nos formatos de saída que o suportam, incluindo em HTML, DocBook, L^AT_EX e PDF online. O modelo é:

```
@link{nome-nó, rótulo, nome-manual}
```

nome-nó é o nome do nó ou âncora alvo. Ou um ou ambos de *rótulo* e *nome-manual* podem ser omitidos. *rótulo*, se fornecido, é o texto a se usar para o link. *nome-manual* é o nome do manual externo em que o alvo aparece; Se não fornecido, a referência é para o manual atual.

@link tem saída gerada semelhante a **@ref**, exceto que ele não produz nenhum texto extra em torno do rótulo do linque no Info ou na saída impressa que o marcaria como uma referência cruzada.

Tenha cuidado ao usar **@link** para produzir linques que sejam necessários para um(a) usuário(a) se mover em um manual, pois esses linques não farão nada em alguns formatos de saída. **@link** é melhor usado para adicionar linques de conveniência que, no entanto, não são essenciais para um(a) leitor(a) entender o texto do manual. Por exemplo, você pode usar **@link** em uma amostra de código para referenciar documentação de um símbolo em uma biblioteca de programação.

5.13 @inforef: Referências cruzadas para material unicamente Info

@inforef é usado para fazer referências cruzadas para documentos Info—mesmo a partir de um manual impresso. Isso originalmente era usado para arquivos Info que não fossem gerados a partir de algum fonte Texinfo. O comando agora está obsoleto e não deveria ser usado. Adicionalmente a ter pouco uso, saída similar pode ser obtida com **@xref**, **@ref** ou **@pxref** com o nome do arquivo Info como o quarto argumento e sem quinto argumento.

O comando recebe ou dois ou três argumentos, na seguinte ordem:

1. O nome do nó.
2. O nome da referência cruzada (opcional).
3. O nome do arquivo Info.

O modelo é:

```
@inforef{nome-nó, nome-referência-cruzada, nome-arquivo-info}
```

5.14 @url, @uref{url[, texto][, substituição]}

@url produz uma referência para um Localizador Uniforme de Recursos (URL). Ele recebe um argumento obrigatório, a URL, e dois argumentos opcionais que controlam o texto que é exibido. Em saída HTML e PDF, **@url** produz um linque que você consegue seguir. (Para meramente indicar uma URL sem criar um linque que as pessoas possam seguir, use **@indicateurl**, veja-se Seção 6.1.15 [**@indicateurl**], Página 61).

@uref é um sinônimo para **@url**. (Originalmente, **@url** tinha o significado de **@indicateurl** e **@uref** era obrigado a produzir um linque funcional, porém, na prática, **@url** quase sempre era mal usado. Então, nós mudamos o significado).

O segundo argumento, se especificado, é o texto a exibir (o padrão é a URL em si); em formatos de saída diferentes de HTML, a URL é gerada em adição a esse texto.

O terceiro argumento, se especificado, é o texto a exibir, mas nesse caso a URL não é gerada em nenhum formato. Isso é útil quando o texto já é suficientemente referencial, como em uma página de manual. Além disso, se o terceiro argumento for fornecido, o segundo argumento será ignorado.

5.14.1 Exemplos @url

Primeiro, aqui está um exemplo da forma mais simples de @url, com apenas um argumento. A URL fornecida é tanto o alvo quanto o texto visível do link:

O sítio FTP GNU oficial é @url{https://ftp.gnu.org/gnu}.

produz:

O sítio FTP GNU oficial é <https://ftp.gnu.org/gnu>.

Forma de dois argumentos de @url

Aqui está um exemplo da forma de dois argumentos:

O @url{https://ftp.gnu.org/gnu, sítio FTP GNU oficial}
mantém programas e textos.

o que produz:

O sítio FTP GNU oficial (<https://ftp.gnu.org/gnu>)
mantém programas e textos.

A saída HTML é esta:

O sítio FTP GNU oficial
mantém programas e textos.

Em outros formatos, a saída gerada é semelhante a isto:

O sítio FTP GNU oficial (<https://ftp.gnu.org/gnu>)
mantém programas e textos.

Forma de três argumentos de @url

Finalmente, um exemplo da forma de três argumentos:

O programa @url{/man.cgi/1/ls,,ls} ...

o que, exceto para HTML, produz:

O programa ls ...

porém com HTML:

O programa ls ...

A propósito, algumas pessoas preferem exibir URLs no inequívoco formato:

<URL:http://dispositivo/caminho>

Você pode usar essa forma no arquivo de entrada de desejar. Nós sentimos que não é necessário incluir o ‘<URL:’ e ‘>’ na saída gerada, pois, para ser útil, qualquer software que tente detectar URLs em texto já tem de detectá-las sem o ‘<URL:’.

5.14.2 Quebra de Linha de URL

TeX permite quebra de linha dentro de URLs com apenas alguns caracteres (os quais são especiais em URLs): ‘&’, ‘.’, ‘#’, ‘?’ e ‘/’ (mas não entre dois caracteres ‘/’). Uma pequena quantidade de espaço elástico também é inserida em torno desses caracteres para ajudar com a quebra de linha.

Para saída gerada HTML, navegadores modernos também farão quebra de linha dentro de URLs exibidas. Se precisar permitir quebras em outros caracteres, você pode inserir @/ conforme necessário (veja-se Seção 13.2 [Quebras de Linha], Página 110).

Por padrão, no $\text{\TeX}$, quaisquer de tais quebras em caracteres especiais ocorrerão depois do caractere. Algumas pessoas preferem que tais quebras aconteçam antes do caractere especial. Isso pode ser controlado com o comando `@urefbreakstyle` (esse comando tem efeito somente no $\text{\TeX}$):

`@urefbreakstyle` *como*

onde o argumento *como* é uma dessas palavras:

- ‘after’ (O padrão) Potencialmente quebra depois dos caracteres especiais.
- ‘before’ Potencialmente quebra antes dos caracteres especiais.
- ‘none’ Não considera a quebra em caracteres especiais em absoluto; quaisquer quebras potenciais precisam ser inseridas manualmente.

5.14.3 @url Formato de Saída PDF

Se o propósito final de um PDF for somente para ser visto online, talvez similar a HTML de alguma maneira incipiente, você pode não querer que as URLs sejam incluídas no texto visível (exatamente como as URLs não são visíveis para leitores(as) de páginas da web). Texinfo fornece uma opção específica de PDF para isso, a qual precisa ser usada dentro de `@tex`:

```
@tex
\global\urefurlonlylinktrue
@end tex
```

O resultado é que `@url{https://www.gnu.org, GNU}` tem a saída gerada visível de apenas ‘GNU’, com um alvo de linque de `https://www.gnu.org`. Normalmente, a saída gerada visível incluiria tanto o rótulo quanto a URL: ‘GNU (`https://www.gnu.org`)’.

Essa opção somente tem efeito quando a saída PDF for produzida com o programa `pdf $\text{\TeX}$` , não com outras maneiras de se obter de Texinfo para PDF (por exemplo, $\text{\TeX}$ para DVI para PDF). Consequentemente, está certo se especificar essa opção incondicionalmente dentro de `@tex`, conforme mostrado acima. Ela é ignorada quando DVI está sendo produzido.

5.15 @cite{referência}

Use o comando `@cite` para o nome de um livro que careça de um arquivo complementar Info. Por exemplo, nós poderíamos referenciar *Um Livro*. O comando seleciona uma fonte inclinada no manual impresso e gera aspas no arquivo Info.

Se um livro for escrito em Texinfo, é melhor usar um comando de referência cruzada, pois um(a) leitor(a) consegue facilmente seguir tal referência no Info. Veja-se Seção 5.8 [`@xref`], Página 48.

5.16 Cores do PDF

Por padrão, links de URLs e de referências cruzadas são impressos em preto na saída gerada PDF. Muito ocasionalmente, entretanto, você pode querer realçar tais links “vivos” com uma cor diferente, como é feito comumente em páginas da web. Texinfo fornece uma opção específica de PDF para se especificar essas cores, a qual precisa ser usada dentro de `@tex`:

```
@tex
\global\def\linkcolor{1 0 0} % vermelha
\global\def\urlcolor{0 1 0} % verde
@end tex
```

`\urlcolor` muda a cor da saída gerada `@url` (tanto a URL real, quanto qualquer rótulo textual), ao passo que `\linkcolor` muda a cor para referências cruzadas para nós, etc. Elas são independentes.

Os três valores fornecidos precisam ser números entre 0 e 1, especificando a quantidade de vermelho, verde e azul, respectivamente.

Essas definições somente tem um efeito quando a saída gerada PDF for produzida com o programa pdf $\text{\TeX}$, não com outros meios de se obter de Texinfo para PDF (por exemplo, $\text{\TeX}$ para DVI para PDF). Consequentemente, está certo se especificar essa opção incondicionalmente dentro de `@tex`, conforme mostrado acima. Ela é ignorada quando DVI está sendo produzido.

Nós não recomendamos colorir apenas por diversão; a menos que você tenha um motivo específico para usar cores, melhor ignorá-las.

6 Marcação de Texto, Palavras e Frases

No Texinfo, você consegue marcar palavras e frases de várias maneiras. Os processadores do Texinfo usam essa informação para determinar como realçar o texto. Você pode especificar, por exemplo, quando uma palavra ou frase é uma ocorrência definidora, uma variável metassintática ou um símbolo usado em um programa. Também, você pode enfatizar texto, de diferentes maneiras.

6.1 Indicando Definições, Comandos, etc.

Texinfo tem comandos para indicar exatamente a que tipo de objeto um pedaço de texto se refere. Por exemplo, endereços de correio eletrônico são marcados por `@email`; dessa maneira, o resultado pode ser um link vivo para enviar mensagem eletrônica quando o formato de saída a suportar. Se o endereço de correio eletrônico simplesmente fosse marcado como “imprimir em uma fonte de máquina de escrever”, isso não seria possível.

6.1.1 Comandos de Realçamento são Úteis

Os comandos servem a uma variedade de propósitos:

`@code{código-modelo}`

Indica texto que é um exemplo literal de um pedaço de um programa. Veja-se Seção 6.1.2 [`@code`], Página 55.

`@kbd{caracteres-de-teclado}`

Indica entrada de teclado. Veja-se Seção 6.1.3 [`@kbd`], Página 56.

`@key{nome-de-tecla}`

Indica o nome convencional para uma tecla em um teclado. Veja-se Seção 6.1.4 [`@key`], Página 57.

`@samp{texto}`

Indica texto que é um exemplo literal de uma sequência de caracteres. Veja-se Seção 6.1.5 [`@samp`], Página 57.

`@verb{texto}`

Escreve uma sequência literal de caracteres. Veja-se Seção 6.1.6 [`@verb`], Página 58.

`@var{variável-metassintática}`

Indica uma variável metassintática. Veja-se Seção 6.1.7 [`@var`], Página 58.

`@env{variável-de-ambiente}`

Indica uma variável de ambiente. Veja-se Seção 6.1.8 [`@env`], Página 59.

`@file{nome-arquivo}`

Indica o nome de um arquivo. Veja-se Seção 6.1.9 [`@file`], Página 59.

`@command{nome-comando}`

Indica o nome de um comando. Veja-se Seção 6.1.10 [`@command`], Página 59.

`@option{opção}`

Indica uma opção de linha de comando. Veja-se Seção 6.1.11 [`@option`], Página 60.

`@dfn{termo}`

Indica o uso introdutório ou definidor de um termo. Veja-se Seção 6.1.12 [`@dfn`], Página 60.

`@cite{referência}`

Indica o nome de um livro. Veja-se Seção 5.15 [`@cite`], Página 52.

`@abbr{abreviação}`

Indica uma abreviação, como ‘Comput.’.

`@acronym{sigla}`

Indica uma sigla. Veja-se Seção 6.1.14 [`@acronym`], Página 60.

`@indicateurl{localizador-uniforme-de-recurso}`

Indica um exemplo (isto é, não funcional) de localizador uniforme de recurso. Veja-se Seção 6.1.15 [`@indicateurl`], Página 61. (Use `@url` (veja-se Seção 5.14 [`@url`], Página 50) para URLs vivas).

`@email{endereço-de-email[, texto-exibido]}`

Indica um endereço de correio eletrônico. Veja-se Seção 6.1.16 [`@email`], Página 62.

6.1.2 `@code{código-amostra}`

Use o comando `@code` para indicar texto que seja um pedaço de um programa e que consiste de tokens sintáticos inteiros. Coloque o texto entre chaves.

Assim, você deveria usar `@code` para uma expressão em um programa, para o nome de uma variável ou função usada em um programa ou para uma palavra chave em uma linguagem de programação.

Use `@code` para nomes de comando em linguagens que remontam linguagens de programação, como Texinfo. Por exemplo, `@code` e `@samp` são produzidos escrevendo-se ‘`@code{@code}`’ e ‘`@code{@samp}`’ no fonte Texinfo, respectivamente.

É incorreto alterar a primeira letra de uma palavra dentro de um comando `@code` quando ele aparecer no início de uma frase. A maioria das linguagens de programação diferencia maiúsculas de minúsculas. Em C, por exemplo, `Printf` é diferente do identificador `printf`, e possivelmente seja um erro de ortografia. Mesmo em linguagens que não diferenciam maiúsculas de minúsculas, é confuso para um(a) leitor(a) humano(a) ver identificadores escritos de diferentes maneiras. Escolha uma grafia e sempre a use. Se não quiser iniciar uma frase com um nome de comando escrito todo em letras minúsculas, você deveria reorganizar a frase.

O argumento `@code` é tipografado em uma fonte de máquina de escrever (mono espaçada). Por exemplo,

A função retorna `@code{nulo}`.

produz isto:

A função retorna `nulo`.

Aqui estão alguns casos para os quais é preferível *não* se usar `@code`:

- Para nomes de comando de shell, como `ls` (use `@command`).
- Para variáveis de ambiente, como `TEXINPUTS` (use `@env`).
- Para opções de shell, como ‘`-c`’, quando tais opções sejam independentes (use `@option`).
- Um comando inteiro de shell frequentemente parece melhor se escrito usando-se `@samp` em vez de `@code`. Nesse caso, a regra é escolher o formato mais agradável.
- Para uma sequência de caracteres menor que um token sintático. Por exemplo, se estiver escrevendo acerca de ‘`goto-ch`’, que apenas é parte do nome para a função `goto-char` da Emacs Lisp, você deveria usar `@samp`.
- Em geral, quando escrever acerca dos caracteres usados em um token; por exemplo, não use `@code` quando estiver explicando quais letras ou símbolos imprimíveis podem ser usados nos nomes das funções. (Use `@samp`). Ainda, você não deveria usar `@code` para marcar texto que seja considerado entrada para programas, a menos que a entrada seja escrita em uma linguagem que seja semelhante a uma linguagem de programação. Por exemplo, você não deveria usar `@code` para os comandos de toques de teclas do GNU Emacs (use `@kbd` em

vez disso), embora você possa usar `@code` para os nomes das funções da Emacs Lisp que os comandos de toques de teclas invocam.

Por padrão, $\text{\TeX}$ considerará quebrar linhas nos caracteres ‘-’ e ‘_’ dentro do `@code` e comandos relacionados. Isso pode ser controlado com `@allowcodebreaks` (veja-se Seção 13.4 [`@allowcodebreaks`], Página 111). Na saída gerada HTML, quebrar linhas depende inteiramente do comportamento do navegador. Para Info, parece melhor nunca fazer tais quebras.

Para Info e texto simples, as aspas normalmente são geradas em torno da saída gerada do comando `@code` e comandos relacionados (por exemplo, `@kbd`, `@command`), exceto em contextos tipo máquina de escrever, como o ambiente `@example` (veja-se Seção 7.4 [`@example`], Página 67) e o `@code` em si, etc. Para controlar quais aspas são inseridas por `texi2any` na saída gerada do ‘`@code`’, etc., vejam-se as variáveis de personalização `OPEN_QUOTE_SYMBOL` e `CLOSE_QUOTE_SYMBOL` (veja-se Seção 20.4.8 [Variáveis de Personalização para Múltiplos Formatos], Página 176).

6.1.3 `@kbd{caracteres-de-teclado}`

Use o comando `@kbd` para caracteres de entrada a serem digitados por usuários(as). Por exemplo, para referenciar os caracteres *M-a*, escreva:

```
@kbd{M-a}
```

e para referenciar os caracteres *M-x shell*, escreva:

```
@kbd{M-x shell}
```

Por padrão, o comando `@kbd` produz uma fonte diferente (máquina de escrever inclinada em vez da de máquina de escrever normal, onde o formato da saída gerada permitir), de forma que os(as) usuários(as) consigam distinguir os caracteres que deveriam supostamente digitar a partir daqueles que o computador gera.

Dado que o uso de `@kbd` varia de manual para manual, você pode controlar a fonte comutando com o comando `@kbinputstyle`. Esse comando não tem efeito sobre a saída Info. Escreva esse comando no início de uma linha com uma palavra como um argumento, uma das seguintes:

- ‘code’ Sempre use a mesma fonte para `@kbd` como `@code`.
- ‘example’ Use a fonte de distinção para `@kbd` somente em `@example` e ambientes similares.
- ‘distinct’ (o padrão) Sempre use a fonte de distinção para `@kbd`.

Você consegue embutir outro comando `@` dentro das chaves de um comando `@kbd`. Aqui, por exemplo, está a maneira para descrever um comando que seria descrito mais detalhadamente como “pressione a tecla ‘r’ e então pressione a tecla RETURN”:

```
@kbd{r @key{RET}}
```

Isso produz: *r RET*. (O presente manual usa o padrão para `@kbinputstyle`).

Você também usa o comando `@kbd` se estiver soletrando as letras que digitar; por exemplo:

```
Para dar o comando @code{logout},
digite os caracteres @kbd{l o g o u t @key{RET}}.
```

Isso produz:

```
Para dar o comando logout, digite os caracteres l o g o u t RET.
```

(Além disso, esse exemplo mostra que você pode adicionar espaços para maior clareza. Se você explicitamente quiser mencionar um caractere espaço como um dos caracteres da entrada, escreva `@key{SPC}` para ele).

6.1.4 @key{*nome-tecla*}

Use o comando @key para o nome convencional para uma tecla em um teclado, como em:

```
@key{RET}
```

Você pode usar o comando @key dentro do argumento de um comando @kbd quando a sequência de caracteres a ser digitada incluir uma ou mais teclas que sejam descritas por nome.

Por exemplo, para produzir *C-x* ESC e *M-TAB* você digitaria:

```
@kbd{C-x @key{ESC}}
```

```
@kbd{M-@key{TAB}}
```

Aqui está uma lista dos nomes recomendados para teclas:

SPC	Espaço
RET	Retorno
LFD	Alimentação de Linha (no entanto, como a maioria dos teclados hoje em dia não tem uma tecla de alimentação de linha, talvez seja melhor chamar esse caractere <i>C-j</i>)
TAB	Tabulação
BS	Backspace
ESC	Escape
DELETE	Delete
SHIFT	Shift
CTRL	Control
META	Meta

Existem sutilezas para se lidar com palavras como ‘meta’ ou ‘ctrl’, que são nomes de teclas modificadoras. Quando mencionar um caractere no qual a tecla modificadora for usada, como *Meta-a*, use o comando @kbd sozinho; não use o comando @key; porém, quando estiver se referindo à tecla modificadora isoladamente, use o comando @key. Por exemplo, escreva ‘@kbd{Meta-a}’ para produzir *Meta-a* e ‘@key{META}’ para produzir META.

6.1.5 @samp{*texto*}

Use o comando @samp para indicar texto que seja um exemplo literal ou ‘amostra’ de uma sequência de caracteres em um arquivo, cadeia de caracteres, padrão, etc. Coloque o texto entre chaves. O argumento aparece dentro de aspas simples; além disso, ele é impresso em uma fonte de largura fixa.

Para corresponder @samp{foo} no final da linha,
use a expressão regular @samp{foo\$}.

produz

Para corresponder ‘foo’ no final da linha, use a expressão regular ‘foo\$’.

Sempre que estiver se referindo a caracteres únicos, você deveria usar @samp, a menos que @kbd ou @key sejam mais apropriados. Além disso, você pode usar @samp para declarações inteiras em C e para comandos inteiros do shell—nesse caso, @samp frequentemente parece melhor que @code. Basicamente, @samp é um pega tudo para o que não for coberto por @code, @kbd, @key, @command, etc.

Somente inclua sinais de pontuação entre chaves se eles fizerem parte da sequência de caracteres que você estiver especificando. Escreva sinais de pontuação fora das chaves se tais sinais de

pontuação fizerem parte do texto em Inglês que envolva a sequência de caracteres. Na frase seguinte, por exemplo, as vírgulas e pontos estão fora das chaves:

```
Em Inglês, as vogais são @samp{a}, @samp{e},
@samp{i}, @samp{o}, @samp{u}, e, ocasionalmente,
@samp{y}.
```

Isso produz:

Em Inglês, as vogais são ‘a’, ‘e’, ‘i’, ‘o’, ‘u’, e, ocasionalmente, ‘y’.

6.1.6 @verb{*chartextchar*}

Use o comando @verb para imprimir uma sequência de caracteres literais.

Semelhante ao comando \verb do L^AT_EX, o texto literal pode ser aspeado usando-se qualquer caractere delimitador único. Coloque o texto literal, incluindo os delimitadores, entre chaves. O texto é impresso em uma fonte de largura fixa:

```
Quantos @verb{|\|} de escape alguém precisa para imprimir esta
sequência @verb{. @a @b.@c.} ou esta @verb{+@'e?`{}!`~\+}?
```

produz

```
Quantos @ de escape alguém precisa para imprimir esta
sequência @a @b.@c ou esta @'e?`{}!`~\?
```

Isso está em contraste com @samp (veja-se a seção anterior), @code e comandos semelhantes; naqueles casos, o argumento é texto normal do Texinfo, onde os três caracteres @{} são especiais, como de costume. Com @verb, nada é especial, exceto o caractere delimitador que você escolher.

O próprio caractere delimitador pode aparecer dentro do texto literal, conforme mostrado acima. Como outro exemplo, ‘@verb{...}’ imprime um ponto (largura fixa).

Não é confiável usar-se @verb dentro de outras construções do Texinfo. Em particular, não funciona usar-se @verb em nada relacionado a referências cruzadas, como títulos de seção ou legendas de figuras.

6.1.7 @var{*variável-metassintática*}

Use o comando @var para indicar variáveis metassintáticas. Uma *variável metassintática* é algo que representa outro pedaço de texto. Por exemplo, você deveria usar uma variável metassintática na documentação de uma função para descrever os argumentos que sejam passados para aquela função.

Não use @var para os nomes de variáveis normais em programas de computador. Esses são nomes específicos, de forma que @code é correto para eles. Por exemplo, a variável `texinfo-tex-command` da Emacs Lisp não é uma variável metassintática; ela é formatada adequadamente usando-se @code.

Tampouco use @var para variáveis de ambiente; @env é correto para elas (veja-se a próxima seção).

O efeito de @var no arquivo Info é o de mudar a escrita do argumento para letras todas maiúsculas. No manual impresso, o argumento é gerado em tipo inclinado.¹

¹ Na saída gerada do T_EX, @var usa uma fonte inclinada de máquina de escrever em contextos de código como @code ou @example. Para usar uma fonte romana, inclinada e de largura variável, em todos os contextos, especifique ‘@clear txicodevaristt’ (veja-se Seção 16.5.1 [set @value], Página 133). Observe que esse sinalizador não faz nada na saída gerada do L^AT_EX.

Por exemplo,

```
Para deletar o arquivo @var{nomearquivo},
digite @samp{rm @var{nomearquivo}}.
```

produz

Para deletar o arquivo *nomearquivo*, digite ‘rm *nomearquivo*’.

(Observe que @var pode aparecer dentro de @code, @samp, @file, etc).

Escreva uma variável metassintática toda em letras minúsculas e sem espaços, e use hifens para torná-la mais legível. Assim, o fonte do Texinfo para a ilustração de como iniciar um manual Texinfo se parece com isto:

```
\input texinfo
@@settitle @var{nome-do-manual}
```

Isso produz:

```
\input texinfo
@settitle nome-do-manual
```

Em alguns estilos de documentação, as variáveis metassintáticas são mostradas com colchetes angulares, por exemplo:

```
..., digite rm <nomearquivo>
```

Entretanto, esse não é o estilo que Texinfo usa.

6.1.8 @env{*variável-ambiente*}

Use o comando @env para indicar variáveis de ambiente, conforme usadas por muitos sistemas operacionais, incluindo GNU. Não a use para variáveis *metassintáticas*; use @var para essas (veja-se a seção anterior).

@env é equivalente a @code nos efeitos dele. Por exemplo:

```
A variável de ambiente @env{PATH} ...
```

produz

A variável de ambiente PATH ...

6.1.9 @file{*nome-arquivo*}

Use o comando @file para indicar texto que seja o nome de um arquivo, buffer, ou diretório, ou seja o nome de um nó no Info. Você também consegue usar o comando para sufixos de nome de arquivo. Não use @file para símbolos em uma linguagem de programação; use @code.

@file é equivalente a @code nos efeitos dele. Por exemplo,

```
Os arquivos @file{.el} estão no
diretório @file{/usr/local/emacs/lisp}.
```

produz

Os arquivos .el estão no diretório /usr/local/emacs/lisp.

6.1.10 @command{*nome-comando*}

Use o comando @command para indicar os nomes de comandos, como ls ou cc.

@command é equivalente a @code nos efeitos dele. Por exemplo:

```
O comando @command{ls} lista conteúdo de diretórios.
```

produz

O comando ls lista conteúdo de diretórios.

Você deveria escrever o nome de um programa na fonte comum de texto, em vez de usar @command, se considerá-lo uma nova palavra do inglês, como ‘Emacs’ ou ‘Bison’.

Quando escrever uma invocação inteira de comando de shell, como em ‘ls -l’, você deveria usar ou @samp ou @code a teu critério.

6.1.11 @option{*nome-opção*}

Use o comando @option para indicar uma opção de linha de comando; por exemplo, -l ou --version ou --output=nomearquivo.

@option é equivalente a @code nos efeitos dele. Por exemplo:

A opção @option{-l} produz uma listagem longa.

produz

A opção -l produz uma listagem longa.

6.1.12 @dfn{*termo*}

Use o comando @dfn para identificar o uso introdutório ou definidor de um termo técnico. Use o comando somente em passagens cujo propósito seja o de introduzir um termo que será usado novamente ou que o(a) leitor(a) deva saber. A mera menção de passagem de um termo pela primeira vez não merece @dfn. O comando seleciona uma fonte inclinada no manual impresso, e gera aspas duplas no arquivo Info. Por exemplo:

Livrar-se de um arquivo é chamado @dfn{deletar} ele.

produz

Livrar-se de um arquivo é chamado *deletar* ele.

Como uma regra geral, uma frase contendo a ocorrência definidora de um termo deveria ser uma definição do termo. A frase não precisa dizer explicitamente que é uma definição, mas deveria conter a informação de uma definição—ela deveria tornar claro o significado.

6.1.13 @abbr{*abreviação*[, *significado*]}

Você pode usar o comando @abbr para abreviações gerais. A abreviação é fornecida como o argumento único entre chaves, como em '@abbr{Comput.}'. Por uma questão de estilo, ou para abreviações específicas, você possivelmente prefira omitir pontos, como em '@abbr{Mr} Stallman'.

@abbr aceita um segundo argumento opcional, destinado a ser usado para o significado da abreviação.

Se a abreviação finalizar com uma letra minúscula e um ponto, e não estiver no final de uma frase, e não tiver segundo argumento, lembre-se de usar o comando @. (veja-se Seção 11.3.3 [Finalizando uma Frase], Página 99) para obter o espaçamento correto. Entretanto, você não tem de usar @. dentro da abreviação em si; Texinfo automaticamente presume que pontos dentro da abreviação não finalizam uma frase.

Em formatos de saída com etiqueta apropriada, como HTML e DocBook, essa etiqueta é usada. Do contrário, o primeiro argumento é impresso como está; se o segundo argumento estiver presente, ele é impresso entre parênteses depois da abreviação. Por exemplo:

@abbr{Comput. J., Jornal do Computador}

produz:

Comput. J. (Jornal do Computador)

Para abreviações consistindo de todas as letras maiúsculas, você possivelmente prefira usar o comando @acronym. Veja-se a próxima seção para mais acerca do uso desses dois comandos.

6.1.14 @acronym{*sigla*[, *significado*]}

Você pode usar o comando @acronym para abreviações escritas todas em letras maiúsculas, como 'NASA'. A abreviação é fornecida como o único argumento entre chaves, como em '@acronym{NASA}'. Por uma questão de estilo, ou para siglas específicas, você possivelmente prefira usar pontos, como em '@acronym{N.A.S.A.}'.

`@acronym` aceita um segundo argumento opcional, destinado a ser usado para o significado da sigla.

Se a sigla estiver no final de uma frase, e se não existir segundo argumento, lembre-se de usar o `@.` ou comando similar (veja-se Seção 11.3.3 [Finalizando uma Frase], Página 99) para obter o espaçamento correto.

No $\text{T}_{\text{E}}\text{X}$, a sigla é impressa em fonte ligeiramente menor. Na saída Info, o argumento é impresso como está. Em qualquer formato, e na saída $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$, se o segundo argumento estiver presente, ele é impresso entre parênteses depois da sigla. Em HTML e em DocBook a etiqueta apropriada é usada.

Por exemplo (dado que GNU é uma sigla recursiva, nós usamos `@acronym` recursivamente):

```
@acronym{GNU, @acronym{GNU} Não é Unix}
```

produz:

```
GNU (GNU Não é Unix)
```

Em algumas circunstâncias, é convencional imprimir nomes de família em letras maiúsculas. Não use `@acronym` para isso, pois um nome não é uma sigla. Use `@sc` (veja-se Seção 6.2.2 [Versaletes], Página 62).

`@abbr` e `@acronym` são comandos intimamente relacionados: ambos sinalizam para o(a) leitor(a) que uma forma abreviada está sendo usada, e possivelmente forneça um significado. Ao escolher quando usar esses dois comandos, por favor tenha o seguinte em mente.

- No uso comum do Inglês, siglas são um subconjunto de abreviações: elas incluem palavras pronunciáveis como ‘NATO’, ‘radar’ e ‘snafu’; algumas fontes também incluem siglas de siglas, como ‘Usenet’; híbridas, como ‘SIGGRAPH’; e inicialismos impronunciáveis como ‘FBI’.
- No Texinfo, uma sigla (mas não uma abreviação) deveria consistir somente de letras maiúsculas e pontos, não minúsculas.
- No $\text{T}_{\text{E}}\text{X}$, uma sigla (mas não uma abreviação) é impressa em uma fonte ligeiramente menor.
- Geralmente acaba sendo bastante difícil e (ou) demorado usar-se consistentemente `@acronym` para todas as sequências de letras maiúsculas. Além do mais, parece estranho para algumas siglas estar no tamanho normal da fonte e outras estarem menores. Assim, uma abordagem mais simples que você possivelmente deseje considerar é a de evitar `@acronym` e apenas tipografar tudo como texto normal todo em maiúsculas: ‘GNU’, produzindo a saída ‘GNU’.
- Em geral, não é essencial usar-se quaisquer desses comandos para todas as abreviações; use teu bom senso. O texto é perfeitamente legível sem eles.

6.1.15 `@indicateurl{localizador-uniforme-de-recurso}`

Use o comando `@indicateurl` para indicar um localizador uniforme de recursos na Internet. Isso é puramente para propósitos de marcação e não produz um linque que você possa seguir (use o comando `@url` ou `@uref` para isso, veja-se Seção 5.14 [`@url`], Página 50). `@indicateurl` é útil para URLs que não existem realmente. Por exemplo:

```
Por exemplo, a URL poderia ser @indicateurl{http://exemplo.org/path}.
```

o que produz:

```
Por exemplo, a URL poderia ser ‘http://exemplo.org/path’.
```

A saída oriunda de `@indicateurl` geralmente é semelhante àquela de `@samp` (veja-se Seção 6.1.5 [`@samp`], Página 57).

6.1.16 @email{*endereço-correio-eletrônico*[, *texto-exibido*]}

Use o comando @email para indicar um endereço de correio eletrônico. Ele recebe um argumento obrigatório, o endereço, e um argumento opcional, o texto a exibir (o padrão é o próprio endereço).

No Info, o endereço é mostrado entre colchetes angulares, precedido pelo texto a exibir, se existir. Na saída impressa, os colchetes angulares são omitidos. Na saída HTML e DocBook, @email produz um linque ‘mailto’. No HTML, um linque ‘mailto’ geralmente abre uma janela de composição de mensagem. Por exemplo:

```
Envie informes de defeitos para @email{bug-texinfo@gnu.org},
sugestões para o @email{bug-texinfo@gnu.org, algum lugar}.
```

produz

```
Envie informes de defeitos para bug-texinfo@gnu.org,
sugestões para o algum lugar.
```

6.2 Enfatizando Texto

Normalmente, Texinfo muda a fonte para marcar palavras no texto de acordo com a categoria a que as palavras pertencem; um exemplo é o comando @code. Na maioria das vezes, essa é a melhor maneira para marcar palavras. Entretanto, ocasionalmente, você desejará enfatizar texto sem indicar uma categoria. Texinfo tem dois comandos para fazer isso. Além disso, Texinfo tem vários comandos que especificam a fonte na qual o texto será gerado. Esses comandos não tem efeito no Info e somente um deles, o comando @r, tem algum uso regular.

6.2.1 @emph{*texto*} e @strong{*texto*}

Os comandos @emph e @strong são para ênfase; @strong é mais forte. Na saída impressa, @emph produz *itálico* e @strong produz **negrito**. Na saída Info, @emph envolve o texto com sublinhados (‘_’), e @strong coloca asteriscos em volta do texto.

Por exemplo,

```
@strong{Cuidado:} @samp{rm * .[^.]*}
remove @emph{todos} os arquivos no diretório.
```

produz o seguinte:

```
Cuidado: ‘rm * .[^.]*’ remove todos os arquivos no diretório.
```

O comando @strong raramente é usado, exceto para marcar o que é, na verdade, um elemento tipográfico, como a palavra ‘Cuidado’ no exemplo precedente.

Cuidado: Não use @strong com a palavra ‘Observação’ seguida por um espaço; Info confundirá a combinação para uma referência cruzada. Use uma frase, como **Por favor note** ou **Cuidado** em vez disso, ou o argumento opcional para @quotation— ‘Observação’ é permitido lá.

6.2.2 @sc{*texto*}: A Fonte de Versaletes

Use o comando ‘@sc’ para configurar texto em UMA FONTE DE VERSALETES (onde possível). Escreva o texto que você quer que esteja em versaletes entre chaves e em letras minúsculas, como isto:

```
Richard @sc{Stallman} a commencé le projet GNU.
```

Isso produz:

```
Richard STALLMAN a commencé le projet GNU.
```

Como mostrado aqui, nós recomendamos reservar @sc para casos especiais onde você queira versaletes tipográfica; nomes de família são um desses, especialmente em outros idiomas diferentes do Inglês, embora não existam regras rígidas acerca de tais coisas.

TEX tipografa quaisquer letras maiúsculas entre as chaves de um comando `@sc` em letras maiúsculas de tamanho real; somente letras minúsculas são impressas na fonte de versaletes. Na saída Info, o argumento para `@sc` é impresso todo em maiúsculas. Em HTML, o argumento é colocado em letras maiúsculas e a saída marcada com a etiqueta `<small>` para reduzir o tamanho da fonte, pois o HTML não consegue facilmente representar versaletes verdadeiras. No L^AT_EX, um comando configurando fontes de versaletes é gerado.

No geral, nós recomendamos usar letras padrão maiúsculas e minúsculas sempre que possível.

6.2.3 Fontes para Impressão

Texinfo fornece um comando para mudar o tamanho da fonte principal do corpo na saída impressa para um documento: `@fonttextsize`. Ele não tem efeito em outra saída. Ele recebe um argumento no restante da linha, o qual precisa ser ou ‘10’ ou ‘11’. Por exemplo:

```
@fonttextsize 10
```

O efeito é o de reduzir a fonte do corpo para um tamanho de 10 pt (o padrão é 11 pt). Fontes para outros elementos, tais como seções e capítulos, são reduzidas de acordo. Isso somente deveria ser usado em conjunto com `@smallbook` (veja-se Seção E.6 [`@smallbook`], Página 280) ou similar, pois fontes de 10 pt em papel padrão (8.5x11 ou A4) são muito pequenas. Uma razão para se usar esse comando é para economizar páginas, e, assim, custo de impressão, para livros físicos.

Texinfo atualmente não tem comandos para alternar a família da fonte a se usar, ou comandos de mudança de tamanho mais gerais.

Texinfo também fornece vários comandos de fonte que especificam mudanças de fonte no manual impresso e (onde possível) na saída HTML e DocBook. Eles não tem efeito no Info. Todos os comandos se aplicam a um argumento seguinte cercados por chaves.

```
@b      seleciona face negrito;
```

```
@i      seleciona uma fonte itálica;
```

```
@r      seleciona uma fonte romana, que é a fonte usual na qual o texto é impresso. Ela
```

```
        pode ou não ser serifada.
```

```
@sansserif
```

```
        seleciona uma fonte sans serifa;
```

```
@slanted seleciona uma fonte inclinada;
```

```
@t      seleciona a fonte de tamanho fixo, estilo máquina de escrever usada por @code;
```

O comando `@r` pode ser útil em ambientes tipo exemplo, para escrever comentários na fonte romana padrão em vez da fonte de largura fixa. Isso aparenta melhor na saída impressa.

Por exemplo,

```
@lisp
(+ 2 2)      ; @r{Adiciona dois mais dois.}
@end lisp
```

produz

```
(+ 2 2)      ; Adiciona dois mais dois.
```

O comando `@t` ocasionalmente pode ser útil para se produzir saída em uma fonte de máquina de escrever onde isso for suportado, mas nenhuma distinção com aspas é necessária no Info ou em texto simples. (Compare-se `@t{foo}` produzindo `foo` com `@code{foo}` produzindo `foo`). Aqui estão alguns possíveis motivos para se usar `@t` em vez de `@code`:

- O argumento é um caractere único
- Já existem aspas de algum tipo cercando o argumento

- É evidente a partir do contexto ou do próprio argumento que o argumento poderia ser código de computador (por exemplo: nome de um grupo de notícias da Usenet)

Em geral, os outros comandos de fonte provavelmente não são úteis; eles existem principalmente para possibilitar documentar a funcionalidade de efeitos específicos de fontes, como no $\text{\TeX}$ e pacotes relacionados.

7 Citações e Exemplos

Citações e exemplos são blocos de texto consistindo de um ou mais parágrafos inteiros que estão separados da maior parte do texto e tratados diferentemente. Eles geralmente estão recuados na saída gerada.

No Texinfo, você sempre inicia uma citação ou exemplo escrevendo um comando `@` sozinho no início de uma linha, e o finaliza escrevendo um comando `@end` que também esteja sozinho no início de uma linha. Por exemplo, você inicia um exemplo escrevendo `@example` sozinho no início de uma linha e finaliza o exemplo escrevendo `@end example` sozinho em uma linha, no início daquela linha e somente com um espaço entre o `@end` e o `example`.

7.1 Comandos de Cercamento de Blocos

Aqui está um resumo de comandos que circundam blocos de texto, também conhecidos como *ambientes*. Eles estão mais explicados nas seções seguintes.

`@quotation`

Indica texto que seja aspeado. O texto é preenchido, recuado (a partir de ambas as margens) , e impresso em uma fonte romana por padrão.

`@indentedblock`

Como `@quotation`, mas o texto é recuado somente na esquerda.

`@example` Ilustra código, comandos, e afins. O texto é impresso em uma fonte de largura fixa, e recuado, mas não preenchido.

`@lisp` Como `@example`, mas especificamente para ilustrar código Lisp. O texto é impresso em uma fonte de largura fixa, e recuado, mas não preenchido.

`@verbatim`

Marca um pedaço de texto que seja para ser impresso literalmente; nenhuma substituição de caractere é feita e todos os comandos são ignorados, até o próximo `@end verbatim`. O texto é impresso em uma fonte de largura fixa, e não é recuado ou preenchido. Espaços extras e linhas em branco são significantes, e tabulações são expandidas.

`@display` Exibe texto ilustrativo. O texto é recuado, mas não preenchido, e nenhuma fonte é selecionada (então, por padrão, a fonte é romana).

`@format` Como `@display` (o texto não é preenchido e nenhuma fonte é selecionada), mas o texto não é recuado.

`@smallquotation`

`@smallindentedblock`

`@smallexample`

`@smalllisp`

`@smalldisplay`

`@smallformat`

Esses comandos `@small...` são exatamente semelhantes aos equivalentes deles `small`, exceto que geram texto em um tamanho de fonte menor, quando possível.

`@flushleft`

`@flushright`

O texto não é preenchido, mas é alinhado com a margem esquerda ou direita, respectivamente.

`@raggedright`

O texto é preenchido, mas justificado somente na esquerda, deixando a margem direita irregular.

@cartouche

Realça o texto, frequentemente um exemplo ou uma citação, desenhando uma caixa com cantos arredondados em torno dele.

O comando **@exdent** é usado dentro das construções acima para desfazer o recuo de uma linha.

O comando **@noindent** pode ser usado depois de uma das construções acima (ou no início de qualquer parágrafo) para evitar que o texto seguinte seja recuado como um novo parágrafo.

7.2 @quotation: Citações de Bloco

O texto de uma citação é processado como texto normal (fonte regular, texto é preenchido), exceto que:

- a margem esquerda está mais próxima do centro da página, de forma que a totalidade da citação esteja recuada; a margem direita também pode estar mais próxima do centro da página
- as primeiras linhas de parágrafos não são recuadas mais que outras linhas; e
- um comando **@author** pode ser fornecido para se especificar o autor da citação.

Este é um exemplo de texto escrito entre um comando **@quotation** e um comando **@end quotation**. Um comando **@quotation** é usado mais frequentemente para indicar texto que é extraído a partir de outro trabalho impresso (real ou hipotético).

Escreva um comando **@quotation** como texto sozinho em uma linha. Essa linha desaparecerá da saída. Marque o final da citação com uma linha iniciando com e contendo somente **@end quotation**. A linha do **@end quotation** também desaparecerá da saída.

@quotation aceita um argumento opcional, fornecido no restante da linha. Esse texto, se presente, é incluído no início da citação, em negrito, ou, caso contrário, enfatizado, e seguido com um ‘:’. Por exemplo:

```
@quotation Observação
Isto é
um foo.
@end quotation
```

produz

Observação: Isto é um foo.

Se o argumento do **@quotation** for uma destas palavras do Inglês (não diferencia maiúsculas de minúsculas):

```
Caution Important Note Tip Warning
```

então a saída DocBook usará etiquetas especiais correspondentes (**<note>**, entre outras) em vez da padrão **<blockquote>**.

Se o(a) autor(a) da citação for especificado(a) no bloco **@quotation** com o comando **@author**, uma linha com o nome do(a) autor(a) é exibida depois da citação:

```
@quotation
As pessoas às vezes me perguntam se é pecado na Igreja do Emacs usar
vi. Usar uma versão livre do vi não é pecado; é uma penitência. Então,
feliz hacking.

@author Richard Stallman
@end quotation
```

produz

As pessoas às vezes me perguntam se é pecado na Igreja do Emacs usar vi. Usar uma versão livre do vi não é pecado; é uma penitência. Então, feliz hacking.

—*Richard Stallman*

7.3 @indentedblock: Blocos recuados de texto

O ambiente `@indentedblock` é semelhante ao `@quotation`, exceto que o texto somente é recuado à esquerda (e não existe argumento opcional para um(a) autor(a)). Assim, a fonte do texto permanece inalterada, e o texto é reunido e preenchido como de costume, mas a margem esquerda é aumentada. Por exemplo:

Este é um exemplo de texto escrito entre um comando `@indentedblock` e um comando `@end indentedblock`. O ambiente `@indentedblock` pode conter qualquer texto ou outros comandos desejados.

Isso é escrito no fonte do Texinfo como:

```
@indentedblock
Este é um exemplo ...
@end indentedblock
```

7.4 @example: Texto de Exemplo

O ambiente `@example` é usado para indicar entrada ou saída de computador que não seja parte do texto em execução. Se você quiser embutir fragmentos de código em frases, use o comando `@code` ou os parentes dele (veja-se Seção 6.1.2 [`@code`], Página 55).

Escreva um comando `@example` sozinho no início de uma linha. Marque o final do bloco com `@end example`. Por exemplo,

```
@example
cp foo @var{dest1}; \
cp foo @var{dest2}
@end example
```

produz

```
cp foo dest1; \
cp foo dest2
```

A saída gerada usa uma fonte de largura fixa e é recuada. Cada linha no arquivo de entrada é uma linha na saída gerada; isto é, o texto fonte não é preenchido. Espaços extras e linhas em branco são significativos. Comandos do Texinfo *são* expandidos; se você quiser que a saída gerada seja a entrada literal, use o ambiente `@verbatim` em vez disso (veja-se Seção 7.5 [`@verbatim`], Página 68).

Exemplos frequentemente estão, falando logicamente, “no meio” de um parágrafo, e o texto que continua após não deveria ser recuado, como no exemplo acima. O comando `@noindent` evita que um pedaço de texto seja recuado como se fosse um parágrafo novo (veja-se Seção 7.12 [`@noindent`], Página 71).

Se você desejar usar a fonte romana normal para um comentário de código, você pode usar o comando `@r` (veja-se Seção 6.2.3 [Fontes], Página 63).

Você pode opcionalmente fornecer argumentos para o comando `@example`, separados por vírgulas, se existir mais que um. Na saída gerada HTML, quaisquer argumentos desse tipo são gerados como nomes de classe, prefixados pela sequência de caracteres ‘`user-`’. Isso pode ser útil para adicionar realce de sintaxe para manuais para amostras de código.

Nós recomendamos que, ao fornecer vários argumentos para `@example`, você use o primeiro argumento para especificar a linguagem do código (por exemplo ‘`C`’, ‘`lisp`’, ‘`Cplusplus`’). Você pode encontrar usos para outros argumentos, como fornecer uma dica de formatação ou marcar

amostras de código para extração e processamento posterior, mas por enquanto nada definitivo é recomendado. Talvez isso mude em lançamentos futuros do Texinfo.

Cuidado: Não use tabulações nas linhas de um exemplo! (Ou em qualquer outro lugar no Texinfo, exceto em ambientes literais). $\TeX$ trata tabulações como espaços simples, e não é isso que elas parecem.

7.5 @verbatim: Texto Literal

Use o ambiente `@verbatim` para imprimir texto que pode conter caracteres especiais ou comandos que não deveriam ser interpretados, como saída ou entrada de computador (`@example` interpreta o texto dele como comandos regulares do Texinfo). Isso é útil especialmente para incluir arquivos gerados automaticamente em um manual do Texinfo.

Em geral, a saída gerada será exatamente a mesma que a entrada. Nenhuma substituição de caracteres é feita; por exemplo, todos os espaços e linhas em branco são significantes, incluindo tabulações. O texto é tipografado em uma fonte de largura fixa, e não recuado ou preenchido.

Escreva um comando `@verbatim` sozinho no início de uma linha. Essa linha desaparecerá da saída gerada. Marque o final do bloco literal com um comando `@end verbatim`, também escrito sozinho no início de uma linha. O `@end verbatim` também desaparecerá da saída gerada.

Por exemplo:

```
@verbatim
{
<tab>@command com caracteres estranhos: @'e
expand<tab>me
}
@end verbatim
```

(onde `<tab>` significa um caractere literal de tabulação). Isso produz:

```
{
    @command com caracteres estranhos: @'e
expand me
}
```

Como as linhas contendo `@verbatim` e `@end verbatim` não produzem saída, tipicamente você deveria colocar uma linha em branco antes do `@verbatim` e outra linha em branco depois do `@end verbatim`. Linhas em branco entre o `@verbatim` iniciante e o `@end verbatim` finalizante aparecerão na saída gerada.

Você consegue obter um literal “pequeno” colocando o `@verbatim` em um ambiente `@smallformat`, conforme mostrado aqui:

```
@smallformat
@verbatim
... ainda literal, porém em uma fonte menor ...
@end verbatim
@end smallformat
```

Finalmente, uma palavra de alerta: não é confiável usar `@verbatim` dentro de outras construções do Texinfo.

Veja-se também Seção 18.3 [`@verbatiminclude`], Página 149.

7.6 @lisp: Marcando um Exemplo da Lisp

O comando `@lisp` é muito semelhante ao `@example`, porém marca o conteúdo como código Lisp. Por exemplo:

```
@lisp
```



```
Código lisp de exemplo
@end lisp
```

A linguagem de programação também pode ser fornecida como um argumento para o comando `@example` (veja-se Seção 7.4 [`@example`], Página 67), de modo que o exemplo acima também poderia ser escrito como:

```
@example lisp
Código lisp de exemplo
@end example
```

Use `@lisp` para preservar informações relativas à natureza do exemplo. Isso é útil, por exemplo, se você escrever uma função que avalia somente e todo o código Lisp em um arquivo do Texinfo. Então você consegue usar o arquivo do Texinfo como uma biblioteca Lisp.

7.7 `@display`: Exemplos Usando a Fonte de Texto

O comando `@display` inicia outro tipo de ambiente, onde a fonte é deixada inalterada, não comutada para máquina de escrever como com `@example`. Cada linha da entrada ainda produz uma linha de saída, e a saída gerada ainda é recuada.

```
Este é um exemplo de texto escrito entre um comando @display
e um comando @end display. O comando @display
recua o texto, mas não o preenche.
```

7.8 `@format`: Exemplos Usando a Largura Total da Linha

O comando `@format` é similar a `@display`, exceto que deixa o texto sem recuo. Como `@display`, ele não seleciona a fonte de largura fixa. Assim,

```
@format
Este é um exemplo de texto escrito entre um comando @code{@@format}
e um comando @code{@@end format}. Como você pode ver
a partir deste exemplo,
o comando @code{@@format} não preenche o texto.
@end format
```

produz

Este é um exemplo de texto escrito entre um comando `@format`
e um comando `@end format`. Como você pode ver
a partir deste exemplo,
o comando `@format` não preenche o texto.

7.9 `@exdent`: Desfazendo o Recuo de Uma Linha

O comando `@exdent` remove qualquer recuo que uma linha possa ter. O comando é escrito no início de uma linha e se aplica somente ao texto que segue o comando que esteja na mesma linha. Não use chaves em volta do texto. O texto em uma linha `@exdent` também é impresso na fonte romana onde o formato da saída gerada permitir isso.

`@exdent` normalmente é usado dentro de exemplos. Assim,

```
@example
Esta linha segue um comando @@example.
@exdent Esta linha está desrecuada.
Esta linha segue a linha desrecuada.
O @@end example vem na próxima linha.
@end example
```

produz

```

    Esta linha segue um comando @example.
Esta linha está desrecuada.
    Esta linha segue a linha desrecuada.
    O @end example vem na próxima linha.

```

Na prática, o comando `@exdent` raramente é usado. Geralmente, você desrecua texto finalizando o exemplo e retornando a página para a largura normal dela.

`@exdent` não tem efeito em todos os formatos de saída.

7.10 @flushleft e @flushright

Os comandos `@flushleft` e `@flushright` alinham as extremidades das linhas nas margens esquerda e direita de uma página, mas não preenchem o texto. Os comandos são escritos em linhas próprias, sem chaves. Os comandos `@flushleft` e `@flushright` são finalizados pelos comandos `@end flushleft` e `@end flushright` em próprias linhas.

Por exemplo,

```

@flushleft
Este texto está
escrito alinhado à esquerda.
@end flushleft

```

produz

```

Este texto está
escrito alinhado à esquerda.

```

`@flushright` produz o tipo de recuo frequentemente usado no endereço de retorno de cartas. Por exemplo,

```

@flushright
Aqui está um exemplo de texto escrito
alinhado à direita. O comando @code{@flushright}
justifica à direita de cada linha, mas deixa a
extremidade esquerda irregular.
@end flushright

```

produz

```

                                     Aqui está um exemplo de texto escrito
                                     alinhado à direita. O comando @flushright
                                     justifica à direita de cada linha, mas deixa a
                                     extremidade esquerda irregular.

```

7.11 @raggedright: Texto Irregular à Direita

O `@raggedright` preenche texto como de costume, mas o texto só é justificado à esquerda; a margem direita fica irregular. O comando é escrito em uma linha própria, sem chaves. O comando `@raggedright` é finalizado por `@end raggedright` em uma linha própria. Esse comando somente tem efeito em formatos de saída onde o texto seja justificado à esquerda, mas não em formatos de saída onde o texto sempre seja configurado como irregular à direita, como Info ou HTML.

O comando `@raggedright` pode ser útil com parágrafos contendo listas de comandos com nomes longos, quando for sabido antecipadamente que justificar o texto em ambas as margens fará com que o parágrafo aparente ruim.

Um exemplo (originário de outro lugar neste manual):

```
@raggedright
Comandos para aspas de ângulo duplo e único:
@code{@@guillemetleft@{@}}, @code{@@guillemetright@{@}},
@code{@@guillemotleft@{@}}, @code{@@guillemotright@{@}},
@code{@@guilsinglleft@{@}}, @code{@@guilsinglright@{@}}.
@end raggedright
```

produz

Comandos para aspas de ângulo duplo e único: @guillemetleft{}, @guillemetright{}, @guillemotleft{}, @guillemotright{}, @guilsinglleft{}, @guilsinglright{}.

7.12 @noindent: Omitindo Recuo

Um exemplo ou outra inclusão pode quebrar um parágrafo em segmentos. Normalmente, os formataadores recuam o texto que segue um exemplo como um parágrafo novo. Você consegue evitar isso caso a caso escrevendo @noindent no início de uma linha, precedendo o texto de continuação. Você também consegue desabilitar o recuo para todos os parágrafos globalmente com @paragraphindent (veja-se Seção E.3 [paragraphindent], Página 278).

Aqui está um exemplo mostrando como eliminar o recuo normal do texto depois de um @example, uma situação comum:

```
@example
Este é um exemplo
@end example

@noindent
Esta linha não está recuada. Como você pode ver, o
início da linha está alinhado totalmente com a
linha que segue depois dele.
```

produz:

```
Este é um exemplo
```

```
Esta linha não está recuada. Como você pode ver, o
início da linha está alinhado totalmente com a
linha que segue depois dele.
```

O uso padrão do @indent é exatamente como acima: no início do que seria de outro modo um parágrafo, para eliminar o recuo que normalmente acontece ali. Ele ou pode ser seguido por texto ou estar em uma linha própria. Não existe razão para usá-lo em outros contextos, como no meio de um parágrafo ou dentro de um ambiente (veja-se Capítulo 7 [Citações e Exemplos], Página 65).

Você consegue controlar o número de linhas em branco na saída do arquivo Info ajustando a entrada conforme desejado: uma linha contendo apenas @noindent não gera uma linha em branco, e nem uma linha @end para um ambiente.

Não coloque chaves depois de um comando @noindent; elas não usadas, pois @noindent é um comando usado fora de parágrafos (veja-se Seção A.1 [Sintaxe de Comando], Página 221).

7.13 @indent: Forçando Recuo

Para complementar o comando @noindent (veja-se a seção anterior), Texinfo fornece o comando @indent para forçar que um parágrafo seja recuado. Por exemplo, este parágrafo (o primeiro nesta seção) está recuado usando um comando @indent.

E, de fato, o primeiro parágrafo de uma seção é o lugar mais provável para se usar `@indent`, para substituir o comportamento normal de não recuo ali (veja-se Seção E.3 [`@paragraphindent`], Página 278). Ele ou pode ser seguido por texto ou estar em uma linha própria.

Como um caso especial, quando `@indent` for usado em um ambiente onde o texto não estiver preenchido, ele produz um espaço de recuo de parágrafo na saída $\text{\TeX}$. (Esses ambientes são onde uma linha de entrada produz uma linha de saída, como `@example` e `@display`; para um resumo de todos os ambientes, veja-se Seção 7.1 [Comandos de Cercamento de Blocos], Página 65).

Não coloque chaves depois de um comando `@indent`; elas não são usadas, pois `@indent` é um comando usado fora de parágrafos (veja-se Seção A.1 [Sintaxe de Comando], Página 221).

7.14 `@cartouche`: Retângulos Arredondados

Onde o formato da saída gerada permitir, o comando `@cartouche` desenhará uma caixa com cantos arredondados ao redor do conteúdo dele. Você consegue usar esse comando para isolar uma parte do manual do fluxo principal. Você também consegue realçar ainda mais um exemplo ou citação com `@cartouche`.

Por exemplo, você poderia escrever um manual no qual um tipo de exemplo esteja envolvido por um `@cartouche` para ênfase. Por exemplo,

```
@cartouche
@example
% pwd
/usr/local/share/emacs
@end example
@end cartouche
```

envolve o exemplo de duas linhas com uma caixa com cantos arredondados, no manual impresso.

A saída gerada oriunda do exemplo se parece com isto (se você estiver lendo isto no Info, você verá que o `@cartouche` não teve efeito):

```
% pwd
/usr/local/share/emacs
```

`@cartouche` recebe um argumento opcional, fornecido no restante da linha. Esse texto, se presente, é o título do `@cartouche`. Ele é gerado em negrito ou, de outra maneira, enfatizado no início do `@cartouche` e é centralizado em alguns formatos de saída.

O seguinte exemplo ilustra um `@cartouche` com um título:

```
@cartouche Importante
Texto explicando algo importante fora do fluxo
principal do texto.
@end cartouche
```

O `@cartouche` com um título se parece com isto:

```
Important

Texto explicando algo importante fora do fluxo principal do texto.
```

Um `@cartouche` é gerado em uma página na saída gerada impressa, semelhante ao `@group` (veja-se Seção 13.9 [`@group`], Página 112).

7.15 @small... Comandos de Bloco

Adicionalmente ao @example regular e comandos similares, Texinfo tem comandos “small” estilo exemplo. Esses são @smallquotation, @smallindentedblock, @smallldisplay, @smallexample, @smallformat e @smalllisp.

Na maioria dos formatos de saída gerada, os comandos @small... são equivalentes aos comandos complementares deles não “small”.

Na saída gerada impressa, entretanto, os comandos @small... tipografam texto em uma fonte menor que os comandos de exemplo não “small”. Assim, por exemplo, exemplos de código podem conter linhas mais longas e ainda caberem em uma página sem a necessidade de serem reescritos.

Marque o fim de um bloco @small... com um correspondente @end small.... Por exemplo, emparelhe @smallexample com @end smallexample.

Aqui está um exemplo da fonte usada pelo comando @smallexample (na maioria dos formatos de saída, a saída gerada será a mesma como de costume):

```
... para ter certeza de que você tem a liberdade de
distribuir cópias de software livre (e cobrar por
esse serviço se você desejar), que você receba o código
fonte ou consiga obtê-lo se quiser, que você pode
mudar o software ou usar partes dele em novos
programas livres; e que você sabe que você pode fazer essas coisas.
```

Os comandos @small... usam o mesmo estilo de fonte que as contrapartes normais deles: @smallexample e @smalllisp usam uma fonte de largura fixa, e os demais usam a fonte regular. Eles também tem o mesmo comportamento em outros aspectos—quando o preenchimento é feito e quando as margens são estreitadas. Entretanto, eles não pegam argumentos, mesmo quando os contrapartes deles pegarem.

Como regra geral, é melhor usar somente os comandos regulares (como @example em vez de @smallexample), a menos que você tenha um bom motivo para isso. Ao processar somente com texinfo.tex, você pode obter um efeito semelhante com ‘@set dispenvsize small’, que usa uma fonte menor para @example e outros ambientes. Na saída HTML, você pode usar CSS para mudar o tamanho da fonte dos ambientes, se necessário; veja-se Seção 22.3 [CSS do HTML], Página 193.

8 Listas e Tabelas

Texinfo tem várias maneiras de construir listas e tabelas. As listas podem ser marcadas ou numeradas; tabelas de duas colunas podem realçar os itens na primeira coluna; tabelas multi colunas também são suportadas.

8.1 Listas Introducentes

Texinfo automaticamente recua o texto em listas ou tabelas, e numera uma lista numerada. Esse último recurso é útil se você modificar a lista, pois não precisa renumerá-la você mesmo(a).

Listas numeradas e tabelas iniciam com o comando `@` apropriado no início de uma linha, e finalizam com o correspondente comando `@end` em uma linha própria. Os comandos de tabela e de lista de itens também exigem que você escreva informações de formatação na mesma linha do comando `@` de inicialização.

Inicie uma lista numerada, por exemplo, com um comando `@enumerate` e finalize a lista com um comando `@end enumerate`. Inicie uma lista de itens com um comando `@itemize`, seguido na mesma linha por um comando de formatação como `@bullet`, e finalize a lista com um comando `@end itemize`.

Preceda cada elemento de uma lista com um comando `@item` ou `@itemx`.

Aqui está uma lista de itens dos diferentes tipos de tabela e listas:

- Listas de itens com e sem marcações.
- Listas numeradas, usando números ou letras.
- Tabelas de duas colunas com realçamento.

Aqui está uma lista numerada com os mesmos itens:

1. Listas de itens com e sem marcações.
2. Listas numeradas, usando números ou letras.
3. Tabelas de duas colunas com realçamento.

E aqui está uma tabela de duas colunas com os mesmos itens e os comandos `@` dela:

`@itemize` Listas de itens com e sem marcações.

`@enumerate`

Listas numeradas, usando números ou letras.

`@table`

`@ftable`

`@vtable` Tabelas de duas colunas, opcionalmente com indexação.

8.2 @itemize: Construindo Uma Lista de Itens

O comando `@itemize` produz uma sequência de “itens”, cada um iniciando com uma bola ou outra marca dentro da margem esquerda, e geralmente recuados.

Inicie uma lista de itens escrevendo `@itemize` no início de uma linha. Siga o comando, na mesma linha, com um caractere ou um comando do Texinfo para gerar uma marca. Geralmente, você usará `@bullet` depois do `@itemize`, mas você pode usar `@minus`, ou qualquer outro comando ou caractere do Texinfo. Se você não especificar um comando de marca, o padrão é `@bullet`.

Depois do `@itemize`, escreva seus itens, cada um iniciando com `@item`. O texto pode seguir na mesma linha que `@item`. O texto de um item pode continuar por mais que um parágrafo.

Aqui está um exemplo do uso de `@itemize`:

```
@itemize @bullet{}
@item
  Algum texto para foo.

@item
  Algum texto
  para bar.
@end itemize
```

Isso produz:

- Algum texto para foo.
- Algum texto para bar.

Quando fornecer um comando de marca como `@bullet` como o argumento para o comando `@itemize`, você pode omitir o ‘{ }’ que normalmente seguiria o comando.

Se você não quiser nenhuma marca, mas ainda quiser itens lógicos, use `@w{ }` (nesse caso, os colchetes são exigidos).

Deveria existir ao menos um `@item` dentro do ambiente `@itemize`. Se nenhum estiver presente, `texi2any` fornecerá um alerta. Se você quiser apenas texto recuado e não uma lista de itens, use `@indentedblock`; veja-se Seção 7.3 [`@indentedblock`], Página 67.

As entradas de índice que ocorrem imediatamente antes de um `@item` são associadas com o `@item`.

Geralmente, você deveria colocar uma linha em branco entre os itens. Isso coloca uma linha em branco no arquivo Info. (O `TeX` insere o espaço vertical apropriado em qualquer caso). Exceto quando as entradas forem muito breves, esses espaços em branco fazem a lista aparentar melhor.

Listas de itens podem ser embutidas em outras listas de itens. Aqui está uma lista marcada com traços embutida em uma lista marcada com bolas:

```
@itemize @bullet{}
@item
  Primeiro item.

@itemize @minus{}
@item
  Item interno.

@item
  Segundo item interno.
@end itemize

@item
  Segundo item externo.
@end itemize
```

Isso produz:

- Primeiro item.
 - Item interno.
 - Segundo item interno.
- Segundo item externo.

8.3 @enumerate: Fazendo Uma Lista Numerada ou Com Letras

`@enumerate` é como `@itemize` (veja-se Seção 8.2 [`@itemize`], Página 74), exceto que os rótulos nos itens são inteiros ou letras sucessivos em vez de bolas.

Escreva o comando `@enumerate` no início de uma linha. O comando não exige um argumento, mas aceita ou um número ou uma letra como uma opção. Sem um argumento, `@enumerate` inicia a lista com o número ‘1’. Com um argumento numérico, como ‘3’, o comando inicia a lista com aquele número. Com uma letra maiúscula ou minúscula, como ‘a’ ou ‘A’, o comando inicia a lista com aquela letra.

Escreva o texto da lista numerada da mesma maneira que uma lista de itens: escreva a linha iniciando com `@item` no início de cada item na enumeração. Está correto ter texto seguindo o `@item`, e o texto para um item pode continuar por vários parágrafos.

Você deveria colocar uma linha em branco entre as entradas na lista. Isso geralmente torna mais fácil ler o arquivo Info.

Aqui está um exemplo do `@enumerate` sem um argumento:

```
@enumerate
@item
Causas subjacentes.

@item
Causas imediatas.
@end enumerate
```

Isso produz:

1. Causas subjacentes.
2. Causas imediatas.

Aqui está um exemplo com um argumento de 3:

```
@enumerate 3
@item
Causas predisponentes.

@item
Causas precipitantes.

@item
Causas perpetuantes.
@end enumerate
```

Isso produz:

3. Causas predisponentes.
4. Causas precipitantes.
5. Causas perpetuantes.

Aqui está um sumário:

a. `@enumerate`

Sem um argumento, produz uma lista numerada, com o primeiro item numerado 1.

b. `@enumerate inteiro-dessinalado`

Com um argumento numérico (dessinalado), inicia uma lista numerada com aquele número. Você pode usar isso para continuar uma lista que você interrompeu com outro texto.

c. `@enumerate letra-maiúscula`

Com uma letra maiúscula como argumento, inicia uma lista na qual cada item é marcado por uma letra, iniciando com aquela letra maiúscula.

d. `@enumerate letra-minúscula`

Com uma letra minúscula como argumento, inicia uma lista na qual cada item é marcado por uma letra, iniciando com aquela letra minúscula.

Você também pode aninhar listas enumeradas, como em uma estrutura de tópicos.

8.4 Fazendo Uma Tabela de Duas Colunas

`@table` é similar a `@itemize` (veja-se Seção 8.2 [`@itemize`], Página 74), mas permite que você especifique um nome ou uma linha de título para cada item. O comando `@table` é usado para produzir tabelas de duas colunas, e é especialmente útil para glossários, exibições explicativas, e resumos de opções de linha de comando.

8.4.1 Usando o Comando `@table`

Use o comando `@table` para produzir uma tabela de duas colunas. Esse comando tipicamente é usado quando você tem uma lista de itens e um texto breve com cada um, como uma lista de definições.

Escreva o comando `@table` no início de uma linha, depois de uma linha em branco, e siga-o, na mesma linha, com um argumento que seja um comando “indicante”, como `@code`, `@samp`, `@var`, `@option`, ou `@kbd` (veja-se Seção 6.1 [Indicando], Página 54). Esse comando será aplicado ao texto na primeira coluna. Por exemplo, `@table @code` fará com que o texto na primeira coluna seja gerado como se ele tivesse sido o argumento para um comando `@code`.

Você pode usar o comando `@asis` como um argumento para `@table`. `@asis` é um comando que não faz nada: se você usar esse comando depois de `@table`, as entradas da primeira coluna serão geradas sem realçamento adicionado (“como está”).

O comando `@table` funciona com outros comandos além daqueles mencionados explicitamente aqui. Entretanto, você só pode usar comandos predefinidos do Texinfo que recebam um argumento entre chaves. Você não pode confiavelmente usar um comando novo definido com `@macro`, embora um `@alias` (para um comando predefinido adequado) seja aceitável. Veja-se Capítulo 17 [Definindo Novos Comandos do Texinfo], Página 138.

Inicie cada entrada da tabela com um comando `@item` no início de uma linha. Escreva o texto para a primeira coluna na mesma linha que o comando `@item`. Escreva o texto para a segunda coluna na linha seguinte à linha do `@item` e em linhas subsequentes. Você pode escrever quantas linhas de texto de apoio desejar, até mesmo vários parágrafos. Mas, somente o texto na mesma linha que `@item` será colocado na primeira coluna (incluindo quaisquer notas de rodapé). Você não precisa digitar nada para uma segunda coluna vazia.

Normalmente, você deveria colocar uma linha em branco entre entradas de tabela. Isso coloca uma linha em branco no arquivo Info, o que aparenta melhor, a menos que as entradas sejam muito breves.

Finalize a tabela com uma linha consistindo de `@end table`. Siga o fim da tabela por uma linha em branco para formatação consistente entre formatos de saída.

Por exemplo, a tabela seguinte realça o texto na primeira coluna com o comando `@samp`:

```
@table @samp
@item foo
Este é o texto para
@samp{foo}.

@item bar
Texto para @samp{bar}.
@end table
```

Isso produz:

```
'foo'      Este é o texto para 'foo'.
'bar'      Texto para 'bar'.
```

Se você quiser listar dois ou mais itens nominados com um bloco de texto, use o comando `@itemx`. (Veja-se Seção 8.4.3 [`@itemx`], Página 78).

O comando `@table` (veja-se Seção 8.4.1 [`@table`], Página 77) não é suportado dentro do `@display`. Como `@display` é orientado a linha, não faz sentido usá-los juntos. Se você deseja recuar uma tabela, tente `@quotation` (veja-se Seção 7.2 [`@quotation`], Página 66) ou `@indentedblock` (veja-se Seção 7.3 [`@indentedblock`], Página 67).

8.4.2 `@ftable` e `@vtable`

Os comandos `@ftable` e `@vtable` são os mesmos que o comando `@table`, exceto que `@ftable` automaticamente insere cada um dos itens na primeira coluna da tabela no índice de funções e `@vtable` automaticamente insere cada um dos itens na primeira coluna da tabela no índice de variáveis. Isso simplifica a tarefa de criar índices. Somente os itens na mesma linha que os comandos `@item` ou `@itemx` são indexados, e são indexados exatamente na forma em que aparecem nessa linha. Veja-se Capítulo 10 [Índices], Página 88, para mais informações acerca de índices.

Inicie uma tabela de duas colunas usando `@ftable` ou `@vtable` escrevendo o comando `@` no início de uma linha, seguido na mesma linha por um argumento que seja um comando do Texinfo, como `@code`, exatamente como você faria para um comando `@table`; e termine a tabela com um comando `@end ftable` ou `@end vtable` em uma linha própria.

Veja-se o exemplo para `@table` na seção anterior.

8.4.3 `@itemx`: Segundo e Itens Subsequentes

Use o comando `@itemx` dentro de uma tabela quando você tiver duas ou mais entradas de primeira coluna para o mesmo item, cada uma das quais deveria aparecer em uma linha própria.

Use `@item` para a primeira entrada, e `@itemx` para todas as entradas subsequentes; `@itemx` sempre precisa seguir um comando `@item`, sem linhas em branco entre eles.

O comando `@itemx` funciona exatamente como `@item`, exceto que não gera espaço vertical extra acima do texto da primeira coluna em alguns formatos de saída. Se você tiver múltiplos comandos `@itemx` consecutivos, não insira quaisquer linhas em branco entre eles.

Por exemplo,

```
@table @code
@item upcase
@itemx downcase
Essas duas funções aceitam um caractere ou uma sequência de caracteres
como argumento, e retornam o correspondente caractere ou a sequência de
caracteres escrito em letras maiúsculas (minúsculas).
@end table
```

Isso produz:

`upcase`

`downcase` Essas duas funções aceitam um caractere ou uma sequência de caracteres como argumento, e retornam o correspondente caractere ou a sequência de caracteres escrito em letras maiúsculas (minúsculas).

(Observe também que esse exemplo ilustra texto de apoio multi linha em uma tabela de duas colunas).

8.5 @multitable: Tabelas Multi Colunas

`@multitable` permite que você construa tabelas com qualquer número de colunas, com cada coluna tendo qualquer largura que você goste.

Você define as larguras da coluna em uma linha `@multitable` própria, e escreve cada linha da tabela real seguindo um comando `@item`, com colunas separadas por um comando `@tab`. Finalmente, `@end multitable` completa a tabela. Detalhes nas seções abaixo.

8.5.1 Larguras de Colunas de Multi Tabelas

Você pode definir as larguras de coluna para uma multi tabela de duas maneiras: como frações do comprimento da linha; ou com uma linha de protótipo. Mesclagem dos dois métodos não é suportada. Em qualquer caso, as larguras são definidas inteiramente na mesma linha que o comando `@multitable`.

1. Para especificar larguras de coluna como frações do comprimento de linha, escreva `@columnfractions` e os números decimais (presumivelmente menor que 1; um zero no início é permitido e ignorado) depois do comando `@multitable`, como em:

```
@multitable @columnfractions .33 .33 .33
```

As frações não precisam totalizar exatamente 1.0, como essas não fazem. Isso permite que você produza tabelas que não precisam do comprimento total da linha.

Ao usar `@columnfractions`, a coluna mais à esquerda pode parecer um pouco mais larga que você esperaria, relativa às outras colunas. Isso é devido ao espaçamento entre colunas estar incluído na largura das outras colunas.

2. Para especificar uma linha de protótipo, escreva a entrada mais longa para cada coluna entre chaves depois do comando `@multitable`. Por exemplo:

```
@multitable {algum texto para a coluna um} {para a coluna dois}
```

A primeira coluna terá então a largura do tipografado ‘algum texto para a coluna um’ e a segunda coluna a largura de ‘para a coluna dois’.

As entradas de protótipo não precisam aparecer na tabela em si.

Apesar que nós usamos texto simples nesse exemplo, as entradas de protótipo podem conter comandos do Texinfo; comandos de marcação, como `@code`, são particularmente prováveis de serem úteis.

Linhas de protótipo não tem efeito na saída HTML.

8.5.2 Linhas de Multi Tabelas

Depois do comando `@multitable` definindo as larguras de colunas (veja-se a seção anterior), você inicia cada linha no corpo de uma multi tabela com `@item`, e separa as entradas de coluna com `@tab`. Quebras de linha não são especiais dentro do corpo da tabela, e você pode quebrar linhas de entrada em teu arquivo fonte se necessário.

Você também pode usar `@headitem` em vez de `@item` para produzir uma *linha de cabeçalho*. A saída do T_EX para tal linha é em negrito, e a saída HTML e DocBook usa a etiqueta `<thead>`. No Info, a linha de cabeçalho é seguida por uma linha separadora feita de traços (caracteres ‘-’).

O comando `@headitemfont` pode ser usado em modelos quando as entradas em uma linha `@headitem` precisarem ser usadas em um modelo. É um sinônimo para `@b`, mas usar `@headitemfont` evita qualquer dependência daquele estilo específico de fonte, no caso de nós fornecermos uma maneira de mudá-lo no futuro.

Aqui está um exemplo completo de uma tabela multi colunas (o texto é originário de *O Manual do GNU Emacs*, veja-se Seção “Splitting Windows” em *O Manual do GNU Emacs*):

```
@multitable @columnfractions .15 .45 .4
@headitem Tecla @tab Comando @tab Descrição
@item C-x 2
@tab @code{split-window-vertically}
@tab Divide a janela selecionada em duas janelas,
com uma acima da outra.
@item C-x 3
@tab @code{split-window-horizontally}
@tab Divide a janela selecionada em duas janelas
posicionadas lado a lado.
@item C-Mouse-2
@tab
@tab Na linha de modo ou barra de rolagem de uma janela,
divide aquela janela.
@end multitable
```

produz:

Tecla	Comando	Descrição
C-x 2	<code>split-window-vertically</code>	Divide a janela selecionada em duas janelas, com uma acima da outra.
C-x 3	<code>split-window-horizontally</code>	Divide a janela selecionada em duas janelas posicionadas lado a lado.
C-Mouse-2		Na linha de modo ou barra de rolagem de uma janela, divide aquela janela.

9 Exibições Especiais

Os comandos neste capítulo permitem que você escreva texto que seja especialmente exibido (permissão de formato de saída), fora do fluxo normal do documento.

Um conjunto de tais comandos é para criar “flutuações”, isto é, figuras, tabelas, e afins, configuradas a partir do texto principal, possivelmente numeradas, legendadas, e (ou) referenciadas a partir de outro lugar no documento. As imagens frequentemente são incluídas nessas exibições.

Outro grupo de comandos é para criar notas de rodapé no Texinfo.

9.1 Flutuadores

Uma *flutuação* é uma exibição que é configurada a partir do texto principal. É tipicamente rotulada como sendo uma “Figura”, “Tabela”, “Exemplo” ou algum tipo similar.

Uma flutuação é assim nomeada porque, a princípio, ela pode ser movida para o final ou topo da página atual, ou para uma página seguinte, na saída impressa. (Flutuação não faz sentido em outros formatos de saída). Em cada formato de saída, exceto para $\text{\LaTeX}$, no entanto, essa flutuação infelizmente ainda não está implementada. Em vez disso, o material flutuante simplesmente é gerado no local atual, mais ou menos como se fosse um `@group` (veja-se Seção 13.9 [`@group`], Página 112).

9.1.1 `@float` [*tipo*][*rótulo*]: Material Flutuante

Para produzir material flutuante, circunde o material que você quer que seja exibido separado entre os comandos `@float` e `@end float`, em suas linhas próprias.

Material flutuante frequentemente usa `@image` para exibir um gráfico já existente (veja-se Seção 9.2 [Imagens], Página 83), ou `@multitable` para exibir uma tabela (veja-se Seção 8.5 [Tabelas Multi Colunas], Página 79). Entretanto, o conteúdo da flutuação pode ser qualquer coisa. Aqui está um exemplo com texto simples:

```
@float Figura,fig:ex1
Esta é uma flutuação de exemplo.
@end float
```

E a saída gerada:

Este é um flutuador de exemplo.

Figure 9.1

Como mostrado no exemplo, `@float` recebe dois argumentos (separados por uma vírgula), *tipo* e *rótulo*. Ambos são opcionais.

tipo Especifica a classe da flutuação; tipicamente uma palavra como “Figura”, “Tabela”, etc. Se não for fornecida, e *rótulo* for, quaisquer referências cruzadas simplesmente usarão um número despojado.

rótulo Especifica um rótulo de referência cruzada para essa flutuação. Se fornecido, a essa flutuação será automaticamente fornecido um número, e aparecerá em qualquer saída `@listoffloats` (veja-se Seção 9.1.3 [`@listoffloats`], Página 82). Referências cruzadas para *rótulo* são permitidas. Por exemplo, ‘veja-se `@ref{fig:ex1}`’ produzirá veja-se Figure 9.1.

Por outro lado, se *rótulo* não for fornecido, então a flutuação não será numerada e conseqüentemente não aparecerá na saída `@listoffloats` ou será referenciável cruzadamente.

Ordinariamente, você especifica ambos, *tipo* e *rótulo*, para obter um flutuador rotulado e numerado.

Na saída do L^AT_EX, o código que carrega o pacote `float` é gerado no preâmbulo se `@float` estiver presente. Um `@float` com o tipo `'figure'` ou `'table'` (sem distinção de maiúsculas e minúsculas) já está definido pelo pacote. Outros tipos de flutuação levam para a definição de um novo ambiente de flutuação, com nomes baseados no tipo de `@float`, com nada mais que letras e `'-'` removido.

No Texinfo, todas as flutuações são numeradas da mesma maneira: com o número do capítulo (ou letra do apêndice), um ponto, e o número da flutuação, o qual simplesmente conta 1, 2, 3, ..., e é zerado a cada capítulo. Cada tipo de flutuação é contado independentemente.

Flutuadores dentro de um `@unnumbered`, ou fora de qualquer capítulo, simplesmente são numeradas consecutivamente a partir de 1.

Essas convenções de numeração não são, atualmente, mudáveis.

9.1.2 `@caption` e `@shortcaption`

Você pode escrever um `@caption` em qualquer lugar dentro de um ambiente `@float`, para definir uma legenda para a flutuação. Não é permitido em qualquer outro contexto. `@caption` recebe um argumento, circundado por chaves. Aqui está um exemplo:

```
@float
Um exemplo de flutuação, com legenda.
@caption{Legenda para flutuação de exemplo.}
@end float
```

A saída gerada é:

Uma flutuação de exemplo, com legenda.

Legenda para flutuação de exemplo.

`@caption` pode aparecer em qualquer lugar dentro da flutuação; ele não é processado até o `@end float`. O texto da legenda geralmente é uma frase ou duas, porém pode consistir de vários parágrafos se necessário.

Na saída gerada, a legenda sempre aparece abaixo da flutuação; isso atualmente não é mudável. Ela é precedida pelo tipo e (ou) número da flutuação, conforme especificado para o comando `@float` (veja-se a seção anterior).

O comando `@shortcaption` igualmente pode ser usado somente dentro de `@float`, e recebe um argumento entre chaves. O texto curto da legenda é usado em vez do texto da legenda em uma lista de flutuações (veja-se a próxima seção). Assim, você pode escrever uma legenda longa para o documento principal, e um título curto para aparecer na lista de flutuações. Por exemplo:

```
@float
... conforme acima ...
@shortcaption{Texto para lista de flutuações.}
@end float
```

O texto para `@shortcaption` pode não conter comentários (`@c`), texto literal (`@verb`), ambientes como `@example`, notas de rodapé (`@footnote`), parágrafos múltiplos ou outras construções complexas. As mesmas restrições se aplicam ao `@caption`, a menos que exista um `@shortcaption`. No L^AT_EX, um `@caption` multi parágrafos levará a um erro quando formatado, a menos que exista um `@shortcaption`.

9.1.3 `@listoffloats`: Sumários para Flutuadores

Você pode escrever um comando `@listoffloats` para gerar uma lista de flutuações para um dado tipo de flutuação (veja-se Seção 9.1.1 [`@float`], Página 81), análogo ao sumário geral

do documento. Tipicamente, ele é escrito no nó `@unnumbered` dele próprio para fornecer um cabeçalho e estrutura, mais como `@printindex` (veja-se Seção 10.5 [Imprimindo Índices e Menus], Página 91).

`@listoffloats` recebe um argumento opcional, o tipo de flutuação. Aqui está exemplo:

```
@node Lista de Figuras
@unnumbered Lista de Figuras
@listoffloats Figura
```

Sem nenhum argumento, `@listoffloats` gera uma lista de flutuações para os quais nenhum tipo de flutuação foi especificado, ou seja, nenhum primeiro argumento para o comando `@float` (veja-se Seção 9.1.1 [`@float`], Página 81).

Aqui está com o que se parece a saída gerada a partir de `@listoffloats`, dada a figura de exemplo anterior neste capítulo:

Figure 9.1: 81

Geralmente, cada linha na lista de flutuações contém o tipo de flutuação (se existir), o número da flutuação, e a legenda, se existir—o argumento `@shortcaption`, se foi especificado, do contrário o argumento `@caption`. O número da página também pode ser incluído, dependendo do formato da saída gerada.

Flutuadores não numerados (aquelas sem rótulos de referência cruzada) são omitidos da lista de flutuações.

A formatação de `@listoffloats` depende do formato da saída gerada. No Info, por exemplo, um `@listoffloats` é formatado como um menu.

Na saída $\text{\LaTeX}$, `\listoffigures` é gerado para o tipo de flutuação ‘figure’ (sem distinção entre maiúsculas e minúsculas), `\listoftables` é gerado para o tipo de flutuação ‘table’ (sem distinção entre maiúsculas e minúsculas). Para outros tipos de flutuação, um comando `\listof` específico é gerado.

9.2 Inserindo Imagens

Você pode inserir uma imagem fornecida em um arquivo externo com o comando `@image`. Embora imagens possam ser usadas em qualquer lugar, incluindo o meio de um parágrafo, nós as descrevemos neste capítulo, pois elas são mais frequentemente parte de uma figura ou exemplo exibida.

9.2.1 Sintaxe da Imagem

Aqui está a sinopse do comando `@image`:

```
@image{nomearquivo[, largura[, altura[, alttexto[, extensão]]]]}
```

O argumento *nomearquivo* é obrigatório, e precisa não ter uma extensão, pois os diferentes processadores suportam formatos diferentes:

- $\text{\TeX}$ (saída DVI) lê o arquivo *nomearquivo.eps* (Formato encapsulado PostScript).
- $\text{\TeX}$ (saída PDF) tenta (na ordem) *nomearquivo.pdf*, *nomearquivo.png*, *nomearquivo.jpg* e *nomearquivo.jpeg*. Ele também tenta as versões em letras maiúsculas das extensões. O formato PDF não suporta imagens EPS, de modo que essas precisam ser convertidas primeiro.
- No Info, *nomearquivo.txt* é incluído literalmente (mais ou menos como se estivesse em `@verbatim`). A saída Info também pode incluir uma referência para *nomearquivo.png* ou *nomearquivo.jpg*. (Veja-se abaixo).
- Em HTML, uma referência para *nomearquivo.png*, *nomearquivo.jpg*, *nomearquivo.jpeg* ou *nomearquivo.gif* (nessa ordem) é gerada. Se nenhum desses existir, fornece um erro, e gera uma referência para *nomearquivo.jpg* de qualquer maneira.

- Em DocBook, todos os seguintes são verificados: *nomearquivo.eps*, *nomearquivo.gif*, *nomearquivo.jpeg*, *nomearquivo.jpg*, *nomearquivo.pdf*, *nomearquivo.png* e *nomearquivo.svg*. Referências são geradas para cada arquivo encontrado. Também, *nomearquivo.txt* é incluído literalmente, se presente. (Supõe-se que o processador DocBook subsequente escolha o adequado).
- Para L^AT_EX, *nomearquivo* sem nenhuma extensão é usado; supõe-se que o processador L^AT_EX subsequente escolha o tipo apropriado de imagem.
- Para saída Info e HTML, o quinto argumento opcional *extensão* para **@image** é usado para a extensão de arquivo, se ele for especificado e o arquivo for encontrado. Qualquer ponto inicial deveria ser incluído em *extensão*. Por exemplo:

```
@image{foo,,,.xpm}
```

Se você quiser instalar arquivos de imagens para uso por leitores Info também, nós recomendamos colocá-los em um subdiretório como ‘*foo-figuras*’ para um pacote *foo*. A cópia dos arquivos para \$(infodir)/*foo-figuras*/ deveria ser feita em teu Makefile.

Os argumentos *largura* e *altura* estão descritos na próxima seção.

Se uma imagem for a primeira coisa em um parágrafo e seguida por mais texto, então você deveria preceder o comando **@image** por **@indent** ou **@noindent** para indicar o início da formatação do parágrafo. Isso é especialmente importante para saída T_EX para obter o recuo correto do parágrafo.

Use **@center** para centralizar uma imagem (veja-se Seção 2.8.2.2 [**@titlefont @center @sp**], Página 20).

Para saída HTML, o *atributo alt* para imagens inline é configurado para o (quarto) argumento opcional *alttexto* para **@image**, se fornecido. Se não fornecido, o nome completo de arquivo da imagem sendo exibida será usado. O *alttexto* é processado como texto do Texinfo, de forma que caracteres especiais, tais como ‘”’ e ‘<’ e ‘&’, são escapados na saída gerada HTML; também, você pode obter uma sequência de caracteres *alt* vazia com **@-** (um comando que não produz saída; veja-se Seção 13.3 [**@- @hyphenation**], Página 111).

Para saída Info, a sequência *alttexto* também é processada como texto e saída do Texinfo. Nesse caso, ‘\’ é escapada como ‘\\’ e ‘”’ como ‘\”’; outros escapes não são feitos.

Na saída Info, uma referência para o arquivo binário da imagem é escrita (tentando *nomearquivo* sufixado com *extensão*, *.extensão*, *.png*, ou *.jpg*, nessa ordem) se um existir. O arquivo *.txt* também é incluído literalmente, se um existir. Dessa maneira, leitores Info que consigam exibir imagens (tais como o navegador Info do Emacs, executando sob X) podem fazer isso, ao passo que leitores Info que somente consigam usar texto (tais como o leitor autônomo Info) conseguem exibir a versão textual.

9.2.2 Escalonamento da Imagem

Os argumentos opcionais *largura* e *altura* para o comando **@image** (veja-se a seção anterior) especificam o tamanho para o qual escalonar a imagem. Eles somente são levados em consideração na saída impressa.

No T_EX, se nenhum for especificado, a imagem será apresentada no tamanho natural dela (fornecido no arquivo); se somente um for especificado, o outro será escalonado proporcionalmente; e se ambos forem especificados, ambos serão respeitados, portanto provavelmente distorcendo a imagem original mudando a proporção de aspecto dela.

A *largura* e a *altura* podem ser especificadas usando qualquer dimensão válida do T_EX, a saber:

pt	ponto (72.27pt = 1in)
pc	pica (1pc = 12pt)

bp	ponto grande (72bp = 1in)
in	Polegada
cm	centímetro (2.54cm = 1in)
mm	milímetro (10mm = 1cm)
dd	ponto de didôt (1157dd = 1238pt)
cc	cicero (1cc = 12dd)
sp	ponto escalonado (65536sp = 1pt)

Por exemplo, o seguinte dimensionará um arquivo `ridt.eps` para uma polegada verticalmente, com a largura dimensionada proporcionalmente:

```
@image{ridt,,1in}
```

Para `@image` funcionar com $\text{\TeX}$, o arquivo `epsf.tex` precisa ser instalado em algum lugar onde o $\text{\TeX}$ consiga encontrá-lo. (O local padrão é `texmf/tex/generic/dvips/epsf.tex`, onde `texmf` é a raiz da árvore de diretórios do teu $\text{\TeX}$). Esse arquivo está incluído na distribuição Texinfo e também está disponível a partir de `ftp://tug.org/tex/epsf.tex`, entre outros lugares.

`@image` pode ser usado dentro de uma linha bem como para figuras exibidas. Portanto, se você pretende que ele seja exibido, assegure-se de deixar uma linha em branco antes do comando, ou a saída gerada executará no texto anterior.

Dimensionamento de imagem atualmente está implementado somente na saída impressa, não em qualquer outro tipo de saída gerada.

9.3 Notas de Rodapé

Uma *nota de rodapé* serve para uma referência que documenta ou elucida o texto principal.¹

As notas de rodapé distraem; use-as com moderação, no máximo, e é melhor evitá-las completamente. As referências bibliográficas padrão geralmente ficam melhor situadas em uma bibliografia, ao final de um documento, que em notas de rodapé ao longo do texto.

9.3.1 Comandos de Notas de Rodapé

No Texinfo, as notas de rodapé são criadas com o comando `@footnote`. Esse comando é seguido imediatamente por uma chave esquerda, depois pelo texto da nota de rodapé e por uma chave direita finalizante. As notas de rodapé podem ser de qualquer comprimento (elas serão quebradas entre páginas, se necessário), mas geralmente são curtas. O modelo é:

```
texto principal@footnote{texto da nota de rodapé}
```

Conforme mostrado aqui, o comando `@footnote` deveria vir logo depois do texto sendo anotado, sem espaço intermediário; caso contrário, o marcador de nota de rodapé poderá acabar iniciando uma linha.

Por exemplo, esta cláusula é seguida por uma nota de rodapé de amostra²; no fonte do Texinfo, se parece com isto:

```
...uma de nota de rodapé de amostra@footnote{Aqui está a
nota de rodapé de amostra.}; no fonte do Texinfo...
```

Como você pode ver, esse fonte inclui dois sinais de pontuação próximos um do outro; nesse caso, `‘.};’` é a sequência. Isso é normal (o primeiro termina a nota de rodapé e o segundo

¹ Uma nota de rodapé deveria complementar ou expandir o texto principal, mas um(a) leitor(a) não deveria precisar ler uma nota de rodapé para entender o texto principal. Para uma discussão completa das notas de rodapé, veja-se *The Chicago Manual of Style*, publicado pela University of Chicago Press.

² Aqui está a nota de rodapé de amostra.

pertence à frase sendo anotada), de forma que não se preocupe que pareça estranho. (Outro estilo, perfeitamente aceitável, é o de colocar a nota de rodapé depois da pontuação pertencente à frase, como em ‘;@footnote{...}’).

Em formatos de saída impressa, a marca de referência para uma nota de rodapé é um número pequeno e sobrescrito; o texto da nota de rodapé aparece na parte inferior da página, abaixo de uma linha horizontal.

No Info, a marca de referência para uma nota de rodapé é um par de parênteses com o número da nota de rodapé entre eles, assim: ‘(1)’. A marca de referência é seguida por um linque de referência cruzada para o texto da nota de rodapé se as notas de rodapé forem colocadas em nós separados (veja-se Seção 9.3.2 [Estilos de Notas de Rodapé], Página 86).

Na saída gerada HTML, as referências de notas de rodapé geralmente são marcadas com um número pequeno e sobrescrito que é renderizado como um linque de hipertexto para o texto da nota de rodapé.

As notas de rodapé não podem ser aninhadas e não podem aparecer em títulos de seção de qualquer tipo ou em outros locais “incomuns”.

Uma dica final: as notas de rodapé no argumento de um comando @item para um @table precisam estar inteiramente na mesma linha que o @item (como de costume). Veja-se Seção 8.4 [Tabelas de Duas Colunas], Página 77.

9.3.2 Estilos de Notas de Rodapé

Formatos online tem dois estilos de nota de rodapé, que determinam onde o texto da nota de rodapé está localizado, o estilo de notas de rodapé ‘end’ e o ‘separate’.

‘end’ Para Info, no estilo de nó ‘end’, todas as notas de rodapé para um nó estão situadas no final desse nó. As notas de rodapé são separadas do resto do nó por uma linha de traços com a palavra ‘Notas de rodapé’ dentro dela. Cada nota de rodapé começa com uma marca de referência ‘(n)’.

Aqui está um exemplo da saída Info para uma nota de rodapé no estilo de fim de nó:

```
----- Notas de rodapé -----
```

```
(1) Aqui está uma nota de rodapé de amostra.
```

Em HTML, quando o estilo da nota de rodapé for ‘end’, ou se a saída não for dividida, as notas de rodapé são colocadas no final de cada arquivo de saída.

‘separate’

Para Info, no estilo de nó ‘separate’, todas as notas de rodapé para um nó são colocadas em um nó próprio construído automaticamente. Nesse estilo, uma “referência de nota de rodapé” segue cada marca de referência ‘(n)’ no corpo do nó. A referência da nota de rodapé é, na verdade, uma referência cruzada que você usa para atingir o nó da nota de rodapé.

O nome do nó com as notas de rodapé é construído pospondo-se ‘-Footnotes’ ao nome do nó que contém as notas de rodapé. (Consequentemente, o nó das notas de rodapé para o nó Footnotes é Footnotes-Footnotes)! O nó das notas de rodapé tem um ponteiro de nó ‘Acima’ que leva de volta para o nó ancestral dele.

Aqui está como a primeira nota de rodapé neste manual aparenta depois de ser formatada para o Info no estilo de nó separar:

```
Arquivo: texinfo.info  Nó: Visão Geral-Notas de rodapé, Acima: Visão Geral
```

```
(1) A primeira sílaba de "Texinfo" é pronunciada como
"speck", não "hex". ...
```

Em HTML, quando o estilo da nota de rodapé for ‘**separate**’ e a saída for dividida, as notas de rodapé são colocadas em um arquivo separado.

A menos que teu documento tenha notas de rodapé longas e importantes (como em, por diga-se, *Decline and Fall* . . . de Gibbon), nós recomendamos o estilo ‘**end**’, pois ele é mais simples para leitores(as) seguirem.

Use o comando `@footnotestyle` para especificar o estilo de nota de rodapé. Escreva esse comando no início de uma linha seguido por um argumento, seja ‘**end**’ para o estilo de nó fim ou ‘**separate**’ para o estilo do nó separar.

Por exemplo,

```
@footnotestyle end
```

ou

```
@footnotestyle separate
```

Escreva um comando `@footnotestyle` no preâmbulo do arquivo Texinfo.

10 Índices

Usando o Texinfo, você consegue gerar índices sem ter que ordenar e agrupar as entradas manualmente. Em um índice, as entradas são listadas em ordem alfabética, juntamente com informações acerca de como encontrar a discussão de cada entrada. Em um manual impresso, essas informações consistem de números de páginas. Em outros formatos, links para o local das entradas do índice ou para os nós associados são gerados.

O Texinfo fornece vários tipos predefinidos de índice: um índice para funções, um índice para variáveis, um índice para conceitos e assim por diante. Você pode combinar índices ou usá-los para outros fins que não os canônicos. Por último, você consegue definir seus próprios novos índices.

10.1 Índices Predefinidos

Texinfo fornece seis índices predefinidos. Aqui estão os significados nominais deles, abreviações e os correspondentes comandos de entrada de índice:

<code>'cp'</code>	<code>(@cindex)</code> Índice de conceito, para conceitos gerais.
<code>'fn'</code>	<code>(@findex)</code> Índice de Função, para nomes de funções e semelhantes a funções (como pontos de entrada de bibliotecas).
<code>'ky'</code>	<code>(@kindex)</code> Índice de pressionamento de tecla, para comandos de teclado.
<code>'pg'</code>	<code>(@pindex)</code> Índice de programas, para nomes de programas.
<code>'tp'</code>	<code>(@tindex)</code> Índice de tipo de dados, para nomes de tipos (como estruturas definidas em arquivos de cabeçalho).
<code>'vr'</code>	<code>(@vindex)</code> Índice de variáveis, para nomes de variáveis (como variáveis globais de bibliotecas).

Nem todo manual precisa de tudo isso, e a maioria dos manuais usa somente dois ou três, no máximo. O presente manual, por exemplo, tem três índices: um índice de conceitos, um índice de variáveis um índice de comandos @. (O último na verdade é o índice de função, mas é chamado de índice de comandos no cabeçalho de capítulo).

Você não é obrigado(a) a usar os índices predefinidos estritamente para os fins canônicos deles. Por exemplo, suponha que você deseje indexar algumas macros do préprocessador C. Você poderia colocá-las no índice de função junto com as funções reais, apenas escrevendo comandos `@findex` para elas; então, quando imprimir o “Índice de Função” como um capítulo não numerado, você poderia dar a ele o título de ‘Índice de Função e Macro’ e tudo seria consistente para o(a) leitor(a).

Por outro lado, é melhor não se afastar muito do significado dos índices predefinidos. Do contrário, caso o teu texto seja combinado com outro texto proveniente de outros manuais, as entradas de índice não corresponderão. Em vez disso, defina teu próprio novo índice (veja-se Seção 10.7 [Novos Índices], Página 93).

Nós recomendamos ter um índice no documento final sempre que possível, independentemente de quantos índices de fonte você usar, pois assim os(as) leitores(as) terão somente um lugar para procurar. Dois ou mais índices de fonte podem ser combinados em um índice de saída gerada, usando-se os comandos `@synindex` ou `@syncodeindex` (veja-se Seção 10.6 [Combinando Índices], Página 92).

Comandos de entrada de índice baseados nos nomes de índice de duas letras também são válidos, por exemplo, `@cpindex` pode ser usado em vez de `@cindex`.

10.2 Definindo as Entradas de um Índice

Os dados para criar um índice vem de muitos comandos individuais de indexação espalhados pelo arquivo fonte do Texinfo. Cada comando diz para adicionar uma entrada para um índice específico; depois da formatação, o índice fornecerá o número da página atual ou o nome do nó como a referência.

Uma entrada de índice consiste de um comando de indexação no início de uma linha seguido, no restante da linha, pela entrada.

Por exemplo, esta seção começa com as seguintes cinco entradas para o índice de conceito:

```
@cindex Definindo entradas de indexação
@cindex Entradas de índice, definindo
@cindex Entradas para um índice
@cindex Especificando entradas de índice
@cindex Criando entradas de índice
```

Cada índice predefinido tem o próprio comando dele de indexação—`@cindex` para o índice de conceito, `@findex` para o índice de função e assim por diante, conforme listado na seção anterior.

As entradas do índice deveriam preceder o material visível que estiver sendo indexado. Por exemplo:

```
@cindex alô
Alô, você!
```

Entre outros motivos, dessa maneira links de indexação seguintes (em qualquer contexto) acabam antes do material, onde os(as) leitores(as) querem estar, em vez de depois.

No Info, um índice geralmente é formatado como um menu. Tente evitar usar dois pontos nas entradas de índice, pois isso pode confundir alguns leitores Info. Veja-se Seção 3.9.4 [Partes do Menu], Página 35, para mais informações acerca da estrutura de uma entrada de menu.

Por padrão, as entradas para um índice de conceito são impressas em uma fonte romana e as entradas para os outros índices são impressas em uma fonte de máquina de escrever. Você pode mudar a maneira como parte de uma entrada é impressa com os comandos usuais do Texinfo, como `@file` para nomes de arquivos (veja-se Capítulo 6 [Marcando Texto], Página 54) e `@r` para a fonte normal romana (veja-se Seção 6.2.3 [Fontes], Página 63).

Você pode especificar uma chave explícita de ordenação para uma entrada de índice usando `@sortas` seguindo ou o comando de índice ou o texto da entrada. Por exemplo: `'@findex @sortas{\} \ @r{(literal \ em @code{@@math})}'` ordena a entrada de índice que isso produz sob barra invertida.

Você pode escolher ignorar determinados caracteres nas entradas de índice para fins de ordenação. Os caracteres que você pode, atualmente, escolher ignorar são `'\'`, `'-'`, `'<'` e `'@'`, que são ignorados fornecendo-se como um argumento para o comando `@set`, respectivamente, `txiindexbackslashignore`, `txiindexhyphenignore`, `txiindexlessthanignore` e `txiindexatsignignore`. Por exemplo, especificar `'@set txiindexbackslashignore'` faz com que a entrada `'\mathop{sup}'` no índice para este manual seja ordenada como se fosse `'mathop{sup}'`, de forma que ela apareça entre as outras entradas começando com `'M'`. Isso evita a necessidade de fornecer chaves explícitas de ordenação para entradas de índice contendo esses caracteres.

Ao usar essas opções, é possível obter uma entrada de índice com uma chave vazia de ordenação. Para evitar isso, especifique uma diretiva `@sortas` na entrada de índice. Por exemplo:

```
@set txiindexbackslashignore
@findex @sortas{\} \
```

10.3 Comandos Avançados de Indexação

O Texinfo fornece vários outros comandos para indexação.

Primeiro, você pode criar entradas de índice *multi nível*, permitindo agrupar muitos subtópicos relacionados sob o mesmo tópico de nível superior. Você faz isso separando as partes de cada entrada com o comando `@subentry`. Esses comandos podem se parecer com isto:

```
@cindex Superhumanos(as) @subentry vilões(ãs)
@cindex Superhumanos(as) @subentry heróis(ínas)
```

Você pode ter até três níveis em uma entrada:

```
@cindex cafeteiras @subentry elétricas @subentry rosa
@cindex cafeteiras @subentry elétricas @subentry azul
```

Você pode usar o comando `@sortas` mencionado anteriormente com qualquer uma ou todas as três partes de uma entrada para fazer com que elas ordenem diferentemente do que fariam por padrão.

Segundo, você pode fornecer uma entrada de índice que aponte para outra, usando o comando `@seeentry` (“veja-se entrada”). Por exemplo:

```
@cindex Índices @seeentry{Expoentes}
```

Essa entrada deveria ser única no teu documento; a ideia é a de redirecionar o(a) leitor(a) para a outra entrada, onde eles(as) encontrarão todas as informações que procuram.

Finalmente, você pode fornecer uma entrada “veja-se também” usando o comando `@seealso`. Essas entradas acompanham as entradas regulares e são agrupadas com elas no índice impresso final. Por exemplo:

```
@cindex Café
@cindex Café @subentry Com leite e açúcar
@cindex Café @subentry Com donuts
@cindex Café @subentry Descafeinado
@cindex Café @seealso{Chá}
```

Ao usar todos esses três comandos avançados, *não* coloque uma vírgula entre as diferentes partes do texto do índice. O programa `texindex`, que ordena as entradas do índice e gera os comandos de formatação de indexação, se encarrega de colocar as vírgulas nos lugares corretos para você.

Não interrompa um índice ou uma entrada `@subentry` com `@sortas` ou comandos avançados com chaves; coloque os comandos com chaves no início ou no final da entrada.

Esses recursos são mais úteis com documentos impressos e ao se traduzir Texinfo para DocBook.

10.4 Criando Entradas de Índice

As entradas do índice de conceito consistem de texto. A melhor maneira de escrever um índice é a de elaborar entradas que sejam concisas, mas claras. Se você conseguir fazer isso, o índice geralmente parecerá melhor se as entradas forem escritas exatamente como apareceriam no meio de uma frase, ou seja, capitalizando somente nomes próprios e siglas que sempre exijam letras maiúsculas. Essa é uma convenção de maiúsculas e minúsculas que nós usamos na maioria dos índices dos manuais GNU.

Se você não sabe como tornar uma entrada concisa, mas clara, torne-a mais longa e clara—não concisa e confusa. Se muitas das entradas tiverem várias palavras, o índice pode parecer melhor se você usar uma convenção diferente: maiusculizar a primeira palavra de cada entrada. Qualquer que seja a convenção de maiúsculas e minúsculas que você usar, use-a de consistentemente.

Em qualquer caso, nunca maiusculize um nome que diferencia maiúsculas de minúsculas, como um nome de função da C ou Lisp ou um comando de shell; isso seria um erro ortográfico.

Entradas em índices diferentes do índice de conceito são nomes de símbolos em linguagens de programação ou nomes de programas; esses nomes geralmente diferenciam maiúsculas de minúsculas, de forma que, igualmente, use letras maiúsculas e minúsculas conforme exigido.

É uma boa ideia tornar as entradas do índice únicas sempre que possível. Dessa forma, as pessoas que usarem a saída gerada impressa ou a completção online das entradas do índice não enxergarão listas indiferenciadas. Considere essa uma oportunidade para tornar mais específicas entradas de índice idênticas, de forma que os(as) leitores(as) consigam encontrar mais facilmente o lugar exato que procuram. Os recursos avançados de indexação descritos em Seção 10.3 [Indexação Avançada], Página 90, podem ajudar com isso também.

Quando você estiver criando entradas de índice, é uma boa prática pensar nas diferentes maneiras pelas quais as pessoas podem procurar algo. Pessoas diferentes *não* pensam nas mesmas palavras quando procuram algo. Um índice útil terá itens indexados sob todas as palavras diferentes que as pessoas possam usar. Por exemplo, um(a) leitor(a) pode pensar que é óbvio que os nomes de duas letras para índices deveriam ser listados sob “Índices, nomes de duas letras”, uma vez que “Índices” são o conceito geral. Todavia, outro(a) leitor(a) pode lembrar-se do conceito específico de nomes de duas letras e procurar a entrada listada como “Nomes de duas letras para índices”. Um bom índice terá ambas as entradas e ajudará ambos(as) os(as) leitores(as).

Como tipografar, a construção de um índice é uma arte especializada, cujas sutilezas possivelmente não sejam apreciadas até que você precise fazê-la você mesmo(a).

índice no Info.

10.5 Imprimindo Índices e Menus

`@printindex` recebe um argumento, uma abreviação de duas letras do índice. Você precisa incluir o comando `@printindex` no local do documento onde quiser que o índice apareça. Isso não acontece automaticamente só porque você usa `@cindex` ou outros comandos geradores de entradas de índice no arquivo Texinfo; esses somente fazem com que os dados brutos para o índice sejam acumulados.

Você deveria preceder o comando `@printindex` com um comando adequado de seção ou capítulo (geralmente `@appendix` ou `@unnumbered`) para fornecer o título do capítulo e inserir o índice no sumário. Preceda o título do capítulo com uma linha `@node`, como de costume.

Por exemplo:

```
@node Índice Variável
@unnumbered Índice Variável

@printindex vr

@node Índice de Conceito
@unnumbered Índice de Conceito

@printindex cp
```

O texto ‘Index’ precisa aparecer no nome do nó que contém o índice para que o índice seja encontrado por leitores Info.

Se você tiver mais que um índice, nós recomendamos colocar o índice de conceito por último.

Outros detalhes da saída do índice em formatos de saída:

- Como parte do processo de criação de um manual impresso com o TeX, você executa um programa chamado `texindex` (veja-se Capítulo 19 [Impresso com TeX], Página 150) para ordenar os dados brutos para produzir um arquivo de índice ordenado. O arquivo de índice ordenado é o que, na verdade, é usado para imprimir o índice.

`@printindex` lê o correspondente arquivo de índice ordenado e produz um índice tradicional de duas colunas, com termos de índice e números de página.

- Na saída Info, `@printindex` produz um menu especial contendo o número da linha da entrada, relativo ao início do nó. Leitores Info podem usar isso para ir para a linha exata de uma entrada, não apenas para o nó que a contém. (Leitores Info mais antigos irão apenas para o nó). Aqui está um exemplo:

* Primeira entrada de índice: Topo. (linha 7)

- Na saída de texto simples, a formatação de `@printindex` geralmente é semelhante a um menu no Info, mostrando o número da linha de cada entrada relativa ao início do arquivo.
- Na saída HTML, `@printindex` produz links para as entradas do índice.
- Na saída DocBook e L^AT_EX, ele simplesmente registra o índice a ser impresso.

10.6 Combinando Índices

Às vezes você irá querer combinar dois índices, como funções e conceitos, talvez porque tenha poucas entradas suficientes que um índice separado pareceria bobo.

Você poderia colocar funções no índice de conceito escrevendo comandos `@cindex` para elas em vez de comandos `@findex` e produzir um manual consistente imprimindo o índice de conceito com o título ‘Índice de Função e Conceito’ e não imprimir o ‘Índice de Função’ em absoluto; mas esse não é um procedimento robusto. Ele funciona somente se o teu documento nunca for incluído como parte de outro documento que seja projetado para ter um índice de função separado; se o teu documento fosse incluído em tal documento, as funções oriundas do teu documento e aquelas oriundas do outro não terminariam juntas. Além disso, para fazer com que os teus nomes de funções apareçam na fonte correta no índice de conceito, você precisaria circundar cada uma delas entre chaves de `@code`.

para o índice mesclado.

10.6.1 `@syncindex`: Combinando Índices Usando `@code`

Quando quiser combinar funções e conceitos em um índice, você deveria indexar as funções com `@findex` e indexar os conceitos com `@cindex` e usar o comando `@syncindex` para redirecionar as entradas do índice de função para o índice de conceito.

O comando `@syncindex` recebe dois argumentos; eles são o nome do índice para redirecionar e o nome do índice para o qual redirecioná-lo. O modelo se parece com isto:

`@syncindex de para`

Para esse propósito, para os índices são fornecidos nomes de duas letras:

<code>‘cp’</code>	Índice de conceito
<code>‘fn’</code>	Índice de função
<code>‘ky’</code>	Índice de tecla
<code>‘pg’</code>	Índice de programa
<code>‘tp’</code>	Índice de tipo de dados
<code>‘vr’</code>	Índice de variável

Escreva um comando `@syncindex` antes ou logo depois da linha de fim do cabeçalho no início de um arquivo Texinfo. Por exemplo, para mesclar um índice de função com um índice de conceito, escreva o seguinte:

`@syncindex fn cp`

Isso fará com que todas as entradas designadas para o índice de função sejam mescladas com o índice de conceito.

Para mesclar ambos, um índice de variável e um índice de função, em um índice de conceito, escreva o seguinte:

```
@syncodeindex vr cp
@syncodeindex fn cp
```

O comando `@syncodeindex` coloca todas as entradas oriundas do índice ‘de’ (o índice redirecionado) na fonte do `@code`, substituindo qualquer fonte padrão que seja usada pelo índice para o qual as entradas sejam agora direcionadas. Dessa forma, se você direcionar nomes de funções oriundas de um índice de função para um índice de conceito, todos os nomes de funções serão impressos na fonte do `@code` como você esperaria.

índice mesclado.

10.6.2 @synindex: Combinando Índices

O comando `@synindex` é quase o mesmo que o comando `@syncodeindex`, exceto que não coloca as entradas do índice ‘de’ na fonte do `@code`; em vez disso, coloca-os na fonte romana. Assim, você usa `@synindex` ao mesclar um índice de conceito em um índice de função.

Veja-se Seção 10.5 [Imprimindo Índices e Menus], Página 91, para informações acerca de imprimir um índice no final de um livro ou criar um menu de índice em um arquivo Info.

10.7 Definindo Novos Índices

Além dos índices predefinidos (veja-se Seção 10.1 [Índices Predefinidos], Página 88), você pode usar os comandos `@defindex` e `@defcodeindex` para definir novos índices. Esses comandos criam novos comandos `@` de indexação com os quais você marca entradas de índice. O comando `@defindex` é usado assim:

```
@defindex nome
```

Novos nomes de índice geralmente são palavras de duas letras, como ‘au’. Por exemplo:

```
@defindex au
```

Isso define um novo índice, chamado o índice ‘au’. Ao mesmo tempo, ele cria um novo comando de indexação, `@auindex`, que você consegue usar para fazer entradas de índice. Use esse novo comando de indexação exatamente como você usaria um comando de indexação predefinido.

Por exemplo, aqui está um título de seção seguido por uma entrada de índice de conceito e duas entradas de índice ‘au’.

```
@section Semântica Cognitiva
@cindex esquemas de imagem cinestésica
@auindex Johnson, Mark
@auindex Lakoff, George
```

(Evidentemente, ‘au’ aqui serve como uma abreviação para “autor”).

Texinfo constrói o novo comando de indexação concatenando o nome do índice com ‘index’; portanto, definir um índice ‘xy’ leva à criação automática de um comando `@xyindex`.

Use o comando `@printindex` para imprimir o índice, como você faz com índices predefinidos. Por exemplo:

```
@node Índice de Autor
@unnumbered Índice de Autor
```

```
@printindex au
```

O `@defcodeindex` é como o comando `@defindex`, exceto que, na saída impressa, ele imprime entradas em uma fonte do `@code` por padrão, em vez de uma fonte romana.

Você deveria definir novos índices antes da linha de fim de cabeçalho de um arquivo Texinfo e (é claro) antes de quaisquer comandos `@synindex` ou `@syncodeindex` (veja-se Seção 2.5 [Cabeçalho do Arquivo do Texinfo], Página 11).

Conforme mencionado anteriormente (veja-se Seção 10.1 [Índices Predefinidos], Página 88), nós recomendamos ter um índice no documento final sempre que possível (independentemente de quantos índices de fonte você usar), pois assim leitores(as) terão somente um lugar para procurar.

Ao criar um índice, `TeX` cria um arquivo cuja extensão é o nome do índice (veja-se [Nomes dos arquivos de índice], Página 152). Portanto, você deveria evitar usar nomes de índices que colidam com extensões usadas para outros propósitos, tais como `‘.aux’` ou `‘.xml’`. `texi2any` já informa um erro se um novo índice conflitar com um nome de extensão bem conhecido.

11 Inserções Especiais

Texinfo fornece vários comandos para inserir caracteres que tenham significado especial no Texinfo, tais como chaves, e para outros elementos gráficos que não correspondam a caracteres simples que você consiga digitar.

11.1 Caracteres Especiais: Inserindo @ {} , \ # &

‘@’ e chaves são os caracteres especiais básicos no Texinfo. Para inserir esses caracteres de forma que apareçam no texto, você precisa colocar um ‘@’ na frente desses caracteres para evitar que o Texinfo os interprete mal. Comandos alfabéticos também são fornecidos.

Os demais caracteres (vírgula, barra invertida, cerquilha, e comercial) são especiais somente em contextos restritos, conforme explicado nas respectivas seções.

11.1.1 Inserindo ‘@’ com @@ e @atchar{}

@@ produz um caractere ‘@’ na saída gerada. Não coloque chaves depois de um comando @@.

@atchar{} também produz um caractere ‘@’ na saída gerada. Ele precisa de chaves seguintes, como de costume para comandos alfabéticos. Em condicionais inline (veja-se Seção 16.4 [Condicionais Inline], Página 131), pode ser necessário evitar usar o caractere literal ‘@’ no fonte (e possivelmente seja mais claro em outros contextos).

11.1.2 Inserindo ‘{ ‘}’ com @{ @} e @l rbracechar{}

@{ produz um ‘{’ na saída gerada e @} produz um ‘}’. Não coloque chaves depois de um comando @{ ou @}.

@lbracechar{} e @rbracechar{} também produzem caracteres ‘{’ e ‘}’ únicos na saída gerada. Eles precisam de chaves seguintes, como de costume para comandos alfabéticos. Em condicionais inline (veja-se Seção 16.4 [Condicionais Inline], Página 131), pode ser necessário evitar usar caracteres de chaves literais no fonte (e possivelmente seja mais claro em outros contextos).

11.1.3 Inserindo ‘,’ com @comma{}

Normalmente, uma vírgula ‘,’ é um caractere normal que pode ser simplesmente digitado em tua entrada onde você precisar dela.

No entanto, Texinfo usa a vírgula como um caractere especial somente em um contexto: para separar argumentos para aqueles comandos do Texinfo, como @node (veja-se Seção 3.3 [Exigências da Linha do Nó], Página 28), @acronym (veja-se Seção 6.1.14 [acronym], Página 60) e @xref (veja-se Capítulo 5 [Referências Cruzadas], Página 43), bem como macros definidas por usuário(a) (veja-se Seção 17.1 [Definindo Macros], Página 138), que recebem mais que um argumento.

Como um caractere de vírgula confundiria a análise do Texinfo para esses comandos, você precisa usar o comando ‘@comma{’ se quiser passar uma vírgula real. Aqui estão alguns exemplos:

```
@acronym{ABC, Uma @comma{} Bizarra}
@xref{Vírgula,, O símbolo @comma{}}
@mymac{Um argumento@comma{} contendo uma vírgula}
```

Embora ‘@comma{’ possa ser usado em praticamente qualquer lugar, não existe necessidade dele em nenhum lugar, exceto nesse caso incomum.

(Incidentalmente, o nome ‘@comma’ carece do sufixo ‘char’ usado nos comandos complementares dele somente por motivos históricos. Não pareceu importante o suficiente para definir um sinônimo).

11.1.4 Inserindo ‘\’ com @backslashchar{}

Normalmente, uma barra invertida ‘\’ é um caractere normal no Texinfo que pode ser simplesmente digitado em tua entrada onde você precisar. O resultado é o de tipografar a barra invertida oriunda da fonte da máquina de escrever.

No entanto, Texinfo usa a barra invertida como um caractere especial em um contexto restrito: para delimitar argumentos formais nos corpos de macros definidas por usuário(a) (veja-se Seção 17.1 [Definindo Macros], Página 138).

Devido aos caprichos da análise de argumentos de macro, é mais confiável passar um comando alfabético que produza uma barra invertida em vez de usar uma \ literal. Daí @backslashchar{}. Aqui está um exemplo de chamada de macro:

```
@mymac{Um argumento@backslashchar{} com uma barra invertida}
```

Documentos do Texinfo também podem usar \ como um caractere de comando dentro de @math (veja-se Capítulo 12 [Inserindo Fórmulas Matemáticas], Página 109). Nesse caso, @\ ou \backslash produz uma barra invertida “matematicista” (proveniente da fonte do símbolo matemático), enquanto @backslashchar{} produz uma barra invertida de máquina de escrever como sempre.

Embora ‘@backslashchar{}’ possa ser usado em praticamente qualquer lugar, não existe necessidade dele, exceto nesses casos incomuns.

11.1.5 Inserindo ‘#’ com @hashchar{}

Normalmente, uma cerquilha ‘#’ é um caractere normal no Texinfo que pode ser simplesmente digitado em tua entrada onde você precisar. O resultado é o de tipografar o caractere cerquilha oriundo da fonte atual.

Esse caractere tem muitos outros nomes, variando por local, como “sinal numérico”, “libra” e “octothorp”. Às vezes também é chamado de “sustenido” ou “sinal sustenido”, pois ele se assemelha vagamente ao símbolo musical com esse nome. Em situações onde Texinfo seja usado, “cerquilha” é o mais comum em nossa experiência.

No entanto, Texinfo usa o caractere cerquilha como um caractere especial em um contexto restrito: para introduzir a assim chamada diretiva #line e variantes (veja-se Seção 17.7 [Processadores Externos de Macro], Página 146).

Portanto, para a finalidade de tipografar um caractere cerquilha real em tal local (por exemplo, em um programa que precise de documentação acerca de #line), é necessário usar @hashchar{} ou alguma outra construção. Aqui está um exemplo:

```
@hashchar{} 10 "exemplo.c"
```

Embora ‘@hashchar{}’ possa ser usado em praticamente qualquer lugar, não existe necessidade dele em nenhum lugar, exceto nesse caso incomum.

11.1.6 Inserindo ‘&’ com @& e @ampchar{}

Normalmente, um “e” comercial “&” é um caractere comum no Texinfo que pode ser simplesmente digitado na tua entrada, onde você precisar. O resultado é a tipografia do caractere “e” comercial.

No entanto, o caractere “e” comercial é tratado especialmente em apenas um contexto restrito. No argumento para um comando de definição (veja-se Capítulo 14 [Comandos de Definição], Página 114), um “e” comercial seguido por uma série de letras pode ser tipografado especialmente ao se processar com T_EX¹ (veja-se Seção 14.8 [Convenções de Comandos de Definição], Página 123). Por exemplo:

```
@defun foo inteiro1 &optional inteiro2 &rest inteiros
```

¹ Essa sintaxe é usada para palavras-chave da Lisp do Emacs. Veja-se Seção “Uma Descrição de Função de Amostra” em *Manual de Referência da Lisp do GNU Emacs*.

```
@code{foo} descrito aqui.
@end defun
```

pode ter ‘&optional’ e ‘&rest’ formatados em negrito, mesmo sem nenhum comando @ envolvido.

Portanto, para a finalidade de tipografar um “e” comercial nesse contexto, é necessário usar @& ou alguma outra construção. Aqui está um exemplo de uma função C++ que recebe uma referência como parâmetro:

```
@deftypefn Function int foo (const std::vector<int>& @var{bar})
  Documentação de @code{foo}.
@end deftypefn
```

Isso fornece a saída

```
int foo (const std::vector<int>& bar) [Function]
  Documentação de foo.
```

Embora ‘@&’ e ‘@ampchar{’ possam ser usados em quase qualquer lugar, não existe necessidade deles em nenhum lugar, exceto nesse caso incomum.

11.2 Inserindo Caracteres de Aspas

Conforme explicado na seção anterior acerca de convenções gerais de entrada do Texinfo (veja-se Seção 2.1 [Convenções], Página 8), arquivos fonte do Texinfo usam o caractere ASCII ` (decimal 96) para produzir uma aspa esquerda (‘), e o ASCII ' (decimal 39) para produzir uma aspa direita (’). Duplicar esses caracteres de entrada (` e ') produz aspas duplas (“ e ”). Essas são as convenções usadas pelo T_EX.

Em exemplos de código de computador, no entanto, ` e ' produzem renderizações típicas para esses caracteres ASCII: o caractere crase (acento grave autônomo) e aspas simples indireccionadas, respectivamente. No passado, glifos direcionados (‘ e ’) foram usados por padrão na saída do T_EX. O Texinfo fornece esses comandos para escolher entre estas renderizações alternativas:

@codequoteundirected on-off

Configure para ‘off’ para gerar o caractere ' em ambientes de código como aspas simples curvas à direita.

@codequotebacktick on-off

Configure para ‘off’ para gerar o caractere ` em ambientes de código como aspas simples curvas à esquerda.

Se você quiser essas configurações somente para parte do documento, @codequote... on restaurará o comportamento normal, como em @codequoteundirected on.

Essas configurações afetam @code, @example, @kbd, @samp, @verb e @verbatim. Veja-se Seção 6.1.1 [Realçamento Útil], Página 54.

Infelizmente, alguns visualizadores de documentos danificarão os caracteres de aspas direccionadas ao copiar e colar. (O leitor livre de PDF xpdf funciona bem, mas outros leitores de PDF, tanto livres quanto não livres, tem problemas).

Esse recurso também pode ser controlado usando-se @set e @clear sobre as variáveis correspondentes txicodequoteundirected e txicodequotebacktick.

11.3 Inserindo Espaço

As seções seguintes descrevem comandos que controlam espaçamentos de vários tipos dentro e depois de frases.

11.3.1 Espaços Múltiplos

Normalmente, vários caracteres de espaço em branco (espaço, tabulação e nova linha) são recolhidos em um espaço.

Ocasionalmente, você pode querer produzir vários espaços consecutivos, seja para fins de exemplo (por exemplo, o que teu programa faz com vários espaços como entrada), ou meramente para fins de aparência em títulos ou listas. Texinfo suporta três comandos: `@SPACE`, `@TAB` e `@NL`, todos os quais inserem um espaço na saída gerada. (Aqui, `@SPACE` representa um caractere ‘@’ seguido por um espaço, ou seja, ‘@ ’, `TAB` representa um caractere real de tabulação, e `@NL` representa um caractere ‘@’ e fim de linha, ou seja, quando ‘@’ é o último caractere em uma linha).

Por exemplo,

```
Exemplo@ @ @ @
espaçoso.
```

produz

```
Exemplo    espaçoso.
```

Não siga nenhum desses comandos com chaves.

Para produzir um espaço inquebrável, veja-se Seção 13.6 [`@tie`], Página 112.

11.3.2 Não Finalizando uma Frase

Quando um ponto, ponto de exclamação ou ponto de interrogação estiverem no final de uma frase, um pouco mais de espaço é inserido depois deles em um manual tipografado.

Geralmente, Texinfo consegue determinar automaticamente quando um ponto termina uma frase. No entanto, comandos especiais são necessários em algumas circunstâncias. Use o comando `@:` depois de um ponto, ponto de interrogação, ponto de exclamação ou dois pontos que deveriam não ser seguidos por espaço extra. Isso é necessário nas seguintes situações:

1. Depois de um ponto que finalize uma abreviação em minúscula que não esteja no final de uma frase.
2. Quando um comentário entre parênteses no meio de uma frase (como este!) terminar com um ponto, ponto de exclamação ou ponto de interrogação, `@:` deveria ser usado depois do parêntese à direita. Similarmente para colchetes à direita e aspas à direita (ambas, simples e duplas).

Por exemplo:

```
‘foo vs.@: bar (ou?)@: baz’,
```

A primeira linha abaixo mostra a saída gerada e, para comparação, a segunda linha mostra o espaçamento quando os comandos ‘`@:`’ não forem usados.

```
foo vs. bar (ou?) baz
foo vs. bar (ou?) baz
```

Se você olhar cuidadosamente, verá um pouco de espaço não essencial depois de ‘`vs.`’ e ‘`(ou?)`’.

Possivelmente te ajude lembrar o que `@:` faz imaginando que ele representa um caractere minúsculo invisível que impede que uma palavra termine com um ponto final.

Alguns comandos do Texinfo forçam espaçamento normal entre palavras, de forma que você não tenha que inserir `@:` onde, do contrário, faria. Esses são os comandos de realçamento tipo código, `@var`, `@abbr` e `@acronym` (veja-se Seção 6.1.1 [Realçamento Útil], Página 54). Por exemplo, em ‘`@code{foo. bar}`’ o ponto não é considerado ser o fim de uma frase, e nenhum espaço extra é inserido.

`@:` não tem efeito na saída HTML ou DocBook.

11.3.3 Finalizando uma Frase

Como mencionado acima, Texinfo normalmente insere espaço adicional depois do fim de uma frase. Ele usa a mesma heurística para isso que o $\TeX$: uma frase termina com um ponto final, ponto de exclamação ou ponto de interrogação, seja precedido ou seguido por pontuação opcional de fechamento, e, então, espaço em branco, e *não* precedido por uma letra maiúscula.

Use `@.` em vez de um ponto, `@!` em vez de um ponto de exclamação e `@?` em vez de um ponto de interrogação ao final de uma frase que termine com uma letra maiúscula. Não coloque chaves depois de quaisquer desses comandos. Por exemplo:

Dê para M.I.B. e para M.E.W@. Também, dê para R.J.C@.

Dê para M.I.B. e para M.E.W. Também, dê para R.J.C.

A saída gerada segue. Na saída gerada impressa e no Info, você pode ver o espaço em branco extra desejado depois do ‘W’ na primeira linha.

Dê para M.I.B. e para M.E.W. Também, dê para R.J.C.

Dê para M.I.B. e para M.E.W. Também, dê para R.J.C.

Em outros formatos de saída, `@.` é equivalente a um simples ‘.’; o mesmo vale para `@!` e `@?`.

A “pontuação de fechamento” mencionada acima é definida como um parêntese à direita (‘)’), colchete à direita (‘]’), ou aspa à direita, seja simples ou dupla (‘’ e ‘’’’; as muitas aspas Unicode adicionais à direita possíveis não estão incluídas). Esses caracteres podem ser considerados invisíveis com respeito a se um ponto fornecido finaliza uma frase. (Essa é a mesma regra do $\TeX$). Por exemplo, os pontos em ‘foo.) Bar’ e ‘foo.’ ‘ Bar’ finalizam frases.

Os significados de `@.` e `@.`, etc. no Texinfo são projetados para funcionar bem com os comandos de movimento de frase do Emacs (veja-se Seção “Sentences” em *O Manual do GNU Emacs*). Possivelmente ajude imaginar que o ‘@’ em ‘@.’, etc., é uma letra minúscula invisível ‘a’ que torna uma letra maiúscula antes dela irrelevante para os propósitos de decidir se o ponto final termina a frase.

Alguns comandos do Texinfo não são considerados como sendo uma abreviação, mesmo que possam terminar com uma letra maiúscula quando expandidos, de forma que você não tenha que inserir `@.` e acompanhantes. Notavelmente, esse é o caso para comandos de realçamento tipo código, argumentos `@var` terminando com uma letra maiúscula, `@LaTeX` e `@TeX`. Por exemplo, essa frase terminou com ‘... @code{@TeX}.’; `@.` não foi necessário. Da mesma forma, em ... `@var{NOMEVARIÁVEL}`. Texto o ponto depois de *NOMEVARIÁVEL* termina a frase; não existe necessidade de usar `@.`.

11.3.4 @frenchspacing val: Espaçamento de Controle de Frase

Na tipografia americana, é tradicional e correto colocar espaço extra ao final de uma frase. Esse é o padrão no Texinfo (implementado no Info e na saída gerada impressa; não em outros formatos de saída). Na tipografia francesa (e outras), esse espaço extra é errado; todos os espaços são uniformes.

Portanto, Texinfo fornece o comando `@frenchspacing` para controlar o espaçamento depois da pontuação. Ele lê o resto da linha como argumento, que precisa ser a única palavra ‘on’ ou ‘off’ (sempre essas palavras, independentemente do idioma do documento). Aqui está um exemplo:

`@frenchspacing on`

Isso é texto. Duas frases. Três frases. Espaçamento francês.

`@frenchspacing off`

Isso é texto. Duas frases. Três frases. Espaçamento não francês.

produz:

Isso é texto. Duas frases. Três frases. Espaçamento francês.

Isso é texto. Duas frases. Três frases. Espaçamento não francês.

`@frenchspacing` também afeta a saída gerada depois de `@.`, `@!` e `@?` (veja-se Seção 11.3.3 [Finalizando uma Frase], Página 99).

`@frenchspacing` não tem efeito na saída gerada de HTML ou DocBook.

11.3.5 `@dmn{dimensão}`: Formatar uma Dimensão

Você pode usar o comando `@dmn` para formatar uma dimensão com exatamente espaço suficiente para tipografia adequada inserida na saída gerada impressa. Em outros formatos de saída gerada, os comandos de formatação não inserem espaço algum.

Para usar o comando `@dmn`, escreva o número e depois siga-o imediatamente, sem espaço intermediário, por `@dmn`, e depois pela dimensão entre chaves. Por exemplo,

Papel A4 tem 8,27`@dmn{in}` de largura.

produz

Papel A4 tem 8,27in de largura.

Nem todo mundo usa esse estilo. Algumas pessoas preferem ‘8.27 in.’ ou ‘8.27 inches’. Nesses casos, no entanto, você precisa usar `@tie` (veja-se Seção 13.6 [`@tie`], Página 112) ou `@w` (veja-se Seção 13.5 [`@w`], Página 111) de forma que nenhuma quebra de linha possa ocorrer entre o número e a dimensão. Além disso, se você escrever um ponto depois de uma abreviação dentro de uma frase (como com ‘in.’ acima), você deveria escrever ‘@:’ depois do ponto para evitar inserir espaços em branco extras na saída gerada, como mostrado aqui. Veja-se Seção 11.3.2 [Não Finalizando uma Frase], Página 98.

11.4 Inserindo Acentos

Aqui está uma tabela com os comandos que Texinfo fornece para inserir acentos flutuantes. Eles todos precisam de um argumento, o caractere a acentuar, que pode ou ser fornecido entre chaves como de costume (`@{e}`), ou, como um caso especial, as chaves podem ser omitidas, em cujo caso o argumento é o próximo caractere (`@'e`). Isso é para tornar o fonte o mais conveniente possível para digitar-se e ler-se, já que caracteres acentuados são muito comuns em alguns idiomas.

Se o comando for alfabético, como `@dotaccent`, então precisa existir um espaço entre o nome do comando e o argumento se chaves não forem usadas. Se o comando for não alfabético, como `@'`, então precisa *não* existir um espaço; o argumento é o exato próximo caractere.

Exceção: o argumento para `@tieaccent` precisa estar circundado de chaves (já que são dois caracteres em vez de um).

Na saída Info e de texto simples, as construções de acento são geradas como caracteres acentuados verdadeiros se a codificação do documento suportar os caracteres exigidos, a menos que a opção `--disable-encoding` seja fornecida para o `texi2any` (veja-se Seção 15.2 [`@documentencoding`], Página 127). Translitterações ASCII são usadas se os caracteres codificados não forem gerados.

Comando	Saída gerada	O quê
<code>@"o</code>	ö	acento trema
<code>@'o</code>	ó	acento agudo
<code>@,{c}</code>	ç	acento cedilha
<code>@=o</code>	ō	acento macron/sobre barra
<code>@^o</code>	ô	acento circunflexo
<code>@'o</code>	ò	acento grave
<code>@~o</code>	õ	acento til
<code>@dotaccent{o}</code>	ó	acento sobre ponto
<code>@H{o}</code>	ő	trema longo húngaro

<code>@ogonek{a}</code>	ą	ogonek
<code>@ringaccent{o}</code>	ô	acento de anel
<code>@tieaccent{oo}</code>	ôo	acento de ligação posterior
<code>@u{o}</code>	ö	acento breve
<code>@ubaraccent{o}</code>	ō	acento sub barra
<code>@udotaccent{o}</code>	ȯ	acento sub ponto
<code>@v{o}</code>	ǎ	acento caron/hacek/check

Esta tabela lista os comandos do Texinfo para inserir outros caracteres comumente usados em idiomas diferentes do inglês.

<code>@exclamdown{}</code>	¡	! de cabeça para baixo
<code>@questiondown{}</code>	¿	? de cabeça para baixo
<code>@aa{}</code> <code>@AA{}</code>	å Å	a,A com círculo
<code>@ae{}</code> <code>@AE{}</code>	æ Æ	ligaduras de ae,AE
<code>@dh{}</code> <code>@DH{}</code>	ð Ð	eth islandês
<code>@dotless{i}</code>	ı	i sem ponto
<code>@dotless{j}</code>	ĵ	j sem ponto
<code>@l{}</code> <code>@L{}</code>	ł Ł	L,l suprimido
<code>@o{}</code> <code>@O{}</code>	ø Ø	O,o com barra
<code>@oe{}</code> <code>@OE{}</code>	œ Œ	ligaduras de oe,OE
<code>@ordf{}</code> <code>@ordm{}</code>	ª º	Ordinais espanhóis
<code>@ss{}</code>	ß	es-zet ou sharp S
<code>@th{}</code> <code>@TH{}</code>	þ Þ	thorn islandês

11.5 Inserindo Aspas

Use caracteres de aspas simples duplicadas para iniciar e terminar citações: ‘ ‘...’ ’. Duas aspas simples são convertidas para aspas duplicadas à esquerda e à direita, “assim”.

Ocasionalmente, você pode precisar produzir duas aspas simples consecutivas; por exemplo, ao documentar uma linguagem de computador como Maxima, onde ‘ ’’ é um comando válido. Você pode fazer isso com a entrada ‘@w{ }’; o comando vazio @w interrompe a combinação nos caracteres de aspas duplas.

O caractere de aspa à esquerda (‘, código ASCII 96) usado no Texinfo é um acento grave nos padrões do conjunto de caracteres ANSI e ISO. Nós o usamos como um caractere de aspas porque assim é como TeX está configurado, por padrão.

Texinfo suporta várias outras marcas de citação usadas em idiomas diferentes do inglês. Abaixo está uma tabela com os comandos que Texinfo fornece para inserir marcas de citação.

Comando	Glifo	Nome Unicode (ponto)
<code>@quotedblleft{}</code>	``	Aspa dupla à esquerda (U+201C)
<code>@quotedblright{}</code>	''	Aspa dupla à direita (U+201D)
<code>@quoteleft{}</code>	`	Aspa simples à esquerda (U+2018)
<code>@quoteright{}</code>	'	Aspa simples à direita (U+2019)
<code>@quotedblbase{}</code>	„	Aspa dupla baixo 9 (U+201E)
<code>@quotesinglbase{}</code>	,	Aspa simples baixo 9 (U+201A)
<code>@guillemetleft{}</code>	«	Aspa dupla angular apontando para a esquerda (U+00AB)

<code>@guillemetright{}</code>	»	Aspa dupla angular apontando para a direita (U+00BB)
<code>@guilsinglleft{}</code>	<	Aspa simples angular apontando para a esquerda (U+2039)
<code>@guilsinglright{}</code>	>	Aspa simples angular apontando para a direita (U+203A)

Para as aspas duplas angulares, nomes de glifos da Adobe e do L^AT_EX também são suportados: `@guillemotleft` e `@guillemotright`. Esses nomes estão incorretos; um “guillemot” é uma espécie de pássaro (um tipo de auk).

As fontes padrão do T_EX suportam as aspas usuais usadas em inglês (aquelas produzidas com aspas simples de ASCII unitárias e duplicadas). Para as outras aspas, T_EX usa fontes European Computer Modern (EC) (`ecrm1000` e outras variantes). Essas fontes estão disponíveis livremente, é claro; você consegue baixá-las a partir de <http://ctan.org/pkg/ec>, entre outros lugares.

As fontes livres EC são fontes bitmap criadas com Metafont. Especialmente para visualização online, as versões Type 1 (vetor) das fontes são preferíveis; elas estão disponíveis no pacote de fontes CM-Super (<http://ctan.org/pkg/cm-super>).

Ambas as distribuições incluem instruções de instalação.

As tradições para uso de aspas variam muito entre os idiomas (https://en.wikipedia.org/wiki/Quotation_mark). Texinfo não fornece comandos ou configurações para tipografar aspas de acordo com as numerosas tradições. Portanto, você tem que escolher os comandos apropriados para o idioma do teu manual. Às vezes, apelidos (veja-se Seção 17.4 [`@alias`], Página 144) conseguem simplificar o uso e tornar o código-fonte mais legível. Por exemplo, em alemão, `@quotedblbase` é usado para a aspa dupla à esquerda, e a aspa dupla à direita é o glifo produzido por `@quotedblleft`, o que é contraintuitivo. Assim, nesse caso, os seguintes apelidos seriam convenientes:

```
@alias lgqq = quotedblbase
@alias rgqq = quotedblleft
```

11.6 @sub e @sup: Inserindo Subscritos e Sobrescritos

Você consegue inserir subscritos e sobrescritos com os comandos `@sub` e `@sup`. Por exemplo:

```
aqui@sub{abaixo}@sup{acima}
```

produz:

```
aquiabaixoacima
```

No Info e texto simples, `@sub{texto}` atualmente é gerado como ‘`_{texto}`’ e `@sup{texto}` como ‘`^{texto}`’, incluindo as chaves literais (para marcar o início e o fim do texto de “roteiro” para o(a) leitor(a)).

Quando o formato da saída gerada (e o programa de exibição) permite (saída impressa, HTML), o sobrescrito é configurado acima do subscrito quando ambos os comandos forem fornecidos consecutivamente.

Para subscritos e sobrescritos em expressões matemáticas, é melhor usar os caracteres ‘`_`’ e ‘`^`’ do T_EX. Veja-se a próxima seção.

11.7 Glifos para Texto

Texinfo tem suporte para alguns glifos adicionais que são comumente usados em texto impresso, mas não disponíveis em ASCII. Claro, existem muitos milhares mais. É possível usar caracteres Unicode como estão no que diz respeito ao `texi2any`, mas o T_EX não é tão sortudo.

11.7.1 `@TeX{} (TEX)` e `@LaTeX{} (LATEX)`

Use o comando `@TeX{} para gerar ‘TEX’`. Em um manual impresso, esse é um logotipo especial que é diferente das três letras comuns. Em outros formatos de saída, apenas se parece com ‘T_EX’.

Similarmente, use o comando `@LaTeX{} para gerar ‘LATEX’`, que é ainda mais especial em manuais impressos (e diferente do incorreto `LaTeX{} para gerar ‘LaTeX’`. Em outros formatos de saída, o resultado é apenas ‘LaTeX’.

A grafia desses comandos é incomum no Texinfo, pois eles usam ambas, letras maiúsculas e minúsculas.

11.7.2 `@copyright{} (©)`

Use o comando `@copyright{} para gerar o símbolo de copirraite, ‘©’`. Onde possível, esse é um ‘c’ dentro de um círculo; do contrário, isso é ‘(C)’.

Juridicamente, não é necessário usar o símbolo de copirraite; a palavra em inglês ‘Copyright’ é suficiente, de acordo com tratado internacional.

11.7.3 `@registeredsymbol{} (®)`

Use o comando `@registeredsymbol{} para gerar o símbolo de registrado(a), ‘®’`. Onde possível, esse é um ‘R’ dentro de um círculo; do contrário, isso é ‘(R)’.

11.7.4 `@dots (...)` e `@enddots (...)`

Uma *reticência* (uma sequência de pontos) seria espaçada incorretamente quando tipografada como uma sequência de pontos, de forma que um comando especial é usado no Texinfo: use o comando `@dots{} para gerar uma reticência normal, que são três pontos em uma linha, apropriadamente espaçados ... assim. Para enfatizar: não escreva simplesmente três pontos no arquivo de entrada; isso poderia funcionar em alguns formatos de saída, mas produziria a quantidade errada de espaço entre os pontos em manuais impressos.`

O comando `@enddots{} gera reticências de final da frase, que também tem três pontos, mas com espaçamento diferente depois, ... Observe atentamente para ver a diferença.`

Aqui está uma reticência: ... Aqui estão três pontos em uma linha: ...

Na saída gerada impressa (e geralmente em HTML), os três pontos em uma linha estão muito mais próximos um do outro que os pontos nas reticências.

11.7.5 `@bullet (●)`

Use o comando `@bullet{} para gerar um ponto redondo grande, ou a coisa mais próxima possível de um. Um asterisco também pode ser usado. Aqui está um marcador: ●`

11.7.6 `@euro (€)`: Símbolo da Moeda Euro

Use o comando `@euro{} para gerar ‘€’`. Onde possível, esse é o símbolo para a moeda Euro. Caso contrário, a palavra ‘Euro’ é usada.

O símbolo do Euro não existe nas fontes padrão do T_EX (que foram projetadas antes da existência jurídica do Euro). Portanto, o T_EX usa uma fonte adicional, chamada `feymr10` (junto com outras variáveis). Ela está disponível livremente, é claro; você consegue baixá-la a partir de <http://ctan.org/pkg/eurosym>, entre outros lugares. A distribuição inclui instruções de instalação.

11.7.7 `@pounds (£)`: Libras Esterlinas

Use o comando `@pounds{} para gerar ‘£’`. Onde possível, esse é o símbolo para a moeda britânica Libra Esterlina.

11.7.8 @textdegree (°): Símbolo de Graus

Use o comando `@textdegree{}` para gerar ‘°’. Onde possível, esse é o símbolo normal para Graus.

11.7.9 @minus (−): Inserindo um Sinal de Menos

Use o comando `@minus{}` para gerar um sinal de menos. Onde a codificação de caractere e a fonte usada na saída gerada permitirem, o símbolo tem o comprimento habitual para um sinal de menos—um pouco mais longo que um hífen, mais curto que um travessão:

‘—’ é um sinal de menos gerado com ‘`@minus{}`’,

‘-’ é um hífen gerado com o caractere ‘-’,

‘—’ é um travessão para texto.

Se você realmente quiser tipografar alguma matemática que faça uma subtração, é melhor usar `@math`, como em `@math{a-b}` (veja-se Capítulo 12 [Inserindo Fórmulas Matemáticas], Página 109).

11.7.10 @geq (≥) e @leq (≤): Inserindo Relações

Use os comandos `@geq{}` e `@leq{}` para gerar sinais de maior que ou igual a e menor que ou igual a, ‘≥’ e ‘≤’. Quando esses símbolos não estiverem disponíveis, as sequências ASCII ‘>=’ e ‘<=’ são geradas.

erros, etc.

11.8 Glifos para Programação

No Texinfo, o código frequentemente é ilustrado em exemplos que são delimitados por `@example` e `@end example`. Em tais exemplos, você pode indicar os resultados da avaliação ou de uma expansão usando ‘⇒’ ou ‘↦’. Da mesma forma, existem comandos para inserir glifos para indicar saída gerada impressa, mensagens de erro, equivalência de expressões, o local do ponto em um editor e sequências de operação da GUI.

Os comandos de inserção de glifos não precisam ser usados dentro de um exemplo, mas na maioria das vezes são. Todos os comandos de inserção de glifos são seguidos por chaves vazias.

11.8.1 Sumário de Glifos

Aqui está um resumo dos comandos de glifo:

⇒	<code>@result{}</code> indica o resultado de uma expressão.
↦	<code>@expansion{}</code> indica os resultados de uma expansão de macro.
⊢	<code>@print{}</code> indica saída impressa.
error	<code>@error{}</code> indica que o texto seguinte é uma mensagem de erro.
≡	<code>@equiv{}</code> indica a equivalência exata de duas formas.
★	<code>@point{}</code> mostra o local do ponto.
A → B	<code>@clicksequence{A @click{}</code> B indica uma sequência de operação da GUI: primeiro A, depois clicar em B, ou escolher B a partir de um menu, ou então selecioná-lo.

11.8.2 @result{ } (⇒): Resultado de uma Expressão

Use o comando `@result{}` para indicar o resultado da avaliação de uma expressão.

O comando `@result{}` é exibido como ‘⇒’, ou uma seta com haste dupla ou (quando essa não estiver disponível) como a sequência ASCII ‘=>’.

Assim, o seguinte,

```
(cdr '(1 2 3))
⇒ (2 3)
```

pode ser lido como “(cdr '(1 2 3)) avalia para (2 3)”.

11.8.3 @expansion{} (⇒): Indicando uma Expansão

Quando uma expressão for uma chamada de macro, ela se expande para uma nova expressão. Você pode indicar o resultado da expansão com o comando @expansion{}.

O comando @expansion{} é exibido como ‘⇒’, ou uma seta longa com uma base plana ou (quando essa não estiver disponível) como a sequência ASCII ‘==>’.

Por exemplo, o seguinte

```
@example lisp
(third '(a b c))
  @expansion{} (car (cdr (cdr '(a b c))))
  @result{} c
@end example
```

produz

```
(third '(a b c))
⇒ (car (cdr (cdr '(a b c))))
⇒ c
```

que pode ser lido como:

(third '(a b c)) expande para (car (cdr (cdr '(a b c)))); o resultado da avaliação da expressão é c.

Frequentemente, como nesse caso, um exemplo aparenta melhor se os comandos @expansion{} e @result{} forem recuados.

11.8.4 @print{} (⊣): Indicando Saída Gerada

Às vezes, uma expressão gerará saída durante a execução dela. Você pode indicar tal saída exibida com o comando @print{}.

O comando @print{} é exibido como ‘⊣’, ou um traço horizontal encostando em uma barra vertical ou (quando isso não estiver disponível) a sequência ASCII ‘-|’.

No exemplo seguinte, o texto impresso é indicado com ‘⊣’, e o valor da expressão segue na última linha.

```
(progn (print 'foo) (print 'bar))
⊣ foo
⊣ bar
⇒ bar
```

Em um arquivo fonte do Texinfo, esse exemplo é escrito como segue:

```
@example lisp
(progn (print 'foo) (print 'bar))
  @print{} foo
  @print{} bar
  @result{} bar
@end example
```

11.8.5 @error{} (`error`): Indicando uma Mensagem de Erro

Um pedaço de código possivelmente cause um erro quando você o avaliar. Você consegue designar a mensagem de erro com o comando `@error{}`.

O comando `@error{}` é exibido como ‘`error`’, ou a palavra ‘erro’ em uma caixa na saída impressa, ou a palavra erro seguida por uma seta em outros formatos ou (quando nenhuma seta estiver disponível) ‘`error-->`’.

Assim,

```
@example lisp
(+ 23 'x)
@error{} Argumento de tipo errado: integer-or-marker-p, x
@end example lisp
```

produz

```
(+ 23 'x)
error Argumento de tipo errado: integer-or-marker-p, x
```

Isso indica que a seguinte mensagem de erro é impressa quando você avaliar a expressão:

```
Argumento de tipo errado: integer-or-marker-p, x
```

A própria palavra ‘`error`’ não é parte da mensagem de erro.

11.8.6 @equiv{} (`≡`): Indicando Equivalência

Às vezes, duas expressões produzem resultados idênticos. Você consegue indicar a equivalência exata de duas formas com o comando `@equiv{}`. O comando `@equiv{}` é exibido como ‘`≡`’, ou um sinal padrão de equivalência matemática (três linhas horizontais paralelas) ou (quando isso não estiver disponível) como a sequência ASCII ‘`==`’.

Assim,

```
@example lisp
(make-sparse-keymap) @equiv{} (list 'keymap)
@end example
```

produz

```
(make-sparse-keymap) ≡ (list 'keymap)
```

Isso indica que avaliar `(make-sparse-keymap)` produz resultados idênticos à avaliação de `(list 'keymap)`.

11.8.7 @point{} (`*`): Indicando Ponto em um Buffer

Às vezes, você precisa mostrar um exemplo de texto em um buffer do Emacs. Em tais exemplos, a convenção é a de incluir o conteúdo inteiro do buffer em questão entre duas linhas de traços contendo o nome do buffer.

Você pode usar o comando ‘`@point{}`’ para mostrar o local do ponto no texto no buffer. (O símbolo para ponto, é claro, não é parte do texto no buffer; ele indica o lugar *entre* dois caracteres onde o ponto está localizado).

O comando `@point{}` é exibido como ‘`*`’, ou uma estrela pontiaguda ou (quando essa não estiver disponível) a sequência ASCII ‘`-!-`’.

O exemplo seguinte mostra o conteúdo do buffer `foo` antes e depois de avaliar um comando Lisp para inserir a palavra `changed`.

```
----- Buffer: foo -----
Este é o *conteúdo de foo.
----- Buffer: foo -----
```

```
(inserção "changed ")
⇒ nil
----- Buffer: foo -----
Este é o *conteúdo mudado de foo.
----- Buffer: foo -----
```

Em um arquivo fonte do Texinfo, o exemplo é escrito assim:

```
@example
----- Buffer: foo -----
Este é o @point{}conteúdo de foo.
----- Buffer: foo -----

(inserção "changed ")
@result{} nil
----- Buffer: foo -----
Este é o @point{}conteúdo mudado de foo.
----- Buffer: foo -----

@end example
```

11.8.8 Sequências de Cliques

Ao documentar interfaces gráficas, é necessário descrever sequências como ‘Clique em ‘Arquivo’, depois escolha ‘Abrir’, depois ...’. O Texinfo oferece os comandos `@clicksequence` e `click` para representar isso, normalmente usados assim:

```
... @clicksequence{Arquivo @click{} Abrir} ...
```

o que produz:

```
... Arquivo → Abrir ...
```

O comando `@click` produz uma seta para a direita; esse glifo também está disponível independentemente por meio do comando `@arrow{}.`

O comando `@clickstyle` que poderia ser usado para mudar o glifo produzido por `@click` está obsoleto; você deveria não mais usá-lo. Você pode usar `@alias` (veja-se Seção 17.4 [`@alias`], Página 144) para mudar o glifo produzido por `@click`.

Você pode substituir:

```
@clickstyle @result
```

by

```
@alias clique=resultado
```

O comando `@clickstyle` será removido em um lançamento futuro.

11.9 Inserindo Unicode: @U

O comando `@U{hexadecimal}` insere uma representação do caractere Unicode U+*hexadecimal*. Por exemplo, `@U{0132}` insere a ligadura holandesa ‘IJ’ (‘IJ’).

O valor *hexadecimal* deveria ter pelo menos quatro dígitos hexadecimais; zeros à esquerda não são adicionados. Em geral, *hexadecimal* precisa especificar um caractere Unicode normal válido; por exemplo, U+10FFFF (o último ponto de código) é inválido por definição e, portanto, não pode ser inserido dessa forma.

@U é útil para inserir glifos ocasionais para os quais Texinfo não tenha comando dedicado, ao mesmo tempo em que permite que o fonte do Texinfo permaneça puramente ASCII de 7 bits para máxima portabilidade.

Esse comando tem muitas limitações—as mesmas limitações de inserir caracteres Unicode em UTF-8 ou outra forma binária. Primeiro e mais importante, $\text{\TeX}$ não sabe nada acerca da maioria do Unicode. Suportar glifos adicionais específicos mediante solicitação é possível, mas não é viável para `texinfo.tex` suportar conjuntos adicionais e inteiros de comandos sequenciados (japonês, urdu, . . .). O comando `@U` não faz nada para mudar isso. Se o caractere especificado não for suportado no $\text{\TeX}$, um erro será fornecido. A saída do $\text{\LaTeX}$ tem mais possibilidades em relação ao UTF-8, mas poderia exigir código extra para carregar fontes e para declarar como os caracteres UTF-8 são gerados. (Veja-se Seção 15.2 [`@documentencoding`], Página 127).

Em HTML e DocBook, a saída proveniente de `@U` é sempre uma referência de entidade do formato `&#xhexadecimal;`, como em `Ĳ` para o exemplo acima. Isso deveria funcionar mesmo quando um documento HTML usar alguma outra codificação (digamos, Latin 1) e o caractere fornecido não for suportado nessa codificação.

No Info e em texto simples, se a codificação da saída não for UTF-8, a saída será a sequência ASCII `'U+hexadecimal'`, como nos seis caracteres ASCII `'U+0132'` para o exemplo acima.

12 Formatando Expressões Matemáticas

Você consegue escrever uma expressão matemática, equação ou fórmula usando o comando `@math`. Escreva a notação matemática do $\text{\TeX}$ entre chaves, assim:

```
@math{\partial_{\alpha} \partial^{\alpha} A^{\beta} = \mu_0 J^{\beta}}
```

`@math` é formatado em linha quando usado dentro de um parágrafo, como $\partial_{\alpha} \partial^{\alpha} A^{\beta} = \mu_0 J^{\beta}$ nesse exemplo.

Por padrão, o comando `@math` não tem nenhum efeito especial na saída Info, meramente gerando o conteúdo literalmente. Você pode usar a variável experimental `INFO_MATH_IMAGES` para produzir imagens; veja-se Seção 20.4.5 [Variáveis de Personalização do Info e de Texto Simples], Página 173.

Para saída impressa, `@math` comuta para o “modo matemático” do $\text{\TeX}$. Nesse contexto, ‘\’ precisa ser usada em vez de ‘@’ para sequências de controle matemático simples do $\text{\TeX}$ para símbolos, funções e assim por diante.

Por padrão, a saída HTML é apenas enfatizada. `texi2any` fornece três opções para exibir expressões matemáticas formatadas corretamente para HTML. Você consegue selecioná-las com a variável `HTML_MATH` (veja-se Seção 22.8.7 [Personalização do HTML para Matemática], Página 211). Com `HTML_MATH` configurada para ‘12h’, `texi2any` tenta usar o programa `latex2html` para produzir arquivos de imagem para material matemático. Com a configuração ‘t4h’, `texi2any` tenta usar o programa `tex4ht`. Com a configuração ‘mathjax’, `texi2any` insere referências nos arquivos da saída gerada para conjuntos de comandos sequenciados MathJax para formatar o material. A opção MathJax exige que JavaScript esteja habilitado no navegador para funcionar. Veja-se também Seção 22.8.7.1 [Variáveis de Personalização da MathJax], Página 212, Seção 22.8.7.2 [Variáveis de Personalização do `latex2html`], Página 213, e Seção 22.8.7.3 [Variáveis de Personalização do `tex4ht`], Página 213.

Para equações exibidas, você pode o comando `@displaymath`. Exemplo:

```
@displaymath
f(x) = {1\over\sigma\sqrt{2\pi}}
e^{-{1\over2}\left({x-\mu\over\sigma}\right)^2}
@end displaymath
```

o que produz:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Em geral, o conteúdo de `@math` ou de `@displaymath` deveria ser somente $\text{\TeX}$ simples, sem comandos Texinfo intercalados. Se usar quaisquer comandos do Texinfo, então você deveria marcá-los com ‘@’, como de costume, em vez de ‘\’ (por exemplo, ‘@var’ em vez de ‘\var’), mas nós não garantimos quais comandos do Texinfo funcionarão.

Embora `@sub` e `@sup` possam funcionar dentro do modo matemático em alguns contextos, é melhor usar os caracteres ‘_’ e ‘^’ do $\text{\TeX}$ para denotar subscritos e sobrescritos dentro de expressões matemáticas.

O código específico do $\text{\LaTeX}$ só funcionará quando o formato de saída for $\text{\LaTeX}$ e com as opções da `HTML_MATH` (embora `tex4ht` precise que `T4H_MATH_CONVERSION` seja configurada para ‘latex’; veja-se Seção 22.8.7.3 [Variáveis de Personalização do `tex4ht`], Página 213).

Devido ao conflito com o comando `@sup` do Texinfo, você pode acessar o comando `\sup` do $\text{\TeX}$ simples como `\mathopsup`, na improvável ocorrência de querer fazer isso (mas somente ao processar com $\text{\TeX}$, não ao gerar $\text{\LaTeX}$ nem com qualquer das opções da `HTML_MATH`).

13 Forçando e Impedindo Quebras

Quebras de linha e de página às vezes podem ocorrer no lugar ‘errado’ em uma ou outra forma de saída. Cabe inteiramente a você garantir que o texto aparente corretamente em todos os formatos de saída.

Por exemplo, em um manual impresso, quebras de página podem ocorrer estranhamente no meio de um exemplo; para impedir isso, você pode manter o texto unido usando um comando de agrupamento que impeça que o texto seja dividido entre duas páginas. Por outro lado, você pode querer forçar uma quebra de página onde nenhuma ocorreria normalmente.

Você pode usar os comandos de quebra, impedimento de quebra ou de paginação para corrigir quebras problemáticas de linha e de página.

13.1 Comandos de Quebra

Os comandos de quebra criam ou permitem quebras de linha e de parágrafo:

- `@*` Força uma quebra de linha.
- `@sp n` Salta *n* linhas em branco.
- `@-` Insere um hífen discricionário.
- `@hyphenation{palavras hi-fe-na-das}`
Define pontos de hífen em *palavras hi-fe-na-das*.

Esses comandos mantêm o texto unido em uma linha:

- `@w{texto}`
Impede que *texto* seja dividido e hifenizado entre duas linhas.
- `@tie{}` Insere um espaço normal entre palavras no qual uma quebra de linha não possa ocorrer.

Os comandos de paginação aplicam-se somente para a saída impressa, pois outros formatos de saída não tem páginas.

- `@page` Inicia uma nova página.
- `@group` Mantém unido o texto que precisa aparecer em uma página.
- `@need milímetros`
Inicia uma nova página se não existir espaço suficiente nesta.

13.2 `@*` e `@/`: Gerar e Permitir Quebras de Linha

O comando `@*` força uma quebra de linha em todos os formatos de saída. O comando `@/` permite uma quebra de linha (somente manual impresso).

Aqui está um exemplo com `@*`:

```
Esta frase está quebrada @*em duas linhas.
```

produz

```
Esta frase está quebrada
em duas linhas.
```

O comando `@/` pode ser útil em URLs longas ou outros identificadores onde o `TeX` não consiga encontrar um bom lugar para quebrar. O `TeX` automaticamente quebrará as URLs nos lugares naturais (veja-se Seção 5.14.2 [Quebra de Linha de URL], Página 51), de forma que use `@/` somente se você precisar. `@/` não tem efeito em outros formatos de saída.

13.3 @- e @hyphenation: Hifenização em Saída Impressa

Embora o algoritmo de hifenização do T_EX geralmente seja muito bom, ele perde pontos de hifenização úteis de vez em quando. (Ou, muito mais raramente, insere uma hifenização incorreta). Então, para documentos com um vocabulário incomum ou ao fazer ajustes finos para uma edição impressa, você possivelmente deseje especificar pontos de hifenização explicitamente. Texinfo suporta dois comandos para isso:

@-

Insere um hífen discricionário, ou seja, um lugar onde uma palavra possa ser quebrada entre linhas com um hífen. Isso é especialmente útil quando você perceber que um hbox cheio demais é devido ao T_EX carecer de uma hifenização (veja-se Seção 19.5 [hboxes lotados], Página 155). T_EX não inserirá ele próprio quaisquer pontos de hifenização em uma palavra contendo @-.

@hyphenation{palavras *hi-fe-na-das*}

Fornece pontos de hifenização para certas palavras. Por exemplo:

@hyphenation{man-u-scrito man-u-scritos}

Conforme mostrado, você coloca um ‘-’ em cada ponto de hifenização. T_EX usa somente os pontos de hifenização especificados quando as palavras corresponderem exatamente, portanto forneça todas as variantes necessárias, tais como plurais.

Saídas não impressas não são hifenizadas, de forma que nenhum desses comandos tem qualquer efeito em outros formatos de saída.

13.4 @allowcodebreaks: Controle Quebras de Linha no @code

Normalmente, T_EX considera a quebra de linhas nos caracteres ‘-’ e ‘_’ dentro do @code e comandos relacionados (veja-se Seção 6.1.2 [@code], Página 55), mais ou menos como se fossem pontos de hifenização “vazios”.

Isso é necessário, pois muitos manuais, especialmente para linguagens da família Lisp, precisam documentar identificadores muito longos. Por outro lado, alguns manuais não tem esse problema, e você pode não desejar permitir uma quebra de linha no sublinhado em, por exemplo, `SIZE_MAX`, ou pior ainda, depois de qualquer um dos quatro sublinhados em `__typeof__`.

Assim, Texinfo fornece este comando:

@allowcodebreaks false

para evitar quebra em ‘-’ ou em ‘_’ dentro de @code. Você pode voltar a permitir tais quebras com @allowcodebreaks true. Escreva esses comandos em linhas dedicadas.

Esses comandos podem ser fornecidos em qualquer lugar no documento. Por exemplo, você pode ter apenas um parágrafo problemático onde precisa desligar as quebras, mas querê-las em geral, ou vice-versa.

Esse comando não tem efeito, exceto na saída do T_EX.

13.5 @w{texto}: Impedir Quebras de Linha

@w{texto} gera *texto*, ao mesmo tempo que proíbe quebras de linha dentro do *texto*.

Assim, você pode usar @w para produzir um espaço não quebrável, fixado na largura de um espaço normal interpalavras:

@w{ } @w{ } @w{ } recuo.

produz:

recuo.

O espaço oriundo de `@w{ }`, além de ser não quebrável, também não esticará nem encolherá. Às vezes isso é o que você quer, por exemplo, se estiver fazendo recuo manual. No entanto, normalmente você quer um espaço normal interpalavra que estique e encolha (na saída impressa); para isso, veja-se o comando `@tie` na próxima seção.

Na saída impressa, você também pode usar o comando `@w` para impedir que um nome longo ou uma frase seja automaticamente hifenizada, por exemplo se acontecer perto do fim de uma linha.

Você também pode usar `@w` para evitar expansão indesejada de palavra-chave em sistemas de controle de fonte. Por exemplo, para escrever literalmente `$Id$` em teu documento, use `@w{$}Id$`. Entretanto, esse truque não é eficaz para arquivos de saída em alguns formatos de saída.

13.6 `@tie{}`: Inserindo um Espaço Inquebrável

O comando `@tie{}` produz um espaço normal interpalavra no qual uma quebra de linha possivelmente não ocorra. Sempre escreva-o com as seguintes chaves (vazias), como de costume para comandos usados dentro de um parágrafo. Aqui está um exemplo:

`@TeX{}` foi escrito por Donald E.`@tie{}`Knuth.

produces:

TeX foi escrito por Donald E. Knuth.

Existem duas diferenças importantes entre `@tie{}` e `@w{ }`:

- O espaço produzido por `@tie{}` esticará e encolherá ligeiramente junto com os espaços normais entre palavras no parágrafo; o espaço produzido por `@w{ }` não variará.
- `@tie{}` permite hifenização das palavras ao redor, enquanto `@w{ }` inibe a hifenização dessas palavras (por motivos T_EXnológicas, ou seja, porque produz um ‘`\hbox`’).

13.7 `@sp n`: Inserir Linhas em Branco

Uma linha que começa com e contém somente `@sp n` gera *n* linhas de espaço em branco. `@sp` também força uma quebra de parágrafo. Por exemplo,

`@sp 2`

gera duas linhas em branco.

O comando `@sp` é usado mais frequentemente dentro de um bloco `@titlepage`.

13.8 `@page`: Comece uma Nova Página

Uma linha contendo somente `@page` inicia uma nova página em um manual impresso. Em outros formatos, sem o conceito de páginas, ele inicia um novo parágrafo. Um comando `@page` frequentemente é usado em uma seção `@titlepage` de um arquivo do Texinfo para iniciar a página de copirraite.

13.9 `@group`: Impedir Quebras de Página

O comando `@group` é usado para manter um exemplo unido em uma página. Use-o dentro de um `@example` ou construção similar para começar um grupo vertical indivisível, que aparecerá inteiramente em uma página na saída impressa. Termine o grupo por uma linha contendo somente `@end group`. Esse comando tem efeito somente na saída do T_EX.

Os comandos `@group` e `@end group` precisam estar dentro dos comandos `@example` e `@end example`, assim:

```
@example
@group
...
@end group
@end example
```

Embora `@group` fizesse sentido conceitualmente em uma ampla variedade de contextos, a implementação atual dele funciona somente dentro de `@example` e variantes, e dentro de `@display`, `@format`, `@flushleft` e `@flushright`. Veja-se Capítulo 7 [Citações e Exemplos], Página 65. (O que todos esses comandos tem em comum é que cada linha da entrada produz uma linha de saída).

13.10 `@need mils`: Impedir Quebras de Página

Uma linha contendo somente `@need n` inicia uma nova página em um manual impresso se menos que n mils (milésimos de polegada) permanecerem na página atual. Não use chaves ao redor do argumento n . O comando `@need` não tem efeito em outros formatos de saída, pois eles não são paginados.

Este parágrafo é precedido por um comando `@need` que inicia uma nova página na saída impressa se restarem menos que 800 mils (oito décimos de polegada) na página. Parece algo assim:

```
@need 800
Este parágrafo é precedido por ...
```

O comando `@need` é útil para evitar órfãs: linhas únicas na parte inferior das páginas impressas.

14 Comandos de Definição

O comando `@defn` e os outros *comandos de definição* te habilitam a descrever funções, variáveis, macros, comandos, opções de usuário(a), formulários especiais e outras entidades em um formato uniforme.

Na saída, o nome da entidade é mostrado com quaisquer argumentos, juntamente com a categoria da entidade —‘Função’, ‘Variável’ ou qualquer outra. Abaixo, o corpo da definição é recuado. O nome da entidade também é inserido no índice apropriado.

Todos os comandos de definição seguem um formato semelhante. Este capítulo começa descrevendo `@defn` antes de detalhar todos os outros comandos de definição.

14.1 O Modelo Para Uma Definição

```
@defn categoria nome argumentos...
  corpo-da-definição
@end defn
```

O comando `@defn` é usado para definições de entidades que se assemelham a funções—entidades que possivelmente recebam argumentos. Escreva o comando `@defn` no início de uma linha e siga-o na mesma linha pela categoria da entidade, o nome da entidade em si e os argumentos dela (se existirem). Em seguida, escreva o corpo da definição nas linhas seguintes. Finalmente, termine a definição com um comando `@end defn` escrito em uma linha própria.

Por exemplo,

```
@defn Command forward-word count
  Esse comando move o ponto para frente @var{count} palavras
  (ou para trás, se @var{count} for negativo). ...
@end defn
```

produz

```
forward-word count [Command]
  Esse comando move o ponto para frente count palavras (ou para trás, se count
  for negativo). ...
```

Maiusculize o nome de categoria como um título. Se o nome da categoria contiver espaços, como na frase ‘Comando Interativo’, circunde-o em chaves. Por exemplo:

```
@defn {Comando Interativo} isearch-forward
...
@end defn
```

Caso contrário, a segunda palavra será confundida com o nome da entidade. Como regra geral, quando quaisquer dos argumentos na linha de título, *exceto* o último, forem mais que uma palavra, você precisa circundá-los em chaves. Isso também possivelmente seja necessário se o texto contiver comandos, por exemplo, ‘{declaraci@'on}’ se você estiver escrevendo em espanhol.

A categoria é gerada em um local diferente para diferentes formatos de saída. Por exemplo, no arquivo Info, a categoria aparece no início da primeira linha da definição. Com a saída do T_EX, a categoria é impressa próxima da margem direita.

`@defn` insere nomes no índice das funções.

Três variações predefinidas e especializadas de `@defn` (`@defun`, `@defmac` e `@defspec`) especificam a categoria para você: “Função”, “Macro” e “Forma Especial” respectivamente. (Em Lisp, uma forma especial é uma entidade muito parecida com uma função). Similarmente, o comando geral `@defvr` é acompanhado por diversas variações especializadas para descrever tipos particulares de variáveis.

14.2 Linhas de Continuação de Comando de Definição

A linha de cabeçalho de um comando de definição pode se tornar muito longa. Portanto, o Texinfo tem uma sintaxe especial que permite que elas sejam continuadas em várias linhas do arquivo fonte: um único ‘@’ no final de cada linha a ser continuada. Aqui está um exemplo:

```
@defun fn-name @
  arg1 arg2 arg3
  Essa é a defun básica continuada.
@end defun
```

produz:

```
fn-name arg1 arg2 arg3 [Função]
  Essa é a defun básica continuada.
```

Como você pode ver, as linhas continuadas estão combinadas, como se tivessem sido digitadas em uma linha do fonte.

Embora esse exemplo mostre somente uma continuação de uma linha, as continuações podem se estender por quaisquer número de linhas, da mesma maneira; coloque um @ no final de cada linha a ser continuada.

Em geral, qualquer número de espaços ou tabulações antes do caractere de continuação @ é recolhido em um espaço. Existe uma exceção: os processadores Texinfo não recolherão totalmente espaços em branco em torno de uma continuação dentro de chaves. Por exemplo:

```
@defn {Nome @
  Categoria} ...
```

A saída gerada (não mostrada) tem espaço excessivo entre ‘Categoria’ e ‘Nome’. Para evitar isso, elimine os espaços em branco indesejados na tua entrada ou coloque a continuação @ fora das chaves.

@ não funciona como um caractere de continuação em *nenhum* outro contexto. Normalmente, ‘@’ seguido por um caractere de espaço em branco (espaço, tabulação, nova linha) produz um espaço normal interpalavra (veja-se Seção 11.3.1 [Espaços Múltiplos], Página 98).

14.3 Argumentos Opcionais e Repetidos

Algumas entidades recebem argumentos opcionais ou repetidos. Uma convenção para indicá-los usa colchetes e reticências: um argumento circundado por colchetes é opcional, e um argumento seguido por reticências é opcional e pode ser repetido mais que uma vez.

Assim, [argumento-opcional] significa que *argumento-opcional* é opcional e *argumentos-repetidos...* representa zero ou mais argumentos. Parênteses são usados quando vários argumentos são agrupados em níveis adicionais da estrutura de lista na Lisp.

Aqui está a linha de definição de um exemplo de uma forma especial (complicada) imaginária:

```
foobar (var [de para [inc]]) corpo... [Forma Especial]
```

Nesse exemplo, os argumentos *de* e *para* são opcionais, mas ambos precisam estar presentes ou ambos ausentes. Se estiverem presentes, *inc* opcionalmente pode ser especificado também. Esses argumentos são agrupados com o argumento *var* em uma lista, para distingui-los de *corpo*, que inclui todos os elementos restantes da forma.

Em um arquivo fonte do Texinfo, essa linha @defspec é escrita assim:

```
@defspec foobar (var [de para [inc]]) corpo@dots{}
```

14.4 Omitindo o Espaço Depois de um Nome de Definição

Por uma questão de estilo ou conforme demandado pela sintaxe de uma linguagem de programação, você possivelmente deseje omitir algum espaço depois de um nome em uma definição, ocorrente antes de um colchete de abertura. Para fazer isso, configure o sinalizador ‘`txidefnamenspace`’ (veja-se Seção 16.5.1 [`@set @value`], Página 133). Por exemplo, esta entrada

```
@set txidefnamenspace
@deffn Índice_m4 interno (string, substring)
@dots{}
@end deffn
```

produz o seguinte:

```
interno(string, substring) [Índice_m4]
...
```

14.5 @deffnx, et al.: Duas ou Mais ‘Primeiras’ Linhas

Para criar duas ou mais linhas ‘primeiras’ ou de cabeçalho para uma definição, siga a primeira linha `@deffn` por uma linha começando com `@deffnx`.

Por exemplo,

```
@deffn {Comando Interativo} isearch-forward
@deffnx {Comando Interativo} isearch-backward
Esses dois comandos de pesquisa são semelhantes, exceto ...
@end deffn
```

produz

```
isearch-forward [Comando Interativo]
isearch-backward [Comando Interativo]
Esses dois comandos de pesquisa são semelhantes, exceto ...
```

Cada comando de definição tem uma forma ‘x’: `@defunx`, `@defvrnx`, `@deftypefunx`, etc.

As formas ‘x’ funcionam similarmente a `@itemx` (veja-se Seção 8.4.3 [`@itemx`], Página 78).

14.6 Os Comandos de Definição

Esta seção descreve todos os comandos de definição do Texinfo.

14.6.1 Funções e Entidades Similares

Esta seção descreve os comandos para descrever funções e entidades similares com argumentos simples:

`@deffn categoria nome argumentos...`

O comando `@deffn` é o comando geral de definição para funções, comandos interativos e entidades similares que podem receber argumentos simples. Você precisa escolher um termo para descrever a categoria da entidade sendo definida; por exemplo, “Função” poderia ser usado se a entidade for uma função. O comando `@deffn` é escrito no início de uma linha e é seguido na mesma linha pela categoria da entidade sendo descrita, o nome dessa entidade específica e os argumentos dela, se existirem. Termine a definição com `@end deffn` em uma linha própria.

Por exemplo, aqui está uma definição:

```
@defn Comando forward-char nchars
Move ponto para frente @var{nchars} caracteres.
@end defn
```

Isso mostra uma definição bastante concisa para um “comando” chamado `forward-char` com um argumento, `nchars`.

No texto da descrição, no entanto, escreva um nome de argumento explicitamente com `@var` para se referir ao valor do argumento. No exemplo acima, nós usamos ‘`@var{nchars}`’ dessa maneira.

No caso extremamente incomum quando um nome de argumento contiver ‘`--`’, ou outra sequência de caracteres que seja tratada especialmente (veja-se Seção 2.1 [Convenções], Página 8), use `@code` ao redor dos caracteres especiais. Isso evita a conversão para traços de ligação e travessões tipográficos.

`@defn nome argumentos...`

O comando `@defun` é o comando de definição para funções com argumentos simples. `@defun` é equivalente a ‘`@defn Função ...`’. Termine a definição com `@end defun` em uma linha própria.

`@defmac nome argumentos...`

O comando `@defmac` é o comando de definição para macros. `@defmac` é equivalente a ‘`@defn Macro ...`’ e funciona como `@defun`.

`@defspec nome argumentos...`

O comando `@defspec` é o comando de definição para formas especiais. (Na Lisp, uma forma especial é uma entidade muito parecida com uma função; veja-se Seção “Special Forms” em *GNU Emacs Lisp Reference Manual*). `@defspec` é equivalente a ‘`@defn {Forma Especial} ...`’ e funciona como `@defun`.

Todos esses comandos criam entradas no índice de funções.

14.6.2 Funções em Linguagens Tipadas

O comando `@deftypefn` e as variantes dele são comandos genéricos para descrever funções. Eles são particularmente adequados para linguagens nas quais você precisa declarar tipos de variáveis e funções, como C e C++.

`@deftypefn categoria tipo-dados nome argumentos...`

O comando `@deftypefn` é o comando geral de definição para funções e entidades similares que podem receber argumentos e que poderiam ser tipadas. O comando `@deftypefn` é escrito no início de uma linha e é seguido na mesma linha pela categoria da entidade sendo descrita, informações acerca do valor retornado, o nome dessa entidade específica e os argumentos dela, se existirem.

Por exemplo,

```
@deftypefn {Função de Biblioteca} int foo_bar @
(int @var{foo}, float @var{bar})
...
@end deftypefn
```

produz:

```
int foo_bar (int foo, float bar) [Função de Biblioteca]
...
```

Isso significa que `foo_bar` é uma “função de biblioteca” que retorna um `int`, e os argumentos dela são `foo` (um `int`) e `bar` (um `float`). ‘`Função de Biblioteca`’ tem de ser circundado por chaves para torná-lo um argumento unitário.

Ao usar o comando `@deftypefn` e variações, você deveria marcar os nomes dos parâmetros com `@var` para distingui-los dos nomes de tipos de dados, de palavras-chave e de outras partes da sintaxe literal da linguagem de programação.

Se você estiver descrevendo um procedimento em uma linguagem que tenha pacotes, como Ada, você pode considerar usar `@deftypefn` da seguinte maneira:

```
@deftypefn pilhas privado push @
    (@var{s}:in out stack; @
    @var{n}:in integer)
...
@end deftypefn
```

(Nesses exemplos os argumentos de `@deftypefn` são mostrados usando-se continuções (veja-se Seção 14.2 [Linhas de Continuação de Comando de Definição], Página 115), mas poderiam estar em uma linha).

Nesse caso, o procedimento é classificado como pertencente ao pacote **pilhas** em vez de classificado como um ‘procedimento’ e o tipo de dados dele está descrito como **privado**. (O nome do procedimento é **push**, e os argumentos dele são *s* e *n*). Saída gerada:

```
privado push (s:in out stack; n:in integer) ... [pilhas]
```

`@deftypefn` e variantes poderiam ser usados quando for conveniente interpretar a lista de argumentos como código literal de computador, marcando os nomes dos argumentos com `@var`. Não é necessário para quaisquer nomes de tipo aparecer, e você pode fornecer um tipo de retorno vazio como ‘{}’.

As informações acerca dos valores de retorno podem ser usadas para nomear as variáveis de retorno. Por exemplo, na Perl, os escalares retornados podem receber os nomes `$result` e `$status` da seguinte maneira:

```
@deftypefn {Sub} {(@var{$resultado}, @var{$situação}) =} @
    process (@var{$entrada})
... Configurar @var{$situação} para 0 no caso de falha ...
@end deftypefn
```

para obter saída como

```
($result, $status) = process ($input) [Sub]
... Set $status to 0 in case of failure ...
```

`@deftypefn` cria uma entrada no índice de funções para *nome*.

`@deftypefun tipo-dados nome argumentos...`

O comando `@deftypefun` é o comando especializado de definição para funções. O comando é equivalente a ‘`@deftypefn Função ...`’.

`@deftypefun` cria uma entrada no índice de funções para *nome*.

Normalmente, o tipo de retorno é impresso na mesma linha que o nome da função e os argumentos, como mostrado acima. No código-fonte, o estilo GNU para funções tipadas é o de colocar o tipo de retorno em uma linha dedicada. Portanto, Texinfo fornece uma opção para fazer isso: `@deftypefnnewline on`.

Isso afeta somente funções genéricas—não funções não tipadas com argumentos simples, como `@defn`, variáveis não tipadas, e outras coisas. Especificamente, afeta os comandos nesta seção e os comandos análogos para linguagens orientadas a objetos, ou seja, `@deftypeop` e `@deftypemethod` (veja-se Seção 14.6.6.2 [Métodos Orientados a Objetos], Página 121).

Especificar-se `@deftypefnnewline off` reverte para o padrão.

14.6.3 Variáveis e Entidades Similares

Aqui estão os comandos para se definir variáveis e entidades similares:

@defvr *categoria nome*

O comando **@defvr** é um comando geral de definição para algo como uma variável—uma entidade que registra um valor. Você precisa escolher um termo para descrever a categoria da entidade sendo definida; por exemplo, “Variável” poderia ser usado se a entidade fosse uma variável. Escreva o comando **@defvr** no início de uma linha e siga-o na mesma linha pela categoria da entidade e o nome da entidade.

Nós recomendamos maiusculizar o nome da categoria, como um título. Se o nome da categoria contiver espaços, como no nome “Opção de Usuário(a)”, circunde-o com chaves. Caso contrário, a segunda palavra será confundida com o nome da entidade. Por exemplo,

```
@defvr {Opção de Usuário(a)} fill-column
Essa variável local e de buffer especifica
a largura máxima das linhas preenchidas.
...
@end defvr
```

Termine a definição com **@end defvr** em uma linha própria.

@defvr cria uma entrada no índice de variáveis para *nome*.

@defvar *nome*

O comando **@defvar** é o comando de definição para variáveis. **@defvar** é equivalente a ‘**@defvr Variável ...**’.

Por exemplo:

```
@defvar kill-ring
...
@end defvar
```

@defvar cria uma entrada no índice de variáveis para *nome*.

@defopt *nome*

O comando **@defopt** é o comando de definição para *opções de usuário(a)*, ou seja, variáveis destinadas para usuários(as) mudarem de acordo com o gosto; Emacs tem muitas dessas (veja-se Seção “Variables” em *O Manual do GNU Emacs*). **@defopt** é equivalente a ‘**@defvr {Opção de Usuário(a)} ...**’.

14.6.4 Variáveis em Linguagens Tipadas

Variáveis em linguagens tipadas são manuseadas de maneira similar a funções em linguagens tipadas. Veja-se Seção 14.6.2 [Funções Tipadas], Página 117. O comando geral de definição **@deftypevr** corresponde a **@deftypefn** e o comando especializado de definição **@deftypevar** corresponde a **@deftypefun**.

@deftypevr *categoria tipo-dados nome*

O comando **@deftypevr** é o comando geral de definição para alguma coisa como uma variável em uma linguagem tipada—uma entidade que registra um valor. Você precisa escolher um termo para descrever a categoria da entidade sendo definida; por exemplo, “Variável” poderia ser usado se a entidade fosse uma variável.

O comando **@deftypevr** é escrito no início de uma linha e é seguido na mesma linha pela categoria da entidade sendo descrita, o tipo de dados e o nome dessa entidade específica.

Por exemplo:

```
@deftypevr {Sinalizador Global} int enable
...
@end deftypevr
```

produz o seguinte:

```
int enable [Sinalizador Global]
...
```

@deftypevar *tipo-dados nome*

O comando **@deftypevar** é o comando especializado de definição para variáveis em linguagens tipadas. **@deftypevar** é equivalente a ‘**@deftypevr Variável ...**’.

Esses comandos criam entradas no índice de variáveis.

14.6.5 Tipos de Dados

Aqui está o comando para tipos de dados:

@defntp *categoria nome atributos...*

O comando **@defntp** é o comando genérico de definição para tipos de dados. O comando é escrito no início de uma linha e é seguido na mesma linha pela categoria, pelo nome do tipo (que é uma palavra como **int** ou **float**) e, em seguida, pelos nomes de atributos de objetos desse tipo. Portanto, você poderia usar esse comando para descrever **int** ou **float**, em cujo caso você poderia usar **tipo de dados** como a categoria. (Um tipo de dados é uma categoria de certos objetos para fins de decidir quais operações podem ser realizadas sobre neles).

Na Lisp, por exemplo, *pair* nomeia um tipo particular de dado, e um objeto desse tipo tem dois slots chamados **CAR** e **CDR**. Aqui está como você escreveria a primeira linha de uma definição de *pair*.

```
@defntp {Tipo de dados} pair car cdr
...
@end defntp
```

@defntp cria uma entrada no índice de tipos de dados.

14.6.6 Programação Orientada a Objetos

Aqui estão os comandos para formatar descrições acerca de objetos abstratos, tais como são usados em programação orientada a objetos. Uma classe é um tipo definido de objeto abstrato. Uma instância de uma classe é um objeto particular que tenha o tipo da classe. Uma variável de instância é uma variável que pertence à classe, mas para a qual cada instância tenha o próprio valor dela.

14.6.6.1 Variáveis Orientadas a Objetos

Estes comandos permitem que você defina diferentes tipos de variáveis em linguagens de programação orientadas a objetos.

@defncv *categoria classe nome*

O comando **@defncv** é o comando geral de definição para variáveis associadas a classes em programação orientada a objetos. O comando **@defncv** é seguido por três argumentos: a categoria da coisa sendo definida, a classe à qual ela pertence e o nome dela. Por exemplo:

```
@defncv {Opção da Classe} Janela padrão-borda
...
@end defncv
```

produz:

`padrão-borda` [Opção da Classe de Janela]

...

`@defcv` cria uma entrada no índice de variáveis.

`@deftypecv categoria classe tipo-dados nome`

O comando `@deftypecv` é o comando de definição para variáveis tipadas de classe em programação orientada a objetos. Ele é análogo ao `@defcv` com a adição do parâmetro *tipo-dados* para especificar o tipo da variável de instância.

`@deftypecv {Opção da Classe} Janela int padrão-borda`

...

`@end deftypecv`

produz:

`int padrão-borda` [Opção da Classe de Janela]

...

`@deftypecv` cria uma entrada no índice de variáveis.

`@defivar classe nome`

O comando `@defivar` é o comando de definição para variáveis de instância em programação orientada a objetos. `@defivar` é equivalente a '`@defcv {Variável da Instância} ...`'. Por exemplo:

`@defivar Janela padrão-borda`

...

`@end defivar`

produz:

`padrão-borda` [Instance Variable of Janela]

...

`@defivar` cria uma entrada no índice de variáveis.

`@deftypeivar classe tipo-dados nome`

O comando `@deftypeivar` é o comando de definição para variáveis tipadas de instância em programação orientada a objetos. Ele é análogo ao `@defivar` com a adição do parâmetro *tipo-dados* para especificar o tipo da variável de instância.

`@deftypeivar Janela int padrão-borda`

...

`@end deftypeivar`

produz:

`int padrão-borda` [Instance Variable of Janela]

...

`@deftypeivar` cria uma entrada no índice de variáveis.

14.6.6.2 Métodos Orientados a Objetos

Esses comandos permitem que você defina diferentes tipos de entidades semelhantes a funções que lembram métodos em linguagens de programação orientadas a objetos. Essas entidades recebem argumentos, como as funções, mas são associadas a classes específicas de objetos.

`@defop categoria classe nome argumentos...`

O comando `@defop` é o comando de definição para essas entidades semelhantes a métodos, com argumentos simples

Por exemplo, alguns sistemas tem construções chamadas *involucrados* que são associadas a classes, como métodos, mas que agem mais como macros que como funções. Você poderia usar `@defop Involucrador` para descrever uma delas.

Ocasionalmente, é útil distinguir métodos e *operações*. Você pode pensar em uma operação como a especificação para um método. Assim, um sistema de janelas pode especificar que todas as classes de janelas tem um método chamado `expor`; nós diríamos que esse sistema de janelas define uma operação `expor` sobre janelas em geral. Normalmente, a operação tem um nome e também especifica o padrão de argumentos; todos os métodos que implementem a operação precisam aceitar os mesmos argumentos, já que aplicações que usem a operação o fazem sem saber qual método a implementará.

Frequentemente faz mais sentido documentar operações que métodos. Por exemplo, desenvolvedores(as) de aplicações de janela precisam saber acerca da operação `expor`, mas não precisam se preocupar se uma determinada classe de janelas tem o próprio método dela para implementar essa operação. Para descrever essa operação, você escreveria:

```
@defop Operação janelas expor
```

O comando `@defop` é escrito no início de uma linha e é seguido na mesma linha pelo nome geral da categoria da operação, pelo nome da classe da operação, pelo nome da operação e pelos argumentos dela, se existirem.

`@defop` cria uma entrada, como ‘`expor sobre janelas`’, no índice de funções.

```
@deftypeop categoria classe tipo-dados nome argumentos...
```

O comando `@deftypeop` é o comando genérico de definição para operações em programação orientada a objetos. Ele é especificamente adequado para linguagens tipadas orientadas a objetos. Ele é semelhante ao `@defop` com a adição do parâmetro *tipo-dados* para especificar informações acerca do valor de retorno do método, por exemplo o tipo de retorno.

```
@defmethod classe nome argumentos...
```

O comando `@defmethod` é o comando de definição para métodos em programação orientada a objetos. Um método é um tipo de função que implementa uma operação para uma classe particular de objetos e as subclasses dela.

`@defmethod` é equivalente a ‘`@defop Método ...`’. O comando é escrito no início de uma linha e é seguido pelo nome da classe do método, pelo nome do método e pelos argumentos dele, se existirem.

Por exemplo:

```
@defmethod classe-barra método-barra argumento
...
@end defmethod
```

ilustra a definição para um método chamado `método-barra` da classe `classe-barra`. O método recebe um argumento.

`@defmethod` cria uma entrada no índice de funções.

```
@deftypemethod classe tipo-dados nome argumentos...
```

O comando `@deftypemethod` é o comando de definição para métodos em linguagens orientadas a objetos, em particular para linguagens tipadas como C++ e Java. Ele é semelhante ao comando `@defmethod` com a adição do parâmetro *tipo-dados* para especificar informações acerca do valor de retorno do método, por exemplo o tipo de retorno.

Os comandos com informação acerca de valores de retorno são afetados pela opção `@deftypefnnewline` (veja-se Seção 14.6.2 [Funções em Linguagens Tipadas], Página 117).

14.7 Comandos Genéricos de Definição

O Texinfo fornece comandos para definições que não produzem entradas automáticas de índice.

Você cria um ambiente genérico de definição com ‘@defblock’ emparelhado com ‘@end defblock’. Nesse ambiente, use uma linha @define ou @deftypeline para cada símbolo que você documentar. Por exemplo:

```
@defblock
@define Macro mac (arg1, arg2)
  Descrição de mac.
@deftypeline Builtin int foo (int @var{bar})
  Descrição de foo.
@end defblock
```

Isso produz a saída:

```
mac (arg1, arg2) [Macro]
  Descrição de macro.
int foo (int bar) [Builtin]
  Descrição de foo.
```

A sintaxe de @define é semelhante àquela de @defn. O primeiro argumento fornece uma categoria para a definição. Siga isso com o nome do símbolo, seguido por quaisquer parâmetros. Você deveria envolver com chaves quaisquer argumentos contendo espaços.

Você usa @deftypeline de maneira semelhante a @deftypefn, seguindo a categoria com um tipo de dado e marcando quaisquer parâmetros com @var. (Veja-se Seção 14.6.2 [Funções Tipadas], Página 117).

Para compartilhar a mesma descrição para vários símbolos, você pode juntar várias linhas @define. Por exemplo:

```
@defblock
@define Função set-var (valor)
@define {Variável Configurável} var
  Descrição de set-var e var.
@end defblock
```

Isso produz a saída:

```
set-var (value) [Function]
var [Settable Variable]
  Descrição de set-var e de var.
```

Possivelmente seja útil definir macros de linha (veja-se Seção 17.5 [Macros de Linha], Página 144) em combinação com esses comandos.

14.8 Convenções para Escrita de Definições

Um manual não precisa e não deveria conter mais que uma definição para um determinado nome. Um apêndice contendo um resumo deveria usar @table em vez dos comandos de definição.

Quando você escrever uma definição usando @defn, @defun ou um dos outros comandos de definição, por favor, tome cuidado para usar argumentos que indiquem o significado, como com o argumento *count* para a função **forward-word**. Além disso, se o nome de um argumento contiver o nome de um tipo, como *integer*, tome cuidado para que o argumento seja realmente desse tipo.

Fontes. Como Texinfo é uma linguagem semântica, você quase nunca precisará especificar estilos de fonte com comandos explícitos de fonte em definições (veja-se Seção 6.2.3 [Fontes],

Página 63). No entanto, você possivelmente precise contornar problemas com formatos de saída e (ou) construções específicos. Aqui estão algumas possibilidades:

- Marcando explicitamente uma palavra-chave da Lisp, como ‘&keyword’, com `@r{&keyword}`, produzindo &keyword.

Observe que tais palavras-chave em argumentos de definição são (atualmente) renderizadas em romano no T_EX, mas essa formatação não é feita em nenhum outro formato de saída.

- ‘`@r{@slanted{argumento}}`’, produzindo *argumento*, semelhante a `@var`, mas evitando maiusculizar o argumento dele na saída do Info.

Neste e no ponto anterior, `@r` reconfigura a fonte daquela sendo usada no contexto da linha de definição (inclinada ou de máquina de escrever) para um estilo romano vertical.

- Você poderia usar `@t` (ou até mesmo ‘`@r{@t{...}`’, usando `@r` para reconfigurar estilos de fonte) para marcar sintaxe literal em uma linha de definição onde, de outra maneira, o texto seria gerado no estilo romano inclinado. `@code` seria inapropriado aqui, pois ele adiciona aspas na saída do Info.

Algun grau de tentativa e erro possivelmente seja necessário para obter o resultado que você quer. Como sempre, como comandos aninhados de fonte se combinam depende do formato de saída, de forma que deveria ser evitada onde possível.

Esperançosamente, tais usos sejam mantidos ao mínimo. Uma possibilidade é a de envolvê-los em `@macro` (veja-se Capítulo 17 [Definindo Novos Comandos do Texinfo], Página 138), permitindo que esses usos sejam facilmente mudados no futuro.

14.9 Uma Definição de Amostra de Função

Aqui está uma definição oriunda do Seção “Calling Functions” em *Manual de Referência da Lisp do GNU Emacs*. O nome da função, `apply`, segue imediatamente depois do comando `@defun` e é seguido, na mesma linha, pela lista de parâmetros.

`apply` *função* &rest *argumentos* [Função]

`apply` chama *função* com *argumentos*, assim como `funcall`, mas com uma diferença: o último dos *argumentos* é uma lista de argumentos para fornecer para *função*, em vez de um argumento unitário. Nós também dizemos que essa lista é *posposta* aos outros argumentos.

`apply` retorna o resultado da chamada de *função*. Assim como com `funcall`, *função* precisa ser ou uma função da Lisp ou uma função primitiva; formas especiais e macros não fazem sentido em `apply`.

```
(setq f 'list)
⇒ lista
(apply f 'x 'y 'z)
[error] Argumento de tipo errado: listp, z
(apply '+ 1 2 '(3 4))
⇒ 10
(apply '+ '(1 2 3 4))
⇒ 10

(apply 'append '((a b c) nil (x y z) nil))
⇒ (a b c x y z)
```

Um exemplo interessante do uso de `apply` é encontrado na descrição de `mapcar`.

No arquivo fonte do Texinfo, esse exemplo deveria se parecer com isto:

`@defun` `apply` função &rest argumentos
`@code{apply}` chama `@var{função}` com `@var{argumentos}`, assim como `@code{funcall}`, mas com uma diferença: o último dos `@var{argumentos}` é uma lista de argumentos para fornecer para `@var{função}`, em vez de um argumento unitário. Nós também dizemos que essa lista é `@dfn{posposta}` aos outros argumentos.

`@code{apply}` retorna o resultado da chamada de `@var{função}`. Assim como com `@code{funcall}`, `@var{função}` precisa ser ou uma função da Lisp ou uma função primitiva; formas especiais e macros não fazem sentido em `@code{apply}`.

```
@example
(setq f 'list)
  @result{} lista
(apply f 'x 'y 'z)
@error{} Argumento de tipo errado: listp, z
(apply '+ 1 2 '(3 4))
  @result{} 10
(apply '+ '(1 2 3 4))
  @result{} 10

(apply 'append '((a b c) nil (x y z) nil))
  @result{} (a b c x y z)
@end example
```

Um exemplo interessante do uso de `@code{apply}` é encontrado na descrição de `@code{mapcar}`.
`@end defun`

Neste manual, essa função está listada no Índice de Comandos sob `apply`.

15 Internacionalização

Texinfo tem algum suporte para escrita em outros idiomas além do Inglês, embora essa área ainda precise de trabalho considerável. (Se você for um(a) dos(as) que ajudam a traduzir as sequências fixas de caracteres escritas em documentos, veja-se Seção 20.5 [Internacionalização de Sequências de Caracteres de Documentos], Página 178).

Para uma lista dos vários caracteres acentuados e especiais que Texinfo suporta, veja-se Seção 11.4 [Inserindo Acentos], Página 100.

15.1 @documentlanguage *ll[_cc]*: Configurar o Idioma do Documento

O comando `@documentlanguage` declara a localidade atual do documento. Escreva-o em uma linha dedicada, perto do começo do arquivo.

```
@documentlanguage ll[_cc]
```

Inclua um código de idioma ISO 639-2 de duas letras (ou de três letras) (*ll*) seguindo o nome do comando, opcionalmente seguido por um sublinhado e código de país ISO 3166 de duas letras ou de três letras (*cc*). Se você tiver um documento multi idioma, a intenção é a de poder usar esse comando várias vezes para declarar cada mudança de idioma.

Assim como com GNU Gettext (veja-se *Gettext*), se o código do país for omitido, o dialeto principal é assumido onde possível. Por exemplo, `de` é equivalente a `de_DE` (alemão como falado na Alemanha).

Esse comando muda a tradução de várias *sequências de caracteres de documento*, tais como “veja-se” em referências cruzadas (veja-se Capítulo 5 [Referências Cruzadas], Página 43), “Função” em defuns (veja-se Capítulo 14 [Comandos de Definição], Página 114), e assim por diante. Opcionalmente, esse comando possivelmente afete como `texi2any` ordena índices (veja-se Seção 20.4.4 [Variáveis Globais de Personalização], Página 168, sob `DOCUMENTLANGUAGE_COLLATION`). Se o idioma do documento não estiver especificado, as sequências padrão de caracteres de documento em inglês serão geradas como estiverem.

Observe que na saída Info, algumas sequências de caracteres, como “Node:”, “Next:”, “Menu:”, entre outras, são palavras-chave e portanto não são traduzidas.

Na saída gerada DocBook, `@documentlanguage` configura o idioma para as seções seguintes.

Para $\text{\LaTeX}$, esse comando faz com que o código carregue o pacote ‘`babel`’ para ser gerado no preâmbulo e `\selectlanguage` para ser gerado.

Para $\text{\TeX}$, esse comando faz com que um arquivo `txi-localidade.tex` seja lido (se ele existir). Se o argumento `@documentlanguage` contiver o sufixo opcional ‘`_cc`’, esse será tentado primeiro. Por exemplo, com `@documentlanguage de_DE`, $\text{\TeX}$ primeiro procura por `txi-de_DE.tex`, depois por `txi-de.tex`.

Esse arquivo `txi-*` é destinado a redefinir as várias palavras em inglês usadas na saída do $\text{\TeX}$, como ‘Chapter’, ‘See’ e assim por diante. Nós estamos cientes de que palavras individuais como essas nem sempre podem ser traduzidas isoladamente, e que uma estratégia muito diferente seria exigida para sequências de caracteres ideográficos (entre outros). Ajuda para melhorar o suporte a idiomas do Texinfo é bem-vinda.

`@documentlanguage` também muda os padrões atuais de hifenização do $\text{\TeX}$, se o programa $\text{\TeX}$ sendo executado tiver o suporte necessário incluído. Isso geralmente não será o caso para `tex` em si, mas geralmente será o caso para distribuições atualizadas dos programas $\text{\TeX}$ estendidos `etex` (saída DVI) e `pdftex` (saída PDF). `texi2dvi` usará os $\text{\TeX}$ s estendidos se estiverem disponíveis (veja-se [Formatar com `texi2dvi`], Página 150).

Como as listas de códigos de idioma e códigos de país são atualizadas com relativa frequência, nós não tentamos listá-los aqui. Os códigos válidos de idioma estão na página inicial oficial do

ISO 639, <http://www.loc.gov/standards/iso639-2/>. Os códigos de país e o sítio oficial do ISO 3166 podem ser encontrados via http://en.wikipedia.org/wiki/ISO_3166.

15.2 @documentencoding codificação: Configurar Codificação de Entrada

Por padrão, a codificação dos documentos de entrada e de saída é assumida ser UTF-8, a vasta codificação global de caracteres, expressa em bytes de 8 bits. UTF-8 é compatível com ASCII de 7 bits. É recomendado usar codificação UTF-8 para os manuais do Texinfo.

O comando `@documentencoding` declara a codificação de entrada do documento e também afeta a codificação da saída. Escreva-o em uma linha dedicada, com uma especificação válida de codificação seguindo, perto do início do arquivo se tua codificação de documento não for a codificação padrão.

```
@documentencoding codificação
```

UTF-8 deveria ser sempre a melhor escolha para a codificação. O Texinfo ainda suporta codificações adicionais, principalmente para compatibilidade com manuais mais antigos¹:

US-ASCII Codificação de caracteres baseada no alfabeto inglês.

ISO-8859-1

ISO-8859-15

ISO-8859-2

Essas especificam as codificações padrão pré UTF-8 para idiomas da Europa Ocidental (os dois primeiros) e da Europa Oriental (o terceiro), respectivamente. A ISO 8859-15 substitui alguns caracteres pouco usados provenientes da 8859-1 (por exemplo, frações pré-compostas) por outros mais comumente necessários, como o símbolo do Euro (€).

Uma descrição completa das codificações está além do nosso escopo aqui; uma referência útil é <http://czyborra.com/charsets/iso8859.html>.

`koi8-r` Essa foi a codificação comumente usada para o idioma russo antes da UTF-8.

`koi8-u` Essa foi a codificação comumente usada para o idioma ucraniano antes da UTF-8.

Na saída do Info, uma seção assim chamada ‘Local Variables’ (veja-se Seção “File Variables” em *O Manual do GNU Emacs*) é gerada incluindo a codificação da saída gerada. Isso permite que leitores Info configurem a codificação apropriadamente. Parece com algo assim:

```
Local Variables:
coding: UTF-8
End:
```

Por padrão, para saídas Info e de texto simples, `texi2any` gera construções de acentos e caracteres especiais (como `@'e`) como a sequência UTF-8 real ou caractere de 8 bits na codificação de saída, quando possível. Se isso não for possível, ou se a opção `--disable-encoding` for fornecida, uma transliteração ASCII será usada.

Na saída HTML, uma etiqueta ‘`<meta>`’ é gerada, na seção ‘`<head>`’ do HTML, que especifica a codificação da saída gerada. Servidores web e navegadores cooperam para usar essas informações, de modo que a codificação correta seja usada para exibir a página, se suportada pelo sistema. Isso se parece com isto:

```
<meta http-equiv="Content-Type" content="text/html;
      charset=utf-8">
```

¹ `texi2any` suporta mais codificações para manuais do Texinfo, potencialmente todas as codificações suportadas por Perl e `iconv` (veja-se Seção “Generic Charset Conversion” em *A Biblioteca C GNU*). No entanto, o suporte em formatos de saída possivelmente seja insuficiente, especialmente para a saída `LaTeX`.

Na saída HTML e $\LaTeX$, se `OUTPUT_CHARACTERS` estiver configurada (veja-se Seção 20.4.8 [Variáveis de Personalização para Múltiplos Formatos], Página 176), constructos de acentos e caracteres especiais, como `@'e` ou ````, serão gerados como a sequência UTF-8 real ou caractere de 8 bits na codificação da saída gerada, onde possível. Caso contrário, entidades HTML serão usadas para esses caracteres no HTML, e macros $\LaTeX$ serão usadas em $\LaTeX$.

Na saída DocBook, se a codificação for diferente de UTF-8, um atributo `encoding` será adicionado à declaração XML. Se `OUTPUT_CHARACTERS` estiver configurada (veja-se Seção 20.4.8 [Variáveis de Personalização para Múltiplos Formatos], Página 176), constructos de acentos, como `@'e`, serão geradas como o caractere de 8 bits ou UTF-8 real na codificação da saída gerada, onde possível. Caso contrário, entidades XML serão usadas para esses constructos.

Na saída $\TeX$, os caracteres que sejam suportados nas fontes Computer Modern padrão são produzidos adequadamente. Por exemplo, isso significa usar acentos construídos em vez de glifos pré-compostos. Usar um caractere ausente gera uma mensagem de aviso, assim como especificar uma codificação não implementada.

Embora os sistemas modernos $\TeX$ suportem quase todos os conjuntos de comandos sequenciados em uso no mundo, esse suporte abrangente não está disponível no `texinfo.tex`, e não é viável duplicar ou incorporar todo esse esforço.

Na saída do $\LaTeX$, o código que carrega o pacote `'inputenc'` é gerado baseado na codificação. Isso, por si só, não garante que todos os caracteres provenientes do documento de entrada possam ser gerados posteriormente. As fontes usadas no caso padrão deveriam abranger os glifos específicos do Texinfo, mas nem todos os possíveis caracteres codificados. Você possivelmente precise carregar fontes diferentes no preâmbulo e usar `\DeclareUnicodeCharacter` com uma codificação UTF-8. Por exemplo:

```
@latex
\DeclareUnicodeCharacter{017B}{\.\Z}
@end latex
```

Referências cruzadas entre arquivos Info em diferentes codificações de caracteres com caracteres não ASCII em nomes dos nós falham. Nós recomendamos fortemente usar somente UTF-8 como a codificação para manuais com caracteres não ASCII nos destinos das referências cruzadas.

16 Texto Condicionalmente Visível

Os *comandos condicionais* permitem que você use texto diferente para formatos de saída diferentes, ou para condições gerais que você define. Por exemplo, você consegue usá-los para especificar texto diferente para o manual impresso e para a saída Info.

Os comandos condicionais compreendem as seguintes categorias.

- Comandos específicos para um formato de saída (Info, T_EX, HTML, ...).
- Comandos específicos para qualquer formato de saída *excluindo* um fornecido (por exemplo, não Info, não T_EX, ...).
- Texto do formatador ‘bruto’ para qualquer formato de saída, passado diretamente com interpretação mínima (mas não nula) dos comandos @.
- Substituições de variáveis independentes de formato e testagem se uma variável estiver configurada ou limpa.

16.1 Comandos Condicionais

O Texinfo tem um ambiente `@ifformato` para cada formato de saída, para permitir inclusão condicional de texto para um formato de saída específico.

`@ifinfo` inicia segmentos de texto que deveriam ser ignorados quando não produzir saída Info, em particular em saída impressa. O segmento de texto aparece somente no arquivo Info e, para compatibilidade histórica, na saída de texto simples.

Os ambientes para os outros formatos são análogos:

```
@ifdocbook ... @end ifdocbook
```

Texto para aparecer somente na saída DocBook.

```
@ifhtml ... @end ifhtml
```

Texto para aparecer somente na saída HTML.

```
@iflatex ... @end iflatex
```

Texto para aparecer somente na saída L^AT_EX.

```
@ifplaintext ... @end ifplaintext
```

Texto para aparecer somente na saída de texto simples.

```
@iftex ... @end iftex
```

Texto para aparecer somente no manual impresso.

```
@ifxml ... @end ifxml
```

Texto para aparecer somente na saída XML.

Os comandos `@if...` e `@end if...` precisam aparecer em linhas dedicadas no teu arquivo fonte. As quebras de linha que seguem os comandos são (mais ou menos) tratadas como espaços em branco, de forma que o texto condicional seja fluído normalmente para um parágrafo circundante.

As construções `@if...` são destinadas para condicionar fonte normal Texinfo; veja-se Seção 16.3 [Comandos Brutos do Formatador], Página 131, para usar comandos subjacentes de formatação diretamente.

Aqui está um exemplo mostrando todos esses Condicionais:

```
@iftex
```

```
Este texto aparecerá somente no manual impresso.
```

```
@end iftex
```

```
@ifinfo
```

```
No entanto, este texto aparecerá somente em Info e em texto simples.
```

```
@end ifinfo
```

```

@ifhtml
E este texto aparecerá somente em HTML.
@end ifhtml
@iflatex
Além disso, este texto aparecerá somente em @LaTeX{}.
@end iflatex
@ifplaintext
Enquanto este texto aparecerá somente em texto simples.
@end ifplaintext
@ifxml
Não obstante, isto aparecerá somente em XML.
@end ifxml
@ifdocbook
No entanto, isto aparecerá somente em DocBook.
@end ifdocbook

```

O exemplo precedente produz a seguinte linha:

Este texto aparecerá somente no manual impresso.

Observe que você vê somente uma das linhas de entrada, dependendo de qual versão do manual estiver lendo.

Em documentos complexos, você pode querer que o Texinfo emita uma mensagem de erro em alguns condicionais que nunca deveriam ser processados. O comando `@errormsg{texto}` fará isso; ele recebe um argumento, o texto da mensagem de erro.

Nós mencionamos `@errormsg{}` aqui, embora ele não esteja estritamente relacionado a condicionais, já que, na prática, é mais provável que ele seja útil nesse contexto. Tecnicamente, ele pode ser usado em qualquer lugar. Veja-se Seção 17.7 [Processadores Externos de Macro], Página 146, para uma ressalva relativa aos números de linha que `@errormsg` emite no T_EX.

16.2 Comandos Condicionais Não

Você pode especificar o texto a ser incluído em algum formato de saída *diferente* de um fornecido, com os ambientes `@ifnot...`:

```

@ifnotdocbook ... @end ifnotdocbook
@ifnothtml ... @end ifnothtml
@ifnotinfol ... @end ifnotinfol
@ifnotlatex ... @end ifnotlatex
@ifnotplaintext ... @end ifnotplaintext
@ifnottex ... @end ifnottex
@ifnotxml ... @end ifnotxml

```

O comando `@ifnot...` e o comando `@end` precisam aparecer em linhas dedicadas no teu arquivo fonte real.

Se o arquivo de saída estiver sendo feito no formato fornecido, a região será *ignorada*. Caso contrário, ela será incluída.

Existe uma exceção (para compatibilidade histórica): texto `@ifnotinfol` será omitido para ambas saídas, Info e de texto simples, não apenas Info. Para especificar texto que apareça somente em Info e não em texto simples, use `@ifnotplaintext`, assim:

```

@ifinfo
@ifnotplaintext
Isto estará no Info, mas não em texto simples.
@end ifnotplaintext
@end ifinfo

```

As regiões delimitadas por esses comandos são fontes Texinfo comuns, como em `@iftex`, e não fontes de formatador bruto, como em `@tex` (veja-se Seção 16.3 [Comandos Brutos do Formatador], Página 131).

16.3 Comandos Brutos do Formatador

Os condicionais `@if...` recém descritos precisam ser usados somente com fonte normal do Texinfo. Por exemplo, a maioria dos recursos do $\TeX$ simples não funcionará dentro do `@iftex`. O propósito do `@if...` é o de fornecer processamento condicional para fonte do Texinfo, não fornecer acesso para recursos de formatação subjacentes. Para isso, Texinfo fornece os assim chamados *comandos brutos de formatador*. Eles deveriam ser usados somente quando verdadeiramente exigidos (a maioria dos documentos não precisa deles).

O primeiro comando bruto de formatador é o `@tex`. Você consegue entrar completamente no $\TeX$ simples e usar ‘\’ nos comandos do $\TeX$, delineando uma região com os comandos `@tex` e `@end tex`. Todos os comandos e códigos de categoria do $\TeX$ simples são restaurados dentro de uma região do `@tex`. A única exceção é a de que o caractere `@` ainda introduz um comando, de forma que `@end tex` possa ser reconhecido. Os processadores do Texinfo não gerarão material em tal região, a menos que a saída $\TeX$ esteja sendo produzida.

Em casos complexos, você pode desejar definir novas macros do $\TeX$ dentro do `@tex`. Você precisa usar `\gdef` para fazer isso, não `\def`, porque as regiões do `@tex` são processadas em um grupo do $\TeX$. Se você precisar fazer várias definições, você pode desejar configurar `\globaldefs=1` (o valor dele será restaurado para zero como de costume quando o grupo terminar em `@end tex`, de forma que não causará problemas com o resto do documento).

Como um exemplo, aqui está uma equação exibida escrita em $\TeX$ simples:

```
@tex
$$ \chi^2 = \sum_{i=1}^N
    \left( \frac{y_i - (a + b x_i)}{\sigma_i} \right)^2 $$
@end tex
```

A saída desse exemplo aparecerá somente em um manual impresso. Se estiver lendo isso em um formato não gerado pelo $\TeX$, você não verá a equação que aparece no manual impresso.

$$\chi^2 = \sum_{i=1}^N \left(\frac{y_i - (a + bx_i)}{\sigma_i} \right)^2$$

Analogamente, você pode usar `@html ... @end html` para uma região de HTML bruto, `@docbook ... @end docbook` para uma região de DocBook bruto, `@latex ... @end latex` para uma região de $\LaTeX$ bruto e `@xml ... @end xml` para uma região de XML bruto.

O comportamento de novas linhas em regiões brutas é inespecificado.

Em todos os casos, no processamento bruto, `@` retém o mesmo significado que no restante do documento. Assim, os processadores do Texinfo precisam reconhecer e até mesmo executar, até certo ponto, o conteúdo das regiões brutas, independentemente do formato final da saída. Portanto, especificar mudanças que afetem globalmente o documento dentro de uma região bruta conduz comportamento imprevisível e geralmente indesejável. Por exemplo, usar o comando `@kbdinputstyle` dentro de uma região bruta é indefinido. O remédio é simples: não faça isso.

16.4 Condicionais Inline: `@inline`, `@inlineifelse`, `@inlineraw`

Texinfo fornece um conjunto de comandos condicionais com argumentos fornecidos entre chaves:

```
@inlinefmt{formato, texto}
```

Processa o *texto* do Texinfo se a saída *formato* estiver sendo gerada.

@inlinefmtifelse{*formato*, *texto-então*, *texto-docontrário*}

Processa o *texto-então* do Texinfo se a saída *formato* estiver sendo gerada; caso contrário, processa *texto-docontrário*.

@inlineraw{*formato*, *texto*}

Similar, porém para *texto* bruto (veja-se Seção 16.3 [Comandos Brutos do Formata-dor], Página 131).

Os nomes de *formato* suportados são:

docbook html info latex plaintext tex xml

Por exemplo,

@inlinefmt{html, @emph{texto somente HTML}}

é quase equivalente a

@ifhtml

@emph{texto somente HTML}

@end ifhtml

exceto que nenhum espaço em branco é adicionado, como acontece no último caso (ambiente).

Nesses comandos, o espaço em branco é ignorado depois da vírgula que separa os argumentos, como de costume, mas *não* é ignorado no final de *texto*.

Para inserir um sinal de arroba literal, uma chave esquerda ou uma chave direita em um dos argumentos, você precisa usar os comandos alfabéticos **@atchar{}** (veja-se Seção 11.1.1 [Inserindo um Símbolo Arroba], Página 95) e **@lbracechar{}** ou **@rbracechar{}** (veja-se Seção 11.1.2 [Inserindo Chaves], Página 95), ou a análise se tornará confusa.

Com **@inlinefmtifelse**, também é necessário usar **@comma{}** para evitar confundir uma ‘,’ no texto com o delimitador. Com **@inlinefmt** e **@inlineraw**, **@comma{}** não é exigido (embora seja aceitável usá-lo), pois esses comandos sempre tem exatamente dois argumentos.

Para T_EX, o *texto* processado não pode conter comandos delimitados por nova linha. O texto a ser ignorado (por exemplo, para não-T_EX) pode, no entanto.

Dois outros condicionais **@inline...** complementam os comandos **@ifset** e **@ifclear**; veja-se a próxima seção.

16.5 Sinalizadores: @set, @clear, condicionais e @value

Você pode direcionar os comandos de formatação do Texinfo para formatar ou para ignorar partes de um arquivo do Texinfo com os comandos **@set**, **@clear**, **@ifset** e **@ifclear**. Aqui está um resumo:

@set *sinalizador* [*valor*]

Configura a variável *sinalizador* para o *valor* opcional, se especificado.

@clear *sinalizador*

Desdefine a variável *sinalizador*, se ou não ela foi definida anteriormente.

@ifset *sinalizador*

Se *sinalizador* estiver configurado, o texto até o próximo comando **@end ifset** será formatado. Se *sinalizador* estiver limpo, o texto até o comando **@end ifset** seguinte será ignorado.

@inlineifset{*sinalizador*, *texto*}

Versão delimitada por chaves de **@ifset**.

@ifclear *sinalizador*

Se *sinalizador* estiver configurado, o texto até o próximo comando **@end ifclear** será ignorado. Se *sinalizador* estiver limpo, o texto até o comando **@end ifclear** seguinte será formatado.


```
@inlineifclear{sinalizador, texto}
```

Versão delimitada por chaves de `@ifclear`.

caracteres.

16.5.1 @set e @value

Você usa o comando `@set` para especificar um valor para um sinalizador, que posteriormente será expandido pelo comando `@value`.

Um nome de *sinalizador* (também conhecido como *variável*) é um identificador que começa com um alfanumérico, um ‘-’ ou um ‘_’. Os caracteres subsequentes, se existirem, não podem ser espaços em branco, ‘@’, chaves, colchetes angulares ou quaisquer de ‘~^+|’; outros caracteres, como ‘%’, possivelmente funcionem. No entanto, é melhor usar somente letras e numerais em um nome de sinalizador, não ‘-’ ou ‘_’ ou outros—eles funcionarão em alguns contextos, mas não em todos, devido às limitações no T_EX.

O valor é o restante da linha de entrada e pode conter qualquer coisa.

Escreva o comando `@set` assim:

```
@set foo Esta é uma sequência de caracteres.
```

Isso configura o valor do sinalizador `foo` para “Esta é uma sequência de caracteres.”.

Os processadores do Texinfo então substituem um comando `@value{sinalizador}` pela sequência de caracteres para a qual *sinalizador* estiver configurado. Assim, quando `foo` for configurado como mostrado acima, os processadores do Texinfo convertem assim:

```
@value{foo}
```

para isto:

```
Esta é uma sequência de caracteres.
```

Você pode escrever um comando `@value` dentro de um parágrafo; mas precisa escrever um comando `@set` em uma linha própria.

Se você escrever o comando `@set` assim:

```
@set foo
```

sem especificar uma sequência de caracteres, o valor de `foo` será a sequência vazia de caracteres.

Se você limpar um sinalizador configurado anteriormente com `@clear sinalizador`, um comando `@value{sinalizador}` subsequente informará um erro.

Por exemplo, se você configurar `foo` conforme segue:

```
@set quanto muito, muito, muito
```

então os processadores transformam

```
É um dia @value{quanto} chuvoso.
```

into

```
É um dia muito, muito, muito chuvoso.
```

Se você escrever

```
@clear quanto
```

então os processadores transformam

```
É um dia @value{quanto} chuvoso.
```

into

```
É um dia {Nenhum valor para "quanto"} chuvoso.
```

`@value` não pode ser usado confiavelmente como o argumento para um comando de acento (veja-se Seção 11.4 [Inserindo Acentos], Página 100). Por exemplo, isto falha:

```
@set minhacarta a
@'@value{minhacarta}
```

16.5.2 @ifset e @ifclear

Quando um *senalizador* for configurado, os comandos de formatação do Texinfo formatam o texto entre pares subsequentes de comandos `@ifset senalizador` e `@end ifset`. Quando o *senalizador* for limpo, os comandos de formatação do Texinfo *não* formatam o texto. `@ifclear` opera analogamente.

Escreva o texto formatado condicionalmente entre comandos `@ifset senalizador` e `@end ifset`, assim:

```
@ifset senalizador
  texto-condicional
@end ifset
```

Por exemplo, você pode criar um documento que tenha duas variantes, como um manual para um modelo ‘grande’ e ‘pequeno’:

```
Você pode usar esta máquina para desenterrar arbustos
sem machucá-los.
```

```
@set grande
```

```
@ifset grande
Ela também pode desenterrar árvores adultas.
@end ifset
```

```
Lembre-se de replantar imediatamente ...
```

No exemplo, os comandos de formatação formatarão o texto entre `@ifset grande` e `@end ifset` porque o sinalizador *grande* está configurado.

Quando *senalizador* for limpo, os comandos de formatação do Texinfo *não* formatam o texto entre `@ifset senalizador` e `@end ifset`; esse texto é ignorado e não aparece na saída gerada.

Por exemplo, se você limpar o sinalizador do exemplo precedente escrevendo um comando `@clear grande` depois do comando `@set grande` (mas antes do texto condicional), então os comandos de formatação do Texinfo ignorarão o texto entre os comandos `@ifset grande` e `@end ifset`. Na saída formatada, esse texto não aparece; você vê somente as linhas que dizem, “Você pode usar esta máquina para desenterrar arbustos sem machucá-los. Lembre-se de replantar imediatamente ...”.

Se um sinalizador for limpo com um comando `@clear senalizador`, então os comandos de formatação formatam o texto entre pares subsequentes de comandos `@ifclear` e `@end ifclear`. Mas se o sinalizador for configurado com `@set senalizador`, então os comandos de formatação *não* formatam o texto entre um comando `@ifclear` e um `@end ifclear`; em vez disso, eles ignoram esse texto. Um comando `@ifclear` se parece com isto:

```
@ifclear senalizador
```

16.5.3 @inlineifset e @inlineifclear

`@inlineifset` e `@inlineifclear` fornecem alternativas delimitadas por chaves para as formas `@ifset` e `@ifclear`, similares aos outros comandos `@inline...` (veja-se Seção 16.4 [Condicionais Inline], Página 131). As mesmas ressalvas acerca de análise de argumentos fornecidas ali se aplicam aqui também.

```
@inlineifset{variável, texto}
```

Processa o *texto* do Texinfo se o sinalizador *variável* estiver definido.

```
@inlineifclear{variável, texto}
```

Processa o *texto* do Texinfo se o sinalizador *variável* não estiver definido.

Exceto para a sintaxe, o comportamento geral e propósitos deles são os mesmos de `@ifset` e `@ifclear`, descritos na seção anterior.

16.5.4 Exemplo de `@value`

Você pode usar o comando `@value` para minimizar o número de lugares que precisa mudar ao registrar uma atualização para um manual. Veja-se Seção C.1 [Textos GNU de Amostra], Página 249, para o texto completo de um exemplo de uso disso para trabalhar com distribuições Automake.

Este exemplo foi adaptado a partir do *Manual do GNU Make*.

1. Configura os sinalizadores:

```
@set EDITION 0.35 Beta
@set VERSION 3.63 Beta
@set UPDATED 14 de agosto de 1992
@set UPDATE-MONTH Agosto de 1992
```

2. Escreva texto para a seção `@copying` (veja-se Seção 2.7.1 [`@copying`], Página 16):

```
@copying
Esta é a Edição @value{EDITION},
atualizada mais recentemente em @value{UPDATED},
do @cite{Manual do GNU Make},
para @code{make}, versão @value{VERSION}.
```

```
Copyright ...
```

```
Permissão é concedida ...
@end copying
```

3. Escreva texto para a página de título, para pessoas leitoras do manual impresso:

```
@documentinfo
@title GNU Make
@subtitle Um Programa para Direcionamento de Recompilação
@subtitle Edição @value{EDITION}, ...
@subtitle @value{UPDATE-MONTH}
@end documentinfo
```

(Em uma capa impressa, uma data listando o mês e o ano parece menos complicada que uma data listando o dia, bem como o mês e o ano).

4. Escreva texto para o nó Top, para as pessoas leitoras do arquivo Info:

```
@ifnottex
@node Top
@top Make

Esta é a Edição @value{EDITION},
atualizada mais recentemente em @value{UPDATED},
do @cite{Manual do GNU Make},
para @code{make}, versão @value{VERSION}.
@end ifnottex
```

Depois de você formatar o manual, as construções `@value` foram expandidas, de forma que a saída contém texto como este:

```
Esta é a Edição 0.35 Beta, atualizada mais recentemente em 14 de agosto de
1992, do `Manual do GNU Make', para `make', versão 3.63 Beta.
```

Quando atualizar o manual, você muda somente os valores dos sinalizadores; você não precisa editar as três seções.

16.6 Testagem para Comandos do Texinfo: `@ifcommanddefined`, `@ifcommandnotdefined`

Ocasionalmente, você possivelmente queira providenciar para que teu manual teste se um dado comando do Texinfo está disponível e (presumivelmente) fazer algum tipo de formatação residual se não estiver. Existem condicionais `@ifcommanddefined` e `@ifcommandnotdefined` para fazer isso. Por exemplo:

```
@ifcommanddefined node
Bom, @samp{@@node} está definido.
@end ifcommanddefined
```

gerará o esperado ‘Bom, ‘@node’ está definido.’.

Esse condicional também considerará verdadeiros quaisquer novos comandos definidos pelo documento via `@macro`, `@alias`, `@definfoenclose` e `@def(code)index` (veja-se Capítulo 17 [Definindo Novos Comandos do Texinfo], Página 138). Ressalva: a implementação do $\TeX$ informa comandos internos do $\TeX$, além de todos os comandos do Texinfo, como sendo “definidos”; a implementação do `texi2any` é confiável a esse respeito, no entanto.

Você pode verificar o arquivo `NEWS` na distribuição do fonte do Texinfo e lincado a partir da página inicial do Texinfo (<https://www.gnu.org/software/texinfo>) para ver quando um comando específico foi adicionado.

Esses condicionais de verificação de comando foram adicionados eles próprios no Texinfo 5.0, lançado em 2013—décadas depois do início do Texinfo. Para a finalidade de testar se eles próprios estão disponíveis, o sinalizador predefinido `txicommandconditionals` pode ser testado, assim:

```
@ifset txicommandconditionals
@ifcommandnotdefined foobarnode
(Bom, @samp{@@foobarnode} não está definido).
@end ifcommandnotdefined
@end ifset
```

Como os sinalizadores (veja-se a seção anterior) foram adicionados no início da existência do Texinfo, não existe problema em supor que eles estejam disponíveis.

Nós recomendamos evitar esses testes sempre que possível—o que geralmente é o caso. Para muitos pacotes de software, é razoável que todos os(as) desenvolvedores(as) tenham uma determinada versão do Texinfo (ou mais recente) instalada, e, portanto, não existe razão para se preocupar com versões mais antigas. (É simples para qualquer um(a) baixar e instalar o fonte do Texinfo; ele não tem quaisquer dependências problemáticas).

O problema das versões do Texinfo geralmente não surge para usuários(as) finais. Com pacotes distribuídos corretamente, os(as) usuários(as) não precisam processar o manual do Texinfo simplesmente para construir e instalar o pacote; eles(as) conseguem usar arquivos pré-formatados de saída do Info (ou outros). Isso é desejável em geral, para evitar dependências desnecessárias entre pacotes (veja-se Seção “Releases” em *Padrões GNU de Codificação*).

16.7 Aninhamento de Condicional

O Texinfo exige que, para um condicional com falha, o texto ignorado seja aninhado corretamente em relação a esse condicional com falha. Aqui está um exemplo:

```
@ifset algumavarariável
@ifset outravarariável
Ambas, algumavarariável e outravarariável estão configuradas.
```

```
@end ifset
@ifclear outravarariável
    algumavarariável está configurada, outravarariável não está.
@end ifclear
@end ifset
```

Se ‘**algumavarariável**’ não estiver configurada, o bloco inteiro é ignorado.

Para permitir que os processadores determinem com segurança quais comandos considerar para fins de aninhamento, todos os comandos condicionais precisam estar em linhas separadas, sem texto (nem mesmo espaços) antes ou depois.

17 Definindo Novos Comandos do Texinfo

Texinfo fornece várias maneiras para definir novos comandos (em todos os casos, não é recomendado tentar redefinir comandos existentes):

- Uma *macro* do Texinfo permite que você defina um novo comando do Texinfo como qualquer sequência de texto e (ou) comandos existentes (incluindo outras macros). A macro pode ter qualquer número de *parâmetros*—texto que você fornece cada vez que usar a macro.
Aliás, essas macros não tem nada a ver com o comando `@defmac`, que serve para documentar macros na área de assunto do manual (veja-se Seção 14.1 [Modelo de Comando de Definição], Página 114).
- ‘`@linemacro`’ permite que você defina uma macro cujos argumentos se estendem até o final da linha e sejam separados por espaços.
- ‘`@alias`’ define um novo nome para um comando existente.
- ‘`@definfoenclose`’ permite que você defina novos comandos com saída personalizada para alguns formatos de saída não impressos.

De modo geral (não apenas para definir novos comandos), é possível invocar qualquer processador externo de macro e fazer com que o Texinfo reconheça as assim chamadas diretivas `#line` para informes de erros.

Se você quiser fazer substituição simples de texto, `@set` e `@value` são a abordagem mais simples (veja-se Seção 16.5 [`@set @clear @value`], Página 132).

17.1 Definindo Macros

Você usa o comando `@macro` do Texinfo para definir uma macro, como isto:

```
@macro nomemacro{parâmetro1, parâmetro2, ...}
  texto ... \parâmetro1\ ...
@end macro
```

Os *parâmetros* *parâmetro1*, *parâmetro2*, ... correspondem aos argumentos fornecidos quando a macro for posteriormente usada no documento (descritos na próxima seção).

Em princípio, *nomemacro* deveria consistir de alfanuméricos e (exceto como primeiro caractere) ‘-’. O caractere ‘_’ é excluído de forma que macros possam ser chamadas dentro de `@math` sem um espaço seguinte (veja-se Capítulo 12 [Inserindo Fórmulas Matemáticas], Página 109). No entanto, para uma macro funcionar consistentemente com $\TeX$, *nomemacro* precisa consistir inteiramente de letras: sem dígitos, hifens ou outros caracteres especiais. Portanto, nós recomendamos usar somente letras. Os comandos do Texinfo não deveriam ser redefinidos como macros.

Se uma macro não precisar de parâmetros, você pode defini-la ou com uma lista vazia (`@macro foo {}`) ou sem chaves em absoluto (`@macro foo`).

A definição ou *corpo* da macro pode conter a maioria dos comandos do Texinfo, incluindo invocações de macro. No entanto, uma definição de macro que defina outra macro não funciona no $\TeX$ devido a limitações no projeto do `@macro`.

No corpo da macro, instâncias de um nome de parâmetro cercado por barras invertidas, como em ‘`\parâmetro1\`’ no exemplo acima, são substituídas pelo argumento correspondente originário da invocação da macro. Você consegue usar nomes de parâmetros qualquer número de vezes no corpo, incluindo zero.

Para obter um ‘\’ simples na expansão da macro, use ‘`\\`’. Qualquer outro uso de ‘\’ no corpo produz um aviso.

Os caracteres de nova linha depois da linha do `@macro` e antes da linha do `@end macro` são ignorados, isto é, não incluídos no corpo da macro. Todos os outros espaços em branco são tratados de acordo com as regras usuais do Texinfo.

Para permitir que uma macro seja usada recursivamente, isto é, em um argumento para uma chamada para si mesma, você precisa defini-la com ‘`@rmacro`’, assim:

```
@rmacro rmac {arg}
a\arg\b
@end rmacro
...
@rmac{1@rmac{texto}2}
```

Isso produz a saída ‘`alatextob2b`’. Com ‘`@macro`’ em vez de ‘`@rmacro`’, uma mensagem de erro é fornecida.

Você pode indefinir uma macro *foo* com `@unmacro foo`. Não é um erro indefinir uma macro que já esteja indefinida. Por exemplo:

```
@unmacro foo
```

17.2 Invocando Macros

Depois que uma macro estiver definida (veja-se a seção anterior), você pode *invocar* (usá-la) em teu documento assim:

```
@nomemacro {arg1, arg2, ...}
```

e o resultado será mais ou menos como se você digitasse o corpo de *nomemacro* naquele local. Por exemplo:

```
@macro foo {p, q}
Junto: \p\ & \q\
@end macro
@foo{a, b}
```

produz:

```
Junto: a & b.
```

Assim, os argumentos e parâmetros são separados por vírgulas e delimitados por chaves; qualquer espaço em branco depois (mas não antes) de uma vírgula é ignorado. As chaves são exigidas na invocação mesmo quando a macro não receba argumentos, consistente com outros comandos do Texinfo. Por exemplo:

```
@macro argless {}
Sem argumentos aqui.
@end macro
@argless{}
```

produz:

```
Sem argumentos aqui.
```

Passar argumentos de macro contendo vírgulas exige cuidado, pois vírgulas também separam os argumentos. Para incluir um caractere de vírgula em um argumento, o método mais confiável é o de usar o comando `@comma{}`. Outro método é o de envolver o argumento com ‘`@asis{...}`’. Para *texi2any*, você também consegue prepor um caractere de barra invertida, como em ‘`\,`’, mas isso não funciona com *T_EX*.

Nem sempre é necessário se preocupar com vírgulas. Para facilitar o uso de macros, duas regras para *aspeamento automático* estão implementadas:

1. Se uma macro receber somente um argumento, todas as vírgulas na invocação dela serão aspeadas por padrão. Por exemplo:

```
@macro TENTEME{texto}
@strong{TENTEME: \texto\}
@end macro
```

```
@TENTEME{Um recurso interessante, embora possa ser perigoso.}
```

produzirá a seguinte saída

TENTEME: Um recurso interessante, embora possa ser perigoso.

E, de fato, pode. Ou seja, não existe controle sobre o número de argumentos passados para macros de um argumento, de forma que seja cuidadoso(a) quando você invocá-las.

2. Se uma invocação de macro incluir outro comando (incluindo um invocação recursiva dela mesma), quaisquer vírgulas na(s) invocação(ões) de comando(s) aninhado(s) serão aspeadas por padrão. Por exemplo, em

```
@say{@strong{Sim, eu faço}, pessoa um}
```

a vírgula depois de ‘Sim’ é implicitamente aspeada. Aqui está outro exemplo, com uma macro recursiva:

```
@macro cat{a,b}
  \a\\b\
@end macro
```

```
@cat{@cat{foo, bar}, baz}
```

produzirá a sequência de caracteres ‘foobarbaz’.

3. Caso contrário, uma vírgula deveria ser explicitamente aspeada, como acima, para ser tratada como parte de um argumento.

A própria barra invertida pode ser aspeada em argumentos de macro com outra barra invertida. Por exemplo:

```
@nomemac {\bleh}
```

passará o argumento ‘\bleh’ para *nomemac*.

`texi2any` também reconhece sequências ‘\{’ e ‘\}’ para chaves, mas elas não são reconhecidas pela implementação em `TEX`. No entanto, raramente deveria existir necessidade delas, pois elas são necessárias somente quando um argumento de macro contiver chaves desbalanceadas.

Se uma macro for definida para receber exatamente um argumento, ela pode ser invocada sem quaisquer chaves, recebendo toda a linha depois do nome da macro como o argumento. Por exemplo:

```
@macro bar {p}
  Duas vezes: \p\ & \p\
@end macro
@bar aah
```

produz:

Duas vezes: aah & aah.

Nesses argumentos, não existe escapagem de caracteres especiais, de forma que cada ‘\’ representa ele mesmo.

Se uma macro for definida para receber mais que um argumento, mas for chamada com somente um (entre chaves), os argumentos restantes serão configurados para a sequência vazia de caracteres e nenhum erro será fornecido. Por exemplo:

```
@macro addtwo {p, q}
  Ambos: \p\\q\
@end macro
@addtwo{a}
```

produz simplesmente:

Ambos: a.

17.3 Detalhes e Ressalvas de Macro

Por projeto, expansão de macro não deveria acontecer nos seguintes contextos:

- linhas `@macro` e `@unmacro`;
- linhas `@if...`, incluindo `@ifset` e similares;
- `@set`, `@clear`, `@value`;
- linhas `@clickstyle`;
- linhas `@end` quando não existir expansão de macro no bloco comando.

Infelizmente, $\TeX$ pode fazer alguma expansão nessas situações, possivelmente ocasionando erros.

Também, algumas construções relacionadas a macro causam problemas com $\TeX$; algumas das ressalvas estão listadas abaixo. Portanto, se você obtiver erros relacionados a macro ao produzir a versão impressa de um manual, você pode tentar expandir as macros com `texi2any` invocando `texi2dvi` com a opção ‘-E’ (veja-se [Formatar com `texi2dvi`], Página 150). Ou, mais confiavelmente, evite completamente as macros do Texinfo e use uma linguagem projetada para processamento de macros, como M4 (veja-se Seção 17.7 [Processadores Externos de Macro], Página 146).

- Como mencionado anteriormente, os nomes de macro precisam consistir inteiramente de letras.
- Não é aconselhável redefinir nenhum nome de comando primitivo do $\TeX$, simples ou Texinfo, como uma macro. Infelizmente, esse é um conjunto de nomes grande e aberto, e os possíveis erros resultantes são imprevisíveis.
- Argumentos para macros recebendo mais que um argumento não podem cruzar linhas.
- Macros contendo um comando que precise estar em uma linha própria, como um condicional, não podem ser invocadas no meio de uma linha. Similarmente, macros contendo comandos orientados a linha ou texto, como ambientes `@example`, possivelmente se comportem imprevisivelmente no $\TeX$.
- Comandos do Texinfo na expansão de uma macro no texto de uma entrada de índice podem acabar sendo tipografados como texto literal (incluindo um sinal “@”), em vez de serem interpretados com o significado pretendido deles.
- Espaços em branco são ignorados no início das linhas.
- Macros não podem ser usadas confiavelmente no argumento para comandos de acento (veja-se Seção 11.4 [Inserindo Acentos], Página 100).
- Macros deveriam não serem usadas em nomes de nós. Veja-se Seção 3.2 [Nomes de Nó], Página 27.
- O escape de barra invertida para vírgulas em argumentos de macro não funciona; `@comma{}` precisa ser usado.
- Terminar um corpo de macro com ‘`@c`’ pode fazer com que o texto seguinte a invocação da macro seja ignorado como um comentário no `texi2any`. Esse não é o caso ao processar com $\TeX$. Isso frequentemente era feito para “comentar” uma quebra de linha indesejada no final de um corpo de macro, mas isso não mais é necessário, pois, de qualquer maneira, a quebra de linha final antes de ‘`@end macro`’ não é incluída no corpo da macro.
- Em geral, você não pode substituir arbitrariamente uma chamada de macro (ou `@value`) por argumentos de comando do Texinfo, mesmo quando o texto for o mesmo. Texinfo não é M4 (nem mesmo $\TeX$ simples). Pode funcionar com alguns comandos, mas falha com outros. É melhor não fazer isso em absoluto. Por exemplo, isto falha:

```
@macro offmacro
off
@end macro
```

```
@headings @offmacro
```

Isso parece equivalente a `@headings off`, mas por motivos TeXnológicos, falha com uma misteriosa mensagem de erro (ou seja, ‘Parágrafo terminou antes que `@headings` estivesse completo’).

- Expandir macros em um contexto matemático pode conduzir a um espaçamento inadequado na saída do TeX. Por exemplo:

```
@iftex
@macro atan
  \mathop{\rm atan}
@end macro
@end iftex
```

```
@math{@atan{}}(x)
```

As chaves seguintes a ‘`@atan`’ são tratadas pelo TeX como uma subfórmula, conduzindo a espaço extra antes do parêntese de abertura. Você consegue contornar isso definindo uma macro bruta do TeX:

```
@tex
\gdef\atan#1{\mathop{\rm atan}}
@end tex
```

O ‘`#1`’ nessa definição absorve as chaves, de forma que o espaçamento seja o esperado para o objeto `\mathop`.

- Se você tiver problemas usando condicionais dentro de uma macro, uma alternativa é a de usar definições separadas de macro dentro de blocos condicionais. Por exemplo, em vez de

```
@macro Mac
@iftex
  texto para saída do TeX
@end iftex
@ifnottex
  texto para saída não do TeX
@end ifnottex
@end macro
```

você pode fazer o seguinte:

```
@iftex
@macro Mac
  texto para saída do TeX
@end macro
@end iftex

@ifnottex
@macro Mac
  texto para saída não do TeX
@end macro
@end ifnottex
```

- Macros não conseguem definir macros na maneira natural. Para fazer isso, você precisa usar condicionais e TeX bruto. Por exemplo:

```
@ifnottex
@macro ctor {nome, arg}
@macro \nome\
  algo envolvendo \arg\ de alguma forma
```

```

@end macro
@end macro
@end ifnottex
@tex
\gdef\ctorx#1{\ctorx#1,}
\gdef\ctorx#1,#2,{\def#1{algo envolvendo #2 de alguma forma}}
@end tex

```

As seguintes limitações são por projeto:

- Se você quiser passar um argumento com o comando @, do Texinfo (para produzir uma cedilha, veja-se Seção 11.4 [Inserindo Acentos], Página 100), você tem de usar @value ou outra solução alternativa. Caso contrário, a vírgula pode ser usada como separadora dos argumentos. Com `texi2any`, a vírgula pode ser escapada por uma barra invertida. Com `TeX`, outra solução alternativa precisa ser usada; portanto, nós recomendamos usar essa solução alternativa.

Por exemplo,

```

@macro mactwo{argprimeiro, argsegundo}
\argprimeiro\+\argsegundo\
@end macro
@set fc Fran@,cois
@mactwo{@value{fc},}

```

produz:

François+.

- `@verbatim` e macros não se misturam; por exemplo, você não pode iniciar um bloco verbatim dentro de uma macro e terminá-lo fora (veja-se Seção 7.5 [`@verbatim`], Página 68). Iniciar qualquer ambiente dentro de uma macro e terminá-lo fora pode ou não funcionar, para esse desiderato.
- Macros que definam macros completamente são corretas, mas não é possível ter definições de macro aninhadas incompletamente. Ou seja, `@macro` e `@end macro` (da mesma forma para `@rmacro`) precisam ser pareados corretamente. Por exemplo, você não pode iniciar uma definição de macro dentro de uma macro e, em seguida, terminar essa definição aninhada fora da macro.

Na implementação do `makeinfo` antes do Texinfo 5.0, os fins de linhas originários da expansão de uma definição do `@macro` não terminavam um argumento delimitado por linha do comando @ (`@chapter`, `@center`, entre outros). Esse não mais é o caso. Por exemplo:

```

@macro duaslinhas{}
aaa
bbb
@end macro
@center @duaslinhas{}

```

No `texi2any` atual, isso é equivalente a:

```

@center aaa
bbb

```

com apenas ‘aaa’ como o argumento para `@center`. Na implementação anterior, ele teria sido analisado assim:

```

@center aaa bbb

```

17.4 ‘@alias novo=existente’

O comando ‘@alias’ define um novo comando para ser exatamente como um existente. Isso é útil para definir nomes adicionais de marcação, preservando assim informações semânticas adicionais na entrada, mesmo que o resultado da saída possa ser o mesmo.

Escreva o comando ‘@alias’ em uma linha própria, seguido pelo novo nome do comando, um sinal de igual e o nome do comando existente. O espaço em branco ao redor do sinal de igual é opcional e ignorado se presente. Assim:

```
@alias novo = existente
```

Por exemplo, se teu documento contiver citações para livros e para alguma outra mídia (filmes, por exemplo), você pode querer definir uma macro @moviecite{} que faz a mesma coisa que um @cite{} comum, mas transmite as informações semânticas extras também. Você faria isso conforme segue:

```
@alias moviecite = cite
```

Macros nem sempre tem o mesmo efeito que apelidos, devido a caprichos da análise de argumentos. Além disso, apelidos são muito mais simples de definir que macros. Então o comando não é redundante.

Infelizmente, não é possível apelidar ambientes do Texinfo; por exemplo, @alias lang=exemplo é um erro.

Apelidos precisam não serem recursivos, direta ou indiretamente.

Não é aconselhável redefinir nenhum nome de comando do T_EX primitivo, do T_EX simples ou do Texinfo como um apelido. Infelizmente, esse é um conjunto muito grande de nomes, e os possíveis erros resultantes oriundos do T_EX são imprevisíveis.

Identificadores de apelidos deveriam ser os mesmos que para nomes de macro, isto é, alfa-numéricos e (exceto como o primeiro caractere) ‘-’. Entretanto, com o T_EX, somente letras deveriam ser usadas. Assim, nós recomendamos usar somente letras.

17.5 Macros de Linha

Você também pode definir uma macro usando o comando ‘@linemacro’. A sintaxe para definições de macros de linha é semelhante àquela para @macro.

No entanto, a sintaxe para usar uma macro definida dessa maneira é diferente. A chamada de macro usa o restante da linha, com argumentos separados por espaços. O recurso de macro de linha é destinado principalmente para permitir que você defina comandos que operem semelhantemente aos comandos internos para fornecer definições (veja-se Capítulo 14 [Comandos de Definição], Página 114).

Por exemplo:

```
@linemacro defbuiltin {nome, argumentos}
@define {Builtin} \nome\ \argumentos\
@end linemacro
```

A macro acima pode ser usada como

```
@defblock
@defbuiltin foo (bar)
Explicação
@end defblock
```

Isso produz o seguinte resultado:

foo (bar) Explicação [Builtin]

Aqui, o primeiro argumento é ‘foo’ e é substituído onde ‘\nome\’ aparecer no corpo da macro. Da mesma forma, ‘(bar)’ é substituído por ‘\argumentos\’.

Você consegue incluir espaços em um argumento de macro envolvendo o argumento entre chaves. Qualquer par de chaves circundando um argumento é removido antes da substituição. Quaisquer argumentos vazios (incluindo o último) precisam ser fornecidos como ‘{}’. Além disso, espaços não iniciais podem aparecer no argumento final na lista de argumentos sem chaves envolventes (como no exemplo abaixo).

Se uma linha de argumento para uma macro de linha terminar com um caractere ‘@’, então esse caractere, juntamente com a nova linha subsequente, será incluído em um argumento de macro, e a linha seguinte no arquivo de entrada também será usada para obter os argumentos para a macro. Observe que qualquer substituição da sequência ‘@<nova linha>’ precisa estar em um contexto válido, como uma linha de definição (veja-se Seção 14.2 [Linhas de Continuação de Comando de Definição], Página 115).

Outro exemplo:

```
@linemacro deffunc {tipo, nome, argumentos}
@findex \nome\
@deftypeline {Func} {\tipo\} \nome\ \argumentos\
@end linemacro

@defblock
@deffunc {long int} F (int @var{um}, int @var{dois}, @
int @var{três}, int @var{quatro}, int @var{cinco})
Explicação
@end defblock
```

Observe como chaves são necessárias em torno de ‘\tipo\’ no corpo da macro caso o argumento de `tipo` contenha um espaço. Esse exemplo produz a saída abaixo e insere uma entrada de índice para `F`.

```
long int F (int um, int dois, int três, int                                     [Func]
quatro, int cinco) Explicação
```

Macros de linha produzem um número inteiro de linhas de saída na expansão delas. Pode existir melhor compatibilidade entre os processadores Texinfo (`texi2any` e `TeX` com `texinfo.tex`) para macros definidas com `@linemacro` que para aquelas definidas com `@macro`.

17.6 @definfoenclose: Realçamento Personalizado

Um comando `@definfoenclose` pode ser usado para definir um comando de realçamento para formatos de saída em linha. Um comando definido usando `@definfoenclose` marca o texto circundando-o em sequências de caracteres que precedem e seguem o texto.

Na prática, existe pouco uso para esse comando e nós não recomendamos o uso dele. O suporte para `@definfoenclose` possivelmente seja removido em lançamentos futuros do Texinfo.

Escreva um comando `@definfoenclose` no início de uma linha seguido por três argumentos separados por vírgula. O primeiro argumento para `@definfoenclose` é o nome do comando @ (sem o @); o segundo argumento é a sequência de caracteres delimitadora inicial; e o terceiro argumento é a sequência de caracteres delimitadora final. Os dois últimos argumentos circundam o texto realçado na saída.

Uma sequência de caracteres delimitadora pode conter espaços. Nem o delimitador inicial nem o final são exigidos. Se você não quiser um delimitador inicial, mas quiser um delimitador final, você precisa seguir o nome do comando com duas vírgulas em uma linha; do contrário, a sequência de caracteres delimitadora final que você pretendia será naturalmente (mal)interpretada como a sequência de caracteres delimitadora inicial.

Um comando de circundação definido dessa maneira recebe um argumento entre chaves, pois é destinado para novos comandos de marcação (veja-se Capítulo 6 [Marcando Texto], Página 54).

Por exemplo, você pode escrever:

```
@definfoenclose phoo,/,\\
```

perto do início de um arquivo do Texinfo para definir `@phoo` como um comando do Info e do HTML que insira ‘//’ antes e ‘\\’ depois do argumento para `@phoo`. Você pode então escrever `@phoo{bar}` onde quiser ‘//bar\\’ realçado no Info e no HTML.

Para formatação do T_EX, você poderia escrever

```
@iftex
@alias phoo = i
@end iftex
```

para definir `@phoo` como um comando que faz com que o T_EX tipografe o argumento para `@phoo` em itálico.

Cada definição se aplica ao próprio formatador dela: uma para T_EX, o outra para formatos em linha. As definições do T_EX precisam estar em ‘@iftex’. O comando `@definfoenclose` não precisa estar em ‘@ifinfo’, a menos que você queira usar definições diferentes para diferentes formatos de saída em linha. Comandos definidos do `@definfoenclose` não tem efeito na saída DocBook e L^AT_EX; os argumentos deles são gerados como são. Um `@alias` também poderia ser usado para esses formatos.

Definições de ‘@definfoenclose’ precisam não serem recursivas, direta ou indiretamente.

17.7 Processadores Externos de Macro: Diretivas de Linha

Macros do Texinfo (e os outros recursos dele de substituição de texto) funcionam bem em casos simples. Se teu documento precisar de processamento incomumente complexo, no entanto, a fragilidade e limitações delas podem ser um problema. Nesse caso, você possivelmente queira usar um processador de macro completamente diferente, como M4 (veja-se *M4*) ou CPP (veja-se *O Preprocessador C*).

Com uma exceção, o Texinfo não precisa saber se a entrada dele é fonte “original” ou preprocessada a partir de algum outro arquivo fonte. Portanto, você consegue organizar teu sistema de construção para invocar quaisquer programas que goste para lidar com expansão de macro ou outras necessidades de pré-processamento. O Texinfo não oferece suporte interno para nenhum preprocessador específico, já que nenhum programa parecia ser suficiente para as exigências de todos os documentos.

A única exceção são os números de linha em mensagens de erro. Nesse caso, o número de linha deveria se referir ao arquivo fonte original, seja ele qual for. Existe um mecanismo bem conhecido para isso: a assim chamada diretiva ‘`#line`’. O Texinfo suporta isso.

17.7.1 Diretiva ‘`#line`’

Uma linha de entrada como esta:

```
#line 100 "foo.ptexi"
```

indica que a próxima linha foi linha 100 do arquivo `foo.ptexi`, e, portanto, isso é ao que uma mensagem de erro deveria se referir. Tanto M4 (veja-se Seção “Recursos de preprocessador” em *GNU M4*) quanto CPP (veja-se Seção “Controle de Linha” em *O Preprocessador C* e Seção “Saída de Preprocessador” em *O Preprocessador C*) conseguem gerar tais linhas.

O programa `texi2any` reconhece essas linhas por padrão, exceto dentro dos blocos `@verbatim` (veja-se Seção 7.5 [`@verbatim`], Página 68). O reconhecimento delas pode ser desativado completamente com `CPP_LINE_DIRECTIVES` (veja-se Seção 20.4.4 [Variáveis Globais de Personalização], Página 168), embora normalmente não exista razão para fazer isso.

Para aqueles poucos programas (M4, CPP, Texinfo) que precisem documentar diretivas ‘`#line`’ e, portanto, tem exemplos que, de outra forma, corresponderiam ao padrão, o comando

`@hashchar{}` pode ser usado (veja-se Seção 11.1.5 [Inserindo um Símbolo Cerquilha], Página 96). A linha de exemplo acima se parece com isto no fonte para este manual:

```
@hashchar{ }line 100 "foo.ptexi"
```

O comando `@hashchar` foi adicionado ao Texinfo em 2013. Se você não quiser depender dele, você também pode usar `@set` e `@value` para inserir o literal `#`:

```
@set hash #
@value{hash}line 1 "exemplo.c"
```

Ou, se adequado, um ambiente `@verbatim` pode ser usado em vez de `@example`. Como mencionado acima, o reconhecimento de `#line` é desabilitado dentro de blocos `verbatim`.

17.7.2 ‘#line’ e T_EX

Conforme mencionado, `texi2any` reconhece as diretivas `#line` descritas na seção anterior. No entanto, `texinfo.tex` não reconhece e não consegue reconhecer. Portanto, tal linha será incorretamente tipografada literal se T_EX a vir. A solução é a de usar as opções de expansão de macro do `texi2any` antes de se executar o T_EX. Existem três abordagens:

- Se você executar `texi2dvi` ou as variantes dele (veja-se [Formatar com `texi2dvi`], Página 150), poderá passar `-E` e `texi2dvi` executará `texi2any` primeiro para expandir macros e eliminar `#line`.
- Se você executar `texi2any`, poderá especificar `--no-ifinfo --iftex -E algumarquivo.out` e, em seguida, fornecer `algumarquivo.out` para `texi2dvi` em um comando separado.
- Ou você pode executar `texi2any --dvi --Xopt -E`. (Ou `--pdf` em vez de `--dvi`). `texi2any` então chamará `texi2dvi -E`.

Uma última ressalva relativa ao uso com T_EX: como as diretivas `#line` não são reconhecidas, os números de linha emitidos pelo comando `@errormsg{}` (veja-se Seção 16.1 [Comandos Condicionais], Página 129), ou pelo próprio T_EX, são os números de linha (incorretos) provenientes do arquivo derivado que o T_EX estiver lendo, em vez dos números de linha especificados pelo preprocessor.

17.7.3 Detalhes da Sintaxe da ‘#line’

Detalhes de sintaxe para a diretiva `#line`: o caractere `#` pode ser precedido ou seguido por espaço em branco, a palavra `line` é opcional, e o nome do arquivo pode ser seguido por uma lista de inteiros separados por espaços em branco (esses são os assim chamados “sinalizadores” gerados pelo CPP em alguns casos). Para aqueles(as) que gostam de saber os detalhes sangrentos, a expressão regular (Perl) real que é correspondida é esta:

```
/^\s*\#\s*(line)? (\d+)(( "[^"]+"))(\s+\d+)*?\s*$/
```

Até onde nós podemos dizer, os sinalizadores de inteiros finais somente ocorrem em conjunto com um nome de arquivo, de forma que isso é refletido na expressão regular.

Como exemplo, a seguir está uma diretiva `#line` sintaticamente válida, ou seja, a linha 1 de `/usr/include/stdio.h`:

```
# 1 "/usr/include/stdio.h" 2 3 4
```

Infelizmente, o nome de arquivo aspeado (“...”) tem que ser opcional, porque o M4 (especialmente) frequentemente pode gerar diretivas `#line` dentro de um arquivo. Como o `line` também é opcional, o resultado é que as linhas podem corresponder ao que você não esperaria, por exemplo,

```
# 1
```

As soluções possíveis estão descritas acima (veja-se Seção 17.7.1 [Diretiva `#line`], Página 146).

18 Arquivos de Inclusão

Quando um processador do Texinfo vê um comando `@include` em um arquivo do Texinfo, ele processa o conteúdo do arquivo nomeado pelo `@include` e o incorpora aos arquivos de saída sendo criados. Arquivos de inclusão permitem que você mantenha um documento grande como uma coleção de partes convenientemente pequenas.

18.1 Como Usar Arquivos de Inclusão

Para incluir outro arquivo em um arquivo do Texinfo, escreva o comando `@include` no começo de uma linha e siga-o na mesma linha pelo nome de um arquivo a ser incluído. Por exemplo:

```
@include buffers.texi
```

As referências a `@value{var}` são expandidas na linha `@include`. Além disso, os únicos comandos `@` permitidos são `@@`, `@{`, `@}` e comandos `@` associados, como `@atchar{}`.

Um arquivo incluído deveria ser simplesmente um segmento de texto que você espera que seja incluído como está no arquivo geral do Texinfo ou no *exterior*; ele não deveria conter as partes padrão de início e fim de um arquivo do Texinfo. Em particular, você não deveria iniciar um arquivo incluído com uma linha dizendo `\input texinfo`; se fizer isso, esse texto será inserido no arquivo de saída literalmente. Da mesma forma, você não deveria terminar um arquivo incluído com um comando `@bye`; nada depois de `@bye` é formatado.

No passado distante, você era obrigado(a) a escrever uma linha `@setfilename` no início de um arquivo incluído, mas não mais. Agora, não importa se você escrever tal linha. Se uma linha `@setfilename` existir em um arquivo incluído, ela será ignorada.

O modo Texinfo do GNU Emacs fornece `texinfo-multiple-files-update` para atualizar ponteiros de nós e menu mestre com vários arquivos de inclusão. Veja-se Seção D.5.3 [Atualizar Múltiplos Arquivos], Página 259.

dele; e um arquivo incluído de amostra.

18.2 Arquivo de Amostra com @include

Aqui está um exemplo de um arquivo externo do Texinfo com arquivos `@include` dentro dele:

```
\input texinfo @c -*-texinfo*-

@settitle Exemplo de Inclusão

@node Top
@top Exemplo de Inclusão

@include foo.texi
@include bar.texi
@include indice-conceitos.texi
@bye
```

Um arquivo incluído, como `foo.texi`, pode se parecer com isto:

```
@node Primeiro
@chapter Primeiro Capítulo

Conteúdo do primeiro capítulo ...
```

O conteúdo completo de `indice-conceitos.texi` pode ser tão simples quanto isto:


```
@node Índice de Conceito
@unnumbered Índice de Conceito
```

```
@printindex cp
```

O arquivo fonte externo do Texinfo para *O Manual de Referência da Lisp do GNU Emacs* é chamado `elisp.texi`. Esse arquivo externo já conteve um menu mestre com 417 entradas e uma lista de 41 arquivos `@include`.

18.3 *arquivo* `@verbatiminclude`: Incluir um Literal de Arquivo

Você pode incluir o conteúdo exato de um arquivo no documento com o comando `@verbatiminclude`:

```
@verbatiminclude nomearquivo
```

O conteúdo de *nomearquivo* é impresso em um ambiente literal (veja-se Seção 7.5 [`@verbatim`], Página 68). Geralmente, o arquivo é impresso exatamente como está, com todos os caracteres especiais e espaços em branco retidos. Nenhum recuo é adicionado; se você quiser recuo, circunde o `@verbatiminclude` com `@example` (veja-se Seção 7.4 [`@example`], Página 67).

referências `@value{variável}` são expandidas na linha `@verbatiminclude`. Isso torna possível incluir arquivos em outros diretórios dentro de uma distribuição, por exemplo:

```
@verbatiminclude @value{top_srcdir}/NEWS
```

(Você ainda tem de ter `top_srcdir` definido no primeiro lugar).

Fora isso, os únicos comandos `@` permitidos são `@@`, `@{`, `@}` e comandos `@` associados, como `@atchar{}`.

Para um método relativo a impressão do conteúdo do arquivo em um tamanho menor de fonte, veja-se o final da seção acerca do `@verbatim`.

19 Formatando e Imprimindo com T_EX

Executar o comando `texi2dvi` ou `texi2pdf` é a maneira mais simples de criar uma saída imprimível. Esses comandos são instalados como parte do pacote do Texinfo.

Em mais detalhes, tipografar e imprimir um arquivo do Texinfo é um processo de várias etapas no qual você usa o programa T_EX para criar um arquivo para impressão (chamado de arquivo DVI ou PDF) e então imprime o arquivo. Opcionalmente, você também pode criar índices usando o comando `texindex` depois da primeira execução do T_EX; e então você precisa executar o T_EX novamente. `texi2dvi` se encarrega de tudo isso, executando T_EX e `texindex` conforme necessário (veja-se [Formatar com `texi2dvi`], Página 150).

Ao usar os comandos do shell, você pode trabalhar diretamente no shell do sistema operacional ou dentro de um shell dentro do GNU Emacs (ou algum outro ambiente computacional). Você pode fornecer comandos de formatação e impressão a partir de um shell dentro do GNU Emacs, assim como qualquer outro comando de shell. Para criar um shell dentro do Emacs, digite `M-x shell` (veja-se Seção “Shell” em *O Manual do GNU Emacs*). Se estiver usando o GNU Emacs, você também pode usar comandos fornecidos pelo modo Texinfo em vez de comandos de shell. Veja-se Seção D.7 [Imprimindo com o Emacs], Página 263.

Para especificar detalhes da saída impressa, como tamanho do papel, veja-se Anexo E [Comandos Globais de Documento], Página 274.

19.1 Use T_EX

O programa de tipografar chamado T_EX é usado para formatar um documento do Texinfo para saída imprimível. T_EX é um programa de tipografia muito poderoso e, quando usado corretamente, faz um trabalho excepcionalmente bom. Ele não está incluso no pacote do Texinfo, sendo uma vasta suíte de software em si.

T_EX é um formatador de documento que é usado pela FSF para a documentação dela. É a maneira mais fácil de obter saída impressa (por exemplo, PDF e PostScript) para manuais do Texinfo. O T_EX é redistribuível livremente e você pode obtê-lo pela Internet ou em mídia física. Veja-se <http://tug.org/texlive>.

19.2 Formatar com `texi2dvi` ou `texi2pdf`

O programa `texi2dvi` se encarrega de todas as etapas para produzir um arquivo DVI do T_EX a partir de um documento do Texinfo. Similarmente, `texi2pdf` produz um arquivo PDF¹.

Para executar `texi2dvi` ou `texi2pdf` sobre um arquivo de entrada `foo.texi`, faça isto (onde ‘`prompt$`’ é o prompt do teu shell):

```
prompt$ texi2dvi foo.texi
prompt$ texi2pdf foo.texi
```

Conforme mostrado nesse exemplo, os nomes dos arquivos fornecidos para `texi2dvi` e `texi2pdf` precisam incluir alguma extensão, como ‘`.texi`’.

Para uma lista de todas as opções, execute ‘`texi2dvi --help`’. Algumas das opções são discutidas abaixo.

Com a opção `--pdf`, `texi2dvi` produz saída PDF em vez de DVI, executando `pdftex` em vez de `tex`. Alternativamente, o comando `texi2pdf` é uma abreviação para executar ‘`texi2dvi --pdf`’. O comando `pdftexi2dvi` também é fornecido como uma conveniência para AUC-T_EX (veja-se *AUC-T_EX*, pois ele prefere meramente antepor ‘`pdf`’ às ferramentas de produção de DVI para ter ferramentas de produção de PDF).

¹ PDF significa ‘Portable Document Format’. Foi inventado pela Adobe Systems para intercâmbio de documentos, baseado na linguagem PostScript deles(as).

Com a opção `--dvipdf`, `texi2dvi` produz saída PDF executando T_EX e então um programa DVI-para-PDF: se a variável de ambiente `DVIPDF` estiver configurada, esse valor será usado, senão o primeiro programa existente entre `dvipdfmx`, `dvipdfm`, `dvipdf`, `dvi2pdf`, `dvitopdf`. Esse método geralmente suporta melhor a tipografia CJK que o `pdftex`.

Com a opção `--ps`, `texi2dvi` produz PostScript em vez de DVI, executando `tex` e então `dvips` (veja-se *Dvips*). (Ou o valor da variável de ambiente `DVIPS`, se configurada).

`texi2dvi` também pode ser usado para processar arquivos do L^AT_EX. Normalmente, `texi2dvi` é capaz de adivinhar a linguagem do arquivo de entrada pelo conteúdo e extensão de nome de arquivo dele; no entanto, se ele adivinhar errado, você consegue especificar explicitamente a linguagem de entrada usando a opção de linha de comando `--language=linguagem`, onde *linguagem* é ou `'latex'` ou `'texinfo'`.

Uma opção útil para `texi2dvi` é `--command=comando`. Isso insere *comando* em uma linha própria no início de um arquivo em uma cópia temporária do arquivo de entrada, antes de executar o T_EX. Com isso, você pode especificar diferentes formatos de impressão, tais como `@smallbook` (veja-se Seção E.6 [`@smallbook`], Página 280), `@fourpaper` (veja-se Seção E.7 [Papel A4], Página 280) ou `@pagesizes` (veja-se Seção E.8 [`@pagesizes`], Página 280), sem realmente mudar o fonte do documento. (Você também pode fazer isso em todo o site com `texinfo.cnf`; veja-se Seção 19.4 [Preparando para T_EX], Página 154).

A opção `-E` (equivalentemente, `-e` e `--expand`) faz a expansão de macro do Texinfo usando `texi2any` em vez da implementação do T_EX (veja-se Seção 17.3 [Detalhes de Macro], Página 141). Cada implementação tem limitações e vantagens próprias delas.

`texi2dvi` recebe a opção `--build=modo` para especificar onde a compilação do T_EX acontece e, como consequência, como os arquivos auxiliares são tratados. O modo de construção também pode ser configurado usando-se a variável de ambiente `TEXI2DVI_BUILD_MODE`. Os valores válidos para *modo* são:

- `'local'` Compile no diretório atual, deixando todos os arquivos auxiliares por aí. Esse é o uso tradicional do TeX.
- `'tidy'` Compile em um diretório local `*.t2d`, onde os arquivos auxiliares são deixados. Os arquivos de saída são copiados de volta para o arquivo original.
Usar o modo `'tidy'` traz várias vantagens:
 - o diretório atual não fica abarrotado com uma abundância de arquivos temporários.
 - a desordem pode ser reduzida ainda mais usando-se `--build-dir=diretório`: todos os diretórios `*.t2d` ficam armazenados lá.
 - a desordem pode ser reduzida a zero usando-se, por exemplo, `--build-dir=/tmp/$USER.t2d` ou `--build-dir=/ $HOME/.t2d`.
 - o arquivo da saída gerada é atualizado depois de cada execução bem-sucedida do T_EX, por para fins de visualização concorrente da saída gerada. Em uma construção `'local'`, o visualizador para durante a execução inteira do T_EX.
 - se a compilação falhar, o estado anterior do arquivo da saída gerada será preservado.
 - As compilações de PDF e de DVI são mantidas em separado subdiretórios, evitando qualquer possibilidade de incompatibilidade dos arquivos auxiliares.

Por outro lado, como a compilação `'tidy'` ocorre em outro diretório, ocasionalmente o T_EX não conseguirá encontrar alguns arquivos (por exemplo, ao usar `\graphicspath`): nesse caso, use `-I` para especificar os diretórios adicionais a considerar.

- `'clean'` O mesmo que `'tidy'`, mas remove o diretório auxiliar depois. Cada compilação, portanto, exige o ciclo completo.

`texi2dvi` usará `etex` se estiver disponível, porque ele executa mais rápido em alguns casos e fornece informações adicionais de rastreamento ao depurar `texinfo.tex`. No entanto, essa versão estendida do T_EX não é exigida, e a saída DVI é idêntica.

`texi2dvi` tenta detectar arquivos auxiliares gerados pelo T_EX, seja usando a opção `-recorder` seja procurando por `'\openout'` no arquivo de registro que uma execução do T_EX produzir. Você pode controlar como `texi2dvi` faz isso com a variável de ambiente `TEXI2DVI_USE_RECORDER`. Os valores válidos são:

- `'yes'` use a opção `-recorder`, sem verificações.
- `'no'` procure por `'\openout'` no arquivo de registro, sem verificações.
- `'yesmaybe'` verifique se a opção `-recorder` é suportada e, se sim, use-a; caso contrário, verifique se o rastreamento de `'\openout'` no arquivo de registro é suportado e, se sim, use-o; caso contrário, será um erro.
- `'nomaybe'` o mesmo que `'yesmaybe'`, exceto que o rastreamento de `'\openout'` no arquivo de registro é verificado primeiro.

O padrão é `'nomaybe'`. Essa variável de ambiente é fornecida para fins de solução de problemas e possivelmente mude ou desapareça no futuro.

19.3 Formatar com `tex`/`texindex`

Você pode fazer a formatação básica de um arquivo do Texinfo com o comando de shell `tex` seguido pelo nome do arquivo do Texinfo. Por exemplo:

```
tex foo.texi
```

T_EX produzirá um *arquivo DVI* bem como muitos arquivos auxiliares contendo informações para índices, referências cruzadas, entre outras. O arquivo DVI (para arquivo *DeVice Independent*) pode ser impresso em praticamente qualquer dispositivo, talvez depois de uma conversão adicional (veja-se a seção anterior).

O comando de formatação `tex` em si não ordena os índices; ele escreve um arquivo de saída dos dados desordenados de índices. Para gerar um índice impresso depois de executar o comando `tex`, você primeiro precisa de um índice ordenado para trabalhar. O comando `texindex` ordena índices. (`texi2dvi`, descrito na seção anterior, executa `tex` e `texindex` conforme necessário).

`tex` gera arquivos desordenados de índices sob nomes seguindo uma convenção padrão: o nome do teu principal arquivo de entrada com alguma extensão `'.texi'` ou similar substituída pelo nome de duas letras do índice. Por exemplo, os arquivos brutos de saída de índices para o arquivo de entrada `foo.texi` seriam, por padrão, `foo.cp`, `foo.vr`, `foo.fn`, `foo.tp`, `foo.pg` e `foo.ky`. Esses são exatamente os argumentos a se fornecer para `texindex`.

Em vez de especificar-se explicitamente todos os nomes de arquivos desordenados de índices, é comum se usar `'??'` como curingas de shell e fornecer o comando nesta forma:

```
texindex foo.??
```

Esse comando executará `texindex` sobre todos os arquivos desordenados de índices, incluindo quaisquer índices de duas letras que você mesmo(a) tiver definido usando `@defindex` ou `@defcodeindex`. Você pode seguramente executar `'texindex foo.??'`, mesmo se existirem arquivos com extensões de duas letras que não sejam arquivos de índice, como `'foo.el'`. O comando `texindex` informa, mas, inobstante, ignora esses arquivos.

Para cada arquivo especificado, `texindex` gera um arquivo ordenado de índice cujo nome é feito anexando `'s'` ao nome de arquivo de entrada; por exemplo, `foo.cps` é feito a partir de `foo.cp`. O comando `@printindex` procura por um arquivo com esse nome (veja-se Seção 10.5

[Imprimindo Índices e Menus], Página 91). T_EX não lê o arquivo bruto de saída de índice e `texindex` não o altera.

Depois que tiver ordenado os índices, você precisa reexecutar `tex` sobre o arquivo do Texinfo. Isso regenera o arquivo de saída, dessa vez com entradas de índices atualizadas.

Por fim, você possivelmente precise executar `tex` mais uma vez para corrigir os números de página nas referências cruzadas.

Para resumir, este é um processo de cinco etapas. (Alternativamente, é um processo de uma etapa: execute `texi2dvi`; veja-se a seção anterior).

1. Execute `tex` sobre teu arquivo do Texinfo. Isso gera um arquivo DVI (com referências cruzadas indefinidas e sem índices) e os arquivos brutos de índices (com extensões de duas letras).
2. Execute `texindex` sobre os arquivos brutos de índices. Isso cria os correspondentes arquivos ordenados de índices (com extensões de três letras).
3. Execute `tex` novamente sobre teu arquivo do Texinfo. Isso regenera o arquivo DVI, dessa vez com índices e referências cruzadas definidas, mas com números de página para as referências cruzadas oriundos da execução anterior, geralmente incorretos.
4. Ordene os índices novamente, com `texindex`.
5. Execute `tex` uma última vez. Desta vez, os números de página corretos serão escritos para as referências cruzadas.

Para gerar PDF, você pode executar o programa `pdftex` em vez do `tex` simples. Ou seja, execute `pdftex foo.texi` em vez de ‘`tex foo.texi`’ nos exemplos acima.

19.3.1 Formatando Documentos Parciais

Ocasionalmente, você possivelmente deseje imprimir um documento quando sabe que ele está incompleto, ou imprimir apenas um capítulo de um documento. Nesse caso, os arquivos auxiliares usuais que o T_EX cria e os avisos que o T_EX fornece acerca de referências cruzadas indefinidas são apenas incômodos. Você consegue evitá-los com o comando `@novalidate`, que você precisa fornecer *antes* de quaisquer comandos de seccionamento ou de referência cruzada.

Assim, o início do teu arquivo aparentaria aproximadamente assim:

```
\input texinfo
@novalidate
...
```

`@novalidate` também desliga a validação no `texi2any`, exatamente como a opção `--no-validate` dele (veja-se Seção 20.1 [Invocando `texi2any`], Página 157).

Além disso, você não precisa executar `texindex` toda vez depois que executar `tex`. O comando de formatação `tex` simplesmente usa quaisquer arquivos ordenados de índices que existam provenientes de um uso anterior do `texindex`. Se eles estiverem desatualizados, isso geralmente é aceitável enquanto você estiver criando ou depurando um documento.

19.3.2 Detalhes do `texindex`

Na versão 6 do Texinfo, lançada em 2015, o programa `texindex` foi completamente reimplementado. A principal diferença funcional é a de que entradas de índice iniciando com uma chave esquerda ou chave direita (respectivamente ‘{’ e ‘}’) conseguem funcionar corretamente. Por exemplo, estas simples entradas de índice são processadas corretamente, incluindo o “index initial” mostrado no índice:

```
@cindex @{
@cindex @}
...
```

```
@printindex cp
```

Embora não uma questão de funcionalidade, os(as) leitores(as) possivelmente estejam interessados(as) em saber que o novo `texindex` é um programa letrado (https://en.wikipedia.org/wiki/Literate_programming) usando Texinfo para documentação e `awk` (portável) para código. Um arquivo fonte, `texindex/ti.twjr` nesse caso, produz o programa executável, um documento imprimível e um documento online.

O sistema é chamado TexiWeb Jr. e foi criado por Arnold Robbins, que também escreveu o novo `texindex`. Não coincidentemente, ele também é o mantenedor de longa data do `gawk` (GNU Awk, veja-se *O Guia do(a) Usuário(a) GNU Awk*). O arquivo `texindex/Makefile.am` mostra um exemplo de uso do sistema.

19.4 Preparando para T_EX

T_EX precisa encontrar o arquivo `texinfo.tex` que o comando ‘`\input texinfo`’ na primeira linha lê. O arquivo `texinfo.tex` informa ao T_EX como lidar com comandos @; Está incluído em todas as distribuições padrão do GNU. A versão mais recente lançada para uso geral está disponível a partir dos servidores GNU usuais e espelhos:

```
https://ftp.gnu.org/gnu/texinfo/texinfo.tex
https://ftpmirror.gnu.org/texinfo/texinfo.tex
```

A versão mais recente de desenvolvimento está disponível a partir do repositório de fonte do Texinfo:

```
https://git.savannah.gnu.org/cgit/texinfo.git/plain/doc/texinfo.tex
```

`texinfo.tex` essencialmente é um arquivo independente; portanto, se você precisar ou quiser tentar uma versão mais recente do que a que veio com o teu sistema, quase sempre basta baixá-la e colocá-la em algum lugar onde o T_EX a encontre. Você consegue substituir qualquer `texinfo.tex` existente por uma versão mais recente (certamente salvando a original em caso de desastre).

Além disso, você deveria instalar o `epsf.tex`, se ele já não estiver instalado, originário em outra distribuição. Mais detalhes estão no final da descrição do comando `@image` (veja-se Seção 9.2 [Imagens], Página 83).

Para usar aspas diferentes daquelas usadas em inglês, você precisará ter as fontes European Computer Modern (por exemplo, `ecrm1000`) e (para saída em PDF) as fontes CM-Super (veja-se Seção 11.5 [Inserindo Aspas], Página 101).

Para usar o comando `@euro`, você precisará das fontes ‘`feym*`’ (por exemplo, `feymr10`). Veja-se Seção 11.7.6 [`@euro`], Página 103.

Todos os arquivos acima deveriam estar instalados por padrão em uma instalação razoável do T_EX.

Opcionalmente, você pode criar um arquivo `texinfo.cnf` para configuração do sítio. Ao processar um arquivo do Texinfo, T_EX procura esse arquivo no caminho dele de pesquisa, que inclui o diretório atual e os diretórios padrão de instalação. Você pode usar esse arquivo para convenções locais. Por exemplo, se `texinfo.cnf` contiver a linha ‘`@afourpaper`’ (veja-se Seção E.7 [Papel A4], Página 280), então todos os documentos do Texinfo serão processados com esse tamanho de página em vigor. Se você não tiver nada para colocar no `texinfo.cnf`, não precisará criá-lo.

Você pode configurar a variável de ambiente `TEXINPUTS` para permitir que o T_EX encontre `texinfo.cnf`. (Isso também funciona para `texinfo.tex` e qualquer outro arquivo que o T_EX possa ler). Por exemplo, se você estiver usando um shell compatível com shell Bourne (`sh`, `bash`, `ksh`, ...), teu arquivo `.profile` poderia conter as linhas:

```
TEXINPUTS=./home/eu/minhabiblioteca:
export TEXINPUTS
```

Essas configurações fariam com que o T_EX primeiro procurasse por um arquivo `\input` no diretório atual, indicado pelo ‘.’, então um diretório `minhabiblioteca` de um(a) usuário(a) hipotético(a) ‘eu’ e finalmente nos diretórios do sistema. (Um ‘:’ à esquerda, à direita ou duplo indica a pesquisa nos diretórios do sistema nesse ponto).

19.5 “hboxes” lotados

T_EX às vezes é incapaz de tipografar uma linha dentro das margens normais. Isso ocorre com mais frequência quando o T_EX aparece no que interpreta como uma palavra longa que não consegue hifenizar, como um endereço de rede de intercomunicação de correio eletrônico ou um identificador muito longo. Quando isso acontece, T_EX imprime uma mensagem de erro como esta:

```
Overfull @hbox (20.76302pt too wide)
```

(No T_EX, as linhas estão em “caixas horizontais”, daí o termo, “hbox”. ‘@hbox’ é um primitivo do T_EX não usado na linguagem do Texinfo).

O T_EX também fornece o número da linha no arquivo fonte do Texinfo e no texto da linha ofensiva, que é marcada em todos os locais que o T_EX considerou a hifenização. Veja-se Seção D.10.3 [Depuração com T_EX], Página 270, para mais informações acerca de erros de tipografia.

Se o arquivo do Texinfo tiver um hbox lotado, você poderá reescrever a frase para que o hbox lotado não ocorra, ou poderá decidir deixá-lo. Uma pequena excursão na margem direita geralmente não importa e pode nem mesmo ser perceptível.

Se você tiver muitas caixas lotadas e (ou) uma antipatia por reescrever, poderá coagir o T_EX a aumentar bastante o espaçamento permitido interpalavras, assim (se você tiver sorte) evitando muitas das quebras ruins de linha, assim:

```
@tex
\global\emergencystretch = .9\hsize
@end tex
```

(Você deveria ajustar a fração conforme necessário). Esse enorme valor para `\emergencystretch` não pode ser o padrão, pois então a saída do tipador geralmente seria de qualidade visivelmente menor; o valor padrão dele é ‘.15\hsize’. `\hsize` é a dimensão do T_EX contendo a largura atual da linha.

Para qualquer caixa lotada que você tenha, T_EX imprimirá um retângulo grande, feio e preto, ao lado da linha que contém o hbox lotado, a menos que seja informado de outra forma. É assim que você notará o local do problema se estiver corrigindo um rascunho.

Para impedir que tal monstruosidade estrague tua impressão final, escreva o seguinte no início do arquivo do Texinfo em uma linha própria, antes do comando `@maketitle` ou do `@titlepage`:

```
@finalout
```

19.6 Imprimindo Japonês e Chinês

Texinfo vem com arquivos de suporte para processamento de entrada nos idiomas japonês e chinês.

Para japonês, você usa o arquivo `texinfo-ja.tex`. Teu arquivo de entrada principal deveria começar com uma linha ‘`\input texinfo-ja`’, em vez da padrão ‘`\input texinfo`’. Use a codificação de caracteres UTF-8 como de costume. Você precisa processar os arquivos ou com XeT_EX ou com LuaT_EX.

Para chinês, você usa o arquivo `texinfo-zh.tex`. Teu arquivo de entrada principal deveria começar com uma linha ‘`\input texinfo-zh`’, em vez da padrão ‘`\input texinfo`’. Use a codificação de caracteres UTF-8 como de costume. Você precisa processar os arquivos com XeT_EX.

Por exemplo, você executaria `'PDFTEX=xetex texi2pdf meu-arquivo-entrada.texi'` para usar XeT_EX para produzir saída em PDF. Você possivelmente tenha que instalar o XeT_EX (ou o LuaT_EX), o pacote `'zhspacing'` e(ou) pacotes de fontes.

Nem o japonês nem o chinês são suportados com o programa padrão T_EX (ou o pdfT_EX), pois ele não suporta as fontes e milhares de glifos necessários para esses idiomas.

A distribuição do Texinfo vem com alguns arquivos de amostra que você pode usar para testar o suporte para esses idiomas: `'short-sample-ja.texi'` para japonês e `'short-sample-zh.texi'` para chinês.

Para mais informações acerca dessas extensões para T_EX, veja-se a página inicial do XeT_EX (<https://tug.org/xetex/>) e o site do LuaT_EX (<https://www.luatex.org/>).

20 texi2any: O Tradutor para Texinfo

texi2any é o tradutor genérico para o Texinfo que consegue produzir formatos de saída diferentes e é altamente personalizável. Ele suporta estes formatos:

- Info (por padrão, ou com `--info`),
- HTML (com `--html`),
- EPUB 3 (com `--epub3`),
- texto simples (com `--plaintext`),
- L^AT_EX (com `--latex`),
- DocBook (com `--docbook`),
- Texinfo XML (com `--xml`).

makeinfo é um apelido para **texi2any**. Por padrão, **texi2any** e **makeinfo** geram saída Info; de fato, não existem diferenças de comportamento baseadas no nome.

Além desses formatos padrão, as opções de linha de comando para **texi2any** podem mudar muitos aspectos da saída gerada. Além disso, os arquivos de inicialização fornecem ainda mais controle sobre a saída final—quase tudo o que não esteja especificado no arquivo de entrada do Texinfo. Os arquivos de inicialização são escritos em Perl, e qualquer coisa que possa ser especificada na linha de comando também pode ser especificada em um arquivo de inicialização.

Duas implementações funcionalmente equivalentes do **texi2any** podem ser construídas no GNU Texinfo, uma em Perl e outra em código nativo compilado (em C). A implementação de código nativo ainda é experimental. O programa Perl pode usar módulos de extensão Perl em código nativo compilado, enquanto o programa de código nativo embute um interpretador Perl para as partes que exigem Perl.

No caso padrão, a implementação Perl é instalada. Você consegue selecionar a implementação experimental C ao tempo da construção, passando a opção `--enable-using-c-texi2any` para **configure**.

20.1 Invocando texi2any a partir de um Shell

Para processar um arquivo do Texinfo, invoque **texi2any** seguido pelo nome do arquivo do Texinfo. Selecione também o formato que você quer produzir com a opção de linha de comando apropriada (o padrão é Info). Assim, para criar o arquivo Info para Bison, digite o seguinte para o shell:

```
texi2any --info bison.texi
```

Você consegue especificar mais que um nome de arquivo de entrada; cada um é processado em sequência. Se um nome de arquivo de entrada for ‘-’, a entrada padrão será lida.

O programa **texi2any** aceita muitas opções. Talvez as mais básicas sejam aquelas que mudam o formato da saída gerada. Por padrão, **texi2any** produz Info.

Cada opção de linha de comando ou é um nome longo precedido por ‘--’ ou é uma única letra precedida por ‘-’. Você pode usar abreviações para os nomes longos de opções, desde que elas sejam únicas.

Por exemplo, você poderia usar o seguinte comando de shell para criar um arquivo Info para **bison.texi** no qual as linhas fossem preenchidas em somente 68 colunas:

```
texi2any --fill-column=68 bison.texi
```

Você pode escrever duas ou mais opções em sequência, assim:

```
texi2any --no-split --fill-column=70 ...
```

(Isso manteria o arquivo Info unido como um arquivo possivelmente muito longo e também configuraria a coluna de preenchimento como 70).

As opções são (aproximadamente em ordem alfabética):

--commands-in-node-names

Essa opção agora não faz nada, mas permanece para compatibilidade. (Ela costumava garantir que os comandos `@` nos nomes dos nós fossem expandidos por todo o documento, especialmente `@value`. Isso agora é feito por padrão).

--conf-dir=dir

Posponha *dir* à lista de pesquisa de diretórios para encontrar arquivos de personalização que podem ser carregados com `--init-file` (veja-se abaixo). O valor de *dir* pode ser um único diretório ou uma lista de vários diretórios separados pelo caractere usual de separador de caminho (':' em sistemas do tipo Unix, ';' no Windows).

--css-include=arquivo

Ao produzir HTML, inclui literalmente o conteúdo de *arquivo*, que deveria conter especificações de folhas de estilos em cascata, no bloco '`<style>`' da saída gerada HTML. Se *arquivo* for '-', lê a entrada padrão. Veja-se Seção 22.3 [CSS do HTML], Página 193.

--css-ref=url

Ao produzir HTML, adiciona uma etiqueta '`<link>`' à saída que referencia uma folha de estilo em cascata em *url*. Isso permite usar folhas de estilo independentes.

-D var

-D 'var valor'

Faz com que a variável *var* do Texinfo seja definida. Isso é equivalente a `@set var` no arquivo do Texinfo (veja-se Seção 16.5 [`@set @clear @value`], Página 132).

O argumento para a opção é sempre uma palavra para o shell; se ela contiver espaços em branco internos, a primeira palavra será considerada como o nome da variável e o restante como o valor. Por exemplo, `-D 'minhavar algumvalor'` é equivalente a `@set minhavar algumvalor`.

--docbook

Gera saída DocBook (em vez de Info).

--document-language=idioma

Use *idioma* para traduzir palavras-chave Texinfo que acabam no documento de saída. O padrão é a localidade especificada pelo comando `@documentlanguage`, se houver, caso contrário, inglês (veja-se Seção 15.1 [`@documentlanguage`], Página 126).

--dvi Gere um arquivo DVI do TeX usando `texi2dvi`, em vez de Info (veja-se Seção 20.3 [Saída Impressa do `texi2any`], Página 165).

--dvipdf Gere um arquivo PDF usando `texi2dvi --dvipdf`, em vez de Info (veja-se Seção 20.3 [Saída Impressa do `texi2any`], Página 165).

--enable-encoding

--disable-encoding

Por padrão, ou com `--enable-encoding`, produza caracteres acentuados e especiais em saída Info e de texto simples com base na codificação do documento. Com `--disable-encoding`, transliterações ASCII de 7 bits são geradas. Também obedecido em outros formatos de saída para ordenação de chaves de índice e para alguma saída de texto simples.

--epub3 Gerar saída EPUB 3

--error-limit=limite

-e limite Informar *limite* de erros antes de abortar (supondo que continuar seria inútil); padrão 100.

`--fill-column=largura`

`-f largura`

Especificar o número máximo de colunas em uma linha; isso é a borda direita de uma linha. Os parágrafos que sejam preenchidos serão preenchidos com essa largura. (Preencher é o processo de quebrar e de conectar linhas para que as linhas sejam do mesmo comprimento, ou menores, que o número especificado como a coluna de preenchimento. As linhas são quebradas entre as palavras). O valor padrão é 72.

`--footnote-style=estilo`

`-s estilo` Configurar o estilo da nota de rodapé como *estilo*: ‘`end`’ para o estilo do nó final (o padrão) ou ‘`separate`’ para o estilo do nó separado. O valor configurado por essa opção substitui o valor configurado em um arquivo do Texinfo por um comando `@footnotestyle` (veja-se Seção 9.3.2 [Estilos de Notas de Rodapé], Página 86).

No Info, quando o estilo da nota de rodapé for ‘`separate`’, `texi2any` cria um novo nó contendo as notas de rodapé encontradas no nó atual. Quando o estilo da nota de rodapé for ‘`end`’, `texi2any` coloca as referências da nota de rodapé no final do nó atual.

No HTML, quando o estilo da nota de rodapé for ‘`end`’, ou se a saída gerada não for dividida, as notas de rodapé são colocadas no final da saída gerada. Se configurada como ‘`separate`’ e a saída gerada for dividida, elas serão colocadas em um arquivo separado.

`--force`

`-F` Normalmente, se o arquivo de entrada contiver erros, os arquivos de saída não serão criados. Com essa opção, eles são preservados.

`--help`

`-h` Imprimir uma mensagem com as opções disponíveis e uso básico, então sair com sucesso.

`--html` Gerar saída HTML (em vez de Info). Por padrão, a saída gerada HTML é dividida em um arquivo de saída por nó e a saída gerada dividida é escrita em um subdiretório baseado no nome do arquivo de entrada. Veja-se Capítulo 22 [Gerando HTML], Página 192.

`-I diretório`

Posponha *diretório* à lista de pesquisa de diretório para encontrar arquivos que sejam incluídos usando o comando `@include`. Por padrão, `texi2any` pesquisa o diretório atual e o diretório principal do caminho do arquivo de entrada, se existir um (e não for o diretório atual). O valor de *diretório* pode ser um diretório ou uma lista de vários diretórios separados pelo caractere separador de caminho usual (‘`:`’ em sistemas do tipo Unix, ‘`;`’ no Windows).

`--ifdocbook`

`--ifhtml`

`--ifinfo`

`--iflatex`

`--ifplaintext`

`--iftex`

`--ifxml` Para o formato fornecido, processar os comandos ‘`@ifformato`’ e ‘`@formato`’ e não processar ‘`@ifnotformato`’, independentemente do formato sendo gerado. Por exemplo, se `--iftex` for fornecido, então os blocos ‘`@iftex`’ e ‘`@tex`’ serão lidos e os blocos ‘`@ifnottex`’ serão ignorados.

```
--no-ifdocbook
--no-ifhtml
--no-ifinfo
--no-iflatex
--no-ifplaintext
--no-iftex
--no-ifxml
```

Para o formato fornecido, não processar os comandos ‘`@ifformato`’ e ‘`@formato`’ e processar ‘`@ifnotformato`’, independentemente do formato sendo gerado. Por exemplo, se `--no-ifhtml` for fornecido, então blocos ‘`@ifhtml`’ e ‘`@html`’ não serão lidos, e blocos ‘`@ifnohtml`’ serão.

```
--info
```

Gerar saída Info. Por padrão, se o arquivo da saída gerada contiver mais que cerca de 300.000 bytes, ele será dividido em subarquivos mais curtos de aproximadamente esse tamanho. O nome do arquivo da saída gerada e de quaisquer subarquivos é determinado pelo nome do arquivo de entrada ou por `@setfilename`, se presente (veja-se [Configurando o Nome do Arquivo da Saída Gerada], Página 12). Veja-se Seção 21.2 [Arquivos de Etiqueta e Divididos], Página 188.

```
--init-file=arquivo
```

Carregar *arquivo* como código para modificar o comportamento e a saída do manual gerado. É comum usar as extensões `.pm` ou `.init` para esses arquivos de personalização, mas isso não é obrigatório; o nome de *arquivo* pode ser qualquer coisa. A opção `--conf-dir` (veja-se acima) pode ser usada para adicionar à lista de diretórios nos quais esses arquivos de personalização são pesquisados.

```
--internal-links=arquivo
```

No modo HTML, produzir um arquivo separado por tabulações contendo três colunas: o linque interno para um item, o tipo do item e o nome do item ou o termo correspondente ao item que foi indexado ou inserido.

Os itens gerados são:

unidades de saída gerada

As unidades de saída são a unidade principal dos documentos de saída. As unidades de saída contêm o conteúdo associado a um nó ou a um comando de seccionamento. O nome do item é o nó associado ou um nome de comando de seccionamento.

Comandos de seccionamento

O nome do comando sem `@`, ajustado para elevar/rebaixar seções é posposto ao nome do item.

nós, âncoras e flutuadores

O tipo do elemento é o nome do comando sem `@`.

intradadas de índice

O tipo do elemento é o nome do índice.

Os itens estão na ordem de classificação natural para o elemento fornecido. Esse despejo pode ser útil para pós-processadores.

```
--latex
```

Gerar saída $\text{\LaTeX}$

```
--macro-expand=arquivo
```

```
-E arquivo
```

Gerar o fonte do Texinfo, com todas as macros do Texinfo expandidas, para *arquivo*. Normalmente, o resultado da expansão de macro é usado internamente por `texi2any` e então descartado.

--no-headers

Não incluir menus ou linhas separadoras de nós na saída gerada.

Ao gerar Info, isso é o mesmo que usar **--plaintext**, resultando em um arquivo de texto simples. Além disso, a saída gerada é a saída padrão, a menos que substituída por **-o**. (Esse comportamento é para retro compatibilidade).

Ao gerar HTML e a saída gerada for dividida, gerar também linques de navegação somente no início de cada arquivo. Se a saída gerada não for dividida, não incluir linques de navegação no topo de cada nó em absoluto. Veja-se Capítulo 22 [Gerando HTML], Página 192.

--node-files**--no-node-files**

Com **--node-files**, ao gerar HTML, criar arquivos de redirecionamento para âncoras e quaisquer nós ainda não gerados com o nome do arquivo correspondente ao nome do nó (veja-se Seção 22.7.2 [Expansão de Nome de Nó do Xref do HTML], Página 198). Isso torna possível o êxito de referências entre manuais em nível de seção e de capítulo (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201).

Se a saída gerada for dividida, isso estará habilitado por padrão. Se a saída gerada não for dividida, **--node-files** habilita a criação dos arquivos de redirecionamento, além do arquivo de saída principal monolítico. **--no-node-files** suprime a criação de arquivos de redirecionamento em qualquer caso. Essa opção não tem efeito com nenhum formato de saída diferente do HTML. Veja-se Capítulo 22 [Gerando HTML], Página 192.

--no-validate**--no-pointer-validate**

Suprimir a fase de validação de ponteiro do `texi2any`—uma coisa perigosa de se fazer. Isso também pode ser feito com o comando `@novalidate` (veja-se Seção 19.3.1 [Formatando Documentos Parciais], Página 153).

Se você não suprimir a validação de ponteiro, `texi2any` verificará a validade das referências cruzadas e entradas de menu no arquivo Texinfo, bem como dos ponteiros de nó, se eles(as) forem fornecidos(as) explicitamente.

--no-warn

Suprimir mensagens de aviso (porém não mensagens de erro).

--number-footnotes**--no-number-footnotes**

Com **--no-number-footnotes**, suprimir a numeração automática de notas de rodapé. Por padrão, notas de rodapé são numeradas sequencialmente dentro de um nó, ou seja, o número da nota de rodapé atual é reconfigurado para 1 no início de cada nó.

--number-sections**--no-number-sections**

Com **--number-sections** (o padrão), produzir números de capítulo, seção e apêndice como em manuais impressos. Isso funciona somente com manuais estruturados hierarquicamente. Você deveria especificar **--no-number-sections** se teu manual não for normalmente estruturado.

--output=file**-o arquivo**

Especificar que a saída gerada deveria ser direcionada para *arquivo*. Isso substitui qualquer nome de arquivo especificado em um comando `@setfilename` encontrado no fonte do Texinfo. Se nem `@setfilename` nem essa opção forem especificados, o

nome do arquivo de entrada será usado para determinar o nome da saída gerada. Veja-se [Configurando o Nome do Arquivo da Saída Gerada], Página 12.

Se *arquivo* for ‘-’, a saída gerada vai para a saída padrão e ‘--no-split’ estará implícita.

Se *arquivo* for um diretório ou terminar com uma ‘/’, as regras usuais serão usadas para determinar o nome do arquivo de saída (ou seja, usar `@setfilename` ou o nome do arquivo de entrada), mas os arquivos serão escritos no diretório *arquivo*. Por exemplo, ‘`texi2any -o bar/ foo.texi`’, com ou sem `--no-split`, escreverá `bar/foo.info`, e possivelmente outros arquivos, sob `bar/`.

Ao gerar HTML e a saída gerada for dividida, *arquivo* será usado como o nome para o diretório no qual todos os arquivos serão escritos. Por exemplo, ‘`texi2any -o bar --html foo.texi`’ escreverá `bar/index.html`, entre outros arquivos.

Ao gerar EPUB, um diretório contêiner para os arquivos e diretórios necessários para o formato EPUB será criado, bem como o arquivo de saída gerada EPUB. Se *arquivo* corresponder a um diretório, o diretório contêiner será colocado dentro de *arquivo*. O arquivo de saída gerada EPUB nunca é colocado nesse diretório. Se *arquivo* corresponder a um arquivo, ele será usado para o nome do arquivo de saída gerada EPUB.

`--output-indent=valor`

Essa opção agora não faz nada, mas permanece para compatibilidade. (É usada para alterar o recuo na saída gerada XML/DocBook).

`-P caminho`

Posponha *caminho* à lista de pesquisa de diretórios para `@include`. Veja-se ‘-I’ acima.

`--paragraph-indent=recuo`

`-p recuo` Configurar o estilo de recuo do parágrafo para *recuo*. O valor configurado por essa opção substitui o valor configurado em um arquivo do Texinfo por um comando `@paragraphindent` (veja-se Seção E.3 [`@paragraphindent`], Página 278). O valor de *recuo* é interpretado conforme segue:

‘asis’ Preservar qualquer recuo existente (ou falta dele) no início dos parágrafos.

‘0’ ou ‘none’ Deletar qualquer recuo existente.

num Recuar cada parágrafo com *num* espaços.

O padrão é o de recuar dois espaços, exceto para parágrafos após o título de uma seção, os quais não são recuados.

`--pdf` Gerar um arquivo PDF usando `texi2dvi --pdf`, em vez de Info (veja-se Seção 20.3 [Saída Impressa do `texi2any`], Página 165).

`--plaintext`

Produzir um arquivo de texto simples (em vez de Info): não incluir menus ou linhas separadoras de nós na saída gerada. Isso resulta em um arquivo de texto simples que você consegue (por exemplo) enviar por correio eletrônico sem complicações ou incluir em uma distribuição (por exemplo, um arquivo `INSTALL`).

Com essa opção, a saída gerada vai para a saída padrão por padrão, em vez de para um arquivo com um nome baseado no nome do arquivo de entrada ou `@setfilename`; isso pode ser substituído por `-o`.

`--ps` Gerar um arquivo PostScript usando `texi2dvi --ps`, em vez de Info (veja-se Seção 20.3 [Saída Impressa do `texi2any`], Página 165).

```
--set-customization-variable variável=valor
-c variável=valor
```

Configurar a variável de personalização *variável* como *valor*. O = é opcional, mas tanto *variável* quanto *valor* precisam estar aspeados para o shell conforme necessário para que o resultado seja uma palavra. Muitos aspectos do comportamento e saída do `texi2any` podem ser controlados por variáveis de personalização, além do que pode ser configurados no documento pelos comandos @ e com outras chaves de linha de comando. Veja-se Seção 20.4 [Variáveis de Personalização], Página 166.

```
--split=como
--no-split
```

Ao gerar Info, por padrão, grandes arquivos de saída são divididos em subarquivos menores, de aproximadamente 300k bytes. Ao gerar HTML, por padrão, cada arquivo de saída contém um nó (veja-se Capítulo 22 [Gerando HTML], Página 192). `--no-split` suprime essa divisão da saída.

Alternativamente, `--split=como` pode ser usada para especificar em qual nível a saída HTML deveria ser dividida. Os valores possíveis para *como* são:

‘chapter’ A saída gerada é dividida em @chapter e outros comandos @ de seccionamento nesse nível (@appendix, entre outros).

‘section’ A saída gerada é dividida em @section e semelhante.

‘nó’ A saída gerada é dividida em cada nó. Esse é o padrão.

A saída gerada de texto simples pode ser dividida semelhante ao HTML. Isso possivelmente seja útil para extrair seções a partir de um documento do Texinfo e disponibilizá-las como arquivos separados.

```
--split-size=número
```

Manter arquivos Info com no máximo *número* caracteres, se possível; o padrão é 300.000. (No entanto, um nó nunca será dividido entre arquivos Info).

```
--trace-includes
```

Imprimir o nome de cada arquivo incluído na saída padrão, incluindo quaisquer arquivos incluídos indiretamente. Um arquivo de inclusão será impresso somente se for realmente encontrado. Quando essa opção é fornecida, `texi2any` não gera em nenhum formato de saída. (O objetivo dessa opção é o de facilitar coletar os fontes de um manual do Texinfo).

```
--transliterate-file-names
```

Habilitar transliteração de caracteres não ASCII em nomes de comandos de nó e de seccionamento para fins de criação de identificadores de nomes de arquivos e de comandos de seccionamento. Os nomes de arquivos e identificadores obtidos dessa maneira são dependentes da plataforma e do processador. Veja-se Seção 22.7.4 [Expansão de Caracteres de 8 bits do Xref do HTML], Página 201.

Arquivos de redirecionamento para âncoras e nós não são afetados, portanto, se `--node-files` estiver habilitada, arquivos com nomes não transliterados também serão criados. É recomendado criar nomes de arquivos não transliterados para nós, de modo que as referências cruzadas para o manual gerado sejam bem-sucedidas. Veja-se [---node-files], Página 161. Veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201.

```
-U variável
```

Faz com que *variável* seja indefinida. Isso é equivalente a @clear *variável* no arquivo do Texinfo (veja-se Seção 16.5 [set @clear @value], Página 132).

- `--verbose` Faz com que `texi2any` exiba mensagens dizendo o que está fazendo. Normalmente, `texi2any` só gera mensagens se existirem erros ou avisos.
- `--version`
- `-V` Imprimir o número da versão, então sair com sucesso.
- `--Xopt sequência` Passar *sequência* (uma palavra de shell) para `texi2dvi`; pode ser repetido (veja-se Seção 20.3 [Saída Impressa do `texi2any`], Página 165).
- `--xml` Gerar saída XML do Texinfo (em vez do Info).

20.2 Variáveis de Ambiente Reconhecidas pelo `texi2any`

`texi2any` também lê a variável de ambiente `TEXINFO_OUTPUT_FORMAT` para determinar o formato da saída gerada, se não substituída por uma opção de linha de comando. O valor deveria ser um de:

`docbook dvi dvipdf epub3 html info latex pdf plaintext ps xml`

Se não configurada ou especificada de outro modo, saída Info é o padrão.

`TEXINFO_OUTPUT_FORMAT` pode assumir alguns outros valores especiais, os quais não correspondem a nenhum formato de saída configurável de linha de comando (veja-se Seção 20.4.3 [Seleção do Formato de Saída], Página 167).

A variável de personalização de mesmo nome também é lida; se configurada, essa substitui uma configuração de variável de ambiente, mas não uma opção de linha de comando. Veja-se Seção 20.4.3 [Seleção do Formato de Saída], Página 167.

Você consegue controlar o uso de módulos de extensão Perl da implementação Perl do `texi2any` configurando a variável de ambiente `TEXINFO_XS`. Esses módulos são código nativo compilado que o código interpretado da Perl consegue usar. Idealmente, esses módulos de extensão deveriam simplesmente funcionar, e a única diferença notável que eles deveriam fazer é que `texi2any` terminasse de executar mais cedo. No entanto, você consegue usar essa variável de ambiente para fins de solução de problemas: por exemplo, se tiver problemas com a saída gerada do `texi2any` variando, dependendo se os módulos de extensão estiverem em uso.

Os seguintes valores da `TEXINFO_XS` são reconhecidos pelo `texi2any`:

- `'default'` O comportamento padrão. Tentar carregar módulos de extensão e voltar silenciosamente para as implementações interpretadas da Perl se isso falhar.
- `'warn'` Tentar carregar módulos de extensão e, se isso falhar, fornecer uma mensagem de aviso antes de voltar para as implementações interpretadas da Perl.
- `'required'` Tentar carregar módulos de extensão e, se isso falhar, fornecer uma mensagem e abortar.
- `'debug'` Tentar carregar módulos de extensão, imprimindo muitas mensagens ao fazer isso.
- `'omit'` Não usar módulos de extensão.

Você pode desabilitar o uso da implementação de código nativo na implementação Perl do `texi2any` para as principais etapas de processamento mais detalhadamente. Se os módulos de código nativo não forem usados em uma etapa, eles não poderão ser usados nas etapas seguintes. São distinguidas três etapas, cada uma associada a uma variável de ambiente:

- `parsing` Corresponde à conversão da entrada do Texinfo em um formato interno de árvore usado para processamento posterior. Configure `TEXINFO_XS_PARSER` como `'0'` para desabilitar o uso da implementação de código nativo do módulo analisador.

structure Corresponde à determinação de seccionamentos e de estruturas de nós. Configure `TEXINFO_XS_STRUCTURE` como ‘0’ para desabilitar o uso da implementação de código nativo de módulos Perl nessa etapa.

conversion

Configure `TEXINFO_XS_CONVERT` como ‘0’ para desabilitar o uso do código nativo para conversão da árvore para o formato de saída. Observe que muitos formatos de saída não têm uma implementação de código nativo para essa etapa.

Isso pode ser útil para contornar defeitos ou incompatibilidades entre a implementação do código nativo e a implementação em código Perl puro.

A implementação C do `texi2any` usa somente código nativo para as etapas de *análise* e de *estruturação*. Para a etapa de *conversão*, o código nativo é usado, se possível. Se o código Perl for chamado, o programa sempre tenta carregar os módulos de extensão Perl disponíveis. A falha ao carregar uma extensão será informada e o programa aborta. Configurar-se `TEXINFO_XS` como `debug` ainda pode ser útil para obter mais informações. Configurar-se `TEXINFO_XS` como `omit` com a implementação C do `texi2any` é um erro.

Você pode controlar o uso do Perl embutido da implementação C do `texi2any` configurando a variável de ambiente `TEXINFO_C_EMBED_PERL` como ‘0’ para desabilitar o uso do Perl embutido. Configure com cuidado, pois os arquivos de inicialização de personalização do HTML, a conversão para a maioria dos formatos de saída e a ordenação ideal do índice exigem Perl.

20.3 Saída Impressa do `texi2any`

Para justificar o nome Texinfo-to-*any*, `texi2any` tem suporte básico para a criação de saída impressa nos vários formatos: $\TeX$ DVI, PDF e PostScript. Isso é feito por meio do método simples de execução do programa `texi2dvi` quando esses formatos de saída forem solicitados, depois de verificar a validade da entrada para dar para os(as) usuários(as) o benefício da verificação de erros do `texi2any`. Se você não quiser essa verificação de erros, talvez porque teu manual execute truques avançados do $\TeX$ junto com `texinfo.tex`, basta invocar `texi2dvi` diretamente.

As opções de formato de saída para isso são `--dvi`, `--dvipdf`, `--pdf` e `--ps`. Veja-se [Formatar com `texi2dvi`], Página 150, para mais detalhes acerca dessas opções e operação geral do `texi2dvi`. Além disso, as opções `--verbose`, `--silent` e `--quiet` são passadas se especificadas; as opções `-I` e `-o` são igualmente passadas com os argumentos delas, e `--debug` sem o argumento dela.

A única opção restante que está relacionada com a invocação do `texi2dvi` é `--Xopt`. Aqui, somente o argumento é passado e múltiplas opções `--Xopt` se acumulam. Isso fornece uma maneira para construir uma linha arbitrária de comando para `texi2dvi`. Por exemplo, executando

```
texi2any --Xopt -t --Xopt @afourpaper --pdf foo.texi
```

é equivalente a executar

```
texi2dvi -t @afourpaper --pdf foo.texi
```

exceto para verificação de validade.

Embora se possa desejar que outras opções para `texi2any` tenham efeito, elas não tem. Por exemplo, executar `‘texi2any --no-number-sections --dvi foo.texi’` ainda resulta em um arquivo DVI com seções numeradas. (Talvez isso pudesse ser melhorado no futuro, se solicitações forem recebidas).

O nome real do comando que é invocado é especificado pela variável de personalização `TEXI2DVI` (veja-se Seção 20.4.8 [Variáveis de Personalização para Múltiplos Formatos], Página 176). Como você pode imaginar, o padrão é `‘texi2dvi’`.

O próprio `texi2any` não gera nenhuma saída normal quando invoca `texi2dvi`, somente mensagens de diagnóstico.

20.4 Variáveis de Personalização

Aviso: Esses nomes e significados de variáveis de personalização podem mudar em qualquer lançamento do Texinfo. Nós sempre tentamos evitar mudanças incompatíveis, mas não podemos absolutamente prometer, pois as necessidades mudam com o tempo.

Muitos aspectos do comportamento e da saída do `texi2any` podem ser modificados modificando-se as assim chamadas *variáveis de personalização*. Eles se enquadram em algumas categorias gerais:

- Aqueles associados com comandos `@`; por exemplo, `@documentlanguage`.
- Aqueles associados com opções de linha de comando; por exemplo, a variável de personalização `SPLIT` está associada com a opção de linha de comando `--split`. `TEXINFO_OUTPUT_FORMAT` é configurada baseada nas opções de linha de comando do formato da saída e permite especificar o formato da saída gerada.
- Aqueles que afetam o comportamento global do `texi2any`, em particular a análise de código Texinfo.
- Aqueles associados a formatos de saída específicos. (Variáveis relevantes somente para saída gerada HTML estão documentadas em Seção 22.8 [Personalização da Saída Gerada HTML], Página 204).
- Aqueles relevantes para vários formatos de saída.

Você pode configurar variáveis de personalização na linha de comando usando `--set-customization-variable 'variável valor'` (aspeando o par variável/valor para o shell) ou `--set-customization-variable variável=valor` (usando `=`). Um valor especial é `'undef'`, o qual configura a variável para esse valor especial “indefinido” da Perl.

As seções abaixo fornecem os detalhes de cada um deles.

20.4.1 Variáveis de Personalização para Comandos `@`

Cada um dos comandos `@` a seguir tem uma variável de personalização associada com o mesmo nome (sem o

<code>@afivepaper</code>	<code>@afourpaper</code>	<code>@afourlatex</code>
<code>@afourwide</code>	<code>@allowcodebreaks</code>	<code>@bsixpaper</code>
<code>@contents</code>	<code>@codequotebacktick</code>	
<code>@codequoteundirected</code>	<code>@deftypefnnewline</code>	<code>@documentdescription</code>
<code>@documentencoding</code>	<code>@documentlanguage</code>	<code>@evenfooting</code>
<code>@evenfootingmarks</code>	<code>@evenheading</code>	<code>@evenheadingmarks</code>
<code>@everyfooting</code>	<code>@everyfootingmarks</code>	<code>@everyheading</code>
<code>@everyheadingmarks</code>	<code>@exampleindent</code>	<code>@firstparagraphindent</code>
<code>@fonttextsize</code>	<code>@footnotestyle</code>	<code>@frenchspacing</code>
<code>@headings</code>	<code>@kbdinputstyle</code>	<code>@microtype</code>
<code>@novalidate</code>	<code>@oddfooting</code>	<code>@oddfootingmarks</code>
<code>@oddheading</code>	<code>@oddheadingmarks</code>	<code>@pagesizes</code>
<code>@paragraphindent</code>	<code>@setfilename</code>	<code>@setchapternewpage</code>
<code>@shortcontents</code>	<code>@smallbook</code>	<code>@summarycontents</code>
<code>@urefbreakstyle</code>	<code>@xrefautomaticsectiontitle</code>	

Configurar essa variável de personalização com um valor `'foo'` é semelhante a executar `@cmd foo`. Porém, não é exatamente a mesma coisa, já que quaisquer efeitos colaterais da análise do fonte do Texinfo não são refeitos. Além disso, algumas variáveis não levam código Texinfo ao gerar formatos específicos, mas sim um argumento que já está formatado. Esse é o caso, por exemplo, do HTML para `documentdescription`.

Observe que se `texi2any` for invocado para processar o arquivo com `TEX` (por exemplo, com a opção `--pdf`), então essas variáveis de personalização possivelmente não sejam passadas para `TEX`.

20.4.2 Variáveis e Opções de Personalização

A tabela a seguir fornece as variáveis de personalização associadas com algumas opções de linha de comando. Veja-se Seção 20.1 [Invocando `texi2any`], Página 157, para o significado das opções.

Option	Variável
<code>--enable-encoding</code>	<code>ENABLE_ENCODING</code>
<code>--document-language</code>	<code>documentlanguage</code>
<code>--error-limit</code>	<code>ERROR_LIMIT</code>
<code>--fill-column</code>	<code>FILLCOLUMN</code>
<code>--footnote-style</code>	<code>footnotestyle</code>
<code>--force</code>	<code>FORCE</code>
<code>--headers</code>	<code>HEADERS, FORMAT_MENU</code>
<code>--internal-links</code>	<code>INTERNAL_LINKS</code>
<code>--macro-expand</code>	<code>MACRO_EXPAND</code>
<code>--no-validate</code>	<code>novalidate</code>
<code>--no-warn</code>	<code>NO_WARN</code>
<code>--node-files</code>	<code>NODE_FILES</code>
<code>--number-footnotes</code>	<code>NUMBER_FOOTNOTES</code>
<code>--number-sections</code>	<code>NUMBER_SECTIONS</code>
<code>--output</code>	<code>OUTFILE, SUBDIR</code>
<code>--paragraph-indent</code>	<code>paragraphindent</code>
<code>--split</code>	<code>SPLIT</code>
<code>--split-size</code>	<code>SPLIT_SIZE</code>
<code>--trace-includes</code>	<code>TRACE_INCLUDES</code>
<code>--transliterate-file-names</code>	<code>TRANSLITERATE_FILE_NAMES</code>
<code>--verbose</code>	<code>VERBOSE</code>

Configurar tal variável de personalização para um valor ‘foo’ é essencialmente o mesmo que especificar `--opt=foo` se a opção receber um argumento, ou `--opt` se não.

20.4.3 Seleção do Formato de Saída

A variável de personalização `TEXINFO_OUTPUT_FORMAT` pega o valor do formato gerado pelo `texi2any`. Isso pode ser um dos formatos usuais de saída que possam ser especificados com opções, ou vários outros formatos especiais de saída:

```
‘docbook’
‘dvi’
‘u’
‘epub3’
‘html’
‘info’
‘pdf’
‘plaintext’
‘ps’
‘xml’
```

Eles correspondem às opções de linha de comando (e aos valores da variável de ambiente `TEXINFO_OUTPUT_FORMAT`) de mesmo nome. O valor da variável de personalização `TEXINFO_OUTPUT_FORMAT` é configurado quando opções de linha de comando (ou valores da variável de ambiente `TEXINFO_OUTPUT_FORMAT`) configuram o formato da saída gerada. Veja-se Seção 20.1 [Invocando `texi2any`], Página 157.

<code>'debugtree'</code>	Em vez de gerar um formato regular de saída, produzir uma representação de texto da árvore obtida pela análise do documento Texinfo de entrada.
<code>'parse'</code>	Fazer somente análise do fonte do Texinfo; não existe saída gerada.
<code>'plaintexinfo'</code>	Produzir o fonte do Texinfo com todas as macros, <code>@include</code> e <code>@value{}</code> , expandidas. Isso é semelhante à configuração de <code>--macro-expand</code> , mas em vez de ser uma saída além da conversão normal, a saída do Texinfo é a saída principal.
<code>'rawtext'</code>	Produzir texto bruto, com formatação mínima. Por exemplo, notas de rodapé são ignoradas e não existe preenchimento de parágrafo. Isso é usado pelo analisador para nomes de arquivos e textos de direitos autorais em comentários do HTML, por exemplo.
<code>'structure'</code>	Fazer somente análise do fonte do Texinfo e determinação da estrutura do documento; não existe saída gerada.
<code>'texinfosxml'</code>	Produzir o documento na representação TexinfoSXML, uma sintaxe para escrever dados XML usando expressões S da Lisp.
<code>'textcontent'</code>	Produzir somente o conteúdo do texto, removidos os comandos; isso é útil para verificação ortográfica ou contagem de palavras, por exemplo. A configuração trivial do conjunto de comandos sequenciais <code>detexinfo</code> está no diretório <code>util</code> do fonte do Texinfo como um exemplo. Ela é uma linha: <pre>exec texi2any -c TEXINFO_OUTPUT_FORMAT=textcontent "\$@"</pre>

20.4.4 Variáveis Globais de Personalização

Esta tabela fornece as variáveis de personalização que afetam o comportamento global do `texi2any`.

CHECK_MISSING_MENU_ENTRY

Quando um bloco `@menu` ocorrer em um nó, verificar se existe uma entrada de menu para todos os nós subordinados na estrutura de seccionamento do documento. Ligada por padrão.

CHECK_NORMAL_MENU_STRUCTURE

Avisa se os ponteiros do nó (configurados ou explícita ou automaticamente) não estão consistentes com a ordem das entradas do menu do nó. Essa é uma verificação de estrutura mais completa que aquela fornecida por `CHECK_MISSING_MENU_ENTRY`. Ligada por padrão.

COLLATION_LANGUAGE

Por padrão, `texi2any` ordena os índices de documentos de acordo com o *Algoritmo de Agrupamento Unicode* (definido em Padrão Técnico Unicode nº 10 (<http://www.unicode.org/reports/tr10/>)), sem adaptação de agrupamento específico do idioma. Se configurado, usar o idioma para adaptação linguística da ordenação de índice.

Atualmente, `texi2any` não avisa se não existir suporte para adaptação linguística devido ou à ausência de software ou à ausência de suporte para o idioma especificado. Em Perl, o módulo `Unicode::Collate::Locale` é usado para adaptação linguística; portanto, se esse módulo não estiver instalado, a variável será ignorada silenciosamente.

Se `USE_UNICODE_COLLATION` estiver configurada como ‘0’, não existirá agrupamento Unicode e, portanto, também não existirá adaptação linguística.

Veja-se também a variável `DOCUMENTLANGUAGE_COLLATION`, descrita abaixo.

`COMMAND_LINE_ENCODING`

Codificação usada para decodificar e codificar argumentos de linha de comando. O padrão é baseado na codificação da localidade. Isso possivelmente afete nomes de arquivos inseridos nos arquivos da saída gerada ou mensagens de erro impressas pelo programa.

Observe que alguns nomes de arquivos e de diretórios provenientes da linha de comando possivelmente não sejam decodificados imediatamente e possivelmente nem mesmo sejam decodificados.

Também é usado para codificar argumentos de linha de comando passados para comandos chamados a partir do `texi2any`. Por exemplo, `latex2html` será chamado a partir do `texi2any` se `HTML_MATH` estiver configurada como ‘12h’.

`CPP_LINE_DIRECTIVES`

Reconhecer diretivas `#line` em uma passagem de “pré processamento” (veja-se Seção 17.7 [Processadores Externos de Macro], Página 146); ligada por padrão.

`DEBUG` Se configurada, saída de depuração será gerada; padrão é desligada (zero).

`DOC_ENCODING_FOR_INPUT_FILE_NAME`

Se configurada, usar as informações de codificação do documento Texinfo de entrada para a codificação de nomes de arquivos de entrada, tais como nomes de arquivos especificados como argumentos `@include` ou `@verbatiminclude`. Se desconfigurada, usar a codificação de localidade. O padrão é configurada, exceto no MS-Windows onde a codificação de localidade é usada por padrão.

Observe que isso é somente para nomes de arquivos; a codificação padrão ou `@documentencoding` é sempre usada para a codificação do conteúdo do arquivo (veja-se Seção 15.2 [`@documentencoding`], Página 127).

A variável `INPUT_FILE_NAME_ENCODING` substitui essa variável.

`DOC_ENCODING_FOR_OUTPUT_FILE_NAME`

Se configurada, usar as informações de codificação do documento Texinfo de entrada para a codificação de nomes de arquivos de saída, tais como arquivos especificados com `--output`. Se desconfigurada, usar a codificação de localidade. O padrão é desconfigurada, de forma que os nomes dos arquivos serão codificados usando a localidade atual.

Observe que isso é somente para nomes de arquivos; `OUTPUT_ENCODING_NAME` será usada para a codificação do conteúdo do arquivo.

A variável `OUTPUT_FILE_NAME_ENCODING` substitui essa variável.

`DOCUMENTLANGUAGE_COLLATION`

Se configurada, tentar usar o idioma especificado em uma diretiva `@documentlanguage` para adaptação específica do idioma na ordenação de índice. Veja-se também a variável `COLLATION_LANGUAGE`, descrita acima. (Veja-se Seção 15.1 [`@documentlanguage`], Página 126).

`DUMP_STRUCTURE`

Para depuração. Se configurada, a árvore construída ao analisar um documento do Texinfo e determinar a estrutura do documento será gerada para o arquivo; se configurada para ‘-’ para erro padrão. Padrão falso.

DUMP_TEXI

Para depuração. Se configurada, nenhuma conversão será feita, somente análise e expansão de macro. Se a opção `--macro-expand` estiver configurada, o fonte do Texinfo também será expandido para o arquivo correspondente. Padrão falso.

DUMP_TREE

Para depuração. Se configurada, a árvore construída ao analisar um documento do Texinfo será gerada para o arquivo; se configurada para `-` para erro padrão. Para compatibilidade retroativa, para um lançamento, também gera para erro padrão se configurada para `1`. Padrão falso.

FORMAT_MENU

Se configurada como `'menu'`, gera menus do Texinfo. Esse é o padrão para Info. `'sectiontoc'` é a configuração padrão para HTML, onde em vez do conteúdo dos blocos do `@menu` ser gerado, uma lista de seções subordinadas é gerada em cada nó. Configurar para `'menu_no_detailmenu'`, em vez de `'menu'` no HTML, se você quiser menus, porém não `@detailmenu` automaticamente gerado no menu mestre do nó Topo. Se configurada como `'nomenu'`, não gera menus.

Essa variável é configurada como `'nomenu'` ao gerar DocBook ou quando `--no-headers` for especificada.

IGNORE_SPACE_AFTER_BRACED_COMMAND_NAME

Se configurada, espaços serão ignorados depois de um comando `@` que use colchetes. Padrão verdadeiro, correspondendo ao comportamento do $\TeX$.

INPUT_FILE_NAME_ENCODING

Codificação usada para nomes de arquivos de entrada. Essa variável substitui qualquer codificação proveniente do documento ou da localidade atual. Normalmente, você não precisa configurar essa variável, mas ela pode ser usada se os nomes dos arquivos estiverem em uma determinada codificação de caracteres em um sistema de arquivos. Uma alternativa é a de configurar `DOC_ENCODING_FOR_INPUT_FILE_NAME` como `0` para usar a codificação de localidade. Veja-se também `OUTPUT_FILE_NAME_ENCODING`.

LOCALE_ENCODING

Codificação de localidade obtida a partir do sistema. Você não deveria precisar configurar explicitamente essa variável.

MAX_MACRO_CALL_NESTING

O número máximo de chamadas recursivas de comandos `@` definidos por meio de `@macro`; o padrão é 100000. O propósito dessa variável é o de evitar recursões infinitas.

MESSAGE_ENCODING

Codificação usada para codificar mensagens emitidas pelo `texi2any`. O padrão é baseado na codificação de localidade.

NO_INDEX Se configurada, as entradas de índice não são registradas e os comandos `@` relacionados aos índices, como `@printindex`, `@synindex` ou `@defindex` não terão efeito. Padrão falso.

NO_USER_COMMANDS

Se configurada, as definições de comando `@` definidas por usuário(a) serão ignoradas. `@alias`, `@definfoenclose`, `@defindex`, `@macro` e comandos `@` semelhantes não terão efeito. Padrão falso.

OUTPUT_ENCODING_NAME

Nome normalizado de codificação usado para arquivos de saída gerada. Deveria ser um nome de conjunto de caracteres usável em HTML, tipicamente um dos nomes de codificação preferidos da IANA. Por padrão, se uma codificação de entrada for configurada (tipicamente por meio de `@documentencoding`), essas informações serão usadas para configurar o nome da codificação da saída gerada, caso contrário, a codificação da saída gerada será baseada na codificação padrão. Veja-se Seção 15.2 [`@documentencoding`], Página 127.

OUTPUT_FILE_NAME_ENCODING

Codificação usada para nomes de arquivos da saída gerada. Essa variável substitui qualquer codificação originária do documento ou da localidade atual.

Normalmente, você não precisa configurar essa variável, mas ela pode ser usada se nomes de arquivos devessem ser criados em uma determinada codificação de caracteres em um sistema de arquivos. Veja-se também `INPUT_FILE_NAME_ENCODING`.

PACKAGE**PACKAGE_VERSION****PACKAGE_AND_VERSION****PACKAGE_URL****PACKAGE_NAME**

O nome abreviado do pacote da implementação, a versão do pacote, o nome do pacote e a versão concatenada, URL do pacote e o nome completo do pacote, respectivamente. Por padrão, essas variáveis são todas configuradas por meio do Autoconf, Automake e `configure`.

PREFIX

O prefixo do arquivo da saída gerada, que é preposto a alguns nomes de arquivos de saída gerada. Por padrão, ele é configurado pelo `@setfilename` ou a partir arquivo de entrada (veja-se [Configurando o Nome do Arquivo da Saída Gerada], Página 12). Como esse valor é usado depende do valor de outras variáveis de personalização ou de opções de linha de comando, como quando a saída gerada for dividida. O padrão é desconfigurada.

PROGRAM

Nome do programa usado. Por padrão, ela está configurada com o nome do programa iniciado, com `‘.pl’` à direita removido.

SORT_ELEMENT_COUNT

Se configurada, o nome de um arquivo para o qual uma lista de elementos (nós ou seções, dependendo do formato da saída gerada) será despejada, ordenada pelo número de linhas que eles contiverem depois da remoção dos comandos `@`; padrão desconfigurada. Isso é usado pelo programa `texi-elements-by-size` no diretório `util/` da distribuição do fonte do Texinfo (veja-se [texi-elements-by-size], Página 248).

SORT_ELEMENT_COUNT_WORDS

Ao despejar o arquivo de elementos por tamanho (veja-se o item anterior), usar contagens de palavras em vez de contagens de linhas; padrão falso.

TEST

Se configurada como um número inteiro positivo, algumas variáveis que normalmente são geradas dinamicamente novamente para cada execução (data, nome do programa, versão) serão configuradas como valores fixos e fornecidos. Alguns nomes de arquivos também podem ter a parte de diretório removida. Também pode ser selecionado código que forneça resultados mais reproduzíveis. Isso é útil para comparar a saída gerada com um arquivo de referência, como é feito para os testes. Recursos não selecionados no caso padrão, mas interessantes para testes, também estarão habilitados. Por exemplo, código relacionado à liberação de memória e a verificações

de memória liberada. Se o valor for maior, mais verificações serão habilitadas, o que pode exigir a instalação de módulos Perl adicionais. O padrão é indefinida.

TEXINFO_OUTPUT_FORMAT

O formato de saída gerado. Configurar a partir da linha de comando com opções como `--html`, como o valor da variável de ambiente `TEXINFO_OUTPUT_FORMAT` (veja-se Seção 20.2 [Variáveis de Ambiente do `texi2any`], Página 164) ou como uma opção de personalização. Veja-se Seção 20.4.3 [Seleção do Formato de Saída], Página 167.

O valor configurado na variável `TEXINFO_OUTPUT_FORMAT` é em geral o mesmo que o nome usado para configurar o formato, mas nem sempre. Se o formato for configurado como `xml`, com `--xml`, com a variável de ambiente `TEXINFO_OUTPUT_FORMAT` ou com uma variável de personalização, a variável de personalização `TEXINFO_OUTPUT_FORMAT` será configurada com um nome de formato de saída mais específico `texinfoxml`. Esse tipo de diferença na nomenclatura deveria não ter nenhuma consequência visível para o(a) usuário(a).

TREE_TRANSFORMATIONS

O valor associado é uma lista separada por vírgulas de transformações que podem ser aplicadas à árvore do Texinfo antes de gerar o resultado. Se mais que um for especificado, o ordenamento será irrelevante; cada um será sempre aplicado no ponto necessário durante o processamento.

Por padrão, as transformações de árvore `‘move_index_entries_after_items’` e `‘relate_index_entries_to_table_entries’` são executadas para saída HTML e DocBook. Aqui está um exemplo de atualização do menu mestre em um documento:

```
texi2any \
  -c TREE_TRANSFORMATIONS=regenerate_master_menu \
  -c TEXINFO_OUTPUT_FORMAT=plaintexinfo \
  meudoc.texi \
  -o /tmp/saida
```

(Advertência: como a saída de `‘plaintexinfo’` expande macros e condicionais do Texinfo, é necessário remover quaisquer dessas diferenças antes de se instalar as atualizações no documento original. Isso possivelmente seja remediado em um lançamento futuro).

As seguintes transformações são atualmente suportadas (muitas são usadas no utilitário `pod2texi` distribuído com Texinfo; veja-se Seção 20.6 [Invocando `pod2texi`], Página 179):

`‘complete_tree_nodes_menus’`

Adicionar entradas de menu ou menus inteiros para nós associados com seções de qualquer nível, com base na árvore de seccionamento.

`‘complete_tree_nodes_missing_menu’`

Adicionar menus inteiros para nós associados com seções sem menu. Os menus são baseados na árvore de seccionamento.

`‘fill_gaps_in_sectioning’`

Adiciona seções `@unnumbered...` vazias em uma árvore para preencher lacunas no seccionamento. Por exemplo, um `@unnumberedsec` será inserido se um `@chapter` for seguido por um `@subsection`.

`‘insert_nodes_for_sectioning_commands’`

Inserir nós para comandos de seccionamento carecentes de um nó correspondente.

`'move_index_entries_after_items'`

Em `@enumerate` e `@itemize`, mover entradas de índice que aparecem logo antes de `@item` para logo depois de `@item`. Linhas de comentários entre entradas de índice também serão movidas. Conforme mencionado, isso sempre é feito para saída HTML e DocBook.

`'regenerate_master_menu'`

Atualizar o menu mestre do nó Top, ou substituindo o (primeiro) `@detailmenu` no menu do nó Top ou criando-o no final do menu do nó Top.

`'relate_index_entries_to_table_entries'`

Em `@table`, `@vtable` e `@ftable`, reassociar as informações de entrada do índice originárias de um comando `@` de índice que aparece logo depois uma linha `@item` com o primeiro elemento do `@item`. Remover o comando `@` de índice da árvore.

`USE_SETFILENAME_EXTENSION`

O padrão é ativado para Info e desativado para outras saídas. Se configurada, usar exatamente o que `@setfilename` fornecer para o nome do arquivo da saída gerada, incluindo a extensão. Você não deveria precisar configurar explicitamente essa variável.

`USE_UNICODE_COLLATION`

Por padrão, `texi2any` ordena os índices de documentos de acordo com o *Algoritmo de Agrupamento Unicode* (definido em Padrão Técnico Unicode nº 10 (<http://www.unicode.org/reports/tr10/>)), que realiza uma ordenação *multi-nível* de sequências de caracteres contendo pontos de código Unicode. (Para fazer isso, o programa usa o módulo `Unicode::Collate`, uma parte padrão das instalações Perl). No entanto, o uso desse algoritmo incorre em tempos de execução notavelmente mais longos ao processar manuais com índices grandes.

Configure essa variável como `'0'` para usar um algoritmo de ordenação mais simples e mais rápido.

Você pode achar isso útil para acelerar o `texi2any` se você não tiver muitos caracteres não-ASCII no texto de entrada do índice ou se não se importar em ter os índices ordenados corretamente, especialmente ao trabalhar no rascunho de um manual.

`USE_UNIDECODING`

Se configurada como falso, não usar o módulo Perl `Text::Unidecode` para transliterar mais caracteres provenientes do código Perl; padrão verdadeiro.

20.4.5 Variáveis de Personalização do Info e de Texto Simples

Esta tabela fornece as variáveis de personalização que se aplicam somente para formatos Info e Texto Simples.

`ASCII_DASHES_AND_QUOTES`

Para saída gerada do Info, quando configurada, usar caracteres ASCII simples para representar aspas, hifens e travessões quando esses forem fornecidos no fonte do Texinfo como `'-'`, `'--'`, `'---'`, `'~'`, `'^^'`, `''` e `'`', em vez de aspas direcionais UTF-8, travessões, e assim por diante. Ligada por padrão.

`ASCII_GLYPH`

Para saída gerada do Info, usar saída gerada ASCII para comandos de glifo, tais como o sinal de direito autoral (`@copyright{}`, tornando-se `'(C)'`) e o símbolo de marcador (`@bullet{}`, tornando-se `'*'`), em vez de outras sequências Unicode. Desligada por padrão.

ASCII_PUNCTUATION

Evite quaisquer sequências UTF-8 e não-ASCII desnecessárias ou gratuitas na saída gerada. Implica `ASCII_DASHES_AND_QUOTES` e `ASCII_GLYPH` e, adicionalmente, afeta a saída gerada de comandos como `@samp` que geram aspas.

AUTO_MENU_DESCRIPTION_ALIGN_COLUMN

Para saída gerada do Info, coluna na qual iniciar uma descrição de entrada de menu fornecida por `@nodedescription` ou `@nodedescriptionblock`. Indefinida por padrão, caso em que 45% do valor da coluna de preenchimento é usado (veja-se Seção 20.1 [Invocando `texi2any`], Página 157).

AUTO_MENU_MAX_WIDTH

Número máximo de colunas em uma linha de entrada de menu no Info ao se adicionar uma descrição originária de `@nodedescription` ou `@nodedescriptionblock`. Indefinida por padrão, caso em que 10% a mais que o valor da coluna de preenchimento é usado (veja-se Seção 20.1 [Invocando `texi2any`], Página 157).

CLOSE_DOUBLE_QUOTE_SYMBOL

Quando uma aspa dupla de fechamento for necessária, para `@dfn` no Info, usar esse caractere. O padrão para Info é configurada da mesma forma que para `OPEN_DOUBLE_QUOTE_SYMBOL`, exceto que o código Unicode é uma aspa dupla de fechamento (veja-se abaixo).

INDEX_SPECIAL_CHARS_WARNING

Se configurada, avisar acerca de `‘:’` na entrada do índice, pois nem todos os leitores Info podem processá-los. Somente para Info e texto simples. Padrão falso, porque os problemas de análise não impedem a navegação; os leitores ainda podem encontrar com relativa facilidade o caminho para o nó em questão.

INFO_MATH_IMAGES

(Experimental). Se configurada, converter o conteúdo de `@math` e de `@displaymath` em imagens e referencie-os no arquivo Info produzido. Essa variável exige uma instalação funcional do `TEX`, o pacote `‘preview’` do `LATEX`, bem como o programa `dvipng`. Pode ser adequado para documentos com muitas fórmulas matemáticas que provavelmente não serão visualizados em um terminal de texto.

Esse recurso é marcado como experimental, pois exigiria suporte extra em programas de visualização do Info (atualmente, somente o modo Info do Emacs) para visualizar as imagens corretamente. As imagens deveriam estar visíveis nas versões existentes do Emacs, mas podem ter o tamanho errado ou parecer desalinhadas com o texto ao redor. É possível que nós mudemos a saída gerada para essa opção com base em comentários adicionais. Veja-se Seção F.8 [Imagem do Formato Info], Página 285, para mais detalhes acerca de como as imagens são geradas atualmente com essa opção.

INFO_SPECIAL_CHARS_QUOTE

Se configurada, sempre que existirem caracteres problemáticos para saída Info em locais como nomes de nós ou itens de menu, envolver a parte da construção onde eles aparecerem com caracteres de aspas, conforme descrito em Anexo F [Especificação do Formato Info], Página 282. O padrão é configurada para Info e desconfigurada para texto simples. Veja-se Seção 3.3 [Exigências da Linha do Nó], Página 28.

INFO_SPECIAL_CHARS_WARNING

Se configurada, avisar acerca de construções problemáticas para saída Info (tais como a sequência de caracteres `‘::’`) em nomes de nós, itens de menu e referências cruzadas. Se não configurada, configurar, a menos que `INFO_SPECIAL_CHARS_QUOTE` esteja configurada. O padrão é configurada para Info e não definida para texto simples.

Avisos semelhantes em entradas de índice são abordados por `INDEX_SPECIAL_CHARS_WARNING`.

`OPEN_DOUBLE_QUOTE_SYMBOL`

Quando uma aspa dupla de abertura for necessária, para saída gerada do ‘`@dfn`’ no Info, usar o caractere especificado. Se `--disable-encoding` estiver configurada ou a codificação do documento não for UTF-8, ‘”’ será usado. Caso contrário, a saída gerada do Info usará aspas duplas esquerdas do Unicode.

20.4.6 Variáveis de Personalização do $\text{\LaTeX}$

aviso: A personalização da saída gerada do $\text{\LaTeX}$ é experimental. Nada está decidido; tudo pode mudar e nós apreciáramos qualquer retro alimentação.

Esta tabela fornece as variáveis de personalização que se aplicam somente para saída gerada do $\text{\LaTeX}$.

`CLASS_BEGIN_USEPACKAGE`

Se configurada, o texto correspondente substituirá o `\documentclass` do $\text{\LaTeX}$; as importações de pacotes que sempre são geradas e são emitidas logo depois de `\documentclass` e as importações de pacotes que dependam da codificação do documento, configurando a entrada e a codificação da fonte (`inputenc` e `fontenc`).

O texto substituído está junto:

```
\documentclass{book}
\usepackage{amsfonts}
\usepackage{amsmath}
\usepackage[gen]{eurosym}
\usepackage{textcomp}
\usepackage{graphicx}
\usepackage{etoolbox}
\usepackage{titleps}
\usepackage[utf8]{inputenc}
\usepackage[T1]{fontenc}
```

`END_USEPACKAGE`

Se configurada, o texto correspondente substituirá as importações de pacote que dependam dos comandos Texinfo usados, e as últimas importações de pacotes que são sempre geradas e saídas geradas depois de todas as outras importações de pacotes. As últimas importações de pacote correspondem a ‘`\usepackage[hidelinks]{hyperref}`’.

Aqui está um exemplo do texto correspondente para um documento com índices, `@need`, `@multitable`, comandos de definição, `@cartouche`, listas e `@float`:

```
\usepackage{imakeidx}
\usepackage{needspace}
\usepackage{array}
\usepackage{embrac}
\usepackage{expl3}
\usepackage{tabularx}
\usepackage[framemethod=tikz]{mdframed}
\usepackage{enumitem}
\usepackage{float}
\usepackage[hidelinks]{hyperref}
```

LATEX_FLOATS_FILE_EXTENSION

Configurar nova extensão de arquivo de definição de flutuações para saída gerada $\text{\LaTeX}$. Padrão `'tfl'`.

20.4.7 Outras Variáveis de Personalização de Formatos

Esta tabela fornece a personalização que se aplica somente para formatos específicos, para formatos sem seção separada de descrição de variáveis de personalização.

TXI_MARKUP_NO_SECTION_EXTENT

Para XML. Não incluir comandos de seccionamento de nível inferior dentro de marcações de comando de seccionamento de nível superior. Padrão falso.

TEXI2DVI Nome do comando usado para produzir PostScript, PDF e DVI; padrão `'texi2dvi'`. Veja-se Seção 20.3 [Saída Impressa do `texi2any`], Página 165.

TEXTCONTENT_COMMENT

Para saída de conteúdo de texto despojado (ou seja, quando `TEXINFO_OUTPUT_FORMAT` estiver configurada como `textcontent`). Se configurada, também gera comentários. Padrão falso.

TEXINFO_DTD_VERSION

Para XML. Versão do DTD usado no preâmbulo da saída gerada XML. O padrão é configurada baseada em uma variável no `configure.ac`.

20.4.8 Variáveis de Personalização para Múltiplos Formatos

Esta tabela fornece a personalização que se aplica para múltiplos formatos.

CLOSE_QUOTE_SYMBOL

Quando uma aspa de fechamento for necessária, por exemplo, para saída gerada do `@samp`, usar esse caractere; padrão `’`; no DocBook. Indefinida no caso padrão em HTML e configurada como `&rsquo`; se `USE_NUMERIC_ENTITY` não estiver configurada, como `’`; se configurada e como um caractere de aspa se `OUTPUT_CHARACTERS` estiver configurada e a codificação da saída gerada incluir esse caractere. O padrão para Info é configurada da mesma forma que para `OPEN_QUOTE_SYMBOL`, exceto que o código Unicode é uma aspa de fechamento (veja-se abaixo).

DOCTYPE Para DocBook, HTML, XML. Especifica o `SystemLiteral`, o identificador de sistema da entidade. Esse é um URI que pode ser usado para recuperar a entidade e identifica o DTD canônico para o documento. O valor padrão é diferente para HTML, DocBook e XML.

EXTENSION

A extensão adicionada ao nome do arquivo da saída gerada. O padrão é diferente para cada formato da saída gerada.

NO_TOP_NODE_OUTPUT

Se configurada, não produzir o conteúdo do nó Top. O nó Top ainda será analisado, mas o conteúdo será descartado. Não configurada no caso padrão para HTML. Configurada no caso padrão para EPUB. Se `SHOW_TITLE` for `'undef'`, configurar-se `NO_TOP_NODE_OUTPUT` também configurará `SHOW_TITLE` para HTML.

Configurar-se `NO_TOP_NODE_OUTPUT`, que remove o nó Top e adiciona uma página de título, corresponde mais à formatação de um livro. Configurar-se `NO_TOP_NODE_OUTPUT` como false, com `SHOW_TITLE` permanecendo `'undef'`, e false, corresponde mais a uma configuração de documento para navegação, com um acesso direto para as informações no nó Top.

Para DocBook, `NO_TOP_NODE_OUTPUT` é configurada como verdadeira. Configurar-se `NO_TOP_NODE_OUTPUT` como falsa faz com que o conteúdo do nó Topo seja gerado. Não é recomendado gerar o nó Topo no DocBook, pois os textos de título e de cópia são sempre gerados. Essa opção é mantida para DocBook para compatibilidade, pois antes de 2022 o nó Topo era gerado no DocBook. Ela poderá ser removida no futuro.

OPEN_QUOTE_SYMBOL

Quando uma aspa de abertura for necessária, por exemplo, para saída gerada do ‘@samp’, usar o caractere especificado; padrão `‘` para DocBook. Indefinida no caso padrão no HTML e configurada como `‘`; se `USE_NUMERIC_ENTITY` não estiver configurada; como `‘` se configurada; e como um caractere de aspas se `OUTPUT_CHARACTERS` estiver configurada e a codificação da saída gerada incluir esse caractere.

Para Info, o padrão depende da codificação habilitada de documento. Se `--disable-encoding` estiver configurada ou a codificação do documento não for UTF-8, ‘’ será usado. Esse caractere geralmente aparece como uma aspa simples indirecionada em sistemas modernos. Caso contrário, a saída gerada Info usará uma aspa esquerda Unicode.

OUTPUT_CHARACTERS

Se não configurada, o padrão gerará caracteres acentuados e especiais em HTML, XML e DocBook usando entidades XML e em $\text{\LaTeX}$ usando macros. Se configurada, gerar caracteres acentuados na saída gerada de HTML, XML, DocBook e $\text{\LaTeX}$ e caracteres especiais na saída HTML e $\text{\LaTeX}$ baseada na codificação do documento. Veja-se Seção 15.2 [`@documentencoding`], Página 127, e Seção 11.4 [Inserindo Acentos], Página 100.

TOP_NODE_UP

Nó Acima para o nó Topo; padrão ‘(dir)’. É presumido que esse nome de nó já esteja formatado para o formato da saída gerada. No HTML pode ser usado em atributo, de modo que não deveria conter nenhum elemento. Usado para saída HTML somente se `TOP_NODE_UP_URL` estiver configurada para substituir o URL; veja-se `TOP_NODE_UP_URL` em Seção 22.8.9 [Lista de Variáveis de Personalização do HTML], Página 214. Veja-se Seção 22.8.2 [Personalização de Nomes de Arquivo e de Linques para HTML], Página 205.

USE_NODES

A unidade principal dos documentos de saída é *unidade de saída*. Normalmente, um nó está associado a um comando de seccionamento seguinte, enquanto um comando de seccionamento está associado a um nó anterior; ambos juntos constituem a unidade de saída.

Para Texto Simples e HTML. Usar preferencialmente nós para decidir onde unidades da saída gerada serão separadas se `USE_NODES` estiver configurada. Se configurada como falso, usar preferencialmente o seccionamento para decidir onde unidades da saída gerada serão separadas. O padrão é verdadeiro.

USE_NUMERIC_ENTITY

Para HTML, XML e DocBook. Se configurada, usar entidades numéricas em vez de entidades nomeadas. Por padrão, configurada como verdadeiro para saída gerada DocBook.

20.5 Internacionalização de Sequências de Caracteres de Documentos

`texi2any` escreve sequências fixas de caracteres no documento de saída em vários locais: referências cruzadas, rodapés de páginas, a página de ajuda, texto alternativo para imagens e assim por diante. A sequência de caracteres escolhida depende do valor de `documentlanguage` ao tempo da sequência de caracteres sendo gerada (veja-se Seção 15.1 [`@documentlanguage`], Página 126, para a interface de comando do Texinfo).

A estrutura Gettext com o domínio ‘`texinfo_document`’ é usada para essas sequências de caracteres (veja-se Seção “Topo” em *Gettext*). Sequências de caracteres traduzidas são escritas como Texinfo e podem incluir comandos `@`. Em sequências de caracteres traduzidas, as partes variáveis da sequência de caracteres são denotadas por ‘`{arg_name}`’, mas não por `%s` e similares. (Essa convenção é comum para o `gettext` em Perl e é totalmente suportada no GNU Gettext; veja-se Seção “Sequências de Caracteres de Formato da Perl” em *GNU Gettext*). Por exemplo, a seguir, ‘`{section}`’ será substituído pelo nome da seção:

veja-se `{section}`

A expansão de uma sequência de caracteres de tradução é feita assim:

1. Primeiro, a sequência de caracteres é traduzida. A localidade é *idiomadocumento.codificaçãodocumento*.

Se a *idiomadocumento* tiver a forma ‘`ll_CC`’, isso será tentado primeiro e depois apenas ‘`ll`’.

2. A seguir, na maioria dos casos, a sequência de caracteres será expandida como Texinfo e convertida. Os argumentos são substituídos; por exemplo, ‘`{arg_name}`’ é substituído pelo argumento real correspondente.

Também é possível que a sequência de caracteres e o argumento já estejam convertidos. Nesse caso, os argumentos simplesmente são substituídos. Similarmente, ‘`{arg_name}`’ é substituído pelo argumento real correspondente.

No exemplo a seguir, ‘`{date}`’, ‘`{program_homepage}`’ e ‘`{program}`’ são os argumentos da sequência de caracteres. Como são usados no `@uref`, a ordem deles não é previsível. ‘`{date}`’, ‘`{program_homepage}`’ e ‘`{program}`’ são substituídos depois da expansão:

Gerado em `@emph{{date}}` usando
`@uref{{program_homepage}, @emph{{program}}}`.

Essa abordagem é reconhecidamente um pouco complicada. A utilidade dela é a de que ela suporta especificar a formatação da sequência de caracteres traduzida independentemente do formato da saída gerada.

Por exemplo, a sequência de caracteres de tradução do ‘`@pxref`’ pode ser como isto:

veja-se seção `{node_file_href}` ‘`{section}`’ em `@cite{{book}}`

que permite especificar-se uma sequência de caracteres independentemente do formato da saída gerada, embora com formatação avançada ela consiga ser traduzida adequadamente em vários idiomas.

Se Perl for usada para tradução de sequências de caracteres no `texi2any`, o pacote `libintl-perl` é usado como a implementação `gettext`; mais especificamente, a implementação Perl pura é usada, de modo que o Texinfo consiga suportar um comportamento consistente em todas as plataformas e instalações, o que, de outra forma, não seria possível. `libintl-perl` está incluído na distribuição do Texinfo e é sempre instalado, para garantir que ele esteja disponível se necessário. Também é possível usar-se o `libintl-perl` do sistema (a escolha pode ser feita em tempo de construção).

20.6 Invocando `pod2texi`: Converter Pod para Texinfo

O programa `pod2texi` traduz arquivo(s) de documentação Pod da Perl para Texinfo. Existem dois modos básicos de operação: gerar um manual independente a partir de cada Pod de entrada ou (se `--base-level=1` ou superior for fornecido) gerar subarquivos Texinfo adequados para uso com `@include`.

O programa `pod2texi` pode ser útil fora do resto do Texinfo; assim, a invocação do `pod2texi` é documentada na linguagem Pod usando o formato de página de manual para seguir a convenção usada em programas autônomos da Perl, com uma versão na web <https://www.gnu.org/software/texinfo/manual/pod2texi.html> e uma versão incluída abaixo. A versão incluída no manual também é um exemplo de uso do `pod2texi`, pois ela é convertida a partir de Pod usando `pod2texi`.

manual.

20.6.1 `pod2texi`

NOME do `pod2texi`

`pod2texi` - converter Pod para Texinfo

SINOPSE do `pod2texi`

`pod2texi` [OPÇÃO]... POD...

DESCRIÇÃO do `pod2texi`

Traduzir arquivo(s) Pod para Texinfo. Existem dois modos básicos de operação. Primeiro, por padrão, cada Pod é traduzido para um manual Texinfo independente.

Segundo, se `--base-level` for configurado como superior a 0, cada Pod será traduzido para um arquivo adequado para `@include`, e mais um arquivo com um menu principal e todo o `@include` será gerado.

OPÇÕES do `pod2texi`

`--appendix-sections`

Usar comandos de seccionamento de apêndice (`@appendix`, ...) em vez dos comandos `@` padrão do Texinfo de seccionamento numerados (`@chapter`, `@section`, ...).

`--base-level=NUM|NOME`

Configura o nível dos comandos `head1`. Pode ser um inteiro ou um comando do Texinfo de seccionamento (sem o `@`): 1 corresponde ao nível `@chapter/@unnumbered`, 2 ao nível `@section` e assim por diante. O padrão é 0, significando que os comandos `head1` ainda são produzidos como capítulos, mas a saída gerada é organizada como um manual independente.

Se o nível não for 0, o arquivo Pod será renderizado como um fragmento de um manual Texinfo adequado para `@include`. Nesse caso, cada arquivo Pod terá um comando de seccionamento adicional cobrindo o arquivo inteiro, um nível acima do valor `--base-level`. Portanto, para transformar cada arquivo Pod em um capítulo de um manual grande, você deveria usar `section` como o nível base.

Para um exemplo de como transformar o Texinfo a partir da própria documentação do Perl, veja-se `contrib/perl-doc-all` na distribuição do fonte do Texinfo.

`--debug=NUM`

Configurar nível de depuração como *NUM*.

-generate-setfilename

Gerar uma linha `@setfilename` para manuais independentes. Pode ser negado com `--no-generate-setfilename`. Ignorado se `--base-level` não for 0.

-headings-as-sections

Usar comandos de cabeçalhos (`@heading, ...`) em vez dos comandos `@` padrão do Texinfo de seccionamento numerados (`@chapter, @section, ...`). O comando de seccionamento que cobre o arquivo inteiro gerado para cada arquivo Pod se `--base-level` não for 0 é um comando numerado.

-help

Exibir ajuda e sair.

-menus

Gerar menus do nó. Se existir um manual principal, o menu do nó Superior dele será sempre gerado, pois um menu mestre será gerado. Outros menus de nós não são gerados no caso padrão.

-outdir=*NOME*

Se existir um manual principal com arquivos de inclusão (cada um correspondendo a um arquivo Pod de entrada), então os arquivos Texinfo gerados serão colocados no diretório *NOME*. O padrão é baseado em `--subdir`.

-output=*NOME*

Nome para o primeiro manual ou para o manual principal, se existir um manual principal. O padrão é o de escrever na saída padrão.

-no-section-nodes

Usar âncoras para seções em vez de nós.

-no-fill-section-gaps

Não preencher lacunas de seccionamento com arquivos `@unnumbered` vazios. Normalmente, é bom manter intacta a hierarquia de seccionamento.

-preamble=*STR*

Inserir *STR* como padrão superior antes do menu e de inclusões. Se *STR* for configurado como `-`, ler o padrão superior a partir da entrada padrão. O padrão superior é um início mínimo para um documento do Texinfo.

-setfilename=*STR*

Usar *STR* no padrão superior antes do menu e de inclusões para `@setfilename` para o manual principal, se `--base-level` não estiver configurado como 0. Ignorado se `--base-level` for 0. Nenhum `@setfilename` será gerado no caso padrão para o manual principal.

-subdir=*NOME*

Se existir um manual principal com arquivos de inclusão (cada um correspondendo a um arquivo Pod de entrada), então esses arquivos de inclusão serão incluídos a partir de *NAME*.

Se `--outdir` for configurado, *NOME* deveria, em geral, ser configurado para o diretório relativo entre o manual principal e o argumento `--outdir`.

-unnumbered-sections

Usar comandos de seccionamento não numerados (`@unnumbered, ...`) em vez dos comandos `@` de seccionamento numerados padrão do Texinfo (`@chapter, @section, ...`).

-top=*TOP*

Nome do elemento `@top` para o manual principal. Pode conter código Texinfo.

`--version`

Exibir informações da versão e sair.

VEJA-SE TAMBÉM do `pod2texi`

Pod::Simple::Texinfo (<https://www.gnu.org/software/texinfo/manual/Pod/Simple/Texinfo.html>). `perlpod` (<https://metacpan.org/pod/perlpod>). O manual do Texinfo. Página inicial do Texinfo: <https://www.gnu.org/software/texinfo/>

COPIRRAITE E LICENÇA do `pod2texi`

Copirraite 2012-2024 Free Software Foundation, Inc.

Este programa é software livre; você pode redistribuí-lo e(ou) modificá-lo sob os termos da Licença Pública Geral GNU conforme publicada pela Free Software Foundation; ou a versão 3 da Licença ou, (a teu critério), qualquer versão posterior.

NÃO EXISTE GARANTIA, na medida permitida pela legislação.

AUTOR do `pod2texi`

Patrice Dumas <bug-texinfo@gnu.org>.

20.7 `texi2html`: Ancestral do `texi2any`

Conceitualmente, o programa `texi2html` é o ancestral do programa `texi2any` atual. `texi2html` foi desenvolvido independentemente, originalmente por Lionel Cons em 1998; na época, `makeinfo` não conseguia gerar HTML. Muitas outras pessoas contribuíram para o `texi2html` ao longo dos anos.

O presente `texi2any` usa pouco do código real do `texi2html` e tem uma abordagem básica bastante diferente para a implementação (ou seja, analisar o documento Texinfo em uma árvore), mas ainda assim existe uma semelhança familiar.

Pelo projeto, o `texi2any` suporta quase todos os recursos do `texi2html` de alguma maneira. No entanto, nós não tentamos manter compatibilidade estrita, de forma que nenhum executável `texi2html` é instalado pelo pacote Texinfo. Uma aproximação pode ser executada com uma invocação como esta:

```
texi2any --set-customization-variable TEXI2HTML=1 ...
```

porém, para enfatizar, isso *não* é um substituto imediato para o `texi2html` anterior. Aqui estão as maiores diferenças:

- Mais descaradamente, as opções de linha de comando do `texi2html` agora são variáveis de personalização, em sua maior parte. Uma tabela de equivalentes aproximados é fornecida abaixo.
- A API de personalização em nível de programa é muito diferente em `texi2any`.
- Índices não podem ser divididos.

Nós não pretendemos reimplementar essas diferenças. Portanto, o caminho a seguir para autores(as) é o de alterar manuais e construir processos, conforme necessário, para usar os novos recursos e métodos do `texi2any`. Os(As) mantenedores(as) do `texi2html` (um dos quais é o autor principal do `texi2any`) não pretendem fazer novos lançamentos.

Aqui está a tabela que mostra as opções do `texi2html` e as variáveis de personalização correspondentes do `texi2any`.

<code>--toc-links</code>	TOC_LINKS
<code>--short-ext</code>	EXTENSION configurada para 'htm'

<code>--prefix</code>	<code>PREFIX</code>
<code>--def-table</code>	<code>DEF_TABLE</code>
<code>--html-xref-prefix</code>	<code>EXTERNAL_DIR</code>
<code>--l2h</code>	<code>HTML_MATH</code> configurada para ‘ <code>l2h</code> ’
<code>--l2h-l2h</code>	<code>L2H_L2H</code>
<code>--l2h-skip</code>	<code>L2H_SKIP</code>
<code>--l2h-tmp</code>	<code>L2H_TMP</code>
<code>--l2h-file</code>	<code>L2H_FILE</code>
<code>--l2h-clean</code>	<code>L2H_CLEAN</code>
<code>--use-nodes</code>	<code>USE_NODES</code>
<code>--monolithic</code>	<code>MONOLITHIC</code>
<code>--top-file</code>	<code>TOP_FILE</code>
<code>--frames</code>	sem equivalente
<code>--menu</code>	<code>FORMAT_MENU</code>
<code>--debug</code>	<code>DEBUG</code>
<code>--doctype</code>	<code>DOCTYPE</code>
<code>--frameset-doctype</code>	sem equivalente
<code>--test</code>	<code>TEST</code>

Finalmente, quaisquer usuários(as) do `texi2html` que busquem informações mais detalhadas consegue verificar a primeira parte do arquivo `doc/texi2oldapi.texi` no repositório do fonte do Texinfo.

21 Criando e Instalando Arquivos Info

Este capítulo fornece algumas informações acerca da saída Info e descreve como instalar arquivos Info. Para a criação de arquivos Info com o `texi2any`, veja-se Capítulo 20 [Tradutor Genérico `texi2any`], Página 157, e com o Emacs, Seção D.6 [Formatação do Info], Página 261. Veja-se Seção 1.3 [Arquivos do Info], Página 4, para informações gerais acerca do formato do arquivo. Veja-se Anexo F [Especificação do Formato Info], Página 282, para uma especificação técnica detalhada do formato Info.

21.1 Instalando um Arquivo do Info

Os arquivos Info geralmente são mantidos no diretório `info`. Você pode ler arquivos Info usando o programa autônomo Info ou o leitor Info integrado ao Emacs. (Veja-se *Info*, para uma introdução ao Info).

21.1.1 O Arquivo de Diretório `dir`

Para que o Info funcione, o diretório `info` precisa conter um arquivo que sirva como diretório de nível superior para o sistema Info. Por convenção, esse arquivo é chamado de `dir`. (Você pode encontrar o local desse arquivo no Emacs digitando `C-h i` para inserir Info e depois digitando `C-x C-f` para ver o local do diretório `info`).

O arquivo `dir` é em si um arquivo Info. Ele contém o menu de nível superior para todos os arquivos Info no sistema. O menu se parece com isto:

```
* Menu:
* Info:      (info).      Sistema de navegação de documentação.
* Emacs:     (emacs).     O editor de texto extensível e
                          autodocumentado.
* Texinfo:   (texinfo).   Com um arquivo fonte, faça ou um manual impresso
                          usando o @TeX{} ou um arquivo Info.
...

```

Cada uma dessas entradas de menu aponta para o nó ‘Top’ do arquivo Info que esteja nomeado entre parênteses. (A entrada do menu não precisa especificar o nó ‘Top’, uma vez que o Info vai para o nó ‘Top’ se nenhum nome de nó for mencionado. Veja-se Seção 3.9.6 [Nós em Outros Arquivos do Info], Página 35).

Assim, a entrada ‘Info’ aponta para o nó ‘Top’ do arquivo `info` e a entrada ‘Emacs’ aponta para o nó ‘Top’ do arquivo `emacs`.

Em cada um dos arquivos do Info, o ponteiro ‘Acima’ do nó ‘Top’ referencia de volta para o arquivo `dir`. Por exemplo, a linha para o nó ‘Topo’ do manual do Emacs se parece com isto no Info:

```
File: emacs Node: Top, Up: (DIR), Next: Distribuição

```

Nesse caso, o nome do arquivo `dir` é escrito em letras maiúsculas—ele pode ser escrito ou em letras maiúsculas ou minúsculas. Isso não é verdade em geral; é um caso especial para `dir`.

Veja-se o arquivo `util/dir-example` na distribuição Texinfo para uma amostra grande do arquivo `dir`.

21.1.2 Listando um Novo Arquivo Info

Para adicionar um novo arquivo Info ao teu sistema, você precisa escrever uma entrada de menu para adicionar ao menu no arquivo `dir` no diretório `info`. Por exemplo, se estivesse adicionando documentação para o GDB, você escreveria a seguinte nova entrada:

```
* GDB: (gdb).          O depurador C de nível de fonte.

```

A primeira parte da entrada do menu é o nome da entrada do menu, seguido por dois pontos. A segunda parte é o nome do arquivo Info, entre parênteses, seguido de um ponto final. A terceira parte é a descrição.

O nome de um arquivo Info frequentemente tem uma extensão `.info`. Assim, o arquivo Info para GDB pode ser chamado ou de `gdb` ou de `gdb.info`. Os programas leitores do Info tentam automaticamente o nome do arquivo com e sem `.info`¹; então é melhor evitar confusão e não escrever `‘.info’` explicitamente na entrada do menu. Por exemplo, a entrada do menu GDB deveria usar somente `‘gdb’` para o nome do arquivo, não `‘gdb.info’`.

outros diretórios.

21.1.3 Arquivos Info em Outros Diretórios

Se um arquivo Info não estiver no diretório `info`, existem três maneiras de especificar o local dele:

1. Escreva o nome do arquivo no arquivo `dir` como a segunda parte do menu.
2. Especifique o nome do diretório Info na variável de ambiente `INFOPATH` em teu arquivo de inicialização `.profile` ou `.cshrc`. (Somente você e outras pessoas que configurarem essa variável de ambiente estarão apto(a) para encontrar arquivos Info cujo local esteja especificado dessa maneira).
3. Se você estiver usando Emacs, liste o nome do arquivo em um segundo arquivo `dir`, no diretório dele; e, em seguida, adicione o nome desse diretório à variável `Info-directory-list` em teu arquivo de inicialização pessoal ou no do sítio.

Essa variável informa ao Emacs onde procurar por arquivos `dir` (os arquivos precisam ser nomeados `dir`). O Emacs mescla os arquivos nomeados de `dir` a partir de cada um dos diretórios listados. (No Emacs versão 18, você consegue configurar a variável `Info-directory` como o nome de somente um diretório).

Por exemplo, para acessar um arquivo de teste no diretório `/home/bob/info`, você poderia adicionar uma entrada como esta ao menu no arquivo `dir` padrão:

*** Teste: (/home/bob/info/info-teste). Arquivo próprio de teste do Bob.**

Nesse caso, o nome absoluto de arquivo do arquivo `info-teste` é escrito como a segunda parte da entrada do menu.

Se não quiser editar o arquivo `dir` do sistema, você pode informar ao Info onde procurar configurando a variável de ambiente `INFOPATH` em teu arquivo de iniciação do shell. Isso funciona tanto com o Emacs quanto com leitores Info independentes.

O Emacs usa a variável de ambiente `INFOPATH` para inicializar o valor da variável `Info-directory-list` do próprio Emacs. O leitor Info independente mescla todos os arquivos chamados `dir` em qualquer diretório listado na variável `INFOPATH` em um menu apresentado a você no nó chamado `‘(dir)Top’`.

Independentemente de como você configurar `INFOPATH`, se o último caractere dela for dois pontos (em sistemas MS-DOS/MS-Windows, use um ponto e vírgula), ele será substituído pelo caminho padrão (compilado internamente). Isso oferece a você uma maneira de aumentar o caminho padrão com novos diretórios sem precisar listar todos os locais padrão. Por exemplo (usando a sintaxe do `sh`):

```
INFOPATH=/home/bob/info:
export INFOPATH
```

pesquisar primeiro `/home/bob/info` e depois os diretórios padrão. Os dois pontos iniciais ou duplos não são tratados especialmente.

¹ Em sistemas MS-DOS/MS-Windows, o Info também tentará a extensão `.inf`.

Quando você cria teu próprio arquivo `dir` para usar com `Info-directory-list` ou com `INFOPATH`, é mais fácil começar copiando um arquivo `dir` existente e substituir todo o texto depois de `* Menu:` pelas tuas entradas desejadas. Dessa maneira, a pontuação e os caracteres especiais `CTRL-` que o Info precisa estarão presentes.

Como uma alternativa final, que funciona somente com o Info do Emacs, você consegue mudar a variável `Info-directory-list`. Por exemplo:

```
(add-hook 'Info-mode-hook '(lambda ()
  (add-to-list 'Info-directory-list
    (expand-file-name "~/info")))))
```

diretório Info.

21.1.4 Instalando Arquivos de Diretório Info

Ao instalar um arquivo Info em teu sistema, você pode usar o programa `install-info` para atualizar o arquivo do diretório Info `dir`. Normalmente o makefile do pacote executa `install-info`, logo depois de copiar o arquivo Info para o local de instalação apropriado dele.

Para a finalidade de que o arquivo Info funcione com `install-info`, você inclui os comandos `@dircategory` e `@direntry...@end direntry` no arquivo fonte do Texinfo.

Usar `@dircategory` para especificar uma categoria para o manual, que determina em qual parte do diretório Info colocá-lo. (Veja-se Seção 2.6 [Categoria de Diretório], Página 14). Usar `@direntry` para especificar a entrada ou entradas de menu a serem adicionadas ao arquivo do diretório Info. Alguns manuais (como o manual que você está lendo agora, ou o do GNU Coreutils; veja-se *GNU Coreutils*) terão diversas entradas. É típico ter uma entrada de diretório para cada programa que um pacote instala, geralmente para o nó ‘Invocando’ do programa. Isso permite que os(as) usuários(as) encontrem facilmente a documentação para os diferentes programas que podem executar. Da mesma forma, um manual para uma biblioteca C pode instalar uma entrada de diretório para cada função que fornece.

Aqui está como esses comandos são usados neste manual:

```
@dircategory Sistema de documentação do Texinfo
@direntry
* Texinfo: (texinfo).          O formato da documentação GNU.
* install-info: (texinfo)Invocando install-info. ...
...
@end direntry
```

Aqui está o que isso produz no arquivo Info:

```
INFO-DIR-SECTION Sistema de documentação do Texinfo
START-INFO-DIR-ENTRY
* Texinfo: (texinfo).          O formato da documentação GNU.
* install-info: (texinfo)Invocando install-info. ...
...
END-INFO-DIR-ENTRY
```

O programa `install-info` vê essas linhas no arquivo Info e é assim que ele sabe o que fazer.

Sempre use os comandos `@direntry` e `@dircategory` próximos ao início da entrada do Texinfo, antes do primeiro comando `@node`. Se você usá-los posteriormente na entrada, `install-info` não os notará.

`install-info` automaticamente reformatará a descrição das entradas de menu que estiver adicionando. Por uma questão de convenção, a descrição da entrada principal (acima, ‘O formato da documentação GNU’) deveria começar na coluna 32, começando em zero (como em `what-cursor-position` no Emacs). Isso fará com que ele se alinhe com a maioria dos outros. É melhor que a descrição para utilitários individuais comece na coluna 48, sempre que possível. Para mais

informações acerca de formatação, vejam-se as opções ‘--calign’, ‘--align’ e ‘--max-width’ em Seção 21.1.5 [Invocando `install-info`], Página 186.

Se você usar `@dircategory` mais de uma vez no fonte do Texinfo, cada uso especificará a categoria ‘atual’; quaisquer comandos `@direntry` subsequentes adicionarão a essa categoria.

21.1.5 Invocando `install-info`

`install-info` insere entradas de menu originárias de um arquivo Info no arquivo `dir` de nível superior no sistema Info (vejam-se as seções anteriores para uma explicação de como o arquivo `dir` funciona). `install-info` também remove entradas de menu do arquivo `dir`. Geralmente é executado como parte da instalação do software ou ao construir um arquivo `dir` para todos os manuais em um sistema. Sinopse:

```
install-info [opção...] [arquivo-info [arquivo-dir]]
```

Se *arquivo-info* ou *arquivo-dir* não forem especificados, as opções (descritas abaixo) que os definem precisarão ser. Não existem padrões de tempo de compilação e a entrada padrão nunca é usada. `install-info` consegue ler somente um arquivo Info e escrever somente um arquivo `dir` por invocação.

Se *arquivo-dir* (independentemente de como especificado) não existir, `install-info` o criará, se possível (sem entradas).

Se algum arquivo de entrada for comprimido com `gzip` (veja-se *Gzip*), `install-info` o descomprimirá automaticamente para leitura. E se *arquivo-dir* estiver comprimido, `install-info` também o deixará comprimido automaticamente depois de escrever quaisquer mudanças. Se o próprio *arquivo-dir* não existir, `install-info` tenta abrir *arquivo-dir.gz*, *arquivo-dir.xz*, *arquivo-dir.bz2*, *arquivo-dir.lz* e *arquivo-dir.lzma*, nessa ordem.

Opções:

--add-once

Especifica que a entrada ou entradas serão colocadas somente em uma seção.

--align=coluna

Especifica a coluna na qual a segunda linha e as subsequentes da descrição da entrada de menu serão formatadas para começar. O padrão para essa opção é ‘35’. É usada em conjunto com a opção ‘--max-width’. *coluna* começa a contar em 1.

--append-new-sections

Em vez de colocar as novas seções em ordem alfabética, coloque-as no final do arquivo DIR.

--calign=coluna

Especifica a coluna na qual a primeira linha da descrição da entrada do menu será formatada para começar. O padrão para essa opção é ‘33’. É usada em conjunto com a opção ‘--max-width’. Quando o nome da entrada do menu exceder essa coluna, a descrição da entrada começará na linha seguinte. *coluna* começa a contar em 1.

--debug Informa o que está sendo feito.

--delete Deletar as entradas em *arquivo-info* originárias de *arquivo-dir*. O nome do arquivo na entrada em *arquivo-dir* precisa ser *arquivo-info* (exceto por um ‘.info’ opcional em qualquer um deles). Não insira quaisquer novas entradas. Quaisquer seções vazias que resultem da remoção também serão removidas.

--description=texto

Especificar a parte explicativa da entrada do menu. Se você não especificar uma descrição (ou por meio de ‘--entry’, ‘--item’ ou dessa opção), a descrição será obtida a partir do próprio arquivo Info.

--dir-file=nome
Especificar o nome do arquivo do diretório Info. Isso é equivalente a se usar o argumento *arquivo-dir*.

--dry-run
Mesmo que '**--test**'.

--entry=texto
Inserir *texto* como uma entrada do diretório Info; *texto* deveria ter a forma de uma linha de item do menu do Info mais zero ou mais linhas extras começando com espaço em branco. Se você especificar mais que uma entrada, todas elas serão adicionadas. Se você não especificar quaisquer entradas, elas serão determinadas a partir das informações no próprio arquivo Info.

--help
Exibir uma mensagem de uso com o uso básico e todas as opções disponíveis e sair com sucesso.

--info-file=arquivo
Especificar o arquivo Info para instalar no diretório. Isso é equivalente a se usar o argumento *arquivo-info*.

--info-dir=dir
Especificar o diretório onde reside o arquivo de diretório *dir*. Equivalente a '**--dir-file=diretório/dir**'.

--infodir=diretório
Mesmo que '**--info-dir**'.

--item=texto
Mesmo que '**--entry=texto**'. Uma entrada de diretório é na verdade um item de menu.

--keep-old
Não substituir entradas preexistentes de menu. Quando '**--remove**' for especificada, essa opção significa que seções vazias não serão removidas.

--max-width=coluna
Especifica a coluna na qual a descrição da entrada do menu será quebrada. *coluna* começa a contar em 1.

--maxwidth=coluna
Mesmo que '**--max-width**'.

--menuentry=texto
Mesmo que '**--name**'.

--name=texto
Especificar a parte do nome da entrada do menu. Se o *texto* não começar com um asterisco '*', será presumido ser o texto depois do '*' e antes dos parênteses que especifica o arquivo Info. Caso contrário, *texto* será tomado literalmente e será tomado como definindo o texto até e incluindo o primeiro ponto (um espaço será posposto se necessário). Se você não especificar o nome (seja por meio de '**--entry**', de '**--item**' ou dessa opção), ele será obtido a partir do próprio arquivo Info. Se o Info não contiver o nome, o nome base do arquivo Info será usado.

--no-indent
Suprimir formatação de novas entradas no arquivo *dir*.

--quiet

--silent
Suprimir avisos, entre outras, para operação silenciosa.

`--remove` Mesmo que `--delete`.

`--remove-exactly`

Também como `--delete`, mas somente entradas se o nome do arquivo Info corresponder exatamente; sufixos `.info` e(ou) `.gz` não são ignorados.

`--section=seção`

Colocar as entradas desse arquivo na seção *seção* do diretório. Se você especificar mais que uma seção, todas as entradas serão adicionadas em cada uma das seções. Se você não especificar nenhuma seção, elas serão determinadas a partir das informações no próprio arquivo Info. Se o arquivo Info não especificar uma seção, as entradas do menu serão colocadas na seção Diversos.

`--section expressãoregular seção`

Mesmo que `--regex=expressãoregular --section=seção --add-once`.

`install-info` tenta detectar quando essa sintaxe alternativa for usada, mas nem sempre adivinha corretamente. Aqui está a heurística que `install-info` usa:

1. Se o segundo argumento para `--section` começar com um hífen, a sintaxe original será presumida.
2. Se o segundo argumento para `--section` for um arquivo que possa ser aberto, a sintaxe original será presumida.
3. Caso contrário, a sintaxe alternativa será usada.

Quando a heurística falhar porque teu título de seção começa com um hífen ou acontecer de ser um arquivo que possa ser aberto, a sintaxe deveria ser mudada para `--regex=expressãoregular --section=seção --add-once`.

`--regex=expressãoregular`

Colocar entradas desse arquivo em alguma seção que corresponda a *expressãoregular*. Se mais que uma seção corresponder, todas as entradas serão adicionadas em cada uma das seções. Especifique *expressãoregular* usando sintaxe básica de expressão regular, mais ou menos como usada com `grep`, por exemplo.

`--test` Suprimir atualização do arquivo de diretório.

`--version`

Exibir informações de versão e sair exitosamente.

21.2 Arquivos de Etiqueta e Arquivos Divididos

Arquivos Info sempre contêm uma *tabela de etiquetas*, para poder saltar rapidamente para nós. Arquivos Info podem ser *não divididos* (também chamados *indivisíveis*) ou *divididos*.

Se o arquivo Info contiver menos que 300.000 caracteres, o arquivo deveria ser não dividido. Nesse caso, a tabela de etiquetas deveria aparecer no final do arquivo Info. Se o arquivo Texinfo contiver mais que 300.000 caracteres, os processadores Texinfo dividirão o grande arquivo Info em subarquivos *indiretos* menores de cerca de 300.000 caracteres cada. Com `texi2any`, a divisão pode ser evitada por `--no-split`, e o tamanho padrão de 300.000 caracteres pode ser modificado com `--split-size` (veja-se Seção 20.1 [Invocando `texi2any`], Página 157).

Quando um arquivo for dividido, o próprio Info faz uso uma versão abreviada do arquivo original que contém apenas a tabela de etiquetas e referências para os arquivos que foram divididos. Os arquivos divididos são chamados de arquivos *indiretos*.

Os arquivos divididos têm nomes que são criados pospondo `'-1'`, `'-2'`, `'-3'` e assim por diante ao nome do arquivo de saída, especificado pelo comando `@setfilename` ou pelo nome do arquivo de entrada. A versão abreviada do arquivo original continua tendo o nome especificado por `@setfilename` ou o nome do arquivo de entrada.

Em um estágio da redação deste documento, por exemplo, o arquivo Info foi salvo como o arquivo `test-texinfo` e esse arquivo se parecia com isto:

```
Arquivo Info: test-texinfo,    -*-Text-*-
produzido pelo texinfo-format-buffer
a partir do arquivo: new-texinfo-manual.texinfo

^_
Indireto:
test-texinfo-1: 102
test-texinfo-2: 50422
test-texinfo-3: 101300
^_~L
Tabela de etiquetas:
(Indireto)
Nó: visão geral^?104
Nó: arquivo info^?1271
Nó: manual impresso^?4853
Nó: convenções^?6855
...
```

(Mas `test-texinfo` tinha muito mais nós que os mostrados aqui). Cada um dos arquivos indiretos, divididos, `test-texinfo-1`, `test-texinfo-2` e `test-texinfo-3`, está listado nesse arquivo depois da linha que diz ‘Indireto:’. A tabela de etiquetas está listada depois da linha que diz ‘Tabela de etiquetas:’.

Na lista de arquivos indiretos, o número seguinte ao nome do arquivo registra o número cumulativo de baítes nos arquivos indiretos precedentes, sem contar a própria lista de arquivos, a tabela de etiquetas ou qualquer texto de permissão no primeiro arquivo. Na tabela de etiquetas, o número seguinte ao nome do nó registra o local do início do nó, em baítes a partir do início da saída gerada (indivisível).

Se você estiver usando `texinfo-format-buffer` para criar arquivos Info, você pode querer executar o comando `Info-validate`. (O comando `texi2any` faz um trabalho tão bom por si só que você não precisa de `Info-validate`). No entanto, você não consegue executar o comando de verificação de nó `M-x Info-validate` sobre arquivos indiretos. Para informações acerca de como evitar que arquivos sejam divididos com `texinfo-format-buffer` e como validar a estrutura dos nós, veja-se Seção D.10.4.1 [Usando `Info-validate`], Página 272.

21.3 Perguntas Frequentes do Formato Info

Aqui estão algumas perguntas que tem sido feitas frequentemente nas listas de discussão e em outros lugares.

Por que quando executo ‘`info foo`’ eu obtenho a mesma saída que ‘`man foo`’?

Verifique se os manuais do Info estão instalados. Nem todas as distribuições GNU/Linux instalam todos os manuais do Info por padrão. Isso é lamentável, pois frequentemente o manual do Info fornece mais informações que a página de manual fornecida.

Por que não usar HTML em vez do Info?

Manuais raramente são escritos no próprio formato Info, mas são gerados a partir do fonte Texinfo pelo programa `texi2any`. `texi2any` consegue gerar HTML e também Info a partir do fonte Texinfo. Você também consegue acessar e baixar manuais em HTML a partir do sítio web do GNU (<https://www.gnu.org/manual/manual.html>). Além disso, algumas distribuições GNU/Linux fornecem pacotes para a instalação de manuais HTML.

Info ainda tem algumas vantagens sobre HTML para documentação instalada localmente. Os navegadores do Info suportam pesquisa de índice e suportam links entre manuais instalados localmente em vários locais. É importante ter-se documentação instalada localmente em tua máquina em vez de depender da Internet; isso torna o acesso à documentação mais confiável e garante que ela corresponda às versões instaladas dos pacotes. Espera-se que o suporte para navegar na documentação do Texinfo instalada localmente em alguma forma de HTML possa ser melhorado no futuro.

Por que fornecer o programa de linha de comando `info` quando o leitor Info do Emacs é melhor?

O leitor Info do Emacs consegue exibir imagens e suporta completamente usar um mouse. Não existem muitas diferenças entre os leitores Info além disso. `info` de linha de comando pode ser configurado para mudar as combinações padrão de teclas, bem como mudar a cor de referências cruzadas e de resultados de pesquisa, e para habilitar suporte à rolagem do mouse. Você não deveria precisar ter experiência com o editor Emacs para usar `info`. (Alguns(mas) recomendam outro programa chamado `pinfo`, mas esse carece de recursos importantes como pesquisa de índice). Alguns(mas) preferem poder percorrer o manual inteiro de uma vez (como acontece com páginas de manual), em vez de ficarem limitados(as) a um ‘nó’ de conteúdo por vez. É claro que não existe consideração pelo gosto, mas um bloco de texto não estruturado torna-se mais estranho à medida que um manual se torna mais longo e mais complicado. (No entanto, se realmente quiser, você consegue visualizar um manual Info de uma só vez no paginador `less` com ‘`info foo | less`’).

Por que eu tenho ‘veja-se’ aparecendo em frente de todos os meus links?

Por padrão, o modo Info do Emacs ou muda o marcador ‘`*note`’ para referências cruzadas para ‘veja-se’ ou o oculta completamente. `info` de linha de comando faz o mesmo se `hide-note-references` estiver configurado. Infelizmente, não existe maneira de fazer isso confiavelmente, já que ambos os comandos `@pxref` e `@ref` no Texinfo geram esse marcador na saída Info, e o texto ‘veja-se’ é apropriado somente para `@pxref`.

Sim, mas como eu obtenho um link simples, sem marcações extras?

Você não pode. Info é um formato de texto simples que é exibido principalmente como está nos visualizadores e sem o texto ‘`*note`’ não existiria nada para marcar o texto como um link.

Para formatos de saída como HTML, você pode usar o comando `@link` para produzir um link simples. Veja-se Seção 5.12 [`@link`], Página 50. No entanto, isso não produz uma referência cruzada funcional na saída Info ou em uma cópia impressa do manual.

Por que linhas contendo links parecem misteriosamente curtas?

Isso se deve ao Emacs (ou `info` com `hide-note-references` configurado como ‘`On`’) ocultando sequências de caracteres ‘`*note`’, conforme mencionado acima.

Pode o formato Info ser estendido para suportar fontes, cores ou texto refluível?

Qualquer extensão seria incompatível com programas de visualização do Info existentes. Marcações extras adicionadas aos arquivos Info seriam exibidas para o(a) usuário(a), tornando os arquivos feios e ilegíveis.

Quando arquivos Texinfo são processados em arquivos Info, as informações acerca de quais comandos do Texinfo foram usados para marcar o texto são perdidas. Além disso, não é possível detectar com segurança se o texto consegue ser refluído (por exemplo um parágrafo de prosa) ou se as quebras de linha deveriam ser mantidas (por exemplo uma amostra de código ou um poema).

O propósito central do Info é o de exibir documentação em terminais de texto. Se você quiser mais, é recomendável usar a saída HTML proveniente do `texi2any`.

22 Gerando HTML

`texi2any` gera saída Info por padrão, mas dada a opção `--html`, ele gerará HTML, para navegadores da web e outros programas. Este capítulo fornece alguns detalhes acerca dessa saída HTML.

`texi2any` tem muitas variáveis de personalização definidas pelo(a) usuário(a) com as quais você consegue influenciar a saída HTML. Veja-se Seção 22.8 [Personalização da Saída Gerada HTML], Página 204. Em particular, existe suporte para realçamento de sintaxe no `@example` (veja-se Seção 22.6 [Realçamento de Sintaxe], Página 196). Você também pode escrever os assim chamados *arquivos de inicialização*, ou *arquivos init* para abreviar, para modificar quase todos os aspectos da saída gerada HTML. Os arquivos de inicialização contêm código e são carregados por `--init-file` (veja-se Seção 20.1 [Invocando `texi2any`], Página 157).

Alguns arquivos de inicialização são mantidos com o Texinfo e instalados no caso padrão. Por exemplo, `chm.pm` produz os arquivos comprimidos intermediários no formato Help do HTML que podem ser posteriormente convertidos para o formato CHM.

A documentação da adaptação da saída HTML do `texi2any` usando arquivos de personalização está em um manual separado. Veja-se *Personalização da Saída do texi2any do GNU Texinfo*.

22.1 Tradução do HTML

O HTML gerado pelo `texi2any` gera saída HTML padrão. A saída gerada é intencionalmente bastante simples para máxima portabilidade e acessibilidade.

Você consegue personalizar a saída gerada via CSS (veja-se Seção 22.3 [CSS do HTML], Página 193) ou outros meios (veja-se Seção 22.8 [Personalização da Saída Gerada HTML], Página 204). Se você não conseguir realizar uma personalização razoável, sintá-se à vontade para informar isso.

Barra de navegação: Por padrão, uma barra de navegação é inserida no início de cada nó, análogo à saída gerada Info. Se a opção `--no-headers` for usada, a barra de navegação só será inserida no início dos arquivos divididos. Elementos de cabeçalho `<link>` na saída dividida suportam navegação semelhante à Info com navegadores que implementem esse recurso.

HTML bruto: `texi2any` incluirá segmentos do fonte do Texinfo entre `@ifhtml` e `@end ifhtml` na saída gerada HTML (mas não qualquer dos outros condicionais, por padrão). Fonte entre `@html` e `@end html` é passado sem mudança para a saída gerada (ou seja, suprimindo a escapagem normal dos caracteres de entrada `<`, `>` e `&` os quais têm significância especial no HTML). Veja-se Seção 16.1 [Comandos Condicionais], Página 129.

Padrões: Não é nosso objetivo intencionalmente, e nem mesmo sempre possível, passar por cada teste de validação concebível sem quaisquer diagnósticos. Diferentes testes de validação têm objetivos diferentes, frequentemente acerca da imposição pedante de algum padrão ou de outro. Nosso objetivo primordial é o de ajudar usuários(as), não o de cumprir cegamente os padrões.

Por favor, informe a saída gerada proveniente de uma execução sem erros do `texi2any` que tenha problemas *práticos* de portabilidade do navegador ou do leitor de EPUB como um defeito (veja-se Seção 1.1 [Informando Defeitos], Página 2).

Na prática, o HTML produzido pelo `texi2any` é lentamente ajustado ao longo do tempo para o padrão HTML mais recente, enquanto também tenta manter compatibilidade com HTML produzido anteriormente. Nós usamos marcação de transição e tentamos ser lentos o suficiente para dar tempo para os diversos leitores de HTML se ajustarem (e para os padrões reincorporarem recursos úteis que foram eliminados. . .).

22.2 Dividindo HTML

Ao dividir a saída gerada em nós (que é o padrão), `texi2any` escreve a saída gerada HTML em (basicamente) um arquivo de saída por `@node` do fonte do Texinfo.

Cada nome de arquivo da saída gerada é o nome do nó com espaços substituídos por ‘-’ e caracteres especiais mudados para ‘_’ seguidos pelo ponto de código deles em hexadecimal (veja-se Seção 22.7 [Xref do HTML], Página 197). Isso é para torná-lo portátil e fácil de usar como um nome de arquivo. No caso incomum de dois nós terem o mesmo nome depois desse tratamento, eles serão escritos consecutivamente no mesmo arquivo, com âncoras HTML, de modo que cada um possa ser referido independentemente.

Se `texi2any` for executado em um sistema que não distinga maiúsculas e minúsculas em nomes de arquivos, nós que sejam iguais, exceto pelas maiúsculas e minúsculas (por exemplo, ‘`index`’ e ‘`Index`’), também serão inseridos no mesmo arquivo de saída com âncoras. Você também consegue fingir que está em um sistema de arquivos que não diferencia maiúsculas de minúsculas, configurando a variável de personalização `CASE_INSENSITIVE_FILENAMES`.

Também é possível dividir nos capítulos ou nas seções com `--split` (veja-se [Dividindo Saída Gerada], Página 163). Nesse caso, os nomes dos arquivos serão construídos depois do nome do nó associado com o comando de seccionamento relevante. Além disso, a menos que `--no-node-files` seja especificada, um arquivo de redireção será gerado para cada nó, para a finalidade de suportar mais confiavelmente as referências cruzadas para esse manual (veja-se Seção 22.7 [Xref do HTML], Página 197).

Ao dividir, os arquivos da saída gerada HTML serão escritos em um subdiretório. O nome do subdiretório é derivado do nome base (ou seja, qualquer extensão será removida), com `_html` posposto. Por exemplo, a saída HTML para `gcc.texi` seria escrita em um subdiretório nominado de ‘`gcc_html/`’. O nome do subdiretório é baseado no `@setfilename` ou no nome do arquivo de entrada (veja-se [Configurando o Nome do Arquivo da Saída Gerada], Página 12).

Em qualquer caso, o arquivo da saída gerada do nível superior dentro do diretório é sempre nominado ‘`index.html`’.

A saída monolítica (`--no-split`) será nominada de acordo com `@setfilename`, se presente, (com qualquer extensão ‘`.info`’ substituída por ‘`.html`’); `--output` (o argumento será usado literalmente); ou baseada no nome do arquivo de entrada como um último recurso (veja-se [Configurando o Nome do Arquivo da Saída Gerada], Página 12).

22.3 CSS do HTML

Folhas de Estilo em Cascata (CSS) é um padrão da Internet para influenciar a exibição de documentos HTML veja-se <http://www.w3.org/Style/CSS/>.

Por padrão, algum código CSS é incluído para implementar melhor a aparência de alguns ambientes do Texinfo. Por exemplo:

```
pre.display { font-family:inherit }
```

O acima diz ao navegador da web para usar a mesma fonte do documento principal dentro de elementos ‘`<pre>`’ usados para ambientes `@display`. Por padrão, o comando ‘`<pre>`’ do HTML usa uma fonte monoespçada.

Por favor, veja-se a referência acima para uma explicação completa do CSS.

Você consegue influenciar o CSS na saída HTML com duas opções do `texi2any`: `--css-include=arquivo` e `--css-ref=url`.

A opção `--css-ref=url` adiciona para cada arquivo de saída HTML uma etiqueta ‘`<link>`’ referenciando um CSS no `url` fornecido. Isso permite usar-se folhas de estilo externas.

A opção `--css-include=arquivo` inclui o conteúdo de `arquivo` na saída HTML, como você esperaria. No entanto, os detalhes são um tanto complicados, conforme descrito a seguir, para fornecer flexibilidade máxima.

A primeira linha do arquivo CSS possivelmente seja uma diretiva ‘@charset’. Se presente, essa diretiva será usada para determinar a codificação do arquivo CSS. A linha não será copiada na saída gerada.

O arquivo CSS possivelmente comece com as assim chamadas diretivas ‘@import’, as quais lincam especificações externas CSS para navegadores usarem ao interpretarem o documento. Novamente, uma descrição completa está além do nosso escopo aqui, mas nós descreveremos como elas funcionam sintaticamente, de modo que consigamos explicar como são manuseadas.

Pode existir mais que uma ‘@import’, mas elas tem de vir primeiro no arquivo, somente com espaços em branco e comentários intercalados, sem definições normais. Comentários em arquivos CSS são delimitados por ‘/* ... */’, como na C. Uma diretiva ‘@import’ precisa estar em uma destas duas formas:

```
@import url(http://exemplo.org/foo.css);
@import "http://exemplo.net/bar.css";
```

Os caracteres cruciais são o ‘@’ no início e o ponto e vírgula terminando a diretiva. Ao ler o arquivo CSS, quaisquer das tais diretivas ‘@’ simplesmente serão copiadas na saída gerada, conforme segue:

- Se *arquivo* contiver somente declarações normais CSS, ele será incluído depois do CSS padrão, portanto substituindo-o.
- Se *arquivo* iniciar com especificações ‘@import’ (veja-se abaixo), então as ‘import’ serão incluídas primeiro (elas têm de vir primeiro, de acordo com o padrão), e então o CSS padrão será incluído. Se precisar substituir o CSS padrão a partir de uma ‘@import’, você consegue fazer isso com o constructo ‘! important’ do CSS, como em:

```
pre.example { font-size: inherit ! important }
```

- Se *arquivo* contiver ambos, ‘@import’ e CSS inline especificações, as ‘@import’ serão incluídas primeiro, então CSS padrão e, por último, o CSS inline proveniente de *arquivo*.
- Qualquer outra diretiva @ diferente de ‘@import’ e de ‘@charset’ será tratada como uma declaração CSS, significando que o CSS padrão será incluído e a seguir o restante do arquivo será preposto.

Se o arquivo CSS estiver malformado ou incorreto, a saída gerada será inespecificada. O significado do arquivo CSS não será interpretado de maneira alguma; os caracteres especiais ‘@’ e ‘;’ serão procurados para que o texto seja copiado cegamente na saída gerada. Comentários no arquivo CSS podem ou não serem incluídos na saída gerada.

Por padrão, classes são adicionadas aos elementos HTML e regras CSS para essas classes são geradas no início de cada arquivo HTML. Configure a variável de personalização NO_CSS para evitar classes sendo adicionadas e CSS sendo usado. Configure INLINE_CSS_STYLE para ter código CSS colocado em elementos HTML no atributo `style` em vez de no início da saída gerada.

Como `texi2any` insere somente código CSS necessário para o HTML realmente gerado, a lista completa do código CSS potencialmente inserido não é visível no documento da saída gerada. Para obter a lista completa de regras CSS, chame `texi2any` com a variável de personalização SHOW_BUILTIN_CSS_RULES configurada para gerar, na saída padrão, as regras CSS padrão integradas e sair, como:

```
texi2any -c SHOW_BUILTIN_CSS_RULES=1
```

22.4 @documentdescription: Texto do Sumário

Ao produzir saída HTML para um documento, um elemento ‘<meta>’ é escrito no ‘<head>’ para dar alguma ideia do conteúdo do documento. Por padrão, essa *descrição* será o título do

documento, tomado a partir do comando `@settitle` (veja-se Seção 2.5.3 [`@settitle`], Página 12). Para mudar isso, use o ambiente `@documentdescription`, como em:

```
@documentdescription
texto descritivo.
@end documentdescription
```

Isso produzirá a seguinte saída no ‘<head>’ do HTML:

```
<meta name=description content="texto descritivo.">
```

22.5 Gerando EPUB

EPUB é um formato projetado para leitura de livros eletrônicos em dispositivos portáteis. `texi2any` gerava EPUB 3.3 em 2024. Um arquivo EPUB é um contêiner de arquivamento ZIP, contendo arquivos informativos, bem como o manual renderizado em HTML.

A geração do arquivo EPUB depende do módulo Perl `Archive::Zip` estar instalado. Essa dependência é verificada em tempo de execução. Por padrão, tentar gerar EPUB sem essa dependência levanta um erro. Porém, se o arquivo de saída EPUB não for gerado, com a variável de personalização `EPUB_CREATE_CONTAINER_FILE` configurada como 0, não será um erro se `Archive::Zip` não estiver instalado.

Os testes do `texi2any` relacionados à geração de EPUB não exigem a instalação do `Archive::Zip`, pois eles configuram `EPUB_CREATE_CONTAINER_FILE` como 0 e mantêm o diretório contendo os arquivos e diretórios necessários para o arquivo EPUB configurando a variável de personalização `EPUB_KEEP_CONTAINER_FOLDER` como 1.

22.5.1 Diretório do Contêiner e Arquivos da Saída Gerada

Um diretório contendo os arquivos e diretórios necessários para o formato EPUB é criado ao se gerar EPUB. O nome desse diretório de contêiner é derivado do nome base do arquivo de entrada (ou seja, qualquer extensão será removida), com `_epub_package` posposto. Se um diretório de saída for especificado, com `--output` ou com a variável de personalização `SUBDIR`, o diretório de contêiner será criado nesse diretório em vez do diretório atual. No início de uma nova execução, o diretório de contêiner, e todo o conteúdo dele, é removido. O diretório de contêiner também é removido depois do arquivo EPUB final ter sido gerado no caso padrão.

Os arquivos HTML produzidos a partir do manual do Texinfo serão gerados em subdiretórios do diretório de contêiner. Arquivos de imagem referenciados a partir do manual do Texinfo, se encontrados, serão copiados para subdiretórios do diretório de contêiner.

O arquivo de saída EPUB é um arquivamento ZIP do diretório de contêiner. O nome do arquivo será derivado do nome base, com a extensão `.epub` posposta. Se um arquivo de saída for especificado, com `--output` ou com a função de personalização `OUTFILE`, esse nome de arquivo será usado. O arquivo EPUB de saída nunca será colocado no diretório especificado por `--output` ou por `SUBDIR`; somente o diretório de contêiner será colocado lá, conforme recém explicado acima.

O arquivo de saída EPUB não será gerado se a variável de personalização `EPUB_CREATE_CONTAINER_FILE` estiver configurada como 0. O diretório de contêiner será deixado depois do arquivo EPUB final ter sido gerado se `EPUB_KEEP_CONTAINER_FOLDER` estiver configurada.

Vea-se Seção 20.1 [Invocando `texi2any`], Página 157.

22.5.2 Referências Cruzadas do EPUB

O formato EPUB não suporta referências a partir de um arquivo EPUB para outro arquivo EPUB. Portanto quaisquer referências para manuais “externos” do Texinfo deveriam ser resolvidas para uma URL externa. `texi2any` produz esses linques com configuração de referência cruzada do HTML (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201). Como os linques no

EPUB resultante estarão incorretos se nenhuma informação for encontrada para as referências cruzadas, `texi2any` emitirá um aviso por padrão para ausência de informações de referências cruzadas. Se esses avisos forem indesejados, configure `CHECK_HTMLXREF` como 0.

22.5.3 Obtendo estrita conformação do EPUB

Por padrão, `texi2any` produz EPUB esperado pelos verificadores de conformidade, tanto quanto possível. Você ainda pode precisar ajustar teu manual Texinfo para obter conformidade total para:

- Certificar-se de que referências para manuais externos resultem em URLs (veja-se Seção 22.5.2 [Referências Cruzadas do EPUB], Página 195).
- Evitar código Texinfo mais indulgente que o XHTML. Em geral Texinfo válido é convertido para XHTML válido, mas existem exceções. Por exemplo, você deveria evitar referências aninhadas e comandos e referências de seccionamento aninhados, conteúdo antes do primeiro `@item` do `@table` e omitir a segunda coluna do `@item` do `@table`.
- Remover espaços nos nomes de arquivo de entrada do Texinfo.
- Remover avanços de formulário que terminem na saída gerada.

22.5.4 HTML Gerado para EPUB

O HTML gerado para EPUB é compatível com XHTML, codificado em UTF-8 e formatado sem os habituais cabeçalhos e rodapés de navegação do HTML.

`texi2any` realiza isso internamente configurando variáveis de personalização, tais como `USE_XML_SYNTAX`, `HEADERS` ou `OUTPUT_FILE_NAME_ENCODING`. Alguns recursos da saída gerada impressa são usados no EPUB. Em particular, o nó Topo não aparece na saída gerada do EPUB, enquanto uma página de título é gerada. Isso é obtido configurando-se `NO_TOP_NODE_OUTPUT`. Veja-se Seção 22.8 [Personalização da Saída Gerada HTML], Página 204.

Os valores das variáveis de personalização `OUTFILE` e `SUBDIR` correspondem inicialmente ao contêiner do diretório EPUB e(ou) ao arquivo da saída EPUB (veja-se Seção 22.5.1 [Arquivo e Diretório da Saída Gerada EPUB], Página 195). Esses valores das variáveis de personalização são desdefinidos ou reconfigurados para os locais no diretório de contêiner onde os arquivos XHTML forem gerados para a geração de HTML.

Esse uso das variáveis de personalização pelo `texi2any` está mencionado aqui porque reconfigurar-se tais variáveis é incomum; no entanto, as variáveis reconfiguradas são usadas internamente para a conversão, e deveria não interagir com nenhuma personalização configurada pelo(a) usuário(a).

22.6 Realçamento de Sintaxe de Exemplos de Códigos no HTML

Você pode usar a variável `HIGHLIGHT_SYNTAX` para realce de sintaxe do código-fonte em blocos `@example` para a saída HTML, com a ajuda de software externo. Você pode configurar `HIGHLIGHT_SYNTAX` para um dos vários valores especiais:

`realce` Usar `highlight` proveniente de <http://www.andre-simon.de/doku/highlight/en/highlight.php>.

`pygments` Usar `pygmentize` proveniente de <https://pygments.org/>.

`source-highlight`

Usar `source-highlight` (veja-se *GNU Source-highlight*).

O primeiro argumento em uma linha `@example` (veja-se Seção 7.4 [`@example`], Página 67) fornece o nome da linguagem usada para realce de sintaxe. Blocos `@example` sem tal argumento

só serão realçados se `HIGHLIGHT_SYNTAX_DEFAULT_LANGUAGE` estiver configurada, variável essa que fornece a linguagem a usar para realçamento de tais blocos.

Se você usar um valor para `HIGHLIGHT_SYNTAX` que não esteja listado acima, ele será um modelo para uma linha de comando que é executada para realçar código. Isso lhe dá mais controle sobre as opções exatas de linha de comando usadas para o programa de realce e permite usar outros programas além dos listados acima. Nesse modelo, o nome da linguagem realçada é substituído por ocorrências de ‘%1’ e ‘%’ é substituído por ‘%%’. `texi2any` passa o conteúdo de cada bloco `@example` (convertido para texto simples) na entrada padrão, e o HTML realçado é lido a partir da saída padrão.

Por exemplo, se você configurar `HIGHLIGHT_SYNTAX` como ‘`colorcode -l=%1`’ e existir um bloco ‘`@example C++`’ no documento, a linha de comando `colorcode -l=C++` será executada com o bloco ‘`@example`’ passado para a entrada padrão do comando executado e o resultado será lido a partir da saída padrão do comando executado.

22.7 Referências Cruzadas do HTML

Referências cruzadas entre manuais do Texinfo em formato HTML tornam-se linques `<a>` padrão do HTML. Esta seção descreve o algoritmo usado, de modo que o Texinfo consiga cooperar com outros programas, como `texi2html`, escrevendo arquivos HTML mutuamente compatíveis.

Este algoritmo pode ou não ser usado para linques *dentro* de saída HTML para um arquivo do Texinfo. Como não surgem problemas de compatibilidade em tais casos, nós não precisamos especificar isso.

Nós tentamos suportar referências para tais manuais “externos” em ambas as formas, monolítica e dividida. Um manual *monolítico* (mono) está inteiramente contido em um arquivo, e um manual *dividido* tem um arquivo para cada nó. (Veja-se Seção 22.2 [Dividindo HTML], Página 193).

O algoritmo foi desenvolvido principalmente por Patrice Dumas em 2003–04.

22.7.1 Básicos do Linque de Referência Cruzada do HTML

Para nossos propósitos, um linque de HTML consiste de quatro componentes: um nome de dispositivo, uma parte de diretório, uma parte de arquivo e uma parte de alvo. Nós sempre assumimos o protocolo `http`. Por exemplo:

```
http://dispositivo/diretório/arquivo.html#alvo
```

As informações para construir um linque vêm do nome do nó e do nome do manual no comando de referência cruzada no fonte do Texinfo (veja-se Capítulo 5 [Referências Cruzadas], Página 43) e de *informações externas* (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201).

No caso usual de manuais HTML publicados na World Wide Web, para a finalidade de se ter referências cruzadas funcionais, informações externas são exigidas para definir os valores de *dispositivo* e de *diretório*. Isso será detalhado posteriormente (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201).

Se nenhuma informação externa estiver disponível para um manual, então padrões serão usados para *dispositivo* e para *diretório* que correspondem a linques de referência cruzada entre uma coleção de manuais localizados lado a lado em um dispositivo. Nesse caso, *dispositivo* está programado para ser o dispositivo local. Esse poderia ser ou a sequência literal de caracteres ‘localhost’ ou ser totalmente omitido junto com ‘http://’, de acordo com as regras para linques HTML.

As partes *diretório* e *arquivo* dependem da natureza relativa de divisão/mono do manual sendo processado e do manual ao qual a referência cruzada referencia:

- Se o manual atual for dividido e o manual referente também for dividido, o diretório será ‘`../referente_html/`’ e o arquivo será o nome expandido do nó (descrito posteriormente).
- Se o manual atual for dividido e o manual referente for mono, o diretório será ‘`../`’ e o arquivo será `referente.html`.
- Se o manual atual for mono e o manual referente for dividido, o diretório será `referente_html/` e o arquivo será o nome expandido do nó.
- Se o manual atual for mono, e o manual referente também for mono, o diretório será `./` (ou apenas a sequência vazia de caracteres) e o arquivo será `referente.html`.

A ideia subjacente é a de que existe um diretório para manuais do Texinfo em HTML, e um determinado *manual* ou está disponível como um arquivo monolítico `manual.html` ou como um subdiretório dividido `manual_html/*.html`.

Outra regra, que vale somente para nomes de arquivos, é que nomes de base de arquivos sejam truncados para 245 caracteres, para permitir que uma extensão seja posposta e ainda cumpra o limite de 255 caracteres, o qual é comum para muitos sistemas de arquivos.

Qualquer parte do diretório no argumento do nome do arquivo do comando de referência cruzada de origem é ignorada. Assim, `@xref{,,,./foo}` e `@xref{,,,foo}` usam ambos ‘foo’ como o nome do manual. Isso ocorre porque tal ligação rígida tentada do diretório seja muito improvável ser útil para todos os formatos de saída que usem o nome do manual.

Finalmente, a parte do *alvo* é sempre o nome expandido do nó.

Quer o manual atual seja dividido ou quer seja mono é determinado por como você executa o `texi2any`; o padrão é o de dividir, com a opção `--no-split` substituindo isso.

Por padrão, `texi2any` gera linques para um manual referente usando a mesma forma do manual atual: se o manual atual for gerado como dividido, então linques para outros manuais serão para as formas divididas deles por padrão; da mesma forma para saída mono.

Entretanto, a forma do manual referente (mono ou dividido) normalmente deveria ser especificada em um arquivo de configuração (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201). Isso garante que não exista desconformidade entre o formato do manual referente que o software gerador assume e o formato em que ele realmente esteja presente.

22.7.2 Expansão de Nome de Nó de Referência Cruzada do HTML

Conforme mencionado na seção anterior, a parte chave do algoritmo de referência cruzada do HTML é a conversão de nomes de nós no fonte do Texinfo em sequências de caracteres adequadas para identificadores do XHTML e nomes de arquivos. As restrições são semelhantes para cada um: letras ASCII simples, números e os caracteres ‘-’ e ‘_’ são tudo o que pode ser usado. (Embora âncoras do HTML possam conter a maioria dos caracteres, XHTML é mais restritivo).

Referências cruzadas no Texinfo podem referir-se ou a nós, ou a âncoras (veja-se Seção 5.11 [`@anchor`], Página 49) ou rótulos flutuantes (veja-se Seção 9.1.1 [`@float`], Página 81). Entretanto, âncoras e rótulos flutuantes são tratados identicamente aos nós nesse contexto, de modo que nós continuaremos a dizer nomes de “nós” para simplicidade.

Uma exceção especial: o nó Top (veja-se Seção 2.10 [O Nó Top], Página 22) é sempre mapeado para o arquivo `index.html`, para corresponder ao software do servidor web. No entanto, o *alvo* do HTML é ‘Top’. Assim (no caso dividido):

```
@xref{Top,,, emacs, O Manual do GNU Emacs}.
⇒ <a href="../emacs_html/index.html#Top">
```

1. As letras ASCII padrão (a-z e A-Z) não são modificadas. Todos os outros caracteres podem ser mudados conforme especificado abaixo.
2. Os números ASCII padrão (0-9) não são modificados, exceto quando um número for o primeiro caractere do nome do nó. Nesse caso, veja-se abaixo.

3. Vários caracteres consecutivos de espaço, tabulação e nova linha serão transformados em apenas um espaço.
4. Espaços iniciais e finais serão removidos.
5. Depois que o acima tiver sido aplicado, cada caractere de espaço restante será convertido em um caractere ‘-’.
6. Outros caracteres ASCII de 7 bits serão transformados em ‘_00xx’, onde xx é o código do caractere ASCII em hexadecimal (minúsculas). Isso inclui ‘_’, o qual é mapeado para ‘_005f’.
7. Se o nome do nó não começar com uma letra, a sequência de caracteres literal ‘g_t’ será prefixada ao resultado. (Devido às regras acima, essa sequência de caracteres nunca pode ocorrer de outra forma; é uma escolha arbitrária, significando “GNU Texinfo”). Isso é necessário porque o XHTML exige que identificadores comecem com uma letra.

Por exemplo:

```
@node Um  nó --- com _'%
⇒ Um-no-_002d_002d_002d-com-_005f_0027_0025
```

Traduções exemplo de caracteres comuns:

- ‘_’ ⇒ ‘_005f’
- ‘-’ ⇒ ‘_002d’
- ‘Um nó’ ⇒ ‘Um-nó’

Em sistemas computacionais normalizantes de maiúsculas e minúsculas, nós que diferem somente por maiúsculas e minúsculas serão mapeados para o mesmo arquivo. Em particular, como mencionado acima, Top sempre mapeia para o arquivo `index.html`. Assim, em um sistema normalizante de maiúsculas e minúsculas, Top e um nó chamado ‘Index’ serão ambos escritos para `index.html`. Felizmente, os alvos servem para distinguir essas maiúsculas e minúsculas, uma vez que nomes de alvos do HTML são sempre sensíveis a maiúsculas e minúsculas, independentemente do sistema operacional.

22.7.3 Expansão do Comando de Referência Cruzada do HTML

Os nomes dos nós podem conter comandos @ (veja-se Seção 3.3 [Exigências da Linha do Nó], Página 28). Esta seção descreve como eles são manuseados.

Primeiro, comentários são removidos.

Em seguida, quaisquer comandos @value (veja-se Seção 16.5.1 [@set @value], Página 133) e invocações de macro (veja-se Seção 17.2 [Invocando Macros], Página 139) são totalmente expandidos.

Então, para os comandos a seguir, o nome do comando e as chaves são removidos e o texto do argumento é transformado recursivamente:

```
@asis @b @cite @code @command @dfn @dmn @dotless
@emph @env @file @i @indicateurl @kbd @key
@samp @sansserif @sc @slanted @strong @sub @sup
@t @U @var @verb @w
```

Além disso, os comandos a seguir são substituídos por texto constante, conforme mostrado abaixo. Se algum desses comandos tiver argumentos não vazios, como em `@TeX{bad}`, é um erro e o resultado será inespecificado. Nesta tabela, ‘(espaço)’ significa um caractere de espaço e ‘(nada)’ significa a sequência vazia de caracteres. A notação ‘U+hhhh’ significa ponto de código Unicode hhhh (em hexadecimal, como sempre).

Existem outras transformações de muitas dessas expansões para produzir o arquivo final ou outro nome de alvo, tais como caracteres de espaço para ‘-’, entre outras, de acordo com as outras regras.

@(newline)	(espaço)
@(space)	(espaço)
@(tab)	(espaço)
@!	‘!’
@*	(espaço)
@-	(nada)
@.	‘.’
@:	(nada)
@?	‘?’
@@	‘@’
@{	‘{’
@}	‘}’
@LaTeX	‘LaTeX’
@TeX	‘TeX’
@arrow	U+2192
@bullet	U+2022
@comma	‘,’
@copyright	U+00A9
@dots	U+2026
@enddots	‘...’
@equiv	U+2261
@error	‘erro-->’
@euro	U+20AC
@exclamdown	U+00A1
@expansion	U+21A6
@geq	U+2265
@leq	U+2264
@minus	U+2212
@ordf	U+00AA
@ordm	U+00BA
@point	U+22C6
@pounds	U+00A3
@print	U+22A3
@questiondown	U+00BF
@registeredsymbol	U+00AE
@result	U+21D2
@textdegree	U+00B0
@tie	(espaço)

Comandos @ de aspas (@quotedblright{} e similares) também são substituídos pelos valores Unicode deles. *Caracteres* de citação normal (por exemplo, ‘ e ’ do ASCII) não são alterados. Veja-se Seção 11.5 [Inserindo Aspas], Página 101.

Quaisquer comandos @acronym, @abbr, @email e @image são substituídos pelo primeiro argumento deles. (Para esses comandos, todos os argumentos subsequentes são opcionais e ignorados aqui). Veja-se Seção 6.1.14 [@acronym], Página 60, Seção 6.1.16 [@email], Página 62, e Seção 9.2 [Imagens], Página 83.

Acentos são manuseados de acordo com a próxima seção.

Qualquer outro comando é um erro e o resultado é inespecificado.

22.7.4 Expansão do Caractere de 8 Bits da Referência Cruzada do HTML

Normalmente, caracteres diferentes do ASCII simples de 7 bits são transformados no(s) correspondente(s) ponto(s) de código do Unicode na Forma de Normalização C, a qual usa caracteres pré-compostos quando disponíveis. (Essa é a forma de normalização recomendada pelo W3C e outros órgãos). Isso vale quando esse ponto de código for 0xffff ou menos, como quase sempre é.

Eles serão então transformados pelas regras acima na sequência de caracteres ‘_hhhh’, onde hhhh é o ponto de código em hexadecimal.

Por exemplo, combinando essa regra e a seção anterior:

```
@node @b{A} @TeX{} @u{B} @point{}@enddots{}
⇒ A-TeX-B_0306-_22C6_002e_002e_002e
```

Aviso: 1) @enddots se expande para três pontos os quais, por sua vez, se expandem para três ‘_002e’s; 2) @u{B} é um ‘B’ com acento breve, o qual não existe como um caractere pré-acentuado do Unicode, portanto se expande para ‘B_0306’ (B com combinação de breve).

Quando o ponto de código do Unicode estiver acima de 0xffff, a transformação será ‘_xxxxxx’, ou seja, dois sublinhados iniciais seguidos por seis dígitos hexadecimais. Como o Unicode declarou que o ponto de código mais alto deles é 0x10ffff, isso é suficiente. (Nós achamos que era melhor definir esse escape extra que usar sempre seis dígitos hexadecimais, já que os dois primeiros seriam quase sempre zeros).

Esse método funciona bem se o nome do nó consistir principalmente de caracteres ASCII e contiver somente alguns de 8 bits. Mas se o documento for escrito em um idioma cuja escrita não seja baseada no alfabeto latino (por exemplo, ucraniano), ele criará nomes de arquivos que consistem quase inteiramente de notações ‘_xxxx’, o que é inconveniente e quase ilegível. Para lidar com esses casos, **texi2any** oferece a opção de linha de comando **--transliterate-file-names**. Essa opção habilita *transliteração* de nomes de nós em caracteres ASCII para fins de criação de nomes de arquivos. A transliteração é baseada em princípios fonéticos, o que torna os nomes dos arquivos gerados mais facilmente compreensíveis.

Para a definição da Forma de Normalização C do Unicode, veja-se o informe UAX#15 do Unicode, <http://www.unicode.org/reports/tr15/>. Muitos documentos e implementações relacionados estão disponíveis em outros lugares na web.

22.7.5 Configuração de Referência Cruzada do HTML

Para outros manuais publicados na World Wide Web estarem aptos para terem referências cruzadas corretas para teu manual, as partes *dispositivo* e *diretório* da referência cruzada precisam ser conhecidas pelos conversores de Texinfo quando eles produzirem HTML para esses outros manuais. Similarmente, para teu manual em formato HTML ter referências cruzadas corretas para outros manuais, **texi2any** precisa saber como construir as URLs para outros manuais.

texi2any lê um arquivo chamado **htmlxref.cnf** e todos os arquivos que terminam com a extensão **.cnf** no diretório **htmlxref.d** para coletar informações para referências cruzadas para outros manuais na saída HTML. **htmlxref.cnf** e **htmlxref.d** são procurados nos seguintes diretórios, com base na Especificação de Diretório Base XDG (<https://specifications.freedesktop.org/basedir-spec/latest/>) com padrões baseados em diretórios de instalação:

./ (o diretório atual)

diretório principal do caminho do arquivo de entrada

Se existir um diretório principal e não for o diretório atual. Por exemplo, se o arquivo de entrada for **algum/diretorio/meummanual.texi**, **htmlxref.cnf** será procurado em **algum/diretorio** e os arquivos correspondentes a **algum/diretorio/htmlxref.d/*.cnf** também serão lidos.

```

./texinfo/
    (sob o diretório atual)

XDG_CONFIG_HOME/texinfo
    (com XDG_CONFIG_HOME uma variável de ambiente)

~/.config/texinfo/
    (onde ~ é o atual diretório HOME do(a) usuário(a), somente se XDG_CONFIG_HOME
    não estiver configurada)

sysconfdir/xdg/texinfo/
    (onde sysconfdir é o diretório de configuração do sistema especificado ao tempo da
    compilação, por exemplo, /usr/local/etc)

XDG_CONFIG_DIRS/texinfo
    (para cada diretório na variável de ambiente XDG_CONFIG_DIRS delimitada por :)

datadir/texinfo/
    (especificado ao tempo de compilação, por exemplo, /usr/local/share)

```

Todos os arquivos encontrados são usados, com as entradas anteriores substituindo as posteriores. A distribuição do Texinfo inclui arquivos que são instalados em `datadir/texinfo/htmlxref.d`, em um dos últimos dos diretórios acima:

`Texinfo_GNU.cnf`
Manuseia muitos manuais GNU.

`Texinfo_nonGNU.cnf`
Manuais livres de projetos não GNU.

Você pode usar um `htmlxref.cnf` local para completar ou para corrigir os arquivos padrão provenientes da distribuição do Texinfo. Isso poderia ser útil para corrigir dados desatualizados ou, até mesmo, para gerar referências cruzadas para arquivos locais para alguns manuais. Outros pacotes de software que gerenciam listas de manuais seguindo a especificação de referências cruzadas do HTML poderiam instalar arquivos `.cnf` em `datadir/texinfo/htmlxref.d`.

A variável de personalização `HTMLXREF_MODE` pode ser configurada para modificar como os arquivos são encontrados. Por exemplo, se configurada como `'none'`, nenhuma informação externa será usada. `HTMLXREF_FILE` configura o nome do arquivo para algo diferente de `htmlxref.cnf`. Veja-se [Variáveis de Personalização do HTML], Página 214.

O arquivo é orientado por linha. Linhas que consistem somente de espaços em branco são ignoradas. Comentários são indicados com um `#` no início de uma linha, opcionalmente precedido por espaço em branco. Como `#` pode ocorrer em URLs (como quase qualquer caractere), ele não inicia, de outro modo, um comentário.

Cada linha não vazia que não seja de comentário precisa ser ou uma *atribuição de variável* ou uma *informação de manual*.

Uma linha de atribuição de variável se parece com isto:

```
nomevariável = valorvariável
```

Espaço em branco ao redor de `=` é opcional e ignorado. A *nomevariável* deveria consistir de letras; diferenciação de maiúsculas e minúsculas é significativa. A *valorvariável* é uma sequência arbitrária de caracteres, continuando até o final da linha. As variáveis são então referenciadas com `'${nomevariável}'`; referências de variáveis podem ocorrer na *valorvariável*.

Uma linha de informação de manual se parece com isto:

```
manual palavrachave prefixourl
```

com *manual* o identificador curto para um manual, *palavrachave* sendo um dos: **mono**, **node**, **section**, **chapter** e *prefixurl* descritos abaixo. Referências de variáveis podem ocorrer somente na *prefixurl*. Por exemplo (usado no arquivo canônico):

```
G = https://www.gnu.org
GS = ${G}/software
hello mono    ${GS}/hello/manual/hello.html
hello chapter ${GS}/hello/manual/html_chapter/
hello section ${GS}/hello/manual/html_section/
hello node    ${GS}/hello/manual/html_node/
```

Se a palavra-chave for **mono**, *prefixurl* fornecerá o dispositivo, o diretório e o nome do arquivo para *manual* como um arquivo monolítico.

Se a palavra-chave for **node**, **section** ou **chapter**, *prefixurl* fornecerá o dispositivo e o diretório para *manual* divididos em nós, seções ou capítulos, respectivamente.

Se *prefixurl* for omitido, os elementos das referências cruzadas corresponderão a linques de referência cruzada entre uma coleção de manuais localizados lado a lado em um dispositivo (veja-se Seção 22.7.1 [Básicos do Linque Xref do HTML], Página 197). Isso poderia ser útil se você gerasse e publicasse uma coleção de manuais e preferisse linhar para os manuais gerados em vez de para as versões encontradas nas configurações de referências cruzadas enviadas com o Texinfo.

Quando **texi2any** estiver gerando um linque para outro manual do Texinfo, ele preferirá usar uma linha para o manual com a mesma opção de divisão que a da saída atual. Ou seja, ao gerar saída monolítica (**--no-split**), uma URL **mono** será preferida; ao gerar saída que seja dividida por nó, uma URL **node** será preferida, entre outras.

Se nenhuma linha de informação com a opção de divisão preferida estiver presente para um determinado manual, então outras linhas para esse manual poderão ser usadas, se presentes. Aqui está a ordem de pesquisa completa para cada opção de divisão:

node	⇒	nó,	seção,	capítulo,	mono
section	⇒	seção,	capítulo,	nó,	mono
chapter	⇒	capítulo,	seção,	nó,	mono
mono	⇒	mono,	capítulo,	seção,	nó

Essas referências entre manuais em nível de seção e de capítulo podem ser bem-sucedidas somente quando o manual alvo foi criado usando-se **--node-files**; esse é o padrão para saída dividida.

Atualmente, os arquivos **Texinfo_GNU.cnf** e **Texinfo_nonGNU.cnf** distribuídos com o GNU Texinfo servem como o principal recurso para localizar manuais HTML do Texinfo na World Wide Web. Como eles são instalados em um local usado pelo **texi2any**, as informações acerca dos manuais HTML encontradas nesses arquivos serão usadas para referências cruzadas por padrão.

Se você tiver acréscimos ou correções para os arquivos distribuídos com o Texinfo, por favor envie uma mensagem eletrônica para **bug-texinfo@gnu.org** como de costume. Se você publicar um manual HTML do Texinfo na World Wide Web, ter um local atualizado listado em **Texinfo_GNU.cnf** ou em **Texinfo_nonGNU.cnf** deveria garantir que todos os manuais HTML gerados pelo **texi2any** usem esse local para referências cruzadas automaticamente. Você pode obter as versões mais recentes a partir de https://ftpmirror.gnu.org/texinfo/htmlxref.d/Texinfo_GNU.cnf e https://ftpmirror.gnu.org/texinfo/htmlxref.d/Texinfo_nonGNU.cnf. Se você quiser manter um arquivo separado distribuído com Texinfo, por favor envie uma mensagem eletrônica para **bug-texinfo@gnu.org** também.

22.8 Personalização da Saída Gerada HTML

Você pode usar muitas variáveis de personalização definíveis por usuário(a) para influenciar a saída HTML. Veja-se Seção 22.8.9 [Lista de Variáveis de Personalização do HTML], Página 214, para a lista completa. As possibilidades oferecidas pelas variáveis de personalização, que vão desde a estrutura geral do documento e variante da linguagem HTML até a formatação de constructos específicos, deveriam cobrir a maioria das necessidades ainda não satisfeitas pela personalização de CSS (veja-se Seção 22.3 [CSS do HTML], Página 193).

Se você precisar de mais controle, poderá conseguir isso por meio do uso de arquivos de inicialização, que são descritos em um manual separado (veja-se *Personalização da Saída do texi2any do GNU Texinfo*).

Se a variável de personalização `TEXI2HTML` estiver configurada, o HTML gerado será o mais compatível possível com `texi2html` (veja-se Seção 20.7 [`texi2html`], Página 181). O efeito dessa variável é bastante incomum, pois ela não modifica diretamente a saída gerada, mas configura muitas outras variáveis de personalização, incluindo algumas que só podem ser configuradas em arquivos de inicialização e estão descritas no manual separado em Personalização de Saída Gerada do `texi2any`. Desconfigurada no caso padrão.

22.8.1 Personalização da Estrutura da Saída Gerada HTML

Você consegue usar variáveis de personalização para controlar a saída gerada de títulos e de sumários de documentos ou de outras partes semelhantes de documentos. Outras variáveis afetam como `texi2any` visualiza a estrutura de um documento e como a saída gerada é dividida em arquivos da saída gerada.

Se `SHOW_TITLE` for `'undef'`, o padrão, nenhum título será gerado no início da saída gerada. Se você configurar essa variável, por padrão o conteúdo de `@titlepage` (ou a saída gerada a partir de `@maketitle`) será usado para o título; desconfigure `USE_TITLEPAGE_FOR_TITLE` para obter uma sequência simples de caracteres de título.

Por padrão, a saída gerada HTML é adaptada para visualização on-line, com o nó Topo sendo gerado como um ponto de partida de nível superior para o leitor. Você pode configurar `NO_TOP_NODE_OUTPUT` para remover o nó Topo, o que resulta em saída gerada que pode ser mais semelhante ao formato de um livro impresso. Por padrão, configurar-se `NO_TOP_NODE_OUTPUT` também gera um título no início do documento, embora você consiga substituir isso configurando `SHOW_TITLE` como 0.

Em cada nó, por padrão, uma lista de seções subordinadas é gerada para navegação. Isso corresponde à variável `FORMAT_MENU` sendo configurada como `'sectiontoc'`. Para usar menus para navegar no documento (como no formato Info; veja-se Seção 3.9 [Menus], Página 33), configure `FORMAT_MENU` como `'menu'`. Configure `FORMAT_MENU` como `'menu_no_detailmenu'` se você quiser menus, mas nenhum `@detailmenu` gerado automaticamente para o menu mestre Topo. Se você não quiser que ou menus ou listas de seções subordinadas para navegação sejam exibidas, configure `FORMAT_MENU` como `'nomenu'`.

Por padrão, o sumário para o documento inteiro é gerado no final da seção `@top`, para ter o local principal para navegação no documento inteiro desde o início. Isso corresponde à variável `CONTENTS_OUTPUT_LOCATION` configurada como `'after_top'`.

Você consegue ter o sumário gerado depois do título (se `SHOW_TITLE` estiver configurada) configurando `CONTENTS_OUTPUT_LOCATION` como `'after_title'`. Para gerar o sumário no final do documento ou em um arquivo separado, se a saída gerada for dividida, configure `CONTENTS_OUTPUT_LOCATION` como `'separate_element'`. Você também pode configurar `CONTENTS_OUTPUT_LOCATION` como `'inline'`: nesse caso, o sumário será gerado onde o comando `@` correspondente, por exemplo `@contents`, for usado.

Você pode achar útil configurar `CONTENTS_OUTPUT_LOCATION` quando `FORMAT_MENU` estiver configurada como `'menu'` para a finalidade de usar menus para navegação. Nesse caso, já deveria

existir um menu mestre na seção `@top` (veja-se Seção 2.10.1 [Partes do Menu Mestre], Página 23), e assim o leitor do teu manual possivelmente não precise de um sumário no mesmo local.

Por padrão, cada nó é gerado em um arquivo separado. Você consegue mudar isso com a opção de linha de comando `--split`, que é equivalente a configurar a variável `SPLIT`. `SPLIT` pode assumir os valores `'chapter'`, `'section'` ou `'node'` ou ser configurada como uma sequência vazia de caracteres para indicar saída não dividida. Veja-se [Dividindo Saída Gerada], Página 163.

Unidades elementares chamadas *unidades de saída* são usadas para saída, com cada arquivo de saída contendo o conteúdo de uma ou mais unidades de saída, dependendo do valor de `SPLIT`. As unidades de saída contêm o conteúdo associado a um nó ou comando de seccionamento. Em fontes típicas de documentos Texinfo, um comando de nó é quase sempre associado com um comando de seccionamento imediatamente seguinte.

A variável de personalização `USE_NODES` determina se o nó ou o comando de seccionamento é considerado o comando principal. Por padrão, nós são usados como o componente principal para a separação das unidades da saída gerada. Você consegue configurar `USE_NODES` como falso para usar preferencialmente seccionamento para decidir onde as unidades da saída gerada serão separadas. Quando seções forem os componentes principais das unidades da saída gerada, nós “isolados” não diretamente associados com um comando de seccionamento serão associados com o comando de seccionamento seguinte, enquanto comandos de seccionamento sem nós constituem unidades da saída gerada. Por outro lado, quando nós forem os principais componentes das unidades da saída gerada, seções isoladas não associadas com nós serão associadas com o nó anterior e nós isolados serão unidades da saída gerada.

Por padrão, a configuração de `USE_NODES` também afeta se os linques Próximo, Acima e Anterior nos cabeçalhos dos nós lincam-se a nós ou a seções, com os nós usados se verdadeira e as seções usadas se falsa. Você consegue substituir isso usando a variável `USE_NODE_DIRECTIONS` (o valor padrão é indefinido). Observe que essa configuração não determina a sequência de caracteres do linque, somente para onde os linques apontam; veja-se Seção 5.5 [xrefautomaticsectiontitle], Página 45, para a personalização da sequência de caracteres do linque. Se os nós e as seções estiverem sistematicamente associados, essa personalização não terá efeito prático.

Você pode adicionar um elemento especial *Sobre*, explicando como navegar, configurando `DO_ABOUT`; configurada como 0 no caso padrão (sem elemento Sobre). Se `PROGRAM_NAME_IN_ABOUT` também estiver configurada, produz o nome do programa e diversas informações relacionadas no elemento especial Sobre; padrão falso.

No caso de saída gerada não dividida, por padrão, somente um arquivo é gerado, o qual contém quaisquer elementos Sobre (com `DO_ABOUT`) e elementos de sumário (com `CONTENTS_OUTPUT_LOCATION` configurada como `'separate_element'`). Se você preferir que esses elementos especiais sejam separados, configure `MONOLITHIC` como 0 para gerar elementos especiais em arquivos separados.

22.8.2 Personalização de Nomes de Arquivo e de Linques para HTML

Existem variáveis de personalização para controlar os nomes dos arquivos da saída gerada e para personalizar a saída de hiperlinques dentro dos arquivos da saída gerada.

Você consegue especificar os nomes dos arquivos de saída com mais controle que meramente a opção de linha de comando `--output` (veja-se Seção 20.1 [Invocando `texi2any`], Página 157). A variável de personalização `PREFIX` substitui o nome base do arquivo fornecido por `@setfilename`, ou o nome do arquivo, e deveria não conter quaisquer componentes de diretório. Para alterar diretórios intermediários, use a variável de personalização `SUBDIR`. Finalmente, você também pode substituir a extensão com a variável de personalização `EXTENSION`. Essa variável deveria ser `undef` se nenhuma extensão for para ser adicionada. Se você configurar `CASE_INSENSITIVE_FILENAMES`, os nomes dos arquivos serão construídos como se o sistema de arquivos fosse insensível

a maiúsculas e minúsculas. Nesse caso, um nome de arquivo será usado para todos os arquivos, diferindo somente por maiúsculas e por minúsculas.

Além disso, a variável de personalização `TOP_FILE` substitui o nome do arquivo da saída gerada para o elemento superior. Isso, em geral, só é levado em conta quando a saída gerada for dividida.

Se `USE_ACCESSKEY` estiver configurada, o atributo `accesskey` será usado para fornecer atalhos de teclado em alguns hiperlinques de navegação. `n` é usado para linques para o próximo nó ou seção, `p` para linques para o nó ou seção anterior e `u` para linques de direção ascendente. No caso padrão, o valor dessa variável de personalização é `undef` e configurado automaticamente baseado na divisão do documento de saída, para evitar configurar `accesskey` para vários valores na mesma página HTML (veja-se Seção 22.2 [Dividindo HTML], Página 193).

Existem variáveis de personalização que são específicas para tipos particulares de linques:

Linques de comandos de referência cruzada

Se o segundo argumento de um comando de referência cruzada estiver configurado, ele será exibido como texto de hiperlinque (veja-se Seção 5.4 [Dois Argumentos], Página 45). Caso contrário, a variável de personalização `XREF_USE_NODE_NAME_ARG` será usada para determinar o texto do hiperlinque. Se configurado como 1, usar o (primeiro) argumento do nome do nó em comandos `@` de referência cruzada para o texto exibido como o hiperlinque. Se configurado como 0, usar o nome do nó se `USE_NODES` estiver configurada; caso contrário, o nome da seção. Se configurado como `'undef'`, usar o primeiro argumento em ambientes pré-formatados, caso contrário, usar o nome do nó ou o nome da seção dependendo de `USE_NODES`. O padrão é `'undef'`.

Linques para flutuações

Se você configurar `XREF_USE_FLOAT_LABEL` (padrão é desligada), o rótulo flutuante será usado em vez do tipo seguido pelo número flutuante (veja-se Seção 9.1.1 [`@float`], Página 81).

Linques para nós ou para seções para entradas de índice

Na formatação de `@printindex`, dois linques são gerados: um linque para a entrada do índice e um linque para o comando `@` raiz que contém a entrada do índice. A variável de customização `NODE_NAME_IN_INDEX` especifica se o nó ou seção deveria ser usado para o linque para o comando `@` raiz. Se verdadeiro, usar nomes de nós nas entradas de índice; caso contrário, usar nomes de seção. Se indefinido, usar o valor de `USE_NODES` para determinar qual comando `@` raiz preferir. O padrão é indefinido.

Linques de menu

Os nomes dos nós são usados em linques de entradas de menu por padrão. Configurar `NODE_NAME_IN_MENU` como falso para usar nomes de seção.

Linques de sumário

Se um Sumário principal e um Sumaríssimo estiverem presentes, as referências cruzadas no Sumaríssimo se lincarão às correspondentes entradas do Sumário no caso padrão. Configurar `SHORT_TOC_LINK_TO_TOC` como 0 para lincar aos comandos de seccionamento.

Se você configurar `TOC_LINKS`, linques de títulos para entradas do sumário serão criados; padrão falso.

Linque de direção Acima do nó Topo

Por padrão, nenhuma referência Acima é gerada para o nó Topo. Você pode configurar `IGNORE_REF_TO_TOP_NODE_UP` para que quaisquer referências para o nó Acima de Topo (por padrão, `'(dir)'`) que aparecem em outras referências cruzadas também

sejam ignoradas. `IGNORE_REF_TO_TOP_NODE_UP` não está configurada no caso padrão. (Veja-se [TOP_NODE_UP], Página 177).

Se o manual estiver referenciado em uma página web juntamente com outros manuais, possivelmente seja relevante linkar para essa página. Configure `TOP_NODE_UP_URL` como o URL usado para o nó Topo que Acima referenciar. Se `TOP_NODE_UP_URL` for configurada, o valor de `TOP_NODE_UP` será usado para o texto do hiperlinque, com ‘(dir)’ como padrão. `TOP_NODE_UP` pode ser usada no atributo, portanto deveria não conter nenhum elemento.

Por exemplo, no projeto GNU <https://www.gnu.org/manual/> coleta links para a maioria dos manuais GNU; portanto, `TOP_NODE_UP_URL` é melhor especificada como `/manual/` se o manual for instalado em `www.gnu.org`.

Veja-se Seção 3.4 [Primeiro Nó], Página 29, para mais acerca dos ponteiros do nó Topo.

linques de imagens

Se `IMAGE_LINK_PREFIX` estiver configurada, o valor associado será preposto aos links do arquivo de imagem; padrão desconfigurada. Isso poderia ser útil se arquivos de imagens estiverem em um subdiretório específico.

As referências cruzadas entre manuais HTML são especificadas precisamente (veja-se Seção 22.7 [Xref do HTML], Página 197). A personalização de locais de manuais é fornecida por meio dos arquivos `htmlxref.cnf` e `htmlxref.d/*.cnf` (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201).

Diretórios e nomes de arquivo padrão são usados para referenciar cruzadamente manuais alvo não encontrados por meio da Configuração do Xref do HTML (veja-se Seção 22.7.1 [Básicos do Linque Xref do HTML], Página 197). Por padrão, um aviso é fornecido para cada manual externo que não esteja nos arquivos de configuração do Xref do HTML. Se não quiser avisos acerca de tais manuais, você deveria configurar `CHECK_HTMLXREF` como ‘0’. Isso poderia ser relevante, por exemplo, se você souber que os manuais estão instalados localmente.

Você consegue usar variáveis de personalização para especificar a Configuração do Xref do HTML mais precisamente. Por padrão, o nome do arquivo usado para configuração do Xref do HTML e o diretório `htmlxref.d` são pesquisados em diretórios, arquivos em `htmlxref.d` com a extensão `.cnf` e todos os arquivos usados para configuração do Xref do HTML encontrados serão usados. Você pode configurar a variável de personalização `HTMLXREF_MODE` para modificar como as referências cruzadas para outras informações de manuais são determinadas. Se `HTMLXREF_MODE` estiver configurada como ‘file’, o nome do arquivo será usado diretamente como a fonte da informação. Se `HTMLXREF_MODE` estiver configurada como ‘none’, nenhuma informação será usada. O caso padrão é obtido com `HTMLXREF_MODE` não definida ou configurada com qualquer outro valor. A variável de personalização `HTMLXREF_FILE` configura o arquivo usado para configuração do Xref do HTML com um valor diferente do padrão, `htmlxref.cnf`. Se `HTMLXREF_FILE` contiver diretórios, eles serão carregados se encontrados, mas não serão pesquisados nos diretórios. Por padrão, o manual distante é considerado estar dividido ou monolítico baseado na divisão do manual sendo produzido. Você pode configurá-lo explicitamente, configurando `EXTERNAL_CROSSREF_SPLIT`.

Existem também variáveis de personalização que modificam vários outros aspectos das referências cruzadas:

- Você consegue configurar o nome do arquivo usado para o nó Topo em referências cruzadas por configurando o valor da variável de personalização `TOP_NODE_FILE_TARGET`; padrão é `index.html`.
- Você especifica um diretório base para externos manuais com `EXTERNAL_DIR`; no caso padrão, não existe nenhum.

- Similarmente, você configura a extensão do arquivo para referenciar cruzadamente a outros manuais com `EXTERNAL_CROSSREF_EXTENSION`.

Entretanto, em geral, o uso dessas tornará referências entre manuais “inválidas” no sentido de que a URL gerada não linkará para o manual externo, a menos que esse manual fosse ele próprio produzido usando a personalização correspondente. Portanto, em geral, é melhor usar apenas a Configuração do Xref do HTML.

Você consegue mudar o comprimento máximo de um nome de arquivo base para referências cruzadas internas configurando `BASEFILENAME_LENGTH`; por padrão, os nomes dos arquivos base são truncados para 245 caracteres.

22.8.3 Personalização da Navegação e de Cabeçalhos

Cabeçalhos e rodapés com um painel de navegação são gerados no caso padrão. Se você usar `--no-headers`, ou se a variável de personalização `HEADERS` estiver desconfigurada, o painel de navegação só será inserido no início dos arquivos divididos.

Por padrão, os elementos `<link>` são gerados no `<head>` do HTML para especificar relacionamentos entre a página HTML e outros recursos, por exemplo, o nó Topo, o próximo nó ou o sumário. Desconfigure `USE_LINKS` para evitar esses elementos.

O `<title>` e a descrição do documento em `<head>` são baseados em `@settitle` (veja-se Seção 2.5.3 [`@settitle`], Página 12). Se o manual for dividido, o nome do nó também será adicionado a esse título HTML. Configure `SECTION_NAME_IN_TITLE` para usar o argumento do comando de estruturamento do capítulo associado em vez do nome do nó.

Por padrão, se uma codificação de entrada for configurada (tipicamente por meio de `@documentencoding`), essas informações serão usadas para configurar o nome da codificação de saída, caso contrário, a codificação da saída gerada será baseada na codificação padrão. Uma etiqueta `<meta>` é gerada, na seção `<head>` do HTML, para especificar a codificação da saída gerada. Veja-se Seção 15.2 [`@documentencoding`], Página 127. Para configurar explicitamente a codificação da saída gerada, configure `OUTPUT_ENCODING_NAME`. A codificação especificada deveria ser um nome de conjunto de caracteres normalizado usável em HTML, tipicamente um dos nomes de codificação preferidos da IANA.

Quando a saída é dividida em nós (veja-se Seção 22.2 [Dividindo HTML], Página 193), o valor da variável de personalização `WORDS_IN_PAGE` especifica o comprimento mínimo aproximado da página no qual um painel de navegação é colocado na parte inferior de uma página. Não é possível calibrar esse valor muito precisamente para resultados exatos: isso ocorre porque a contagem de palavras usadas para a verificação é baseada na saída HTML para o nó até o momento (incluindo qualquer barra de navegação na parte superior da página), o que produz um valor um pouco maior que o número de palavras que seriam visíveis para o(a) usuário(a) no navegador da web deles(as).

A aparência do painel de navegação é afetada pelas seguintes variáveis de personalização, todas falsas no caso padrão:

`DATE_IN_HEADER`

Colocar a data de geração do documento no cabeçalho.

`HEADER_IN_TABLE`

Usar tabelas para formatação de cabeçalho em vez de um simples `<div>`.

`ICONS`

Usar ícones para o painel de navegação.

`PROGRAM_NAME_IN_FOOTER`

Se configurada, gerar o nome do programa e diversas informações relacionadas nos rodapés das páginas.

VERTICAL_HEAD_NAVIGATION

Se configurada, um painel vertical de navegação é usado.

Configurar-se **ICONS** é necessário, mas não suficiente para obter ícones para botões de direção, uma vez que nenhuma imagem de botão é especificada no caso padrão. Além disso, você precisa configurar as variáveis de personalização **ACTIVE_ICONS** e **PASSIVE_ICONS**. Você só consegue fazer isso usando um arquivo de inicialização (veja-se Seção 20.1 [Invocando **texi2any**], Página 157). Veja-se *Personalização da Saída do texi2any do GNU Texinfo*.

22.8.4 Personalização de Recursos do HTML

Por padrão, o HTML é gerado, mas você consegue configurar **USE_XML_SYNTAX** para a finalidade de gerar código compatível com XML. A principal diferença é que `</>` em vez de `<>` encerra etiquetas como `<img>` ou `<link>` que não têm uma etiqueta de fechamento (essas etiquetas criam os assim chamados *elementos void*).

Para configurar o **DOCTYPE**, configure a variável de personalização **DOCTYPE**. O padrão é o **DOCTYPE** simples sem DTD usado para HTML5.

Você consegue configurar um arquivo compatível com XML começando:

```
texi2any --html meu_manual.texi -c USE_XML_SYNTAX \
  -c DOCTYPE='<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html>'
```

Você também poderia configurar o **DOCTYPE** como HTML 4.01 Transitório:

```
texi2any --html meu_manual.texi -c DOCTYPE=\
'<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">'
```

Por padrão, entidades são usadas para caracteres duplicados de aspas simples (veja-se Seção 11.5 [Inserindo Aspas], Página 101) e `'---` e `--'` (veja-se Seção 2.1 [Convenções], Página 8). Configure **USE_ISO** como `'0'` no caso improvável de você querer uma saída mais simples. Por padrão, entidades textuais são usadas quando possível. Configure **USE_NUMERIC_ENTITY** para usar somente entidades numéricas. Configure **OUTPUT_CHARACTERS** para gerar caracteres acentuados baseados na codificação da saída gerada em vez de nas entidades.

Por padrão, um atributo personalizado, conforme permitido pelo padrão, é usado para fornecer o nome do manual alvo em referências cruzadas. Se não for desejável, por exemplo, gerar XHTML estrito, você pode configurar **NO_CUSTOM_HTML_ATTRIBUTE** para evitar atributos personalizados sendo gerados.

Linques copiáveis são gerados para alvos de linque para os comandos de definição (veja-se Capítulo 14 [Comandos de Definição], Página 114), comandos de tabela (veja-se Seção 8.4 [Tabelas de Duas Colunas], Página 77) onde uma entrada de índice estiver definida e cabeçalhos. Um linque aparece como um sinal ‘¶’ que aparece quando você passa o ponteiro do mouse sobre o texto do título. Desconfigure **COPIABLE_LINKS** para evitar que linques copiáveis sejam gerados.

22.8.5 Personalização de Código HTML Inserido

Existem variáveis para configurar o código HTML inserido em vários pontos da saída gerada.

Aspa de abertura e aspas de fechamento necessárias, por exemplo, para saída **@samp**, são configuradas como:

```
&lquo; e &rquo;
```

Por padrão, com **OPEN_QUOTE_SYMBOL** e **CLOSE_QUOTE_SYMBOL** desconfiguradas.

```
&#8216; e &#8217;
```

Se você configurar **USE_NUMERIC_ENTITY**.

Um caractere de aspa de abertura e de fechamento

Se você configurar `OUTPUT_CHARACTERS` e a codificação da saída gerada incluir esse caractere.

`OPEN_QUOTE_SYMBOL` e `CLOSE_QUOTE_SYMBOL`

Se você configurar essas variáveis de personalização.

Para mudar o código HTML para quebras (regra horizontal) inseridas em vários contextos, configure `DEFAULT_RULE`, que é usado para a saída da maioria dos separadores horizontais no HTML resultante, e configure `BIG_RULE` para o separador mais largo usado algumas vezes.

Você consegue configurar `HTML_ROOT_ELEMENT_ATTRIBUTES` para adicionar atributos ao elemento `<html>`. Você pode definir a variável `EXTRA_HEAD` para adicionar texto dentro do elemento `<head>` do HTML. Similarmente, o valor de `AFTER_BODY_OPEN` é adicionado logo depois que `<body>` for gerado. Essas variáveis ficam vazias por padrão.

Você consegue configurar os atributos do elemento `<body>` definindo a variável de personalização `BODY_ELEMENT_ATTRIBUTES`. Por padrão, o atributo `lang` é configurado como o idioma do documento (veja-se Seção 15.1 [`@documentlanguage`], Página 126).

Você pode definir a variável `PRE_BODY_CLOSE` para adicionar texto logo antes do elemento `</body>` do HTML. Nada é adicionado por padrão.

Similarmente, as seguintes variáveis permitem algum controle útil da formatação do sumário e do sumaríssimo:

`BEFORE_TOC_LINES`

Inserido antes do texto do sumário.

`AFTER_TOC_LINES`

Inserido depois do texto do sumário.

`BEFORE_SHORT_TOC_LINES`

Inserido antes do texto do sumaríssimo.

`AFTER_SHORT_TOC_LINES`

Inserido depois do texto do sumaríssimo.

Ao tempo desta escrita, os valores padrão dessas variáveis eram configurados como etiquetas de abertura e de fechamento para um elemento `<div>` que encerra os sumários.

22.8.6 Personalização da Saída Gerada Específica do HTML

A saída gerada de constructos específicos é personalizável.

No HTML, níveis de cabeçalho traduzem para elementos `<hn>` onde *n* é o nível do cabeçalho. Os seguintes níveis de cabeçalho são personalizáveis:

`CHAPTER_HEADER_LEVEL`

Nível de formatação de cabeçalho usado para comandos de seccionamento em nível de capítulo; padrão `'2'`.

`FOOTNOTE_END_HEADER_LEVEL`

`FOOTNOTE_SEPARATE_HEADER_LEVEL`

Nível de formatação de cabeçalho usado para o cabeçalho das notas de rodapé com o estilo de nota de rodapé `'end'` ou para o estilo de nota de rodapé `'separate'`; padrão `'4'`. Veja-se Seção 9.3.2 [Estilos de Notas de Rodapé], Página 86.

`MAX_HEADER_LEVEL`

Nível máximo de formatação de cabeçalho usado (números mais altos de níveis de formatação de cabeçalho correspondem a níveis mais baixos de seccionamento); padrão `'4'`.

Você pode modificar a aparência de vários constructos configurando:

DEF_TABLE

Se configurada, uma construção `<table>` para `@defn` e comandos `@` semelhantes será usada (parecendo mais com a saída gerada do $\text{\TeX}$), em vez de listas de definição; padrão falso.

INDENTED_BLOCK_COMMANDS_IN_TABLE

Se configurada, usar tabelas para recuo de comandos de blocos recuados; padrão falso.

INDEX_ENTRY_COLON

Símbolo usado entre a entrada de índice e o nó, ou seção, associado; padrão é uma sequência vazia de caracteres.

MENU_ENTRY_COLON

Símbolo usado entre a entrada do menu e a descrição; padrão ‘:’.

MENU_SYMBOL

Símbolo usado em frente das entradas de menu quando nomes de nós forem usados para formatação de entradas de menu; o padrão é indefinido e configurada como `&bull`; se `USE_NUMERIC_ENTITY` não estiver configurada, e como `’`; se configurada.

NUMBER_FOOTNOTES

NO_NUMBER_FOOTNOTE_SYMBOL

Por padrão, notas de rodapé são numeradas. Com `--no-number-footnotes` ou se `NUMBER_FOOTNOTES` estiver configurada como 0, um ‘*’ será usado ou o valor da variável de personalização `NO_NUMBER_FOOTNOTE_SYMBOL`, se configurada.

22.8.7 Personalização do HTML para Matemática

Você pode selecionar vários métodos para renderizar matemática em HTML (veja-se Capítulo 12 [Inserindo Fórmulas Matemáticas], Página 109). Isso é controlado pela variável de personalização `HTML_MATH`. Por padrão, a variável de personalização está desconfigurada e a saída HTML é apenas enfatizada. Você pode selecionar outras opções para a finalidade de exibir fórmulas matemáticas formatadas corretamente. Para algumas dessas opções, se você configurar `--iftex`, as seções `@tex` serão convertidas para HTML e se você configurar `--iflatex`, as seções `@latex` serão convertidas para HTML.

12h Usar o programa `latex2html` para produzir arquivos de imagem para material matemático.

`latex2html` processará matemática $\text{\LaTeX}$ em comandos matemáticos, incluindo matemática $\text{\TeX}$ compatível com $\text{\LaTeX}$. `latex2html` é usado para traduzir seções `@tex` e `@latex` para HTML se as seções correspondentes não forem ignoradas. Observe que `latex2html` só consegue processar $\text{\LaTeX}$, inclusive ao processar seções `@tex`.

O processamento com `latex2html` deveria deixar os arquivos no diretório da saída gerada, com ‘-12h’ no nome deles, em particular um arquivo de cache para evitar refazer a tradução para HTML se já feita.

Vea-se Seção 22.8.7.2 [Variáveis de Personalização do `latex2html`], Página 213.

mathjax Insere referências nos arquivos de saída para conjuntos de comandos sequenciados MathJax para formatar fórmulas matemáticas. A opção MathJax exige que o JavaScript esteja habilitado no navegador para funcionar.

MathJax lida com matemática $\text{\LaTeX}$, incluindo matemática $\text{\TeX}$ comumente usada e compatível com $\text{\LaTeX}$. Alguns constructos específicos (por exemplo, alguns usos de `\text`) não são suportados, mas isso não deveria ser um problema prático.

Se os comandos matemáticos forem realmente processados, um rótulo da web de licença JavaScript será gerado para conjuntos de comandos sequenciados MathJax (veja-se [Rótulos web de licença JavaScript], Página 214).

Veja-se Seção 22.8.7.1 [Variáveis de Personalização da MathJax], Página 212.

t4h Usar o programa `tex4ht` para produzir HTML para material matemático. `tex4ht` processará matemática $\text{\LaTeX}$ em comandos matemáticos, incluindo matemática $\text{\TeX}$ compatível com $\text{\LaTeX}$. `tex4ht` é usado para traduzir seções `@tex` e `@latex` para HTML se as seções não forem ignoradas. As seções `@tex` são processadas no modo $\text{\TeX}$, enquanto as seções `@latex` são processadas como $\text{\LaTeX}$. O processamento com `tex4ht` deveria deixar os arquivos no diretório da saída gerada, com `'_tex4ht'` no nomes deles.

Veja-se Seção 22.8.7.3 [Variáveis de Personalização do `tex4ht`], Página 213.

Por padrão, com `CONVERT_TO_LATEX_IN_MATH` indefinida, configurar-se `HTML_MATH` também configura `CONVERT_TO_LATEX_IN_MATH`. Nesse caso, comandos `@` do Texinfo dentro de `@math` e de `@displaymath` serão convertidos para $\text{\LaTeX}$, antes de converter o `@math` ou o `@displaymath` para HTML.

22.8.7.1 Variáveis de Personalização da MathJax

Esta tabela lista as variáveis de personalização que podem ser usadas quando MathJax estiver sendo usado, o que será o caso quando `HTML_MATH` estiver configurada como `'mathjax'`.

MATHJAX_CONFIGURATION

Dados a adicionar-se ao objeto de configuração global `'MathJax'`. Especifique como uma lista separada por vírgulas de pares de *componente*: `{ nome: valor, ... }`.

Por exemplo, você poderia configurar `MATHJAX_CONFIGURATION` para

```
opções: { enableMenu: falso },
carregador: {
  carregar: [\'[tex]/physics\'],
  versionWarnings: falso
},
tex: {
  pacotes: {\'[+]\': [\\'physics\']}
```

para desabilitar o menu do clique direito da MathJax e para carregar a extensão `physics` da MathJax. (Veja-se Configurando MathJax (<https://docs.mathjax.org/en/latest/options/index.html>) na documentação da MathJax).

Se você substituir qualquer um dos padrões para essa configuração gerada pelo `texi2any`, você deveria certificar-se de que tuas novas configurações alinhem-se com a saída HTML produzida pelo `texi2any`.

MATHJAX_SCRIPT

URL do arquivo do componente MathJax (por exemplo `tex-svg.js`) que você estiver usando. `texi2any` fornece um valor padrão para essa variável, mas você é incentivado(a) a hospedar você mesmo(a) esse arquivo em teu sítio da web, de forma que não dependa da hospedagem de terceiros(as).

MATHJAX_SOURCE

Uma URL do código-fonte completo na forma dela preferida para modificação, ou instruções para obter tal código-fonte, para o arquivo do componente nomeado por `MATHJAX_SCRIPT`. `'Forma preferida para modificação'` significa que essa não deveria

estar em uma forma ‘minificada’. Usada na página de rótulos de licença (veja-se [Rótulos web de licença JavaScript], Página 214).

Novamente, `texi2any` fornece um valor padrão para essa variável, mas você é incentivado(a) a hospedar você mesmo(a) o código-fonte para MathJax e as dependências dele. Isso é para a finalidade de tornar o código-fonte disponível confiavelmente e para reduzir tua dependência e a de teus usuários(as) dos sistemas de distribuição de terceiros(as).

22.8.7.2 Variáveis de Personalização do `latex2html`

Esta tabela lista as variáveis de personalização que podem ser usadas quando `latex2html` estiver sendo usado para converter seções `@math`, `@displaymath`, `@latex` e `@tex` para HTML. Estas variáveis de personalização serão relevantes somente se `HTML_MATH` estiver configurada como ‘`l2h`’.

Para realmente converter seções `@tex`, `--iftex` deveria ser usada; e para realmente converter seções `@latex`, `--iflatex` deveria ser usada.

L2H_CLEAN

Se configurada, os arquivos intermediários gerados em relação ao `latex2html` serão removidos; padrão verdadeiro.

L2H_FILE Se configurada, o arquivo fornecido será usado como arquivo de iniciação do `latex2html`; padrão desconfigurada.

L2H_HTML_VERSION

A versão do HTML usada na chamada `latex2html`; padrão desconfigurada.

L2H_L2H O programa invocado como `latex2html`; padrão é `latex2html`.

L2H_SKIP Se configurada com um valor verdadeiro, a chamada real para `latex2html` será ignorada; o conteúdo gerado anteriormente será reusado. Se configurada como 0, o cache não será usado em absoluto. Se configurada como ‘`undef`’, o cache será usado para tantos fragmentos `TEX` quanto possível e para qualquer restante o comando será executado. O padrão é ‘`undef`’.

L2H_TMP Configurar o diretório usado para arquivos temporários. Nenhum dos componentes do nome de arquivo nesse nome de diretório pode começar com ‘`.`’; caso contrário, `latex2html` falhará (por causa do `dvips`). O padrão é a sequência vazia de caracteres, a qual significa o diretório atual.

22.8.7.3 Variáveis de Personalização do `tex4ht`

Esta tabela lista as variáveis de personalização que podem ser usadas quando `tex4ht` estiver sendo usado para converter seções `@math`, `@displaymath`, `@tex` e `@latex` para HTML. Estas variáveis de personalização serão relevantes somente se `HTML_MATH` estiver configurada como ‘`t4h`’.

Para realmente converter seções `@tex`, `--iftex` deveria ser usada; e para realmente converter seções `@latex`, `--iflatex` deveria ser usada.

T4H_LATEX_CONVERSION

Se configurada, o tipo de conversão usado para seções `@latex`. As possibilidades são ‘`latex`’, ‘`tex`’ e ‘`texi`’. Configurada como ‘`latex`’ se não definida.

T4H_MATH_CONVERSION

Se configurada, o tipo de conversão usado para `@math` e `@displaymath`. As possibilidades são ‘`latex`’, ‘`tex`’ e ‘`texi`’. Configurada como ‘`tex`’ se não definida.

T4H_TEX_CONVERSION

Se configurada, o tipo de conversão usado para seções `@tex`. As possibilidades são ‘`latex`’, ‘`tex`’ e ‘`texi`’. Configurada como ‘`tex`’ se não definida.

22.8.8 Interface e Licenças JavaScript

Você pode adicionar uma interface de navegação JavaScript experimental à saída HTML configurando o valor de `INFO_JS_DIR` como o nome de um diretório no qual colocar o código para essa interface. Por exemplo, ‘`texi2any --html -c INFO_JS_DIR=js manual.texi`’ coloca arquivos em um diretório ‘`js`’ abaixo da saída gerada. Essa interface fornece algumas das funcionalidades dos navegadores Info em um navegador da Web, como navegação por teclado e pesquisa de índice. Isso funciona apenas com saída HTML não dividida.

Por padrão, uma *página web de rótulos de licença JavaScript* é configurada para fornecer informações de licenciamento e de código-fonte para qualquer JavaScript usado pela saída gerada HTML (veja-se <https://www.gnu.org/licenses/javascript-labels.html>). Para evitar gerar uma página web de rótulos de licença JavaScript, configure `JS_WEBLABELS` como ‘`omit`’. Com `JS_WEBLABELS` configurada como ‘`generate`’, o padrão, gerar uma página de rótulos em `JS_WEBLABELS_FILE`, `js_licenses.html` no caso padrão, e lincar a ela nos arquivos da saída gerada HTML. Com `JS_WEBLABELS` configurada como ‘`reference`’, lincar ao arquivo de rótulos fornecido por `JS_WEBLABELS_FILE` na saída gerada e não gerar um arquivo de rótulos. Essa última configuração é útil se você mantiver separadamente um arquivo de rótulos para um sítio web maior que inclua teu manual.

Arquivos de rótulos podem ser gerados se `INFO_JS_DIR` estiver configurada ou se `HTML_MATH` estiver configurada como ‘`mathjax`’ (veja-se [Conjuntos de comandos sequenciados MathJax], Página 211).

22.8.9 Lista de Variáveis de Personalização do HTML

Esta tabela lista as variáveis de personalização que se aplicam para saída gerada HTML.

AFTER_BODY_OPEN

Texto adicionado no início de cada arquivo HTML; padrão desconfigurada.

AFTER_SHORT_TOC_LINES**AFTER_TOC_LINES**

Texto gerado depois do sumário para `AFTER_SHORT_TOC_LINES` e depois do sumário para `AFTER_TOC_LINES` se configurada; caso contrário, uma sequência padrão de caracteres será usada. Ao tempo desta escrita, um elemento `</div>` estava fechado.

Em geral, você deveria configurar `BEFORE_SHORT_TOC_LINES` se `AFTER_SHORT_TOC_LINES` estiver configurada, e você deveria configurar `BEFORE_TOC_LINES` se `AFTER_TOC_LINES` estiver configurada.

BASEFILENAME_LENGTH

O comprimento máximo de um nome de arquivo base para referências cruzadas internas; padrão 245.

BEFORE_SHORT_TOC_LINES**BEFORE_TOC_LINES**

Se configurada, o texto gerado antes do sumário para `BEFORE_SHORT_TOC_LINES` e antes do sumário para `BEFORE_TOC_LINES`, caso contrário, uma sequência padrão de caracteres será usada. Ao tempo desta escrita, um elemento `<div ...>` está aberto.

Em geral, você deveria configurar `AFTER_SHORT_TOC_LINES` se `BEFORE_SHORT_TOC_LINES` estiver configurada, e você deveria configurar `AFTER_TOC_LINES` se `BEFORE_TOC_LINES` estiver configurada.

- BIG_RULE** Regra usada antes e depois do elemento superior e antes dos elementos especiais, mas não para rodapés e cabeçalhos; padrão
- BODY_ELEMENT_ATTRIBUTES**
O texto de atributos do `<body>`. Por padrão, configura o atributo `lang` do HTML para o idioma do documento (veja-se Seção 15.1 [`@documentlanguage`], Página 126).
- CASE_INSENSITIVE_FILENAMES**
Construa nomes de arquivos de saída como se o sistema de arquivos não diferenciase maiúsculas de minúsculas (veja-se Seção 22.2 [Dividindo HTML], Página 193); padrão falso.
- CHAPTER_HEADER_LEVEL**
Nível de formatação de cabeçalho usado para comandos de seccionamento em nível de capítulo; padrão ‘2’.
- CHECK_HTMLXREF**
Verificar se os manuais que são o alvo de referências cruzadas externas (veja-se Seção 5.6 [Quatro e Cinco Argumentos], Página 46) estão presentes em `htmlxref.cnf` ou em `htmlxref.d/*.cnf` (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201); padrão verdadeiro.
- CONTENTS_OUTPUT_LOCATION**
Se configurada como ‘`after_top`’, produzir o conteúdo no final da seção `@top`. Se configurada como ‘`inline`’, produzir o conteúdo onde o `@contents` e comandos `@` semelhantes estão localizados. Se configurada como ‘`separate_element`’ produzir o conteúdo em elementos separados, ou no final do documento, se não dividido, ou em um arquivo separado. Se configurada como ‘`after_title`’ os índices serão produzidos depois do título; padrão ‘`after_top`’.
- CONVERT_TO_LATEX_IN_MATH**
Se configurada, tentar converter quaisquer comandos `@` do Texinfo dentro de `@math` e de `@displaymath` para $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$, antes de converter o `@math` ou o `@displaymath` para HTML. Padrão `undef`. Se indefinida, configurar se `HTML_MATH` estiver configurada.
- COPIABLE_LINKS**
Se configurada, produzir links copiáveis para os comandos de definição (veja-se Capítulo 14 [Comandos de Definição], Página 114), comandos de tabela (veja-se Seção 8.4 [Tabelas de Duas Colunas], Página 77) onde uma entrada de índice estiver definida e cabeçalhos. Um link aparece como um sinal ‘¶’ que aparece quando você passa o ponteiro do mouse sobre o texto do cabeçalho.
- DATE_IN_HEADER**
Colocar a data de geração do documento no cabeçalho; desligado por padrão.
- DEF_TABLE**
Se configurada, uma construção `<table>` para `@defn` e comandos `@` semelhantes será usada (parecendo mais com a saída gerada do $\text{T}_{\text{E}}\text{X}$), em vez de listas de definição; padrão falso.
- DEFAULT_RULE**
Regra usada entre elementos, exceto antes e depois do elemento superior, e antes de elementos especiais, e para rodapés e cabeçalhos; padrão `<hr>`.
- DO_ABOUT** Se configurada como 0, nunca fazer um elemento especial Sobre; se configurada como 1, sempre fazer um elemento especial Sobre; padrão 0.
- EPUB_CREATE_CONTAINER_FILE**
Se configurada como 0, não gera o arquivo de saída EPUB. O padrão é configurada como 1.

EPUB_KEEP_CONTAINER_FOLDER

Se configurada, mantém o diretório contendo os diretórios e arquivos necessários para EPUB. O arquivo de saída EPUB é um arquivamento ZIP desse diretório. O padrão é não definida. Configurada se não definida e **TEST** ou **DEBUG** estiver configurada. Veja-se Seção 22.5.1 [Arquivo e Diretório da Saída Gerada EPUB], Página 195.

EXTERNAL_CROSSREF_SPLIT

Para referências cruzadas para outros manuais, isso determina se o outro manual é considerado dividido ou monolítico. Por padrão, ela é configurada baseada no valor de **SPLIT**. Veja-se Seção 22.7 [Xref do HTML], Página 197, e veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201.

EXTERNAL_DIR

Diretório base para manuais externos; padrão nenhum. É melhor usar o mecanismo geral de referência cruzada externa (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201) do que essa variável.

EXTERNAL_CROSSREF_EXTENSION

Extensão de arquivo para referências cruzadas para outros manuais.

EXTRA_HEAD

Texto adicional aparecendo em `<head>`; padrão desconfigurada.

FOOTNOTE_END_HEADER_LEVEL

Nível de formatação de cabeçalho usado para o cabeçalho das notas de rodapé com o estilo de nota de rodapé ‘end’; padrão ‘4’. Veja-se Seção 9.3.2 [Estilos de Notas de Rodapé], Página 86.

FOOTNOTE_SEPARATE_HEADER_LEVEL

Nível de formatação de cabeçalho usado para o cabeçalho das notas de rodapé com o estilo de nota de rodapé ‘separate’; padrão ‘4’. Veja-se Seção 9.3.2 [Estilos de Notas de Rodapé], Página 86.

HEADER_IN_TABLE

Usar tabelas para formatação de cabeçalho em vez de um simples elemento `<div>`; padrão falso.

HIGHLIGHT_SYNTAX

Se configurada, blocos `@example` com informações de idioma como primeiro argumento serão realçados na saída gerada HTML. Também é possível especificar um padrão para o idioma com **HIGHLIGHT_SYNTAX_DEFAULT_LANGUAGE**. O realce de sintaxe exige um programa externo para gerar o HTML realçado. O valor de **HIGHLIGHT_SYNTAX** permite selecionar um programa específico. As possibilidades são `source-highlight`, `highlight`, `pygments`, com qualquer outro valor sendo concatenado para o idioma para gerar a linha de comando para o programa externo (veja-se Seção 22.6 [Realçamento de Sintaxe], Página 196).

HIGHLIGHT_SYNTAX_DEFAULT_LANGUAGE

O idioma padrão usado para realce de sintaxe quando não existir informações de idioma.

HTML_MATH

Método a usar-se para renderizar `@math` (veja-se Seção 22.8.7 [Personalização do HTML para Matemática], Página 211). Isso pode ser desconfigurada, configurada como ‘`mathjax`’ (veja-se Seção 22.8.7.1 [Variáveis de Personalização da MathJax], Página 212), configurada como ‘`12h`’, que usa `latex2html` (veja-se Seção 22.8.7.2 [Variáveis de Personalização do `latex2html`], Página 213), ou configurada como

‘`t4h`’, que usa `tex4ht` (veja-se Seção 22.8.7.3 [Variáveis de Personalização do `tex4ht`], Página 213). Por padrão, configurar-se `HTML_MATH` também configura `CONVERT_TO_LATEX_IN_MATH`.

HTML_ROOT_ELEMENT_ATTRIBUTES

Usar essa string para o elemento raiz `<html>` do documento HTML. Padrão indefinido.

HTMLXREF_FILE

Configurar o nome do arquivo usado para referências cruzadas para outros manuais. Se não definida, `htmlxref.cnf` será usado (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201). Não definida no caso padrão. Se `TEST` estiver configurada, `HTMLXREF_MODE` estiver configurada para o padrão e `HTMLXREF_FILE` não estiver definida, informações acerca de referências cruzadas para outros manuais não serão usadas.

Se `HTMLXREF_MODE` estiver configurada como ‘`file`’, o nome do arquivo será usado diretamente como a fonte de informação; caso contrário, o nome do arquivo será pesquisado em diretórios e todos os arquivos encontrados serão usados (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201). Arquivos `htmlxref.d/* .cnf` também serão usados nesse caso.

HTMLXREF_MODE

Como informações de referências cruzadas para outros manuais são determinadas. Se configurada como ‘`none`’, nenhuma informação será usada. Se configurada como ‘`file`’, as informações serão determinadas a partir de um caminho de arquivo, `htmlxref.cnf` no caso padrão, ou o valor de `HTMLXREF_FILE`. Se não definida (o padrão) ou configurada para qualquer outro valor, pesquisar pelo nome do arquivo `HTMLXREF_FILE` e pelo diretório `htmlxref.d` nos diretórios e usar todos os arquivos `HTMLXREF_FILE` e arquivos no diretório `htmlxref.d` com a extensão `.cnf` (veja-se Seção 22.7.5 [Configuração do Xref do HTML], Página 201).

ICONS Usar ícones para o painel de navegação; padrão falso.

IMAGE_LINK_PREFIX

Se configurada, o valor associado será preposto aos links do arquivo de imagem; padrão desconfigurada.

INDENTED_BLOCK_COMMANDS_IN_TABLE

Se configurada, usar tabelas para recuo de comandos de blocos recuados; padrão falso.

INDEX_ENTRY_COLON

Símbolo usado entre a entrada de índice e o nó, ou seção, associado; padrão é uma sequência vazia de caracteres.

INFO_JS_DIR

(Experimental). Adicionar uma interface de navegação JavaScript ao manual. O valor da variável é o diretório para colocar o código para essa interface, de forma que você executaria o programa como por exemplo ‘`texi2any --html -c INFO_JS_DIR=js manual.texi`’ para colocar arquivos em um diretório ‘`js`’ sob a saída gerada. Isso fornece algumas das funcionalidades dos navegadores Info em um navegador da Web, como navegação por teclado e pesquisa de índice. Isso funciona somente com saída HTML não dividida.

A interface deveria fornecer uma funcionalidade alternativa aceitável se JavaScript ou recursos do navegador da Web não estiverem disponíveis. No entanto, por favor, seja cuidadoso(a) ao usar essa opção, caso você torne tua documentação mais difícil de acessar para alguns dos seus usuários(as).

IGNORE_REF_TO_TOP_NODE_UP

Ignorar referências para TOP_NODE_UP, o nó acima para o nó Top.

INLINE_CSS_STYLE

Colocar CSS diretamente nos elementos HTML, em vez de no início da saída gerada; padrão falso.

JS_WEBLABELS**JS_WEBLABELS_FILE**

Especificar como usar uma página *rótulos web de licença JavaScript* para fornecer informações de licenciamento e código-fonte para qualquer JavaScript usado nos arquivos HTML para o manual. (Veja-se <https://www.gnu.org/licenses/javascript-labels.html>).

Com o valor ‘generate’ (o padrão), gerar uma página de rótulos em JS_WEBLABELS_FILE e linkar a ela nos arquivos de saída HTML. Fazer isso somente se realmente referenciar arquivos JavaScript (ou com HTML_MATH configurada como ‘mathjax’ ou ao usar a interface experimental de navegação JS quando INFO_JS_DIR estiver configurada). Com essa configuração, JS_WEBLABELS_FILE precisa ser um nome relativo de arquivo.

Com o valor ‘reference’, linkar para o arquivo de rótulos fornecido por JS_WEBLABELS_FILE na saída gerada e não gerar um arquivo de rótulos. Essa configuração é útil se você mantiver separadamente um arquivo de rótulos para um sítio maior que inclua teu manual.

Com ‘omit’, nem gerar nem linkar para um arquivo de rótulos.

MAX_HEADER_LEVEL

Nível máximo de formatação de cabeçalho usado (números mais altos de níveis de formatação de cabeçalho correspondem a níveis mais baixos de seccionamento); padrão ‘4’.

MENU_ENTRY_COLON

Símbolo usado entre a entrada do menu e a descrição; padrão ‘:’.

MENU_SYMBOL

Símbolo usado em frente das entradas de menu quando nomes de nós forem usados para formatação de entradas de menu; o padrão é indefinido e configurada como • se USE_NUMERIC_ENTITY não estiver configurada, e como ’ se configurada.

MONOLITHIC

Produzir somente um arquivo incluindo o sumário. Configurada por padrão, mas relevante somente quando a saída gerada não for dividida.

NO_CSS Não usar CSS; padrão falso. Veja-se Seção 22.3 [CSS do HTML], Página 193.

NO_CUSTOM_HTML_ATTRIBUTE

Não produzir HTML com atributos personalizados em elementos; padrão falso.

NO_NUMBER_FOOTNOTE_SYMBOL

Símbolo usado para notas de rodapé se NUMBER_FOOTNOTES for falso. O padrão é *.

NODE_NAME_IN_INDEX

Se verdadeiro, usar nomes de nós nas entradas de índice; caso contrário, preferir nomes de seções. Se indefinida, usar valor de USE_NODES no HTML. O padrão é indefinida.

NODE_NAME_IN_MENU

Se configurada, usar nomes de nós nas entradas de menu; caso contrário, preferir nomes de seção; padrão verdadeiro.

PRE_BODY_CLOSE

Se configurada, o texto fornecido aparecerá no rodapé de cada arquivo HTML; padrão desconfigurada.

PROGRAM_NAME_IN_ABOUT

Usada quando um elemento Sobre for gerado. Se configurada, produzir o nome do programa e diversas informações relacionadas no elemento especial Sobre; padrão falso.

PROGRAM_NAME_IN_FOOTER

Se configurada, gerar o nome do programa e diversas informações relacionadas nos rodapés das páginas; padrão falso.

SECTION_NAME_IN_TITLE

Se configurada, quando a saída gerada for dividida, usar o argumento do comando de estruturação do capítulo (por exemplo, `@chapter` ou `@section`) no `<title>` em vez do argumento para `@node`.

SHORT_TOC_LINK_TO_TOC

Se configurada, as referências cruzadas no sumário curto se lincam às entradas correspondentes do Sumário, se um Sumário for gerado; padrão verdadeiro.

SHOW_BUILTIN_CSS_RULES

Produzir as regras CSS padrão integradas na saída padrão e sair.

SHOW_TITLE

Se configurada, gerar o título no início do documento; padrão `'undef'`. Se configurada como `'undef'`, configurar-se `NO_TOP_NODE_OUTPUT` também configura `SHOW_TITLE` para HTML.

TEXI2HTML

Gerar HTML e tentar ser o mais compatível possível com `texi2html`; padrão falso.

TOC_LINKS

Se configurada, linques de títulos para entradas de índice serão criados; padrão falso.

TOP_FILE Esse nome de arquivo pode ser usado para o arquivo de nível superior. A extensão é configurada apropriadamente, se necessário. Isso é usado para substituir o padrão e, em geral, só é levado em consideração quando a saída gerada for dividida e para HTML.

TOP_NODE_FILE_TARGET

Nome do arquivo usado para o nó Top em referências cruzadas; o padrão é `index.html`.

TOP_NODE_UP_URL

Uma URL usada para nó Topo que Acima referencia; o padrão é `undef`; nesse caso, nenhuma referência Acima de nó Topo é gerada. Para substituir o nome do ponteiro Acima no caso de `TOP_NODE_UP_URL` estiver configurada e para outros formatos, veja-se `TOP_NODE_UP` em Seção 20.4.8 [Variáveis de Personalização para Múltiplos Formatos], Página 176. Veja-se Seção 22.8.2 [Personalização de Nomes de Arquivo e de Linques para HTML], Página 205. Para mais acerca dos ponteiros do nó Topo, veja-se Seção 3.4 [Primeiro Nó], Página 29.

USE_ACCESSKEY

Usar `accesskey` em referências cruzadas; padrão `undef`. Se `undef`, configurar se a saída gerada for dividida.

USE_ISO Usar entidades para caracteres de aspas simples duplicadas (veja-se Seção 11.5 [Inserindo Aspas], Página 101) e ‘---’ e ‘--’ (veja-se Seção 2.1 [Convenções], Página 8); padrão verdadeiro.

USE_LINKS
Gerar elementos `<link>` na saída `<head>` do HTML; padrão verdadeiro.

USE_NODE_DIRECTIONS
Se verdadeiro, usar nós para determinar para onde linkar próximo, acima e anterior nos cabeçalhos dos nós. Se falso, usar seções. Se indefinida, usar valor de **USE_NODES**. O padrão é indefinida. Observe que essa configuração não determina a sequência de caracteres do linque, somente para onde os links apontam; veja-se Seção 5.5 [xrefautomaticsectiontitle], Página 45, para a personalização da sequência de caracteres do linque. Se nós e seções estiverem sistematicamente associados, essa personalização não terá efeito prático.

USE_TITLEPAGE_FOR_TITLE
Usar o `@titlepage` completo (ou saída proveniente de `@maketitle`) como o título, não uma simples sequência de caracteres de título; padrão verdadeiro. Relevante somente se **SHOW_TITLE** estiver configurada.

USE_XML_SYNTAX
Usar sintaxe compatível com XML/XHTML

VERTICAL_HEAD_NAVIGATION
Se configurada, um painel vertical de navegação é usado; padrão falso.

WORDS_IN_PAGE
Quando a saída é dividida por nós, especifica o comprimento mínimo aproximado da página em que um painel de navegação é colocado na parte inferior de uma página. Para evitar que os botões de navegação fiquem na parte inferior de uma página, configure isso para um número suficientemente grande. O padrão é 200.

XREF_USE_FLOAT_LABEL
Se configurada, para o nome flutuante em referências cruzadas, usar o rótulo flutuante em vez do tipo seguido pelo número flutuante (veja-se Seção 9.1.1 [`@float`], Página 81). O padrão é desligado.

XREF_USE_NODE_NAME_ARG
Relevante somente para comandos de referência cruzada sem nome de referência cruzada (segundo argumento). Se configurada como 1, usar o argumento (primeiro) do nome do nó em comandos `@` de referência cruzada para o texto exibido como o hiperlinque. Se configurada como 0, usar o nome do nó se **USE_NODES** estiver configurada; caso contrário, o nome da seção. Se configurada como ‘undef’, usar o primeiro argumento em ambientes pré-formatados; caso contrário, usar o nome do nó ou o nome da seção, dependendo de **USE_NODES**. O padrão é ‘undef’.

Anexo A Detalhes do Comando @

Aqui estão os detalhes dos comandos @: informações acerca da sintaxe deles, uma lista dos comandos e informações acerca de onde os comandos podem aparecer.

A.1 Sintaxe de Comando @

Texinfo tem os seguintes tipos de comando @:

1. Comandos de chave

Esses comandos começam com @ seguido por uma letra ou uma palavra, seguida por um argumento entre chaves. Por exemplo, o comando `@dfn` indica o uso introdutório ou definidor de um termo; ele é usado conforme segue: ‘No Texinfo, comandos @ são comandos de `@dfn{marcação}`’.

2 . Comandos de linha

Esses comandos ocupam uma linha inteira. A linha começa com @, seguido pelo nome do comando (uma palavra); por exemplo, `@center` ou `@cindex`. Se nenhum argumento for necessário, a palavra será seguida pelo fim de linha. Se existir um argumento, ele será separado do nome do comando por um espaço. Chaves não são usadas.

3 . Comandos de bloco

Esses comandos são escritos no início de uma linha, com texto geral nas linhas seguintes, finalizados por um comando `@end` correspondente em uma linha própria. Por exemplo, `@example`, depois as linhas de um exemplo de codificação e, em seguida, `@end example`. Alguns desses comandos de bloco recebem argumentos, como os comandos de linha recebem; por exemplo, `@enumerate A` abrindo um ambiente finalizado por `@end enumerate`. Aqui ‘A’ é o argumento.

4. Comandos de inserção de símbolos sem argumentos

Esses comandos começam com @ seguido por uma palavra seguida por chaves esquerda e direita. Esses comandos inserem símbolos especiais no documento; eles não recebem argumentos. Alguns exemplos: `@dots{}` $\Rightarrow$ ‘...’, `@equiv{}` $\Rightarrow$ ‘ $\equiv$ ’, `@TeX{}` $\Rightarrow$ ‘ $\TeX$ ’ e `@bullet{}` $\Rightarrow$ ‘•’.

5. Comandos não alfabéticos

Os nomes dos comandos em todas as categorias acima consistem de caracteres alfabéticos, quase inteiramente em letras minúsculas. Ao contrário desses, os comandos não alfabéticos consistem de um @ seguido por um sinal de pontuação ou outro caractere que não é parte do alfabeto latino. Comandos não alfabéticos quase sempre são parte do texto dentro de um parágrafo. Os comandos não alfabéticos incluem `@{`, `@}`, `@.`, `@SPACE` e a maioria dos comandos de acento.

6. Comandos variados

Existem vários comandos que não se enquadram em nenhuma das categorias acima; por exemplo, o comando obsoleto `@refill`, que é sempre usado no final de um parágrafo imediatamente seguinte ao ponto final ou a outro caractere de pontuação. `@refill` não recebe argumentos e não exige chaves. Da mesma forma, `@tab` usado em um bloco `@multitable` não recebe argumentos e não é seguido por chaves.

Assim, os comandos alfabéticos se enquadram em classes que tem diferentes sintaxes de argumento. Você não consegue dizer a qual classe um comando pertence pela aparência do nome dele, mas pode saber pelo significado do comando: se o comando representar um glifo, ele estará na classe 4 e não exigirá um argumento; se fizer sentido usar o comando entre outros textos como parte de um parágrafo, o comando estará na classe 1 e precisará ser seguido por um argumento

entre chaves. Os comandos não alfabéticos, como @:, são exceções à regra; eles não precisam de chaves.

O objetivo de se ter sintaxe diferente para comandos é o de tornar os arquivos do Texinfo mais fáceis para ler e também o de ajudar os comandos de parágrafo e de preenchimento do GNU Emacs a funcionarem adequadamente.

A.2 Lista de Comandos @

Aqui está uma lista alfabética dos comandos @ no Texinfo. Colchetes, [], indicam argumentos opcionais; uma reticência, ‘...’, indica texto repetido.

@espaçoembranco

Um @ seguido por um espaço, tabulação ou nova linha produz um espaço entre palavras normal e extensível. Veja-se Seção 11.3.1 [Espaços Múltiplos], Página 98.

@! Produzir um ponto de exclamação que termine uma frase (geralmente depois de uma letra maiúscula no final da frase). Veja-se Seção 11.3.3 [Finalizando uma Frase], Página 99.

@"

@' Gerar um trema ou um acento agudo, respectivamente, sobre o próximo caractere, como em ö e em ó. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@&

@ampchar{}

Gerar um E Comercial. Veja-se Seção 11.1.6 [Inserindo um E Comercial], Página 96.

@* Forçar uma quebra de linha. Veja-se Seção 13.2 [Quebras de Linha], Página 110.

@,{c} Gerar um acento de cedilha sob c, como em ç. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@- Inserir um ponto discricionário de hifenização. Veja-se Seção 13.3 [@- @hyphenation], Página 111.

@. Produzir um ponto final que termine uma frase (geralmente depois de uma letra maiúscula no final da frase). Veja-se Seção 11.3.3 [Finalizando uma Frase], Página 99.

@/ Não produz saída, porém permite uma quebra de linha. Veja-se Seção 13.2 [Quebras de Linha], Página 110.

@: Dizer aos processadores de saída impressa para evitarem inserir espaços em branco extras depois de um ponto final, ponto de interrogação, ponto de exclamação ou dois pontos imediatamente anteriores. Veja-se Seção 11.3.2 [Não Finalizando uma Frase], Página 98.

@= Gerar um acento macron (barra) sobre o próximo caractere, como em ō. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@? Produzir um ponto de interrogação que termine uma frase (geralmente depois de uma letra maiúscula no final da frase). Veja-se Seção 11.3.3 [Finalizando uma Frase], Página 99.

@@

@atchar{}

Inserir um sinal de arroba, ‘@’. Veja-se Seção 11.1.1 [Inserindo um Símbolo Arroba], Página 95.

@\

`@backslashchar{}`

Inserir uma barra invertida, ‘\’; `@backslashchar{}` funciona em qualquer lugar, enquanto `@\` funciona somente dentro de `@math`. Veja-se Seção 11.1.4 [Inserindo uma Barra Invertida], Página 96, e Capítulo 12 [Inserindo Fórmulas Matemáticas], Página 109.

@~

@`

Gerar um acento circunflexo (chapéu) ou grave, respectivamente, sobre o próximo caractere, como em ô e è. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@{

`@lbracechar{}`

Inserir uma chave esquerda ‘{’. Veja-se Seção 11.1.2 [Inserindo Chaves], Página 95.

@}

`@rbracechar{}`

Inserir uma chave direita ‘}’. Veja-se Seção 11.1.2 [Inserindo Chaves], Página 95.

@~

Gerar um acento til sobre o próximo caractere, como em Ñ. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

`@AA{}`

`@aa{}`

Gerar as letras maiúsculas e minúsculas do anel A escandinavo, respectivamente: Å, å. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

`@abbr{abreviação}`

Indicar uma abreviação geral, como em ‘Comput.’. Veja-se Seção 6.1.13 [`@abbr`], Página 60.

`@acronym{sigla}`

Indicar uma sigla em letras todas maiúsculas, como em ‘NASA’. Veja-se Seção 6.1.14 [`@acronym`], Página 60.

`@AE{}`

`@ae{}`

Gerar as ligaduras AE maiúsculas e minúsculas, respectivamente: Æ, æ. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

`@afivepaper`

Mudar dimensões de página para o tamanho de papel A5. Veja-se Seção E.7 [Papel A4], Página 280.

`@afourlatex`

`@afourpaper`

`@afourwide`

Mudar dimensões de página para o tamanho de papel A4. Veja-se Seção E.7 [Papel A4], Página 280.

`@alias novo=existente`

Tornar o comando ‘`@novo`’ um sinônimo para o comando existente ‘`@existente`’. Veja-se Seção 17.4 [`@alias`], Página 144.

`@allowcodebreaks verdadeiro-falso`

Controlar quebra em ‘-’ e ‘_’ na saída impressa. Veja-se Seção 13.4 [`@allowcodebreaks`], Página 111.

`@anchor{nome}`

Definir *nome* como o local atual para uso como um alvo de referência cruzada. Veja-se Seção 5.11 [`@anchor`], Página 49.

@appendix *título*

Começar um apêndice. O título aparece no sumário. Veja-se Seção 4.4 [**@unnumbered @appendix**], Página 38.

@appendixsec *título***@appendixsection *título***

Começar uma seção de apêndice dentro de um apêndice. O título da seção aparece no sumário. **@appendixsection** é uma ortografia mais longa do comando **@appendixsec**. Veja-se Seção 4.7 [**@unnumberedsec @appendixsec @heading**], Página 39.

@appendixsubsec *título*

Começar uma subseção do apêndice. O título aparece no sumário. Veja-se Seção 4.9 [**@unnumberedsubsec @appendixsubsec @subheading**], Página 40.

@appendixsubsubsec *título*

Começar uma subsubseção do apêndice. O título aparece no índice. Veja-se Seção 4.10 [**@subsubsection**], Página 40.

@arrow{} Gerar um glifo de seta para a direita: ‘→’. Usado por padrão para **@click**. Veja-se Seção 11.8.8 [Sequências de Cliques], Página 107.

@asis Manter o argumento como está. Usado seguinte a **@table**, **@ftable** e **@vtable** para imprimir a primeira coluna da tabela sem realçar (“como está”). Veja-se [**@asis**], Página 77.

@author *autor(a)*

Configurar um(a) autor(a) do manual na página de título. Veja-se Seção 2.8.2.1 [**@title @subtitle @author**], Página 19. Configurar um(a) autor(a) de citação em **@quotation**. Veja-se Seção 7.2 [**@quotation**], Página 66.

@b{*texto*}

Configurar *texto* em uma fonte **negrito**, se possível. Veja-se Seção 6.2.3 [Fontes], Página 63.

@bullet{}

Gerar um ponto redondo grande, ●, ou a coisa mais próxima possível de um. Frequentemente usado com **@itemize**. Veja-se Seção 11.7.5 [**@bullet**], Página 103.

@bsixpaper

Mudar dimensões de página para o tamanho de papel B6. Veja-se Seção E.7 [Papel A4], Página 280.

@bye

Parar processamento de um arquivo. Os processadores não veem nada no arquivo de entrada seguinte a **@bye**. Veja-se Seção 2.12 [Finalizando um Arquivo], Página 25.

@c *comentário*

Começar um comentário no Texinfo. O resto da linha não aparece em nenhuma saída gerada. Um sinônimo para **@comment**. **DEL** também inicia um comentário. Veja-se Seção 2.2 [Comentários], Página 8.

@caption Definir a legenda completa para um **@float**. Veja-se Seção 9.1.2 [**@caption @shortcaption**], Página 82.

@cartouche

Destacar um exemplo ou citação desenhando-se uma caixa com cantos arredondados ao redor dela, se possível. Emparelhar com **@end cartouche**. Veja-se Seção 7.14 [**@cartouche**], Página 72.

@center *linha-de-texto*

Centralizar a linha de texto seguinte ao comando. Veja-se Seção 2.8.2.2 [**@titlefont @center @sp**], Página 20.

@centerchap *linha-de-texto*

Como **@chapter**, porém centraliza o título do capítulo. Veja-se Seção 4.3 [**@chapter**], Página 38.

@chapheading *título*

Imprimir um título semelhante a um capítulo sem numeração, porém omita-o do sumário. Veja-se Seção 4.5 [**@majorheading @chapheading**], Página 39.

@chapter *título*

Começar um capítulo numerado. O título do capítulo aparece no sumário. Veja-se Seção 4.3 [**@chapter**], Página 38.

@cindex *entrada*

Adicionar *entrada* ao índice de conceitos. Veja-se Seção 10.4 [Definindo as Entradas de um Índice], Página 90.

@cite{*referência*}

Realçar o nome de um livro ou outra referência que não tenha arquivo Info complementar. Veja-se Seção 5.15 [**@cite**], Página 52.

@clear *senalizador*

Desconfigurar *senalizador*, evitando que os comandos de formatação do Texinfo formatem texto entre pares subsequentes de comandos **@ifset** *senalizador* e **@end ifset** e evitando que **@value**{*senalizador*} se expanda para o valor para o qual *senalizador* estiver configurado. Veja-se Seção 16.5 [**@set @clear @value**], Página 132.

@click{ } Representar um “clique” em uma GUI. Usado dentro de **@clicksequence**. Veja-se Seção 11.8.8 [Sequências de Cliques], Página 107.

@clicksequence{*ação @click*{ } *ação*}

Representa uma sequência de cliques em uma GUI. Veja-se Seção 11.8.8 [Sequências de Cliques], Página 107.

@clickstyle *@comando*

Esse comando está obsoleto; não o use. Executar *@comando* para cada **@click**; o padrão é **@arrow**. As chaves vazias usuais seguintes a *@comando* são omitidas. Veja-se Seção 11.8.8 [Sequências de Cliques], Página 107.

@code{*código-modelo*}

Indicar uma expressão, um token sintaticamente completo de um programa ou um nome de programa. Veja-se Seção 6.1.2 [**@code**], Página 55.

@codequotebacktick *on-off***@codequoteundirected** *on-off*

Controlar saída gerada de ` e ' em exemplos de código. Veja-se Seção 11.2 [Inserindo Caracteres de Aspas], Página 97.

@comma{ } Inserir um caractere de vírgula ‘,’; necessário somente quando uma vírgula literal fosse considerada como um separador de argumentos. Veja-se Seção 11.1.3 [Inserindo uma Vírgula], Página 95.

@command{*nome-comando*}

Indicar um nome de comando, como em **ls**. Veja-se Seção 6.1.10 [**@command**], Página 59.

@comment *comentário*

Começar um comentário no Texinfo. O resto da linha não aparece em nenhuma saída gerada. Um sinônimo para **@c**. Veja-se Seção 2.2 [Comentários], Página 8.

@contents

Imprimir um sumário completo ou especificar que um sumário deveria ser gerado, para formatos que podem gerar um sumário. Veja-se Seção 2.9 [Gerando um Sumário], Página 21.

@copying Especificar titulares dos direitos autorais e condições de cópia para o documento. Emparelhar com **@end copying**. Veja-se Seção 2.7.1 [copying], Página 16.

@copyright{}

Gerar o símbolo de copirraite ©. Veja-se Seção 11.7.2 [copyright], Página 103.

@defblock

Iniciar um bloco contendo definições. Emparelhar com ‘**@end defblock**’. Veja-se Seção 14.7 [Comandos Genéricos de Definição], Página 123.

@defcodeindex nome-índice

Definir um novo índice e o comando de indexação dele. Imprimir entradas em uma fonte @code. Veja-se Seção 10.7 [Definindo Novos Índices], Página 93.

@defcv categoria classe nome**@defcvx categoria classe nome**

Formatar uma descrição para uma variável associada com uma classe na programação orientada a objetos. Recebe três argumentos: a categoria da coisa sendo definida, a classe à qual ela pertence e o nome dela. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

@defn categoria nome argumentos...**@defnx categoria nome argumentos...**

Formatar uma descrição para uma função, comando interativo ou entidade semelhante que possa receber argumentos. **@defn** recebe como argumentos a categoria da entidade sendo descrita, o nome dessa entidade específica e os argumentos dela, se existirem. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

@defindex nome-índice

Definir um novo índice e o comando de indexação dele. Imprimir entradas em uma fonte romana. Veja-se Seção 10.7 [Definindo Novos Índices], Página 93.

@definfoenclose novocomando, antes, depois

Criar um novo comando **@novocomando** para formatos online que marca texto colocando-o em sequências de caracteres que precedem e seguem o texto. Veja-se Seção 17.6 [definfoenclose], Página 145.

@defivar classe instância-variável-nome**@defivarx classe nome-variável-instância**

Formatar uma descrição para uma variável de instância na programação orientada a objetos. O comando é equivalente a ‘**@defcv {Instância Variável} ...**’. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

@defline categoria nome argumentos...

Usar em um ambiente **@defblock** para fornecer a linha de protótipo de título para um símbolo sendo definido. Esse comando não cria nenhuma entrada de índice. Veja-se Seção 14.7 [Comandos Genéricos de Definição], Página 123.

@defmac nomemacro argumentos...**@defmacx nomemacro argumentos...**

Formatar uma descrição para uma macro; equivalente a ‘**@defn Macro ...**’. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

`@defmethod classe nome-método argumentos...`

`@defmethodx classe nome-método argumentos...`

Formatar uma descrição para um método na programação orientada a objetos; equivalente para ‘`@defop Método ...`’. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

`@defop categoria classe nome argumentos...`

`@defopx categoria classe nome argumentos...`

Formatar uma descrição para uma operação na programação orientada a objetos. `@defop` recebe como argumentos o nome da categoria de operação, o nome da classe da operação, o nome da operação e os argumentos dela, se existirem. Veja-se Capítulo 14 [Comandos de Definição], Página 114, e Seção 14.6.6 [Objetos Abstratos], Página 120.

`@defopt nome-opção`

`@defoptx nome-opção`

Formatar uma descrição para uma opção de usuário(a); equivalente para ‘`@defvr {Opção de Usuário(a)} ...`’. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

`@defspec nome-forma-especial argumentos...`

`@defspecx nome-forma-especial argumentos...`

Formatar uma descrição para uma forma especial; equivalente para ‘`@deffn {Forma Especial} ...`’. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

`@deftp categoria nome-do-tipo atributos...`

`@deftpx categoria nome-do-tipo atributos...`

Formatar uma descrição para um tipo de dados; os argumentos dela são a categoria, o nome do tipo (por exemplo, ‘`int`’) e, em seguida, os nomes dos atributos dos objetos desse tipo. Veja-se Capítulo 14 [Comandos de Definição], Página 114, e Seção 14.6.5 [Tipos de Dados], Página 120.

`@deftypecv categoria classe tipo-dados nome`

`@deftypecvx categoria classe tipo-dados nome`

Formatar uma descrição para uma variável de classe tipada na programação orientada a objetos. Veja-se Capítulo 14 [Comandos de Definição], Página 114, e Seção 14.6.6 [Objetos Abstratos], Página 120.

`@deftypefn categoria tipo-dados nome argumentos...`

`@deftypefnx categoria tipo-dados nome argumentos...`

Formatar uma descrição para uma função ou entidade semelhante que possa receber argumentos e que seja tipada. `@deftypefn` recebe como argumentos a categoria da entidade sendo descrita, o tipo, o nome da entidade e os argumentos dela, se existirem. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

`@deftypefnnewline ligada-desligada`

Especifica se os tipos de retorno para `@deftypefn` e similares são impressos sozinhos nas linhas; o padrão é desligado. Veja-se Seção 14.6.2 [Funções em Linguagens Tipadas], Página 117.

`@deftypefun tipo-dados nome-função argumentos...`

`@deftypefunx tipo-dados nome-função argumentos...`

Formatar uma descrição para uma função em uma linguagem tipada. O comando é equivalente para ‘`@deftypefn Função ...`’. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

@deftypeivar *classe tipo-dados nome-variável*

@deftypeivarx *classe tipo-dados nome-variável*

Formatar uma descrição para uma variável tipada de instância na programação orientada a objetos. Veja-se Capítulo 14 [Comandos de Definição], Página 114, e Seção 14.6.6 [Objetos Abstratos], Página 120.

@deftypeline *categoria tipo-dados nome argumentos...*

Usar dentro de um ambiente **@defblock** para fornecer a linha de protótipo de título para um símbolo sendo definido, com tipos de dados. Esse comando não cria quaisquer entradas de índice. Veja-se Seção 14.7 [Comandos Genéricos de Definição], Página 123.

@deftypemethod *classe tipo-dados nome-método argumentos...*

@deftypemethodx *classe tipo-dados nome-método argumentos...*

Formatar uma descrição para um método tipado na programação orientada a objetos. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

@deftypeop *categoria classe tipo-dados nome argumentos...*

@deftypeopx *categoria classe tipo-dados nome argumentos...*

Formatar uma descrição para uma operação tipada na programação orientada a objetos. Veja-se Capítulo 14 [Comandos de Definição], Página 114, e Seção 14.6.6 [Objetos Abstratos], Página 120.

@deftypevar *tipo-dados nome-variável*

@deftypevarx *tipo-dados nome-variável*

Formatar uma descrição para uma variável em uma linguagem tipada. O comando é equivalente para ‘**@deftypevr** *Variável ...*’. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

@deftypevr *categoria tipo-dados nome*

@deftypevr *categoria tipo-dados nome*

Formatar uma descrição para algo como uma variável em uma linguagem tipada—uma entidade que registra um valor. Recebe como argumentos a categoria da entidade sendo descrita, o tipo e o nome da entidade. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

@defun *nome-função argumentos...*

@defunx *nome-função argumentos...*

Formatar uma descrição para uma função; equivalente para ‘**@defn** *Função ...*’. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

@defvar *nome-variável*

@defvarx *nome-variável*

Formatar uma descrição para uma variável; equivalente para ‘**@defvr** *Variável ...*’. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

@defvr *categoria nome*

@defvr *categoria nome*

Formatar uma descrição para qualquer tipo de variável. **@defvr** recebe como argumentos a categoria da entidade e o nome da entidade. Veja-se Capítulo 14 [Comandos de Definição], Página 114.

@detailmenu

Marcar a listagem detalhada de nós (opcional) em um menu mestre. Veja-se Seção 2.10.1 [Partes do Menu Mestre], Página 23.

- @dfn{termo}**
Indica o uso introdutório ou definidor de um termo. Veja-se Seção 6.1.12 [**@dfn**], Página 60.
- @DH{}**
@dh{} Gerar a letra islandesa maiúscula e minúscula eth, respectivamente: Ð, ð. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.
- @dircategory partediretório**
Especificar uma categoria para o manual. Veja-se Seção 2.6 [Categoria de Diretório], Página 14.
- @direntry**
Iniciar a entrada do menu do diretório Info para esse arquivo. Emparelhar com **@end direntry**. Veja-se Seção 21.1.4 [Instalando Entradas de Diretório], Página 185.
- @display** Iniciar uma espécie de exemplo. Como **@example** (recuar o texto, não preencher), mas não selecionar uma nova fonte. Emparelhar com **@end display**. Veja-se Seção 7.7 [**@display**], Página 69.
- @displaymath**
Formatar um bloco de matemática no formato “display”. Veja-se Capítulo 12 [Inserindo Fórmulas Matemáticas], Página 109.
- @dmn{dimensão}**
Formatar uma unidade de medida, como em 12pt. Veja-se Seção 11.3.5 [**@dmn**], Página 100.
- @docbook** Entrar no DocBook completamente. Emparelhar com **@end docbook**. Veja-se Seção 16.3 [Comandos Brutos do Formatador], Página 131.
- @documentdescription**
Configurar o texto de descrição do documento, incluído na saída gerada HTML. Emparelhar com **@end documentdescription**. Veja-se Seção 22.4 [**@documentdescription**], Página 194.
- @documentencoding codificação**
Declarar a codificação de entrada como *codificação*. Veja-se Seção 15.2 [**@documentencoding**], Página 127.
- @documentinfo**
Iniciar o bloco de metadados do documento. Emparelhar com ‘**@end documentinfo**’. Veja-se Seção 2.8.1 [Metadados de Documento], Página 18.
- @documentlanguage CC**
Declarar o idioma do documento como a abreviatura ISO-639 de dois caracteres *CC*. Veja-se Seção 15.1 [**@documentlanguage**], Página 126.
- @dotaccent{c}**
Gerar um acento de ponto sobre o caractere *c*, como em ó. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.
- @dotless{i-ou-j}**
Gerar *i* sem ponto (‘*i*’) e *j* sem ponto (‘*j*’). Veja-se Seção 11.4 [Inserindo Acentos], Página 100.
- @dots{}** Gerar reticências, ‘...’. Veja-se Seção 11.7.4 [**@dots**], Página 103.
- @email{endereço[, texto-exibido]}**
Indica um endereço de correio eletrônico. Veja-se Seção 6.1.16 [**@email**], Página 62.

@emph{texto}

Enfatizar *texto*. Veja-se Seção 6.2 [Enfatizando Texto], Página 62.

@end ambiente

Termina *ambiente*, como em ‘@end example’. Veja-se [comandos @], Página 8.

@enddots{}

Gerar reticências no final da frase, como esta: ... Veja-se Seção 11.7.4 [dots], Página 103.

@enumerate [número-ou-letra]

Começar uma lista numerada, usando @item para cada entrada. Opcionalmente, iniciar lista com *número-ou-letra*. Emparelhar com @end enumerate. Veja-se Seção 8.3 [enumerate], Página 76.

@env{variável-de-ambiente}

Indicar um nome de variável de ambiente, como PATH. Veja-se Seção 6.1.8 [env], Página 59.

@equiv{}

Indicar para o(a) leitor(a) a equivalência exata de duas formas com um glifo: ‘≡’. Veja-se Seção 11.8.6 [equiv], Página 106.

@error{}

Indicar para o(a) leitor(a) com um glifo que o texto seguinte é uma mensagem de erro: ‘error’. Veja-se Seção 11.8.5 [error], Página 106.

@errormsg{mensagem}

Informar *mensagem* como um erro para erro padrão e saia sem sucesso. Os comandos do Texinfo dentro de *mensagem* são expandidos para texto simples. Veja-se Capítulo 16 [Condiçionais], Página 129, e Seção 17.7 [Processadores Externos de Macro], Página 146.

@euro{}

Gerar o símbolo da moeda Euro. Veja-se Seção 11.7.6 [euro], Página 103.

@evenfooting [esquerda] @| [centro] @| [direita]

@evenheading [esquerda] @| [centro] @| [direita]

Especificar os rodapés das páginas resp. títulos para páginas pares (à esquerda). Veja-se Seção E.2.3 [Como Fazer Teus Próprios Cabeçalhos], Página 276.

@everyfooting [esquerda] @| [centro] @| [direita]

@everyheading [esquerda] @| [centro] @| [direita]

Especificar os rodapés das páginas resp. títulos para cada página. Não relevante Info. Veja-se Seção E.2.3 [Como Fazer Teus Próprios Cabeçalhos], Página 276.

@example

Começar um exemplo. Recuar o texto, não preencher e selecionar uma fonte de largura fixa. Emparelhar com @end example. @example aceita argumentos opcionais, separados por vírgulas. É recomendado configurar o primeiro argumento para o idioma do código de exemplo. Veja-se Seção 7.4 [example], Página 67.

@exampleindent recuo

Recuar ambientes semelhantes a exemplos por *recuo* número de espaços (talvez 0). Veja-se Seção E.5 [exampleindent], Página 279.

@exclamdown{}

Gerar um ponto de exclamação invertido. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@exdent linha-do-texto

Remover qualquer recuo que uma linha possa ter. Veja-se Seção 7.9 [exdent], Página 69.

@expansion{}

Indicar o resultado de uma expansão de macro para o(a) leitor(a) com um glifo especial: ‘ $\mapsto$ ’. Veja-se Seção 11.8.3 [**@expansion**], Página 105.

@file{nomearquivo}

Realçar o nome de um arquivo, buffer, nó, diretório, entre outros. Veja-se Seção 6.1.9 [**@file**], Página 59.

@finalout

Evitar que o T_EX imprima grandes retângulos pretos de aviso ao lado de linhas muito largas. Veja-se Seção 19.5 [hboxes lotados], Página 155.

@index entrada

Adicionar *entrada* ao índice de funções. Veja-se Seção 10.4 [Definindo as Entradas de um Índice], Página 90.

@firstparagraphindent palavra

Controlar o recuo do primeiro parágrafo depois dos cabeçalhos das seções de acordo com *palavra*, entre ‘none’ ou ‘insert’. Veja-se Seção E.4 [**@firstparagraphindent**], Página 279.

@float

Ambiente para definir material flutuante. Emparelhar com **@end float**. Veja-se Seção 9.1 [Flutuadores], Página 81.

@flushleft**@flushright**

Não preencher texto; deixar (direita) justificar cada linha, deixando a extremidade direita (esquerda) irregular. Deixar a fonte como está. Emparelhar com **@end flushleft** (**@end flushright**). Veja-se Seção 7.10 [**@flushleft @flushright**], Página 70.

@fonttextsize 10–11

Mudar o tamanho da fonte do corpo principal na saída impressa. Veja-se Seção 6.2.3 [Fontes], Página 63.

@footnote{texto-da-nota-rodapé}

Inserir uma nota de rodapé, para uma referência que documente ou elucide o texto principal. O texto da nota de rodapé é impresso na parte inferior da página na saída impressa. Em outros formatos, o texto da nota de rodapé pode ser gerado no mesmo nó, em um nó separado ou simplesmente ser marcado como sendo texto de nota de rodapé. Veja-se Seção 9.3 [Notas de Rodapé], Página 85.

@footnotestyle estilo

Especificar um estilo de nota de rodapé, ‘end’ para o estilo do nó final ou ‘separate’ para o estilo separado. No estilo separado, as notas de rodapé são colocadas em um nó ou arquivo separado. Veja-se Seção 9.3.2 [Estilos de Notas de Rodapé], Página 86.

@format

Começar uma espécie de exemplo. Como **@display**, mas não recuar. Emparelhar com **@end format**. Veja-se Seção 7.4 [**@example**], Página 67.

@frenchspacing ligado-desligado

Controlar espaçamento depois de pontuação. Veja-se Seção 11.3.4 [**@frenchspacing**], Página 99.

@ftable comando-formatação

Começar uma tabela de duas colunas, usando **@item** para cada entrada. Inserir automaticamente cada um dos itens da primeira coluna no índice de funções. Emparelhar com **@end ftable**. O mesmo que **@table**, exceto pela indexação. Veja-se Seção 8.4.2 [**@ftable @vtable**], Página 78.

- @geq{}** Gerar um sinal de maior que ou igual a, ‘ $\geq$ ’. Veja-se Seção 11.7.10 [**@geq @leq**], Página 104.
- @group** Proibir quebras de página dentro do texto seguinte. Emparelhar com **@end group**. Ignorado no Info. Veja-se Seção 13.9 [**@group**], Página 112.
- @guillemetleft{}**
@guillemetright{}
@guillemotleft{}
@guillemotright{}
@guilsinglleft{}
@guilsinglright{}
 Aspas duplas e simples: « » < >. **@guillemotleft** e **@guillemotright** são sinônimos para **@guillemetleft** e **@guillemetright**. Veja-se Seção 11.5 [Inserindo Aspas], Página 101.
- @H{c}** Gerar o longo acento húngaro sobre *c*, como em *ő*.
- @hashchar{}**
 Inserir um caractere cerquilha ‘#’; necessário somente quando uma cerquilha literal introduzisse a diretiva **#line**. Veja-se Seção 11.1.5 [Inserindo um Símbolo Cerquilha], Página 96, e Seção 17.7 [Processadores Externos de Macro], Página 146.
- @heading título**
 Imprimir um título semelhante a uma seção não numerada, mas omitir do sumário. Veja-se Seção 4.7 [**@unnumberedsec @appendixsec @heading**], Página 39.
- @headings ligado-desligado-simples-duplo**
 Ligar ou desligar cabeçalhos de página, e(ou) especificar cabeçalhos de página de um lado ou de dois lados para impressão. Veja-se Seção E.2.1 [**@headings**], Página 275.
- @headitem**
 Começar uma linha de cabeçalho em uma tabela múltipla. Veja-se Seção 8.5.2 [Linhas de Multi Tabelas], Página 79.
- @headitemfont{texto}**
 Configurar *texto* na fonte usada para linhas de cabeçalho de múltiplas tabelas; principalmente útil em modelos de multitabelas. Veja-se Seção 8.5.2 [Linhas de Multi Tabelas], Página 79.
- @html** Inserir HTML completamente. Emparelhar com **@end html**. Veja-se Seção 16.3 [Comandos Brutos do Formatador], Página 131.
- @hyphenation{palavras hi-fe-na-das}**
 Definir explicitamente pontos de hifenização. Veja-se Seção 13.3 [**@- @hyphenation**], Página 111.
- @i{texto}**
 Configurar *texto* em uma fonte *itálica*, quando possível. Veja-se Seção 6.2.3 [Fontes], Página 63.
- @ifclear variáveltexi**
 Se a variável Texinfo *variáveltexi* não estiver configurada, formatar o texto a seguir. Emparelhar com **@end ifclear**. Veja-se Seção 16.5 [**@set @clear @value**], Página 132.
- @ifcommanddefined comandotexi**
@ifcommandnotdefined comandotexi
 Se o código Texinfo ‘**@comandotexi**’ (não) estiver definido, formatar o texto a seguir. Emparelhar com o **@end secomando...** correspondente. Veja-se Seção 16.6 [Testagem para Comandos do Texinfo], Página 136.

@ifdocbook

@ifhtml

@ifinfo

@iflatex

@ifplaintext

@ifxml Iniciar texto que aparecerá somente no formato de saída fornecido. A saída @ifinfo aparece em ambas as saídas geradas, Info e (para compatibilidade histórica) texto simples. Emparelhar com @end ifdocbook resp. @end ifhtml... Veja-se Capítulo 16 [Condicionais], Página 129.

@ifnotdocbook

@ifnohtml

@ifnotlatex

@ifnotplaintext

@ifnottex

@ifnotxml

Começar texto para ser ignorado em um formato de saída, mas não nos outros. O texto @ifnohtml é omitido da saída HTML, entre outras. Emparelhar com o @end ifnotformato correspondente. Veja-se Capítulo 16 [Condicionais], Página 129.

@ifnotinfo

Começar texto para aparecer na saída diferente da Info e (para compatibilidade histórica) texto simples. Emparelhar com @end ifnotinfo. Veja-se Capítulo 16 [Condicionais], Página 129.

@ifset *variáveltexi*

Se a variável Texinfo *variáveltexi* estiver configurada, formatar o texto a seguir. Emparelhar com @end ifset. Veja-se Seção 16.5 [@set @clear @value], Página 132.

@iftex Começar texto para aparecer somente na saída do T_EX. Emparelhar com @end iftex. Veja-se Capítulo 16 [Condicionais], Página 129.

@ignore Iniciar texto que não aparecerá em nenhuma saída gerada. Emparelhar com @end ignore. Veja-se Seção 2.2 [Comentários e Texto Ignorado], Página 8.

@image{*nomearquivo*, [*largura*], [*altura*], [*alt*], [*ext*]}

Incluir imagem gráfica em *nomearquivo* externo dimensionado para *largura* e(ou) *altura* fornecidos, usando texto *alt* e procurando por '*nomearquivo.ext*' no HTML. Veja-se Seção 9.2 [Imagens], Página 83.

@include *nomearquivo*

Ler o conteúdo do arquivo fonte do Texinfo *nomearquivo*. Veja-se Capítulo 18 [Arquivos de Inclusão], Página 148.

@indent Inserir recuo de parágrafo. Veja-se Seção 7.13 [@indent], Página 71.

@indentedblock

Recuar um bloco de texto arbitrário à esquerda. Emparelhar com @end indentedblock. Veja-se Seção 7.3 [@indentedblock], Página 67.

@indicateurl{*indicarurl*}

Indicar texto que é um localizador uniforme de recursos para a World Wide Web. Veja-se Seção 6.1.15 [@indicateurl], Página 61.

@inforef{*nome-nó*, [*nome-entrada*], *nome-arquivo-info*}

Fazer uma referência cruzada para um arquivo Info para o qual não existe manual impresso. Veja-se Seção 5.13 [@inforef], Página 50.

`@inlinefmt{formato, texto}`

Inserir *texto* somente se o formato da saída gerada for *formato*. Veja-se Seção 16.4 [Condiçõais Inline], Página 131.

`@inlinefmtifelse{formato, texto, texto-senão}`

Inserir *texto* se o formato da saída gerada for *formato*, caso contrário, *texto-senão*.

`@inlineifclear{variável, texto}`

`@inlineifset{variável, texto}`

Inserir *texto* somente se a variável Texinfo *variável* (não) estiver configurada.

`@inlineraw{formato, texto-bruto}`

Inserir *texto* como em uma condicional bruta, somente se o formato da saída gerada for *formato*.

`\input arquivo-definições-macro`

Usar o arquivo de definições de macro especificado. Esse comando é usado somente na primeira linha de um arquivo Texinfo para fazer com que T_EX faça uso do arquivo de definições de macro `texinfo`. A `\` em `\input` é usada em vez de um `@` porque T_EX não reconhece `@` até depois que tiver lido o arquivo de definições. Veja-se Seção 2.5 [Cabeçalho do Arquivo do Texinfo], Página 11.

`@insertcopying`

Inserir o texto previamente definido com o ambiente `@copying`. Veja-se Seção 2.7.2 [`@insertcopying`], Página 17.

`@item`

Indicar o início de um parágrafo marcado para `@itemize` e `@enumerate`; indicar o início do texto de uma entrada da primeira coluna para `@table`, `@ftable` e `@vtable`. Veja-se Capítulo 8 [Listas e Tabelas], Página 74.

`@itemize marca-gerando-caractere-ou-comando`

Começar uma lista desordenada: parágrafos recuados com uma marca, tal como `@bullet`, dentro da margem esquerda no início de cada item. Emparelhar com `@end itemize`. Veja-se Seção 8.2 [`@itemize`], Página 74.

`@itemx`

Como `@item` em `@table`, `@ftable` e `@vtable`, porém não gerar espaço vertical extra acima do texto do item. Assim, quando vários itens tiverem a mesma descrição, usar `@item` para o primeiro e `@itemx` para os demais. Veja-se Seção 8.4.3 [`@itemx`], Página 78.

`@kbd{caracteres-de-teclado}`

Indicar caracteres de entrada a serem digitados por usuários(as). Veja-se Seção 6.1.3 [`@kbd`], Página 56.

`@kbinputstyle estilo`

Especificar quando `@kbd` deveria usar uma fonte distinta da de `@code` de acordo com *estilo*: `code`, `distinct`, `example`. Veja-se Seção 6.1.3 [`@kbd`], Página 56.

`@key{nome-de-tecla}`

Indicar o nome de uma tecla em um teclado. Veja-se Seção 6.1.4 [`@key`], Página 57.

`@kindex entrada`

Adicionar *entrada* ao índice de chaves. Veja-se Seção 10.4 [Definindo as Entradas de um Índice], Página 90.

`@L{}`

`@l{}`

Gerar as letras L suprimidas em polônês maiúsculo e minúsculo, respectivamente: Ł, ł.

`@LaTeX{}`

Gerar a logomarca do L^AT_EX. Veja-se Seção 11.7.1 [`@LaTeX` do `@TeX`], Página 103.

- @latex** Entrar no L^AT_EX completamente. Emparelhar com **@end latex**. Veja-se Seção 16.3 [Comandos Brutos do Formador], Página 131.
- @leq{}** Gerar um sinal de menor que ou igual a, ‘ $\leq$ ’. Veja-se Seção 11.7.10 [**@geq @leq**], Página 104.
- @linemacro *nomemacro* {parâmetros}**
Definir uma nova macro que receba o restante da linha como argumento e se expanda para um número inteiro de linhas completas. Veja-se Seção 17.5 [Macros de Linha], Página 144.
- @link{*nomenó, rótulo, nome-manual*}**
Criar um linque simples sem marcação visível ou referência de página. Veja-se Seção 5.12 [**@link**], Página 50.
- @lisp** Começar um exemplo de código Lisp. Recuar texto, não preencher, e selecionar fonte de largura fixa. Emparelhar com **@end lisp**. Veja-se Seção 7.6 [**@lisp**], Página 68.
- @listoffloats**
Produzir uma listagem de **@floats**, semelhante a um sumário. Veja-se Seção 9.1.3 [**@listoffloats**], Página 82.
- @lowersections**
Mudar capítulos subsequentes para seções, seções para subseções e assim por diante. Veja-se Seção 4.12 [**@raisesections** e **@lowersections**], Página 41.
- @macro *nomemacro* {parâmetros}**
Definir um novo comando Texinfo **@nomemacro{parâmetros}**. Emparelhar com **@end macro**. Veja-se Seção 17.1 [Definindo Macros], Página 138.
- @maketitle**
Gerar uma página de título e, possivelmente, também a página seguinte. Veja-se Seção 2.8.1 [Metadados de Documento], Página 18.
- @majorheading *título***
Imprimir um título semelhante a um capítulo sem numeração, mas omitir do sumário. Isso gera mais espaços em branco verticais antes do título que o comando **@chapheading**. Veja-se Seção 4.5 [**@majorheading @chapheading**], Página 39.
- @math{*expressão-matemática*}**
Formatar uma expressão matemática. Veja-se Capítulo 12 [Inserindo Fórmulas Matemáticas], Página 109.
- @menu** Marcar o início de um menu de nós. Nenhum efeito em um manual impresso. Emparelhar com **@end menu**. Veja-se Seção 3.9 [Menus], Página 33.
- @microtype *ligada-desligada***
Ative ou desative microtipo. Veja-se Seção E.9 [Microtipografia], Página 280.
- @minus{}** Gerar um sinal de menos, ‘ $-$ ’. Veja-se Seção 11.7.9 [**@minus**], Página 104.
- @multitable *especificação-largura-coluna***
Começar uma tabela com várias colunas. Começar cada linha com **@item** ou **@headitem** e separar colunas com **@tab**. Emparelhar com **@end multitable**. Veja-se Seção 8.5.1 [Larguras de Colunas de Multi Tabelas], Página 79.
- @namedanchor{*nome-âncora, nome-xref*}**
Definir um alvo para uma referência cruzada. Veja-se Seção 5.11 [**@anchor**], Página 49.
- @need *n*** Iniciar uma nova página em um manual impresso se menos que *n* mils (milésimos de polegada) restarem na página atual. Veja-se Seção 13.10 [**@need**], Página 113.

@node *nome*, [*próximo*], [*anterior*], [*acima*]

Começar um novo nó. Somente o primeiro argumento é obrigatório. Veja-se Seção 3.1 [Escrevendo um Nó], Página 26.

@nodedescription *descrição-nó*

Fornecer uma breve elaboração da finalidade de um nó. Veja-se Seção 3.8 [Descrições de Nó], Página 32.

@nodedescriptionblock

Emparelhar com ‘@end nodedescriptionblock’. Usado para iniciar uma descrição mais longa de nó. Veja-se Seção 3.8 [Descrições de Nó], Página 32.

@noindent

Evitar que o texto seja recuado como se fosse um novo parágrafo. Veja-se Seção 7.12 [noindent], Página 71.

@novalidate

Suprimir validação de referências de nós e omitir criação de arquivos auxiliares com T_EX. Usar antes de quaisquer comandos de seccionamento ou de referência cruzada. Veja-se [Validação de Ponteiro], Página 161.

@O{}

@o{} Gerar as letras maiúsculas e minúsculas O com barra, respectivamente: Ø, ø.

@oddfooting [*esquerda*] @| [*centro*] @| [*direita*]

@oddheading [*esquerda*] @| [*centro*] @| [*direita*]

Especificar rodapés de páginas resp. títulos para páginas ímpares (à direita). Veja-se Seção E.2.3 [Como Fazer Teus Próprios Cabeçalhos], Página 276.

@OE{}

@oe{} Gerar as ligaduras OE maiúsculas e minúsculas, respectivamente: Œ, œ. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@ogonek{c}

Gerar um diacrítico ogonek sob o próximo caractere, como em ą. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@option{nome-opção}

Indicar uma opção de linha de comando, como -1 ou em --help. Veja-se Seção 6.1.11 [option], Página 60.

@ordf{}

@ordm{} Gerar os ordinais espanhóis femininos e masculinos, respectivamente: ^a, ^o. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@page

Começar uma nova página em um manual impresso. Veja-se Seção 13.8 [page], Página 112.

@pagesizes [*largura*] [, *altura*]

Mudar dimensões de página. Veja-se [pagesizes], Página 280.

@paragraphindent *recuo*

Recuar parágrafos por *recuo* número de espaços (talvez 0); preservar recuo do arquivo de origem se *recuo* for *asis*. Veja-se Seção E.3 [paragraphindent], Página 278.

@part *título*

Começar um grupo de capítulos ou de apêndices; incluído no sumário. Veja-se Seção 4.11 [part], Página 40.

@pindex entrada

Adicionar *entrada* ao índice de programas. Veja-se Seção 10.4 [Definindo as Entradas de um Índice], Página 90.

@publication

Iniciar informações acerca da publicação do documento. Emparelhar com ‘@end publicação’. Veja-se Seção 2.8.1 [Metadados de Documento], Página 18.

@point{} Indicar a posição do ponto em um buffer para o(a) leitor(a) com um glifo: ‘★’. Veja-se Seção 11.8.7 [point], Página 106.

@pounds{}

Gerar o símbolo da libra esterlina, ‘£’. Veja-se Seção 11.7.7 [pounds], Página 103.

@print{} Indicar saída impressa para o(a) leitor(a) com um glifo: ‘↵’. Veja-se Seção 11.8.4 [print], Página 105.

@printindex nome-índice

Gerar o índice para *nome-índice*. Veja-se Seção 10.5 [Imprimindo Índices e Menus], Página 91.

@pxref{nó, [entrada], [título-nó], [arquivo-info], [manual-impresso]}

Fazer uma referência para ser usada entre parênteses. Começa com um ‘veja-se’ minúsculo em um manual impresso. O primeiro argumento é obrigatório, exceto para referências para manuais inteiros. Para se referir a outro manual como um todo, o *manual-impresso* e(ou) o *arquivo-info* são os únicos argumentos exigidos. Veja-se Seção 5.10 [pxref], Página 48.

@questiondown{}

Gerar um ponto de interrogação invertido. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@quotation

Estreitar as margens para indicar texto que seja citado a partir de outra obra. Recebe argumento opcional especificando texto do prefixo. Emparelhar com @end quotation. Veja-se Seção 7.2 [quotation], Página 66.

@quotedblleft{}**@quotedblright{}****@quoteleft{}****@quoteright{}****@quotedblbase{}****@quotesinglbase{}**

Produzir várias aspas: “ ” ‘ ’ „ „. Veja-se Seção 11.5 [Inserindo Aspas], Página 101.

@r{texto}

Configurar *texto* na fonte romana normal, se possível. Veja-se Seção 6.2.3 [Fontes], Página 63.

@raggedright

Preencher texto; justifique cada linha à esquerda, deixando a extremidade direita irregular. Deixar fonte como está. Emparelhar com @end raggedright. Veja-se Seção 7.11 [raggedright], Página 70.

@raisesections

Mudar seções subsequentes para capítulos, subseções para seções e assim por diante. Veja-se Seção 4.12 [Elevar/rebaixar seções], Página 41.

`@ref{nó, [entrada], [título-nó], [arquivo-info], [manual-impresso]}`

Fazer uma referência simples que não comece com nenhum texto especial. Seguir comando com um sinal de pontuação. O primeiro argumento é obrigatório, exceto para referências para manuais inteiros. Para referenciar outro manual como um todo, o *manual-impresso* e(ou) o *arquivo-info* são os únicos argumentos exigidos. Veja-se Seção 5.9 [`@ref`], Página 48.

`@registeredsymbol{}`

Gerar o símbolo jurídico ®. Veja-se Seção 11.7.3 [`@registeredsymbol`], Página 103.

`@result{}`

Indicar o resultado de uma expressão para o(a) leitor(a) com um glifo especial: ‘⇒’. Veja-se Seção 11.8.2 [`@result`], Página 104.

`@ringaccent{c}`

Gerar um acento circular sobre o próximo caractere, como em ô. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

`@samp{texto}`

Indicar um exemplo literal de uma sequência de caracteres, em geral. Veja-se Seção 6.1.5 [`@samp`], Página 57.

`@sansserif{texto}`

Configurar *texto* em uma fonte sans serif, se possível. Veja-se Seção 6.2.3 [Fontes], Página 63.

`@sc{texto}`

Configurar *texto* em uma fonte minúscula, se possível, e maiúscula no Info. Veja-se Seção 6.2.2 [Versaletes], Página 62.

`@section título`

Começar uma seção dentro de um capítulo. O título da seção aparece no sumário. Dentro de `@chapter` e de `@appendix`, o título da seção é numerado; dentro de `@unnumbered`, a seção é inumerada. Veja-se Seção 4.6 [`@section`], Página 39.

`@seealso{texto-índice}`

Usar em uma entrada de índice para referenciar o(a) leitor(a) para outra entrada de índice relevante. Veja-se Seção 10.3 [Indexação Avançada], Página 90.

`@seeentry{texto-índice}`

Usar em uma entrada de índice para redirecionar o(a) leitor(a) para outra entrada de índice. Veja-se Seção 10.3 [Indexação Avançada], Página 90.

`@set variáveltexinfo [sequênciacaracteres]`

Definir a variável Texinfo *variáveltexinfo*, opcionalmente com o valor *sequênciacaracteres*. Veja-se Seção 16.5 [`@set @clear @value`], Página 132.

`@setchapternewpage impar-ligado-desligado`

Especificar se capítulos iniciam em novas páginas e, em caso afirmativo, se em novas páginas ímpares (à direita). Veja-se Seção E.1 [`@setchapternewpage`], Página 274.

`@setfilename nome-arquivo-info`

Fornecer um nome para ser usado para os arquivos de saída. Esse comando é ignorado para formatação do TeX. Veja-se Seção 2.5.2 [`@setfilename`], Página 12.

`@settitle título`

Especificar o título para cabeçalhos de páginas em um manual impresso e o título padrão do documento para HTML. Veja-se Seção 2.5.3 [`@settitle`], Página 12.

@shortcaption

Definir a legenda curta para um @float. Veja-se Seção 9.1.2 [@caption @shortcaption], Página 82.

@shortcontents

Imprimir um sumário curto, somente com entradas em nível de capítulo, ou especificar que um sumário curto deveria ser gerado. Para formatos que podem gerar um sumário curto. Veja-se Seção 2.9 [Gerando um Sumário], Página 21.

@shorttitlepage título

Gerar uma página mínima de título. Veja-se Seção 2.8 [Página de Título e Página de Copirraite], Página 17.

@slanted{texto}

Configurar *texto* em uma fonte *inclinada*, se possível. Veja-se Seção 6.2.3 [Fontes], Página 63.

@smallbook

Na saída impressa, usar um formato de 7 por 9,25 polegadas em vez do formato normal de 8,5 por 11 polegadas. Veja-se Seção E.6 [@smallbook], Página 280.

@smalldisplay

Começar uma espécie de exemplo. Como @display, mas usar um tamanho de fonte menor onde possível. Emparelhar com @end smalldisplay. Veja-se Seção 7.15 [small], Página 73.

@smallexample

Começar um exemplo. Como @example, mas usar um tamanho de fonte menor onde possível. Emparelhar com @end smallexample. Veja-se Seção 7.15 [small], Página 73.

@smallformat

Começar uma espécie de exemplo. Como @format, mas usar um tamanho de fonte menor onde possível. Emparelhar com @end smallformat. Veja-se Seção 7.15 [small], Página 73.

@smallindentedblock

Como @indentedblock, mas usar um tamanho de fonte menor onde possível. Emparelhar com @end smallindentedblock. Veja-se Seção 7.15 [small], Página 73.

@smalllisp

Começar um exemplo de código Lisp. O mesmo que @smallexample. Emparelhar com @end smalllisp. Veja-se Seção 7.15 [small], Página 73.

@smallquotation

Como @quotation, mas usar um tamanho de fonte menor onde possível. Emparelhar com @end smallquotation. Veja-se Seção 7.15 [small], Página 73.

@sortas {chave}

Usado nos argumentos para indexar comandos para fornecer uma sequência de caracteres pela qual a entrada do índice deveria ser ordenada. Veja-se Seção 10.2 [Comandos de Indexação], Página 89.

@sp n

Ignorar *n* linhas em branco. Veja-se Seção 13.7 [@sp], Página 112.

@ss{}

Gerar a letra alemã S sustenido es-zet, ß. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@strong {texto}

Enfatizar *texto* com mais força que @emph. Veja-se [Enfatizando Texto], Página 62.

@sub {texto}

Configurar *texto* como um subscrito. Veja-se Seção 11.6 [Inserindo Subscritos e Sobrescritos], Página 102.

@subentry

Usar em uma entrada de índice para separar partes de uma entrada multinível. Veja-se Seção 10.3 [Indexação Avançada], Página 90.

@subheading título

Imprimir um título semelhante a uma subseção não numerado, porém omitir do sumário de um manual impresso. Veja-se Seção 4.9 [@unnumberedsubsec @appendixsubsec @subheading], Página 40.

@subsection título

Começar uma subseção dentro de uma seção. O título da subseção aparece no sumário. Mesma numeração dependente do contexto que @section. Veja-se Seção 4.8 [@subsection], Página 40.

@subsubheading título

Imprimir um título semelhante a uma subsubseção não numerado, porém omitir do sumário de um manual impresso. Veja-se Seção 4.10 [@subsubsection], Página 40.

@subsubsection título

Começar uma subsubseção dentro de uma subseção. O título da subsubseção aparece no sumário. Mesma numeração dependente do contexto que @section. Veja-se Seção 4.10 [@subsubsection], Página 40.

@subtitle título

Configurar um subtítulo para a página de título. Veja-se Seção 2.8.2.1 [@title @subtitle @author], Página 19.

@summarycontents

Imprimir ou especificar um sumário curto. Sinônimo para @shortcontents. Veja-se Seção 2.9 [Gerando um Sumário], Página 21.

@sup {texto}

Configurar *texto* como sobrescrito. Veja-se Seção 11.6 [Inserindo Subscritos e Sobrescritos], Página 102.

@syncodeindex índice-origem índice-destino

Mesclar o índice nominado no primeiro argumento no índice nominado no segundo argumento, formatando as entradas a partir do primeiro índice com @code. Veja-se Seção 10.6 [Combinando Índices], Página 92.

@synindex índice-origem índice-destino

Mesclar o índice nominado no primeiro argumento com o índice nominado no segundo argumento. Não mudar a fonte das entradas *índice-origem*. Veja-se Seção 10.6 [Combinando Índices], Página 92.

@t{texto}

Configurar *texto* em uma fonte de **largura fixa**, semelhante a máquina de escrever. Veja-se Seção 6.2.3 [Fontes], Página 63.

@tab

Separar colunas em uma linha de uma multitabela. Veja-se Seção 8.5.2 [Linhas de Multi Tabelas], Página 79.

@table comando-formatação

Começar uma tabela de duas colunas (lista de descrição), usando @item para cada entrada. Escrever cada entrada da primeira coluna na mesma linha que @item.

Entradas da primeira coluna são impressas na fonte resultante de *comando-formatação*. Emparelhar com `@end table`. Veja-se Seção 8.4 [Criando uma Tabela de Duas Colunas], Página 77. Veja-se também Seção 8.4.2 [`@ftable @vtable`], Página 78, e Seção 8.4.3 [`@itemx`], Página 78.

`@TeX{}` Gerar a logomarca do T_EX. Veja-se Seção 11.7.1 [`@LaTeX` do `@TeX`], Página 103.

`@tex` Entrar no T_EX completamente. Emparelhar com `@end tex`. Veja-se Seção 16.3 [Comandos Brutos do Formatador], Página 131.

`@textdegree{}`

Gerar o símbolo grau. Veja-se Seção 11.7.8 [`@textdegree`], Página 104.

`@thischapter`

`@thischaptername`

`@thischapternum`

`@thissection`

`@thissectionname`

`@thissectionnum`

`@thispart`

`@thisfile`

`@thispage`

`@thistitle`

Permitido somente em um cabeçalho ou rodapé. Representa, respectivamente, o número e o nome do capítulo atual (no formato ‘Capítulo 1: Título’), somente o nome do capítulo atual, somente o número do capítulo atual, o número e o nome da seção atual, somente o nome da seção atual, somente o número da seção atual, o nome atual da parte, o nome do arquivo, o número da página atual e o título do documento. Veja-se Seção E.2.3 [Como Criar Teus Próprios Cabeçalhos], Página 276.

`@TH{}`

`@th{}` Gerar a letra maiúscula e minúscula islandesa thorn, respectivamente: Þ, þ. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

`@tie{}` Gerar um espaço normal entre palavras no qual uma quebra de linha não é permitida. Veja-se Seção 13.6 [`@tie`], Página 112.

`@tieaccent{cc}`

Gerar um acento de ligação sobre os próximos dois caracteres *cc*, como em ‘ôô’. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

`@tindex entrada`

Adicionar *entrada* ao índice de tipos de dados. Veja-se Seção 10.4 [Definindo as Entradas de um Índice], Página 90.

`@title título`

Configurar o título para a página de título. Veja-se Seção 2.8.2.1 [`@title @subtitle @author`], Página 19.

`@titlefont{texto}`

Imprimir *texto* em uma fonte maior que a normal, se possível. Veja-se Seção 2.8.2.2 [`@titlefont @center @sp`], Página 20.

`@titlepage`

Iniciar a página de título. Escrever o comando em uma linha própria, emparelhado com `@end titlepage`. A página de título não será gerada, no caso padrão, em formatos online. Veja-se Seção 2.8.2 [`@titlepage`], Página 18.

@today{} Inserir a data atual, no estilo ‘1 de janeiro de 1900’. Veja-se Seção E.2.3 [Como Fazer Teus Próprios Cabeçalhos], Página 276.

@top *título*

Marcar o **@node** mais alto no arquivo, que precisa ser definido na linha imediatamente precedente ao comando **@top**. O título é formatado como um cabeçalho de nível de capítulo. No $\text{\TeX}$, o comando **@top** é meramente um sinônimo para **@unnumbered**.

@U{*hexadecimal*}

Produz uma representação do caractere Unicode $U+hexadecimal$. Veja-se Seção 11.9 [Inserindo Unicode], Página 107.

@u{*c*}

@ubaraccent{*c*}

@udotaccent{*c*}

Gerar um acento breve, um sublinhado ou um ponto, respectivamente, sobre ou sob o caractere *c*, como em ö, o, o. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@unmacro *nomemacro*

Indefine a macro **@*nomemacro*** se ela tiver sido definida. Veja-se Seção 17.1 [Definindo Macros], Página 138.

@unnumbered *título*

Iniciar um capítulo que aparece sem números de capítulo de qualquer tipo. O título aparece no sumário. Veja-se Seção 4.4 [**@unnumbered @appendix**], Página 38.

@unnumberedsec *título*

Iniciar uma seção que aparece sem números de seção de qualquer tipo. O título aparece no sumário. Veja-se Seção 4.7 [**@unnumberedsec @appendixsec @heading**], Página 39.

@unnumberedsubsec *título*

Iniciar uma subseção não numerada. O título aparece no sumário. Veja-se Seção 4.9 [**@unnumberedsubsec @appendixsubsec @subheading**], Página 40.

@unnumberedsubsubsec *título*

Iniciar uma subsubseção não numerada. O título aparece no sumário. Veja-se Seção 4.10 [**@subsubsection**], Página 40.

@uref{*url*[, *texto-exibido*][, *substituição*}

@url{*url*[, *texto-exibido*][, *substituição*}

Define uma referência cruzada para um localizador externo uniforme de recursos, por exemplo, para a World Wide Web. Veja-se Seção 5.14 [**@url**], Página 50.

@urefbreakstyle *estilo*

Especificar como **@uref/@url** deveria quebrar em caracteres especiais: **after**, **before**, **none**. Veja-se Seção 5.14 [**@url**], Página 50.

@v{*c*}

Gerar acento caron sobre o caractere *c*, como em ö. Veja-se Seção 11.4 [Inserindo Acentos], Página 100.

@value{*variáveltexinfo*}

Inserir o valor, se existir, da variável do Texinfo *variáveltexinfo*, previamente definida por **@set**. Veja-se Seção 16.5 [**@set @clear @value**], Página 132.

@var{*variável-metassintática*}

Realçar uma variável metassintática, que é algo que representa outro pedaço de texto. Veja-se Seção 6.1.7 [**@var**], Página 58.

@verb{*delimitador literal delimitador*}

Emitir *literal*, delimitada pelo caractere único *delimitador*, exatamente como está (na fonte de largura fixa), incluindo quaisquer espaços em branco ou caracteres especiais do Texinfo. Veja-se Seção 6.1.6 [**@verb**], Página 58.

@verbatim

Emitir o texto do ambiente exatamente como está (na fonte de largura fixa). Emparelhar com **@end verbatim**. Veja-se Seção 7.5 [**@verbatim**], Página 68.

@verbatiminclude *nomearquivo*

Emitir o conteúdo de *nomearquivo* exatamente como está (na fonte de largura fixa). Veja-se Seção 18.3 [**@verbatiminclude**], Página 149.

@vindex *entrada*

Adicionar *entrada* ao índice de variáveis. Veja-se Seção 10.4 [Definindo as Entradas de um Índice], Página 90.

@vskip *quantidade*

Em um manual impresso, inserir espaços em branco de forma a empurrar o texto no restante da página em direção ao final da página. Usado na formatação da página de copirraite com o argumento ‘*opt plus 1filll*’. (Observe a grafia de ‘*filll*’). Veja-se [Direitos Autorais], Página 19.

@vtable *comando-formatação*

Iniciar uma tabela de duas colunas, usando **@item** para cada entrada. Inserir automaticamente cada um dos itens da primeira coluna no índice de variáveis. Emparelhar com **@end vtable**. O mesmo que **@table**, exceto pela indexação. Veja-se Seção 8.4.2 [**@table @vtable**], Página 78.

@w{*texto*}

Proíbe quebras de linha em *texto*. Veja-se Seção 13.5 [**@w**], Página 111.

@xml

Entrar no XML completamente. Emparelhar com **@end xml**. Veja-se Seção 16.3 [Comandos Brutos do Formatador], Página 131.

@xref{*nó*, [*entrada*], [*título-nó*], [*arquivo-info*], [*manual-impresso*]}

Fazer uma referência que comece com ‘Veja’ em um manual impresso. Seguir o comando com um sinal de pontuação. O primeiro argumento é obrigatório, exceto para referências para manuais inteiros. Para se referir a outro manual como um todo, o *manual-impresso* e(ou) o *arquivo-info* são os únicos argumentos exigidos. Veja-se Seção 5.8 [**@xref**], Página 48.

@xrefautomaticsectiontitle *on-off*

Por padrão, usar o título da seção em vez do nome do nó em referências cruzadas, inclusive nos cabeçalhos dos nós em HTML. Veja-se Seção 5.5 [Três Argumentos], Página 45.

@xrefname *nome-xref*

Usar imediatamente depois de uma linha **@node** para fornecer texto para linkar a esse nó quando **xrefautomaticsectiontitle** estiver ativado, sem imprimir um cabeçalho. Veja-se Seção 4.13 [Nós Desestruturados], Página 42.

A.3 Contextos do Comando @

Aqui nós descrevemos aproximadamente quais comandos @ podem ser usados em quais contextos. Não é exaustivo nem pretende ser uma referência completa. As discrepâncias entre as informações aqui e as implementações dos processadores Texinfo provavelmente serão resolvidas em favor das implementações.

Por *texto geral* abaixo, nós queremos dizer qualquer coisa, exceto seccionamento e outros comandos de documento de nível externo, como `@section`, `@node` e `@setfilename`.

Comandos condicionais `@c`, `@comment` e `@if ... @end if` podem aparecer em qualquer lugar (exceto que os condicionais ainda precisam estar sozinhos nas linhas). `@caption` e `@shortcaption` podem aparecer somente em `@float`, porém podem conter texto geral. Conteúdo do `@footnote` da mesma forma.

Comandos @ com colchetes marcando texto (como `@strong`, `@sc`, `@asis`) podem conter comandos do formatador bruto, como `@html`, mas nenhum outro comando de bloco (outros comandos terminados por `@end`) e não podem ser divididos em parágrafos, mas podem conter texto geral.

Além da restrição de comando de bloco, em `@center`, `@exdent` e `@item` em linhas do `@table`, comandos @ que só fazem sentido em um parágrafo não são aceitos, como `@indent`.

Além do acima, comandos de seccionamento não podem conter `@anchor`, `@footnote` ou `@verb`.

Além do acima, os comandos restantes (`@node`, `@anchor`, `@printindex`, `@ref`, `@math`, `@cindex`, `@url`, `@image` e assim por diante) não podem conter comandos de referência cruzada (`@ref`, `@xref`, `@pxref` e `@inforef`).

Para informações precisas e completas, nós sugerimos uma olhada na suíte de teste nos fontes, que exaustivamente testa combinações.

A.4 Comandos @ Obsoletos

Aqui estão comandos @ do Texinfo que estão obsoletos ou foram completamente removidos. Esta seção é para fins históricos.

@refill Esse comando costumava recarregar e recuar o parágrafo depois de todos os outros processamentos terem sido feitos. Ele não mais é necessário, pois todos os formata-dores agora recarregam automaticamente conforme necessário, mas você ainda pode vê-lo no fonte para alguns manuais, pois ele não causa danos.

@setcontentsaftertitlepage

No passado, os comandos de conteúdo às vezes eram colocados no final do arquivo, depois de quaisquer índices e logo antes do `@bye`, mas nós não mais recomendamos isso. Esse comando poderia ser usado por um(a) usuário(a) imprimindo um manual, para forçar a impressão do conteúdo depois da página de título (após a linha '`@end titlepage`') mesmo se o comando `@contents` estivesse no final do manual.

@setshortcontentsaftertitlepage

Isso colocava o sumário curto depois do comando '`@end titlepage`', mesmo se o comando `@shortcontents` estivesse no final.

Anexo B Dicas e Sugestões

Escritores(as) de manuais GNU deveriam consultar Seção “Manuais GNU” em *Padrões GNU de Codificação*. Aqui estão algumas outras dicas para escrever documentação do Texinfo:

- Escrever no presente, não no passado ou no futuro.
- Escrever ativamente! Por exemplo, escrever “Nós recomendamos que . . .” em vez de “É recomendado que . . .”.
- Projetar teu manual de modo que possa ser lido sequencialmente, na medida do possível. As pessoas se cansam de ficar indo e voltando para encontrar informações que deveriam ser apresentadas a elas conforme necessário.
- Usar 70 ou 72 como tua coluna de preenchimento. Linhas mais longas são difíceis para ler.
- Incluir um aviso de copirraite e permissões de cópia.

Índice, Índice, Índice!

- Escrever muitas entradas de índice, de maneiras diferentes. Você consegue escrever entradas de índice a medida que escreve o corpo do texto, ou escrevê-los depois.
- Escrever entradas de índice somente onde um tópico for discutido significativamente. Por exemplo, não é útil indexar “informações de depuração” em um capítulo acerca de informe de defeitos. Alguém que queira saber acerca de informações de depuração certamente não as encontrará nesse capítulo.
- Escrever os comandos de indexação que se refiram a uma seção inteira imediatamente depois do comando de seção. Escrever os comandos de indexação que se refiram a um parágrafo antes desse parágrafo; dessa maneira, uma entrada de índice aponta para a primeira página de um parágrafo que esteja dividido entre páginas.

No exemplo a seguir, uma linha em branco vem após a entrada do índice para “Pulando”:

```
@section O Cachorro e a Raposa
@cindex Saltando, em geral
@cindex Pulando

@cindex Cachorro, preguiçoso, saltou sobre
@cindex Cachorro preguiçoso saltou sobre
@cindex Raposa, salta sobre cachorro
@cindex Raposa rápida salta sobre cachorro
A rápida raposa marrom salta sobre o cachorro preguiçoso.
```

(Observe que o exemplo mostra entradas para o mesmo conceito que são escritas de maneiras diferentes—‘cachorro preguiçoso’ e ‘cachorro, preguiçoso’—portanto, leitores(as) conseguem pesquisar o conceito de maneiras diferentes).

- Consistentemente maiusculize a primeira palavra de cada entrada do índice de conceitos ou consistentemente use letras minúsculas. Entradas concisas frequentemente exigem letras minúsculas; entradas mais longas para capitalização. Qualquer que seja a convenção de caso que você usar, por favor, use uma ou outra consistentemente! Misturar os dois estilos parece ruim.
- Sempre capitalize ou use letras maiúsculas para as palavras em um índice para o qual isso seja adequado, como nomes de países ou siglas. Sempre use maiúsculas e minúsculas apropriadas para nomes que diferenciam maiúsculas de minúsculas, como aqueles em C ou Lisp.

Frases Completas

Frases completas são mais fáceis de ler que . . .

- Escreva entradas em uma lista detalhada como frases completas; ou, pelo menos, como frases completas. Expressões incompletas . . . estranhas . . . como esta.
- Escrever a frase ou frase introdutória para uma lista ou tabela com vários itens como uma expressão completa. Não escreva “Você consegue configurar.”; em vez disso, escrever “Você consegue configurar essas variáveis:”. A expressão anterior parece cortada.

Apresentando Novos Termos

- Introduzir novos termos para que um(a) leitor(a) que não os conheça consiga compreendê-los a partir do contexto; ou escrever uma definição para o termo.

Por exemplo, a seguir, os termos “check in”, “register” e “delta” estão todos aparecendo pela primeira vez; a frase de exemplo deveria ser reescrita de modo que eles sejam compreensíveis.

A função principal te auxilia no submeter/registrar de um arquivo em teu sistema de controle de versão e no registro de sucessivos conjuntos de mudanças nele como deltas.

- Usar o comando `@dfn` em torno de uma palavra sendo apresentada, para indicar que o(a) leitor(a) deveria não esperar já saber o significado e deveria esperar aprender o significado a partir dessa passagem.

Exemplos Ruins

Aqui estão alguns exemplos de escrita ruim para se evitar:

Ao terminar de editar o arquivo, você precisa realizar um `@dfn{registro}`.

Neste exemplo, digamos, “. . . você precisa `@dfn{registrar}` a nova versão.” Isso flui melhor.

SCCS, RCS e outros sistemas de controle de versão realizam funções semelhantes de maneiras amplamente semelhantes (é essa semelhança que torna possível um modo de controle unificado como esse).

Neste exemplo, digamos, “. . . torna possível uma interface unificada, tal como o modo VC.”

Pontos Finais Fora das Aspas

Colocar pontos finais e outros sinais de pontuação *fora* das citações, a menos que a pontuação seja parte da citação. Essa prática vai contra algumas convenções editoriais nos Estados Unidos da América do Norte, mas habilita o(a) leitor(a) a distinguir entre o conteúdo da citação e a passagem inteira.

Por exemplo, você deveria escrever a seguinte frase com o ponto final fora das aspas finais:

Evidentemente, ‘au’ é uma abreviação para ‘`autor(a)’’.

já que ‘au’ *não* serve como uma abreviação para ‘autor(a).’ (com um ponto final depois da palavra).

Linhas em Branco

- Inserir uma linha em branco entre um comando de seccionamento e a primeira frase ou parágrafo seguinte, ou entre os comandos de indexação associados ao comando de seccionamento e a primeira frase ou parágrafo seguinte, conforme mostrado na dica acerca de indexação. Isso torna o fonte mais fácil de ler.
- Sempre inserir uma linha em branco antes de um comando `@table` e depois de um comando `@end table`; mas nunca inserir uma linha em branco após um comando `@table`.

Por exemplo,

Tipos de raposa:

```
@table @samp
@item Rápida
Pular sobre cachorros preguiçosos.

@item Marrom
Também pula sobre cachorros preguiçosos.
@end table
```

```
@noindent
Por outro lado, ...
```

Inserir linhas em branco antes e depois de `@itemize ... @end itemize` e de `@enumerate ... @end enumerate` da mesma maneira.

Espaços

Não usar espaços para formatar um arquivo do Texinfo, exceto dentro de `@example ... @end example` e outros ambientes e comandos literais.

Por exemplo, `TEX` preenche o seguinte:

```
@kbd{C-x v}
@kbd{M-x vc-next-action}
  Realizar a próxima operação lógica
  sobre o arquivo controlado por versão
  correspondente ao buffer atual.
```

de modo que se parece com isto:

```
C-x v M-x vc-next-action Realizar a próxima operação lógica sobre o arquivo
controlado por versão correspondente ao buffer atual.
```

Nesse caso, o texto deveria ser formatado com `@table`, `@item` e `@itemx`, para criar uma tabela.

@code, @samp, @var e ‘---’

- Usar `@code` em torno de símbolos da Lisp, incluindo nomes de comando. Por exemplo,


```
A função principal é @code{vc-next-action}, ...
```
- Evitar colocar letras como ‘s’ imediatamente depois de um comando ‘@code’. Essas letras aparentam ruins.
- Usar `@var` em torno de metavariáveis. Não escrever colchetes angulares ao redor delas.
- Usar três hifens em uma linha, ‘---’, para indicar um travessão longo. O formatador do Info reduz três hifens para dois; um travessão longo é tipografado em outros formatos de saída.

Nós de Invocação de Programa

Você pode invocar programas como Emacs, GCC e `gawk` a partir de um shell. A documentação para cada programa deveria conter uma seção que descreva isso. Infelizmente, se os nomes e títulos dos nós para essas seções forem todos diferentes, será difícil para usuários(as) encontrá-los.

Portanto, existe uma convenção para nominar tais seções com uma frase começando com a palavra ‘Invocando’, como em ‘Invocando Emacs’; dessa maneira, usuários(as) conseguem encontrar a seção facilmente.

Funções C

Quando você usar `@example` para descrever as convenções de chamada de uma função da C, usar a sintaxe ANSI C, assim:

```
void dld_init (char *@var{caminho});
```

E na discussão subsequente, referenciar os valores dos argumentos escrevendo os mesmos nomes dos argumentos, novamente realçados com `@var`.

Além disso, é melhor evitar escrever `#include` acima da declaração apenas para indicar que a função está declarada em um arquivo de cabeçalho. A prática pode dar a impressão errada de que `#include` pertence próximo à declaração da função. Ou indique explicitamente qual arquivo de cabeçalho contém a declaração ou, melhor ainda, nomeie o arquivo de cabeçalho usado para um grupo de funções no início da seção que descreva as funções.

Comprimento do Nó

Manter nós (seções) para um comprimento razoável, qualquer que seja o razoável no contexto determinado. Não hesitar em dividir nós longos em subnós e ter uma extensa estrutura de árvore; é para isso que eles existem. Muitas vezes, leitores(as) provavelmente tentarão encontrar um ponto específico no manual, usando pesquisa, indexação ou simplesmente adivinhação, em vez de ler tudo do começo ao fim.

Você consegue usar o utilitário `texi-elements-by-size` para ver uma lista de todos os nós (ou seções) no documento, ordenados por tamanho (ou linhas ou palavras), para encontrar candidatos para divisão. Ele está no subdiretório `util/` dos fontes do Texinfo.

Capitalização

- Coloque “Texinfo” em maiúscula; é um nome. Não escrever o ‘x’ ou o ‘i’ em maiúsculas.
- Coloque “Info” em maiúscula; é um nome.
- Escrever $\text{T}_{\text{E}}\text{X}$ usando o comando `@TeX{}`. Observe as letras maiúsculas ‘T’ e ‘X’. Esse comando faz com que os formatadores tipografem o nome de acordo com os desejos de Donald Knuth, que escreveu o $\text{T}_{\text{E}}\text{X}$. (Da mesma forma `@LaTeX{}` para $\text{L}_{\text{A}}\text{T}_{\text{E}}\text{X}$).

E Finalmente ...

- Escrever observações para você mesmo(a) no finalzinho de um arquivo Texinfo depois do `@bye`. Nenhum dos processadores processa texto após o `@bye`; é como se o texto estivesse dentro de `@ignore ... @end ignore`.

Anexo C Arquivos de Amostra do Texinfo

Este apêndice inclui textos para serem usados nos manuais GNU.

C.1 Textos GNU de Amostra

A seguir está uma amostra de documento do Texinfo com os textos completos que deveriam ser usados (adaptados conforme necessário) nos manuais GNU.

Além dos textos jurídicos, também serve como exemplo prático de quantos elementos em um sistema GNU podem afetar o manual. Se você não estiver familiarizado(a) com todos esses diferentes elementos, não se preocupe. Eles não são obrigatórios e um manual perfeitamente bom pode ser escrito sem eles. No entanto, eles estão incluídos aqui porque muitos manuais se beneficiam (ou poderiam) deles.

Veja-se [Amostra Curta], Página 9, para um exemplo mínimo de um arquivo do Texinfo.

Aqui estão algumas observações acerca do exemplo:

- O `version.texi` no comando `@include` é mantido automaticamente pelo Automake (veja-se Seção “Texinfo” em *GNU Automake*). Ele configura os valores de ‘VERSION’, ‘UPDATED’ e ‘UPDATED-MONTH’ usados em outro lugar. Se tua distribuição não usar Automake, mas você usar Emacs, você pode achar o pacote `time-stamp.el` útil (veja-se Seção “Marcas Temporais” em *O Manual do GNU Emacs*).
- O comando `@syncodeindex` reflete a recomendação de usar apenas um índice sempre que possível, para facilitar para os(as) leitores(as) consultar as entradas do índice.
- O `@dircategory` especifica uma categoria para o manual. Ele é usado para construir o diretório Info. Veja-se Seção 2.6 [Categoria de Diretório], Página 14, que inclui uma variedade de nomes recomendados de categorias. Veja-se Seção 21.1.4 [Instalando Entradas de Diretório], Página 185.
- O nó ‘Invocando’ é um padrão GNU para ajudar usuários(as) a encontrar informações básicas acerca do uso da linha de comando de um determinado programa. Veja-se Seção “Detalhes da Estrutura do Manual” em *Padrões GNU de Codificação*.
- Esta amostra mostra como incluir a Licença de Documentação Livre em um manual usando-se o comando `@include`. O arquivo `fdl.texi` está disponível no Texinfo e em outras distribuições do fonte GNU. Ele também está disponível no sítio web do GNU (em <https://www.gnu.org/licenses/fdl-1.3.html>) junto com orientações para usá-lo.
- Se a Fundação do Software Livre não for a titular do copirraite, então usar o nome apropriado.
- Para documentos que expressem tuas opiniões, sentimentos ou experiências pessoais, é mais apropriado usar uma licença que permita somente cópia literal, em vez da Licença de Documentação Livre. Veja-se Seção C.2 [Licença de Cópia Literal], Página 251.

Aqui está um documento de amostra:

```
\input texinfo    @c -*-texinfo-*-
@comment %**start of header
@include version-pt_BR.texi
@settitle Amostra GNU @value{VERSION}
@syncodeindex pg cp
@comment %**end of header
@copying
Este manual é para Amostra GNU (versão @value{VERSION}, @value{UPDATED}),
o qual é um exemplo na documentação do Texinfo.
```

Copirraite @copyright{} 2026 Fundação do Software Livre, Inc.

@quotation

É concedida permissão para copiar, distribuir e(ou) modificar este documento sob os termos da Licença GNU de Documentação Livre, Versão 1.3 ou qualquer versão posterior publicada pela Fundação do Software Livre; sem Seções Invariantes, sem Textos de Capa Frontal e sem Textos de Contracapa. Uma cópia da licença está incluída na seção intitulada ``Licença GNU de Documentação Livre''.

@end quotation

@end copying

@dircategory Sistema de documentação Texinfo

@direntry

* amostra: (amostra)Invocando amostra. amostra: (amostra)Invocando amostra

@end direntry

@documentinfo

@title Amostra GNU

@subtitle para versão @value{VERSION}, @value{UPDATED}

@author A.U. Thor (@email{bug-sample@gnu.org})

@end documentinfo

@maketitle

@contents

@node Top

@top Amostra GNU

Este manual é para Amostra GNU (versão @value{VERSION}, @value{UPDATED}).

@node Invocando amostra

@chapter Invocando amostra

@pindex amostra

@cindex invocando @command{amostra}

Este é um manual de amostra. Não existe nenhum programa de amostra para invocar, mas se existisse, você poderia ver o uso básico e opções de linha de comando dele aqui.

@node Licença GNU de Documentação Livre

@appendix Licença GNU de Documentação Livre

@include fdl-versao_1.3-pt_BR.texi

@node Índice

@unnumbered Índice

@printindex cp

@bye

C.2 Licença de Cópia Literal

Para manuais de software e outras documentações, é fundamental usar uma licença que permita redistribuição e atualização livres, de modo que quando um programa livre for mudado, a documentação também possa ser atualizada.

Por outro lado, para documentos que expressem tuas opiniões, sentimentos ou experiências pessoais, é mais apropriado usar uma licença que permita somente cópia literal.

Aqui está um texto de amostra para tal licença, permitindo somente cópia literal. Este é apenas o próprio texto da licença. Para um documento completo de amostra, vejam-se as seções anteriores.

@copying

Este documento é uma amostra para permissão somente de cópia literal.

Copirraite @copyright{} 2026 Fundação do Software Livre, Inc.

@quotation

É concedida permissão para fazer e distribuir cópias literais de todo este documento sem royalties, desde que o aviso de copirraite e este aviso de permissão sejam preservados.

@end quotation

@end copying

C.3 Licença de Cópia Toda Permissiva

Para manuais de software e outras documentações, é importante usar uma licença que permita redistribuição e atualização livres, de modo que quando um programa livre for mudado, a documentação também possa ser atualizada.

Por outro lado, para pequenos arquivos de suporte, manuais curtos (menos de 300 linhas) e documentação aproximada (arquivos README, arquivos INSTALL, entre outros), a Licença de Documentação Livre completa seria um exagero. Eles podem usar uma licença simples e totalmente permissiva.

Aqui está um texto de amostra para uma tal licença totalmente permissiva. Este é apenas o próprio texto da licença. Para um documento completo de amostra, vejam-se as seções anteriores.

Copirraite @copyright{} 2026 Fundação do Software Livre, Inc.

A cópia e distribuição deste arquivo, com ou sem modificação, são permitidas em qualquer meio sem royalties, desde que o aviso de copirraite e este aviso sejam preservados.

Anexo D Usando Modo Texinfo

Você pode editar um arquivo do Texinfo com qualquer editor de texto que escolher. Um arquivo do Texinfo não é diferente de qualquer outro arquivo ASCII. No entanto, o GNU Emacs vem com um modo especial, chamado modo Texinfo, que fornece comandos e ferramentas do Emacs para ajudar a facilitar teu trabalho.

D.1 Visão Geral do Modo Texinfo

O modo Texinfo oferece recursos especiais para trabalhar com arquivos do Texinfo. Você consegue:

- Inserir comandos @ frequentemente usados.
- Criar automaticamente linha de @node
- Mostrar a estrutura de um arquivo fonte do Texinfo.
- Automaticamente criar ou atualizar os ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ de um nó.
- Automaticamente criar ou atualizar menus.
- Automaticamente criar um menu mestre.
- Formatar uma parte, ou todo, de um arquivo para Info.
- Tipografar e imprimir parte, ou todo, de um arquivo.

Talvez os dois recursos mais úteis sejam aqueles para inserir comandos @ frequentemente usados e para criar ponteiros de nós e menus.

D.2 Os Comandos Usuais de Edição do GNU Emacs

Na maioria dos casos, os comandos usuais do modo Texto funcionam da mesma forma no modo Texinfo e no modo Texto. O modo Texinfo adiciona novos comandos e ferramentas de edição aos recursos de edição de uso geral do GNU Emacs. A principal diferença diz respeito ao preenchimento. No modo Texinfo, a variável de separação de parágrafo e a tabela de sintaxe são redefinidas, de modo que os comandos do Texinfo que deveriam estar em linhas próprias não sejam incluídos inadvertidamente nos parágrafos. Assim, o comando *M-q* (*fill-paragraph*) preencherá um parágrafo, mas não misturará um comando de indexação em uma linha adjacente a ele no parágrafo.

Além disso, o modo Texinfo configura a variável *page-delimiter* com o valor de *texinfo-chapter-level-regex*; por padrão, essa é uma expressão regular que corresponde aos comandos para capítulos e os equivalentes deles, como apêndices. Com esse valor para o delimitador de página, você pode pular de título de capítulo para título de capítulo com os comandos *C-x]* (*forward-page*) e *C-x [* (*backward-page*) e restringir a um capítulo com o comando *C-x n p* (*narrow-to-page*). (Veja-se Seção “Páginas” em *O Manual do GNU Emacs*, para detalhes acerca dos comandos de página).

GNU Emacs entra automaticamente no modo Texinfo quando você visitar um arquivo com uma extensão *.texinfo*, *.texi* ou *.txi*. Além disso, o Emacs comuta para o modo Texinfo quando você visitar um arquivo que tenha ‘*-*-texinfo-**’ na primeira linha dele. Se alguma vez você estiver em outro modo e desejar comutar para o modo Texinfo, digite *M-x texinfo-mode*.

Como todos os outros recursos do Emacs, você pode personalizar ou aprimorar o modo Texinfo conforme desejar. Em particular, os atalhos de teclado são muito fáceis de mudar. As combinações de teclas descritas aqui são as pré configuradas ou padrão.

D.3 Inserindo Comandos Frequentemente Usados

O modo Texinfo fornece comandos para inserir vários comandos @ frequentemente usados no buffer. Você consegue usar esses comandos para salvar as teclas digitadas.

Os comandos de inserção são invocados digitando-se *C-c* duas vezes e depois a primeira letra do comando @:

C-c C-c c

M-x texinfo-insert-@code

Inserir @code{} e colocar o cursor entre as chaves.

C-c C-c d

M-x texinfo-insert-@dfn

Inserir @dfn{} e colocar o cursor entre as chaves.

C-c C-c e

M-x texinfo-insert-@end

Inserir @end e tentar inserir a palavra correta a seguir, como ‘exemplo’ ou ‘tabela’. (Esse comando não trata listas aninhadas corretamente, mas insere a palavra apropriada na lista imediatamente anterior).

C-c C-c i

M-x texinfo-insert-@item

Inserir @item e colocar o cursor no início da próxima linha.

C-c C-c k

M-x texinfo-insert-@kbd

Inserir @kbd{} e colocar o cursor entre as chaves.

C-c C-c n

M-x texinfo-insert-@node

Inserir @node e uma linha de comentário listando a sequência para os nós ‘Próximo’, ‘Anterior’ e ‘Acima’. Deixar um ponto depois do @node.

C-c C-c o

M-x texinfo-insert-@noindent

Inserir @noindent e colocar o cursor no início da próxima linha.

C-c C-c r

M-x texinfo-insert-dwim-@ref

Essa função e ligação foram adicionadas no Emacs 27.1. Insere um de @pxref{}, @xref{} ou @ref{} baseado no texto ao redor do ponto; chamá-la perto de um parêntese aberto anterior não fechado resulta em @pxref{}, no início de uma frase ou em (point-min) produz @xref{}, qualquer outro local (inclusive dentro de uma palavra), resultará em @ref{}. Um argumento numérico diz quantas palavras as chaves deveriam envolver. Coloca o cursor entre as chaves.

C-c C-c s

M-x texinfo-insert-@samp

Inserir @samp{} e colocar o cursor entre as chaves.

C-c C-c t

M-x texinfo-insert-@table

Inserir @table seguido por um SPC e deixar o cursor depois de SPC.

C-c C-c v

M-x texinfo-insert-@var

Inserir @var{} e colocar o cursor entre as chaves.

C-c C-c x

M-x texinfo-insert-@example

Inserir @example e colocar o cursor no início da próxima linha.

C-c C-c {

M-x texinfo-insert-braces

Inserir {} e colocar o cursor entre as chaves.

C-c }

C-c]

M-x up-list

Mover-se de entre um par de chaves em direção a depois da chave de fechamento. Digitar-se **C-c]** é mais fácil que digitar **C-c }**, que é, no entanto, mais mnemônico; daí as duas combinações de teclas. (Além disso, você consegue sair de entre chaves digitando **C-f**).

Para colocar um comando como **@code{...}** em torno de uma palavra *existente*, posicionar o cursor em frente da palavra e digitar **C-u 1 C-c C-c c**. Isso facilita a edição de texto simples existente. O valor do argumento prefixo informa ao Emacs quantas palavras após o ponto incluir entre chaves—‘1’ para uma palavra, ‘2’ para duas palavras e assim por diante. Usar um argumento negativo para incluir a palavra, ou palavras, anterior. Se você não especificar um argumento de prefixo, o Emacs insere a sequência de caracteres do comando @ e posiciona o cursor entre as chaves. Esse recurso funciona somente para aqueles comandos @ que operam sobre uma palavra ou palavras dentro de uma linha, tais como @kbd e @var.

Este conjunto de comandos de inserção foi criado depois de se analisar a frequência com que diferentes comandos @ são usados no *Manual do GNU Emacs* e no *Manual do GDB*. Se desejar adicionar teus próprios comandos de inserção, você pode vincular uma macro de teclado a uma tecla, usar abreviações ou estender o código em **texinfo.el**.

C-c C-c C-d (texinfo-start-menu-description) é um comando de inserção que funciona diferentemente dos outros comandos de inserção. Ele insere uma seção de um nó ou título de capítulo no espaço para a descrição em uma linha de entrada de menu. (Uma entrada de menu tem três partes: o nome da entrada, o nome do nó e a descrição. Somente o nome do nó é exigido, mas uma descrição ajuda a explicar do que se trata o nó. Veja-se Seção 3.9.4 [As Partes de um Menu], Página 35).

Para usar **texinfo-start-menu-description**, posicionar o ponto em uma linha de entrada de menu e digitar **C-c C-c C-d**. O comando procura e copia o título que acompanha o nome do nó e insere o título como uma descrição; ele posiciona o ponto no início do texto inserido, de modo que você consiga editá-lo. A função não insere o título se a linha de entrada do menu já contiver uma descrição.

Esse comando é apenas uma ajuda para se escrever descrições; ele não faz todo o trabalho. Você precisa editar o texto inserido, pois um título tende a usar as mesmas palavras que um nome de nó, mas uma descrição útil usa palavras diferentes.

D.4 Mostrando a Estrutura de Seccionamento de um Arquivo

Você consegue mostrar a estrutura de seccionamento de um arquivo do Texinfo usando o comando **C-c C-s (texinfo-show-structure)**. Esse comando lista as linhas que começam com os comandos @ para @chapter, @section e similares. Ele constrói o que equivale a um sumário. Essas linhas são exibidas em outro buffer chamado o buffer ‘*0ccur*’. Nesse buffer, você consegue posicionar o cursor sobre uma das linhas e usar o comando **C-c C-c (occur-mode-goto-occurrence)**, para pular para o local correspondente no arquivo do Texinfo.

C-c C-s

M-x texinfo-show-structure

Mostrar o `@chapter`, `@section` e outras linhas de um arquivo do Texinfo.

C-c C-c

M-x occur-mode-goto-occurrence

Ir para a linha no arquivo do Texinfo correspondente à linha sob o cursor no buffer `*Occur*`.

Se você chamar `texinfo-show-structure` com um argumento de prefixo digitando `C-u C-c C-s`, ele listará não apenas essas linhas com os comandos `@` para `@chapter`, `@section` e similares, mas também as linhas `@node`. Você consegue usar `texinfo-show-structure` com um argumento de prefixo para verificar se os ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ de uma linha `@node` estão corretos.

Frequentemente, quando estiver trabalhando em um manual, você estará interessado(a) somente na estrutura do capítulo atual. Nesse caso, você consegue marcar a região do buffer em que está interessado(a) usando o comando `C-x n n` (`narrow-to-region`) e `texinfo-show-structure` funcionará apenas sobre essa região. Para ver o buffer inteiro novamente, use `Cx n w` (`widen`). (Veja-se Seção “Estreitando” em *O Manual do GNU Emacs*, para mais informações acerca dos comandos de estreitamento).

Além de fornecer o comando `texinfo-show-structure`, o modo Texinfo configura o valor da variável delimitadora de página para corresponder aos comandos `@` de nível de capítulo. Isso te habilita a usar os comandos `C-x ]` (`forward-page`) e `C-x [` (`backward-page`) para avançar e retroceder por capítulo, e a usar o comando `C-x n p` (`narrow-to-page`) para restringir a um capítulo. Veja-se Seção “Páginas” em *O Manual do GNU Emacs*, para mais informações acerca dos comandos de página.

D.4.1 Usando texinfo-show-structure

Nem sempre é fácil manter-se a par dos nós, capítulos, seções e subseções de um arquivo do Texinfo. Isso é especialmente verdadeiro se você estiver revisando, ou adicionando a, um arquivo do Texinfo que outra pessoa tenha escrito.

No GNU Emacs, no modo Texinfo, o comando `texinfo-show-structure` lista todas as linhas que comecem com os comandos `@` que especifiquem a estrutura: `@chapter`, `@section`, `@appendix` e assim por diante. Com um argumento (`C-u` como argumento de prefixo, se interativo), o comando também mostra as linhas `@node`. O comando `texinfo-show-structure` está vinculado a `C-c C-s` no modo Texinfo, por padrão.

As linhas são exibidas em um buffer chamado o buffer `*Occur*`, recuado por nível hierárquico. Por exemplo, aqui está uma parte do que foi produzido executando-se `texinfo-show-structure` neste manual:

```
Lines matching "~@\\(chapter \\|sect\\|subs\\|subh\\|
unnum\\|major\\|chapheading \\|heading \\|appendix\\)"
in buffer texinfo.texi.
...
4177:@chapter Nodes
4198:  @heading Dois Caminhos
4231:  @section Ilustração de Nó e de Menu
4337:  @section O Comando @code{@@node}
4393:    @subheading Escolhendo Nomes de Nó e de Ponteiro
4417:    @subsection Como Escrever uma Linha @code{@@node}
4469:    @subsection Dicas de Linha @code{@@node}
...
```

Isso diz que as linhas 4337, 4393 e 4417 de `texinfo.texi` começam com os comandos `@section`, `@subheading` e `@subsection` respectivamente. Se você mover o cursor para a janela `*0occur*`, poderá posicionar o cursor sobre uma das linhas e usar o comando `C-c C-c` (`occur-mode-goto-occurrence`), para pular para o local correspondente no arquivo Texinfo. Veja-se Seção “Usando Occur” em *O Manual do GNU Emacs*, para mais informações acerca do `occur-mode-goto-occurrence`.

A primeira linha na janela `*0occur*` descreve a expressão regular especificada por `texinfo-cabeçalho-padrão`. Essa expressão regular é o padrão que `texinfo-show-structure` procura. Veja-se Seção “Usando Expressões Regulares” em *O Manual do GNU Emacs*, para mais informações.

Quando você invocar o comando `texinfo-show-structure`, o Emacs exibirá a estrutura do buffer inteiro. Se quiser ver a estrutura de apenas uma parte do buffer, de um capítulo, por exemplo, use o comando `C-x n n` (`narrow-to-region`) para marcar a região. (Veja-se Seção “Estreitando” em *O Manual do GNU Emacs*). Foi assim que o exemplo usado acima foi gerado. (Para ver o buffer inteiro novamente, use `C-x n w` (`widen`)).

Se você chamar `texinfo-show-structure` com um argumento de prefixo digitando `C-u C-c C-s`, ele listará as linhas que comecem com `@node`, bem como as linhas que comecem com os comandos de sinal `@` para `@chapter`, `@section` e similares.

Você consegue se lembrar da estrutura de um arquivo do Texinfo olhando a lista na janela `*0occur*`; e se você nomeou incorretamente um nó ou deixou de fora uma seção, poderá corrigir o erro.

D.4.2 Usando occur

Às vezes, o comando `texinfo-show-structure` produz muitas informações. Talvez você queira se lembrar da estrutura geral de um arquivo do Texinfo e fique impressionado(a) pela lista detalhada produzida por `texinfo-show-structure`. Nesse caso, você consegue usar o comando `occur` diretamente. Para fazer isso, digite:

```
M-x occur
```

e então, quando solicitado(a), digite uma *regex*, uma expressão regular para o padrão que você quiser corresponder. (Veja-se Seção “Expressões Regulares” em *O Manual do GNU Emacs*). O comando `occur` funciona desde o local atual do cursor no buffer até o final do buffer. Se você quiser executar `occur` sobre o buffer inteiro, coloque o cursor no início do buffer.

Por exemplo, para ver todas as linhas que contêm a palavra `@chapter`, basta digitar `@chapter`. Isso produzirá uma lista dos capítulos. Também listará todas as frases com `@chapter` no meio da linha.

Se você quiser ver somente as linhas que comecem com a palavra `@chapter`, digite `^@chapter` quando solicitado(a) por `occur`. Se você quiser ver todas as linhas que terminem com uma palavra ou frase, termine a última palavra com um `$`; por exemplo, `capturando erros$`. Isso pode ser útil quando você quiser ver todos os nós que sejam parte do mesmo capítulo ou seção e, portanto, têm o mesmo ponteiro ‘Acima’.

Veja-se Seção “Usando Occur” em *O Manual do GNU Emacs*, para mais informações.

D.5 Atualizando Nós e Menus

O comando `texi2any` criará um arquivo do Info para um arquivo do Texinfo organizado hierarquicamente que careça dos ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ (veja-se Seção 3.1 [Escrevendo um Nó], Página 26). Assim, em geral, não existe necessidade para ponteiros explícitos ‘Próximo’, ‘Anterior’ e ‘Acima’. Nessa configuração, os menus serão adicionados automaticamente para nós sem um menu explícito. (Veja-se Capítulo 20 [Tradutor Genérico `texi2any`], Página 157, para mais informações acerca do `texi2any`).

Se você ainda quiser ponteiros explícitos, o modo Texinfo fornece comandos para criar ou atualizar automaticamente menus e ponteiros de nós. Os comandos são chamados de comandos de “atualização” porque o uso mais frequente deles é para atualizar um arquivo do Texinfo depois de você ter trabalhado nele; mas você consegue usá-los para inserir os ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ em uma linha `@node` que não tenha nenhum e para criar menus em um arquivo que não tenha nenhum.

D.5.1 Os Comandos de Atualização

Você consegue usar os comandos de atualização para:

- inserir ou atualizar os ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’, de um nó,
- inserir ou atualizar o menu para uma seção, e
- criar um menu mestre para um arquivo fonte do Texinfo.

Você também consegue usar os comandos para atualizar todos os nós e menus em uma região ou em um arquivo inteiro do Texinfo.

Os comandos de atualização funcionam somente com arquivos convencionais do Texinfo, que sejam estruturados hierarquicamente como livros. Nesses arquivos, uma linha com um comando de estruturação precisa seguir logo depois de cada linha `@node`, exceto para a linha ‘Top’ do `@node`.

Você consegue escrever o comando de estruturação na linha que segue imediatamente depois de uma linha `@node` ou então na linha que segue depois de uma linha de `@comment` único ou de uma linha de `@ifinfo` único. Você não pode interpor mais que uma linha entre a linha `@node` e a linha do comando de estruturação; e você pode interpor somente uma linha `@comment` ou uma linha `@ifinfo`.

Comandos que funcionam sobre um buffer inteiro exigem que o nó ‘Top’ seja seguido por um nó com um `@chapter` ou comando de nível equivalente. Os comandos de atualização de menu não criarão um menu principal ou mestre para um arquivo do Texinfo que tenha somente nós de nível `@chapter`! Os comandos de atualização de menu criam somente nós de menus *dentro* para nós de nível inferior. Para criar um menu de capítulos, você precisa fornecer um nó ‘Top’.

Os comandos de atualização de menu removem entradas de menu que referenciem a outros arquivos do Info, uma vez que elas não referenciam a nós dentro do buffer atual. Isso é uma deficiência. Em vez de usar entradas de menu, você consegue usar referências cruzadas para referenciar a outros arquivos do Info. Nenhum dos comandos de atualização afeta referências cruzadas.

O modo Texinfo tem cinco comandos de atualização que são usados mais frequentemente: dois são para atualizar os ponteiros de nó ou menu de um nó único (ou região); dois são para atualizar cada ponteiro de nó e menu em um arquivo; e um, o comando `texinfo-master-menu`, é para criar um menu mestre para um arquivo completo e, opcionalmente, para atualizar cada nó e menu no arquivo inteiro do Texinfo.

O comando `texinfo-master-menu` é o comando primário:

C-c C-u m

M-x texinfo-master-menu

Criar ou atualizar um menu mestre que inclua todos os outros menus (incorporando as descrições originárias de menus preexistentes, se existirem).

Com um argumento (argumento de prefixo, **C-u**, se interativo), primeiro criar ou atualizar todos os nós e todos os menus regulares no buffer antes de construir o menu mestre. (Veja-se Seção 2.10 [O Nó Top e o Menu Mestre], Página 22, para mais acerca de um menu mestre).

Para `texinfo-master-menu` funcionar, o arquivo do Texinfo precisa ter um nó ‘Top’ e pelo menos um nó subsequente.

Depois de editar extensivamente um arquivo do Texinfo, você pode digitar o seguinte:

```
C-u M-x texinfo-master-menu
```

ou

```
C-u C-c C-u m
```

Isso atualiza todos os nós e menus completamente e de uma vez.

Os outros comandos principais de atualização realizam trabalhos menores e são projetados para a pessoa que atualiza nós e menus enquanto escreve um arquivo do Texinfo.

Os comandos são:

C-c C-u C-n

M-x texinfo-update-node

Inserir os ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ para o nó em que o ponto está (ou seja, para o ponto anterior da linha `@node`). Se a linha `@node` tiver ponteiros preexistentes ‘Próximo’, ‘Anterior’ ou ‘Acima’, os ponteiros antigos serão removidos e novos serão inseridos. Com um argumento (argumento de prefixo, *C-u*, se interativo), esse comando atualiza todas as linhas `@node` na região (que é o texto entre o ponto e a marca).

C-c C-u C-m

M-x texinfo-make-menu

Criar ou atualizar o menu no nó em que o ponto está. Com um argumento (*C-u* como argumento de prefixo, se interativo), o comando cria ou atualiza menus para os nós que ou estejam dentro ou sejam uma parte da região.

Sempre que `texinfo-make-menu` atualizar um menu existente, as descrições originárias desse menu serão incorporadas no novo menu. Isso é feito copiando as descrições originárias do menu existente para as entradas no novo menu que tenham os mesmos nomes de nós. Se os nomes de nós forem diferentes, as descrições não serão copiadas para o novo menu.

C-c C-u C-e

M-x texinfo-every-node-update

Inserir ou atualizar os ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ para cada nó no buffer.

C-c C-u C-a

M-x texinfo-all-menus-update

Criar ou atualizar todos os menus no buffer. Com um argumento (*C-u* como argumento de prefixo, se interativo), primeiro inserir ou atualizar todos os ponteiros de nó antes de trabalhar nos menus.

Se um menu mestre existir, o comando `texinfo-all-menus-update` o atualiza; mas o comando não cria um novo menu mestre se já não existir nenhum. (Usar o comando `texinfo-master-menu` para isso).

Ao trabalhar em um documento que não merece um menu mestre, você pode digitar o seguinte:

```
C-u C-c C-u C-a
```

ou

```
C-u M-x texinfo-all-menus-update
```

Isso atualiza todos os nós e menus.

A variável `texinfo-column-for-description` especifica a coluna para a qual descrições de menu são recuadas. Por padrão, o valor é 32, embora possa ser útil reduzi-lo para 24. Você pode configurar a variável por meio de personalização (veja-se Seção “Personalização” em *O*

Manual do GNU Emacs) ou com o comando `M-x set-variable` (veja-se Seção “Examinando e Configurando Variáveis” em *O Manual do GNU Emacs*).

Além disso, o comando `texinfo-indent-menu-description` pode ser usado para recuar descrições existentes de menu para uma coluna especificada. Finalmente, se desejar, você consegue usar o comando `texinfo-insert-node-lines` para inserir linhas `@node` ausentes em um arquivo. Em particular, você consegue ignorar completamente linhas `@node` em teu primeiro rascunho e então usar o comando `texinfo-insert-node-lines` para criar linhas `@node` para você. No entanto, nós não recomendamos essa prática. É melhor nominar o próprio nó ao mesmo tempo em que você escrever um segmento, de modo que consiga facilmente fazer referências cruzadas. Referências cruzadas úteis são um recurso especialmente importante de um bom manual do Texinfo. (Veja-se Seção D.5.5 [Outros Comandos de Atualização], Página 261, para mais informações).

D.5.2 Exigências de Atualização

Para usar os comandos de atualização, você precisa organizar o arquivo do Texinfo hierarquicamente com capítulos, seções, subseções e similares. Ao construir a hierarquia do manual, não ‘pule para baixo’ mais que um nível por vez: você pode seguir o nó ‘Top’ com um capítulo, mas não com uma seção; você pode seguir um capítulo com uma seção, mas não com uma subseção. Entretanto, você pode ‘pular para cima’ qualquer número de níveis ao mesmo tempo—por exemplo, de uma subseção para um capítulo.

Cada linha `@node`, com exceção da linha para o nó ‘Top’, precisa ser seguida por uma linha com um comando estruturante como `@chapter`, `@section` ou `@unnumberedsubsec`.

Cada combinação de linha `@node`/linha de comando estruturante precisa ou se parecer com esta:

```
@node      Comentários, Mínimo, Convenções, Visão Geral
@comment  nome-nó, próximo, anterior, acima
@section  Comentários
```

ou com esta (sem a linha `@comment`):

```
@node Comentários, Mínimo, Convenções, Visão Geral
@section Comentários
```

ou com esta (sem os ponteiros explícitos de nó):

```
@node Comentários
@section Comentários
```

Neste exemplo, ‘Comentários’ é o nome de ambos, do nó e da seção. O próximo nó é denominado ‘Mínimo’ e o nó anterior é denominado ‘Convenções’. A seção ‘Comentários’ está dentro do nó ‘Visão Geral’, o qual é especificado pelo ponteiro ‘Acima’.

Se um arquivo tiver um nó ‘Top’, ele precisa ser chamado ‘`top`’ ou ‘`Top`’ e ser o primeiro nó no arquivo.

Os comandos de atualização de menu criam um menu de seções dentro de um capítulo, um menu de subseções dentro de uma seção e assim por diante. Isso significa que você precisa ter um nó ‘Top’ se quiser um menu de capítulos.

D.5.3 Atualizar Arquivo Externo e Incluir Arquivos

O modo Texinfo do GNU Emacs fornece o comando `texinfo-multiple-files-update`. Esse comando cria ou atualiza ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ dos arquivos incluídos, bem como aqueles no arquivo Texinfo externo ou geral, e cria ou atualiza um menu principal no arquivo externo. Dependendo se você o chama com argumentos opcionais, o comando atualiza somente os ponteiros na primeira linha `@node` dos arquivos incluídos ou todos eles.

Com `C-u` como argumento de prefixo, criar e inserir um menu mestre no arquivo externo. Com um argumento de prefixo e numérico, como `C-u 2`, primeiro atualizar todos os menus e todos os ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ de todos os arquivos incluídos antes de criar e de inserir um menu mestre no arquivo externo.

Em mais detalhes:

`M-x texinfo-multiple-files-update`

Chamado sem quaisquer argumentos:

- Criar ou atualizar os ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ da primeira linha `@node` em cada arquivo incluído em um arquivo do Texinfo externo ou geral.
- Criar ou atualizar os ponteiros de nó do nível ‘Top’ do arquivo externo ou geral.
- Criar ou atualizar um menu principal no arquivo externo.

`C-u M-x texinfo-multiple-files-update`

Chamado com `C-u` como um argumento de prefixo:

- Criar ou atualizar ponteiros na primeira linha `@node` em cada arquivo incluído.
- Criar ou atualizar os ponteiros de nó do nível ‘Top’ do arquivo externo.
- Criar e inserir um menu mestre no arquivo externo. O menu mestre é feito a partir de todos os menus em todos os arquivos incluídos.

`C-u 8 M-x texinfo-multiple-files-update`

Chamado com um argumento de prefixo e numérico, tal como `C-u 8`:

- Criar ou atualizar *todos* os ponteiros ‘Próximo’, ‘Anterior’ e ‘Acima’ de todos os arquivos incluídos.
- Criar ou atualizar *todos* os menus de todos os arquivos incluídos.
- Criar ou atualizar os ponteiros de nó do nível ‘Top’ do arquivo externo ou geral.
- E então criar um menu mestre no arquivo externo. Isso é semelhante a invocar-se `texinfo-master-menu` com um argumento quando você estiver trabalhando com apenas um arquivo.

Observe o uso do argumento de prefixo no uso interativo: com um argumento de prefixo regular, apenas `C-u`, o comando `texinfo-multiple-files-update` insere um menu mestre; com um argumento de prefixo e numérico, como `C-u 8`, o comando atualiza *cada* ponteiro e menu em *todos* os arquivos e então insere um menu mestre.

D.5.4 Exigências de Arquivos de Inclusão

Se você planeja usar o comando `texinfo-multiple-files-update`, o arquivo externo do Texinfo que lista os arquivos incluídos nele não deveria conter nada além das partes inicial e final de um arquivo do Texinfo e vários comandos `@include` listando os arquivos incluídos. Ele não deveria nem incluir índices, os quais deveriam ser listados em um arquivo incluído próprio.

Além disso, cada um dos arquivos incluídos precisa conter exatamente um nó de nível mais alto (convencionalmente, `@chapter` ou equivalente), e esse nó precisa ser o primeiro nó no arquivo incluído. Além disso, cada um desses nós de nível mais alto em cada arquivo incluído precisa estar no mesmo nível hierárquico na estrutura do arquivo. Normalmente, cada um é um nó `@chapter`, `@appendix` ou `@unnumbered`. Assim, normalmente, cada arquivo incluído contém um, e apenas um, capítulo ou nó de nível equivalente.

O arquivo externo deveria conter apenas *um* nó, o nó ‘Top’. Ele deveria *não* conter quaisquer nós além do único nó ‘Top’. O comando `texinfo-multiple-files-update` não os processará.

D.5.5 Outros Comandos de Atualização

Além dos principais comandos de atualização, o modo Texinfo possui vários comandos de atualização usados menos frequentemente:

M-x texinfo-insert-node-lines

Inserir linhas `@node` antes de `@chapter`, `@section` e outros comandos de seccionamento sempre que eles estiverem ausentes em uma região em um arquivo do Texinfo.

Com um argumento (`C-u` como argumento de prefixo, se interativo), o comando `texinfo-insert-node-lines` não apenas insere linhas `@node`, mas também insere os títulos dos capítulos ou das seções como os nomes dos nós correspondentes. Além disso, ele insere os títulos como nomes de nós em linhas `@node` preexistentes que careçam de nomes. Como nomes de nós deveriam ser mais concisos que títulos de seções ou de capítulos, você precisa editar manualmente nomes de nós inseridos desse modo.

Por exemplo, o seguinte marca um buffer inteiro como uma região e insere linhas `@node` e títulos por toda parte:

```
C-x h C-u M-x texinfo-insert-node-lines
```

Esse comando insere títulos como nomes de nós nas linhas `@node`; o comando `texinfo-start-menu-description` (veja-se Seção D.3 [Inserindo], Página 253) insere títulos como descrições nas entradas do menu, uma ação diferente. Porém, em ambos os casos, você precisa editar o texto inserido.

M-x texinfo-indent-menu-description

Recuar cada descrição no menu seguinte ao ponto para a coluna especificada. Você pode usar esse comando para ter mais espaço para descrições. Com um argumento (`C-u` como argumento de prefixo, se interativo), o comando `texinfo-indent-menu-description` recua cada descrição em cada menu na região. No entanto, esse comando não recua a segunda e as linhas subsequentes de uma descrição multilinha.

M-x texinfo-sequential-node-update

Inserir os nomes dos nós imediatamente seguintes e anteriores ao nó atual como os ponteiros ‘Próximo’ ou ‘Anterior’, independentemente do nível hierárquico desses nós. Isso significa que o nó ‘Próximo’ de uma subseção pode muito bem ser o próximo capítulo. Nós ordenados sequencialmente são úteis para romances e outros documentos que você lê sequencialmente. (No entanto, no Info, o comando `g *` permite que você examine o arquivo sequencialmente, de modo que nós ordenados sequencialmente não são estritamente necessários). Com um argumento (argumento de prefixo, se interativo), o comando `texinfo-sequential-node-update` atualiza sequencialmente todos os nós na região.

D.6 Formatação para Info

O modo Texinfo fornece vários comandos para formatar parte ou tudo de um arquivo do Texinfo para Info.

D.6.1 Executando `texi2any/makeinfo` no Emacs

O programa `texi2any` fornece mensagens de erro melhores que qualquer um dos comandos de formatação do Emacs. Nós o recomendamos. O programa `texi2any` é independente do Emacs.

Você pode executar `texi2any` (ou `makeinfo`) no modo Texinfo do GNU Emacs usando ou o comando `makeinfo-region` ou o `makeinfo-buffer`. No modo Texinfo, os comandos estão vinculados a `C-c C-m C-r` e `C-c C-m C-b` por padrão.

C-c C-m C-r

M-x makeinfo-region

Formatar a região atual para Info.

C-c C-m C-b

M-x makeinfo-buffer

Formatar o buffer atual para Info.

Quando você invoca **makeinfo-region** a saída gerada vai para um buffer temporário. Quando você invoca **makeinfo-buffer**, a saída gerada vai para o arquivo configurado com **@setfilename** (veja-se Seção 2.5.2 [**@setfilename**], Página 12).

Os comandos **makeinfo-region** e **makeinfo-buffer** do Emacs executam o programa **texi2any** em um buffer temporário de shell. Se **texi2any** encontrar quaisquer erros, o Emacs exibe as mensagens de erro no buffer temporário.

Você pode analisar as mensagens de erro digitando **C-x ` (next-error)**. Isso faz com que o Emacs vá e posicione o cursor na linha no fonte do Texinfo que **texi2any** acha que causou o erro. Veja-se Seção “Executando **make** ou Compiladores Geralmente” em *O Manual do GNU Emacs*, para mais informações acerca de se usar o comando **next-error**.

Além disso, você consegue encerrar o shell no qual o comando **texi2any** estiver executando ou fazer o buffer do shell exibir a saída gerada mais recente dele.

C-c C-m C-k

M-x makeinfo-kill-job

Terminar a atual tarefa **texi2any** (ou **makeinfo**) executando (a partir de **makeinfo-region** ou **makeinfo-buffer**).

C-c C-m C-l

M-x makeinfo-recenter-output-buffer

Reexibir o buffer do shell do **texi2any** para exibir a saída gerada mais recente dele.

(Observe que os comandos paralelos para terminar e recentralizar uma tarefa do T_EX são **C-c C-t C-k** e **C-c C-t C-l**. Veja-se Seção D.7.1 [Impressão no Modo Texinfo], Página 263).

Você pode especificar opções para **texi2any** configurando a variável **makeinfo-options** ou com o comando **M-x customize** ou com o **M-x set-variable**, ou configurando a variável em teu arquivo de inicialização **.emacs**.

Por exemplo, você poderia escrever o seguinte em teu arquivo **.emacs**:

```
(setq makeinfo-options
  "--paragraph-indent=0 --no-split
  --fill-column=70 --verbose")
```

Para mais informações, veja-se [Opções do **texi2any**], Página 157, bem como “Interface de Fácil Personalização”, “Examinando e Configurando Variáveis” e “Arquivo de Iniciação” em *O Manual do GNU Emacs*.

D.6.2 Os comandos **texinfo-format...**

No GNU Emacs, no modo Texinfo, você consegue formatar parte, ou todo, de um arquivo do Texinfo com o comando **texinfo-format-region**. Isso formata a região atual e exibe o texto formatado em um buffer temporário chamado ***Info Region***.

Similarmente, você consegue formatar um buffer com o comando **texinfo-format-buffer**. Esse comando cria um novo buffer e gera o arquivo do Info nele. Digitar **C-x C-s** salvará o arquivo do Info sob o nome especificado pela linha **@setfilename**, que precisa estar próxima ao início do arquivo do Texinfo.

C-c C-e C-r

texinfo-format-region

Formatar a região atual para Info.

C-c C-e C-b

texinfo-format-buffer

Formatar o buffer atual para Info.

Os comandos `texinfo-format-region` e `texinfo-format-buffer` fornecem alguma verificação de erros, e outras funções podem fornecer ajuda adicional para encontrar erros de formatação. Esses procedimentos estão descritos em um apêndice; veja-se Seção D.10 [Capturando Erros], Página 269. No entanto, o programa `texi2any` fornece melhor verificação de erros (veja-se Seção D.6.1 [`texi2any` no Emacs], Página 261).

Uma peculiaridade dos comandos `texinfo-format-buffer` e `texinfo-format-region` é que eles não recuam (nem preenchem) parágrafos que contenham comandos `@w` ou `@*`.

D.7 Formatando e Imprimindo com o Emacs

GNU Emacs pode ser usado para formatação e impressão com $\text{\TeX}$, a partir de um Shell do Emacs. O modo Texinfo também fornece comandos de teclas predefinidos para formatação e impressão.

D.7.1 Formatação e Impressão no Modo Texinfo

O modo Texinfo fornece vários comandos de teclas predefinidos para formatação e impressão do $\text{\TeX}$. Esses incluem comandos para ordenar índices, examinar a fila da impressora, encerrar o trabalho de formatação e recentralizar a exibição do buffer no qual as operações ocorrem.

Frequentemente, quando estiver escrevendo um documento, você desejará tipografar e imprimir somente parte de um arquivo para ver como ele ficará. Você consegue usar o `texinfo-tex-region` e comandos relacionados para essa finalidade. Use o comando `texinfo-tex-buffer` para formatar todo um buffer.

Para que `texinfo-tex-region` ou `texinfo-tex-buffer` funcionem, o arquivo *precisa* começar com uma linha `\input texinfo` e precisa incluir uma linha `@settitle`. O arquivo precisa terminar com `@bye` sozinho em uma linha. (Ao usar `texinfo-tex-region`, você precisa envolver a linha `@settitle` com linhas de início de cabeçalho e de fim de cabeçalho).

C-c C-t C-b

M-x texinfo-tex-buffer

Executar `texi2dvi` no buffer. Além de executar $\text{\TeX}$ no buffer, esse comando automaticamente cria ou atualiza índices conforme necessário.

C-c C-t C-r

M-x texinfo-tex-region

Executar $\text{\TeX}$ na região atual.

Se comandos `@` relacionados à saída impressa estiverem entre as linhas de início de cabeçalho e de fim de cabeçalho, então `texinfo-tex-region` formatará a região de acordo. Por exemplo, se você escrever o comando `@smallbook` entre as linhas de início de cabeçalho e de fim de cabeçalho, `texinfo-tex-region` formatará a região no tamanho do livro “small”.

C-c C-t C-i

M-x texinfo-texindex

Executar `texindex` para ordenar os índices de um arquivo do Texinfo formatado com `texinfo-tex-region`. O comando `texinfo-tex-region` não executa `texindex` automaticamente; ele somente executa o comando de tipografia `tex`. Você precisa

executar o comando `texinfo-tex-region` uma segunda vez depois de ordenar os arquivos brutos de índice com o comando `texindex`. (Normalmente, você não formata um índice ao formatar uma região, somente quando formata um buffer. Agora que o comando `texi2dvi` existe, existe pouca ou nenhuma necessidade para esse comando).

C-c C-t C-p

M-x texinfo-tex-print

Imprimir um arquivo DVI que foi criado com `texinfo-tex-region` ou `texinfo-tex-buffer`.

C-c C-t C-q

M-x tex-show-print-queue

Mostrar a fila de impressão.

C-c C-t C-d

M-x texinfo-delete-from-print-queue

Excluir um trabalho da fila de impressão; você será solicitado(a) a fornecer o número do trabalho mostrado por um comando *C-c C-t C-q* anterior (`texinfo-show-tex-print-queue`).

C-c C-t C-k

M-x tex-kill-job

Eliminar o trabalho $\text{\TeX}$ atualmente executando iniciado ou por `texinfo-tex-region` ou por `texinfo-tex-buffer` ou qualquer outro processo executando no buffer do shell do Texinfo.

C-c C-t C-x

M-x texinfo-quit-job

Encerre um trabalho de formatação do $\text{\TeX}$ que foi interrompido devido a um erro enviando um `x` para ele. Quando você faz isso, o $\text{\TeX}$ preserva um registro do que fez em um arquivo `.log`.

C-c C-t C-l

M-x tex-recenter-output-buffer

Reexibir o buffer do shell no qual os comandos de impressão e de formatação do $\text{\TeX}$ são executados para mostrar a saída gerada mais recente dele.

Assim, a sequência usual de comandos para formatar um buffer é a seguinte (com comentários à direita):

C-c C-t C-b Executar `texi2dvi` no buffer.

C-c C-t C-p Imprimir o arquivo DVI.

C-c C-t C-q Exibir a fila da impressora.

Os comandos de formatação do $\text{\TeX}$ do modo Texinfo iniciam um subshell no Emacs chamado o `*tex-shell*`. Os comandos `texinfo-tex-command`, `texinfo-texindex-command` e `tex-dvi-print-command` são todos executados nesse shell.

Você consegue observar os comandos operarem no buffer `*tex-shell*` e consegue comutar, para e de, e usar o buffer `*tex-shell*` como faria com qualquer outro buffer de shell.

Os comandos de formatação e de impressão dependem dos valores de várias variáveis. Os valores padrão são:

Variável	Valor padrão
<code>texinfo-texi2dvi-command</code>	<code>"texi2dvi"</code>
<code>texinfo-tex-command</code>	<code>"tex"</code>
<code>texinfo-texindex-command</code>	<code>"texindex"</code>
<code>texinfo-delete-from-print-queue-command</code>	<code>"lprm"</code>
<code>texinfo-tex-trailer</code>	<code>"@bye"</code>
<code>tex-start-of-header</code>	<code>"%**start"</code>
<code>tex-end-of-header</code>	<code>"%**end"</code>
<code>tex-dvi-print-command</code>	<code>"lpr -d"</code>
<code>tex-show-queue-command</code>	<code>"lpq"</code>

Você pode mudar os valores dessas variáveis com o comando `M-x set-variable` (veja-se Seção “Examinando e Configurando Variáveis” em *O Manual do GNU Emacs*), ou com teu arquivo de inicialização `.emacs` (veja-se Seção “Arquivo de Inicialização” em *O Manual do GNU Emacs*).

A partir da versão 20, o GNU Emacs oferece uma interface amigável, chamada *Customize*, para mudança de valores de variáveis definíveis pelo(a) usuário(a). Veja-se Seção “Interface de Personalização Fácil” em *O Manual do GNU Emacs*, para mais detalhes acerca disso. As variáveis do Texinfo podem ser encontradas no grupo ‘Development/Docs/Texinfo’, depois de invocar o comando `M-x customize`.

D.7.2 Usando a Lista de Variáveis Locais

Outra maneira de aplicar o comando de formatação do $\text{\TeX}$ a um arquivo do Texinfo é a de colocar esse comando em uma *lista de variáveis locais* no final do arquivo do Texinfo. Você consegue então especificar os comandos `tex` ou `texi2dvi` como um `compile-command` e fazer com que o Emacs execute-o digitando `M-x compile`. Isso cria um shell especial chamado o buffer do `*compilation*` no qual o Emacs executa o comando de compilação. Por exemplo, no final do arquivo `gdb.texi`, depois do `@bye`, você poderia colocar o seguinte:

```
Variáveis locais:
compile-command: "texi2dvi gdb.texi"
Fim:
```

Essa técnica é mais frequentemente usada por programadores(as) que também compilam programas dessa maneira; veja-se Seção “Compilação” em *O Manual do GNU Emacs*.

D.8 Sumário do Modo Texinfo

No modo Texinfo, cada conjunto de comandos tem atalhos de teclado padrão que começam com as mesmas teclas. Todos os comandos que sejam personalizadamente criados para o modo Texinfo começam com `C-c`. As teclas são um tanto mnemônicas.

Comandos de Inserção

Os comandos de inserção são invocados digitando-se `C-c` duas vezes e depois a primeira letra do comando `@` a ser inserida. (Pode fazer mais sentido mnemonicamente usar `C-c C-i`, para ‘inserção personalizada’, mas `C-c C-c` é rápido de digitar).

<code>C-c C-c c</code>	Inserir ‘ <code>@code</code> ’.
<code>C-c C-c d</code>	Inserir ‘ <code>@dfn</code> ’.
<code>C-c C-c e</code>	Inserir ‘ <code>@end</code> ’.
<code>C-c C-c i</code>	Inserir ‘ <code>@item</code> ’.
<code>C-c C-c n</code>	Inserir ‘ <code>@node</code> ’.
<code>C-c C-c s</code>	Inserir ‘ <code>@samp</code> ’.

`C-c C-c v` Inserir '@var'.
`C-c {` Inserir chaves.
`C-c ]`
`C-c }` Mover para fora das chaves cercantes.

`C-c C-c C-d` Inserir título da seção de um nó
no espaço para a descrição
em uma linha de entrada do menu.

Mostrar Estrutura

O comando `texinfo-show-structure` frequentemente é usado em uma região estreitada.

`C-c C-s` Listar todos os títulos.

O Comando Mestre de Atualização

O comando `texinfo-master-menu` cria um menu mestre; e também pode ser usado para atualizar cada nó e menu em um arquivo.

`C-c C-u m`
`M-x texinfo-master-menu`
Criar ou atualizar um menu mestre.

`C-u C-c C-u m` Com `C-u` como um argumento de prefixo, primeiro criar ou atualizar todos os nós e menus regulares e, em seguida, criar um menu mestre.

Atualizar Ponteiros

Os comandos de atualização de ponteiro são invocados digitando-se `C-c C-u e`, então, ou `C-n` para `texinfo-update-node` ou `C-e` para `texinfo-every-node-update`.

`C-c C-u C-n` Atualizar um nó.
`C-c C-u C-e` Atualizar cada nó no buffer.

Menus de Atualização

Invocar os comandos de atualização de menu digitando `C-c C-u e`, então, ou `C-m` para `texinfo-make-menu` ou `C-a` para `texinfo-all-menus-update`. Para atualizar ambos, nós e menus, ao mesmo tempo, preceder `C-c C-u C-a` com `C-u`.

`C-c C-u C-m` Criar ou atualizar um menu.

`C-c C-u C-a` Criar ou atualizar todos os menus em um buffer.

`C-u C-c C-u C-a` Com `C-u` como um argumento de prefixo, primeiro criar ou atualizar todos os nós e, então, criar os atualizar todos os menus.

Formatar para Info

Os comandos de formatação do Info que sejam escritos em Emacs Lisp são invocados digitando-se `C-c C-e e`, então, ou `C-r` para uma região ou `C-b` para o buffer inteiro.

Os comandos de formatação do Info que sejam baseados no programa `texi2any/makeinfo` são invocados digitando-se `C-c C-m e`, então, ou `C-r` para uma região ou `C-b` para o buffer inteiro.

Usar os comandos `texinfo-format...`:

<code>C-c C-e C-r</code>	Formatar a região.
<code>C-c C-e C-b</code>	Formatar o buffer.

Usar `texi2any/makeinfo`:

<code>C-c C-m C-r</code>	Formatar a região.
<code>C-c C-m C-b</code>	Formatar o buffer.
<code>C-c C-m C-l</code>	Recentralizar o buffer da saída do <code>texi2any</code> .
<code>C-c C-m C-k</code>	Encerra o trabalho de formatação do <code>texi2any</code> .

Tipografia e Impressão

Os comandos de tipografia e de impressão do `TeX` são invocados digitando-se `C-c C-t` e, em seguida, outro comando de controle: `C-r` para `texinfo-tex-region`, `C-b` para `texinfo-tex-buffer` e assim por diante.

<code>C-c C-t C-r</code>	Executar <code>TeX</code> sobre a região.
<code>C-c C-t C-b</code>	Executar <code>texi2dvi</code> sobre o buffer.
<code>C-c C-t C-i</code>	Executar <code>texindex</code> .
<code>C-c C-t C-p</code>	Imprimir o arquivo DVI.
<code>C-c C-t C-q</code>	Mostrar a fila de impressão.
<code>C-c C-t C-d</code>	Deletar um trabalho da fila de impressão.
<code>C-c C-t C-k</code>	Elimina o trabalho atual de formatação do <code>TeX</code> .
<code>C-c C-t C-x</code>	Fecha um trabalho de formatação do <code>TeX</code> atualmente interrompido.
<code>C-c C-t C-l</code>	Recentralizar a saída gerada de buffer.

Outros Comandos de Atualização

Os comandos de atualização restantes não têm atalhos padrão de teclado porque raramente eles são usados.

`M-x texinfo-insert-node-lines`
 Inserir linhas `@node` ausentes na região.
 Com `C-u` como argumento de prefixo,
 usar títulos de seção como nomes de nós.

`M-x texinfo-multiple-files-update`
 Atualizar um documento com vários arquivos.
 Com `C-u 2` como um argumento de prefixo,
 criar ou atualizar todos os nós e menus
 em todos os arquivos incluídos primeiro.

`M-x texinfo-indent-menu-description`
 Recuar descrições.

`M-x texinfo-sequential-node-update`
 Inserir ponteiros de nó em sequência estrita.

D.9 Formatação Direta de arquivos do Info

Em geral, não existe necessidade de editar ou de processar arquivos do Info. A maior parte do conteúdo do arquivo do Info é texto simples, mas as tabelas de etiquetas exigem computar-se deslocamentos para nós e para arquivos indiretos (veja-se Seção 21.2 [Arquivos de Etiqueta e Divididos], Página 188), o que é impraticável. Os processadores do Texinfo produzem Info bem formatados a partir da entrada Texinfo. Além disso, muitas informações originárias do fonte

do Texinfo são perdidas no arquivo do Info. O fonte do Texinfo é portanto mais adequado para analisar um manual, seja para informe de erros, para estatísticas, ou para configurar traduções.

Ainda assim, especialmente com arquivos do Info escritos manualmente, existem algumas situações em que pode ser interessante processar arquivos do Info diretamente, em particular para computar deslocamentos de nós dos arquivos do Info e para computar deslocamentos de arquivo dos arquivos divididos do Info. Dois comandos do Emacs fazem isso. **Info-tagify** adiciona uma tabela de etiquetas para um arquivo não dividido carecendo de uma. Essa função também é útil para validação de arquivo do Info (veja-se Seção D.10.4 [Executando **Info-validate**], Página 272). Também é possível dividir um arquivo não dividido do Info com **Info-split**.

D.9.1 Etiquetando um Arquivo

Os processadores do Texinfo criam tabelas de etiquetas automaticamente. A única exceção surge quando **texinfo-format-buffer** é chamado com um prefixo, como **C-u M-x texinfo-format-buffer**, para criar um arquivo não dividido do Info sem uma tabela de etiquetas (a qual pode ser útil em alguns casos para validação do arquivo do Info, veja-se Seção D.10.4.2 [Desdividir e Etiquetar], Página 272). Adicionar uma tabela de etiquetas é, portanto, mais útil geralmente para arquivos do Info escritos manualmente.

Para criar uma tabela de etiquetas para um arquivo não dividido, visite o arquivo do Info que você deseja etiquetar e digite:

M-x Info-tagify

(Observe o ‘I’ maiúsculo no **Info-tagify**). Isso cria um arquivo do Info com uma tabela de etiquetas. Uma tabela de etiquetas faz parte de um bem formado arquivo do Info (veja-se Seção 21.2 [Arquivos de Etiqueta e Divididos], Página 188). Uma tabela de etiquetas também é necessária para validar ou para dividir o arquivo do Info.

D.9.2 Dividindo um Arquivo Manualmente

Você deveria dividir um arquivo grande ou então deixar o comando **texinfo-format-buffer** ou **makeinfo-buffer** fazer isso para você automaticamente. (Geralmente você deixará um dos comandos de formatação fazer esse trabalho para você. Veja-se Seção D.6 [Formatação do Info], Página 261).

Os arquivos divididos são chamados de subarquivos indiretos.

As tabelas de etiquetas são criadas automaticamente pelo comando de formatação; você só precisa criar uma tabela de etiquetas se estiver fazendo o trabalho manualmente. Veja-se Seção D.9.1 [Etiquetando], Página 268, para informações acerca da criação de uma tabela de etiquetas.

Visite o arquivo do Info que você deseja etiquetar e dividir manualmente e digite os dois comandos:

M-x Info-tagify

M-x Info-split

(Observe que o ‘I’ no ‘Info’ está maiúsculo).

Quando você usa o comando **Info-split**, o buffer é modificado em um (pequeno) arquivo do Info que lista os subarquivos indiretos. Esse arquivo deveria ser salvo no lugar do arquivo original visitado. Os subarquivos indiretos são escritos no mesmo diretório em que o arquivo original está, com nomes gerados pospondo-se ‘-’ e um número ao nome original do arquivo.

O arquivo primário ainda funciona como um arquivo do Info, mas ele contém apenas a tabela de etiquetas e um diretório de subarquivos. Veja-se Seção 21.2 [Arquivos de Etiqueta e Divididos], Página 188.

D.10 Capturando Erros

Além de erros no conteúdo da tua documentação, existem dois tipos de erros que você pode cometer com o Texinfo: você pode cometer erros com comandos @ e pode cometer erros com a estrutura dos nós e dos capítulos.

Para encontrar problemas com comandos @, você pode executar `TEX` ou um comando de formatação de região sobre a região que tenha um problema; na verdade, você pode executar esses comandos sobre cada região enquanto escreve.

Para encontrar problemas com a estrutura de nós e de capítulos, você consegue usar `C-c C-s` (`texinfo-show-structure`) e o comando `occur` relacionado e você consegue usar o comando `M-x Info-validate`.

D.10.1 texi2any Preferido

O programa `texi2any` faz um excelente trabalho de capturar erros e informá-los—muito melhor que `texinfo-format-region` ou `texinfo-format-buffer`. Além disso, as diversas funções para automaticamente criar e atualizar ponteiros de nós e menus removem muitas oportunidades para erro humano.

Use `texi2any` (ou as manifestações dele no modo Texinfo, `makeinfo-region` e `makeinfo-buffer`) para formatar teu arquivo e verificar se existem outros erros. Essa é a melhor maneira de trabalhar com o Texinfo. Mas se não puder usar `texi2any`, ou se teu problema for muito intrigante, então você pode querer usar as ferramentas descritas nesta seção.

D.10.2 Capturando Erros com formatação do Info

Depois de ter escrito parte de um arquivo do Texinfo, você consegue usar o comando `texinfo-format-region` ou o `makeinfo-region` para ver se a região formata corretamente.

Provavelmente, entretanto, você está lendo esta seção porque, por algum motivo, não pode usar o comando `makeinfo-region`; portanto, o restante desta seção presume que você está usando `texinfo-format-region`.

Se você tiver cometido um erro com um comando @, `texinfo-format-region` parará o processamento no momento ou depois do erro e exibirá uma mensagem de erro. Para ver onde no buffer ocorreu o erro, comute para o buffer `*Info Region*`; o cursor estará em uma posição que é posterior ao local do erro. Além disso, o texto não será formatado após o local onde o erro ocorreu (ou mais precisamente, onde ele foi detectado).

Por exemplo, se acidentalmente encerrar um menu com o comando `@end menus` com um ‘s’ no final, em vez de `@end menu`, você verá uma mensagem de erro que diz:

```
@end menus não é manuseado pelo texinfo
```

O cursor parará no ponto no buffer onde o erro ocorrer, ou não muito depois dele. O buffer se parecerá com isto:

```
----- Buffer: *Info Region* -----
* Menu:

* Usando texinfo-show-structure:: Como usar
                                `texinfo-show-structure'
                                para capturar erros.
* Executando Info-validate::    Como verificar se existem
                                nós não referenciados.

@end menus
*
----- Buffer: *Info Region* -----
```

O comando `texinfo-format-region` às vezes fornece mensagens de erro um pouco estranhas. Por exemplo, a seguinte referência cruzada falha para formatar:

```
(@xref{Capturando Erros, para mais informações}).
```

Neste caso, `texinfo-format-region` detecta a chave de fechamento ausente, mas exibe uma mensagem que diz ‘Parênteses não balanceados’ em vez de ‘Chaves não balanceadas’. Isso ocorre porque o comando de formatação procura desconformidades entre chaves como se elas fossem parênteses.

Às vezes, `texinfo-format-region` falha para detectar erros. Por exemplo, a seguir, a chave de fechamento é trocada pelo parêntese de fechamento:

```
(@xref{Capturando Erros), para mais informações.}
```

A formatação produz:

```
(*Observação para mais informações: Capturando Erros)
```

A única maneira para você detectar esse erro é a de perceber que a referência deveria ter se parecido com isto:

```
(*Observe Capturando Erros::, para mais informações).
```

Aliás, se estiver lendo este nó no Info e digitar `f RET` (`Info-follow-reference`), você gerará uma mensagem de erro que diz:

```
Esse nó não existe: "Capturando Erros) A única maneira ...
```

Isso ocorre porque o Info percebe o exemplo do erro como a primeira referência cruzada neste nó e se você digitasse um `RET` imediatamente depois de digitar o comando `f` do Info, o Info tentará ir para o nó referenciado. Se você digitasse `f catch TAB RET`, o Info completará o nome do nó do exemplo escrito corretamente e te levará ao nó ‘Capturando Erros’. (Se você tentar isso, poderá retornar do nó ‘Capturando Erros’ digitando `l` (`Info-last`)).

D.10.3 Depuração com T_EX

Você também pode capturar erros ao formatar um arquivo com T_EX.

Normalmente, você vai querer fazer isso depois de ter executado `texinfo-format-buffer` (ou, melhor, `makeinfo-buffer`) sobre o mesmo arquivo, porque `texinfo-format-buffer` às vezes exibe mensagens de erro que fazem mais sentido que o T_EX. (Veja-se Seção D.10.2 [Depuração com Info], Página 269, para mais informações).

Por exemplo, T_EX foi executado sobre um arquivo do Texinfo, parte do qual é mostrado aqui:

```
----- Buffer: texinfo.texi -----
nome do arquivo Texinfo como extensão. Os
@samp{??} são `curingas' que fazem com que o shell
substitua todos os arquivos brutos de índice. (@xref{ordenando
índices, para mais informações acerca da ordenação de
índices}).
----- Buffer: texinfo.texi -----
```

(A referência cruzada carece de uma chave de fechamento). T_EX produziu a seguinte saída, depois da qual parou:

```
----- Buffer: *tex-shell* -----
Argumento fugitivo?
{ordenando índices, para mais informações acerca de
ordenação de índices}. @ETC.
! O parágrafo terminou antes que @xref estivesse completa.
<para ser lido novamente>
@par
```

```
?
----- Buffer: *tex-shell* -----
```

Nesse caso, T_EX produziu uma mensagem de erro precisa e compreensível:

O parágrafo terminou antes que @xref estivesse completa.

‘@par’ é um comando interno do T_EX sem relevância para o Texinfo. ‘1.27’ significa que T_EX detectou o problema na linha 27 do arquivo do Texinfo. O ‘?’ é o prompt que o T_EX usa nessa circunstância.

Infelizmente, o T_EX nem sempre é tão útil e, às vezes, você precisa realmente ser um Sherlock Holmes para descobrir o que deu errado.

De qualquer forma, se você se deparar com um problema como esse, poderá fazer uma de três coisas.

1. Você pode dizer ao T_EX para continuar executando e ignorar apenas esse erro digitando RET no prompt ‘?’.
2. Você pode dizer ao T_EX para continuar executando e para ignorar todos os erros da melhor maneira possível digitando **r** RET no prompt ‘?’.

Essa frequentemente é a melhor coisa a se fazer. No entanto, tenha cuidado: um erro possivelmente produza uma cascata de mensagens adicionais de erro, à medida que as consequências dele forem sentidas ao longo do restante do arquivo. Para parar o T_EX quando ele estiver produzindo tal avalanche de mensagens de erro, digite **C-c** (ou **C-c C-c**, se você estiver executando um shell dentro do Emacs).

3. Você pode dizer ao T_EX para interromper essa execução digitando **x** RET no prompt ‘?’.

Se estiver executando o T_EX dentro do Emacs, você precisará comutar para o buffer do shell e a linha na qual o T_EX oferece o prompt ‘?’.

Às vezes, o T_EX formatará um arquivo sem produzir mensagens de erro, mesmo que exista um problema. Isso geralmente ocorre se um comando não for finalizado, mas o T_EX puder, de qualquer maneira, continuar o processamento. Por exemplo, se você falhar para finalizar uma lista detalhada com o comando @end itemize, o T_EX escreverá um arquivo DVI que você conseguirá imprimir. A única mensagem de erro que o T_EX lhe dará será o comentário um tanto misterioso:

```
(@end ocorreu dentro de um grupo no nível 1)
```

No entanto, se você imprimir o arquivo DVI, descobrirá que o texto do arquivo que segue a lista detalhada estará inteiramente recuado como se fosse parte do último item da lista detalhada. A mensagem de erro é a maneira como o T_EX diz que esperava encontrar um comando @end em algum lugar no arquivo; mas não conseguiu determinar onde era necessário.

Outra fonte de erros notoriamente difíceis de encontrar é um comando @end group ausente. Se você ficar perplexo(a) com erros incompreensíveis, procure primeiro um comando @end group ausente.

Se o arquivo do Texinfo carecer de linhas de cabeçalho, T_EX poderá parar no início da execução dele e exibir uma saída que se parece com a seguinte. O ‘*’ indica que T_EX está aguardando entrada.

```
Este é o TeX, versão 3.14159 (Web2c 7.0)
(test.texinfo [1])
*
```

Nesse caso, simplesmente digite **\end** RET depois do asterisco. Em seguida, escreva as linhas de cabeçalho no arquivo do Texinfo e execute o comando T_EX novamente. (Observe o uso da barra invertida, ‘\’. T_EX usa ‘\’ em vez de ‘@’; e nessa circunstância, você está trabalhando diretamente com T_EX, não com Texinfo).

D.10.4 Encontrando Nós Mal Referenciados

Você consegue usar o comando **Info-validate** para verificar se quaisquer de ‘Próximo’, ‘Anterior’, ‘Acima’ ou outros ponteiros de nó falham ao apontar para um nó. Esse comando verifica se cada ponteiro de nó aponta para um nó existente. O comando **Info-validate** funciona somente sobre arquivos do Info, não sobre arquivos do Texinfo.

O programa **texi2any** valida ponteiros automaticamente, de modo que você não precisa usar o comando **Info-validate** se estiver usando **texi2any**. Com a variável de personalização **CHECK_NORMAL_MENU_STRUCTURE** configurada, **texi2any** também avisará se os ponteiros dos nós (ou configurados explicita ou automaticamente) não estiverem consistentes com a ordem das entradas do menu dos nós. **texi2any** não verifica se cada ponteiro ‘Próximo’ está correspondido por um ‘Anterior’ (no nó para onde o ‘Próximo’ aponta) que aponte para trás, pois possivelmente seja correto para uma estrutura de documento não padrão.

Você possivelmente necessite usar **Info-validate** somente se estiver inapto(a) para executar **texi2any** e, em vez disso, precise criar um arquivo do Info usando **texinfo-format-region** ou **texinfo-format-buffer**, ou se escrever um arquivo do Info a partir do zero.

D.10.4.1 Usando Info-validate

Para usar **Info-validate**, visite o arquivo do Info que você deseja verificar e digite:

```
M-x Info-validate
```

Observe que o comando **Info-validate** exige um ‘I’ maiúsculo. Você possivelmente necessite também criar uma tabela de etiquetas antes de executar **Info-validate**. Veja-se Seção D.9.1 [Etiquetando], Página 268.

Se teu arquivo for válido, você receberá uma mensagem que diz “Arquivo parece válido”. No entanto, se você tiver um ponteiro que não apontar para um nó, mensagens de erro serão exibidas em um buffer chamado ***problems in info file***.

Por exemplo, **Info-validate** foi executado sobre um arquivo de teste que continha somente o primeiro nó deste manual. Uma das mensagens dizia:

```
No nó "Visão Geral", inválido Próximo: Modo Texinfo
```

Isso significava que o nó chamado ‘Visão Geral’ tinha um ponteiro ‘Próximo’ que não apontava para nada (o que era verdade nesse caso, já que o arquivo de teste tinha somente um nó nele).

Agora suponha que nós adicionássemos um nó chamado ‘Modo Texinfo’ ao nosso caso de teste, mas não especificássemos um ‘Anterior’ para esse nó. Então obteríamos a seguinte mensagem de erro:

```
No nó "Modo Texinfo", deveria ter Anterior: Visão Geral
```

Isso ocorre porque, com uma estrutura padrão de documento, cada ponteiro ‘Próximo’ deveria ser correspondido por um ‘Anterior’ (no nó para onde o ‘Próximo’ aponta) que aponte para trás.

Info-validate também verifica se todas as entradas de menu e referências cruzadas apontam para nós reais.

Info-validate exige uma tabela de etiquetas e não funciona com arquivos que tiverem sido divididos. (O comando **texinfo-format-buffer** divide automaticamente arquivos grandes). Para a finalidade de usar **Info-validate** sobre um arquivo grande, você precisa executar **texinfo-format-buffer** com um argumento, de modo que ele não divida o arquivo do Info; e você precisa criar uma tabela de etiquetas para o arquivo não dividido.

D.10.4.2 Criando um Arquivo Desdividido e Adicionando uma Tabela de Etiquetas

Você consegue executar **Info-validate** somente sobre um arquivo do Info que tenha uma tabela de etiquetas. O comando não funcionará sobre os subarquivos indiretos que forem gerados

quando um arquivo mestre for dividido. Se tiver um arquivo grande (mais que 300.000 bytes ou mais), você necessita executar o comando `texinfo-format-buffer`, ou o `makeinfo-buffer`, de tal maneira que ele não crie subarquivos indiretos. Você também precisará criar uma tabela de etiquetas para o arquivo do Info. Depois que tiver feito isso, você pode executar `Info-validate` e procurar nós mal referenciados.

A primeira etapa é a de criar um arquivo do Info não dividido. Para evitar que `texinfo-format-buffer` divida um arquivo do Texinfo em arquivos do Info menores, forneça um prefixo para o comando `M-x texinfo-format-buffer`:

```
C-u M-x texinfo-format-buffer
```

ou então

```
C-u C-c C-e C-b
```

Quando você fizer isso, `texinfo-format-buffer` não dividirá o arquivo e não criará uma tabela de etiquetas para ele.

Depois de criar um arquivo do Info não dividido, você precisa criar uma tabela de etiquetas para ele (veja-se Seção D.9.1 [Etiquetando], Página 268). Visite o arquivo do Info não dividido criado pelo `texinfo-format-buffer` e digite:

```
M-x Info-tagify
```

A terceira etapa é a de validar o arquivo do Info:

```
M-x Info-validate
```

(Observe o ‘I’ maiúsculo no `Info-validate`). Resumidamente, as etapas são:

```
C-u M-x texinfo-format-buffer
```

```
M-x Info-tagify
```

```
M-x Info-validate
```

Depois que tiver validado a estrutura do nó, você pode reexecutar `texinfo-format-buffer` da maneira normal, de modo que ele construa uma tabela de etiquetas e divida o arquivo automaticamente, ou você pode criar a tabela de etiquetas e dividir o arquivo manualmente (veja-se Seção D.9.2 [Dividindo], Página 268).

D.11 Formatação em Lote

Você pode formatar arquivos do Texinfo para Info usando `batch-texinfo-format` e modo de lote do Emacs. Você pode executar o Emacs em modo de lote a partir de qualquer shell, incluindo um shell dentro do Emacs. (Veja-se Seção “Opções Iniciais” em *O Manual do GNU Emacs*).

Aqui está um comando de shell para formatar todos os arquivos que terminem em `.texinfo` no diretório atual:

```
emacs -batch -funcall batch-texinfo-format *.texinfo
```

O Emacs processa todos os arquivos listados na linha de comando, mesmo se um erro ocorrer enquanto tenta formatar alguns deles.

Execute `batch-texinfo-format` somente com Emacs em modo de lote conforme mostrado; ele não é interativo. Ele mata o modo de lote do Emacs após a conclusão.

`batch-texinfo-format` é conveniente se você carecer do `texi2any` e quiser formatar vários arquivos do Texinfo de uma vez. Ao usar o modo de Lote, você cria um novo processo do Emacs. Isso libera teu Emacs atual, de modo que você consiga continuar trabalhando nele. (Quando executar `texinfo-format-region`, ou `texinfo-format-buffer`, você não consegue usar esse Emacs para mais nada até que o comando termine).

Anexo E Comandos Globais de Documento

Aqui estão comandos adicionais que afetam o documento como um todo. A maioria desses comandos serve para personalizar a aparência da saída gerada impressa. Geralmente, todos são fornecidos antes do nó Top, se forem fornecidos.

E.1 @setchapternewpage: Páginas em Branco Antes dos Capítulos

Em um livro oficialmente encadernado, o texto geralmente é impresso em ambos os lados do papel, os capítulos começam nas páginas da direita e as páginas da direita têm números ímpares. Mas em informes curtos, frequentemente o texto é impresso somente em um lado do papel. Também em informes curtos, os capítulos às vezes não começam em novas páginas, mas são impressos na mesma página do final do capítulo anterior, depois de uma pequena quantidade de espaço em branco vertical.

Você consegue usar o comando `@setchapternewpage` com vários argumentos para especificar como os capítulos deveriam ser iniciados na saída impressa e se os cabeçalhos deveriam ser formatados para impressão em um ou ambos os lados do papel (impressão em um lado ou frente e verso).

Escreva o comando `@setchapternewpage` no início de uma linha seguido pelo argumento dele. Por exemplo, você escreveria o seguinte para fazer com que cada capítulo começasse em uma nova página ímpar:

```
@setchapternewpage odd
```

Você consegue especificar uma das três alternativas com o comando `@setchapternewpage`:

`@setchapternewpage off`

Tipografar um novo capítulo na mesma página do último capítulo, depois de pular alguns espaços em branco verticais. Além disso, formatar os cabeçalhos das páginas para impressão em um lado.

`@setchapternewpage on`

Iniciar novos capítulos em novas páginas e formatar cabeçalhos de páginas para impressão em um lado. Essa é a forma mais frequentemente usada para informes curtos ou para impressão pessoal. Esse é o padrão.

`@setchapternewpage odd`

Iniciar novos capítulos em novas páginas ímpares (páginas destros) e tipografadas para impressão frente e verso. Essa é a forma mais frequentemente usada para livros e manuais.

Texinfo não tem um comando `@setchapternewpage even`, porque não existe tradição de impressão de iniciar capítulos ou livros em uma página par.

Se não gostar dos cabeçalhos padrão que `@setchapternewpage` configurar, você consegue controlá-los explicitamente com o comando `@headings`. Veja-se Seção E.2.1 [`@headings`], Página 275.

No início de um manual ou livro, as páginas não são numeradas—por exemplo, as páginas de título e de copirraite de um livro não são numeradas. Por convenção, páginas de sumário e iniciais são numeradas com algarismos romanos e não em sequência com o resto do documento.

O `@setchapternewpage` não tem efeito em formatos de saída que não tenham páginas, como Info e HTML.

Nós recomendamos não incluir nenhum comando `@setchapternewpage` no fonte do teu documento, pois tal paginação desejada não é intrínseca ao documento. Para uma execução específica de cópia impressa, se você não quiser a saída padrão (sem páginas em branco, mesmos cabeçalhos em todas as páginas), use a opção `--texinfo` para `texi2dvi` para especificar a saída que você quer.

E.2 Cabeçalhos de Página

A maioria dos manuais impressos contém cabeçalhos ao longo do topo de cada página, exceto nas páginas de título e de copirraite. Alguns manuais também contêm rodapés. Cabeçalhos e rodapés não têm significado no Info ou em outros formatos de saída.

O Texinfo fornece dois formatos padrão de cabeçalho, um para manuais impressos em um lado de cada folha de papel e outro para manuais impressos em ambos os lados do papel. Por padrão, nada é especificado para o rodapé de um arquivo do Texinfo, de modo que o rodapé permanece em branco.

O Texinfo também tem vários comandos de cabeçalho e de rodapé que você consegue usar para gerar teus próprios formatos de cabeçalho e de rodapé.

No Texinfo, cabeçalhos e rodapés são linhas únicas no topo e no final das páginas; você não consegue criar cabeçalhos ou rodapés de múltiplas linhas. Cada linha de cabeçalho ou de rodapé é dividida em três partes: uma parte esquerda, uma parte central e uma parte direita. Qualquer parte, ou uma linha inteira, pode ser deixada em branco. O texto da parte esquerda de uma linha de cabeçalho ou de rodapé é configurado alinhado à esquerda; o texto da parte central fica centralizado; e o texto da parte direita é configurado alinhado à direita.

E.2.1 O Comando @headings

O comando `@headings` raramente é usado. Ele especifica quais tipos de cabeçalhos e rodapés de página imprimir em cada página. Normalmente, isso é controlado pelo comando `@setchapternewpage`. Você precisará do comando `@headings` somente se o comando `@setchapternewpage` não fizer o que você quiser.

Você consegue usar `@headings` conforme segue:

`@headings off`

Desativar impressão dos cabeçalhos das páginas.

`@headings single`

Ativar cabeçalhos de página apropriados para impressão em um lado.

`@headings double`

Ativar cabeçalhos de página apropriados para impressão frente e verso.

`@headings singleafter`

`@headings doubleafter`

Ativar cabeçalhos `single` ou `double`, respectivamente, depois que a página atual for gerada.

`@headings on`

Ativar os cabeçalhos das páginas: `single` se '`@setchapternewpage on`', `double` caso contrário.

Por exemplo, suponha que você escreva `@setchapternewpage off` antes do comando `@titlepage` para iniciar um novo capítulo na mesma página do final do último capítulo. Esse comando também faz com que os cabeçalhos das páginas sejam tipografados para impressão em um lado. Para fazer com que os cabeçalhos das páginas sejam tipografados para impressão frente e verso, escreva `@headings double`.

Você consegue impedir que quaisquer cabeçalhos de página sejam gerados escrevendo `@headings off` em uma linha própria, como esta:

```
@headings off
```

E.2.2 Formatos Padrão de Cabeçalho

O Texinfo fornece dois formatos padrão de cabeçalho, um para manuais impressos em um lado de cada folha de papel e outro para manuais impressos em ambos os lados do papel.

O formato padrão para impressão em um lado consiste de uma linha de cabeçalho na qual a parte esquerda contém o nome do capítulo, a parte central fica em branco e a parte direita contém o número da página. Uma página de um lado se parece com isto:

```

-----
| capítulo      número da página |
|                               |
| Início do texto ...          |
| ...                               |
|                               |

```

No formato frente e verso padrão, a parte esquerda da página esquerda (número par) contém o número da página, a parte central fica em branco e a parte direita contém o título (especificado pelo comando `@settitle`). A parte esquerda da página direita (número ímpar) contém o nome do capítulo, a parte central fica em branco e a parte direita contém o número da página. Uma página par e uma página ímpar, lado a lado como em um livro aberto, se parecem com isto:

```

-----
| número da página      título | | número da página      capítulo |
|                               | |                               |
| Início do texto...        | | Mais texto...          |
| ...                               | | ...                     |
|                               | |                               |

```

O nome do capítulo é precedido pela palavra “Capítulo”, o número do capítulo e dois pontos. Isso torna mais fácil acompanhar onde você está no manual.

Observe que nas páginas onde um novo capítulo começa, algum texto pode ser omitido da linha do cabeçalho.

Por padrão, nada é especificado para o rodapé de um arquivo do Texinfo, de modo que o rodapé permanece em branco.

E.2.3 Como Fazer Teus Próprios Cabeçalhos

Você pode usar os cabeçalhos padrão fornecidos com o Texinfo ou especificar os teus próprios. Por padrão, o Texinfo não tem rodapés, de modo que se você especificá-los, o tamanho de página disponível para o texto principal será ligeiramente reduzido.

Texinfo fornece seis comandos para se especificar cabeçalhos e rodapés:

- `@everyheading` e `@everyfooting` geram cabeçalhos e rodapés de página que são os mesmos para ambas, páginas pares e ímpares.
- Os comandos `@evenheading` e `@evenfooting` geram cabeçalhos e rodapés para páginas pares (à esquerda).
- `@oddheading` e `@oddfooting` geram cabeçalhos e rodapés para páginas ímpares (à direita).

Você precisa cancelar os comandos predefinidos de cabeçalho com o comando `@headings off` antes de definir tuas próprias especificações.

Aqui está como colocar o nome do capítulo à esquerda, o número da página no centro e a data à direita de cada cabeçalho para ambas, páginas pares e ímpares:

```
@headings off
```

```
@everyheading @thischapter @| @thispage @| @today{}
```

Você precisa dividir a parte esquerda originária da parte central e a parte central originária da parte direita inserindo ‘@|’ entre as partes. Caso contrário, o comando de especificação não será capaz de dizer onde o texto para uma parte termina e a próxima parte começa.

Cada parte pode conter texto ou comandos @. O texto é impresso como se a parte estivesse dentro de um parágrafo comum no corpo da página. Os comandos @ substituem-se pelo número da página, data, nome do capítulo ou qualquer outra coisa.

Aqui estão os seis comandos de cabeçalho e de rodapé:

```
@everyheading esquerda @| centro @| direita
```

```
@everyfooting esquerda @| centro @| direita
```

Os comandos ‘every’ especificam o formato para ambas, páginas pares e ímpares. Esses comandos são para documentos que forem impressos em um lado de cada folha de papel ou para documentos nos quais você queira cabeçalhos ou rodapés simétricos.

```
@evenheading esquerda @| centro @| direita
```

```
@oddheading esquerda @| centro @| direita
```

```
@evenfooting esquerda @| centro @| direita
```

```
@oddfooting esquerda @| centro @| direita
```

Os comandos ‘par’ e ‘ímpar’ especificam o formato para páginas pares e páginas ímpares. Esses comandos são para livros e manuais que forem impressos nos dois lados de cada folha de papel.

Use a série ‘@this...’ dos comandos @ para fornecer os nomes dos capítulos e seções e o número da página. Você pode usar os comandos ‘@this...’ nas partes esquerda, central ou direita dos cabeçalhos e rodapés.

Aqui estão os comandos ‘@this...’:

```
@thispage
```

Expande para o número da página atual.

```
@thissection
```

Expande para o número e nome da seção atual, no formato ‘Seção 1.1: Título’.

```
@thissectionname
```

Expande para o nome da seção atual.

```
@thissectionnum
```

Expande para o número da seção atual, no formato ‘1.1’.

```
@thischapter
```

Expande para o número e nome do capítulo atual, no formato ‘Capítulo 1: Título’.

```
@thischaptername
```

Expande para o nome do capítulo atual.

```
@thischapternum
```

Expande para o número do capítulo atual ou letra do apêndice atual.

```
@thispart
```

Expande para o nome da parte atual, conforme especificado pelo último comando @part.

```
@thistitle
```

Expande para o nome do documento, conforme especificado pelo comando @settitle.

@thisfile

Somente para arquivos **@include**: expande para o nome do arquivo **@include** atual. Se o atual arquivo fonte do Texinfo não for um arquivo **@include**, esse comando não terá efeito. Esse comando *não* fornece o nome atual do arquivo fonte do Texinfo, a menos que seja um arquivo **@include**. (Veja-se Capítulo 18 [Arquivos de Inclusão], Página 148, para mais informações acerca de arquivos **@include**).

Você também pode usar o comando **@today{}**, o qual expande para a data atual, no formato ‘1 de janeiro de 1900’.

Outros comandos **@** e texto são impressos em um cabeçalho ou rodapé como se estivessem no corpo de uma página. É útil incorporar texto, especialmente quando você estiver escrevendo rascunhos:

```
@headings off
@everyheading @emph{Rascunho!} @| @thispage @| @thischapter
@everyfooting @| @| Versão: 0.27: @today{}
```

Cuidado com títulos muito longos: eles podem se sobrepor a outra parte do cabeçalho ou do rodapé e apagá-la.

Se você tiver capítulos e(ou) seções muito curtos, vários deles poderão aparecer em uma página. Você pode especificar quais capítulos e seções quer que **@thischapter**, **@thissection** e outras macros se refiram nessas páginas, como segue:

@everyheadingmarks *referência*

@everyfootingmarks *referência*

O argumento *referência* pode ser ou **top** (os comandos **@this...** se referirão ao capítulo/seção no topo de uma página) ou **bottom** (os comandos refletirão a situação na parte inferior de uma página). Esses comandos ‘**@every...**’ especificam o que fazer em ambas, páginas pares e ímpares.

@evenheadingmarks *referência*

@oddheadingmarks *referência*

@evenfootingmarks *referência*

@oddfootingmarks *referência*

Esses comandos ‘**@even...**’ e ‘**@odd...**’ especificam o que fazer somente em páginas pares ou ímpares, respectivamente. O argumento *referência* é o mesmo que com os comandos ‘**@every...**’.

Escreva esses comandos imediatamente depois dos comandos **@...contents** ou depois do comando **@end titlepage** se você não tiver um sumário ou se ele estiver impresso no final do teu manual. Esses comandos não têm efeito no L^AT_EX.

Por padrão, para T_EX, os comandos **@this...** refletem a situação na parte inferior de uma página tanto nos cabeçalhos quanto nos rodapés.

E.3 @paragraphindent: Controlando Recuo do Parágrafo

Os processadores do Texinfo podem inserir espaços em branco no início da primeira linha de cada parágrafo, recuando assim esse parágrafo. Você pode usar o comando **@paragraphindent** para especificar esse recuo. Escreva um comando **@paragraphindent** no início de uma linha seguido ou por ‘**asis**’ ou por um número:

```
@paragraphindent recuo
```

O recuo fica de acordo com o valor de *recuo*:

asis Não mude o recuo existente (não implementado na saída impressa).

none

0 Omita todo o recuo.

n Recuar por *n* caracteres de espaço na saída Info, por *n* ems na saída impressa.

O valor padrão de *reco* é 3. `@paragraphindent` é ignorado para saída HTML.

É melhor escrever o comando `@paragraphindent` antes da linha de fim do cabeçalho no início de um arquivo do Texinfo, de modo que os comandos de formatação de região recuem parágrafos conforme especificado. Veja-se [Início do Cabeçalho], Página 14.

E.4 `@firstparagraphindent`: Recuo Depois de Cabeçalhos

Como você pode ver neste manual, o primeiro parágrafo de qualquer seção não é recuado por padrão. Tipograficamente, o recuo é um separador de parágrafo, o que significa que é desnecessário quando uma nova seção começa. Esse recuo é controlado com o comando `@firstparagraphindent`:

`@firstparagraphindent palavra`

O primeiro parágrafo depois de um cabeçalho é recuado de acordo com o valor de *palavra*:

none Impede que o primeiro parágrafo seja recuado (padrão). Essa opção é ignorada pelo `makeinfo` se `@paragraphindent asis` estiver em vigor.

insert Inclui recuo normal do parágrafo. Isso respeita o recuo do parágrafo configurado por um comando `@paragraphindent` (veja-se Seção E.3 [`@paragraphindent`], Página 278).

`@firstparagraphindent` é ignorado para saída HTML e DocBook.

É melhor escrever o comando `@firstparagraphindent` antes da linha de final do cabeçalho no início de um arquivo do Texinfo, de modo que os comandos de formatação de região recuem parágrafos conforme especificado. Veja-se [Início do Cabeçalho], Página 14.

E.5 `@exampleindent`: Recuo do Ambiente

Os processadores do Texinfo recuam cada linha de `@example` e ambientes semelhantes. Você pode usar o comando `@exampleindent` para especificar esse recuo. Escreva um comando `@exampleindent` no início de uma linha seguido ou por ‘*asis*’ ou por um número:

`@exampleindent reco`

O recuo fica de acordo com o valor de *reco*:

asis Não mude o recuo existente (não implementado na saída impressa).

0 Omita todo o recuo.

n Recuar ambientes por *n* caracteres de espaço na saída Info, por *n* ems na saída impressa.

O valor padrão de *reco* é 5 espaços no Info e 0,4in na saída impressa, o que é um pouco menos. (A redução é para ajudar a encaixar mais caracteres nas linhas físicas nos manuais impressos).

É melhor escrever o comando `@exampleindent` antes da linha de fim do cabeçalho no início de um arquivo do Texinfo, de modo que os comandos de formatação de região recuem parágrafos conforme especificado. Veja-se [Início do Cabeçalho], Página 14.

E.6 @smallbook: Imprimindo Livros “Pequenos”

Por padrão, o T_EX tipografa páginas para impressão em um formato de 8,5 por 11 polegadas, que é o tamanho “carta” comumente usado nos Estados Unidos da América do Norte. No entanto, você pode direcionar T_EX ou L^AT_EX para tipografar um documento em um formato de 7 por 9,25 polegadas que seja adequado para livros encadernados, inserindo o seguinte comando em uma linha isolada no início do arquivo do Texinfo, antes da página de título:

```
@smallbook
```

(Como muitos livros têm cerca de 7 por 9,25 polegadas, esse comando poderia ter sido melhor chamado de comando @regularbooksize, mas passou a ser chamado de comando @smallbook por comparação com o formato de 8,5 por 11 polegadas).

Veja-se Seção 19.2 [Formatar com texi2dvi ou texi2pdf], Página 150, e Seção 19.4 [Preparando para T_EX], Página 154, para outras maneiras de formatar com @smallbook com T_EX que não exigem mudar o arquivo fonte.

E.7 Imprimir em Papel A4

Você pode formatar um documento para impressão em papel tamanho ISO 216 A4 com o comando @afourpaper. Escreva o comando em uma linha própria próxima ao início do arquivo do Texinfo, antes da página de título.

Você pode ou não preferir a formatação resultante do comando @afourlatex. Existe também @afourwide para papel A4 em formato largo, @afivepaper para papel A5 e @sixpaper para papel B6.

Veja-se Seção 19.2 [Formatar com texi2dvi ou texi2pdf], Página 150, e Seção 19.4 [Preparando para T_EX], Página 154, para outras maneiras de formatar para diferentes tamanhos de papel que não exigem mudar o arquivo fonte.

E.8 @pagesizes [largura][, altura]: Tamanhos Personalizados de Página

Você pode especificar explicitamente a altura e (opcionalmente) a largura da área principal de texto na página com o comando @pagesizes. Escreva isso em uma linha própria perto do início do arquivo do Texinfo, antes da página de título. A altura vem primeiro, depois a largura, se desejado, separada por vírgula. Exemplos:

```
@pagesizes 200mm,150mm
```

e

```
@pagesizes 11.5in
```

Isso seria razoável para impressão em papel tamanho B5. Para enfatizar, esse comando especifica o tamanho da *área de texto*, não o tamanho do papel (que é 250 mm por 177 mm para B5, 14 in por 8,5 in para ofício).

Para fazer mudanças mais elaboradas, como mudar quaisquer das margens da página, você precisa definir um novo comando no texinfo.tex ou no texinfo.cnf.

Veja-se Seção 19.2 [Formatar com texi2dvi ou texi2pdf], Página 150, e Seção 19.4 [Preparando para T_EX], Página 154, para outras maneiras de especificar @pagesizes que não exigem mudança do arquivo fonte.

E.9 Microtipografia

Algumas versões do T_EX, especificamente pdfT_EX e LuaT_EX, têm recursos de *microtipografia*. Esses envolvem esticar glifos de fonte levemente e permitir que texto se projete muito levemente para dentro das margens. O uso deles dá ao T_EX mais flexibilidade na quebra de um parágrafo

em linhas e pode melhorar a aparência do parágrafo ao reduzir a hifenização e produzir uma cor tipográfica mais consistente.

`texinfo.tex` usa esses recursos (‘microtipos’ para abreviar) quando disponíveis. Você pode desativar o microtipo especificando ‘`@microtype off`’ em teu arquivo de entrada; da mesma forma, ‘`@microtype on`’ ativa o microtipo novamente.

As fontes usadas na saída do $\text{\LaTeX}$ possivelmente não sejam escalonáveis. Se as fontes não forem escalonáveis, usar-se microtipografia poderá deflagrar um erro ao converter a saída do $\text{\LaTeX}$. Portanto, a microtipografia não está ativada no caso padrão na saída do $\text{\LaTeX}$. É possível ativá-lo com ‘`@microtype on`’. Uma maneira de obter fontes escalonáveis para a fonte usada na saída padrão do $\text{\LaTeX}$ é a de instalar o pacote de fontes CM-Super (<http://ctan.org/pkg/cm-super>).

O comando `@microtype` não faz nada para outros formatos de saída.

E.10 Ampliação

Você pode tentar direcionar o $\text{\TeX}$ para tipografar páginas maiores ou menores que o normal com o comando `\mag` do $\text{\TeX}$. Tudo o que for tipografado é dimensionado proporcionalmente para maior ou para menor. (`\mag` significa “ampliação”). Esse *não* é um comando `@` do Texinfo, mas é um comando bruto do $\text{\TeX}$ que é prefixado com uma barra invertida. Você tem que escrever esse comando entre `@tex` e `@end tex` (veja-se Seção 16.3 [Comandos Brutos do Formatador], Página 131).

Siga o comando `\mag` com um ‘=’ e depois um número que seja 1.000 vezes a ampliação que você desejar. Por exemplo, para imprimir páginas em tamanho normal 1,2, escreva o seguinte próximo ao início do arquivo do Texinfo, antes da página de título:

```
@tex
\global\mag=1200
@end tex
```

Com algumas tecnologias de impressão, você consegue imprimir cópias de tamanho normal que aparentem melhor que o normal, fornecendo um mestre maior que o normal para tua gráfica. Elas fazem a redução, portanto aumentando efetivamente a resolução.

Dependendo do teu sistema, arquivos DVI preparados com um `\mag` não padrão possivelmente não imprimam ou possivelmente imprimam somente com determinadas ampliações. Esteja preparado(a) para experimentar.

Anexo F Especificação do Formato Info

Aqui nós descrevemos os detalhes técnicos do formato do Info.

Nesta descrição formal, os caracteres `<>*()|=#` são usados para a linguagem da própria descrição. Outros caracteres são literais. Os constructos formais usados são típicos: `<...>` indica um nome de metavariável, `'=`' significa definição, `'*`' repetição, `'?`' opcional, `'(`' agrupamento, `'|`' alternância e `'#`' comentário.

Em geral, programas que leem arquivos do Info deveriam tentar não diferenciar maiúsculas de minúsculas em palavras-chave que ocorrerem no arquivo (por exemplo, `'Tabela de Etiquetas'` e `'Tabela de etiquetas'` deveriam ser equivalentes) para a finalidade de suportar programas de geração do Info que usem diferentes maiusculizações.

Nós especificamos parênteses literais (aqueles que são parte do formato Info) com `<lparen>` e `<rparen>`, significando os caracteres únicos `'(`' e `)'` respectivamente. Finalmente, a sequência de dois caracteres `'~x'` significa o caractere único `'CTRL-x'`, para alguma `x`.

Esta definição de formato foi escrita cerca de 25 anos depois que o formato Info foi criado. Assim, no caso de conflitos entre esta definição e a prática real, a prática vence. Ela também pressupõe algum conhecimento geral do Texinfo; ela é destinada a ser um guia para implementadores(as) em vez de um padrão técnico rígido. Nós possivelmente retro referenciamos outras partes deste manual para exemplos e definições, em vez de redundantemente explicitar cada detalhe.

F.1 Formato Info: Um Manual Inteiro

Para começar, um manual do Info ou é *não dividido* (contido inteiramente em um arquivo) ou é *dividido* (em vários arquivos).

A sintaxe para um manual não dividido é:

```
<arquivo do info não dividido> =
<preâmbulo>
<nó>*
<tabela de etiquetas>?
<variáveis locais>?
```

Quando dividido, existe um *arquivo principal*, o qual contém somente ponteiros para os nós fornecidos em outros *subarquivos*. O arquivo principal se parece com isto:

```
<arquivo dividido principal do info> =
<preâmbulo>
<tabela indireta>
<tabela de etiquetas>
<variáveis locais>?
```

Os subarquivos em um manual dividido têm a seguinte sintaxe:

```
<subarquivo dividido do info> =
<preâmbulo>
<nó>*
```

Observe que a tabela de etiquetas não é opcional para arquivos divididos, pois ela é usada com a tabela indireta para deduzir em qual subarquivo um nó específico está.

Várias das seções em um arquivo Info (tais como nós ou tabelas de etiquetas) começam com uma sequência:

```
<separador> = (^L)?~(^L)?^J
```

Ou seja, um caractere `'CTRL-'` seguido por uma nova linha, com caracteres formfeed opcionais.

F.2 Formato Info: Preâmbulo

O <preâmbulo> é o texto no início de todos os arquivos de saída. Não se destina a ser visível por padrão em um visualizador do Info, mas pode ser exibido mediante solicitação do(a) usuário(a).

```
<preâmbulo> =
<identificação>      # "Este é NOMEARQUIVO, produzido por ..."
<copiando texto>     # Expansão do texto de @copying.
<entradas do diretório> # Derivado de @dircategory e @direntry.
```

Essas peças são:

<linha de identificação>
Uma sequência arbitrária de caracteres iniciando o arquivo da saída gerada, seguida por uma linha em branco.

<copiando texto>
A expansão de um ambiente @copying, se o manual tiver um (veja-se Seção 2.7.1 [copying], Página 16).

<entradas do diretório>
O resultado de quaisquer comandos @dircategory e @direntry presentes no manual (veja-se Seção 21.1.4 [Instalando Entradas de Diretório], Página 185).

F.3 Formato Info: Tabela Indireta

```
<tabela indireta> =
<separador>
Indireto:
(<nome do arquivo>: <bytepos>)*
```

A tabela indireta é escrita no arquivo principal somente no caso de saída dividida. Ela especifica, como um número inteiro decimal, a posição inicial do byte (baseado em zero) que o primeiro nó de cada subarquivo teria se os subarquivos fossem concatenados em ordem, sem incluir o arquivo de nível superior. O primeiro nó do conteúdo real é apontado pela primeira entrada.

Como exemplo, suponha que a saída dividida seja gerada para o manual do GDB. O arquivo de nível superior `gdb.info` conterá algo assim:

```
<separador>
Indireto:
gdb.info-1: 1878
gdb.info-2: 295733
...
```

Isso informa aos visualizadores do Info que o primeiro nó do manual ocorre no byte 1.878 do arquivo `gdb.info-1` (que seria depois do preâmbulo desse arquivo). O primeiro nó no subarquivo `gdb.info-2` começaria no byte 295.733 se `gdb.info-2` fosse posposto a `gdb.info-1`, incluindo quaisquer seções de preâmbulo em ambos os arquivos.

Infelizmente, programas de criação do Info como o `makeinfo` nem sempre implementaram essas regras perfeitamente, devido a vários defeitos e descuidos. Portanto, visualizadores robustos do Info deveriam voltar para procurar por um nó “próximo” à posição fornecida, em vez de desistir imediatamente se a posição não estiver exatamente no início de um nó.

F.4 Formato Info: Tabela de Etiquetas

```
<tabela de etiquetas> =
<separador>
```

```

Tabela de Etiquetas:
(<lparen>Indireto<rparen>)?
(Nó|Ref): <nodeid>^?<bytepos>
<separador>
Fim da Tabela de Etiquetas

```

A linha ‘(Indireto)’ aparece somente no caso de saída dividida.

A tabela de etiquetas especifica a posição inicial do baite de cada nó e âncora no arquivo. No caso de saída dividida, ela será escrita somente no arquivo de saída principal.

Cada linha define um identificador ou como uma âncora ou como um nó, conforme especificado. Por exemplo, ‘Nó: Topo^?1647’ diz que o nó chamado ‘Topo’ começa no baite 1.647 enquanto ‘Ref: VisãoGeral-NotaRodapé-1^?30045’ diz que a âncora chamada ‘VisãoGeral-NotaRodapé-1’ começa no baite 30.045. É um erro definir o mesmo identificador nos dois sentidos.

No caso de saída não dividida, as posições dos bairtes referem-se simplesmente ao local no arquivo da saída gerada. No caso de saída dividida, as posições dos bairtes referem-se a um arquivo imaginário criado pela concatenação de todos os arquivos divididos (mas não o arquivo de nível superior). Veja-se a seção anterior.

Aqui está um exemplo:

```

^_
Tabela de Etiquetas:
Nó: Topo^?89
Nó: Ch1^?292
^_
Final da Tabela de Etiquetas

```

Isso especifica um manual com dois nós, ‘Topo’ e ‘Ch1’, nas posições de baite 89 e 292 respectivamente. Como a linha ‘(Indireto)’ não está presente, o manual não é dividido.

Seções de preâmbulo ou outras seções de arquivos que não sejam nós não têm uma entrada na tabela de etiquetas.

F.5 Formato Info: Variáveis Locais

A seção de variáveis locais é opcional. Ela atualmente é usada para fornecer as informações de codificação e(ou) o código do idioma do documento.

```

<variáveis locais> =
<separador>
Variáveis Locais:
codificação: <codificação>
(Info-documentlanguage: <idioma>)?
Fim:

```

Veja-se Seção 15.2 [documentencoding], Página 127, para informações acerca da codificação do documento.

‘<idioma>’ deveria ser da forma *ll* ou *ll_cc*; veja-se Seção 15.1 [documentlanguage], Página 126. Somente um idioma pode ser especificado na seção de variáveis locais, mesmo que @documentlanguage ocorra várias vezes em um documento do Texinfo.

F.6 Formato Info: Nós Regulares

Nós regulares se parecem com isto:

```

<nó> =
<separador>

```


Arquivo: <fn>, Nó: <id1>, (Próximo: <id2>,)? (Anterior: <id3>,)? Acima: <id4>

<texto geral, até o próximo ^_ ou final do arquivo>

Pelo menos um espaço ou tabulação precisa estar presente depois de cada dois pontos e vírgula, mas qualquer número de espaços será ignorado. Os identificadores de nó <id> têm o seguinte formato:

```
<id> = (<lparen><arquivoinfo><rparen>)?<especificações-nó>?
<especificações-nó> = <nomenó> | <del><nomenó><del>
<del> = ^?
```

Esse <nó> define <id1> no arquivo <fn>, o qual tipicamente é ‘nomemanual’ ou ‘nomemanual.info’. Nenhum componente <arquivoinfo> entre parênteses pode aparecer dentro de <id1>.

Cada um dos identificadores depois de Próximo, Anterior e Acima se refere a nós ou a âncoras dentro de um arquivo. Esses ponteiros normalmente se referem ao mesmo arquivo, mas ‘(dir)’ é frequentemente usado para apontar para o arquivo de diretório de nível superior. Se um componente <arquivoinfo> for usado, então o nome do nó poderá ser omitido; nesse caso, o identificador do nó se referirá ao nó ‘Topo’ dentro do arquivo referenciado.

Os ponteiros Próximo e Anterior são opcionais. O ponteiro Acima tecnicamente também é opcional, embora muito provavelmente indique um erro na estruturação do nó. Convencionalmente, os nós são organizados para formar uma árvore, mas isso não é uma exigência do formato.

Nomes de nós contendo pontos, vírgulas, dois pontos ou parênteses podem confundir leitores do Info. Se for necessário referir-se a um nó cujo nome contenha algum desses, o <nomenó> deveria ser envolvido por um par de caracteres DEL (‘CTRL-?’, caractere número 127). `makeinfo` adiciona esses caracteres quando necessário no caso padrão. Observe que nem todos os leitores do Info reconhecem essa sintaxe. Veja-se [Restrições de Nomes de Nó do Info], Página 28.

O <texto geral> do nó pode incluir os constructos especiais descritos a seguir.

F.7 Formato Info: Menu

Convencionalmente, menus aparecem no final dos nós, mas o formato Info não impõe restrições ao local deles.

```
<menu> =
* Menu:
(<entrada de menu> | <comentário de menu>)*
```

As partes de uma <entrada de menu> também são descritas em Seção 3.9.4 [Partes do Menu], Página 35. Elas têm a mesma sintaxe das referências cruzadas, com um ‘*’ inicial em vez de ‘* (N|n)ote’ (veja-se Seção F.10 [Referência Cruzada do Formato Info], Página 287). Índices estendem o formato do menu para especificar a linha de destino; veja-se Seção F.9 [Impressão de Índice do Formato Info], Página 286.

Um <comentário de menu> é qualquer linha que não comece com ‘*’ e que apareça ou no início do menu ou seja separada de uma entrada do menu por uma ou mais linhas em branco. Esses comentários são destinados para serem exibidos como parte do menu, como estão (veja-se Seção 3.9.1 [Escrevendo um Menu], Página 33).

F.8 Formato Info: Imagem

O comando `@image` resulta na seguinte diretiva especial dentro do arquivo Info (veja-se Seção 9.2 [Imagens], Página 83):

```
<image> =
```

```

~@~H[image src="<arquivo da imagem>"
      (dpi=<dpi>)?
      (depth=<profundidade>)?
      (text="<conteúdo do arquivo de texto>")?
      (alt="<texto alternativo>")?
~@~H]

```

As quebras de linha e recuos nessa descrição são editoriais; o espaço em branco entre as diferentes partes da diretiva nos arquivos Info é arbitrário.

Nas strings `<arquivo de imagem>`, `<conteúdo do arquivo de texto>` e `<texto alternativo>`, ‘”’ é aspeado como ‘\”’ e ‘\’ é aspeado como ‘\\’. As especificações de texto e de alternativo são opcionais.

O valor de `alternativo` serve ao mesmo propósito que no HTML: uma descrição em prosa da imagem. Em exibições somente de texto ou sistemas de fala, por exemplo, o valor de `alternativo` pode ser usado em vez de exibir o `<arquivo de imagem>` (tipicamente gráfico).

O `<conteúdo do arquivo de texto>`, se presente, deveria ser considerado uma representação ASCII da imagem, para possível uso em uma exibição somente de texto.

O formato não prescreve a escolha entre exibir o `<arquivo de imagem>`, o `<texto alternativo>` ou o `<conteúdo do arquivo de texto>`.

As opções `dpi` e `depth` são geradas somente para a variável experimental ‘INFO_MATH_IMAGES’ do `texi2any` (veja-se Seção 20.4.5 [Variáveis de Personalização do Info e de Texto Simples], Página 173).

`dpi` fornece os pontos por polegada (*DPI*) da imagem. Programas de visualização do Info poderiam usar isso para dimensionar a imagem, de modo que ela tenha um tamanho apropriado para o texto ao redor. Aqui está uma fórmula possível para a altura de uma imagem em pixels:

$$h_{\text{exibir}} = (h_{\text{arquivo}}/\text{dpi}) * (72.27/10) * f$$

Aqui, h_{exibir} é a altura em pixels na qual se exibir a imagem, h_{arquivo} é a altura em pixels da imagem sem escala, dpi é o DPI proveniente do arquivo Info e f é o tamanho da fonte em pixels da fonte usada atualmente.

O 72,27 nessa fórmula é o número de pontos em uma polegada (conforme usado pelo $\text{T}_{\text{E}}\text{X}$). O 10 é o tamanho de fonte padrão 10 point do $\text{T}_{\text{E}}\text{X}$, em outras palavras, o tamanho de 1 em. O quociente (72,27/10) fornece o número de ems por polegada e converte a altura da imagem em polegadas para a altura da imagem em ems.

`depth` fornece a profundidade da imagem abaixo da linha de base, em pixels da imagem (antes do dimensionamento). Programas de visualização deveriam usá-lo para deslocar a imagem para baixo para alinhá-la adequadamente com o texto ao redor.

F.9 Formato Info: Impressão de Índice

Um índice no formato Info é uma espécie de menu, com uma diretiva adicional no início para marcá-lo como um menu de índice.

```

<impressãoíndice> =
~@~H[índice~@~H]
* Menu:

```

```

<entrada de índice>*

```

Os itens de `<entrada de índice>` são semelhantes às entradas normais do menu, mas a descrição em formato livre é substituída pelo número da linha onde as entradas ocorrem no texto:

```

<entrada de índice> =

```

* <texto de entrada>: <especificação do nó>. <especificação de linha>

<especificação de linha> =
<lparen>linha <linhanúmero><rparen>

A parte inicial, '<texto de entrada>: <nó de entrada>.', deveria estar em uma única linha. O <texto de entrada> é o termo do índice.

Embora <texto de entrada> seja seguido por dois pontos, ele próprio pode conter dois pontos, de modo que leitores do Info deveriam tentar incluir o máximo possível da linha no <texto de entrada>. (No entanto, <texto de entrada> pode não conter os caracteres DEL que podem ocorrer em <especificação do nó>).

<linhanúmero> é um número inteiro sem sinal, fornecido relativo ao início do <nó de entrada>. Uma quebra de linha opcional pode ocorrer antes de '<especificação de linha>'.

Aqui está um exemplo:

```
^@^H[índice^@^H]
* Menu:
```

```
* trovão:           Fenômenos Meteorológicos.           (linha 5)
```

Isso significa que uma entrada de índice para 'trovão' aparece na linha 5 do nó 'Fenômenos Meteorológicos'.

F.10 Formato Info: Referência Cruzada

Uma referência cruzada geral no formato Info tem uma das duas formas seguintes:

```
<referência cruzada> =
* (N|n)ote <identificador>::
| * (N|n)ote <rótulo>:<identificador>(.|,)

<identificador> = (<lparen><arquivoinfo><rparen>)?<especificação do nó>?
<rótulo> = <texto do rótulo> | <del><texto do rótulo><del>
```

Nenhum espaço deveria ocorrer entre o caractere '*' e o seguinte 'N' ou 'n'. '*Note' deveria ser usado no início de uma frase, caso contrário, '*note' deveria ser usado. (Alguns leitores do Info, tais como o do Emacs, conseguem exibir '*Note' e '*note' como 'Veja-se' e 'veja-se' respectivamente). Em ambos os casos, <texto do rótulo> é um texto descritivo.

Em ambas as formas, o <identificador> se refere a um nó ou âncora, da mesma maneira que uma referência na linha de informações do nó (veja-se Seção F.6 [Nós Regulares do Formato Info], Página 284). O opcional entre parênteses '<arquivoinfo>' é o nome do manual sendo referenciado e <especificação do nó> fornece o nó ou âncora dentro desse manual.

A segunda forma tem um rótulo descritivo. Uma referência cruzada nessa forma geralmente deveria ser terminada com uma vírgula ou ponto final, para tornar viável encontrar o final do <identificador>.

Se <rótulo> contiver dois pontos (:), ele deverá ser envolvido por um par de caracteres . Da mesma forma, caracteres de aspeamento podem ser usados para o nome do nó se ele contiver caracteres problemáticos; então, uma vírgula ou ponto final não é necessária. Conforme afirmado anteriormente, esse mecanismo de aspeamento não é suportado em todos os programas de leitura do Info.

O formato não prescreve como encontrar outros manuais para resolver tais referências.

Aqui estão alguns exemplos:

```
*observar Licença GNU de Documentação Livre::
*observar Tabela de Etiquetas: Tabela de Etiquetas do formato Info, para
```

```
detalhes.  
*Observar Visão Geral: (make)Topo.  
*Observar ^?:^?: (bash)Builtins do Shell Bourne.  
*Observar alloca.h: (gnulib)^?alloca.h^?.
```

O primeiro mostra uma referência para um nó no manual atual usando a forma curta.

O segundo também se refere a um nó no manual atual, chamado ‘Tabela de Etiquetas do Formato Info’; a ‘Tabela de Etiquetas’ antes de ‘:’ é somente um rótulo nessa referência específica, e o ‘**para detalhes.**’ é o texto pertencente à frase, não parte da referência.

O terceiro exemplo se refere ao nó ‘Topo’ em outro manual, nomeadamente ‘**make**’, com ‘Visão Geral’ sendo o rótulo para essa referência cruzada.

O quarto exemplo mostra um caractere de dois pontos sendo aspeado em um rótulo e o quinto exemplo mostra um ponto final sendo aspeado em um nome de nó.

Veja-se Capítulo 5 [Referências Cruzadas], Página 43.

Anexo G Licença GNU de Documentação Livre

Versão 1.3, 03 de novembro de 2008

Direitos autorais © 2000, 2001, 2002, 2007, 2008 Free
Software Foundation, Inc. <http://fsf.org/>

A qualquer pessoa é permitido copiar e distribuir cópias literais deste documento de licença, porém modificá-lo não é permitido.

0. PREÂMBULO

O propósito desta licença é tornar um manual, livro de texto, ou outro documento funcional e útil livre no sentido da liberdade: para assegurar a qualquer pessoa a liberdade efetiva para copiar e redistribuí-lo, com ou sem modificações, ambos comercialmente ou não comercialmente. Secundariamente, esta Licença preserva para o autor e editor uma maneira de obter crédito pelos seus trabalhos, ao mesmo tempo não sendo considerado responsável por modificações feitas por outros.

Esta Licença é uma espécie de “copyleft” (“esquerdos autorais”), o que significa que trabalhos derivados do documento devem necessariamente eles mesmos serem livres no mesmo sentido. Ela complementa a Licença Pública Geral GNU, a qual é uma licença de esquerdos autorais projetada para software livre.

Nós projetamos esta Licença para utilizá-la para manuais para software livre, porque software livre precisa de documentação livre: um programa livre deveria vir com manuais provendo as mesmas liberdades que o software provê. Porém esta Licença não é limitada a manuais de software; ela pode ser utilizada para qualquer trabalho textual, independentemente de questões de assunto ou se o trabalho textual for publicado como um livro impresso. Nós recomendamos esta Licença principalmente para trabalhos cujo propósito seja instrução ou referência.

1. APLICABILIDADE E DEFINIÇÕES

Esta Licença se aplica a qualquer manual ou outro trabalho, em qualquer meio, que contenha um aviso colocado pelo detentor dos direitos autorais dizendo que ele pode ser distribuído sob os termos desta Licença. Tal aviso concede uma licença mundial, livre de patente, ilimitada na duração, para utilizar aquele trabalho sob as condições nela declaradas. O “Documento”, abaixo, se refere a quaisquer desses manuais ou trabalhos. Qualquer membro do público é um titular da licença, e é mencionado como “você”. Você aceita a licença se você copiar, modificar ou distribuir o trabalho em uma forma que exija permissão sob lei de direitos autorais.

Uma “Versão Modificada” do Documento significa qualquer trabalho contendo o Documento ou uma porção dele, seja literalmente copiado, ou com modificações e/ou traduzido em outra língua.

Uma “Seção Secundária” é um apêndice nomeado ou uma seção pré-textual do Documento que lida exclusivamente com o relacionamento dos editores ou autores do Documento para com o assunto global do Documento (ou com questões relacionadas) e não contém nada que possa se conformar diretamente com aquele assunto global. (Assim, se o Documento for em parte um livro texto de matemática, uma Seção Secundária não pode explicar nada acerca de cálculos matemáticos). O relacionamento poderia ser uma questão de conexão histórica com o assunto ou com questões relacionadas, ou de posicionamento legal, comercial, filosófico, ético ou político respeitante a eles.

As “Seções Invariantes” são certas Seções Secundárias cujos títulos são projetados, como sendo aqueles de Seções Invariantes, no aviso que diz que o Documento é publicado sob esta Licença. Se uma seção não se encaixa na definição de Secundária acima, então a seção não está autorizada a ser designada como Invariante. O Documento pode conter zero Seções

Invariantes. Se o Documento não identifica quaisquer Seções Invariantes, então não existe nenhuma.

Os “Textos de Capa” são certas passagens curtas de texto que são listadas, como Textos de Primeira Capa ou Textos de Quarta-Capa, no aviso que diz que o Documento é publicado sob esta Licença. Um Texto de Primeira Capa pode ter no máximo cinco (05) palavras, e um Texto de Quarta Capa pode ter no máximo vinte e cinco (25) palavras.

Uma cópia “Transparente” do Documento significa uma cópia legível por máquina, representada em um formato cuja especificação está disponível para o público em geral, que é adequada para revisar o documento diretamente com editores de texto genéricos ou (para imagens compostas de pixels) programas de pintura genéricos ou (para desenhos) algum editor de desenho disponível amplamente, e que seja adequado para entrada a formatadores de texto ou para tradução automática a uma variedade de formatos próprios para entrada a formatadores de texto. Uma cópia feita em um formato de arquivo contrário ao Transparente, cuja linguagem de marcação, ou ausência de linguagem de marcação, tenha sido organizada para frustrar ou desencorajar modificações subsequentes por leitores, não é Transparente. Um formato de imagem não é Transparente se utilizado para qualquer quantidade substancial de texto. Uma cópia que não é “Transparente” é chamada “Opaca”.

Exemplos de formatos adequados para cópias Transparentes incluem ASCII puro sem marcações; formato de entrada Texinfo; formato de entrada LaTeX; SGML ou XML utilizando um DTD disponível publicamente; HTML simples conformante com o padrão; PostScript ou PDF projetado para modificação humana. Exemplos de formatos transparentes de imagens incluem PNG, XCF e JPG. Formatos opacos incluem formatos proprietários que podem ser lidos e editados somente por processadores proprietários de palavra; SGML ou XML para os quais o DTD e/ou as ferramentas de processamentos não estejam disponíveis genericamente; e o HTML gerado por máquina; PostScript ou PDF produzidos por alguns processadores de palavra apenas para propósitos de saída.

A “Página de Título” significa, para um livro impresso, a própria página de título, mais tantas páginas seguintes quantas sejam necessárias para manter, legivelmente, o material que esta Licença exige para aparecer na página de título. Para trabalhos em formatos que não tenham qualquer página de título como tal, “Página de Título” significa o texto próximo da mais proeminente aparição do título do trabalho, precedendo o início do corpo do texto.

O “editor” significa qualquer pessoa ou entidade que distribui cópias do Documento ao público.

Uma seção “Intitulada XYZ” significa uma subunidade nomeada do Documento cujo título ou é precisamente XYZ ou contém XYZ entre parênteses seguinte ao texto que traduz XYZ em outra linguagem. (Aqui XYZ significa um nome específico de seção mencionado abaixo, tais como “Agradecimentos”; “Dedicatórias”; “Patrocínios”; ou “Histórico”). “Preservar o Título” de tal seção quando você modificar o Documento significa que ele permanece uma seção “Intitulada XYZ” de acordo com essa definição.

O Documento pode incluir Declarações de Garantia próximas ao aviso que declara que esta Licença se aplica ao Documento. Essas Declarações de Garantia são consideradas como inclusas por referência nesta Licença, porém somente com relação à negação de garantias: qualquer outra implicação que essas Declarações de Garantia possam ter é inválida e não tem efeito sobre o significado desta Licença.

2. CÓPIA LITERAL

Você pode copiar e distribuir o Documento em qualquer meio, ambos comercialmente e não comercialmente, contanto que esta Licença, os avisos de direitos autorais, e o aviso de licença dizendo que esta Licença se aplica ao Documento estejam reproduzidas em todas as cópias, e que você não adiciona quaisquer outras condições, quaisquer que sejam, àquelas desta Licença. Você não pode utilizar medidas técnicas para obstruir ou controlar a leitura

ou posteriores cópias das cópias que você fizer ou distribuir. Entretanto, você pode aceitar remuneração em troca das cópias. Se você distribui um número de cópias grande o suficiente, você deve necessariamente também seguir as condições na seção três (3).

Você também pode ceder cópias, sob as mesmas condições declaradas acima, e você pode publicamente exibir cópias.

3. CÓPIAS EM QUANTIDADE

Se você publicar cópias impressas (ou cópias em mídia que geralmente tem capas impressas) do Documento, em número maior que cem (100), e o aviso de licença do Documento exigir Textos de Capa, você deve necessariamente encartar as cópias em capas que transportem, claramente e legivelmente, todos estes Textos de Capa: Textos de Primeira Capa na primeira capa, e Textos de Quarta Capa na capa traseira. Ambas as capas devem necessariamente também claramente e legivelmente identificar você como o editor dessas cópias. A capa frontal deve necessariamente apresentar o título completo com todas as palavras do título igualmente proeminentes e visíveis. Você pode adicionar outros materiais nas capas adicionalmente. As cópias com modificações limitadas às capas, tanto quanto preservem o título do Documento e satisfaçam essas condições, podem ser tratadas como cópias literais em relação a outros aspectos.

Se os textos exigidos para ambas as capas forem muito volumosos para caber legivelmente, você deveria colocar os primeiros listados (tantos quantos caibam razoavelmente) na capa atual, e continuar o restante em páginas adjacentes.

Se você publicar ou distribuir cópias Opacas do Documento em número maior que cem (100), você deve necessariamente ou incluir uma cópia Transparente, legível por máquina, junto com cada cópia Opaca, ou declarar, na ou com cada cópia Opaca, uma localização de rede de computador, a partir da qual o público usuário de rede geral tenha acesso para baixar, utilizando protocolos de rede de padrão público, uma cópia Transparente completa do Documento, livre do material adicionado.

Se você se utilizar da última opção, você deve necessariamente adotar razoavelmente passos prudentes, quando você iniciar a distribuição de cópias Opacas em quantidade, para se assegurar que essa cópia Transparente permanecerá então acessível na localização declarada até pelo menos um ano após a última vez que você distribuiu uma cópia Opaca (diretamente ou por intermédio dos seus agentes ou varejistas) daquela edição ao público.

É pedido, mas não exigido, que você contate os autores do Documento bem antes de redistribuir qualquer número grande de cópias, para dá-los a oportunidade de lhe fornecer uma versão atualizada do Documento.

4. MODIFICAÇÕES

Você pode copiar e distribuir uma Versão Modificada do Documento sob as condições das seções dois (2) e três (3) acima, contanto que você publique a Versão Modificada precisamente sob esta Licença, com a Versão Modificada preenchendo a função do Documento, portanto licenciando a distribuição e modificação da Versão Modificada a quem quer que possua uma cópia dela. Adicionalmente, você deve necessariamente fazer estas coisas na Versão Modificada:

- A. Utilize na Página de Título (e nas capas, se existentes) um título distinto daquele do Documento, e daqueles das versões prévias (as quais deveriam, se existiu alguma, serem listadas na seção Histórico do Documento). Você pode utilizar o mesmo título que uma versão prévia, se o editor original daquela versão conceder permissão.
- B. Liste na Página de Título, como autores, uma ou mais pessoas ou entidades responsáveis pela autoria das modificações na Versão Modificada, junto com ao menos cinco dos autores principais do Documento (todos os autores principais, se tiver menos que cinco), a menos que eles liberem você dessa exigência.

- C. Declare na Página de Título o nome do editor da Versão Modificada, como o editor.
- D. Preserve todos os avisos de direitos autorais do Documento.
- E. Adicione um aviso apropriado de direitos autorais para suas modificações, adjacente aos outros avisos de direitos autorais.
- F. Inclua, imediatamente após os avisos de direitos autorais, um aviso de licença concedendo ao público permissão para utilizar a Versão Modificada sob os termos desta Licença, na forma mostrada no Adendo abaixo.
- G. Preserve, naquele aviso de licença, as listas completas de Seções Invariantes e Textos de Capa exigidos dados no aviso de licença do Documento.
- H. Inclua uma cópia inalterada desta Licença.
- I. Preserve a seção intitulada “Histórico”, Preserve seu Título, e adicione a ele um item declarando ao menos o título, ano, novos autores, e editor da Versão Modificada, conforme dado na Página de Título. Se não existir uma seção intitulada “Histórico” no Documento, crie uma declarando o título, ano, autores, e editor do Documento, conforme dado em sua Página de Título, então adicione um item descrevendo a Versão Modificada, conforme declarado na frase prévia.
- J. Preserve a localização de rede, se existente, dada no Documento para acesso público a uma cópia Transparente do Documento, e da mesma forma as localizações de rede dadas no Documento para versões prévias nas quais foi baseado. Essas podem ser colocadas na seção “Histórico”. Você pode omitir uma localização de rede para um trabalho que foi publicado nos últimos quatro anos anteriores à publicação do próprio do Documento, ou se o editor original da versão à qual a localização de rede se refere conceder permissão.
- K. Para cada seção Intitulada “Agradecimentos” ou “Dedicatórias”, Preserve o Título da seção, e preserve na seção toda a substância e tonalidade de cada um dos agradecimentos a contribuidores e/ou dedicatórias dadas nela.
- L. Preserve todas as Seções Invariantes do Documento, inalteradas em seus textos e em seus títulos. Os números de Seção ou o equivalente não são considerados parte dos títulos de seção.
- M. Delete quaisquer seções Intituladas “Patrocínios”. Tal seção não pode ser incluída na Versão Modificada.
- N. Não re intitule qualquer seção existente para Intitulada “Patrocínios” ou para conflitar no título com qualquer Seção Invariante.
- O. Preserve quaisquer Declarações de Garantia.

Se a Versão Modificada incluir novas seções pré textuais ou apêndices que se qualifiquem como Seções Secundárias e não contenham material copiado a partir do Documento, você pode, a sua escolha, designar algumas ou todas essas seções como Invariantes. Para fazer isso, adicione seus títulos à lista das Seções Invariantes no aviso de licença da Versão Modificada. Esses títulos devem necessariamente serem distintos de quaisquer outros títulos de seções.

Você pode adicionar uma seção Intitulada “Patrocínios”, contanto que ela não contenha nada além de patrocínios da sua Versão Modificada por vários patrocinadores—por exemplo, declarações de avaliadores ou aquelas de que o texto foi aprovado por uma organização como a definição autorizativa de um padrão.

Você pode adicionar uma passagem de até cinco palavras, como um Texto de Primeira Capa, e uma passagem de até vinte e cinco palavras, como um Texto de Quarta Capa, ao final da lista dos Textos de Capa na Versão Modificada. Somente uma passagem de Texto de Primeira Capa e uma de Texto de Quarta Capa podem ser adicionadas por (ou mediante acordos feitos por) qualquer uma entidade. Se o Documento já inclui um texto de capa para a mesma capa, previamente adicionado por você ou por acordo feito pela mesma entidade

pela qual você está atuando, você não pode adicionar outro; porém você pode substituir o antigo, na permissão explícita do editor prévio que adicionou o antigo.

O(s) autor(s) e editor(s) do Documento, por esta Licença, não concedem permissão para utilizar seus nomes para publicidade para ou para afirmar ou implicar patrocínio de qualquer Versão Modificada.

5. COMBINANDO DOCUMENTOS

Você pode combinar o Documento com outros documentos publicados sob esta Licença, sob os termos definidos na seção quatro (4) acima para versões modificadas, contanto que você inclua na combinação todas as Seções Invariantes de todos os documentos originais, não modificados, e listá-los todos como Seções Invariantes do seu trabalho combinado no seu aviso de licença, e você preserva todas as Declarações de Garantias deles.

O trabalho combinado precisa conter somente uma cópia desta Licença, e múltiplas Seções Invariantes idênticas podem ser substituídas por uma cópia única. Se existirem múltiplas Seções Invariantes com o mesmo nome, mas conteúdos diferentes, torne o título de cada uma de tal seção único adicionando ao final dele, entre parênteses, o nome do autor ou editor original daquela seção se conhecido, ou, do contrário, um número único. Faça o mesmo ajuste aos títulos da seção na lista de Seções Invariantes no aviso de licença do trabalho combinado.

Na combinação, você deve necessariamente combinar quaisquer seções Intituladas “Histórico” nos vários documentos originais, formando uma seção Intitulada “Histórico”; de mesma maneira, combine quaisquer seções Intituladas “Agradecimentos”, e quaisquer seções Intituladas “Dedicatórias”. Você deve necessariamente deletar todas as seções Intituladas “Patrocínios”.

6. COLEÇÕES DE DOCUMENTOS

Você pode produzir uma coleção consistente do Documento e outros documentos publicados sob esta Licença, e substitua as cópias individuais desta Licença nos vários documentos por uma cópia única que esteja incluída na coleção, contanto que você siga as regras desta Licença para cópias literais de cada um dos documentos em todos os outros aspectos.

Você pode extrair um documento único de tal coleção, e distribuí-lo individualmente sob esta Licença, contanto que você insira uma cópia desta Licença no documento extraído, e siga esta Licença em todos os outros aspectos relativos à cópias literais daquele documento.

7. AGREGAÇÃO COM TRABALHOS INDEPENDENTES

Uma compilação do Documento ou seus derivados com outros documentos separados e independentes ou trabalhos, dentro ou junto a volume de armazenamento ou meio de distribuição, é chamado em “agregado” se os direitos autorais resultantes da compilação não forem utilizados para limitar os direitos legais dos usuários da compilação além do que os trabalhos individuais permitem. Quando o Documento for incluído em um agregado, esta Licença não se aplica aos outros trabalhos no agregado, os quais não são eles próprios trabalhos derivados do Documento.

Se a exigência do Texto de Capa da seção três (3) for aplicável a essas cópias do Documento, então se o Documento for menor que a metade do agregado inteiro, os Textos de Capa do Documento podem ser colocados em capas que encartem o Documento dentro do agregado, ou o equivalente eletrônico de capas se o Documento estiver em formato eletrônico. Do contrário, eles devem necessariamente aparecer nas capas impressas que encartem o agregado inteiro.

8. TRADUÇÃO

Tradução é considerada um tipo de modificação, de forma que você pode distribuir traduções do Documento sob os termos da seção quatro (4). A substituição de Seções Invariantes por traduções exige permissão especial de seus detentores dos direitos autorais, porém você pode incluir traduções de algumas ou todas as Seções Invariantes adicionalmente às versões

originais dessas Seções Invariantes. Você pode incluir uma tradução desta Licença, e todos os avisos de licença no Documento, e quaisquer Declarações de Garantia, contanto que você inclua também a versão original em Inglês desta Licença e as versões originais daqueles avisos e declarações. No caso de uma divergência entre a tradução e a versão original desta Licença ou um aviso ou declaração, a versão original prevalecerá.

Se uma seção no Documento for intitulada “Agradecimentos”, “Dedicatórias”, ou “Histórico”, a exigência (seção 4) de Preservar seu Título (seção 1) tipicamente exigirá a modificação do título atual.

9. FINALIZAÇÃO

Você não pode copiar, modificar, sublicenciar, ou distribuir o Documento, exceto conforme expressamente provido sob esta Licença. Qualquer tentativa clandestina de copiar, modificar, sublicenciar, ou distribuir o Documento é inválida, e automaticamente finalizará seus direitos sob esta Licença.

Entretanto, se você cessar todas as violações a esta Licença, então a sua licença oriunda de um detentor de direitos autorais em particular está restabelecida (a) provisoriamente, a menos e até que o detentor dos direitos autorais explicita e finalmente cancele sua licença; e (b) permanentemente, se o detentor dos direitos autorais falhar em notificar você da violação, por algum meio razoável, antes de sessenta (60) dias após a cessação.

Além disso, a sua licença oriunda de um detentor de direitos autorais em particular está restabelecida permanentemente se o detentor dos direitos autorais notificar você sobre a violação por algum meio razoável, essa for a primeira vez que você recebeu um aviso de violação desta Licença (para qualquer trabalho) oriunda daquele detentor de direitos autorais, e você sanar a violação antes de decorridos trinta (30) dias após o seu recebimento do aviso.

A finalização dos seus direitos sob esta seção não finaliza as licenças de varejistas que tenham recebido cópias ou direitos de você sob esta Licença. Se os seus direitos tiverem sido finalizados e não permanentemente restabelecidos, o recebimento de uma cópia de algum ou de tudo do mesmo material não concede a você direitos de utilizá-lo.

10. REVISÕES FUTURAS DESTA LICENÇA

A Free Software Foundation pode publicar novas, revisadas versões da Licença de Documentação Livre GNU de tempos em tempos. Tais novas versões serão similares na essência à presente versão, porém podem diferir em detalhes para abarcar novos problemas ou assuntos. Veja-se <http://www.gnu.org/copyleft/>.

Para cada versão da Licença é dado um número distintivo de versão. Se o Documento especifica que uma versão numerada em particular desta Licença “ou qualquer versão posterior” se aplica a ele, você tem a opção de seguir os termos e condições ou da versão especificada ou de qualquer versão posterior que tenha sido publicada (não como um rascunho) pela Free Software Foundation. Se o Documento não especifica um número de versão desta Licença, você pode escolher qualquer versão já publicada (não como um rascunho) pela Free Software Foundation. Se o Documento especifica que um procurador pode decidir quais versões futuras desta Licença podem ser utilizadas, essa declaração pública do procurador de aceitação de uma versão permanentemente autoriza você a escolher aquela versão para o Documento.

11. RELICENCIAMENTO

“Sítio de Colaboração Massiva Multi autor” (ou “Sítio MMC”) significa qualquer servidor da Rede Mundial de Computadores que publica trabalhos sujeitos a direitos autorais e também provê facilidades proeminentes para qualquer pessoa editar esses trabalhos. Um wiki público que qualquer pessoa pode editar é um exemplo de tal servidor. Uma “Colaboração Massiva Multi autor” (ou “MMC”) contida no sítio significa qualquer conjunto de trabalhos sujeitos a direitos autorais assim publicados no sítio MMC.

“CC-BY-SA” significa a licença Creative Commons Attribution-Share Alike 3.0 publicada pela Creative Commons Corporation, uma corporação sem fins lucrativos com seu domicílio empresarial situado em São Francisco, Califórnia, Estados Unidos da América do Norte, bem como versões futuras de esquerdos autorais dessa licença publicadas pela mesma organização.

“Incorporar” significa publicar ou republicar um Documento, no todo ou em parte, como parte de outro Documento.

Um MMC é “elegível para relicenciamento” se ele for licenciado sob esta Licença, e se todos os trabalhos que foram primeiro publicados sob esta Licença em algum lugar que não esse MMC, e subsequentemente incorporados, no todo ou em parte, no MMC, (1) não tinham textos de capa ou seções invariantes; e (2) estavam assim incorporados antes de 01 de novembro de 2008.

O operador de um Sítio MMC pode republicar um MMC contido no sítio sob CC-BY-SA, no mesmo sítio, a qualquer tempo antes de 01 de agosto de 2009, contanto que o MMC seja elegível para relicenciamento.

ADENDO: Como utilizar esta Licença para seus documentos

Para utilizar esta Licença em um documento que você escreveu, inclua um cópia da Licença no documento e coloque os seguintes avisos de direitos autorais e licença pouco depois da página de título:

```
Direitos autorais (C) ano seu nome.
Permissão é concedida para copiar, distribuir e/ou modificar este
documento sob os termos da Licença de Documentação Livre GNU, Versão
1.3 ou qualquer versão posterior publicada pela Free Software
Foundation; sem Seções Invariantes, sem Textos de Primeira Capa, e sem
Textos de Quarta Capa. Uma cópia da licença está inclusa na seção
intitulada ``Licença de Documentação Livre GNU''.
```

Se você tiver Seções Invariantes, Textos de Primeira Capa e Textos de Quarta Capa, substitua a linha “sem. . .Capa” por isto:

```
com as Seções Invariantes sendo liste seus títulos, com os
Textos de Primeira Capa sendo lista, e com os Textos de Quarta
Capa sendo lista.
```

Se você tiver Seções Invariantes sem Textos de Capa, ou alguma outra combinação dos três, mescle essas duas alternativas para adequar a situação.

Se o seu documento contém exemplos não triviais de código de programação, nós recomendamos publicar esses exemplos em paralelo, sob sua escolha de licença de software livre, tal como a Licença Pública Geral GNU, para permitir seu uso em software livre.

Índice de Comandos

Esta é uma lista alfabética de todos os comandos @ e de diversas funções do Emacs Lisp. Para facilitar o uso da lista, os comandos são listados sem ‘@’ precedente.

!	@
! (fim de frase) 99	@ (‘@’ literal) 95
"	\
" (acento trema) 100	\ (‘\’ literal na matemática) 96
&	{
& (‘&’ literal) 96	{ (‘{’ literal) 95
,	}
' (acento agudo) 100	} (‘}’ literal) 95
*	~
* (força quebra de linha) 110	~ (acento til) 100
,	A
, (acento cedilha) 100	aa 101
—	AA 101
— (hífen discriçãoário) 111	abbr 60
.	acronym 60
. (fim de frase) 99	ae 101
/	AE 101
/ (permite quebra de linha) 110	afivepaper 280
:	afourlatex 280
: (suprimir espaço de fim de frase) 98	afourpaper 280
=	afourwide 280
= (acento macron) 100	alias 144
?	allowcodebreaks 111
? (fim de frase) 99	ampchar 96
^	anchor 49
^ (acento circunflexo) 100	appendix 38
`	appendixsec 39
` (acento grave) 100	appendixsection 39
	appendixsubsec 40
	appendixsubsubsec 40
	apply 124
	arrow 107
	asis 77
	atchar 95
	author 19
	B
	b (fonte negrito) 63
	\backslash 96
	backslashchar 96
	blocorecuado 67
	bsixpaper 280
	bullet 103
	bye 25

C

c.....	8
caption.....	82
cartouche.....	72
center.....	20
centerchap.....	38
chapheading.....	39
chapter.....	38
cindex.....	88
citação.....	66
cite.....	52
clear.....	133
click.....	107
clicksequence.....	107
clickstyle.....	107
code.....	55
codequotebacktick.....	97
codequoteundirected.....	97
columnfractions.....	79
comentário.....	8
comma.....	95
command.....	59
complete_tree_nodes_menus.....	172
complete_tree_nodes_missing_menu.....	172
conteúdo.....	21
copying.....	16
copyright.....	103
cpindex.....	88

D

defblock.....	123
defcodeindex.....	93
defcv.....	120
defcvx.....	116
deffn.....	116
deffnx.....	116
defindex.....	93
definfoenclose.....	145
defivar.....	121
defivarx.....	116
defline.....	123
defmac.....	117
defmacx.....	116
defmethod.....	122
defmethodx.....	116
defop.....	121
defopt.....	119
defoptx.....	116
defopx.....	116
defspec.....	117
defspecx.....	116
deftp.....	120
deftpx.....	116
deftypecv.....	121
deftypecvx.....	116
deftypefn.....	117
deftypefnnewline.....	118
deftypefnx.....	116
deftypefun.....	118
deftypefunx.....	116
deftypeivar.....	121
deftypeivarx.....	116
deftypeline.....	123

deftypemethod.....	122
deftypemethodx.....	116
deftypeop.....	122
deftypeopx.....	116
deftypevar.....	120
deftypevarx.....	116
deftypevr.....	119
deftypevrx.....	116
defun.....	117
defunx.....	116
defvar.....	119
defvarx.....	116
defvr.....	119
defvrx.....	116
DEL (caractere de comentário).....	9
detailmenu.....	23
dfn.....	60
dh.....	101
DH.....	101
dircategory.....	14
direntry.....	185
displaymath.....	109
dmn.....	100
docbook.....	131
documentdescription.....	194
documentencoding.....	127
documentinfo.....	18
documentlanguage.....	126
dotaccent.....	100
dotless.....	101
dots.....	103

E

email.....	62
\emergencystretch.....	155
emph.....	62
end.....	221
enddots.....	103
enumerate.....	76
env.....	59
equiv.....	106
error.....	106
errormsg.....	130
errormsg, e números de linha no T _E X.....	147
estilo de nota de rodapé.....	87
euro.....	103
evenfooting.....	277
evenfootingmarks.....	278
evenheading.....	277
evenheadingmarks.....	278
everyfooting.....	277
everyfootingmarks.....	278
everyheading.....	277
everyheadingmarks.....	278
example.....	67
exampleindent.....	279
exclamdown.....	101
exdent.....	69
exibição.....	69
expansion.....	105

F

file	59
fill_gaps_in_sectioning	172
finalout	155
findex	88
firstparagraphindent	279
float	81
flushleft	70
flushright	70
fnindex	88
fn-name	115
fonttextsize	63
foo	97
foobar	115, 117
footnote	85
formato	69
forward-word	114
frenchspacing	99
ftable	78

G

\gdef dentro de @tex	131
geq	104
\globaldefs dentro de @tex	131
group	112
guillemetleft	101
guillemetright	101
guillemotleft	101
guillemotright	101
guilsinglleft	101
guilsinglright	101

H

H (acento trema húngaro)	100
hashchar	96
hbox	155
heading	39
headings	275
headitem	79
headitemfont	79
html	131
hyphenation	111

I

i (fonte itálica)	63
ifclear	134
ifcommanddefined	136
ifcommandnotdefined	136
ifdocbook	129
ifhtml	129
ifinfo	129
iflatex	129
ifnotdocbook	130
ifnothtml	130
ifnotinfo	130
ifnotlatex	130
ifnotplaintext	130
ifnottex	130
ifnotxml	130
ifplaintext	129
ifset	134

iftex	129
ifxml	129
ignore	9
image	83
include	148
indent	71
indicateurl	61
inforef	50
Info-validate	272
inlinefmt	131
inlinefmtifelse	131
inlineifclear	134
inlineifset	134
inlineraw	131
\input (iniciação do T _E X bruto)	9
insert_nodes_for_sectioning_commands	172
insertcopying	17
interno	116
isearch-backward	116
isearch-forward	116
item em @itemize	74
item em @multitable	79
item em @table	77
itemize	74
itemx	78

K

kbd	56
kbdinputstyle	56
key	57
kindex	88
kyindex	88

L

l	101
latex	131
LaTeX	103
lbracechar	95
leq	104
linemacro	144
link	50
\linkcolor	52
lisp	68
listoffloats	82
lowersections	41
L	101

M

macro	138
\mag (ampliação bruta do T _E X)	281
majorheading	39
makeinfo-buffer	262
makeinfo-kill-job	262
makeinfo-recenter-output-buffer	262
makeinfo-region	262
maketitle	18
math	109
\mathopsup	109
menu	33
microtype	281
minus	104

<code>move_index_entries_after_items</code>	173
<code>multitable</code>	79

N

<code>nó</code>	26
<code>namedanchor</code>	49
<code>need</code>	113
<code><newline></code>	98
<code>next-error</code>	262
<code>nodesdescription</code>	32
<code>nodesdescriptionblock</code>	32
<code>noindent</code>	71
<code>novalidate</code>	153

O

<code>o</code>	101
<code>occur</code>	256
<code>occur-mode-goto-occurrence</code>	255
<code>oddfooting</code>	277
<code>oddfootingmarks</code>	278
<code>oddheading</code>	277
<code>oddheadingmarks</code>	278
<code>oe</code>	101
<code>OE</code>	101
<code>ogonek</code>	100
<code>option</code>	60
<code>ordf</code>	101
<code>ordm</code>	101
<code>O</code>	101

P

<code>page</code>	112
<code>page, dentro de @titlepage</code>	19
<code>pagesizes</code>	280
<code>paragraphindent</code>	278
<code>part</code>	40
<code>pgindex</code>	88
<code>phoo</code>	145
<code>pindex</code>	88
<code>point</code>	106
<code>pounds</code>	103
<code>print</code>	105
<code>printindex</code>	91
<code>process</code>	118
<code>publication</code>	18
<code>push</code>	118
<code>pxref</code>	48

Q

<code>questiondown</code>	101
<code>quotedblbase</code>	101
<code>quotedblleft</code>	101
<code>quotedblright</code>	101
<code>quoteleft</code>	101
<code>quoteright</code>	101
<code>quotesinglbase</code>	101

R

<code>r (fonte romana)</code>	63
<code>raggedright</code>	70
<code>raisesections</code>	41
<code>rbracechar</code>	95
<code>ref</code>	48
<code>@refill</code>	244
<code>regenerate_master_menu</code>	173
<code>registeredsymbol</code>	103
<code>relate_index_entries_to_table_entries</code>	173
<code>result</code>	104
<code>ringaccent</code>	100
<code>rmacro</code>	138

S

<code>samp</code>	57
<code>sansserif</code>	63
<code>sc</code>	62
<code>section</code>	39
<code>seealso</code>	90
<code>seeentry</code>	90
<code>set</code>	133
<code>setchapternewpage</code>	274
<code>@setcontentsaftertitlepage</code>	244
<code>setfilename</code>	12
<code>@setshortcontentsaftertitlepage</code>	244
<code>setttitle</code>	12
<code>shortcaption</code>	82
<code>shortcontents</code>	21
<code>shorttitlepage</code>	17
<code>slanted</code>	63
<code>smallbook</code>	280
<code>smalldisplay</code>	73
<code>smallexample</code>	73
<code>smallformat</code>	73
<code>smallindentedblock</code>	73
<code>smalllisp</code>	73
<code>smallquotation</code>	73
<code>sortas</code>	89
<code>sp</code>	112
<code>sp (espaçamento entre linhas da página de título)</code> ..	20
<code><space></code>	98
<code>ss</code>	101
<code>strong</code>	62
<code>sub</code>	102
<code>subentry</code>	90
<code>subheading</code>	40
<code>subsection</code>	40
<code>subsubheading</code>	40
<code>subsubsection</code>	40
<code>subtitle</code>	19
<code>summarycontents</code>	21
<code>sup</code>	102
<code>syncodeindex</code>	92
<code>synindex</code>	93

T

t (fonte de máquina de escrever).....	63
tab	79
<tab>	98
tabela	77
tex	131
texinfo-all-menus-update	258
texinfo-every-node-update	258
texinfo-format-buffer	263
texinfo-format-region	263
texinfo-indent-menu-description	261
texinfo-insert-braces	254
texinfo-insert-@code	253
texinfo-insert-@dfn	253
texinfo-insert-dwim-@ref	253
texinfo-insert-@end	253
texinfo-insert-@example	253
texinfo-insert-@item	253
texinfo-insert-@kbd	253
texinfo-insert-@node	253
texinfo-insert-node-lines	261
texinfo-insert-@noindent	253
texinfo-insert-@samp	253
texinfo-insert-@table	253
texinfo-insert-@var	253
texinfo-make-menu	258
texinfo-master-menu	257
texinfo-multiple-files-update	259
texinfo-sequential-node-update	261
texinfo-show-structure	255
texinfo-start-menu-description	254
texinfo-tex-buffer	263
texinfo-update-node	258
textdegree	104
TeX	103
th	101
thischapter	277
thischaptername	277
thischapternum	277
thisfile	277
thispage	277
thispart	277
thissection	277
thissectionname	277
thissectionnum	277
thistitle	277
TH	101
tie	112
tieaccent	100

tindex	88
title	19
titlefont	20
titlepage	18
today	278
top	29
tpindex	88

U

u (acento breve).....	100
ubaraccent	100
udotaccent	100
unmacro	139
unnumbered	38
unnumberedsec	39
unnumberedsubsec	40
unnumberedsubsubsec	40
up-list	254
uref	50
urefbreakstyle	51
\urefurlonlylinktrue	52
url	50
\urlcolor	52
U	107

V

v (caron).....	100
value	133
var	58
verb	58
verbatim	68
verbatiminclude	149
vindex	88
vrindex	88
vskip pulo vertical do T _E X	19
vtable	78

W

w	111
---------	-----

X

xml	131
xref	48
xrefautomaticsectiontitle	46
xrefname	42

Índice de Variáveis

Esta é uma lista alfabética de todas as variáveis, principalmente variáveis de personalização.

A

after, valor para @urefbreakstyle	52
AFTER_SHORT_TOC_LINES	210
AFTER_TOC_LINES	210
ASCII_DASHES_AND_QUOTES	173
ASCII_GLYPH	173
ASCII_PUNCTUATION	174
AUTO_MENU_DESCRIPTION_ALIGN_COLUMN	174
AUTO_MENU_MAX_WIDTH	174

B

BASEFILENAME_LENGTH	208
before, valor para @urefbreakstyle	52
BEFORE_SHORT_TOC_LINES	210
BEFORE_TOC_LINES	210
BIG_RULE	210
BODY_ELEMENT_ATTRIBUTES	210

C

CASE_INSENSITIVE_FILENAMES	205
CHAPTER_HEADER_LEVEL	210
CHECK_HTMLXREF	207
CHECK_MISSING_MENU_ENTRY	168
CHECK_NORMAL_MENU_STRUCTURE	168
CLASS_BEGIN_USEPACKAGE	175
CLOSE_DOUBLE_QUOTE_SYMBOL	174
CLOSE_QUOTE_SYMBOL	176
para HTML	209
code, valor para @kbinputstyle	56
COLLATION_LANGUAGE	168
COMMAND_LINE_ENCODING	169
CONTENTS_OUTPUT_LOCATION	204
CONVERT_TO_LATEX_IN_MATH	212
COPIABLE_LINKS	209
CPP_LINE_DIRECTIVES	169

D

DATE_IN_HEADER	208
DEBUG	169
DEF_TABLE	211
DEFAULT_RULE	210
dispenssize	73
distinct, valor para @kbinputstyle	56
DO_ABOUT	205
DOC_ENCODING_FOR_INPUT_FILE_NAME	169
DOC_ENCODING_FOR_OUTPUT_FILE_NAME	169
DOCTYPE	176
para HTML	209
documentlanguage variável de personalização	178
DOCUMENTLANGUAGE_COLLATION	169
DUMP_STRUCTURE	169
DUMP_TEXI	170
DUMP_TREE	170

E

enable	120
ENABLE_ENCODING	167
END_USEPACKAGE	175
EPUB_CREATE_CONTAINER_FILE	195
evitando Archive::Zip dependência	195
EPUB_KEEP_CONTAINER_FOLDER	195
ERROR_LIMIT	167
estilo de nota de rodapé	159
example, valor para @kbinputstyle	56
EXTENSÃO,	
para HTML	205
EXTENSION	176
EXTERNAL_CROSSREF_EXTENSION	207
EXTERNAL_CROSSREF_SPLIT	207
EXTERNAL_DIR	207
EXTRA_HEAD	210

F

FILLCOLUMN	167
FOOTNOTE_END_HEADER_LEVEL	210
FOOTNOTE_SEPARATE_HEADER_LEVEL	210
FORCE	167
FORMAT_MENU	170
para HTML	204

H

HEADER_IN_TABLE	208
HEADERS	167
para HTML	208
HIGHLIGHT_SYNTAX	196
HIGHLIGHT_SYNTAX_DEFAULT_LANGUAGE	196
HTML_MATH	211
HTML_ROOT_ELEMENT_ATTRIBUTES	210
HTMLXREF_FILE	207
HTMLXREF_MODE	207

I

ICONS	208
IGNORE_REF_TO_TOP_NODE_UP	206
IGNORE_SPACE_AFTER_BRACED_COMMAND_NAME	170
IMAGE_LINK_PREFIX	207
INDENTED_BLOCK_COMMANDS_IN_TABLE	211
INDEX_ENTRY_COLON	211
INDEX_SPECIAL_CHARS_WARNING	174
INFO_JS_DIR	214
INFO_MATH_IMAGES	174
INFO_SPECIAL_CHARS_QUOTE	174
INFO_SPECIAL_CHARS_WARNING	174
INFOPATH	184
INLINE_CSS_STYLE	194
INPUT_FILE_NAME_ENCODING	170
INTERNAL_LINKS	167

J

JS_WEBLABELS	214
JS_WEBLABELS_FILE	214

L

L2H_CLEAN	213
L2H_FILE	213
L2H_HTML_VERSION	213
L2H_L2H	213
L2H_SKIP	213
L2H_TMP	213
LATEX_FLOATS_FILE_EXTENSION	176
LOCALE_ENCODING	170

M

MACRO_EXPAND	167
MATHJAX_CONFIGURATION	212
MATHJAX_SCRIPT	212
MATHJAX_SOURCE	212
MAX_HEADER_LEVEL	210
MAX_MACRO_CALL_NESTING	170
MENU_ENTRY_COLON	211
MENU_SYMBOL	211
MESSAGE_ENCODING	170
MONOLITHIC	205

N

NO_CSS	194
NO_CUSTOM_HTML_ATTRIBUTE	209
NO_INDEX	170
NO_NUMBER_FOOTNOTE_SYMBOL	211
NO_TOP_NODE_OUTPUT	176
para HTML	204
NO_USER_COMMANDS	170
NO_WARN	167
NODE_FILES	167
NODE_NAME_IN_INDEX	206
NODE_NAME_IN_MENU	206
none, valor para @urefbreakstyle	52
novalidate	167
NUMBER_FOOTNOTES	167, 211
NUMBER_SECTIONS	167

O

OPEN_DOUBLE_QUOTE_SYMBOL	175
OPEN_QUOTE_SYMBOL	177
para HTML	209
OUTFILE	167
OUTPUT_CHARACTERS	177
para HTML	209
OUTPUT_ENCODING_NAME	171
para HTML	208
OUTPUT_FILE_NAME_ENCODING	171

P

PACKAGE	171
PACKAGE_AND_VERSION	171
PACKAGE_NAME	171
PACKAGE_URL	171
PACKAGE_VERSION	171
padrão-borda de Janela	121
page-delimiter	255
paragraphindent	167
PRE_BODY_CLOSE	210
PREFIX	171
PREFIXO,	
para HTML	205
PROGRAM	171
PROGRAM_NAME_IN_ABOUT	205
PROGRAM_NAME_IN_FOOTER	208

S

SECTION_NAME_IN_TITLE	208
SHORT_TOC_LINK_TO_TOC	206
SHOW_BUILTIN_CSS_RULES	194
SHOW_TITLE	204
SORT_ELEMENT_COUNT	171
SORT_ELEMENT_COUNT_WORDS	171
SPLIT	167
para HTML	205
SPLIT_SIZE	167
SUBDIR	167
para HTML	205
SystemLiteral	176

T

T4H_LATEX_CONVERSION	213
T4H_MATH_CONVERSION	213
T4H_TEX_CONVERSION	214
TEST	171
TEXI2DVI	176
TEXI2HTML	204
texinfo_document Domínio do Gettext	178
TEXINFO_C_EMBED_PERL	165
TEXINFO_DTD_VERSION	176
TEXINFO_OUTPUT_FORMAT	172
TEXINFO_XS	164
TEXINFO_XS_CONVERT	165
TEXINFO_XS_PARSER	164
TEXINFO_XS_STRUCTURE	165
TEXINPUTS	154
TEXTCONTENT_COMMENT	176
TOC_LINKS	206
TOP_FILE	206
TOP_NODE_FILE_TARGET	207
TOP_NODE_UP	177
para HTML	207
TOP_NODE_UP_URL	207
TRACE_INCLUDES	167
TRANSLITERATE_FILE_NAMES	167
TREE_TRANSFORMATIONS	172
TXI_MARKUP_NO_SECTION_EXTENT	176
txicodequotebacktick	97
txicodequoteundirected	97
txicommandconditionals	136
txidefnamenspace	116

txiindexatsignignore.....	89
txiindexbackslashignore.....	89
txiindexhyphenignore.....	89
txiindexlessthanignore.....	89

U

UPDATED Variável do automake.....	249
UPDATED-MONTH Variável do automake.....	249
USE_ACCESSKEY.....	206
USE_ISO.....	209
USE_LINKS.....	208
USE_NODE_DIRECTIONS.....	205
USE_NODES.....	177
para HTML.....	205
USE_NUMERIC_ENTITY.....	177
para HTML.....	209
USE_SETFILENAME_EXTENSION.....	173
USE_TITLEPAGE_FOR_TITLE.....	204

USE_UNICODE_COLLATION.....	173
USE_UNIDECODER.....	173
USE_XML_SYNTAX.....	209

V

VERBOSE.....	167
VERSION Variável do automake.....	249
VERTICAL_HEAD_NAVIGATION.....	209

W

WORDS_IN_PAGE.....	208
--------------------	-----

X

XREF_USE_FLOAT_LABEL.....	206
XREF_USE_NODE_NAME_ARG.....	206

Índice Geral

!

! 101

"

„ (aspa baixo 9 dupla) 101
" (caractere de aspa indireccionado duplo) 101

&

'&#x hexadecimal;', saída oriunda de @U 108

,

, 101
" 101

(

(dir) como nó Acima do nó Top 29

,

, (aspa baixo 9 simples) 101

—

- (na sequência de caracteres alt da imagem) . 84, 111

<

< 101
« 101

>

> 101
» 101

?

? 101

—

-, ponto de interrupção dentro do @code 111

‘

‘ 101
` 101

@

@' como continuação em comandos de definição .. 115

Â

Âncoras 49

Í

Índice **cp** (conceito) 88
Índice **fn** (função) 88
Índice **ky** (pressionamento de tecla) 88
Índice **pg** (programa) 88
Índice **tp** (tipo de dado) 88
Índice **vr** (variável) 88
Índices 88
Índices de ordenação 150
Índices, combinando-os 92
Índices, definindo novos 93
Índices, imprimindo e menus 91
Índices, no formato Info 286
Índices, nomes de duas letras 92
Índices, ordenação 150

Ó

Órfãs, linhas, impedindo 113

A

â 101
A saída HTML,
matemática 109
ã 101
Å 101
Abreviações para teclas 57
Abreviações, etiquetando 60
accesskey,
em saída gerada HTML de menus 33
na saída HTML de nós 30
variável de personalização para 219
Acento agudo 100
Acento breve 100
Acento cedilha 100
Acento Check 100
Acento circunflexo 100
Acento de anel 100
Acento de ligação posterior 100
Acento grave 100
autônomo 97
versus aspa à esquerda 101
Acento Hacek 100
Acento macron 100
Acento ponto 100
Acento sub barra 100
Acento sublinhado 100
Acento til 100
Acento trema 100
Acento trema húngaro 100
Acentos flutuantes, inserindo 100
Acentos, inserindo 100
<acknowledgements> Etiqueta do DocBook 39
--add-once, para install-info 186
Adicionando um novo arquivo Info 183
æ 101
Æ 101
Agrupando duas definições juntas 116

Agrupar (manter texto unido verticalmente)	112
Ajuste fino, e hifenização	111
<code>--align=coluna</code> , para <code>install-info</code>	186
Altura da área de texto	280
Altura das imagens	84
Alvos das referências cruzadas, arbitrários	49
Alvos para referências cruzadas, arbitrários	49
Ambiente de flutuação	81
Ambiente verbatim	68
Amostra de arquivo de inclusão	148
Amostra de arquivo <code>@include</code>	148
Analísadores Texinfo, desencorajando mais	7
Analisando erros	262
Aninhando Condicionais	136
Anos, na linha de direitos autorais	16
Apelidos de comando	144
Apelidos, comando	144
<code>--append-new-sections</code> , para <code>install-info</code>	186
Arbusto	134
<code>Archive::Zip</code> , para saída do arquivo EPUB	195
Argumentos opcionais e repetidos	115
Argumentos repetidos e opcionais	115
Argumentos, repetidos e opcionais	115
Arquivo de configuração do Texinfo abrangente ao sítio	154
Arquivo de inicialização para entrada do $\text{\TeX}$	154
Arquivo de inicialização, botões de direção de ícone do HTML	209
Arquivo de saída EPUB	195
Arquivo <code>dir</code> , criando teu próprio	184
Arquivo do Texinfo de amostra, sem comentários	10
Arquivo do Texinfo, terminando	25
Arquivo DVI	152
Arquivo <code>@include</code> de amostra	148
Arquivo Info dividido	188
Arquivo Info não dividido	188
Arquivo <code>INSTALL</code> , gerando	162
Arquivo mínimo do Texinfo (exigências)	9
arquivo <code>NEWS</code> para Texinfo	136
Arquivo, terminando	25
Arquivos auxiliares, omitindo	153
Arquivos de amostra do Texinfo	249
Arquivos de inclusão, e níveis de seção	41
Arquivos <code>dir</code> bzípadados, lendo	186
Arquivos <code>dir</code> comprimidos por <code>lzip</code> , lendo	186
Arquivos <code>dir</code> comprimidos por <code>LZMA</code> , lendo	186
Arquivos <code>dir</code> comprimidos por <code>XZ</code> , lendo	186
Arquivos <code>dir</code> comprimidos, lendo	186
Arquivos <code>dir</code> e diretórios Info	184
Arquivos <code>dir</code> , comprimidos	186
Arquivos do Info, criando uma tabela de etiquetas	268
dividindo manualmente	268
Arquivos fonte, caracteres usados	8
Arquivos Info	4
instalação	183
listando um novo	183
ASCII, portabilidade de documentos fonte, usando	107
Aspa angular dupla apontando à direita	101
Aspa angular dupla apontando à esquerda	101
Aspa angular simples apontando à direita	101
Aspa angular simples apontando à esquerda	101
Aspa baixo 9 dupla	101
Aspa baixo 9 simples	101

Aspas à direita	101
Aspas à esquerda	101
Aspas alemãs	101
Aspas angulares	101
Aspas angulares apontando à direita	101
Aspas angulares apontando à esquerda	101
Aspas angulares duplas	101
Aspas angulares simples	101
Aspas duplas	101
Aspas francesas	101
Aspas simples	101
aspas simples indireccionadas	97
Aspas, alemãs	101
Aspas, francesas	101
Aspas, inserindo	101
Aspeamento automático de vírgulas para algumas macros	139
Aspeamento, automático para algumas macros	139
Comandos <code>@</code>	8
em nomes de nós	28
lista dos	222
sintaxe	221
variáveis de personalização para	166
Atributo <code>Alt</code> para imagens	84
Atualizando nós e menus	256
Auk, espécies de pássaro	102
<code>automake</code> , e informações de versão	249
Automaticamente inserir nós, menus	256

B

Barra de navegação, na saída gerada HTML	192
Barra invertida em macros	138
Barra invertida, e macros	139
Barra invertida, em argumentos de macro	140
Barra invertida, inserindo	96
Beebe, Nelson	4
Berry, Karl	5
Bloco <code><head></code> , adicionando ao	210
Blocos recuados de texto	67
Bolio	6
Bo $\text{\TeX}$	6

C

<code>-c variável=valor</code>	163
Cíceros	85
Código de categoria para comentários no $\text{\TeX}$	9
Códigos de categoria, do $\text{\TeX}$ simples	131
Códigos de idioma	126
Códigos de idioma ISO 639-2	126
Códigos de país	126
Códigos de país ISO 3166	126
Cabeçalho de um arquivo do Texinfo	11
Cabeçalho do arquivo do Texinfo	11
Cabeçalho para arquivos do Texinfo	11
Cabeçalhos	275
Cabeçalhos de comando de definição, continuando	115
Cabeçalhos de página	275
Cabeçalhos, recuo depois	279
Caixa com cantos arredondados	72
Caixa, preto horrível em cópia impressa	155
<code>--calign=coluna</code> , para <code>install-info</code>	186
Capítulos, estruturamento de	37

Definições de macro, linguagem de programação .	116
Definições de macro, Texinfo	138
Definindo entradas de indexação	89
Definindo macros	138
Definindo novos índices	93
Definindo novos comandos do Texinfo	138
--delete , para install-info	186
Delimitador de página no modo Texinfo	255
Depuração com formatação do Info	269
Depuração com formatação do T_EX	270
Depurando a estrutura do Texinfo	269
depurando documento, com representação de	
árvore	167
Descrição de documento	194
Descrição de menu, iniciar	254
Descrição do documento	194
Descrição para menu, iniciar	254
--description=texto , para install-info	186
Detalhes da sintaxe, ' #line '	147
Detalhes de macro	141
Detalhes do uso de macro	141
detexinfo	168
Diacrítico Ogonek	100
Dicas	245
Dicas de uso	245
Diferentes comandos de referência cruzada	43
Dimensões em tamanhos de imagem	84
Dimensionando imagens	84
dir , criado por install-info	186
Direita irregular, sem preencher	70
Diretório de contêiner de EPUB	195
Diretório de contêiner para EPUB	195
Diretório dir para instalação do Info	183
Diretiva ' #line '	146
detalhes da sintaxe	147
Diretivas ' #line ',	
não processando com T_EX	147
--dir-file=nome , para install-info	187
--disable-encoding	158
Distorcendo imagens	84
Dividindo um arquivo do Info manualmente	268
Dividir saída HTML	193
Divisão de arquivo de saída	163
Divisão dos arquivos de saída	163
--docbook	158
DocBook e seções prévias	38
DocBook, incluindo bruto	131
--document-language	158
Documentos hierárquicos, e menus	33
Documentos parciais, formatando	153
Dois itens nomeados para @table	78
Dois pontos em nome de nó	28
Dois pontos, último na INFOPATH	184
--dry-run , para install-info	187
DTD, para Texinfo XML	4
Duas 'Primeiras' Linhas para @defn	116
Dumas, Patrice	6, 197
--dvi	158
DVI, saída em	150
--dvi pdf	158
--dvi pdf , para texi2dvi	150
dvi pdf mx	150
dvips	151
dvips (programa)	3
δ	101
Ð	101
E	
-e limite	158
-E arquivo	160
E Comercial, inserindo	96
Edições críticas	86
Elevando e rebaixando seções	41
eliminando comandos do Texinfo	168
Emacs	252
--enable-encoding	158
Encontrando nós mal referenciados	272
Enfatização de texto, fonte para	62
Enfatizando texto	62
Englobamento	152
Entrada de menu de duas partes	35
Entrada de menu menos desordenada	35
Entrada de menu organizada	35
Entrada de teclado	56
Entrada de usuário(a)	56
Entradas de índice	89
Entradas de índice, conselhos acerca de redação ...	90
Entradas de índice, criando	90
Entradas de menu com dois "dois pontos"	35
Entradas dois pontos de menu	35
Entradas exclusivas de índice	91
Entradas para um índice	89
Entradas, criando índice	90
--entry=texto , para install-info	187
Enumeração	76
epsf.tex	85
epsf.tex , instalando	154
--epub3	158
EPUB, gerando	195
Equação exibida, em T_EX simples	131
Equação, exibida, em T_EX simples	131
Equações exibidas	109
Equações, exibidas	109
Equivalência, indicando	106
--error-limit=limite	158
Erros, analisando	262
Erros, capturando	269
Esboço da estrutura do arquivo, mostrando	254
Esboço semelhante ao conteúdo da estrutura do	
arquivo	254
Escapagem para HTML	192
Escopo limitado do Texinfo	2
Escrevendo entradas de índice	90
Escrevendo um menu	33
Escrevendo uma linha de @node	26
Escrita de linha de nó	26
escrita de linha de @node	26
Escrita de Texinfo	2
Espaçamento entre linhas	112
Espaçamento francês	99
Espaçamento, ao final de frases	99
Espaçamento, no meio de frases	98
Espaço amarrado	112
Espaço em branco em macros	138
Espaço em branco em nome de nó	28
Espaço em branco, continuações recolhidas em	
torno	115
Espaço em branco, controlando em condicionais ..	131
Espaço em branco, inserindo	98

Espaço fino entre número, dimensão	100
Espaço inquebrável, fixado	111
Espaço não quebrável, fixado	111
Espaço não quebrável, variável	112
Espaço, depois de frases	99
Espaço, inserindo	97
Espaço, inserindo horizontal	98
Espaço, inserindo vertical	112
Espaços em macros	138
Espaços em nome de nó	28
Espaços múltiplos	98
Espaços, em menus	33
Especificação '@charset', em arquivos CSS	193
especificação de cor RGB	52
Especificação de formato do Info	282
Especificação do formato Info	282
Especificações '@import', em arquivos CSS	194
Especificando entradas de índice	89
Esquerda irregular, sem preencher	70
Estilo de nota de rodapé, 'end'	86
'separate'	86
Estilo de nota de rodapé de nó 'end'	86
Estilo de nota de rodapé 'separate'	86
Estrutura de documento, do Texinfo	30
Estrutura de retaguarda, formatos de saída	3
Estrutura de seccionamento de arquivo do Texinfo, mostrando	254
Estrutura de seccionamento de arquivo, mostrando	254
Estrutura de seccionamento de um arquivo, mostrando	254
Estrutura de um arquivo, mostrando	254
Estrutura do documento Texinfo	30
Estrutura dupla, dos documentos do Texinfo	30
Estrutura, capturando erros em	269
Estrutura, dos documentos do Texinfo	30
Estruturamento de árvore	37
Estruturamento de capítulos	37
Es-zet	101
etex	152
Eth	101
etiqueta <blockquote> do DocBook	66
Etiqueta <body>, atributos da	210
etiqueta <caution> do DocBook	66
Etiqueta <head> do HTML, e <link>	220
etiqueta <important> do DocBook	66
Etiqueta <link> do HTML, no <head>	220
Etiqueta <meta> do HTML, e descrição de documento	194
Etiqueta <meta> do HTML, e especificação de conjunto de caracteres	127
etiqueta <note> do DocBook	66
etiqueta <small>	62
Etiqueta <thead> do HTML/DocBook	79
etiqueta <tip> do DocBook	66
Etiqueta <title> do HTML	12
etiqueta <warning> do DocBook	66
Evitando o recuo do primeiro parágrafo	279
Executando macros	139
Executando makeinfo no Emacs	261
Executando texi2any no Emacs	261
Executando um formataador Info	261
Exemplo de menu	34
Exemplo de nó Top	22

Exemplo de recuo	279
Exemplo Lisp	68
Exemplos da Lisp em fontes menores	73
Exemplos em fontes menores	73
Exemplos, formatando-os	67
Exemplos, glifos para	104
Exibições especiais	81
Exigência de nomes únicos de nós	28
Exigências da linha de nó	28
Exigências de arquivos de inclusão	260
Exigências de atualização	259
Exigências de formatação	9
Exigências mínimas para formatação	9
Exigências para arquivos de inclusão	260
Exigências para comandos de atualização	259
Exigências para formatação	9
Exigido no arquivo do Texinfo	9
Expandindo macros	139
Expansão das macros, contextos para	141
Expansão de \$Id, impedindo	112
Expansão de macro, contextos para	141
Expansão de macro, indicando	105
Expansão de nome de nó, em referências cruzadas do HTML	198
Expansão de palavras-chave, impedindo	112
Expansão dos caracteres de 8 bits nas referências cruzadas do HTML	201
Expansão, de nomes de nó nas referências cruzadas do HTML	198
Expansão, Indicando	105
Expressão regular, para '#line'	147
Expressões em um programa, indicando	55
Expressões matemáticas, inserindo	109
Expressões S, formato de saída	168

F

-f largura	159
Fórmulas, matemáticas	109
Fazendo quebras de linha e de página	110
Fazendo referências cruzadas	43
Fazendo um manual impresso	150
Fazendo uma tabela de etiquetas manualmente ..	268
feymr10	103
instalando	154
--fill-column=largura	159
filll dimensão do T _E X	19
Finalizando um arquivo do Texinfo	25
Finalizando uma Frase	99
Flutuação não numerada, criando	81
Flutuadores	81
fazendo não numeradas	81
lista de	82
numeração de	82
Flutuando, ainda não implementado	81
Folhas de Estilos em Cascata, e saída HTML	193
Fonte de máquina de escrever	63
Fonte de tamanho fixo	63
Fonte de versaletes	62
Fonte do Euro, instalando	154
fonte Euro	103
Fonte inclinada	63
Fonte inclinada de máquina de escrever, para @kbd	56
Fonte itálica	63

Fonte mono espaçada	63
Fonte negrito	63
Fonte oblíqua	63
Fonte padrão	63
Fonte para linhas de cabeçalho multi tabelas	79
Fonte romana	63
Fonte sans serifa	63
Fontes CM-Super	102
instalando	154
para L ^A T _E X	281
Fontes Computer Modern	128
Fontes EC	102
Fontes EC, instalando	154
Fontes European Computer Modern	102
instalando	154
Fontes menores	63
Fontes para índices	93
Fontes para impressão	63
--footnote-style=estilo	159
--force	159
-F	159
Forçando quebras de linha e de página	110
Forçando recuo	71
Forçar quebra de linha	110
Forma de dois argumentos das referências cruzadas	45
Forma de Normalização C, Unicode	201
Forma de três argumentos das referências cruzadas	45
Forma de um argumento das referências cruzadas	44
Formas de quatro e cinco argumentos das referências cruzadas	46
Formatação de dimensão	100
Formatação de exibição	69
Formatação de shell com tex e texindex	152
Formatação direta de arquivos do Info	267
Formatação e impressão de buffer	263
Formatação e impressão de região	263
Formatação em lote do Info	273
Formatação em lote para Info	273
Formatação Info	261
Formatação para Info	261
Formatando cabeçalhos e rodapés	275
Formatando com tex e texindex	152
Formatando documentos parciais	153
Formatando exemplos	67
Formatando, comandos	8
Formatar com o comando de compilação	265
Formatar e imprimir no modo Texinfo	263
Formatar e imprimir versão impressa	150
Formatar uma dimensão	100
Formatar, imprimir a partir do shell do Emacs	150
formato de imagem eps	83
formato de imagem jpeg	83
formato de imagem png	83
formato de imagem XPM	84
Formato do arquivo fonte	2
Formato Info, e menus	34
Formatos adicionais de saída	7
Formatos de imagem	83
Formatos de saída	3
Formatos de saída, com XML do Texinfo como entrada	7
Formatos de saída, suportando mais	7
Formatos para imagens	83
Fotos, inserindo	83
Fox, Brian	5

Frases, espaçamento depois	99
Funções tipadas	117
Funções, em linguagens tipadas	117
Furacões	48
Futuro das implementações do Texinfo	6

G

Gerando arquivos de texto simples com --no-headers	161
Gerando arquivos de texto simples com --plaintext	162
Gerando EPUB	195
Gerando HTML	192
Gerando menus com índices	91
Gerando títulos de página	21
Glifo de avaliação	104
Glifos para programação	104
Glifos para texto	102
Glifos textuais	102
GNU Emacs	252
gravador de arquivo para T _E X	152
Guillemets	101
Guillemets duplas	101
Guillemets simples	101
Guillemots	101

H

-h	159
Hífen, comparado ao sinal menos	104
Hífen, ponto de interrupção dentro do @code	111
'hbox', lotado	155
'hboxes' lotados	155
--help, para texi2any	159
--help, para texindex	187
help2man	7
Hifenização, ajudando T _E X a fazer	111
Hifenização, impedindo	112
Hifens no fonte, dois ou três em uma linha	8
Hiperlink simples	50
Histórico do Texinfo	5
href, produzindo HTML	50
--html	159
HTML bruto	192
HTML, e CSS	193
HTML, incluindo bruto	131
htmlxref.cnf	201
htmlxref.d/*.cnf	201
http-equiv, e especificação de conjunto de caracteres	127

I

i (i sem ponto)	101
i, j sem ponto	101
-I <i>diretório</i>	159
I18n, de strings do documento	178
Idioma chinês	155
Idioma do documento, declarando	126
Idioma japonês	155
Idioma, declarando	126
--ifdocbook	159
--ifhtml	159
--ifinfo	159
--iflatex	159
--ifplaintext	159
--iftex	159
--ifxml	159
Ignorado antes de @setfilename	12
Imagens	83
dimensionamento	84
no formato Info	285
texto alternativo para	84
Imagens SVG, usadas em DocBook	84
Impedindo quebras de linha e de página	110
Implementação de referência	6
Implementação, <i>texi2any</i> como referência	6
Impressão ampliada	281
Impressão de região no modo Texinfo	263
Imprimindo um índice	91
Imprimindo uma região ou buffer	263
Imprimindo versão impressa	150
Imprimir e formatar no modo Texinfo	263
Imprimir, formatar a partir do shell do Emacs ...	150
Início da linha de cabeçalho	14
Incluindo texto das permissões	17
Incluindo um literal de arquivo	149
Inclusão de Arquivos	148
inclusões de imagem pdf	83
Indefinindo macros	139
Indexação avançada	90
Indexação, avançada	90
Indexando entradas de tabela automaticamente ...	78
Indicando avaliação	104
Indicando comandos, definições, etc.	54
Indo para outros nós de arquivos do Info	35
--info	160
Info instalado em outro diretório	184
Info validação de um arquivo grande	272
Info; nós de outros arquivos	35
--info-dir=diretório, para install-info	187
--info-file=arquivo, para install-info	187
Informando Defeitos	2
Inicialização da entrada do TeX	154
Iniciando capítulos	274
--init-file=arquivo	160
Inquebrável, espaço variável	112
Inserções especiais	95
Inserindo @ ('@' literal)	95
Inserindo #	96
Inserindo &	96
Inserindo acentos	100
Inserindo aspas	101
Inserindo caracteres de citação	97
Inserindo caracteres e símbolos especiais	95
Inserindo comandos frequentemente usados	253

Inserindo espaço	97
Inserindo pontos	103
Inserindo recuo	71
Inserindo reticências	103
Inserir nós, menus automaticamente	256
Instalando Info em outro diretório	184
Instalando um arquivo Info	183
install-info	186
Interface de navegação JavaScript para HTML ...	214
Internacionalização	126
Internacionalização de strings do documento	178
--internal-links=arquivo	160
Introdução a Texinfo	2
Inundação	48
Invocação de macro	139
Invocações recursivas de macro	138
Invocando macros	139
Invocando nós, incluindo no arquivo dir	185
Invocando pod2texi	179
Irregular à direita, com preenchimento	70
Islandês	101
--item=texto, para install-info	187
Itenização	74

J

j (j sem ponto)	101
Japonês com LuaTeX	155
Japonês com XeTeX	155

K

--keep-old, para install-info	187
Knuth, Donald	5

L

l	101
lang, atributo do HTML	215
--language, para texi2dvi	151
L	101
Largura da área de texto	280
Largura das imagens	84
Largura e altura do texto	280
Larguras de colunas de multi tabelas	79
Larguras de colunas, definindo para multi tabelas .	79
Larguras, definindo coluna de multi tabelas	79
--latex	160
L ^A T _E X, incluindo bruto	131
L ^A T _E X, processando com texi2dvi	151
Legendas curtas, para listas de flutuações	82
Legendas, para flutuações	82
libintl-perl Implementação Gettext	178
Libras, símbolo	103
Licença de cópia literal	251
Licença de cópia toda permissiva	251
Licença de Documentação Livre, incluindo inteira.	249
Licença GNU de Documentação Livre, incluindo inteira	249
Licença para cópia literal	251
Licença para cópia toda permissiva	251
Linha de cabeçalho, em tabela	79
Linha de fim do cabeçalho	14

Linha de protótipo, larguras de coluna definidas por	79
Linha inicial de um arquivo do Texinfo	11
linha ‘\input’ do fonte ignorada	12
linha ‘\openout’ em arquivo de registro	152
Linhas de continuação em comandos de definição	115
Linhas de multi tabelas	79
Linhas em branco	112
Linhas, de uma multi tabela	79
Link simples	50
Links coloridos, em saída PDF	52
Links, colorindo em saída PDF	52
Linque para EMail	62
Linques de navegação, omitindo	161
Linques internos, do HTML	160
Lista alfabética de comandos @	222
Lista de flutuações	82
Lista de verificação para informes de defeitos	2
Lista dos comandos @	222
Listagem do arquivo <code>dir</code>	183
Listando um novo arquivo Info	183
Listas de definição, tipografando	77
Listas e tabelas, construindo	74
Literal, arquivo de inclusão	149
Literal, pequeno	68
Livre, software	1
Livro, imprimindo pequeno	280
Local de Menu	34
Localidade, declarando	126
Logo do L ^A T _E X	103
Logo do T _E X	103
Logomarcas, T _E X	103
LuaTeX	156

M

Métodos, orientados a objetos	121
Mínimo do arquivo do Texinfo	9
Módulos de extensão da Perl (XS)	164
<code>--macro-expand=arquivo</code>	160
Macros	138
Macros de linha	144
Macros recebendo linha inteira como um argumento	144
Macros, indefinindo	139
Maiúscula, não alterando em <code>@code</code>	55
Maiúsculas e minúsculas em nome de nó	28
<code>makeinfo</code>	157
<code>makeinfo</code> dentro do Emacs	261
Mantendo texto unido verticalmente	112
Manuais divididos, formato Info de	282
Manuais divididos, para referências cruzadas do HTML	203
Manuais monolíticos, para referências cruzadas do HTML	203
Manuais não divididos, formato Info de	282
Manual e livro impresso, características	5
Manual inteiro, no formato Info	282
Manual, referenciamento para como um todo	47
Marcação semântica	2
Marcando palavras e frases	54
Marcando texto dentro de um parágrafo	54
Margens na página, não controláveis	280
<code>--max-width=coluna</code> , para <code>install-info</code>	187
Mensagem de erro, indicando	106

Mensagens de erro, números de linha em	146
Menu de exemplo	34
Menu detalhado	23
Menu mestre	23
Menu, Escrevendo	33
Menu, mestre	23
Menu, partes	35
<code>--menuentry=texto</code> , para <code>install-info</code>	187
Menus	33
Menus gerados com índices	91
Menus, Local de	34
Menus, no formato Info	285
Menus, omitindo com <code>--no-headers</code>	161
Menus, omitindo com <code>--plaintext</code>	162
META, tecla	57
Metadados de documento	17
Milímetros	85
Mils, argumento para <code>@need</code>	113
Modelo de definição	114
Modelo para uma definição	114
Modo Texinfo	252
Modo, usando Texinfo	252
Mostrando a estrutura de seccionamento de um arquivo	254
Mostrando a estrutura de um arquivo	255

N

Não finalizando uma frase	98
Nó Acima do nó Top	29
Nó Anterior do nó Top	29
Nó Próximo do nó Top	29
Nó Top	22
Nó Top é primeiro	29
Nó, definido	26
Nó, ‘Top’	22
Nós de outros arquivos do Info	35
Nós desestruturados	42
Nós do Info, no formato Info	284
Nós em outros arquivos do Info	35
Nós mais longos, achando	171
Nós mal referenciados	272
Nós, capturando erros	269
Nós, deletando ou renomeando	49
Nós, verificando mal referenciados	272
Número de versão, para <code>install-info</code>	188
Números de linha, em mensagens de erro	146
<code>--name=texto</code> , para <code>install-info</code>	187
NASA, como sigla	60
NLS	5
<code>--node-files</code>	161
<code>--node-files</code> , e referências cruzadas do HTML	203
<code>--no-headers</code>	161
<code>--no-ifdocbook</code>	160
<code>--no-ifhtml</code>	160
<code>--no-ifinfo</code>	160
<code>--no-iflatex</code>	160
<code>--no-ifplaintext</code>	160
<code>--no-iftex</code>	160
<code>--no-ifxml</code>	160
<code>--no-indent</code> , para <code>install-info</code>	187
Nome do arquivo da saída gerada	12
Nome do arquivo do Info, escolhendo	12
Nomes de arquivos de índice	152

Nomes de comandos, indicando	59
Nomes de duas letras para índices	92
Nomes de família, todas letras maiúsculas	61
Nomes de macro, caracteres válidos em	138
Nomes de macros, caracteres válidos de	138
Nomes de nó precisam ser únicos	28
Nomes de nó, caracteres inválidos em	28
Nomes de nós, escolhendo	27
Nomes de programas, indicando	59
Nomes dos arquivos de índice	152
Nomes para índices	92
Nomes predefinidos para índices	92
Nomes recomendados para teclas	57
<code>--no-node-files</code>	161
<code>--no-number-footnotes</code>	161
<code>--no-number-sections</code>	161
<code>--no-pointer-validate</code>	161
<code>--no-split</code>	163
Notas de Rodapé	85
Notas de rodapé aninhadas	86
<code>--no-validate</code>	161
Novas linhas, evitando em condicionais	131
Novo arquivo Info, listando-o no arquivo <code>dir</code>	183
Novos índices, definindo	93
Novos comandos do Texinfo, definindo	138
<code>--no-warn</code>	161
<code>--number-sections</code>	161
Numeração de flutuações	82
Numeração de páginas	275

O

ø	101
<code>-o</code> <code>arquivo</code>	161
O'Dea, Brendan	7
Obras da FSF, Titular dos direitos autorais das	16
Obtendo $\TeX$	150
Ocorrências, listando com <code>@occur</code>	256
Octotherp, inserindo	96
œ	101
Œ	101
Omitindo recuo	71
Ondas do mar	48
Opções de linha de comando do <code>texi2html</code>	181
Opções de usuário(a), marcando	119
opções do <code>makeinfo</code>	157
opções do <code>texi2any</code>	157
Opções do <code>texi2html</code>	181
Opções para <code>makeinfo</code>	157
Opções para o <code>texi2any</code>	157
Opções, variáveis de personalização para	167
Ordenação de índices do $\TeX$	150
ordenamento de índice	89
Ordenando nós por tamanho	171
ordenar chaves para entradas de índice	89
Ordinais de romance	101
Ordinais, Romance	101
Ordinal feminino	101
Ordinal masculino	101
<code>--output=file</code>	161
<code>--outputindent</code>	162
Ø	101
Outro diretório Info	184

P

<code>-p</code> <code>recuo</code>	162
<code>-P</code> <code>caminho</code>	162
Página órfã de título	17
Página de direitos autorais	17
Página de manual, referência a	51
Página de rótulos web de licença JavaScript	214
Página de título	18
órfã	17
Página preliminar, texto em	23
Páginas maiores ou menores	281
Páginas Part	40
Páginas, começando ímpares	274
Padrões de hifenização, dependentes do idioma	126
Painel de navegação, final da página	220
Palavra Copyright, sempre em inglês	16
Palavras chave, indicando	55
Palavras e frases, marcando-as	54
Palavras reservadas, indicando	55
Palavras-chave de controle de versão, impedindo expansão de	112
Papel A4 europeu	280
Papel A4, impressão em	280
Papel A5, impressão em	280
Papel B5, impressão em	280
Papel ofício, impressão em	280
Parágrafo, marcando texto dentro	54
Parâmetros para macros	138
Parêntesis em nome de nó	28
<code>--paragraph-indent=recuo</code>	162
Parte da formatação e impressão de arquivos	263
Partes da referência cruzada	43
Partes de <code>@menu</code>	35
Partes de um menu	35
Partes de um menu mestre	23
Partes de uma referência cruzada	43
<code>--pdf</code>	162
<code>--pdf</code> , para <code>texi2dvi</code>	150
<code>pdftex</code>	153
<code>pdftex</code> , e imagens	83
<code>pdftexi2dvi</code>	150
Perl Embutido no <code>texi2any</code> da C	165
Permissões de cópia	15
Permissões de documentos	15
Permissões impressas	17
Permissões, impressas	17
Permitir quebra de linha	110
Personalização de referências cruzadas, no HTML	207
Personalização do DOCTYPE	209
Personalização do $\TeX$ para Texinfo	154
Personalizar o pacote Emacs (Development/Docs/Texinfo)	265
Picas	84
<code>--plaintext</code>	162
Pod da Perl, convertendo para Texinfo	179
Pod, convertendo para Texinfo	179
<code>pod2texi</code>	179
Polegadas	85
Ponto de código do caractere Unicode, inserindo por	107
Ponto em nome de nó	28
Ponto, indicando em um buffer	106
Pontos (dimensão)	84
Pontos de Didôt	85

Pontos de interrupção dentro de URLs.....	51
Pontos escalonados	85
Pontos grandes	85
Pontos, inserindo	98, 103
Pontuação de fechamento, e finalização de frase ...	99
Pontuação de final de frase	99
Pontuação não terminando frase	98
Preâmbulo	13
Preâmbulo do Texinfo	13
Preâmbulo, no formato Info	283
Precisa de espaço no final da página	113
Prefácio, etc., e DocBook	38
<preface> Etiqueta do DocBook	39
Preparando para T _E X	154
Primeira linha de um arquivo do Texinfo	11
Primeiro nó	29
Primeiro parágrafo, suprimindo o recuo de	279
Problemas, capturando	269
Processadores de macro, externos	146
Processadores externos de macro	146
Procurando por nós mal referenciados	272
arquivo de inicialização .profile	154
Profundidade da área de texto	280
Programação letrada, com Texinfo e awk	154
Programação orientada a objetos	120
Programação, glifos para	104
Pronúncia de Texinfo	2
Proporção de aspecto das imagens	84
Proprietário(a) dos direitos autorais das obras da FSF	16
--ps	162
--ps, para texi2dvi	151

Q

Quebras de linha, controlando	110
estranhas	110
URLs	51
Quebras de linha, impedindo	111
Quebras de página, estranhas	110
Quebras de página, forçando	112
Quebras em uma linha	110
Quebras, dentro do @code	111
--quiet, para install-info	187

R

Realçamento, personalizado	145
Realçando texto	54
Rebaixando e elevando seções	41
Recolhendo espaços em branco em torno de continuações	115
Recuando ambientes	279
Recuo de parágrafos, controle de	278
Recuo do ambiente	279
Recuo, desfazendo	69
Recuo, forçando	71
Recuo, omitindo	71
Recuo, supressão do primeiro parágrafo	279
Recursos do Texinfo, adaptando-se a	136
Reduzindo tamanho de fonte	63
Referência de entidade em HTML et al.	108
Referência para comandos @	222

Referências	43
Referências cruzadas	43
Referências cruzadas do HTML	197
básicos do linque	197
configuração	201
expansão de comando	199
expansão de nome de nó	198
expansão do caractere de 8 bits	201
personalização de configuração	207
Referências cruzadas usando @inforef	50
Referências cruzadas usando @link	50
Referências cruzadas usando @pxref	48
Referências cruzadas usando @ref	48
Referências cruzadas usando @xref	48
Referências cruzadas, na saída gerada HTML	197
Referências Cruzadas, no formato Info	287
Referências usando @inforef	50
Referências usando @pxref	48
Referências usando @ref	48
Referências usando @xref	48
Referenciamento para um manual inteiro	47
Referenciando outros arquivos do Info	35
--regex=expressão regular, para install-info ..	188
Reid, Brian	5
Remendos, contribuindo	3
--remove, para install-info	188
--remove-exactly, para install-info	188
representação de árvore, para depuração	167
Representação de documentos em árvore	6
Ressalvas para uso de macro	141
Restrições a nomes de nó	28
Resultado de uma expressão	104
Resumo do documento	194
Retângulo preto em cópia impressa	155
Retângulo, preto em cópia impressa	155
Retângulos arredondados, ao redor do texto	72
Retângulos pretos horríveis em cópia impressa ...	155
Reticências, inserindo	103
ridt.eps	85
Robbins, Arnold	154
Rodapé de navegação	220
Rodapés	275

S

--s estilo	159
S alemão	101
Símbolo arroba, inserindo	95
Símbolo cerquilha, inserindo	96
Símbolo de copirraite	103
Símbolo de Graus	104
Símbolo de número, inserindo	96
Símbolo de registrado(a)	103
Símbolo do euro e codificações	127
Símbolo do Euro, produzindo	103
Saída de página de manual, não suportada	7
Saída de texto ASCII com --plaintext	162
saída de texto cru	168
Saída de texto simples com --plaintext	162
Saída do Info, e codificação	127
Saída do Info, visão geral	3
Saída DocBook, visão geral	4
Saída DVI, visão geral	3
Saída EPUB 3, visão geral	3

Saída gerada de HTML,	
visão geral	3
Saída gerada de texto simples, visão geral	3
Saída gerada final	155
Saída gerada HTML,	
compatibilidade de navegador	192
Saída gerada matemática para HTML	109
Saída gerada PDF, visão geral	4
Saída gerada PostScript, visão geral	3
Saída HTML,	
codificações	127
dividir	193
matemática,	
personalização	211
Saída impressa, indicando	105
Saída impressa, por meio do texi2any	165
Saída LaTeX, visão geral	4
Saída PDF	150
Saída PDF de URLs	52
Saída SXML	168
Saída Texinfo XML, visão geral	4
Saída XML do Texinfo, como entrada	7
Saída XML DocBook, visão geral	4
Saída XML Texinfo, visão geral	4
Saída, em PDF	150
Saída, impressa por meio do texi2any	165
Scribe	5
Se texto visível condicionalmente	129
Seção da variável local, no formato Info	284
Seção Variáveis Locais, para codificação	127
Seções, elevando e rebaixando	41
Seccionamento	37
--section expressão regular seção , para	
install-info	188
--section=seção , para install-info	188
Separadores de nós, omitindo com --no-headers	161
Separadores de nós, omitindo com --plaintext	162
Sequência de cliques da GUI	107
Sequência dos cliques	107
Sequências de caracteres de documentos, tradução	
de	126
Sequências de cliques	107
--set-customization-variable	
variável=valor	163
Sharp S	101
Shell do Emacs, formatar, imprimir a partir de ...	150
Shell do GNU Emacs, formatar, imprimir a partir	
de	150
Shell, executando makeinfo em	261
Shell, executando texi2any em	261
Shell, formatar, imprimir a partir de	150
Siglas, etiquetando	60
--silent , para install-info	187
Sinal de menos	104
Sinal sustenido (não), inserindo	96
Sintáticas, convenções	8
Sintaxe abstrata, Representação de documentos em	
árvore	6
Sintaxe de comando	221
Sintaxe URI para Info	5
Sintaxe, argumentos opcionais e repetidos	115
Sintaxe, dos comandos @	221
Sintaxe, Representação de documentos em árvore ..	6
Small, exemplos	73
Smith, Gavin	5

Software livre	1
source-highlight	196
--split=como	163
--split-size=número	163
ß	101
Stallman, Richard M.	5
Strings do documento de saída, internacionalização	
de	178
Strings do documento, internacionalização de	178
Strings do formato Perl para tradução	178
Subarquivos indiretos	188
Sublinhado, ponto de interrupção dentro do @code ..	111
Subscritos e sobrescritos, texto	102
Sugestões para Texinfo, fazendo	2
Sumário	21
Sumário curto	21
Sumário do documento	194
Sumário, depois da página de título	244
Sumário, gerar	21
Sumário, para flutuações	82
Suprimindo o recuo do primeiro parágrafo	279
Suprimindo recuo	71

T

Título de documento, especificando	12
Tabela de etiquetas	188
Tabela de etiquetas, fazendo manualmente	268
Tabela de Etiquetas, no formato Info	283
Tabela Indireta, no formato Info	283
Tabelas com indexação	78
Tabelas e listas, construindo	74
Tabelas, fazendo duas colunas	77
Tabelas, fazendo multi colunas	79
Tamanho de fonte, reduzindo	63
Tamanho do livro impresso	280
Tamanho do papel, A4	280
Tamanho pequeno do livro	280
Tamanhos de página para livros	280
Tamanhos de página, personalizados	280
Tamanhos personalizados de página	280
Teclas de controle, especificando	57
Teclas Meta, especificando	57
Teclas, nomes recomendados	57
--test , para install-info	188
Testagem para comandos do Texinfo	136
Testes da linguagem do Texinfo	7
Testes, da linguagem do Texinfo	7
TeX e diretivas '#line'	147
TeX simples	131
TeX, como obter	150
texi2any	157
texi2any dentro do Emacs	261
texi2any , como implementação de referência	6
texi2dvi (script de shell)	150
texi2html	181
texi2oldapi.texi , para texi2html	182
texi2pdf (script de shell)	150
texi-elements-by-size	171
texindex	152
Texinfo visão geral	2
Texinfo, e programação letrada	154
Texinfo, estória	5
Texinfo, introdução a	2

<code>texinfo.cnf</code> instalação.....	154
<code>texinfo.dtd</code>	4
<code>texinfo.tex</code> , instalando.....	154
<code>texiwebjr</code>	154
<code>Text::Unidecode</code>	173
Texto condicional e delimitado por chaves.....	131
Texto condicionalmente visível.....	129
Texto das permissões, incluindo.....	17
Texto de reprodução, incluindo.....	17
texto do <code><body></code> , personalizando.....	215
Texto ignorado.....	9
Texto literal interno à linha.....	58
Texto não processado.....	9
Texto, marcando.....	54
Texto, visível condicionalmente.....	129
Textos completos, GNU.....	249
Textos de amostra, GNU.....	249
Textos GNU de amostra.....	249
<code>þ</code>	101
Thorn.....	101
<code>Þ</code>	101
<code>ti.twjr</code>	154
<code>time-stamp.el</code>	249
Tipo de retorno, linha própria para.....	118
Tipos de fonte de índice.....	89
Titular dos direitos autorais para obras da FSF...	16
Tokens sintáticos, indicando.....	55
Traço de ligação, produzindo.....	8
Traço, ponto de interrupção dentro do <code>@code</code>	111
Traços no fonte.....	8
<code>--trace-includes</code>	163
Tradução do HTML.....	192
Traduzindo strings nos documentos de saída.....	178
Transliteração dos caracteres de 8 bits nas referências cruzadas do HTML.....	201
<code>--transliterate-file-names</code>	163
Travessão, comparado ao sinal de menos.....	104
Travessão, produzindo.....	8
<code>txi-cc.tex</code>	126
<code>txixml2texi</code>	4

U

Unicode e $\TeX$	107
<code>Unicode::Collate</code>	173
<code>@url</code> , exemplos de uso.....	51
URL, exemplos de exibição.....	51
indicando.....	61
referenciando.....	50

<code><URL...></code> convenção, não usada.....	51
URLs, colorindo em saída PDF.....	52
URLs, saída PDF de.....	52
Usando <code>Info-validate</code>	272
Usando Texinfo em geral.....	2

V

<code>-v</code>	164
Vários traços no fonte.....	8
Vírgula depois de referência cruzada.....	44
Vírgula em nome de nó.....	28
Vírgula, em argumentos de macro.....	139
Vírgula, inserindo.....	95
Validação de ponteiro.....	161
suprimindo.....	153
Validando um arquivo grande.....	272
Valor de uma expressão, indicando.....	104
Variáveis de personalização para comandos <code>@</code>	166
Variáveis de personalização para opções.....	167
Variáveis locais.....	265
Variáveis tipadas.....	119
Variáveis, em linguagens tipadas.....	119
Variáveis, orientadas a objetos.....	120
Variável de ambiente <code>INFOPATH</code>	184
Variável de ambiente <code>TEXINFO_OUTPUT_FORMAT</code> ...	164
Variável de ambiente <code>TEXINPUTS</code>	154
Verbatim, pequeno.....	68
<code>--verbose</code>	164
verificação ortográfica.....	168
Verificando nós mal referenciados.....	272
Versão impressa, imprimindo.....	150
Versões do Texinfo, adaptando-se a.....	136
<code>--version</code> , para <code>install-info</code>	188
<code>--version</code> , para <code>texi2any</code>	164
Verticalmente mantendo texto unido.....	112
Visão Geral do Texinfo.....	2
Visibilidade de texto condicional.....	129

W

<code>@w</code> , para itens em branco.....	75
---	----

X

<code>xdvi</code>	3
$\XeTeX$	156
<code>--xml</code>	164
XML, incluindo bruto.....	131
<code>--Xopt sequência</code>	164